HYDRIS AI

HYDRIS FIELD GLASSES · COMPLETE DOCUMENTATION

Hydris Field Glasses: Complete Product Documentation

Requirements, architecture, data, APIs, apps, glasses, design, deployment, security, quality, operations and roadmap in one volume

DOCUMENT	VERSION	STATUS	DATE
HFG-ALL	2.0	Draft for review	October 2026

Document control

Document	Hydris Field Glasses: Complete Product Documentation (HFG-ALL)
Version	2.0
Status	Draft for review
Owner	Product and engineering, Hydris
Audience	Everyone building, deploying, operating, reviewing or evaluating Hydris Field Glasses
Product	Hydris Field Glasses: website, operator console, glasses app, Android app, API
Source of truth	docs/product/_combined.md in tarun-km/hydrisview (this Word file is generated from it)
Classification	Confidential: Hydris, Inc. and named partners

Revision history

Version	Date	Author	Changes
2.0	October 2026	Product and engineering, Hydris	Single volume combining all thirteen product documents

Contents

1 Product Documentation Suite 10
1.1 The documents 10
1.2 Where to start 11
1.3 The product on one page 12
1.4 How the Word editions are made 13
1.5 Status conventions 15
2 Product Requirements Document 16
2.1 Executive summary 16
2.2 Vision and product principles 17
2.3 Problem and market context 18
2.4 Personas 20
2.5 Jobs to be done and user journeys 22
2.6 Product surfaces and scope 23
2.7 Functional requirements 26
2.8 Non-functional requirements 63
2.9 Data and AI requirements 66
2.10 Success metrics and KPIs 69
2.11 Release plan and milestones 70
2.12 Dependencies 73
2.13 Risks and mitigations 76
2.14 Open questions 77
2.15 Glossary 77
2.16 Appendix: traceability to other documents 78
3 Architecture 80
3.1 Architectural drivers 80
3.2 System context 82
3.3 Containers 83
3.4 API: a modular monolith 85
3.5 Domain model 88
3.6 Data architecture and tenancy 89
3.7 Identity, sessions and tokens 91
3.8 Devices and pairing 93
3.9 Telemetry and alarms 97
3.10 Realtime 98
3.11 Audit document scanning 100

3.12 Voice assistant ... 103
3.13 Client applications ... 104
3.14 Deployment topologies ... 107
3.15 Cross-cutting concerns ... 111
3.16 Architecture decision records ... 113
3.17 Quality attributes ... 115
3.18 Scalability path ... 116
3.19 Technical debt register ... 117
3.20 Future architecture ... 119
4 Data Model ... 121
4.1 Data principles ... 121
4.2 Domain overview ... 122
4.3 Entity relationship model ... 124
4.4 Data dictionary ... 133
4.5 Enumerations ... 158
4.6 JSON column schemas ... 160
4.7 State models ... 163
4.8 Tenancy and row-level security ... 168
4.9 Module access patterns ... 171
4.10 Indexing and query patterns ... 172
4.11 Data quality rules ... 173
4.12 Data lifecycle and retention ... 175
4.13 Volumes and growth ... 177
4.14 Binary and image storage ... 178
4.15 Migrations ... 179
4.16 Seed and demo data ... 180
4.17 Privacy classification ... 181
4.18 Mobile and client-side data ... 183
4.19 Analytics and event model ... 184
4.20 Future schema changes ... 185
4.21 Open questions and unverified facts ... 186
5 API Reference ... 187
5.1 How to read this reference ... 187
5.2 Overview and conventions ... 189
5.3 Authentication and sessions ... 192
5.4 Authorization ... 198

5.5 Errors ... 202
5.6 Rate limiting ... 207
5.7 Idempotency and concurrency ... 208
5.8 Realtime events ... 208
5.9 Health ... 211
5.10 Authentication module ... 212
5.11 Organization module ... 215
5.12 Plant hierarchy and records module ... 218
5.13 Telemetry module ... 224
5.14 Alarms module ... 228
5.15 Inspections module ... 232
5.16 Maintenance module ... 236
5.17 Devices and pairing module ... 239
5.18 Realtime module ... 242
5.19 Insights module ... 242
5.20 Voice assistant module ... 243
5.21 Audit documents module ... 243
5.22 Glasses UI profiles module ... 246
5.23 Simulation module ... 247
5.24 Public website module ... 248
5.25 Background jobs and cron ... 248
5.26 Client libraries ... 248
5.27 Change history and deprecation policy ... 249
5.28 Planned API ... 249
6 Mobile App Specification ... 251
6.1 Product summary and goals ... 251
6.2 Relationship to the Meta AI app ... 253
6.3 Architecture ... 254
6.4 Navigation ... 258
6.5 Screen specifications ... 260
6.6 Data layer and networking ... 278
6.7 Audit scan pipeline ... 283
6.8 Voice ... 288
6.9 Notifications ... 289
6.10 Permissions ... 290
6.11 Design system ... 291

6.12 Accessibility 292
6.13 Performance budgets 293
6.14 Offline behaviour and sync 294
6.15 Security 294
6.16 Build and release 295
6.17 Testing 298
6.18 Feature status and roadmap 299
7 Glasses Integration 301
7.1 Scope and status 301
7.2 Hardware and capabilities 302
7.3 Runtimes and architecture 304
7.4 The Meta AI app dependency 306
7.5 DAT lifecycle 306
7.6 Web bridge contract 314
7.7 Glasses web viewport 317
7.8 Pairing with the Hydris account 320
7.9 Setup guide 321
7.10 Troubleshooting 322
7.11 Limitations and known gaps 323
7.12 Glasses state model and console view 323
7.13 Data flows and network use 324
7.14 Hardware acceptance tests 325
7.15 Frequently asked questions 326
7.16 Privacy 326
7.17 Roadmap 327
8 Design System and UX Guidelines 328
8.1 Design principles 328
8.2 Design system architecture 330
8.3 Brand and colour system 331
8.4 Typography 337
8.5 Spacing, radius and elevation 339
8.6 Iconography 341
8.7 Component library 341
8.8 Data visualisation guidelines 351
8.9 Layout grids 351
8.10 Navigation patterns 357

8.11 Interaction patterns ... 361
8.12 Motion ... 373
8.13 Accessibility ... 374
8.14 Content and voice guidelines ... 375
8.15 Localisation ... 376
8.16 Dark and light themes ... 376
8.17 Design QA checklist ... 377
8.18 Governance and contribution ... 378
8.19 Future work ... 380
9 Deployment and Infrastructure ... 382
9.1 Deployment at a glance ... 382
9.2 Where Hydris runs today ... 383
9.3 Target topologies ... 386
9.4 Environment matrix ... 394
9.5 Configuration reference ... 395
9.6 Build and release pipeline ... 397
9.7 Proposed CI/CD with GitHub Actions ... 398
9.8 Database migrations and rollback ... 401
9.9 Background jobs and scheduling ... 404
9.10 Android app distribution ... 405
9.11 Observability ... 408
9.12 Backup and disaster recovery ... 410
9.13 Scaling and capacity ... 412
9.14 Cost model ... 413
9.15 India data residency and regulatory hosting ... 414
9.16 Customer onboarding and provisioning ... 416
9.17 Go-live checklist ... 417
9.18 Known gaps and decisions ... 418
10 Security and Privacy ... 420
10.1 Security principles ... 420
10.2 System overview and trust boundaries ... 421
10.3 Data classification ... 422
10.4 Threat model ... 422
10.5 Authentication and sessions ... 425
10.6 Device and glasses security ... 427
10.7 Authorization ... 430

10.8 API security ... 432
10.9 Secrets management ... 433
10.10 Mobile security ... 435
10.11 AI safety ... 435
10.12 Camera and bystander privacy ... 438
10.13 Worker privacy and monitoring ethics ... 439
10.14 Compliance mapping ... 439
10.15 Retention schedule ... 440
10.16 Audit logging ... 440
10.17 Vulnerability management ... 441
10.18 Incident response ... 442
10.19 Security roadmap ... 443
10.20 Known gaps register ... 443
11 QA and Test Plan ... 445
11.1 Purpose, scope and principles ... 445
11.2 Quality strategy and test pyramid ... 446
11.3 Current automated coverage ... 448
11.4 Test environments and data ... 455
11.5 Manual test charters by surface ... 457
11.6 Hardware and device matrix ... 459
11.7 Acceptance criteria by feature ... 460
11.8 Non-functional testing ... 474
11.9 Defect management ... 477
11.10 Release quality gates and pilot exit criteria ... 479
11.11 Traceability from requirements to tests ... 481
11.12 Proposed CI test pipeline ... 483
11.13 Quality metrics dashboard ... 485
11.14 Risk-based testing ... 486
11.15 Schedule, roles and open questions ... 488
12 Operations and Support Runbook ... 489
12.1 Operating model ... 489
12.2 Customer onboarding ... 492
12.3 Routine operations ... 498
12.4 Glasses fleet management ... 500
12.5 Android app distribution and updates ... 505
12.6 Audit document operations ... 506

12.7 Alarm operations and escalation 509
12.8 Support procedures and troubleshooting 511
12.9 Incident management 524
12.10 Monitoring and health checks 526
12.11 Data requests 527
12.12 Demo environment management 528
12.13 Release and change management 530
12.14 Training curriculum 531
12.15 Operations KPIs 532
12.16 Support contact and escalation templates 532
12.17 Appendices 533
13 Product Roadmap and Future Features 535
13.1 How to read this roadmap 535
13.2 Where the product is today 536
13.3 Vision and horizons 537
13.4 Timeline Q4 2026 to Q4 2027 539
13.5 Themes and epics 540
13.6 Future features in depth 542
13.7 Prioritisation 552
13.8 Known gaps and tech debt to close 553
13.9 Dependencies and risks 555
13.10 Success metrics per horizon 556
13.11 Release cadence 557
13.12 Open questions 557

This volume combines the full Hydris Field Glasses documentation suite. Each part is one document; its chapters keep their original order.

1 Product Documentation Suite

> **NOTE**
> **HFG-IDX v2.0** · Draft for review · Owner: Product, Hydris. The index to every Hydris Field Glasses product document, who each one is for, and how they fit together

Hydris Field Glasses puts live plant telemetry, guided inspections, approved procedures, audit-record scanning and a grounded voice assistant in the field of view of wastewater and effluent treatment operators. One account powers four surfaces: the website, the operator console, the glasses app and the native Android app, all backed by one API.

This suite is the reference for building, deploying, operating and evolving that product. Every document has a Markdown source in this folder (the source of truth, reviewed like code) and a branded Word edition in `word/` with a cover, document control, a contents page, numbered chapters, diagrams and captioned tables.

1.1 The documents

Table 1. The Hydris Field Glasses documentation suite

#	Document	ID	Answers	Primary readers
00	Documentation Index	HFG-IDX	What exists and where to start	Everyone
01	Product Requirements	HFG-PRD	What we build, for whom, why, and how we measure success	Product, leadership, engineering, pilots
02	Architecture	HFG-ARC	How the system is structured and why	Engineering
03	Data Model	HFG-DAT	Every table, field, state and tenancy rule	Engineering, data, security
04	API Reference	HFG-API	Every endpoint, payload, error and event	Engineering, integrators
05	Mobile App Specification	HFG-MOB	The native Android app, screen by screen	Mobile engineering, design, QA
06	Glasses Integration	HFG-GLS	How Ray-Ban Meta and display glasses connect	Engineering, field support
07	Design System and UX	HFG-DSN	Tokens, components and hands-free interaction rules	Design, front-end, mobile
08	Deployment and Infrastructure	HFG-DEP	Where Hydris runs and how it ships	Platform engineering
09	Security and Privacy	HFG-SEC	Threats, controls, privacy and compliance mapping	Security, legal, customers
10	QA and Test Plan	HFG-QA	How quality is proven, release by release	QA, engineering
11	Operations Runbook	HFG-OPS	Onboarding, support, troubleshooting, incidents	Customer success, support
12	Roadmap and Future Features	HFG-RMP	Where the product goes next, and in what order	Leadership, product, partners

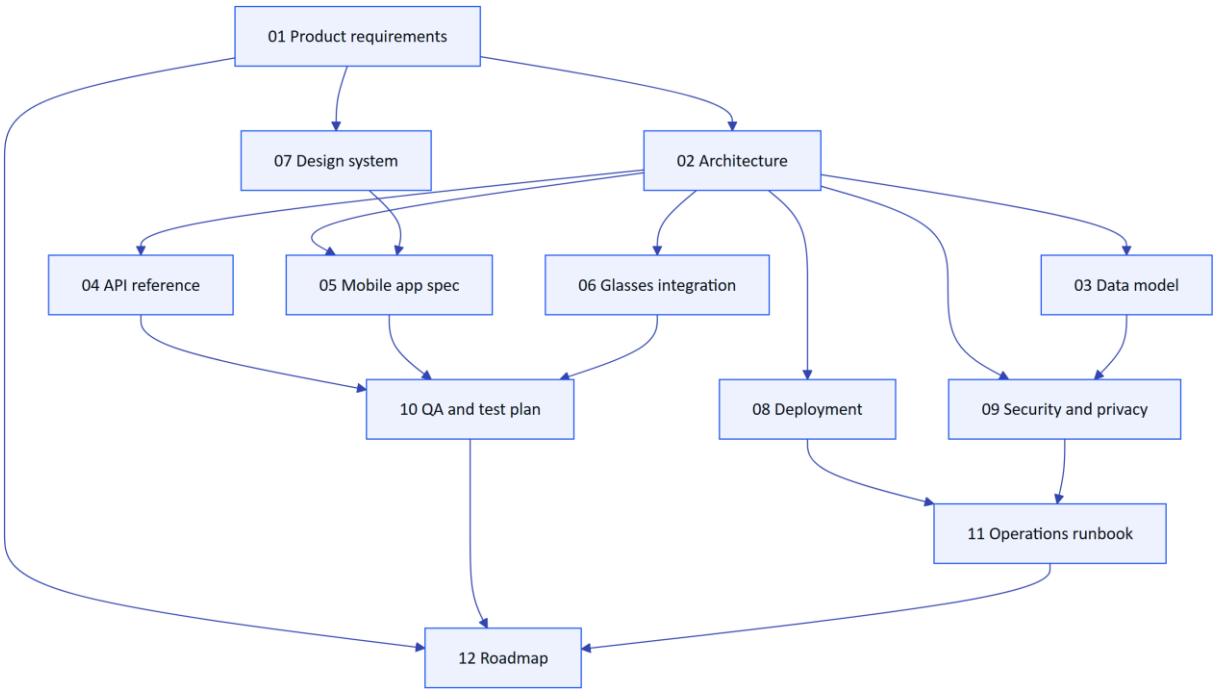

Figure 1. *How the documents relate: requirements flow into design and build documents, which flow into run and prove documents*

1.2 Where to start

Table 2. Suggested reading paths by role

If you are	Read first	Then
New to Hydris	01 Product requirements (executive summary and personas)	02 Architecture (system context), 12 Roadmap
Back-end engineer	02 Architecture, 03 Data model	04 API reference, 09 Security, 10 QA
Android engineer	05 Mobile app spec, 06 Glasses integration	07 Design system, 04 API reference
Web or console engineer	07 Design system, 04 API reference	02 Architecture, 10 QA
Designer	07 Design system, 01 Product requirements (journeys)	05 Mobile app spec
Platform or DevOps	08 Deployment, 09 Security	11 Operations runbook
Customer success or support	11 Operations runbook	06 Glasses integration, 01 Product requirements
Customer security reviewer	09 Security and privacy	03 Data model, 08 Deployment
Leadership or investor	01 Product requirements, 12 Roadmap	08 Deployment (cost model)

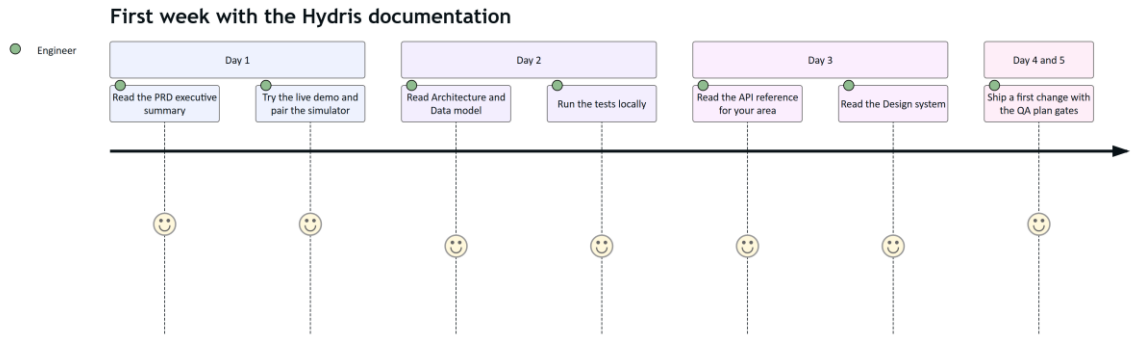

Figure 2. *Reading journey for a new engineer joining the Hydris team*

1.3 The product on one page

4
product surfaces
website, console, glasses app, Android app

1
account everywhere
the same data on every surface

2
people per audit record
field review, then a different approver

12
documents
in this suite

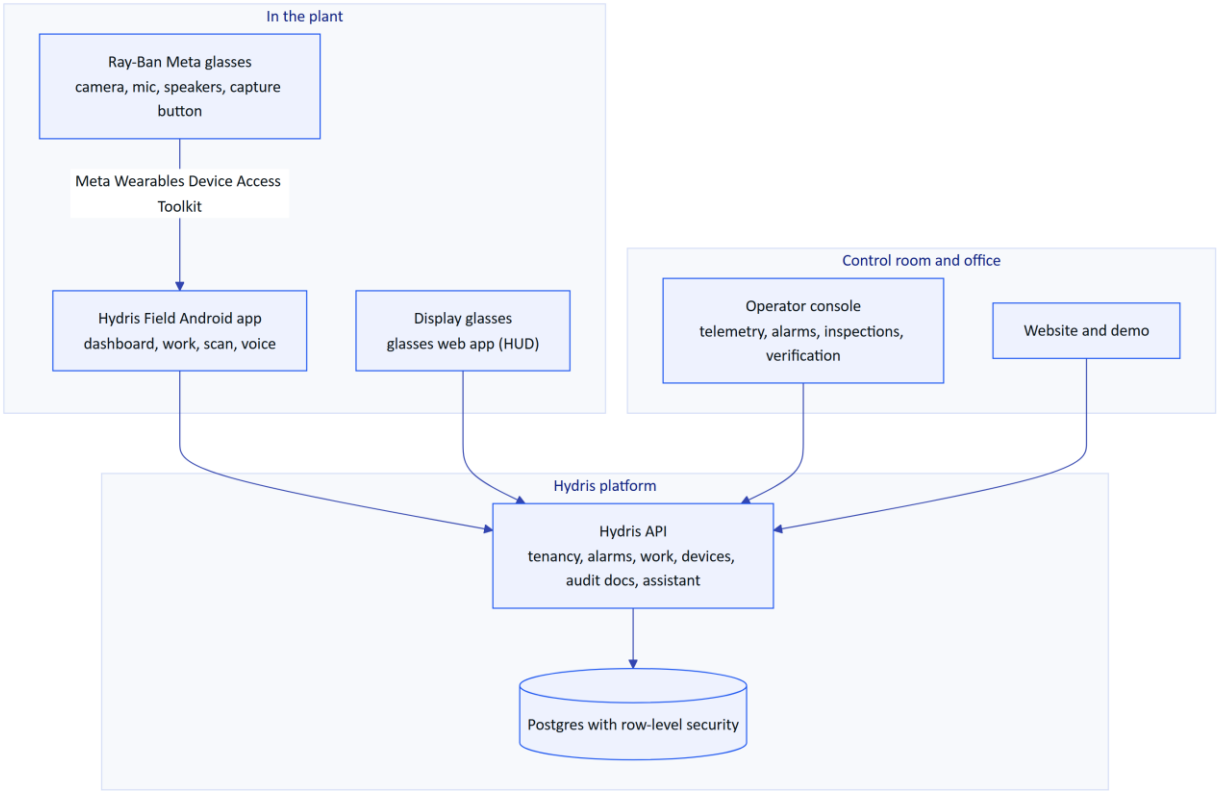

Figure 3. *Hydris Field Glasses at a glance*

The core loops the product supports:

- **See and act on the plant:** live readings and alarms reach the operator on the glasses or phone, with acknowledgement and investigation from the field.
- **Do the work right:** inspections and maintenance rounds with checklists and approved SOPs, hands-free.

- **Turn paper into verified records:** scan a paper audit sheet with the glasses or phone camera; values are read twice on the device, checked for plausibility, confirmed by a reviewer against the paper and approved by a second person before a searchable PDF is produced.
- **Ask the plant:** "Hey Hydris, what is the current pH?" is answered from the account's own data, never invented.

1.4 How the Word editions are made

The Word files are generated, so they never drift from the Markdown. Edit the Markdown, then rebuild.

```
cd docs/product/tools
npm install
node check.js                    # every diagram renders, front matter and house style
node build.js                    # Markdown and Mermaid to branded Word files in ../word
powershell -File finalize.ps1    # Windows with Word: fill contents pages, export PDFs to ../word/pdf
```

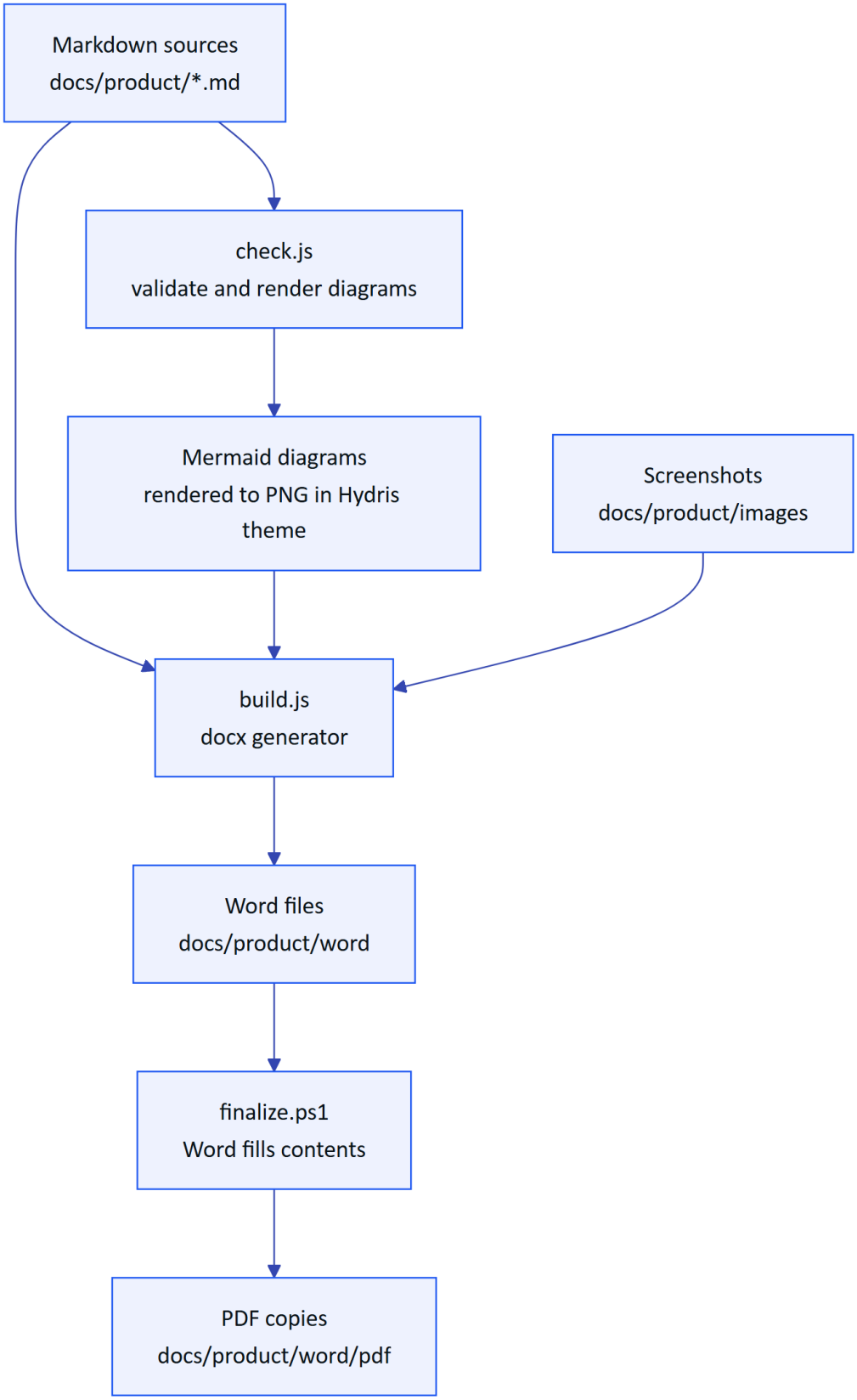

Figure 4. *Documentation build pipeline*

> **NOTE**
> Authoring conventions (captions, callouts, KPI tiles, diagram types, house style) are in `tools/AUTHORING.md`. Screenshots of the web surfaces can be refreshed from the live site with `node screenshots.js`.

1.5 Status conventions

Every document uses the same status words so readers can tell what exists today from what is planned.

Table 3. Status vocabulary used across the suite

Status	Meaning
Built	In the code on `main` and exercised by tests or the live demo
Partial	Exists but incomplete, or only on some surfaces
Planned	Agreed direction, not yet built
Target	A goal value, not a measurement
Estimate	A modelled value, labelled with its assumptions

> **IMPORTANT**
> Ray-Ban Meta support depends on Meta's Wearables Device Access Toolkit and the Meta AI app, which stays installed on the phone for pairing, permissions and firmware. Hydris replaces the Meta AI app for daily work, not for those functions. See 06 Glasses integration.

2 Product Requirements Document

> **NOTE**
>
> **HFG-PRD v2.0** · Draft for review · Owner: Product, Hydris. What Hydris Field Glasses must do for wastewater operators, why, how we will know it works, and what ships when

This document is the single statement of what Hydris Field Glasses is for and what it must do. It is written for the people who build the product, the people who sell and deploy it, and the plant teams who will use it. Every statement about the current product was checked against the repository on `main` at commit `07cca23` (22 October 2026). Where the code and an earlier document disagree, the code wins and this document says so.

Each requirement carries a stable ID, a MoSCoW priority, a status and an acceptance criterion. Status labels mean exactly this:

- **Built**: implemented on `main` and exercised by the product today (in the live demo or in automated tests).
- **Partial**: implemented in part, or implemented in code but not yet validated where it matters (for example on physical Ray-Ban Meta glasses).
- **Planned**: not in the repository. It is a commitment to build, with a target phase in the release plan.

Targets (latency, accuracy, adoption) are labelled as targets. None of them is a measured result unless the text names the measurement.

2.1 Executive summary

Wastewater plant operators spend their shift walking the plant: reading instruments, checking pumps and blowers, sampling effluent, writing values on paper logs, and responding to alarms that usually reach a control room before they reach the person standing next to the equipment. The records they produce are the evidence a plant shows to its regulator, and much of that evidence still lives on paper.

Hydris Field Glasses puts the live state of the plant, the next task and the approved procedure in the operator's line of sight, hands free, and turns every reading, check and paper record they touch into a trustworthy digital record. It does this through five connected surfaces that share one API, one tenant model and one permission matrix:

1. A **public website** with a live, synthetic glasses demo.
2. An **operator console** for supervisors, managers and administrators.
3. A **glasses heads-up app** (web) that runs on smart-glasses runtimes, in a phone layout, and inside a browser simulator.
4. A **native Android app**, Hydris Field, which connects Ray-Ban Meta glasses through Meta's Wearables Device Access Toolkit (DAT 1.0.1).
5. A **multi-tenant API** with row-level security, realtime events and an append-only audit trail.

67	5	143	12
automated tests passing 26 domain + 41 API, run locally on 22 Oct 2026	**product surfaces** website, console, glasses app, Android app, API	**API routes** across 15 modules in apps/api/src/modules	**glasses screens** UI_SCREENS in packages/domain/src/uiprofile.ts

6	16	42	10
roles 38 permissions in packages/domain/src/roles.ts	**measurands** pH to residual chlorine, with unit normalization	**database tables** 23 enums, RLS on every tenant table	**demo scenarios** deterministic, demo tenants only

The platform core is built and running at `https://hydrisview.vercel.app` in demo mode (named guest accounts on synthetic plant data). The main gaps before a customer pilot are: validation of the glasses paths on physical Ray-Ban Meta hardware, email delivery for invitations and password resets, continuous integration, a scheduled-job cadence that suits real data on serverless hosting, and an offline outbox in the Android app. The release plan (chapter "Release plan and milestones") sequences these into a hardening phase, a pilot at one or two Indian plants in Q1 2027, and general availability in Q2 2027.

DECISION

Hydris is read-only towards the plant. It never writes to PLCs, SCADA or equipment. The grounded assistance endpoint says so in every response ("Hydris does not control equipment", `apps/api/src/modules/insights.ts`), and the voice assistant's tools are all read-only (`apps/api/src/modules/voice-assistant.ts`). This keeps Hydris outside the plant's control-system safety case and is a precondition for every requirement in this document.

IMPORTANT

Ray-Ban Meta glasses reach third-party apps only through Meta's DAT, and the glasses are paired, registered and permissioned through the Meta AI app. Hydris cannot replace the Meta AI app; it must stay installed on every operator phone. The Android app checks for it (`GlassesManager.metaAiInstalled()`, package `com.facebook.stella`) and sends the user there to connect.

2.2 Vision and product principles

2.2.1 Vision

Every operator in a water or wastewater plant sees what matters, does the next right thing, and leaves behind a record that stands up to an audit, without putting down their tools.

In three years we want Hydris to be the default way an Indian ETP, STP or CETP runs its rounds, responds to alarms and keeps its compliance records, and a credible choice for plants in other markets with similar discharge-monitoring obligations.

2.2.2 Product principles

These principles settle design arguments. When two requirements conflict, the principle higher in this list wins.

Table 4. Product principles and how the code already applies them

#	Principle	What it means in practice	Where the code applies it
P1	Safety first, read-only towards the plant	Never issue a control command. Point to the approved procedure and the supervisor for safety-critical actions	`/assist` safety notice; assistant system prompt in `voice-assistant.ts`
P2	Never show a guess as a value	Missing data is "no data", never zero. Stale values are marked and excluded from health roll-ups	`freshness()` in `packages/domain/src/freshness.ts`
P3	Every number has provenance	A reading says where it came from (instrument, manual, lab, simulator) and when	`TELEMETRY_SOURCES`, DB check `telemetry_synthetic_source_ck`
P4	History is never overwritten	Corrections append; the original stays visible	Telemetry corrections, audit document `extracted` values, `audit_events`
P5	Humans verify, machines assist	AI and OCR propose; a person confirms; a different person approves	Audit document four-eyes rule in `audit-docs.ts`
P6	Glance, then act	A screen on the glasses answers one question. Voice answers are one or two sentences	HUD screens; assistant `SYSTEM` prompt
P7	Hands busy, still in control	Every operator action works with directional input or voice; the camera only runs on explicit intent	`useHudInput` in `apps/web/src/glasses/input.ts`; scan screens
P8	Tenant isolation is enforced by the database, not by convention	RLS on every tenant table; every request runs as `hydris_app` with `app.org_id`	`drizzle/0001_rls.sql`, `db/tenant.ts`
P9	Synthetic data can never reach a customer tenant	Simulator readings are refused for non-demo organizations	`ingest()` in `modules/telemetry.ts`
P10	One account everywhere	The same user, permissions and data on console, glasses and phone	Shared API; Android `HydrisApi.kt` uses the web session contract

2.2.3 What Hydris is not

- Not a SCADA, DCS or historian. It reads from them (Planned connectors) and never replaces them.
- Not a general-purpose chatbot. The assistant answers only from the tenant's own data and approved documents.
- Not a laboratory information management system. It records lab results as telemetry with `source = lab` and will integrate with a LIMS later (Planned).
- Not a consumer glasses app. It is an enterprise tool distributed to managed devices.

2.3 Problem and market context

2.3.1 The operator's day today

Effluent treatment plants (ETPs), sewage treatment plants (STPs), water treatment plants (WTPs) and common effluent treatment plants (CETPs) run continuously, with shift operators walking fixed rounds. The facility kinds Hydris models are exactly these (`FACILITY_KINDS` in `packages/domain/src/catalog.ts`). A typical round covers intake and screening, equalization, aeration, clarification, filtration, disinfection, sludge handling and the discharge point, which are the process areas Hydris models (`PROCESS_AREA_KINDS`).

Four problems recur:

1. **Paper rounds drift from reality.** Readings are written on clipboards, transcribed later, and diverge from instrument values. Nobody can tell which value was observed and which was copied.
2. **Alarms reach the wrong person.** Alarms land in a control room or on a panel. The operator nearest the pump finds out last. Acknowledging an alarm and actually fixing the condition get conflated, so the record shows alarms "cleared" that were never resolved.
3. **Compliance evidence is unsearchable.** Logbooks, lab sheets, calibration certificates and inspection checklists are paper. Producing evidence for an inspection means digging through files.
4. **Phones are the wrong tool at the asset.** Gloves, ladders, wet hands and sampling bottles make phones awkward. Smart glasses free the hands, but there is no industrial software for them.

2.3.2 Regulatory context

Hydris is designed around discharge-monitoring regimes, and it states only what is well established about them:

- **India.** Discharges are regulated under the Water (Prevention and Control of Pollution) Act, 1974, through consents to establish and operate issued by State Pollution Control Boards (SPCBs) and Pollution Control Committees. General discharge standards are set under the Environment (Protection) Rules, 1986, with plant-specific limits in each consent. In 2014 the Central Pollution Control Board (CPCB) directed the 17 categories of highly polluting industries, and CETPs, to install online continuous effluent and emission monitoring systems (OCEMS) connected to the CPCB and SPCB servers (CPCB online monitoring).
- **United States.** Point-source discharges need National Pollutant Discharge Elimination System (NPDES) permits, with monitoring results reported through Discharge Monitoring Reports.
- **European Union.** The Urban Waste Water Treatment Directive (originally 91/271/EEC, recast in 2024) sets treatment and monitoring requirements for urban wastewater.

The common thread is that plants must monitor continuously, keep records that can be retrieved and trusted, and show who did what and when. Hydris does not certify compliance. It produces records (telemetry with provenance, inspection results, verified audit documents and an append-only audit trail) that make compliance easier to demonstrate.

> **NOTE**
>
> Hydris does not replace OCEMS reporting to CPCB or SPCB servers, and it is not a certified OCEMS. A plant's OCEMS analyzers can feed Hydris through the gateway ingestion API (FR-022) so operators see the same values the regulator sees.

2.3.3 Why glasses, why now

Three things changed. Consumer-grade camera glasses (Ray-Ban Meta) are light, affordable and socially acceptable on a plant floor. Meta opened a device access toolkit (DAT) that lets third-party Android and iOS apps use the glasses' camera, microphone, capture button and speakers. And large language models can now answer spoken questions from structured plant data reliably enough to be useful, provided they are grounded and constrained (chapter "Data and AI requirements").

2.3.4 Competitive landscape (qualitative)

Table 5. Alternatives a plant uses today and where Hydris differs

Alternative	Strength	Gap Hydris fills
Paper logbooks and clipboards	Familiar, no devices needed	No provenance, no search, no realtime alarms, transcription errors
SCADA/HMI in the control room	Authoritative live data and control	Not at the asset; no rounds, no paper-record digitization
Generic mobile forms apps	Cheap digital checklists	Not hands-free, no live telemetry context, no alarm lifecycle, no verified OCR
Remote-assist video apps for glasses	Expert sees what the operator sees	No plant data model, no rounds, no records
CMMS / EAM suites	Strong maintenance planning	Heavy, desktop-first, weak in the field, no glasses

2.4 Personas

Hydris has seven personas. Six map one to one onto the roles in `packages/domain/src/roles.ts`; the platform administrator is a flag on the user (`users.is_platform_admin`).

Table 6. Personas, roles and primary surfaces

Persona	Role in code	Primary surface	Top needs
Field operator (Ravi)	`field_operator`	Glasses, Android app	Next task, live values at the asset, acknowledge alarms, record readings, scan paper logs, ask by voice
Shift supervisor (Meena)	`supervisor`	Console, Android app	Dispatch rounds, triage and assign alarms, review inspections, verify scanned documents
Maintenance engineer (Arjun)	`maintenance_engineer`	Android app, glasses	Work orders, checklists, investigate and resolve alarms, SOPs at the asset
Plant manager (Priya)	`factory_manager`	Console	KPIs and reports, alarm rules, approve audit documents, devices and people
Compliance auditor (Mr Rao)	`viewer` (read plus `report.read`)	Console, PDF exports	Tamper-evident trail, approved documents, exports for the regulator
Organization admin (IT lead)	`org_admin`	Console	Members and invitations, sites, glasses UI profiles, all permissions
Hydris platform admin	`users.is_platform_admin`	Console `/console/platform`	Create and suspend customer organizations

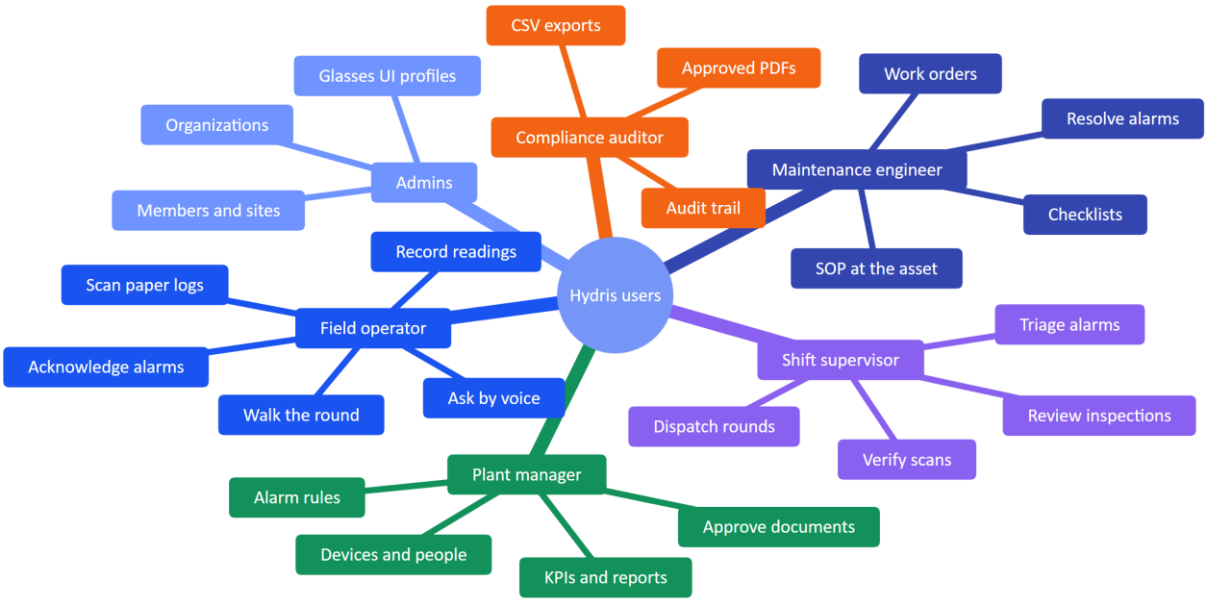

Figure 5. *Personas and their jobs to be done*

2.4.1 Persona detail

Field operator

Ravi is 28, a diploma holder with six years on ETP shifts. He works 8-hour rotating shifts, often alone at night. He wears gloves and safety glasses, carries a sampling bottle and a radio, and has an Android phone in his pocket. His English reading is good; he speaks Hindi and Tamil at home. He wants fewer forms, clear instructions, and to be told quickly when something near him is wrong. He fears being blamed for a reading he copied from someone else.

What success looks like for Ravi: his round is on the glasses in order, each asset shows its live values with how fresh they are, he records a reading with two taps or a sentence, and his paper log becomes a verified PDF without retyping.

Shift supervisor

Meena runs a shift of four operators across two plants. She lives in the console during the day and on her phone at night. She needs to see open alarms by severity, who has them, and which rounds are late. She verifies scanned audit documents against the paper before her manager approves them.

Maintenance engineer

Arjun moves between pumps, blowers and dosing systems. He needs the work order, the checklist and the equipment manual at the asset, and he needs to close the loop on alarms: investigate, fix, resolve with a note.

Plant manager

Priya is accountable to the SPCB for discharge compliance and to her management for uptime and cost. She configures alarm rules, approves audit documents, decides which glasses features each role sees, and reads the reports.

Compliance auditor

Mr Rao is an internal or third-party auditor. He needs read-only access to approved records, the verification trail of every scanned document, and exports. He must never be able to change anything.

Administrators

The organization admin manages members, invitations, sites and glasses UI profiles. The Hydris platform admin creates and suspends customer organizations and has no access to tenant data by default.

2.5 Jobs to be done and user journeys

2.5.1 Jobs to be done

Table 7. Jobs to be done, with the requirements that serve them

#	Job statement	Served by
J1	When I start my round, I want my assigned inspections in order, so I do not miss an asset	FR-040, FR-041, FR-059
J2	When I stand at an asset, I want its live readings and how fresh they are, so I can trust what I record	FR-021, FR-026, FR-095
J3	When an alarm fires near me, I want to acknowledge it and see the recommended action and SOP, so the right person is accountable	FR-031, FR-032, FR-033, FR-112
J4	When I fill a paper log or receive a lab sheet, I want to capture it once and have it verified, so the record is searchable and defensible	FR-060 to FR-069
J5	When my hands are busy, I want to ask by voice and hear a short answer grounded in our data	FR-071, FR-072, FR-074
J6	As a manager, I want to control what each role sees on the glasses without shipping an app update	FR-054
J7	As an admin, I want to pair, reassign, suspend or revoke a pair of glasses in seconds	FR-050 to FR-053, FR-057
J8	As an auditor, I want every approved document and every change traceable to a person and a time	FR-065, FR-066, FR-082
J9	As a prospect, I want to try the product in minutes without talking to sales	FR-083, FR-091, FR-102

2.5.2 Journey: an operator's shift

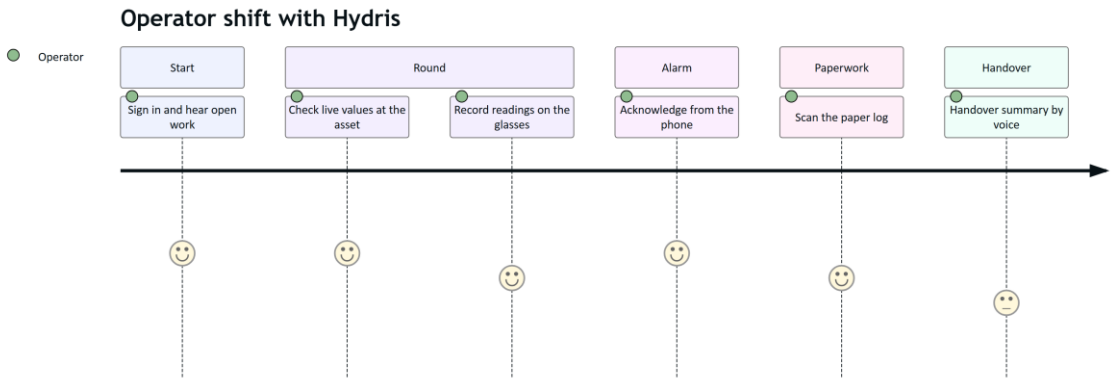

Figure 6. *Field operator journey across one shift (satisfaction 1 to 5, target state)*

The shift journey shows where the product is strong today (alarm acknowledgement, scanning, voice lookups) and where it is weakest: the handover summary is a Planned capability, and inspection execution on the Android app is Planned (FR-044). Inspections are executed on the glasses heads-up app today.

2.5.3 Journey: from paper record to approved evidence

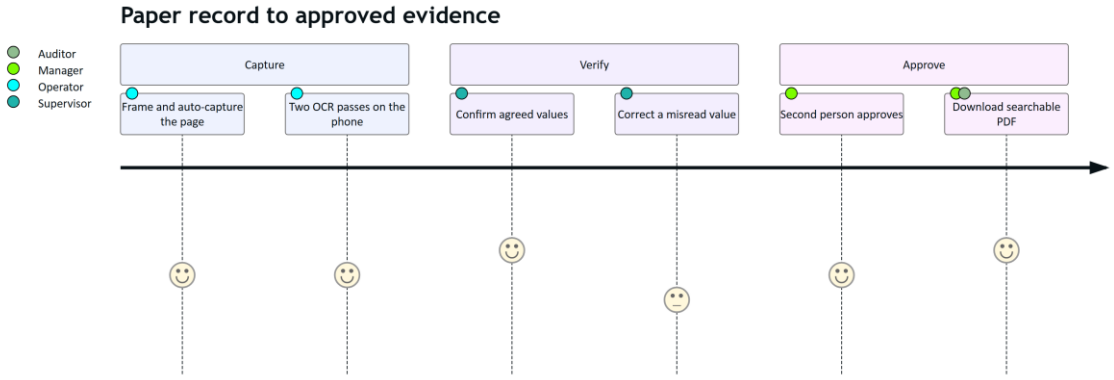

Figure 7. *Supervisor and manager journey for a scanned audit document*

2.5.4 Journey: evaluating Hydris

A prospect lands on the website, opens the live glasses demo (`/glasses?demo=1`, fully client-side synthetic data), and then signs in to the console or the Android app in demo mode with just their name (`POST /auth/demo`). Each guest gets an `org_admin` account in the demo tenant with personal copies of the demo owner's scheduled inspections and open maintenance tasks (`apps/api/src/core/demo-guests.ts`). Nothing they do affects other guests' work, and all data is clearly labelled synthetic.

2.6 Product surfaces and scope

2.6.1 System context

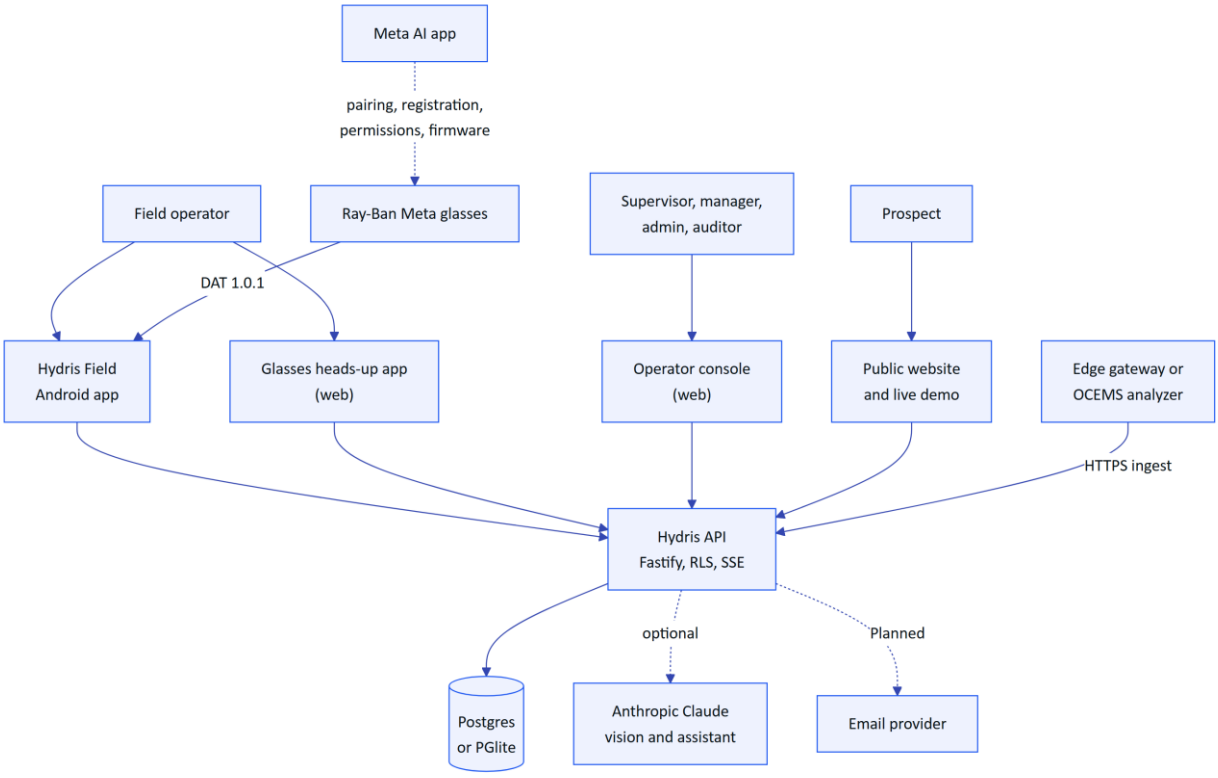

Figure 8. *System context: people, Hydris surfaces and external systems*

2.6.2 Surfaces

Table 8. Product surfaces, where they live and their status

Surface	Code	What it does	Status
Public website	`apps/web/src/app/(site)`	Seven pages (`/`, `/product`, `/technology`, `/operations`, `/demo`, `/contact`, `/privacy`), live glasses demo, contact and demo-request form	Built
Operator console	`apps/web/src/app/console/**`	Sixteen navigation sections in five groups (Operate, Work, Plant, People and devices, Insight) plus Admin, account and platform pages	Built
Glasses heads-up app	`apps/web/src/app/glasses`, `apps/web/src/glasses/*`	Twelve screens, directional input, offline inspection queue, audit scan, voice assistant, phone layout	Built (web). Native glasses display rendering Planned
Simulator and pairing	`/simulator`, `/pair`	Browser glasses simulator; approve a pairing code by QR or typed code	Built
Android app "Hydris Field"	`apps/android`	Sign-in, home, work, alarms, sensors, audit scan with ML Kit OCR, "Hey Hydris", glasses connection and registration through DAT	Partial: built in code, glasses paths not validated on hardware
Android WebView bridge	`apps/glasses-bridge-android`	Hosts the glasses web app and bridges DAT camera, speech, capture button and voice invocation	Partial: superseded by the native app for pilot
API	`apps/api`	143 routes in 15 modules, RLS tenancy, SSE realtime, jobs, demo mode	Built

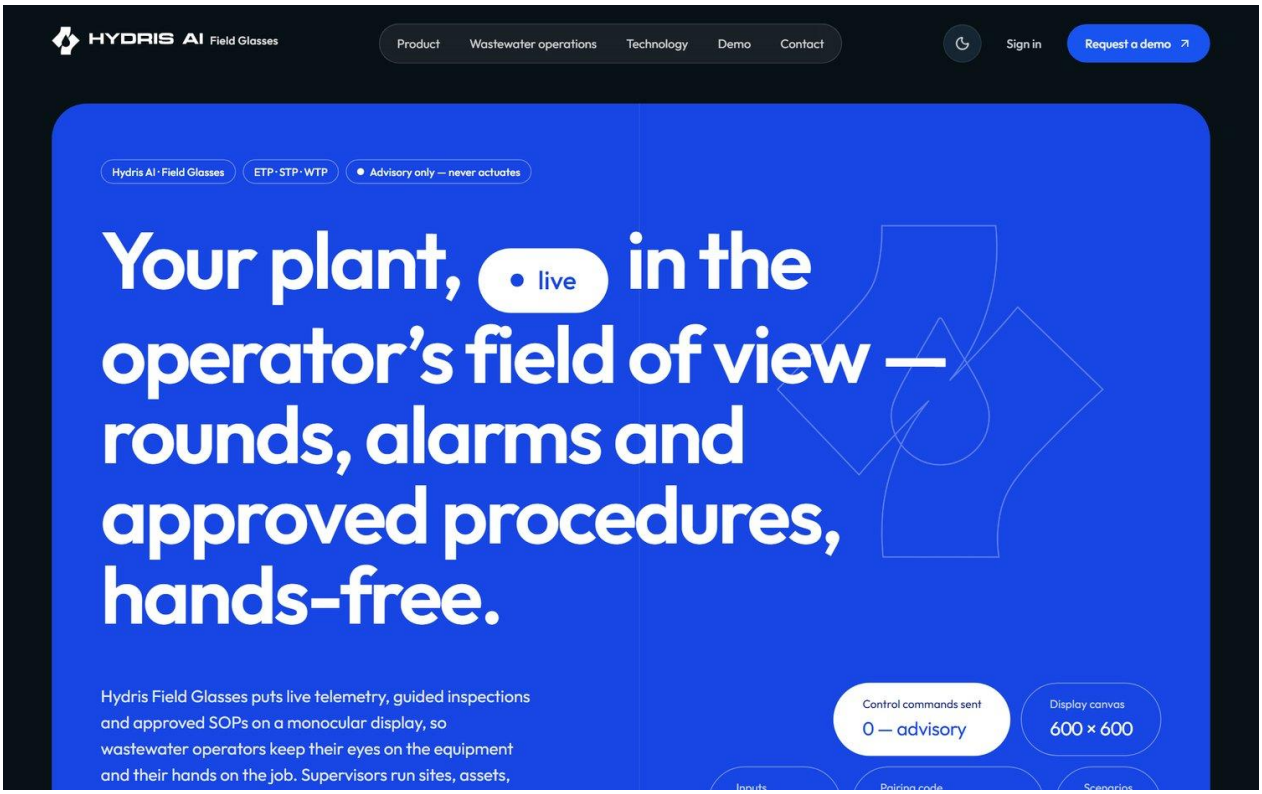

Figure 9. *The public website home page*

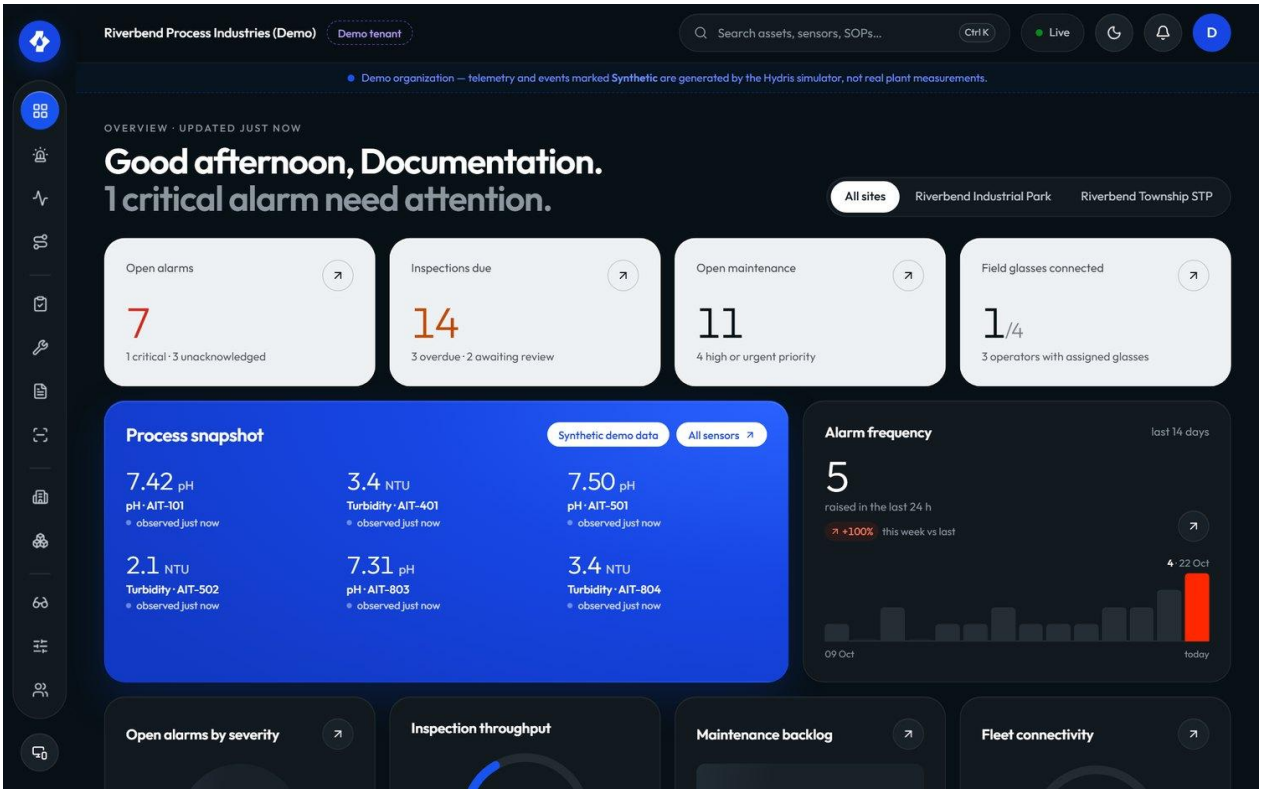

Figure 10. *The operator console overview: KPIs, open alarms and live measurements*

2.6.3 Feature map

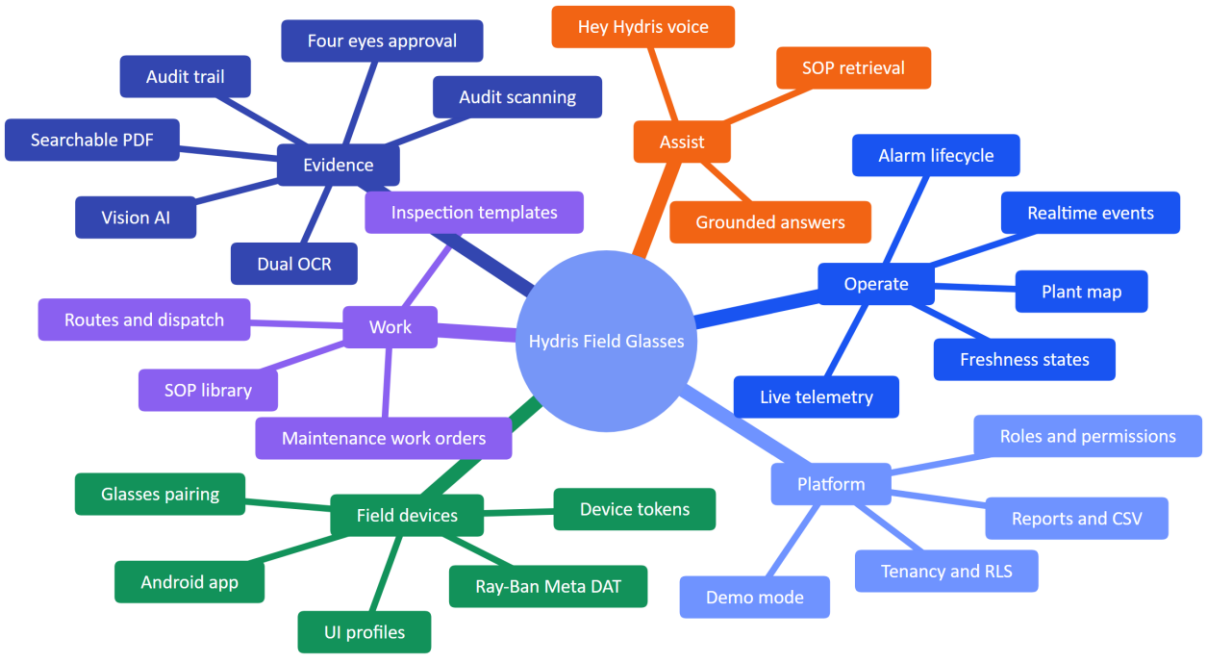

Figure 11. *Feature map of the platform by capability area*

2.6.4 In scope and out of scope

Table 9. Scope boundaries for the pilot and GA releases

Area	In scope	Out of scope
Plant control	Read-only display of values, alarms and procedures	Any write to PLC, SCADA or equipment; setpoint changes
Data ingestion	HTTPS gateway push, manual and lab entry; MQTT and OPC UA adapters (Planned)	Direct fieldbus integration (Modbus, Profibus) inside Hydris
Devices	Ray-Ban Meta through DAT on Android; glasses web app on other runtimes and in a phone layout	Building custom glasses hardware; iOS before GA
Compliance	Records with provenance, verification and audit trail; exports	Certified OCEMS reporting to CPCB or SPCB servers; legal e-signature certification before FR-068
AI	Grounded answers from tenant data; vision extraction with human verification	Autonomous decisions; free-form generation of procedures; answers from the open internet
Accounts	Invitations, MFA, sessions, SSO (Planned)	Self-serve signup for customer tenants (demo mode only)
Languages	English UI and voice	Hindi, Tamil, Marathi and Gujarati voice (Planned, Advanced phase)

2.7 Functional requirements

2.7.1 How to read the requirement tables

Each table lists ID, requirement, MoSCoW priority (**M**ust, **S**hould, **C**ould, **W**on't this release), status, acceptance criterion and the surfaces involved (W website, C console, G glasses app, A Android app, P API). IDs from version 1.0 are unchanged. New IDs take the next free number inside the area's block of ten; new

areas start a new block (FR-090 Android app, FR-100 website and demo, FR-110 documents and SOPs, FR-120 insights and compliance).

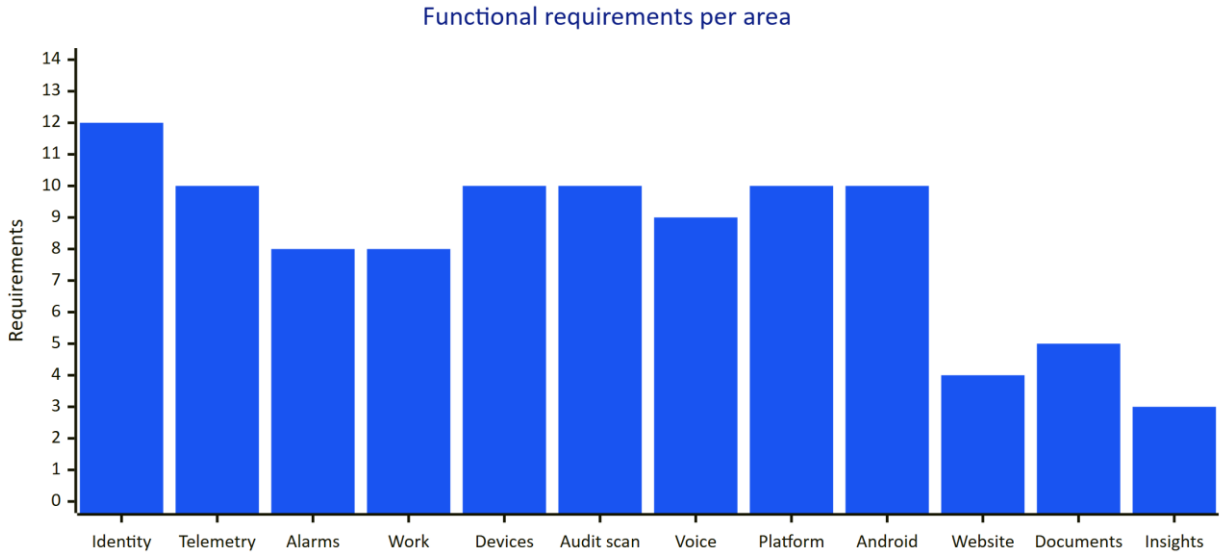

Figure 12. *Requirements per functional area in this edition*

2.7.2 Identity, tenancy and access

Hydris is multi-tenant from the first line of code. Every tenant table has row-level security; every request runs `SET LOCAL ROLE hydris_app` with the organization in `app.org_id`, so the database enforces isolation even if application code has a bug. Roles are a fixed matrix of six roles and 38 permissions; the API enforces it on every request and clients use it only to decide what to show.

Table 10. Identity, tenancy and access requirements

ID	Requirement	Pri	Status	Acceptance criteria	Surface
FR-001	Email and password sign-in with lockout after 5 failed attempts for 15 minutes	M	Built	Sixth attempt within the window returns `423 account_locked`; lockout clears after 15 min (`TTL.lockoutMs`)	C, A, P
FR-002	TOTP MFA (setup, enable, disable, verify step) with the secret encrypted by `DATA_KEY`	S	Built	MFA user cannot obtain a session without a valid code; secret is never stored in clear	C, P
FR-003	Refresh-token rotation with family revocation on reuse (30 s multi-tab grace)	M	Built	Reusing a rotated refresh token outside the grace window revokes the whole family	C, G, A, P
FR-004	Invitations with a 7-day token and role assignment that cannot escalate above the inviter (`assignableRoles`)	M	Built (delivery depends on FR-086)	A factory manager cannot invite an org admin; expired tokens are refused	C, P
FR-005	Password reset with a 30-minute single-use token	M	Partial	Token works once and expires in 30 min. Gap: the email is written to `mail_outbox` and not sent	C, P
FR-006	Multi-organization membership and organization switch	S	Built	A user in two orgs switches with `/auth/switch-org` and sees only the new org's data	C, P

ID	Requirement	Pri	Status	Acceptance criteria	Surface
FR-007	Site-scoped permissions for every role except `org_admin`	M	Built	A site-scoped supervisor cannot read alarms of another site (403/404)	All
FR-008	Database-enforced tenant isolation (RLS)	M	Built	Cross-tenant reads return no rows even with a crafted ID (API test suite)	P
FR-009	Session listing and revocation	S	Built	Revoking a session ends it on the next refresh	C, P
FR-010	Platform organization management (create, suspend)	M	Built	Suspended org members cannot sign in to that org	C, P
FR-011	Single sign-on with SAML 2.0 and OIDC per organization	S	Planned (GA)	An org admin configures an IdP; users sign in with it; local passwords can be disabled	C, A, P
FR-012	Fixed role matrix: six roles, 38 permissions, no escalation path	M	Built	`can(role, permission)` is the only check; unit tests cover the matrix	All

NOTE

The Android app does not yet complete an MFA challenge. If an account has MFA enabled, the app tells the user to sign in on the web console first (`LoginScreen.kt`). That gap is tracked as FR-099.

2.7.3 Plant model and telemetry

The plant model is a hierarchy: site, facility (ETP, STP, WTP, CETP, other), process area (13 kinds), asset (14 types), and sensors on assets. Sensors measure one of 16 measurands with defined units and decimals (`MEASURAND_DEFS`). Readings carry a source (`gateway`, `manual`, `lab`, `simulator`) and a quality. Freshness is derived from the sensor's expected interval: live within 2 times the interval, delayed within 5 times, stale beyond, and `no_data` when nothing was ever received.

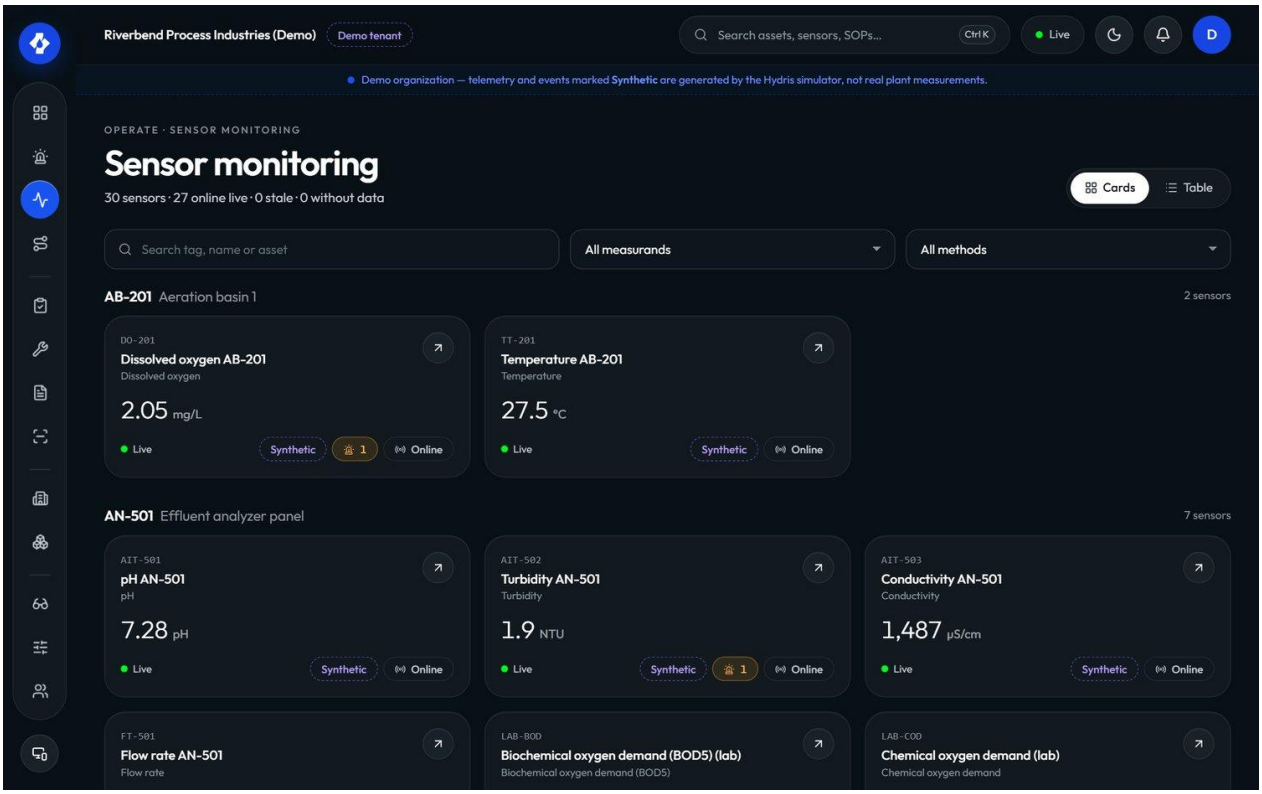

Figure 13. *Sensor monitoring in the console: live values with freshness and provenance*

Table 11. Plant model and telemetry requirements

ID	Requirement	Pri	Status	Acceptance criteria	Surface
FR-020	Site, facility, process area and asset hierarchy with map positions, archive and restore	M	Built	Archived assets disappear from glasses and assistant answers and can be restored	C, G, P
FR-021	Sensors for 16 measurands with unit normalization	M	Built	A reading in a compatible unit is stored in the canonical unit	C, P
FR-022	HTTPS gateway ingestion `POST /ingest/v1/readings` with an `hgw_` API key, up to 500 readings per call, idempotent on sensor and timestamp	M	Built	Replaying the same batch creates no duplicates; a revoked key gets 401	P
FR-023	Manual and lab entry with provenance	M	Built	Entries show source "Manual" or "Lab" everywhere they appear	C, P
FR-024	Append-only telemetry corrections (correct or invalidate)	M	Built	The original reading stays visible with its correction and author	C, P
FR-025	History, raw readings and CSV export per sensor	S	Built	CSV export opens in a spreadsheet without formula injection (leading `=+-@` escaped)	C, A, P
FR-026	Freshness states live, delayed, stale, no data; missing data never shown as a value	M	Built	A sensor with no readings shows "No data", never 0	All
FR-027	MQTT and OPC UA ingestion adapters	S	Planned (Advanced)	A gateway publishing over MQTT or OPC UA lands readings with the same guarantees as FR-022	P
FR-028	Read-only SCADA and historian connectors	C	Planned (Advanced)	Values from a historian appear with source and timestamp; no write path exists	P

ID	Requirement	Pri	Status	Acceptance criteria	Surface
FR-029	Synthetic data is labelled everywhere and refused for non-demo tenants	M	Built	Simulator readings into a customer org are rejected with `synthetic_not_allowed`; DB check ties `synthetic` to `source = simulator`	All

2.7.4 Alarms

Alarms come from threshold rules (high-high, high, low, low-low) with a deadband (hysteresis) and an on-delay, or from a stale-data rule. An alarm has a severity (critical, high, medium, low) and moves through a five-state lifecycle. Acknowledging records that a person has seen it; it does not change the process. Resolving records that the condition was dealt with and requires a note. Closing archives the event after review.

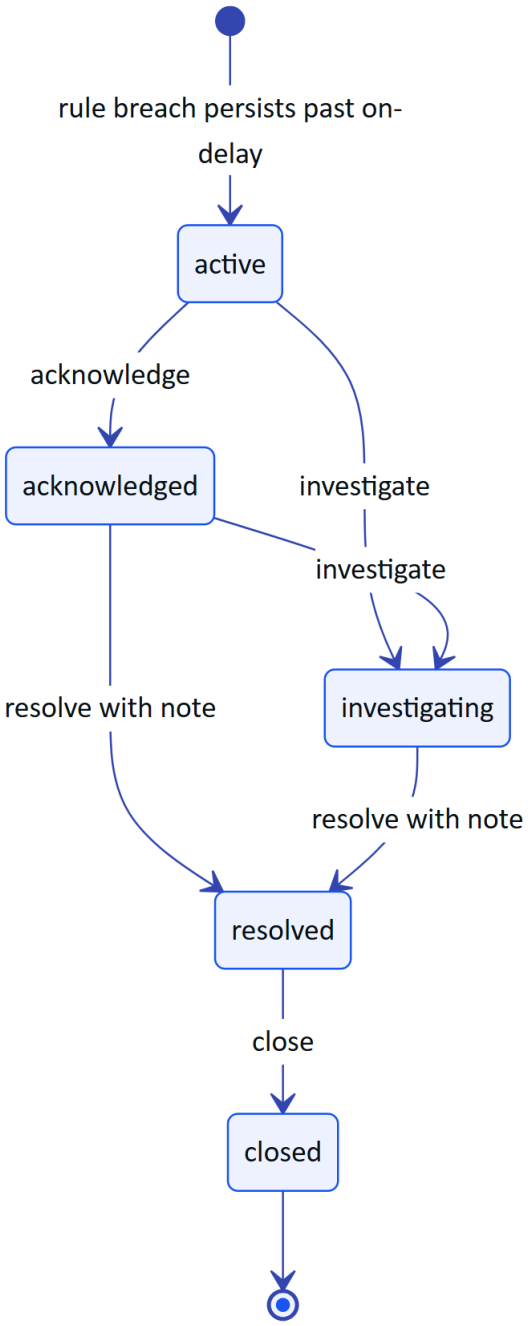

Figure 14. *Alarm lifecycle (packages/domain/src/alarms.ts)*

Table 12. **Alarm transitions, permissions and who holds them**

Action	From	To	Permission	Note required	Held by
Acknowledge	active	acknowledged	`alarm.acknowledge`	No	All roles except viewer
Investigate	active, acknowledged	investigating	`alarm.investigate`	No	Admin, manager, supervisor, maintenance engineer
Resolve	acknowledged, investigating	resolved	`alarm.resolve`	Yes	Admin, manager, supervisor, maintenance engineer
Close	resolved	closed	`alarm.close`	No	Admin, manager, supervisor

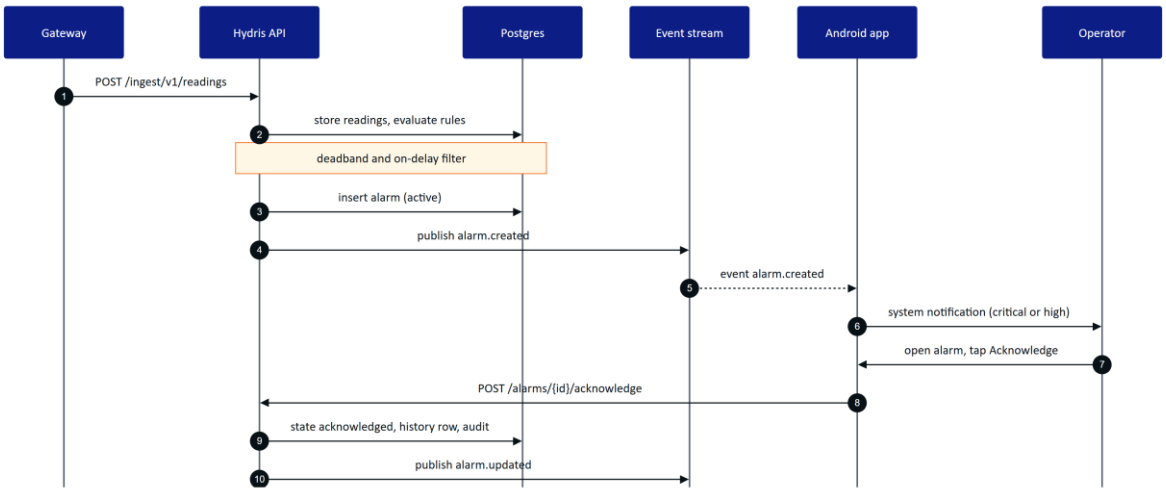

Figure 15. *From a sensor reading to an acknowledged alarm on the operator's phone*

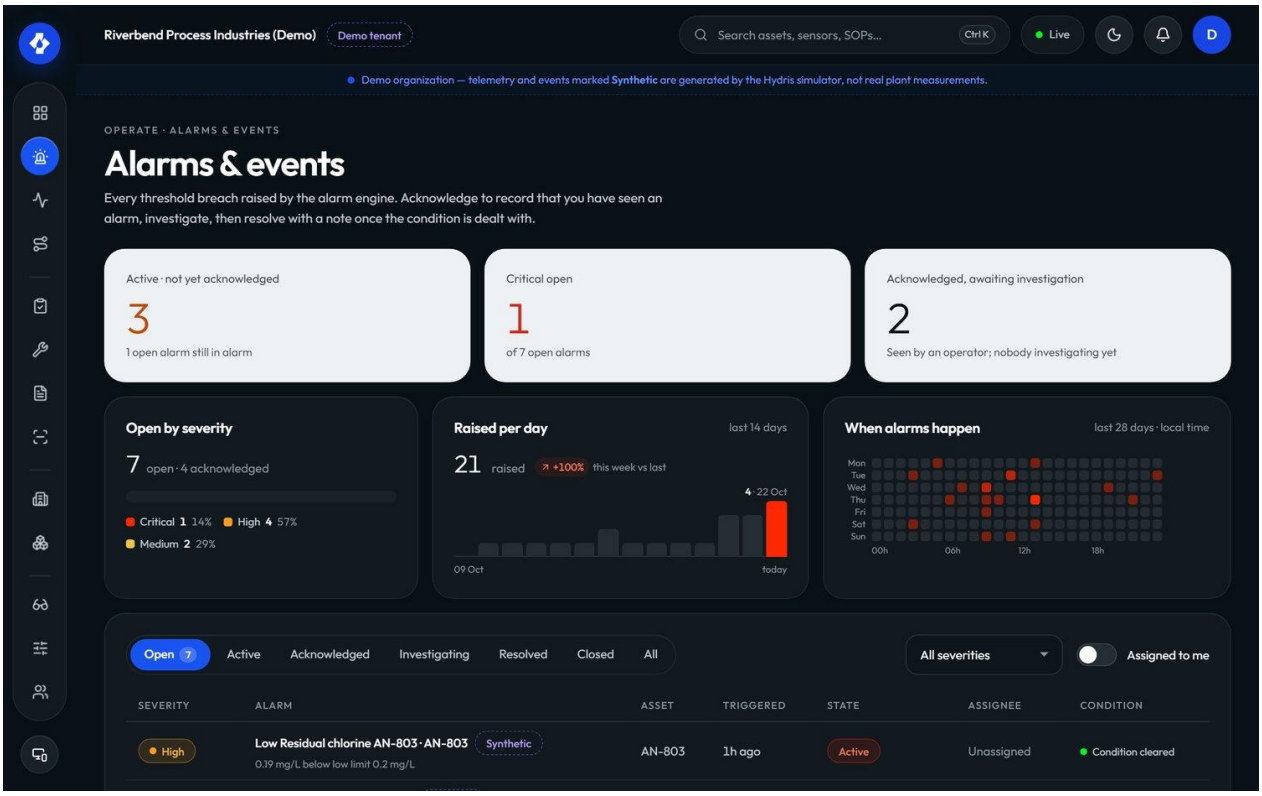

Figure 16. *Alarms and events in the console*

Table 13. Alarm requirements

ID	Requirement	Pri	Status	Acceptance criteria	Surface
FR-030	Threshold rules (high-high, high, low, low-low, stale) with deadband and on-delay	M	Built	A value oscillating inside the deadband does not raise or clear repeatedly; a breach shorter than the on-delay raises nothing	C, P
FR-031	Lifecycle active, acknowledged, investigating, resolved, closed; no resolve from active; resolution note required	M	Built	`POST /alarms/{id}/resolve` without a note returns `400 note_required`	C, G, A, P

ID	Requirement	Pri	Status	Acceptance criteria	Surface
FR-032	Assignment, notes, history, recommended action and linked SOP	M	Built	Alarm detail shows history with actors, the rule's procedure and follow-up tasks	C, A, P
FR-033	Realtime push of alarm events to console, glasses and Android	M	Built	A new alarm appears without reload within the NFR-002 target	C, G, A, P
FR-034	Push notifications through Firebase Cloud Messaging when the app is not running	S	Planned (GA)	A critical alarm reaches a phone whose app process is stopped	A, P
FR-035	Stale-data alarms evaluated on a schedule	M	Partial	Long-running server evaluates every 30 s (`startJobs`). Gap: on Vercel the cron runs once a day (`vercel.json`, `0 3 * * *`)	P
FR-037	Escalation when an alarm stays unacknowledged (notify supervisor after a configurable time)	S	Planned (Pilot)	An unacknowledged critical alarm notifies the site supervisor after N minutes; the escalation is in the alarm history	C, A, P
FR-038	Alarm shelving with reason and expiry	C	Planned (GA)	A shelved alarm is hidden from the operator's active list until expiry; shelving is audited	C, P

NOTE

FR-036 from the draft of this edition (Android alarm notifications) was folded into FR-097 to keep Android requirements in one block. The ID is retired and will not be reused.

2.7.5 Inspections and maintenance

Inspection templates are versioned and made of steps of five kinds: check, reading (with expected range), choice (with abnormal options), text and photo. Routes group templates in order and are dispatched to operators. An inspection moves through scheduled, in progress, paused, submitted, then approved or returned (rejected), or cancelled. A reading outside its expected range, a failed check or an abnormal choice marks the step abnormal automatically (`stepOutcome`).

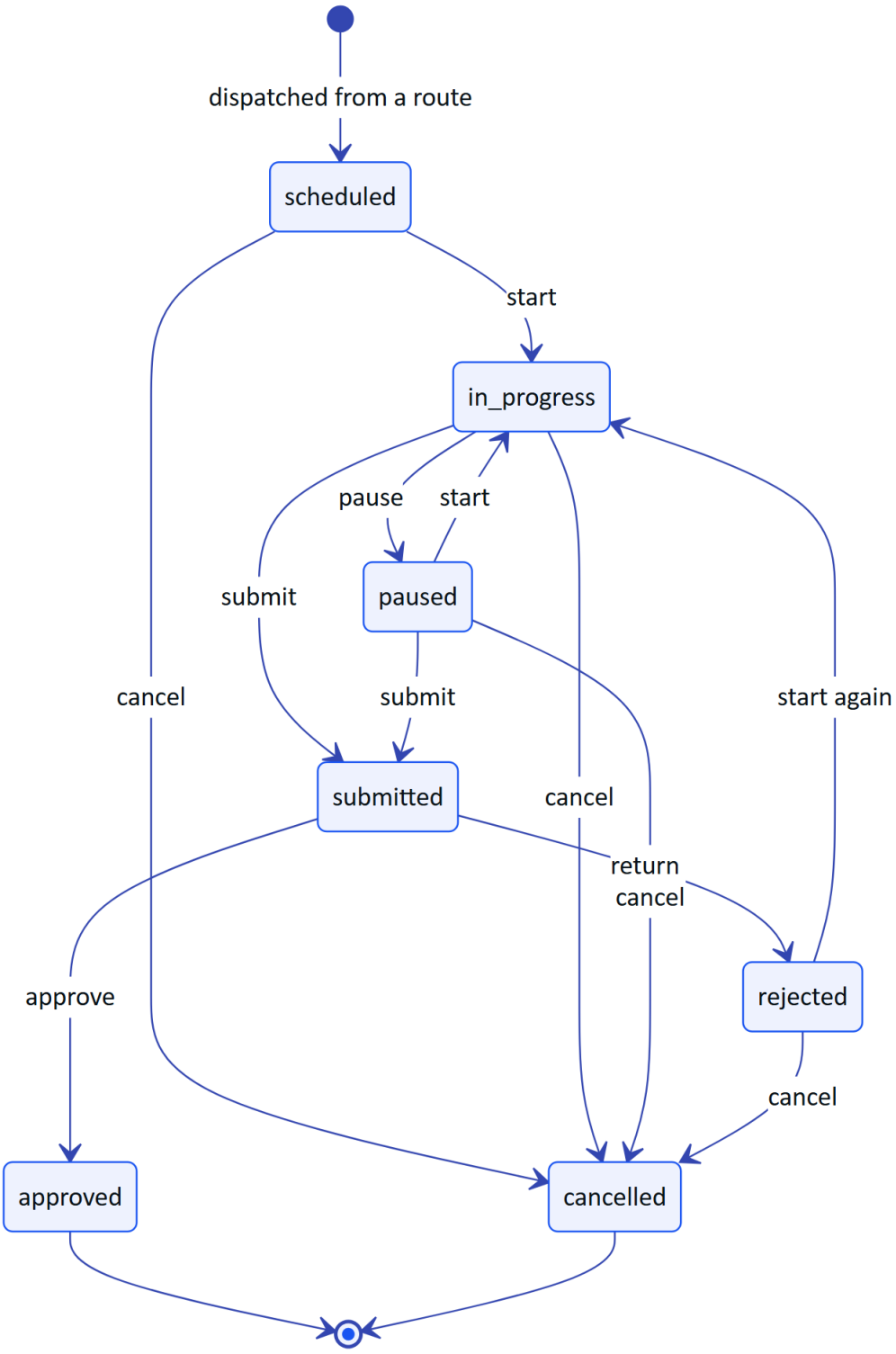

Figure 17. *Inspection lifecycle (packages/domain/src/workflows.ts)*

On the glasses, every step response is queued locally first and then flushed; replays are idempotent because each response carries a client operation ID (`apps/web/src/glasses/data.ts`). An operator who loses connectivity mid-round keeps working and the record syncs when the network returns. The "Glasses lose connectivity" demo scenario (`glasses_network_loss`) exercises exactly this.

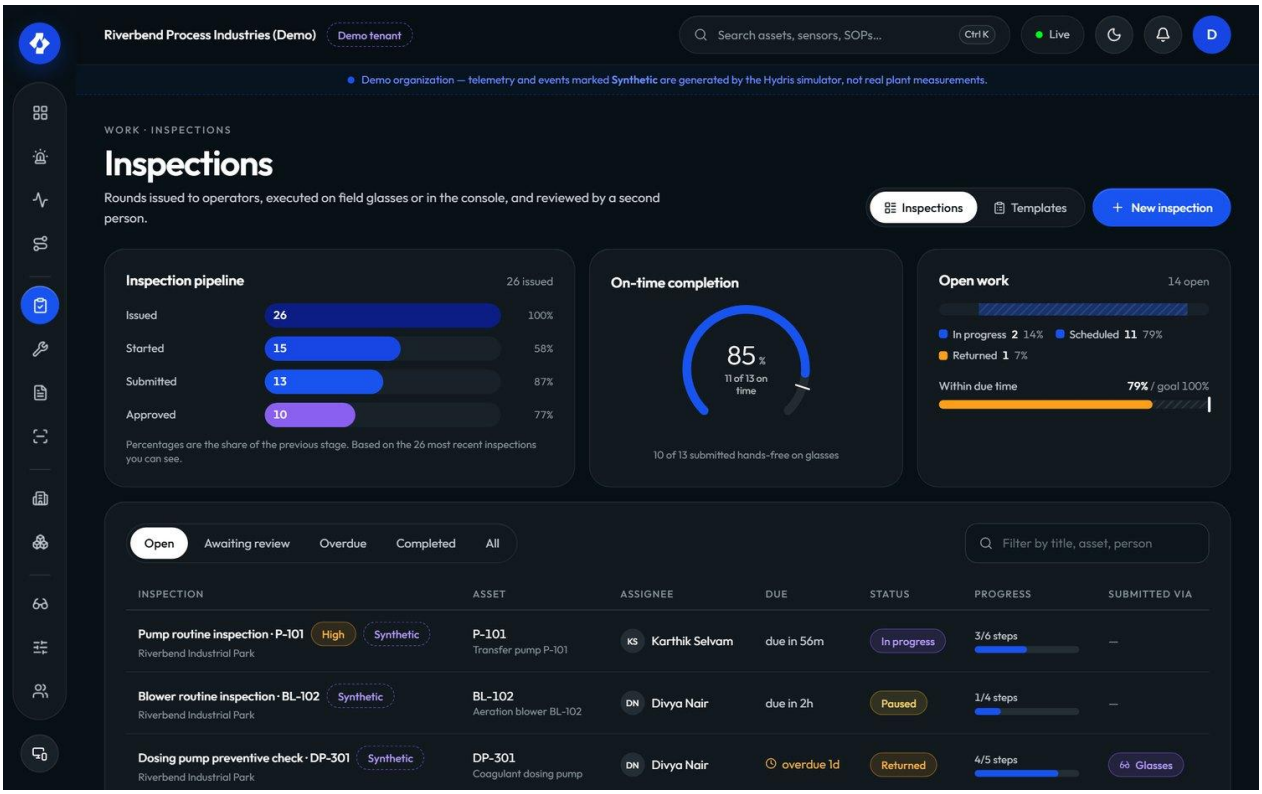

Figure 18. *Inspections in the console*

Maintenance tasks are corrective, preventive, inspection follow-ups or alarm follow-ups, with a priority (low to urgent), a checklist and an activity log. Their lifecycle is open, assigned, in progress, on hold, completed or cancelled, with reopen from completed or cancelled.

Table 14. Inspection and maintenance requirements

ID	Requirement	Pri	Status	Acceptance criteria	Surface
FR-040	Versioned inspection templates (check, reading, choice, text, photo) and routes with dispatch	M	Built	Editing a template creates a new version; dispatched inspections keep the version they were created with	C, P
FR-041	Execution on the glasses with an offline queue and idempotent replay	M	Built	Responses recorded offline appear once after reconnect (Playwright journey)	G, P
FR-042	Submit, review, approve or return	M	Built	A returned inspection can be restarted and resubmitted; history shows both submissions	C, G, P
FR-043	Maintenance tasks with checklist, activity and follow-ups from alarms and inspections	M	Built	A follow-up created from an alarm links back to it and shows on the alarm detail	C, A, P
FR-044	Inspection execution in the Android app (step entry, submit)	S	Planned (Pilot)	An operator completes a round on the phone with the same validation and offline guarantees as FR-041	A, P
FR-045	Photo steps captured with the glasses or phone camera	S	Partial	Photo steps and attachments exist (`/attachments`); the glasses app shows "attach from console" because camera capture is not wired to photo steps	G, A, P
FR-046	Tick maintenance checklist items from the Android app	M	Built	Ticking an item on the phone updates the console task in realtime	A, P

ID	Requirement	Pri	Status	Acceptance criteria	Surface
FR-047	Recurring preventive maintenance and inspection schedules (calendar or runtime based)	S	Planned (GA)	A weekly route generates inspections automatically; a runtime-hours rule creates a task when crossed	C, P

2.7.6 Devices and glasses

A pair of glasses becomes a Hydris device through a pairing flow shaped like the OAuth 2.0 Device Authorization Grant (RFC 8628). The glasses ask for a challenge and display a short code and a QR link; a signed-in person approves it on another device and chooses the site and operator; the glasses, holding a separate poll secret that is never displayed, then claim a device credential. The QR code never contains a credential.

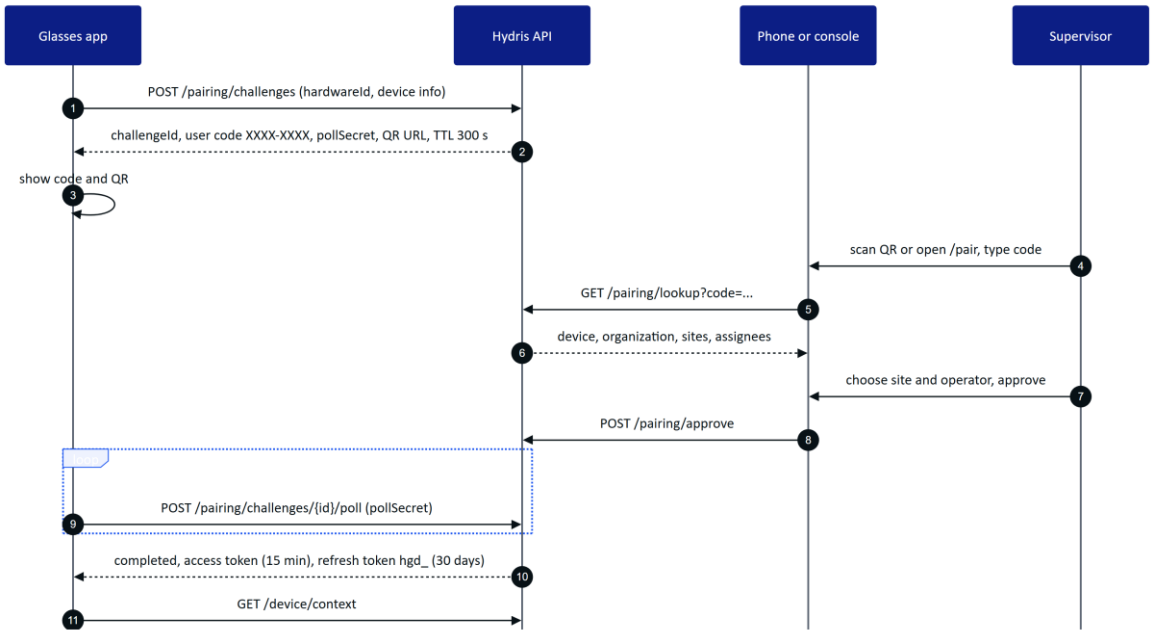

Figure 19. *Glasses pairing, an RFC 8628-style device flow (apps/api/src/modules/devices.ts)*

Device credentials are short-lived access tokens (15 minutes) plus a rotating 30-day refresh token with reuse detection. A session version on the device record lets an administrator invalidate every outstanding token at once. Suspending, revoking or unassigning a device takes effect on its next request and on the realtime stream's 30-second revalidation.

Device permissions are capped: whatever the assigned operator's role, a device can only do what `DEVICE_PERMISSIONS` allows. Managers further shape the glasses experience with **UI profiles**: versioned configurations of which screens are enabled and in what order, which home widgets show, which measurements are allowed and in which order per asset type, which actions are permitted, and layout (density, text scale, handedness). The server enforces the same configuration it sends to the device.

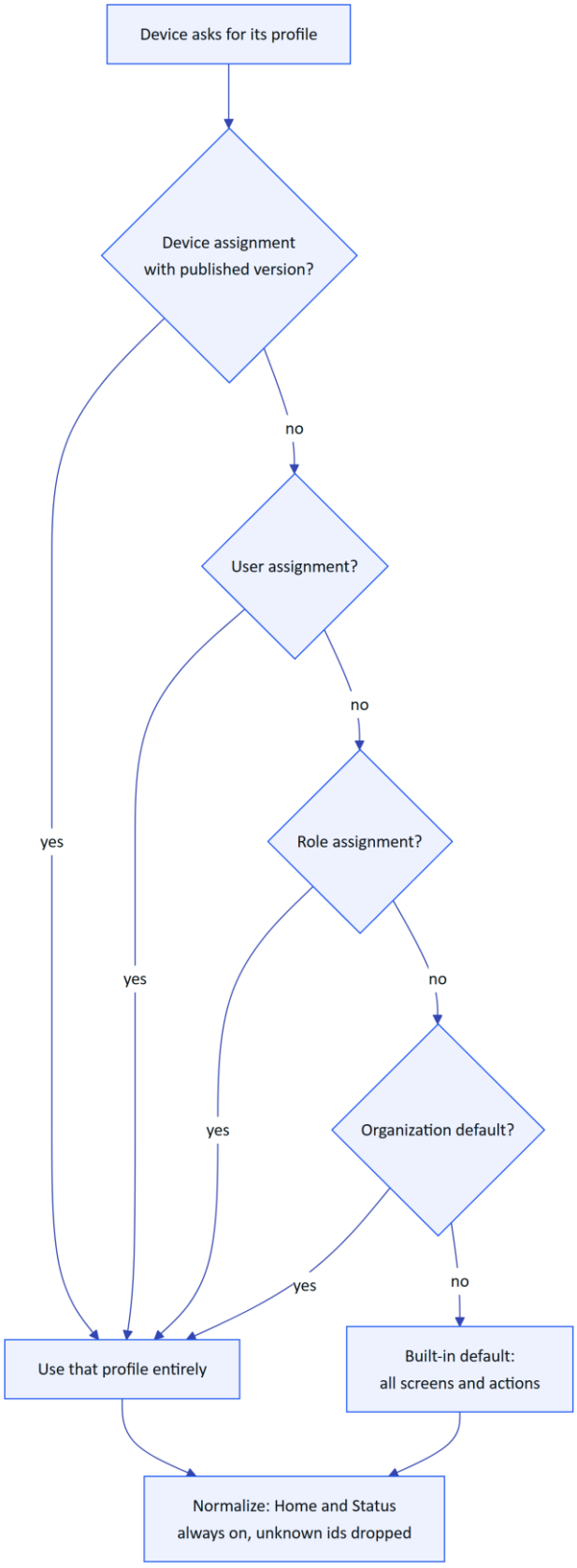

Figure 20. *How the effective glasses UI profile is resolved (packages/domain/src/uiprofile.ts)*

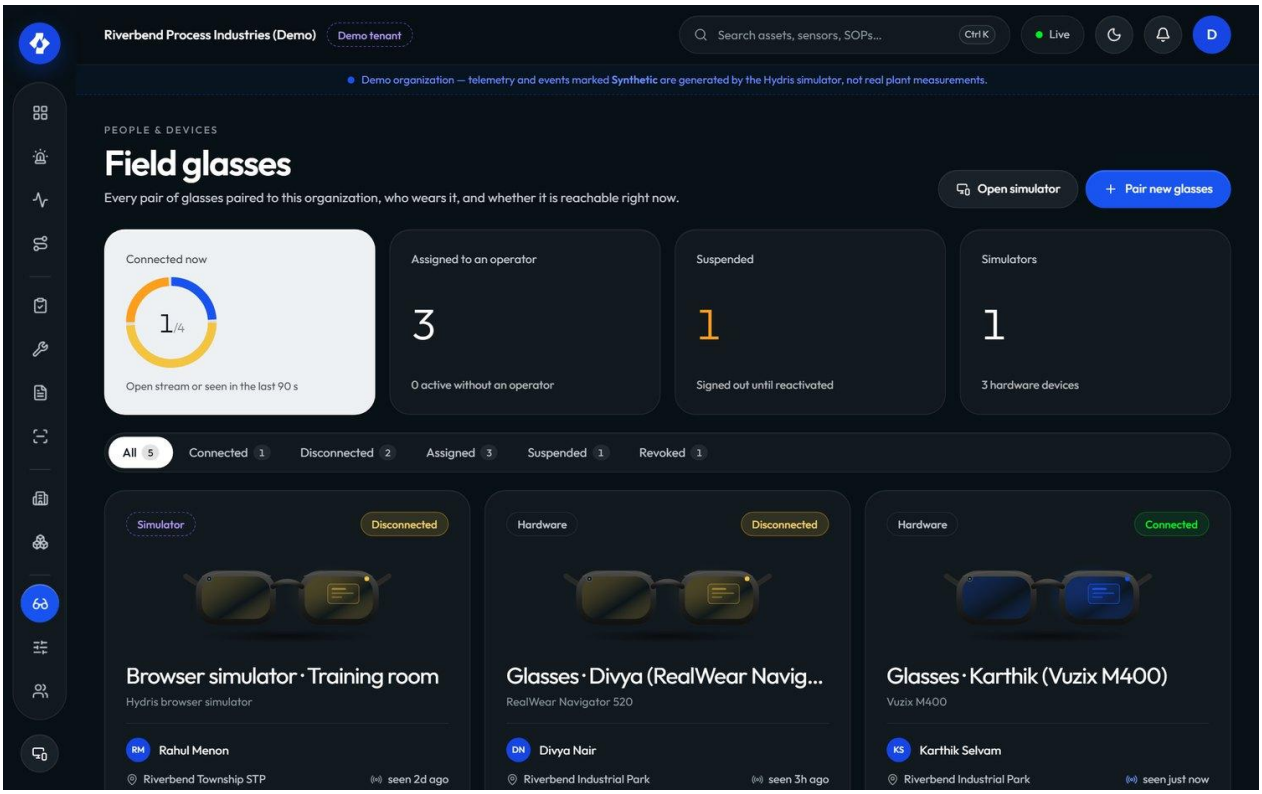

Figure 21. *Field glasses in the console: pairing state, assignment and last seen*

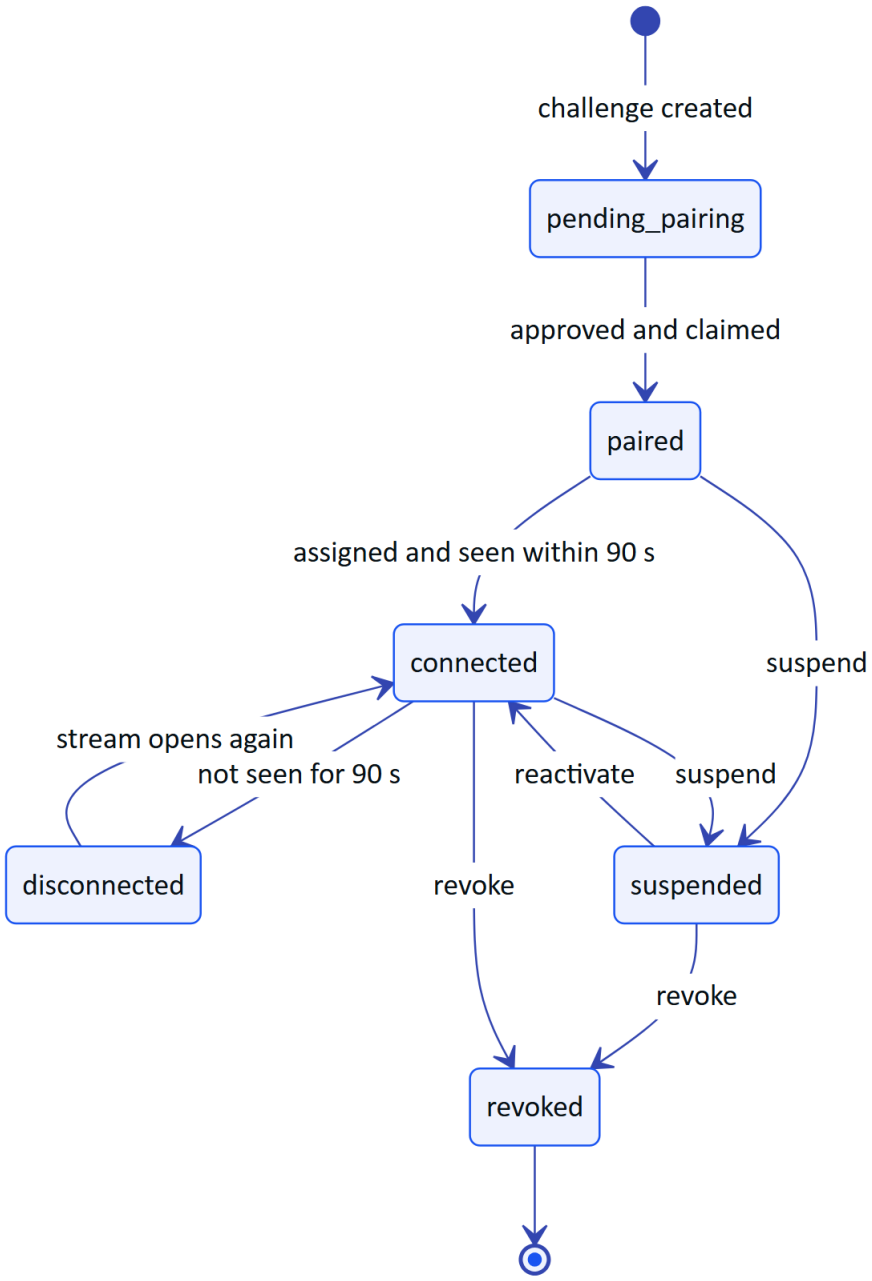

Figure 22. *Composite device states shown to administrators (packages/domain/src/devices.ts)*

Table 15. **Device and glasses requirements**

ID	Requirement	Pri	Status	Acceptance criteria	Surface
FR-050	Pairing with an 8-character code (no 0, O, 1, I, L, U), QR link, 5-minute TTL, 3 s polling, approval with site and assignee	M	Built	Polling faster than 3 s returns `slow_down`; an expired code cannot be approved; a used code returns `already_used` (Playwright journey)	G, C, A, P
FR-051	Device tokens: 15-minute access, 30-day rotating `hgd_` refresh with reuse detection, session version	M	Built	Reusing a rotated device refresh token revokes the token family and is audited	G, A, P
FR-052	Suspend, reactivate, revoke, reassign; revocation effective on next request and within 30 s on the event stream	M	Built	A revoked device's open stream receives `session.ended` within 30 s	C, P

ID	Requirement	Pri	Status	Acceptance criteria	Surface
FR-053	Device permission ceiling (`DEVICE_PERMISSIONS`)	M	Built	A device paired to an org admin still cannot manage members	P
FR-054	UI profiles with draft, publish, rollback; precedence device, user, role, org; live push	S	Built	Publishing a profile changes the glasses screens without a reload; rollback restores the previous version	C, G, P
FR-055	Ray-Ban Meta camera through DAT (preview stream and full-resolution photo)	M	Partial	Code complete in `GlassesManager.kt` and the WebView bridge. Gap: not validated on physical glasses	A
FR-056	Glasses capture button takes the page photo or starts a scan	S	Partial	Short press emits a capture in `GlassesManager.captureButton`. Gap: not validated on hardware	A
FR-057	The Android app registers the connected glasses as a Hydris device using the pairing endpoints	M	Partial	`Repository.registerGlasses` runs challenge, lookup, approve (first permitted site) and poll, then stores device tokens. Gap: depends on a real DAT device id; not validated on hardware	A, P
FR-058	Log out and unpair from the glasses menu, ending the device's server sessions	M	Built	After `/device/logout` the device's refresh token is rejected and the glasses can be paired again by anyone	G, A, P
FR-059	Glasses heads-up app with 12 screens and a six-key directional input vocabulary, plus a phone layout	M	Built	Every screen is reachable with up, down, left, right, select and back; browser back never leaves the app	G

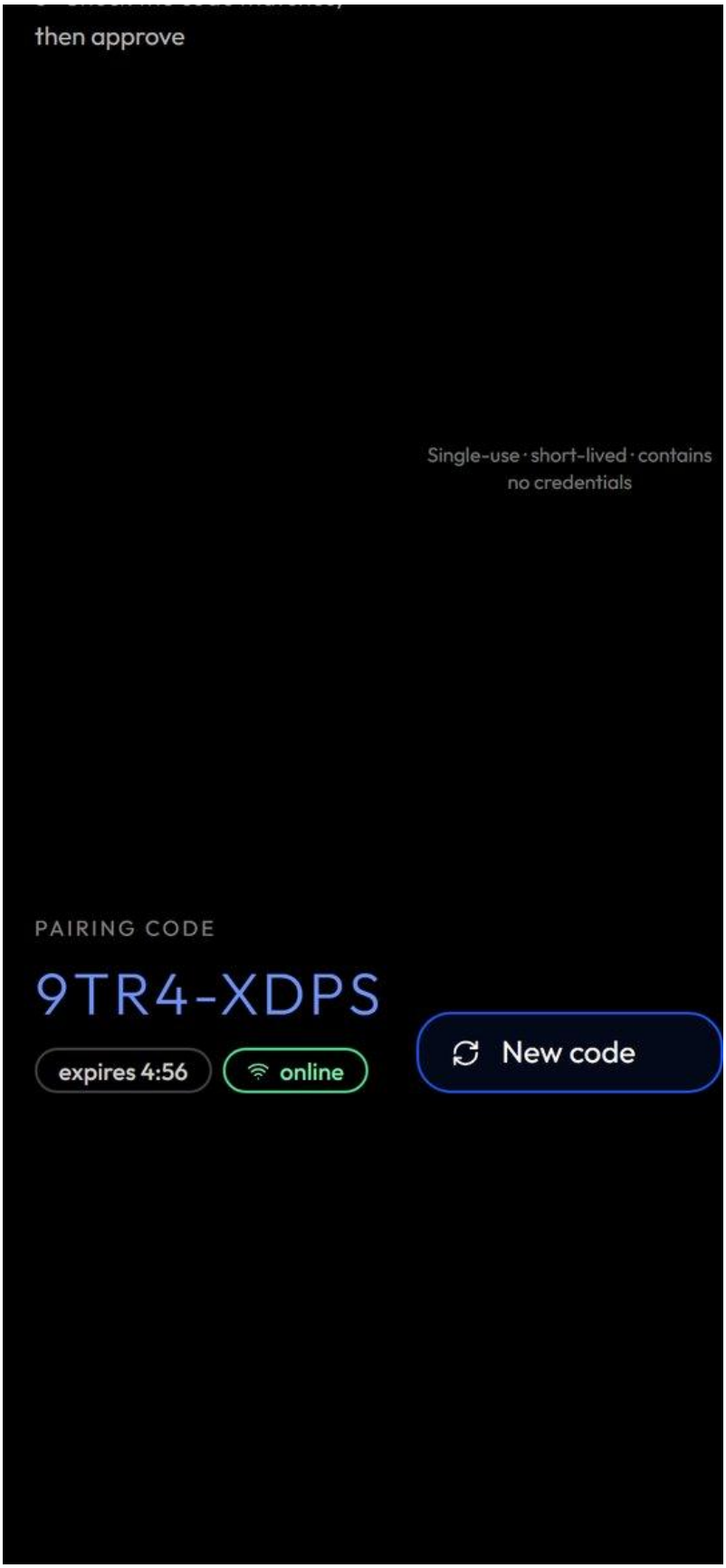

Figure 23. *The glasses app in its phone layout with a pairing code*

2.7.7 Android app: Hydris Field

The native Android app is the operator's daily driver and the only supported way to reach Ray-Ban Meta glasses for the pilot. It is written in Kotlin with Jetpack Compose, targets Android 12 and later (min SDK 31, matching DAT), and uses the same API and session contract as the web console. It has five tabs (Home, Work, Scan, Hydris, Glasses) plus alarm and sensor detail screens (`apps/android/app/src/main/java/ai/hydris/field/ui/Root.kt`).

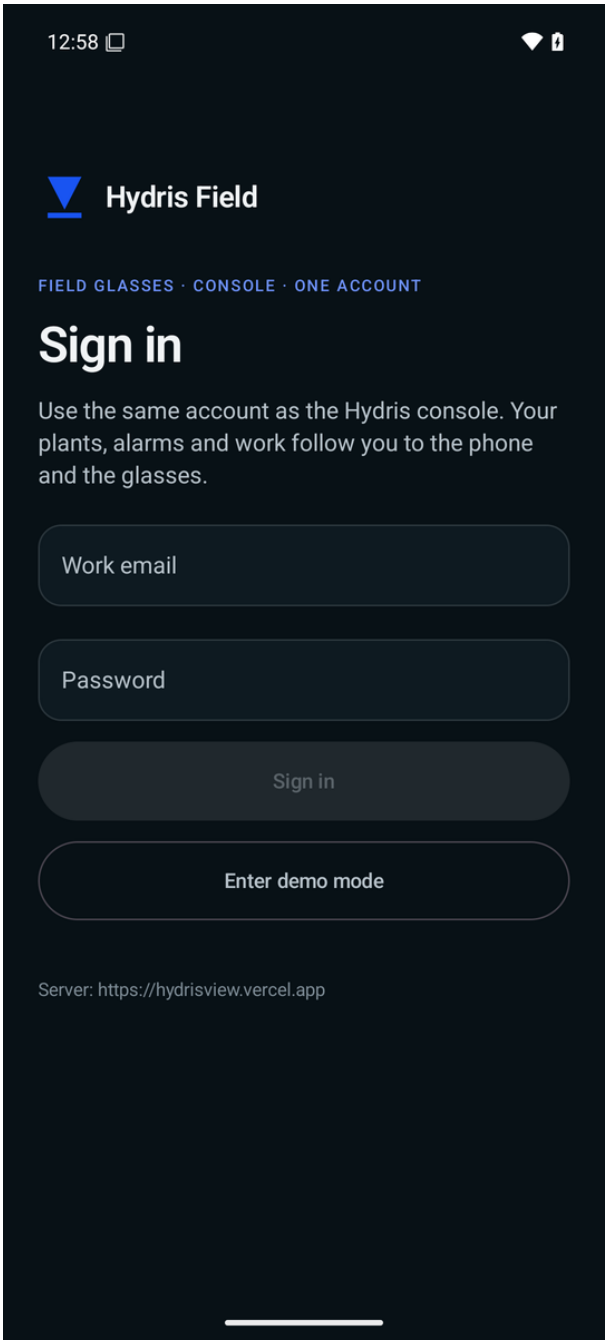

Figure 24. *Sign-in on the Android app*

Figure 25. *Home: greeting, 14-day alarm trend, asset health and live measurements*

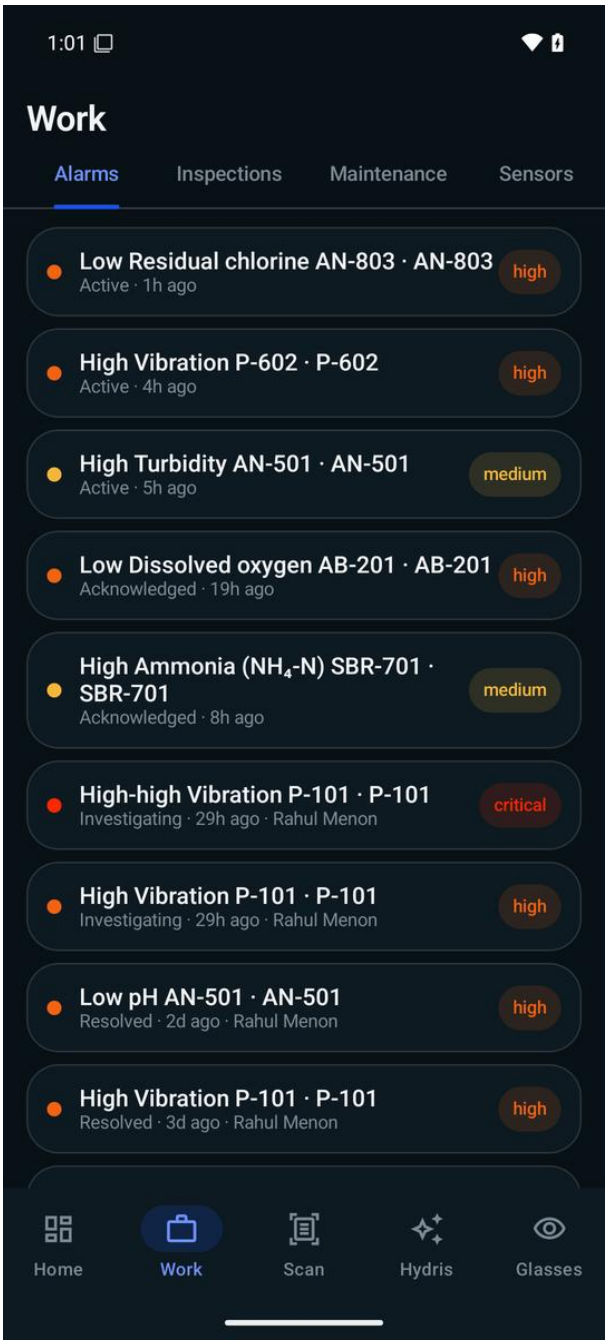

Figure 26. *Work: alarms tab with severity and state*

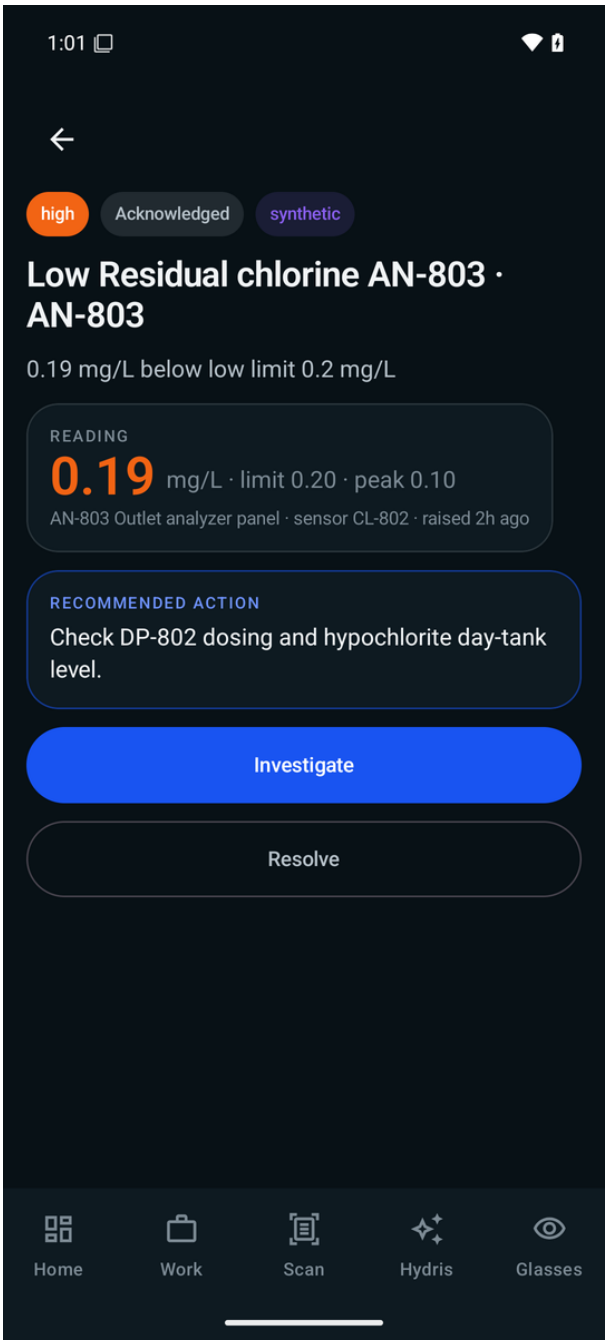

Figure 27. *Alarm detail with reading, limit, peak and the next lifecycle action*

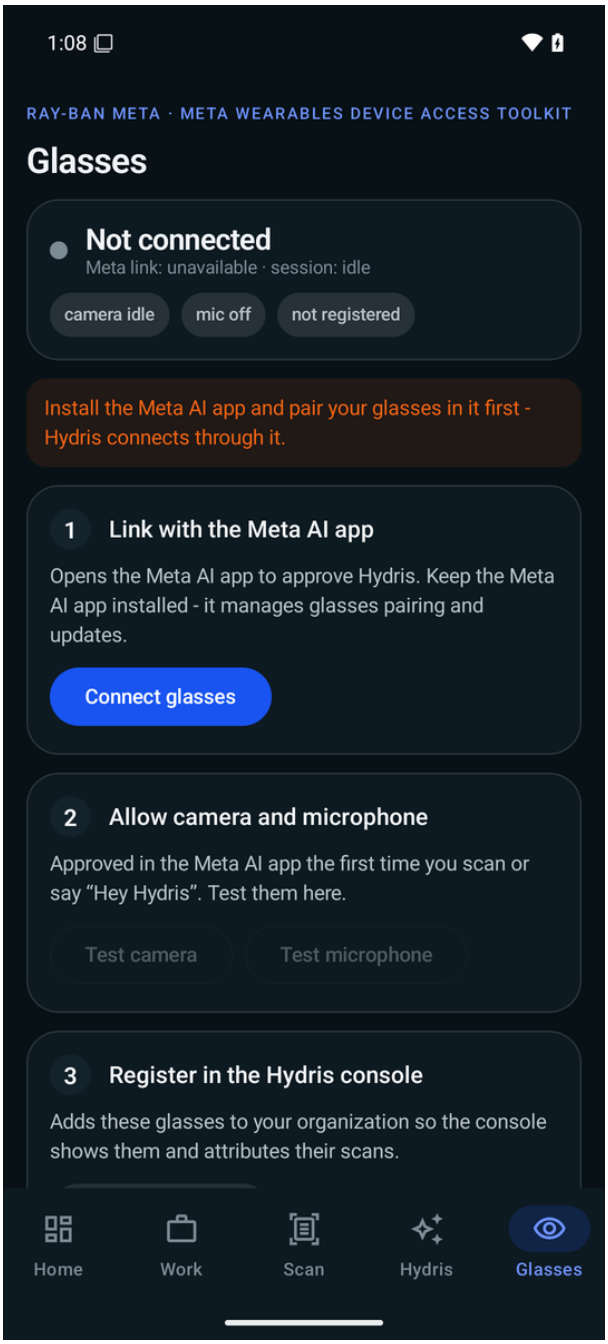

Figure 28. *Glasses: link with the Meta AI app, allow camera and microphone, register in the console*

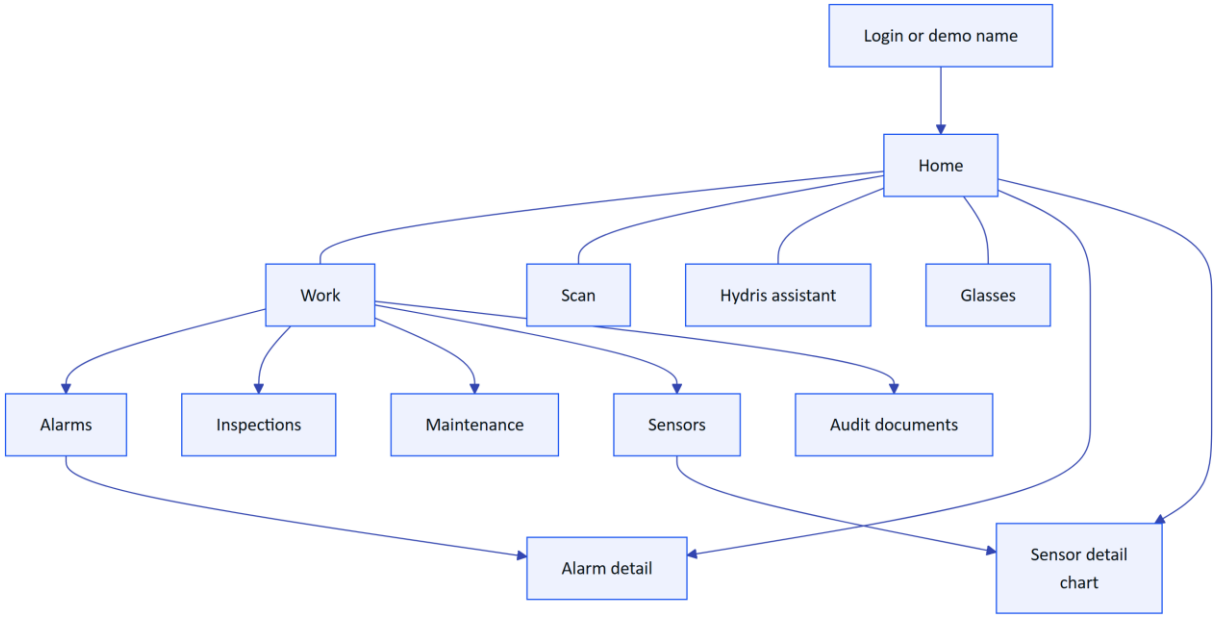

Figure 29. *Android app navigation map*

Table 16. **Android app requirements**

ID	Requirement	Pri	Status	Acceptance criteria	Surface
FR-090	Sign in with the console account; restore the session on restart from the refresh cookie, stored AES-256-GCM encrypted with a non-exportable Android Keystore key	M	Built	Killing and reopening the app restores the session without a password; no credential is stored in plain text (`SecureStore.kt`)	A
FR-091	Demo mode sign-in with a name	M	Built	Entering a name of 2 or more characters opens a personal demo account	A
FR-092	Home: greeting, 14-day alarm trend, asset health, live measurements with freshness and synthetic labels, recent activity	M	Built	Values marked synthetic in demo; stale values show their state	A
FR-093	Work tabs: alarms, inspections, maintenance, sensors, audit documents	M	Built	Each tab lists only what the user's site scope allows	A
FR-094	Alarm detail with lifecycle actions	M	Partial	Acknowledge, investigate and close work. Gap: Resolve sends no note, so the API returns `note_required`; the app also shows actions without checking `allowedActions`	A, P
FR-095	Sensor detail with a 144-point history chart and min, typical and max	S	Built	Chart covers the history window with typical range from `MEASURAND_DEFS`	A

ID	Requirement	Pri	Status	Acceptance criteria	Surface
FR-096	Audit scan from the glasses camera, the phone camera or an imported photo, with guided A4 capture, flattening, two ML Kit OCR passes and up to 6 pages	M	Built (phone). Glasses path Partial (FR-055)	A scan of the sample sheet reaches the console in review with values and engine agreement	A, P
FR-097	Realtime alarm notifications: active critical or high alarms raise a system notification while the app runs	M	Built	`alarm.created` on the SSE stream raises a high-priority notification; reconnects resume with `Last-Event-ID`	A, P
FR-098	Offline outbox for actions taken without connectivity (Room-backed, idempotent replay)	S	Planned (GA)	Alarm acknowledgements and checklist ticks made offline are sent once when the network returns	A
FR-099	Complete MFA sign-in in the app	S	Planned (Pilot)	An MFA-enabled operator signs in on the phone with a TOTP code	A, P

RISK

FR-094 is a correctness gap a pilot operator will hit on day one: tapping Resolve in the Android app fails because the API requires a resolution note (FR-031). Fix before pilot: add a note dialog for Resolve and drive the buttons from the API's `allowedActions`.

2.7.8 Audit document scanning

Audit scanning turns a paper record (inspection checklist, lab report, permit, calibration certificate, maintenance log) into a verified, searchable digital document. The design rule is dual verification: a machine cross-check of every value between independent readings, then a human review of every value and an independent approval by a second person.

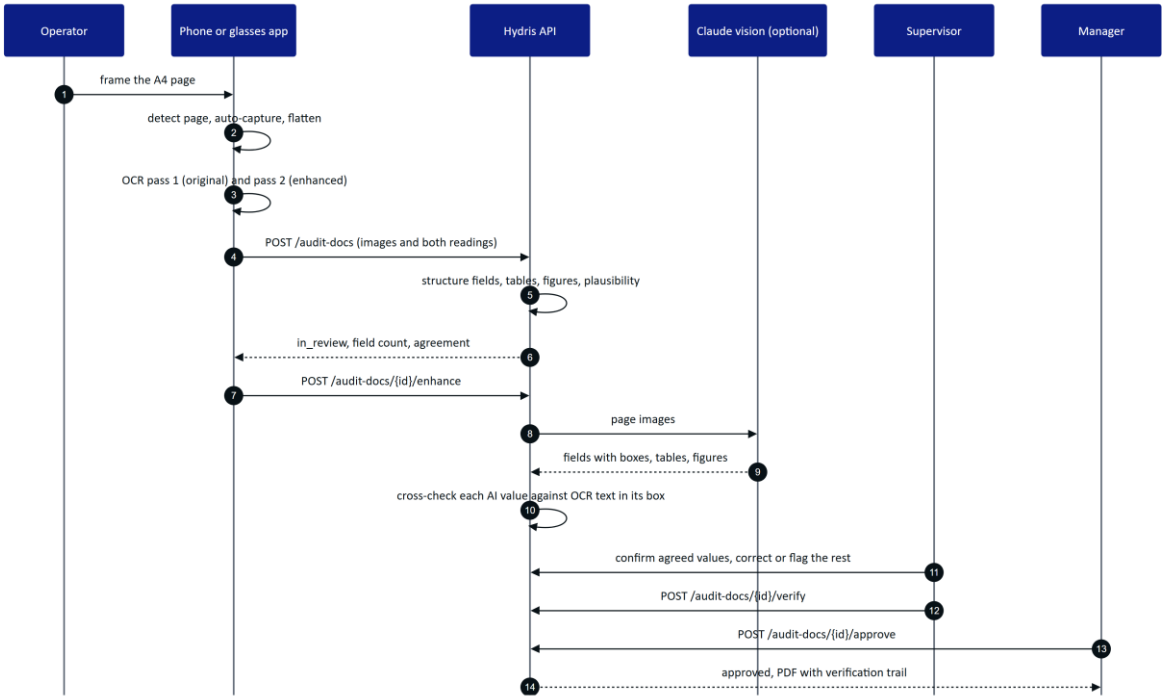

Figure 30. *Scan to verified PDF: capture, machine cross-check, human review, approval*

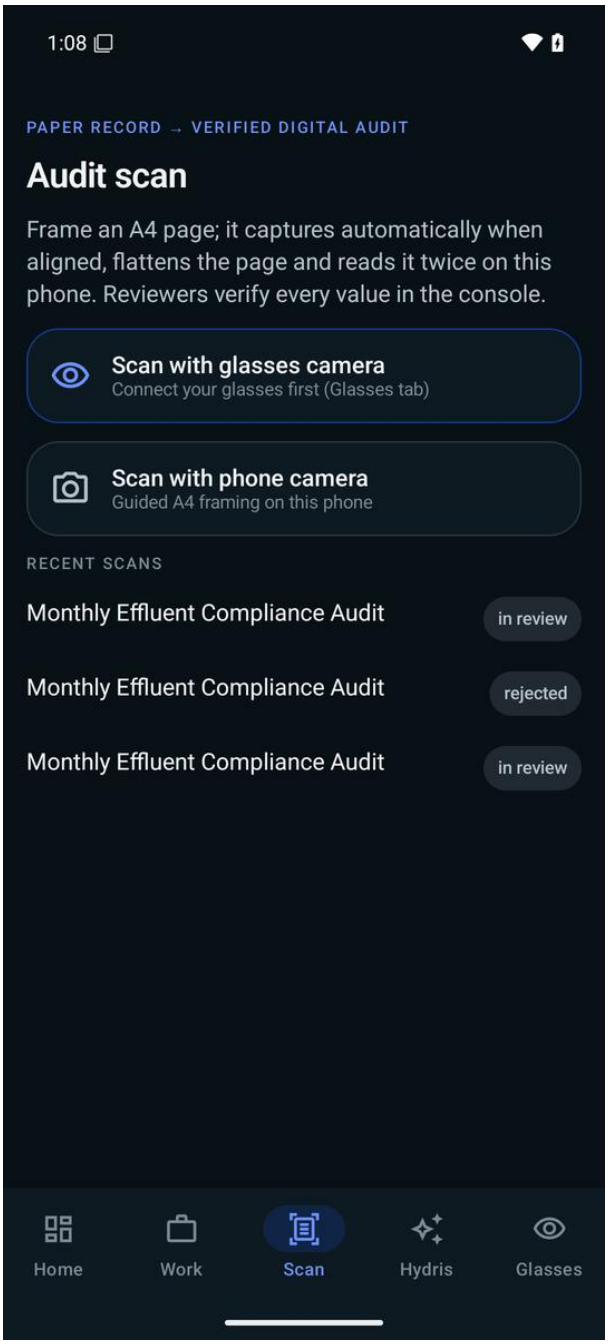

Figure 31. *Scan menu on Android: glasses camera, phone camera or import*

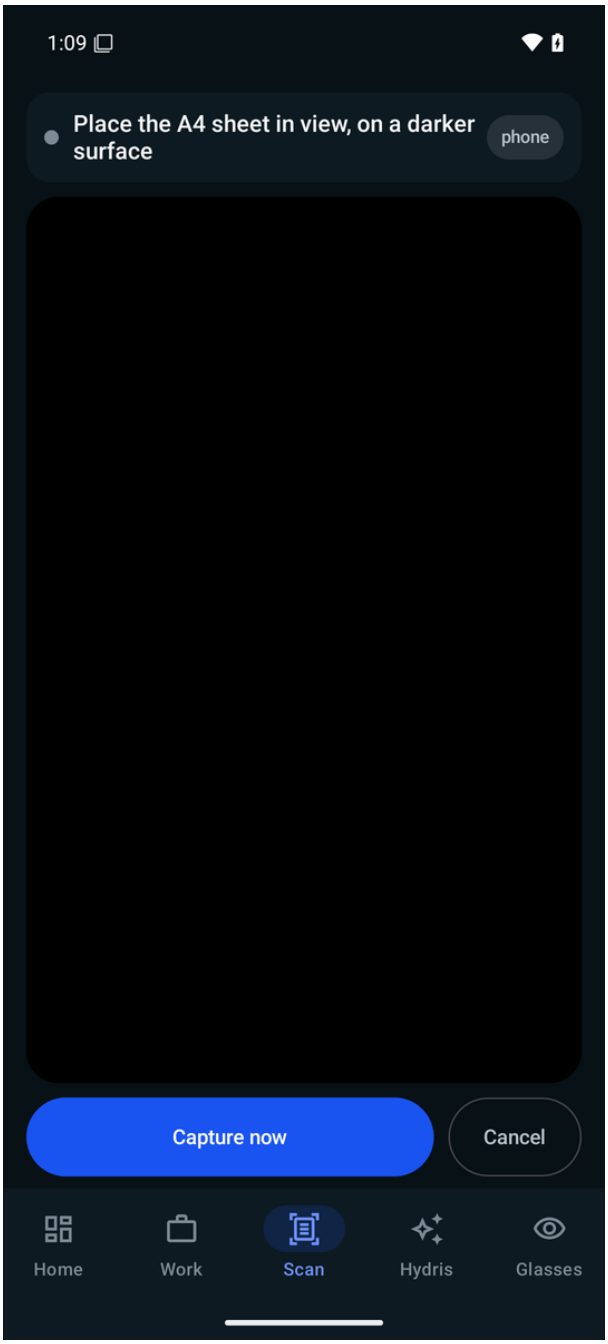

Figure 32. *Guided A4 capture: page detected, aligned and steady before auto-capture*

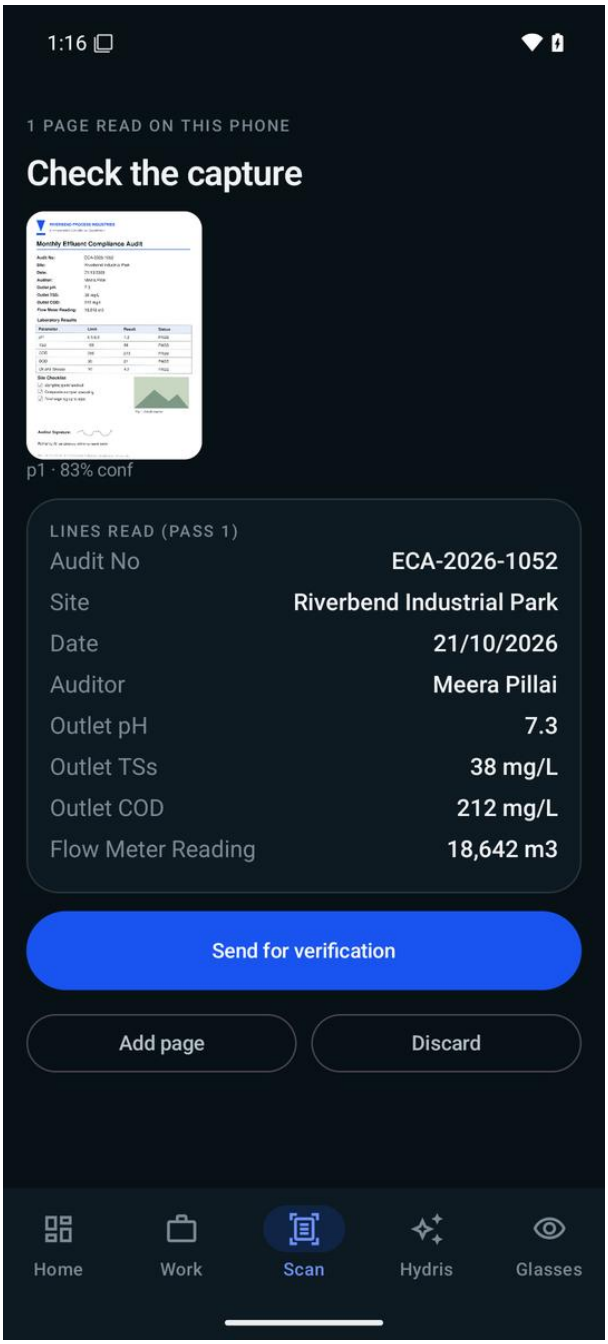

Figure 33. *Review the capture: confidence per page and the label and value pairs found*

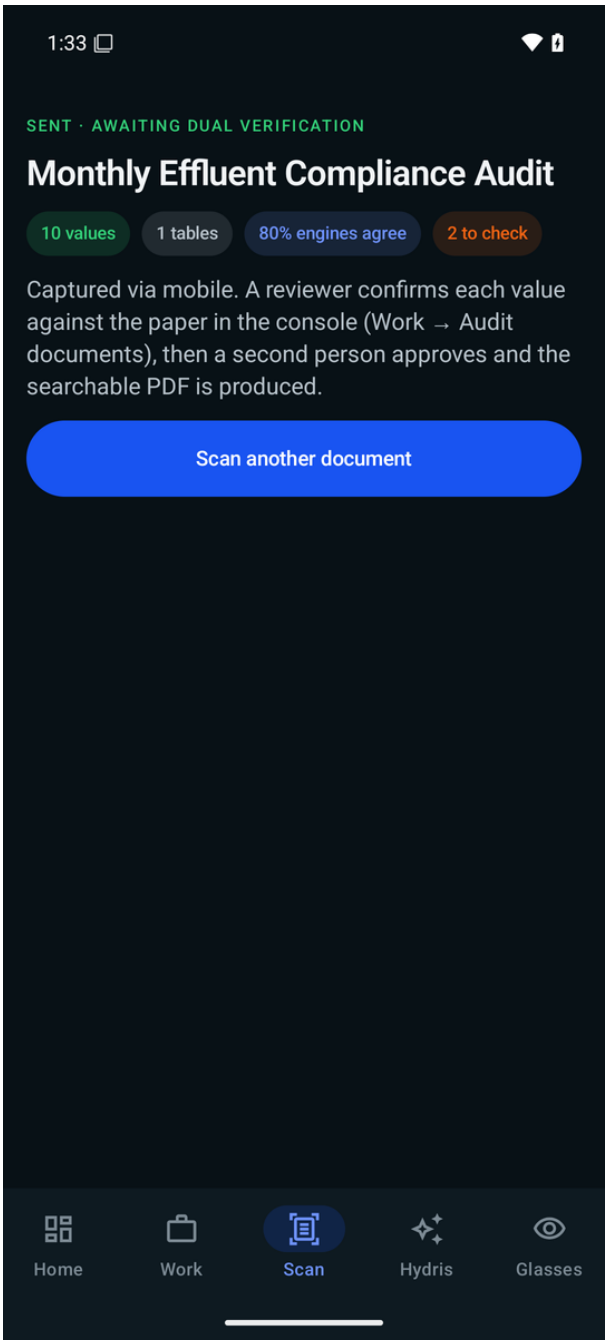

Figure 34. *Sent for verification: the next steps for reviewer and approver*

The document lifecycle is enforced by the API (`apps/api/src/modules/audit-docs.ts`). Every review action is appended to `audit_document_events`; corrections keep the first extracted value; concurrent edits are rejected with a revision check.

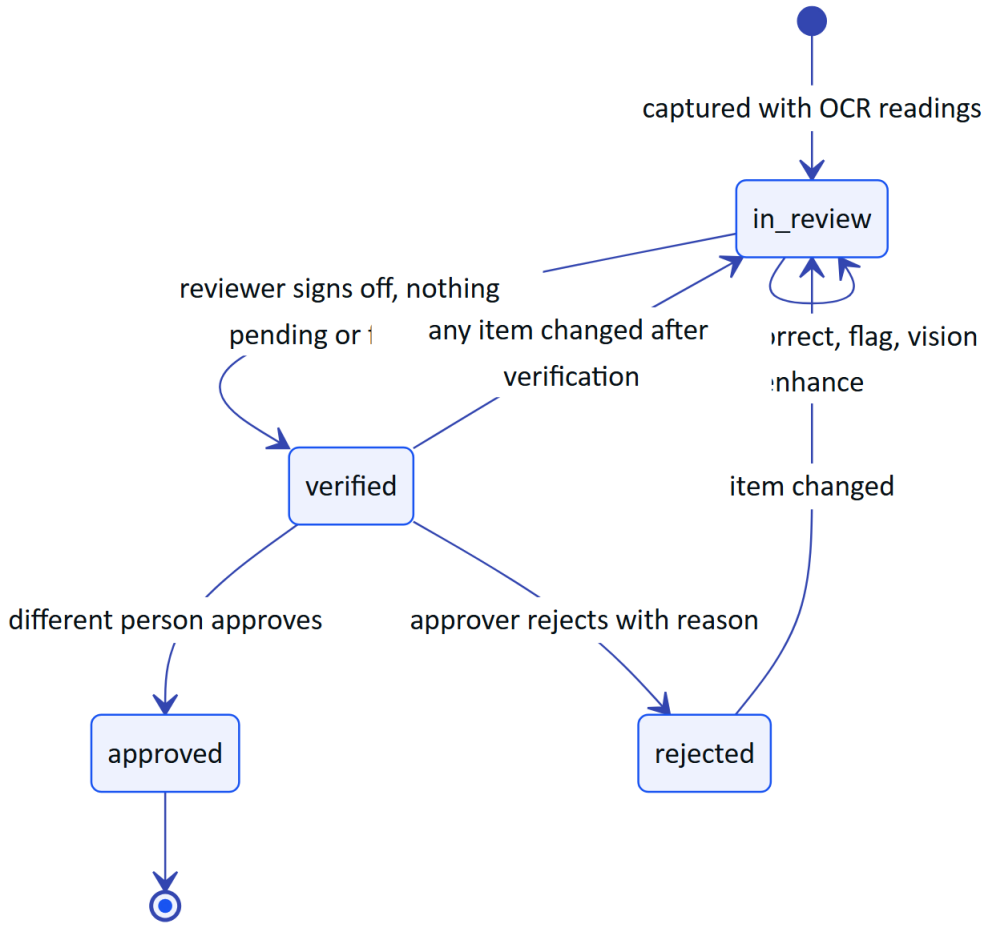

Figure 35. *Audit document lifecycle and the four-eyes rule*

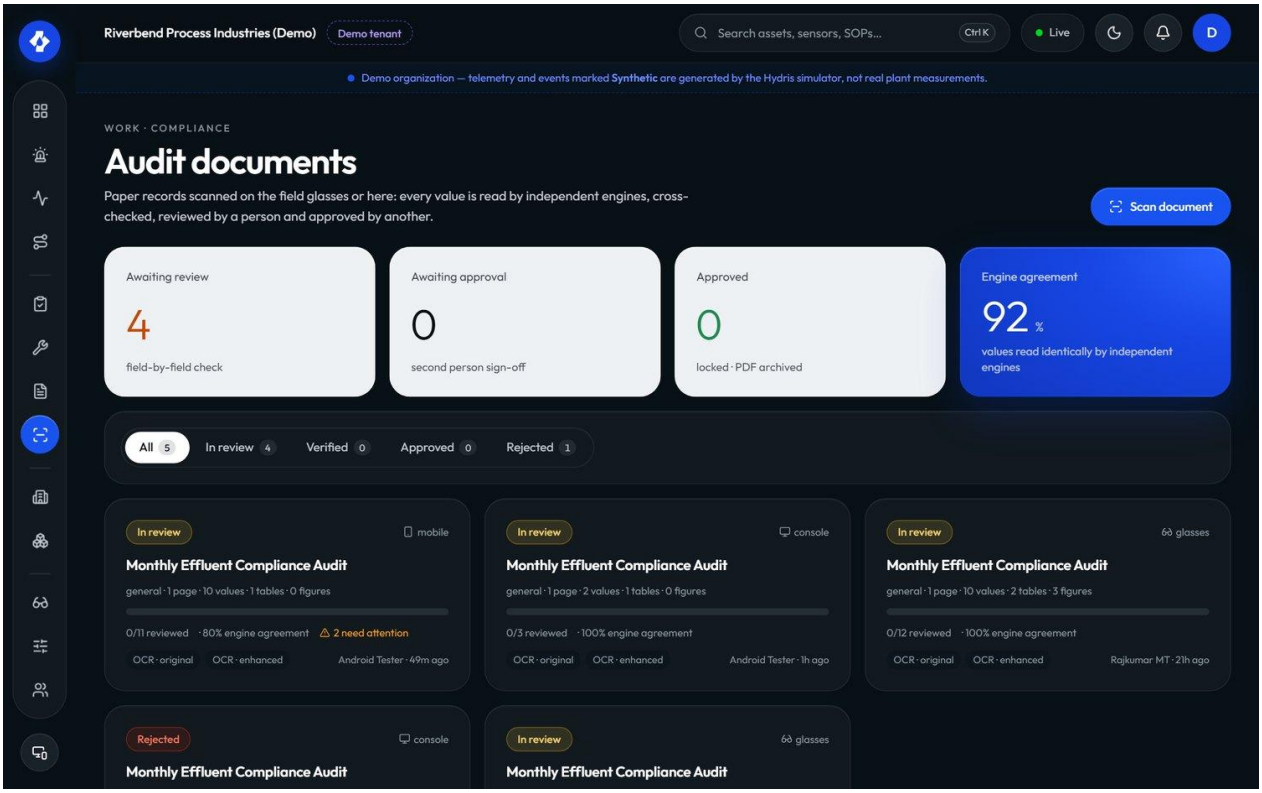

Figure 36. *Audit documents in the console: status, engines, agreement and review progress*

Table 17. Audit document scanning requirements

ID	Requirement	Pri	Status	Acceptance criteria	Surface
FR-060	Guided A4 capture with live page detection, auto-capture and perspective correction	M	Built (web `docdetect.ts`, Android `DocDetect.kt` with the same thresholds)	Capture fires only when the page is aligned, covers enough of the frame and is sharp	G, A, C
FR-061	Dual-pass on-device OCR (`ocr:original`, `ocr:enhanced`)	M	Built (Tesseract.js on web, ML Kit on Android)	Two readings per page arrive at the API; no page image leaves the device before upload	G, A, C
FR-062	Structuring into typed fields, tables and figures, with plausibility checks (pH 0 to 14, percentages, temperatures)	M	Built (`auditdoc.ts`)	An implausible pH is flagged with a message on the field	P
FR-063	Optional vision engine (Claude) cross-checked against the OCR text inside each value's box	S	Built (needs `ANTHROPIC_API_KEY`)	Each vision value lists the OCR sources that agree or disagree; without a key the OCR result is kept and `aiStatus` is `unavailable`	P
FR-064	Field review: confirm, correct, flag, reset; original value kept; verify requires zero pending and zero flagged items	M	Built	Verify with one flagged item returns `400 flagged_items`; a correction keeps `extracted`	C, P
FR-065	Independent approval (four eyes); self-approval only in demo tenants and recorded as such	M	Built	The verifier approving their own document in a customer org gets 403	C, P
FR-066	PDF with verification trail, searchable original and reconstructed page	M	Built (`core/audit-pdf.ts`)	Text in the PDF original is selectable; the cover lists reviewer, approver and times	C, P
FR-067	Up to 6 pages per document, each image at most 900 KB, JPEG or PNG validated by magic bytes	M	Built	A seventh page or a 1 MB page is refused with a clear message	P
FR-068	Electronic signature with re-authentication and signature meaning (Part 11-style)	S	Planned (Advanced)	Approving asks for the password or MFA again and records the meaning of the signature	C, P
FR-069	Bulk-confirm values where at least two independent engines match with confidence of 0.8 or more	S	Built (`confirm-agreed`)	Only multi-source, matching, confident items change to confirmed; the event lists their IDs	C, P

DECISION

Verification and approval are signed only by a person in the console (`p.kind !== 'user'` is refused). A device can capture and pre-review, but it cannot verify or approve. This keeps the evidentiary steps tied to an authenticated human session, not to a shared pair of glasses.

2.7.9 Voice assistant and grounded assistance

"Hey Hydris" is a hands-free assistant. The wake phrase is detected on the client with a pattern tolerant of common recognition spellings ("hydras", "hi dris"); the question goes to `POST /assistant/ask`. The API answers with one of two engines:

- **Claude** (`claude-opus-5-5` with tool use) when an Anthropic credential is configured: natural phrasing and multi-step lookups through five read-only tools (`get_readings`, `get_open_alarms`, `get_my_work`, `get_asset`, `search_procedures`), at most five tool rounds.
- **A deterministic rules engine** otherwise, and as the fallback when the model fails: readings, alarms, my work, asset status, procedures and help. It never guesses; anything else gets "I can't answer that from your plant data".

Every tool is scoped to the caller (RLS plus site scope), so the assistant can only say what the caller could see in the console. Answers return `facts` (label, value, unit, time, source) so the client can show the evidence under the spoken answer.

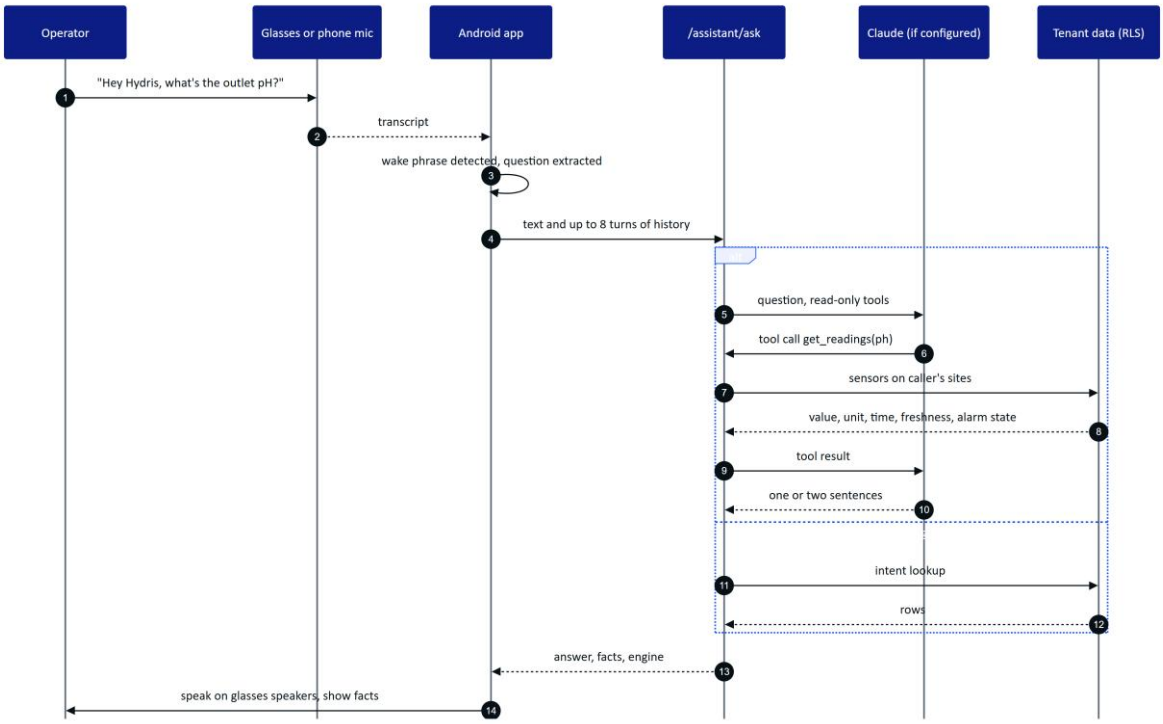

Figure 37. *A spoken question answered from the caller's own plant data*

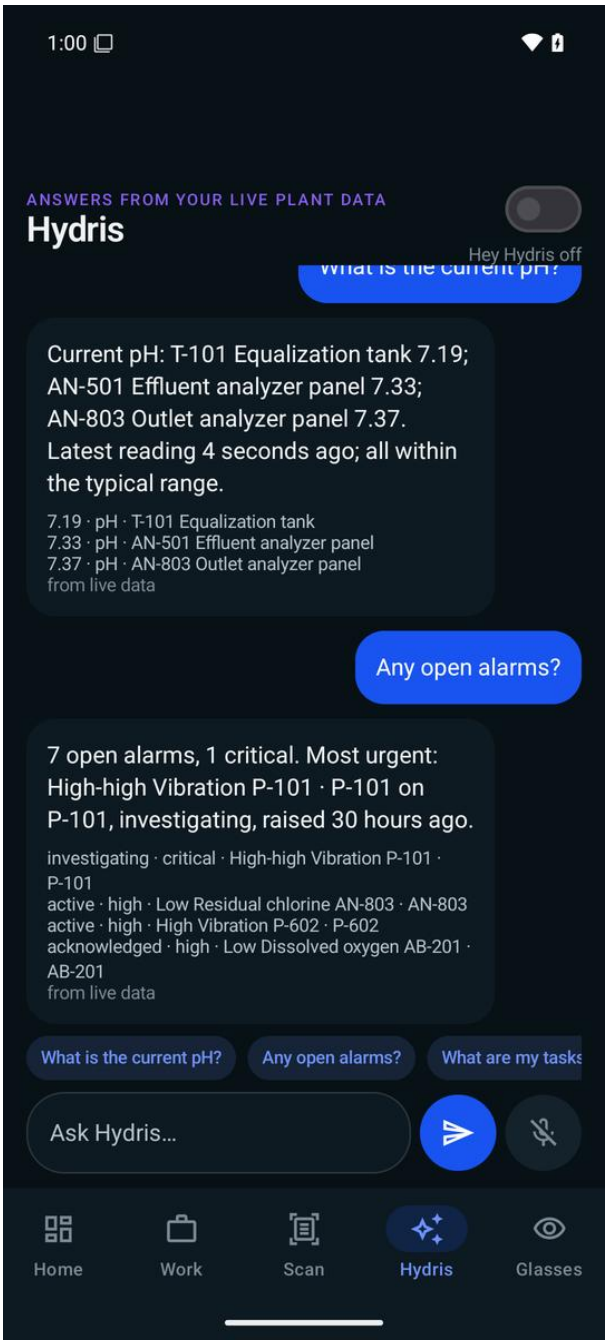

Figure 38. *The Hydris assistant on Android: answers with the facts behind them*

Table 18. Voice and assistance requirements

ID	Requirement	Pri	Status	Acceptance criteria	Surface
FR-070	Grounded SOP retrieval (`/assist`) with passages from approved documents and a safety notice	M	Built	Responses contain only approved documents and always include the safety notice	G, C, P
FR-071	"Hey Hydris" wake phrase and short spoken answers	M	Built (web Web Speech API; Android SpeechRecognizer and TextToSpeech)	Saying the wake phrase and a question produces a spoken answer; "stop" or "thanks" ends the follow-up window	G, A

ID	Requirement	Pri	Status	Acceptance criteria	Surface
FR-072	`/assistant/ask` answers only from tenant data through scoped tools; rules fallback never guesses	M	Built	An unknown question returns intent `unknown`; every number in an answer appears in `facts` for rules answers	G, A, P
FR-073	Multi-language voice (Hindi, Tamil, Marathi, Gujarati)	S	Planned (Advanced)	A Hindi question returns a Hindi answer with the same grounding guarantees	G, A, P
FR-074	Glasses microphone through DAT Speech with phone-microphone fallback; answers played through TextToSpeech on the glasses' open-ear speakers	M	Partial	Phone path built in `VoiceEngine.kt`. Gap: glasses microphone path not validated on hardware	A
FR-075	Answers show the facts behind them with source, and label synthetic data	M	Built	A demo answer's facts show "synthetic demo data"	G, A, P
FR-076	Follow-up questions with conversation history (up to 8 turns accepted by the API)	S	Built	"And the inlet?" after a pH question is answered for the inlet	G, A, P
FR-077	"Hey Meta, start Hydris" voice invocation opens the assistant	C	Partial (WebView bridge only; DAT feature is experimental)	Voice invocation intent opens the assistant screen	A
FR-078	The assistant has no write tools and cannot change any record or equipment state	M	Built	The tool list contains only read tools; the system prompt forbids operating equipment	P

2.7.10 Documents and SOPs

Table 19. Document and SOP requirements

ID	Requirement	Pri	Status	Acceptance criteria	Surface
FR-110	Document library (SOP, manual, datasheet, safety notice, troubleshooting) with draft, approved and archived states, versions, and site, asset or asset-type scope	M	Built	Only `document.manage` holders approve; versions increment on change	C, P
FR-111	Only approved documents reach the glasses, the assistant and `/assist`	M	Built	A draft SOP never appears in a glasses search or assistant answer	G, A, P
FR-112	Alarm rules link a procedure document shown with the alarm	M	Built	Alarm detail shows the linked SOP title and opens it	C, G, P
FR-113	SOPs read on the glasses are cached for offline reading	S	Built (`glasses/data.ts` cache)	A previously opened SOP opens with no network	G
FR-114	Upload PDF manuals and SOPs with text extraction for retrieval	C	Planned (GA)	An uploaded PDF becomes searchable by `/assist` after approval	C, P

2.7.11 Insights, reports and platform

Table 20. Platform requirements

ID	Requirement	Pri	Status	Acceptance criteria	Surface
FR-080	Realtime server-sent events with `Last-Event-ID` replay; memory or database mode	M	Built	Reconnecting with the last ID replays missed events; DB mode works across serverless instances	C, G, A, P
FR-081	Reports (inspections, alarms, maintenance, devices, operators) in JSON and CSV	M	Built	Alarm report includes median minutes to acknowledge; inspection report includes on-time rate and glasses share	C, P
FR-082	Append-only audit trail	M	Built	No API path updates or deletes audit events	C, P
FR-083	Demo mode: zero-configuration synthetic tenant, named guests, cross-instance pairing	M	Built	A deployment with no environment variables serves a working demo	All
FR-084	Deterministic simulation scenarios, demo tenants only	M	Built	Ten scenarios run; starting one in a customer org is refused	C, P
FR-085	In-app notifications	M	Built	Notifications appear in realtime and can be marked read	C, G, A, P
FR-086	Email delivery (SMTP, SES or Resend) for `mail_outbox`	M	Planned (Hardening)	Invitation and reset emails arrive; failures retry and are visible to admins	P
FR-087	Audit log page with filters	M	Built	Auditor filters by action and entity and sees actor, time and IP	C, P
FR-088	Global search across assets, sensors, documents and work	S	Built (`/search`)	Typing an asset tag finds the asset and its open work	C, P
FR-089	Overview dashboard with KPIs, open alarms, health and live measurements	M	Built (`/overview`)	Console overview and Android home load from the same endpoint	C, A, P

Table 21. Insights and compliance requirements

ID	Requirement	Pri	Status	Acceptance criteria	Surface
FR-120	Compliance summary per discharge point: daily minimum, maximum and average against consent limits, with data availability	S	Planned (Pilot)	A manager exports a monthly summary for the final outlet with gaps and exceedances marked	C, P
FR-121	Scheduled report delivery by email	C	Planned (GA)	A weekly alarm report arrives every Monday (depends on FR-086)	P
FR-122	Shift handover summary generated from the shift's alarms, work and readings, grounded and editable before sending	C	Planned (GA)	The outgoing operator reviews a summary whose every number links to a record	A, C, P

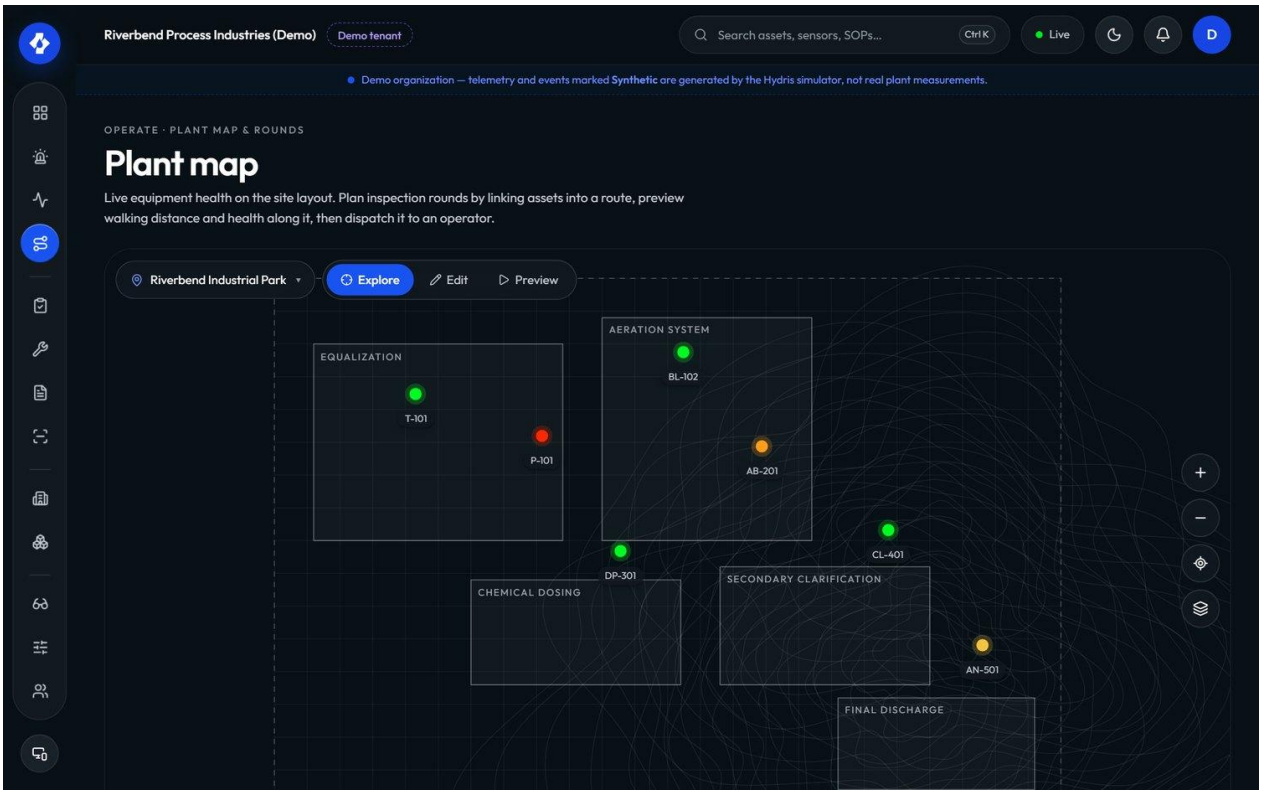

Figure 39. *Plant map and rounds in the console*

2.7.12 Website, demo and public

Table 22. Website, demo and public requirements

ID	Requirement	Pri	Status	Acceptance criteria	Surface
FR-100	Marketing website with seven pages and a privacy page	M	Built	Pages render on phone and desktop, light and dark	W
FR-101	Contact and demo-request form `POST /public/inquiries`, rate limited to 5 per 10 minutes per client	M	Built	The sixth submission within 10 minutes is refused	W, P
FR-102	Live glasses demo from the website (`/glasses?demo=1&scenario=...`), fully client-side, and the browser simulator	M	Built	The demo runs with no API calls and shows scenario events	W, G
FR-103	Named demo guests: each guest is an `org_admin` in the demo tenant with personal copies of scheduled inspections and open tasks	M	Built	Two guests acting at once do not see each other's personal work changes	C, A, P

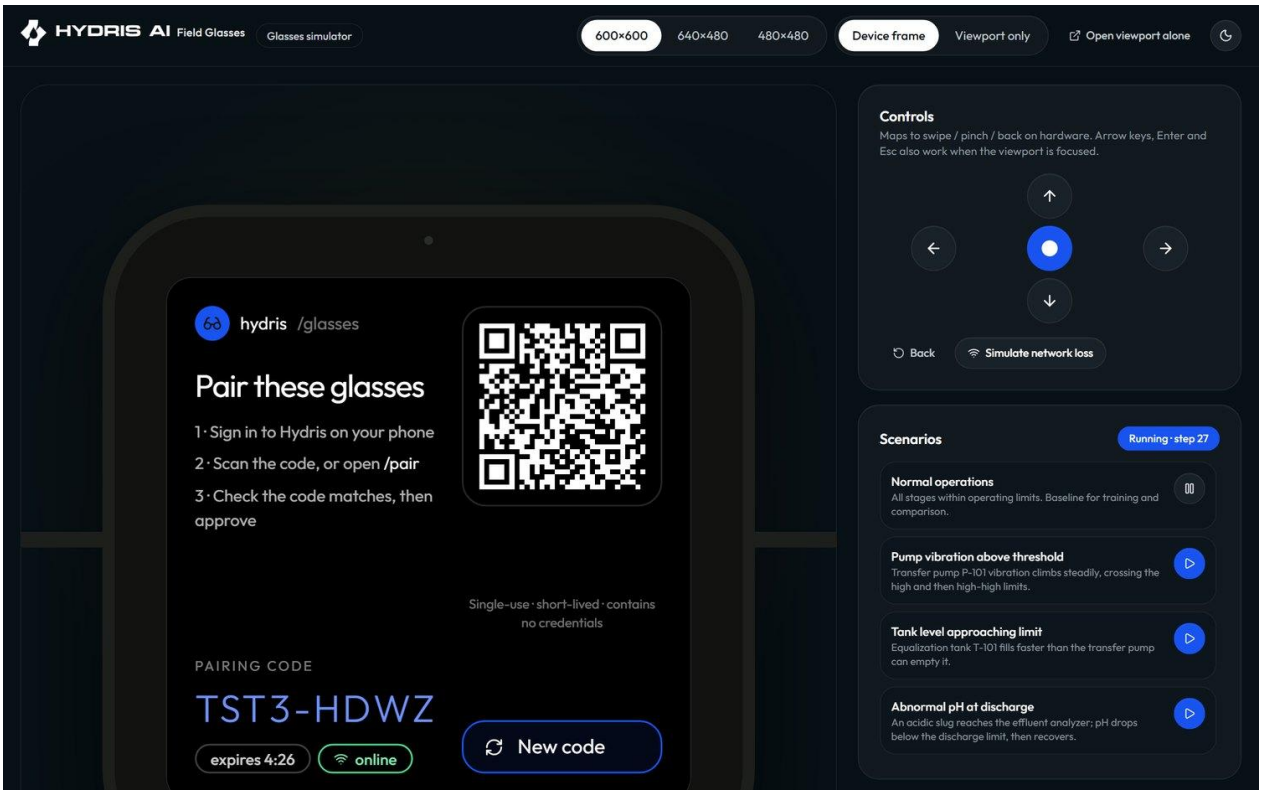

Figure 40. *The browser glasses simulator used for demos and testing*

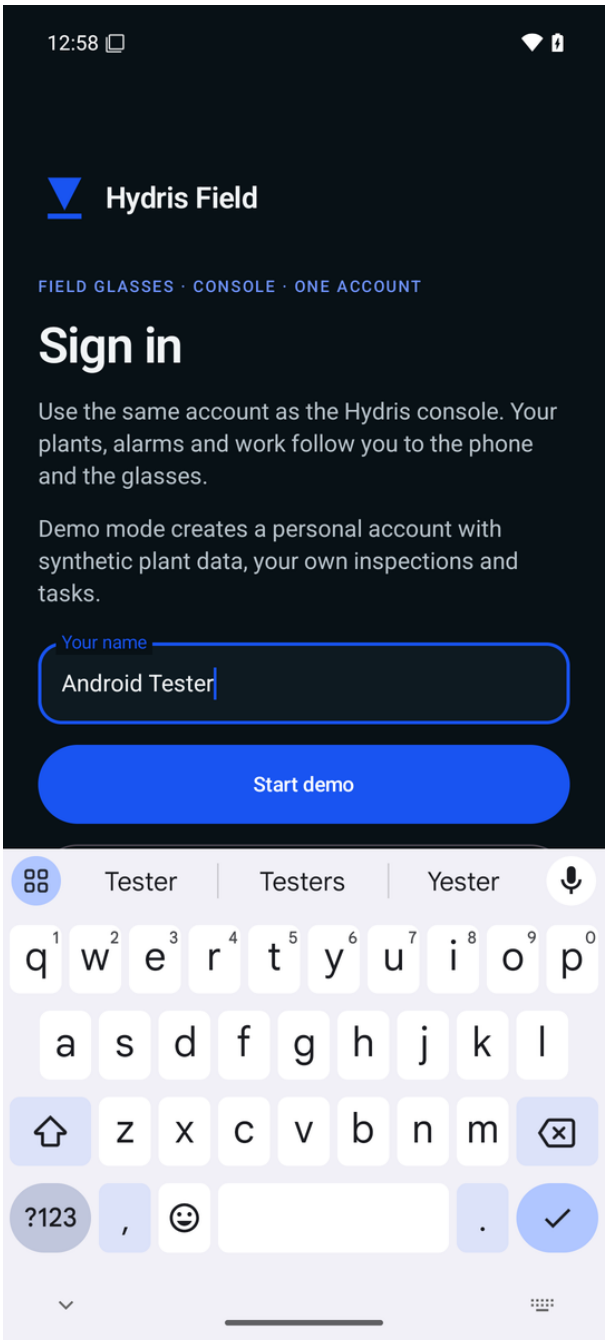

Figure 41. *Demo mode on Android: enter your name to get a personal demo account*

2.7.13 Requirement status overview

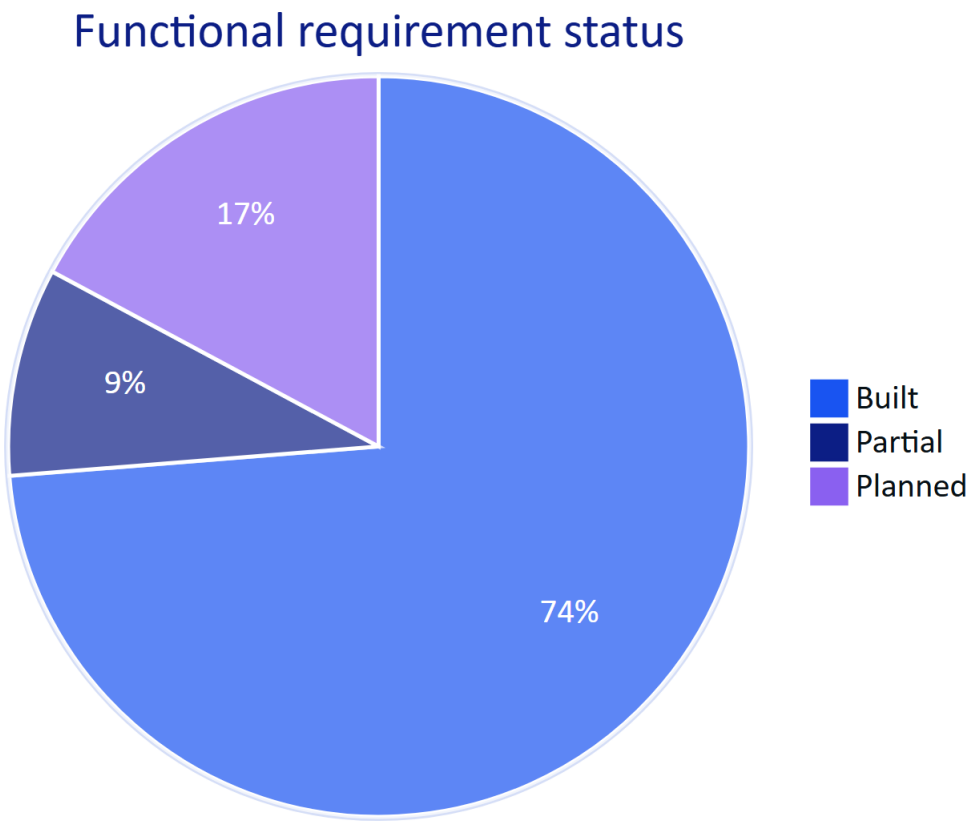

Figure 42. *Status of all functional requirements in this edition*

2.8 Non-functional requirements

Non-functional requirements apply across surfaces. IDs are new in this edition (version 1.0 listed the same areas without IDs). Values are targets unless a measurement is named.

2.8.1 Performance and responsiveness

Table 23. Performance requirements

ID	Requirement	Pri	Status	Target and verification
NFR-001	API read latency	M	Partial (not yet measured)	Target p95 under 300 ms for reads in-region; verify with load tests before pilot
NFR-002	Realtime alarm delivery from creation to client	M	Built (design); not measured	Target 2 s or less. Memory mode is immediate; DB mode polls every 1.5 s (`realtime.ts`)
NFR-003	Voice answer time from end of speech to start of spoken answer	M	Partial (not measured)	Targets: 3 s with the rules engine, 8 s with the model engine
NFR-004	OCR time per page on a mid-range phone	S	Partial (not measured)	Target 15 s or less per page for both passes
NFR-005	Glasses home screen ready after wake	S	Planned (measure)	Target 2 s from cached context; fresh data within 5 s

2.8.2 Availability, scalability and operations

Table 24. Availability and operations requirements

ID	Requirement	Pri	Status	Target and verification
NFR-006	Service availability	M	Planned (measure)	Targets: 99.5 % monthly in pilot, 99.9 % at GA, measured by external uptime checks
NFR-007	Horizontal scale of realtime	M	Built	`REALTIME_MODE=db` supports serverless and multiple instances; memory mode is single-instance only
NFR-008	Scheduled jobs (stale rules, pairing expiry, event pruning) run at a cadence suitable for real data	M	Partial	Long-running server: 30 s, 60 s and hourly timers. Vercel: daily cron only; fix in Hardening
NFR-009	Backups and restore	M	Planned	Daily backups, point-in-time recovery on managed Postgres, restore tested each quarter
NFR-010	Continuous integration on every change	M	Planned (Hardening)	Typecheck, unit, API and e2e tests on every pull request; no `.github` workflows exist today

2.8.3 Security and privacy

Table 25. Security and privacy requirements

ID	Requirement	Pri	Status	Target and verification
NFR-011	Tenant isolation enforced in the database	M	Built	RLS on every tenant table; requests run as `hydris_app` without BYPASSRLS; API tests cover cross-tenant access
NFR-012	Short-lived credentials	M	Built	15-minute access tokens (12 h in demo mode only); rotating refresh tokens with reuse detection
NFR-013	Rate limiting	M	Built	Global 600 requests per minute per client; sign-in 10 per minute; pairing endpoints 10 to 40 per minute; public form 5 per 10 minutes
NFR-014	Secrets at rest	M	Built	MFA secrets encrypted with `DATA_KEY`; tokens stored as SHA-256 hashes; Android credentials in Keystore-backed AES-256-GCM
NFR-015	Camera only on explicit intent; no continuous recording	M	Built	Camera streams only while a scan screen is open (`startPreview` and `stopPreview`); no background capture
NFR-016	Images stored only for audit documents and attachments	M	Built	No other path persists camera frames
NFR-017	Data residency in India for Indian customers	S	Planned (GA)	Production database, storage and model inference in an Indian region, documented per tenant
NFR-018	Independent penetration test before GA	M	Planned (GA)	No open high or critical findings at launch

> **WARNING**
> The glasses web app keeps its device refresh token in browser `localStorage` (`apps/web/src/glasses/link.ts`). That is acceptable for the simulator and demo, not for production glasses. The native Android app already stores device tokens in the Keystore-backed `SecureStore`, which is one reason the native app, not the WebView bridge, is the pilot path.

2.8.4 Accessibility, glanceability and hands-free use

Table 26. Accessibility and hands-free requirements

ID	Requirement	Pri	Status	Target and verification
NFR-019	Glasses text scale per profile	M	Built	Scales 0.9, 1, 1.15 and 1.3 (`UI_TEXT_SCALES`), clamped to 0.85 to 1.4
NFR-020	Additive-display friendly design	M	Built (assumption to verify on hardware)	Pure black renders as transparent; 600 by 600 viewport assumption (`GLASSES_DISPLAY`)
NFR-021	Every glasses action reachable with six directional keys or voice	M	Built	Keyboard, swipe and simulator controls map to up, down, left, right, select, back
NFR-022	Spoken answers are short	M	Built	One or two sentences (assistant system prompt); units spoken in words
NFR-023	Console and Android meet WCAG 2.2 AA	S	Partial (not audited)	Automated and manual accessibility audit before GA; contrast in light and dark themes
NFR-024	Large touch targets for gloved use on Android	S	Built	Primary buttons are 52 to 54 dp tall in the scan, login and alarm screens

2.8.5 Offline, battery and device constraints

Table 27. Offline and device requirements

ID	Requirement	Pri	Status	Target and verification
NFR-025	Inspection responses survive connectivity loss on the glasses	M	Built	Queue plus idempotent replay; Playwright journey and `glasses_network_loss` scenario
NFR-026	Last context and SOPs readable offline on the glasses	S	Built	Cached with timestamp; shown as cached, never as live
NFR-027	Android offline outbox	S	Planned (GA)	See FR-098
NFR-028	Battery: a 4-hour shift on Ray-Ban Meta with up to 20 scans	S	Planned (measure)	Camera streams only on the scan screen; Android preview frames throttled to at most one per 100 ms; speech restarted only while enabled
NFR-029	Android 12 or later (min SDK 31)	M	Built	Matches DAT requirements

2.8.6 Data integrity and maintainability

Table 28. Data integrity and maintainability requirements

ID	Requirement	Pri	Status	Target and verification
NFR-030	Provenance on every reading; synthetic data impossible in customer tenants	M	Built	DB check `telemetry_synthetic_source_ck`; ingest refuses simulator data for non-demo orgs
NFR-031	Optimistic concurrency on review records	M	Built	Audit document edits carry a revision; stale writes get `409 stale`
NFR-032	Shared domain logic between server and clients	M	Built	State machines, permissions, freshness and voice parsing live in `packages/domain`; Android ports keep identical thresholds
NFR-033	Automated test coverage of security-critical paths	M	Built	67 tests passing (26 domain, 41 API) on 22 Oct 2026; 4 Playwright journeys
NFR-034	Observability	S	Planned (Pilot)	Structured logs, error tracking and uptime alerts with an on-call runbook

2.9 Data and AI requirements

2.9.1 Data principles

1. **Tenant data stays in the tenant.** All AI calls are made with data the caller is already allowed to see, fetched through the same RLS-scoped queries as the console.
2. **The model never sees more than it needs.** The assistant receives tool results for the question asked, not a dump of the database. Vision extraction receives only the page images of the document being processed.
3. **Nothing the model says becomes a record without a person.** Vision-extracted values enter the review as `pending`; a reviewer confirms each one and a second person approves.
4. **Every AI output is attributable.** Audit documents record which engines produced each value (`engines`, `sources`); assistant responses say which engine answered (`engine: claude-opus-5-5` or `hydris-rules`).

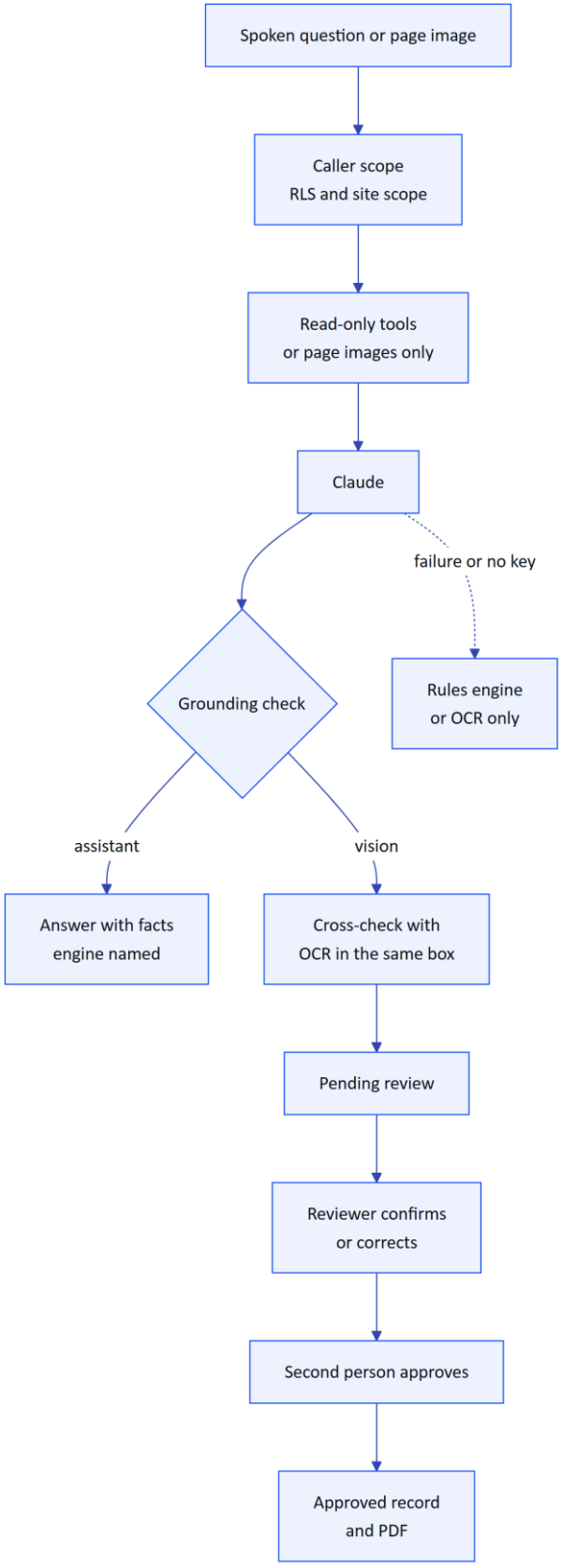

Figure 43. *Guardrails between the language model and the plant record*

2.9.2 Grounding requirements

Table 29. Data and AI requirements

ID	Requirement	Pri	Status	Acceptance criteria
NFR-035	The assistant uses only values returned by tools in the same conversation	M	Built (prompt and tool design)	Evaluation set: zero answers containing a number absent from tool results (target)
NFR-036	The assistant states the age of a reading and says plainly when it is stale, missing, out of range or in alarm	M	Built	Rules answers include "measured N minutes ago"; stale readings add "not current, check the sensor"
NFR-037	Graceful degradation without the model	M	Built	Any model error falls back to the rules engine; vision failure keeps OCR results with `aiStatus = failed` and the reason
NFR-038	Vision transcribes faithfully: no correcting, normalizing or inferring illegible values	M	Built (system prompt)	Illegible areas appear in warnings with low confidence
NFR-039	Human-in-the-loop for every AI-extracted value	M	Built	No document reaches `verified` with pending or flagged items
NFR-040	Model choice and data handling documented per tenant, including region	S	Planned (GA)	Tenant settings show which model provider and region process its data

2.9.3 Accuracy targets

Table 30. AI and OCR quality targets (targets, not measurements)

Measure	Target for pilot	How it will be measured
Fields where independent engines agree (`agreement = match`)	90 % or more on typed forms	Agreement score per document in the console; pilot sample of 200 pages
Fields a reviewer has to correct	under 5 %	Ratio of `corrected` to all reviewed fields
Vision false confidence (value confirmed by engine agreement but wrong on paper)	under 0.5 %	Blind re-check of 100 approved documents
Assistant answers with facts (not "can't answer")	85 % or more of real operator questions	Intent logging on `/assistant/ask` (engine, intent)
Assistant answers with a number not in its facts	0	Automated check on logged answers and facts

> **IMPORTANT**
>
> The assistant and vision engine run on `claude-opus-5-5` when `ANTHROPIC_API_KEY` is set (`voice-assistant.ts`, `core/vision-extract.ts`). Before a customer pilot the tenant must agree to that processing, and the deployment must record where inference runs. Until then, pilots can run with the rules engine and OCR only, which are fully on Hydris infrastructure and the device.

2.10 Success metrics and KPIs

2.10.1 North star

Verified field records per active operator per week: the number of inspections submitted, alarms acknowledged within target, and audit documents approved, divided by the number of operators who used Hydris that week. It rises only when operators use the product in the field and the records it produces are trusted.

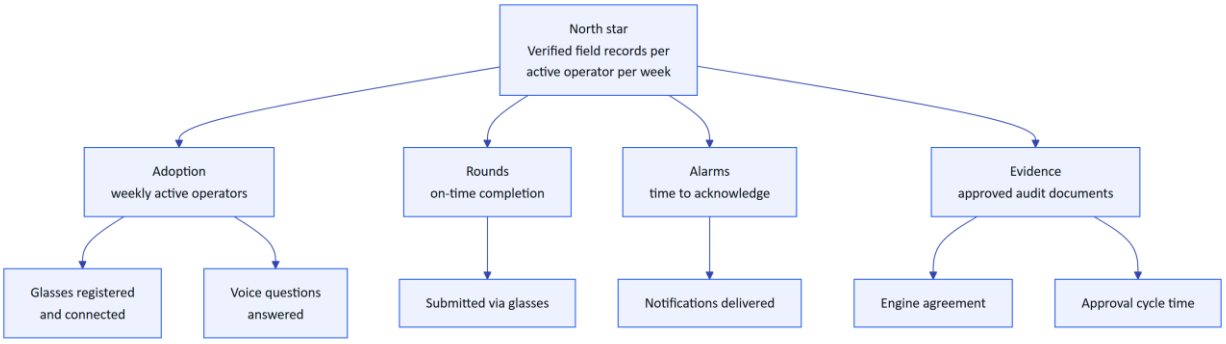

Figure 44. *KPI tree: the north star and the input metrics that drive it*

2.10.2 Metrics and targets

Table 31. KPIs with definitions, sources and pilot targets

Area	Metric	Definition and source	Pilot target
Adoption	Weekly active operators	Operators with at least one record in a week divided by licensed operators	70 % or more
Rounds	On-time completion	`onTimeRate` in the inspections report	95 % or more
Rounds	Time per inspection	Median start to submit, compared with the paper baseline	30 % lower than paper
Rounds	Share submitted via glasses	`viaGlasses` in the inspections report	50 % or more
Alarms	Time to acknowledge	`medianMinutesToAcknowledge` in the alarms report	under 5 minutes for critical and high
Alarms	Acknowledged but unresolved backlog	Count of alarms in acknowledged or investigating older than 24 h	Visible daily; trending down
Evidence	Engine agreement	Average agreement score of approved documents	90 % or more
Evidence	Reviewer edits	Corrected fields divided by reviewed fields	under 5 %
Evidence	Approval cycle	Capture to approval, median	under 24 hours
Voice	Answered with facts	Share of questions with intent other than `unknown`	85 % or more
Trust	Hallucinated numbers	Numbers in answers missing from facts	0
Security	Cross-tenant incidents	Confirmed data exposure across tenants	0
Security	Device revocation time	Revoke to `session.ended` on the stream	30 s or less

> **NOTE**
> Most adoption and round metrics already have a data source in `/reports/:kind` (`apps/api/src/modules/insights.ts`). The voice and agreement metrics need lightweight logging that is Planned for the pilot (part of NFR-034).

2.11 Release plan and milestones

2.11.1 What has shipped

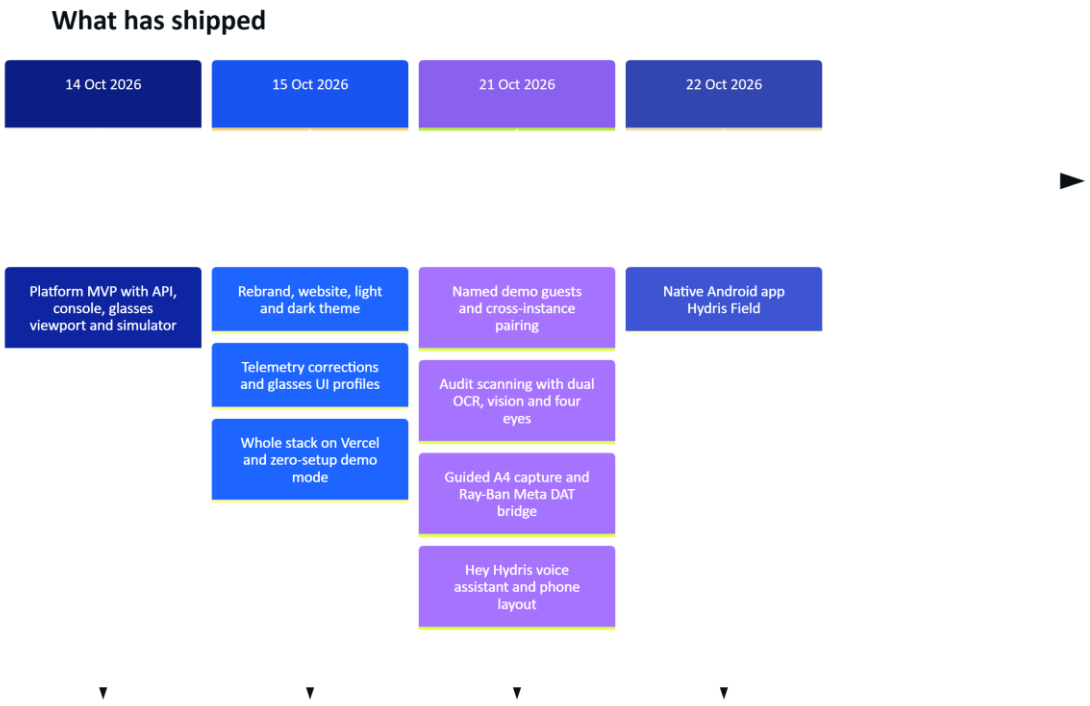

Figure 45. *Build history on main (git log, October 2026)*

2.11.2 Phases

Table 32. Release phases, scope and exit criteria

Phase	When	Scope	Exit criteria
0. MVP	Done (to October 2026)	API, console, glasses app, simulator, pairing, UI profiles, audit scanning, voice, demo mode, native Android app in code	Live demo; 67 tests passing
1. Hardening	Q4 2026	Email transport (FR-086), CI (NFR-010), job cadence on serverless (FR-035, NFR-008), Android fixes (FR-094, FR-099), audit page images to object storage, observability baseline	All Must requirements for pilot are Built or have an agreed workaround
2. Hardware validation	Q4 2026 to Q1 2027	Ray-Ban Meta on the test matrix: camera, capture button, speech, registration (FR-055, FR-056, FR-057, FR-074); battery measurement (NFR-028)	Every glasses requirement moves from Partial to Built or is descoped with a phone fallback

Phase	When	Scope	Exit criteria
3. Pilot	Q1 2027	One or two Indian plants (ETP or STP), 10 to 30 operators, gateway ingestion, compliance summary (FR-120), escalation (FR-037), Android inspections (FR-044)	KPI baselines captured; pilot targets met or explained; customer sign-off
4. GA	Q2 2027	FCM push (FR-034), Android offline outbox (FR-098), SSO (FR-011), India-region production (NFR-017), pen test (NFR-018), recurring schedules (FR-047), Play or MDM distribution	Store or MDM release approved; security review passed
5. Advanced	H2 2027 onwards	MQTT and OPC UA (FR-027), SCADA connectors (FR-028), multi-language voice (FR-073), e-signatures (FR-068), native glasses display, predictive maintenance, LIMS, iOS	Driven by customer demand

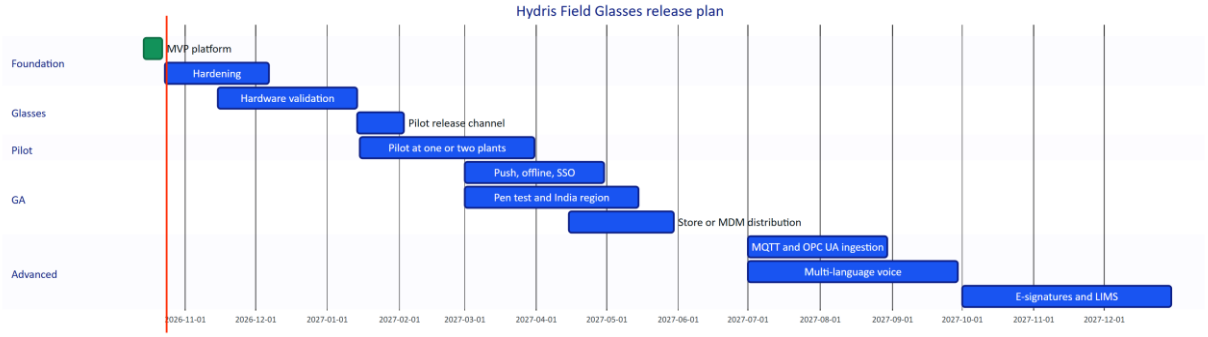

Figure 46. *Release plan (indicative dates)*

2.11.3 Prioritisation

Planned and Partial work was scored for value to the pilot customer and engineering effort by the product team. The chart is a planning aid, not a measurement.

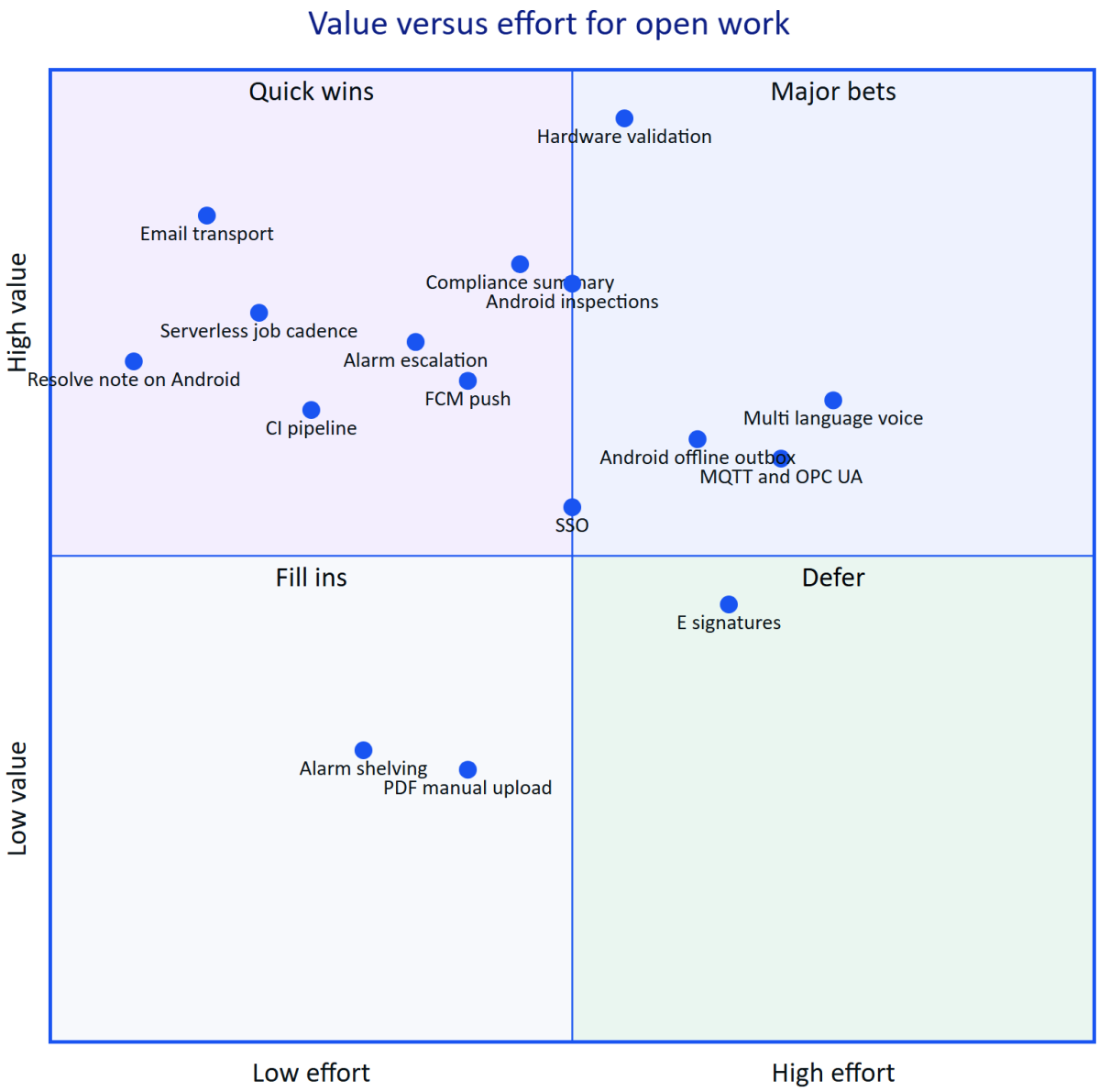

Figure 47. *Planned and partial work by value and effort (product team estimate)*

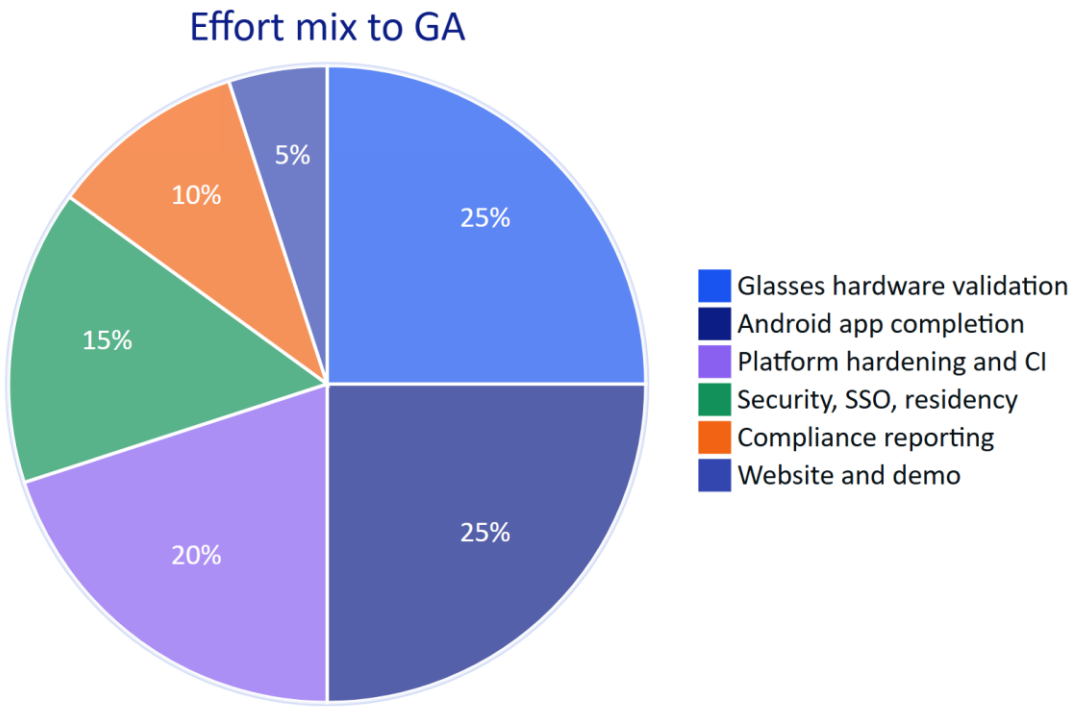

Figure 48. *Where the remaining effort goes before GA (product team estimate, share of engineering weeks)*

DECISION

The pilot runs on the native Android app, not the WebView bridge. The native app keeps device tokens in the Android Keystore, uses ML Kit for faster on-device OCR, and talks to DAT directly. The bridge remains useful for demonstrations of the web glasses app on hardware.

2.12 Dependencies

Table 33. External dependencies, what Hydris uses them for, and the fallback

Dependency	Used for	Owner	Risk if unavailable	Fallback
Meta AI app (`com.facebook.stella`)	Glasses pairing, DAT registration, camera and microphone permissions, firmware	Meta	Glasses features unavailable	Phone camera and phone microphone paths work without it
Meta Wearables DAT 1.0.1 and Developer Mode	Camera stream, photo, capture button, speech	Meta	Cannot reach the glasses	Same as above
Wearables Developer Center registration	Production app ID and client token; publishing beyond developer mode	Meta	Cannot distribute beyond test devices	Developer Mode for the pilot; apply early
Vercel	Hosting web, API route handlers, cron, Blob storage	Vercel	Outage of the hosted service	Container deployment (Docker, Render blueprint) is supported
Postgres (Neon or managed)	Primary database with RLS	Provider	Outage	Point-in-time recovery; PGlite for local and demo

Dependency	Used for	Owner	Risk if unavailable	Fallback
Anthropic API	Assistant model and vision extraction	Anthropic	Degraded answers, no vision	Rules engine and OCR-only extraction are built
Google ML Kit text recognition	On-device OCR in the Android app	Google	Lower OCR quality	Tesseract.js path in the web app
Email provider (Planned)	Invitations, resets, scheduled reports	To be chosen	Invitations and resets do not arrive	Admin copies links from `mail_outbox`

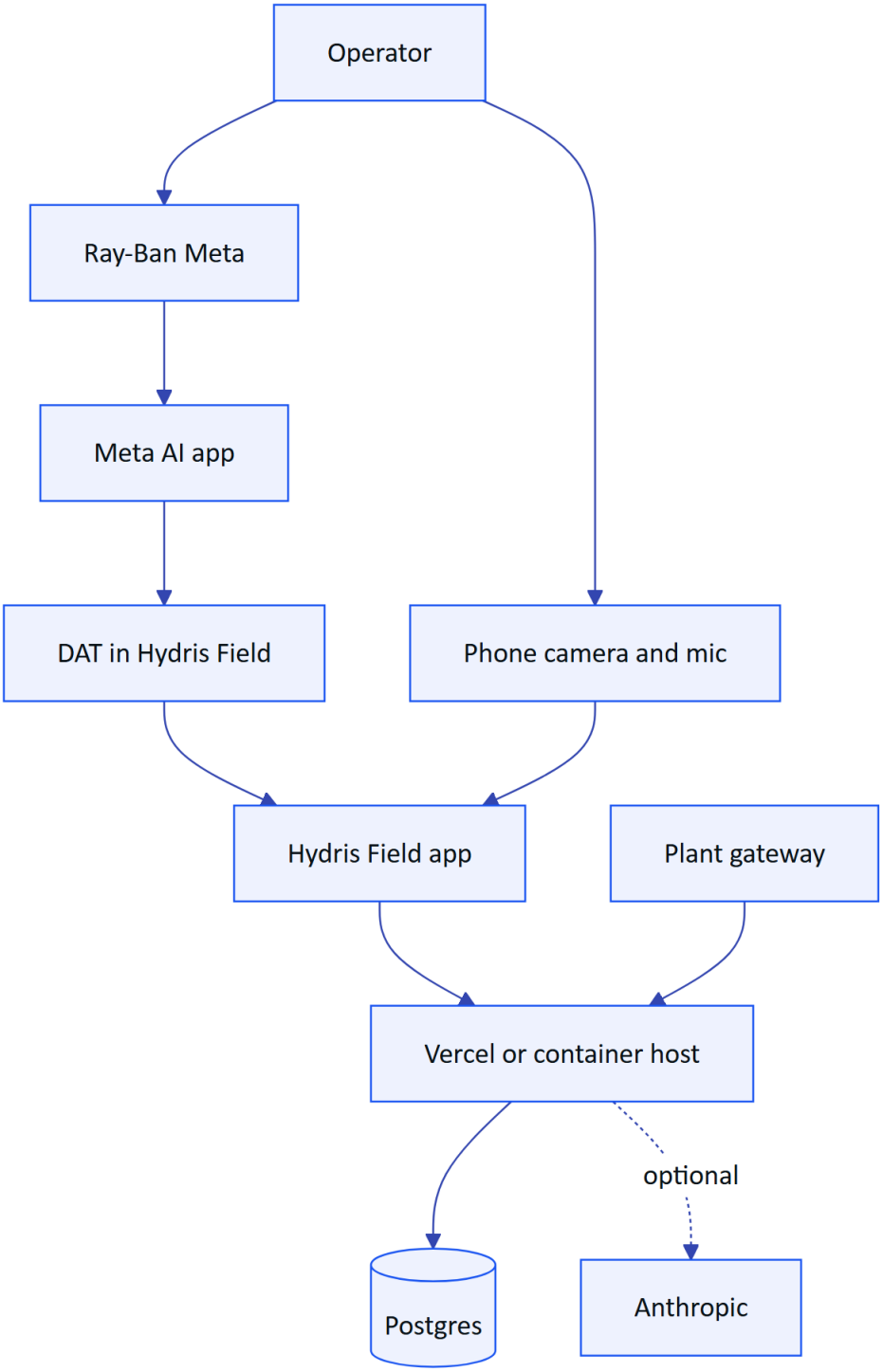

Figure 49. *Dependency chain from operator to plant data*

2.13 Risks and mitigations

Table 34. Product risks, impact, likelihood and mitigations

#	Risk	Impact	Likelihood	Mitigation	Owner
R1	Meta DAT publishing remains gated; store submissions not open when we need them	Cannot distribute glasses features at scale	Medium	Pilot in Developer Mode and Developer Center release channels; phone paths ship independently; apply early	Product
R2	Meta AI app dependency (pairing, registration, permissions, firmware) changes behaviour	Support burden, version drift	Medium	Onboarding checklist; the Glasses screen checks for the Meta AI app and shows link state and a connection log	Mobile
R3	Glasses paths fail on real hardware	Scan and voice on glasses unusable	Medium	Phase 2 hardware matrix before pilot; phone fallback on every glasses feature	Mobile, QA
R4	Email not wired	Invitations and resets do not reach users	High until fixed	FR-086 in Hardening; interim admin copy of links	Platform
R5	Serverless cron runs once a day	Stale-data alarms late on Vercel deployments with real data	High on Vercel Hobby	Move to per-minute cron (Pro plan) or a container host; NFR-008	Platform
R6	No CI	Regressions reach the live demo	Medium	NFR-010 in Hardening	Platform
R7	Android Resolve fails without a note	Operators cannot close the loop on the phone	High	FR-094 fix in Hardening	Mobile
R8	LLM availability, cost or data-residency objections	Degraded answers; pilot blocked by policy	Medium	Rules engine and OCR-only modes; region-pinned provider option; NFR-040	Product
R9	Camera glasses not accepted in a customer's security zones; bystander privacy	Feature disabled on site	Medium	Camera only on intent; UI profile can disable the Scan screen; site camera policy agreed in the pilot contract	Product
R10	Regulators or auditors do not accept Hydris PDFs as primary records	Paper remains the record	Medium	Keep the searchable original scan inside every PDF; FR-068 e-signatures; early conversations with the SPCB through the pilot customer	Product
R11	Device refresh token in `localStorage` on the web glasses app	Token theft on a compromised device	Low for pilot (native app)	Native app with Keystore storage is the pilot path	Security
R12	Audit page images stored in the database	Database growth and cost	Medium at scale	Move page images to object storage in Hardening	Platform

RISK

The single largest schedule risk is R3 combined with R1: everything that makes Hydris unique on glasses (camera scanning, capture button, glasses microphone) is code-complete but unproven on hardware, and distribution beyond developer mode is in Meta's hands. The plan protects the pilot by making every glasses feature work on the phone first.

2.14 Open questions

Table 35. Open questions, owner and when they must be answered

#	Question	Owner	Needed by
Q1	Is India among Meta's supported countries for full DAT capabilities and Developer Center access? Written confirmation needed	Product	Before Phase 2
Q2	When do DAT 1.0 store submissions open, and what review does an unlisted enterprise app need?	Product	Before GA planning
Q3	Will pilot customers allow camera glasses inside their plants, and in which zones?	Customer success	Pilot contract
Q4	Which regulators accept a Hydris-generated PDF as the primary record rather than a scanned copy?	Product, pilot customer	Pilot exit
Q5	Is a separate iOS app needed for pilot or GA customers?	Product	GA planning
Q6	Must LLM inference run in India for Indian customers, and through which provider?	Security	Before enabling the model for a customer
Q7	Should acknowledgement on the glasses count as acknowledgement by the assigned operator, or by the device, for compliance reports?	Product, compliance	Pilot
Q8	What consent-limit data model does FR-120 need (per parameter, per discharge point, per consent version)?	Product	Pilot
Q9	Which email provider meets deliverability and residency needs (SES Mumbai, Resend, other)?	Platform	Hardening
Q10	Should the operator handover summary (FR-122) be generated by the model or by deterministic templates?	Product	GA planning

2.15 Glossary

Table 36. Terms used in this document

Term	Meaning
ETP	Effluent treatment plant: treats industrial wastewater before discharge
STP	Sewage treatment plant: treats domestic wastewater
WTP	Water treatment plant: produces process or drinking water
CETP	Common effluent treatment plant: shared ETP serving an industrial cluster
CPCB / SPCB	Central and State Pollution Control Boards in India

Term	Meaning
OCEMS	Online continuous effluent and emission monitoring system
Consent to operate	SPCB permission that sets a plant's discharge limits
Measurand	A quantity measured by a sensor (pH, dissolved oxygen, COD, and so on)
Freshness	Live, delayed, stale or no data, derived from a sensor's expected interval
Deadband	Hysteresis band that stops an alarm chattering around its threshold
On-delay	Time a breach must persist before an alarm is raised
RLS	Row-level security in Postgres, used for tenant isolation
SSE	Server-sent events, the realtime channel from API to clients
DAT	Meta Wearables Device Access Toolkit for Ray-Ban and Oakley Meta glasses
Meta AI app	Meta's companion app that pairs, registers and updates the glasses
UI profile	Versioned configuration of what a pair of glasses shows and allows
Four eyes	Rule that a second, different person approves what the first verified
Agreement	Whether independent OCR and vision readings of a value match
MoSCoW	Must, Should, Could, Won't prioritisation
Demo mode	Zero-configuration deployment with an in-memory synthetic tenant

2.16 Appendix: traceability to other documents

Each requirement area is specified in more depth in the other documents of the suite. The table shows where to look.

Table 37. Traceability from requirement areas to the documentation suite

Requirement area	ARCHITECTURE	DATA_MODEL	API_REFERENCE	MOBILE_APP_SPEC	GLASSES_INTEGRATION	D
Identity and tenancy (FR-001 to FR-012)	Tenancy, token model	Identity tables, RLS	Auth, org	Sign-in		
Telemetry (FR-020 to FR-029)	Telemetry flow	Plant tables	Plant, telemetry	Sensor screens		D
Alarms (FR-030 to FR-038)	Alarm flow	Alarm tables	Alarms	Alarm screens		S
Work (FR-040 to FR-047)	Offline sync	Work tables	Inspections, maintenance	Work tab	HUD screens	
Devices (FR-050 to FR-059)	Pairing flow	Device tables	Devices, UI profiles	Glasses screen	Capability matrix	H
Android (FR-090 to FR-099)	Containers		All client routes	Whole document	DAT flows	T
Audit scanning (FR-060 to FR-069)	Audit scan flow	Audit document tables	Audit documents	Scan screens	Camera bridge	S
Voice (FR-070 to FR-078)	Voice flow		Assistant	Assistant screen	Speech bridge	V

Requirement area	ARCHITECTURE	DATA_MODEL	API_REFERENCE	MOBILE_APP_SPEC	GLASSES_INTEGRATION	D
Documents (FR-110 to FR-114)		Documents	Plant documents			
Platform and insights (FR-080 to FR-089, FR-120 to FR-122)	Realtime	Platform tables	Insights, realtime	Home		C
Website and demo (FR-100 to FR-103)	Demo topology		Public, auth demo	Demo sign-in		W
Non-functional (NFR-001 to NFR-040)	All	Retention	Errors, limits	Performance budgets	Limits	A

NOTE

Every document referenced in this table is part of the suite in `docs/product/` and its Word edition in `docs/product/word/`.

3 Architecture

> **NOTE**
>
> **HFG-ARC v2.0** · Draft for review · Owner: Engineering, Hydris. How Hydris Field Glasses is built today, why it is built that way, and how it grows from a pilot plant to a regional fleet

This document describes the software architecture of Hydris Field Glasses: the operator console, the glasses heads-up app and simulator, the native Android companion app "Hydris Field", the API and the shared domain logic. It is written against the repository at commit `07cca23` on `main`. Every statement about the current system names the file it comes from. Where the code and an older document disagree, the code wins and this document follows the code.

The architecture has one organising idea: **one tenant-isolated API, one shared domain model, many thin clients**. The website, the console, the glasses HUD, the simulator and the Android app all speak the same JSON and Server-Sent Events contract to one Fastify application. That application can run as a long-lived server, as a container, or inside a Next.js route handler on Vercel, without code changes.

15
API modules
Fastify plugins under `apps/api/src/modules`

147
HTTP routes
Including `/api/health`; counted from the route registrations

42
Database tables
`apps/api/src/db/schema.ts`, 9 SQL migrations

4
Deployable units
Web (with built-in API), API container, Android app, WebView bridge

Status labels used throughout:

Table 38. Status labels

Label	Meaning
Built	On `main`, exercised by tests or the live demo at https://hydrisview.vercel.app
Partial	On `main` but incomplete, unverified on hardware, or limited to one topology
Planned	Not in the repository. Design intent only

3.1 Architectural drivers

3.1.1 Business goals and constraints

The architecture is shaped by a small set of goals that come from the product requirements (`PRD.md`) and from the environment the product runs in: water and wastewater plants (ETPs, STPs, CETPs) where operators wear gloves, stand on ladders and work far from the control room.

Table 39. Architectural drivers

Driver	What it demands of the architecture	Where it shows up
Read-only to the plant	No write path to PLC, SCADA or DCS. Telemetry is pushed in, never pulled out of control networks	`POST /api/ingest/v1/readings` only accepts readings (`modules/telemetry.ts`)

Driver	What it demands of the architecture	Where it shows up
Strict tenant isolation	A missing `WHERE` clause must not leak another customer's data	Postgres RLS on every tenant table (`drizzle/0001_rls.sql`, `db/tenant.ts`)
Hands-free field use	Short, spoken, glanceable interactions; tolerate flaky Wi-Fi	Glasses HUD (`apps/web/src/glasses`), voice (`modules/voice-assistant.ts`), offline queue (`glasses/link.ts`)
Defensible records	Every reading, correction and approval must be attributable and append-only	`audit_events`, `telemetry_corrections`, `audit_document_events` grants
Zero-configuration demo	A prospect opens a URL and gets a live, believable plant	Demo mode (`apps/api/src/demo.ts`, `core/demo-shared.ts`)
Small team, fast iteration	One language, one repository, one deployable for most customers	TypeScript monorepo, modular monolith
Hardware churn	Meta glasses SDKs change quickly; the display glasses are not yet broadly available	Device integration isolated behind `SecureStore`, camera and voice adapters

3.1.2 Architecture principles

1. **The API is the product boundary.** Clients hold no business rules that the API does not also enforce. `packages/domain/src/roles.ts` says it plainly: the matrix is the single source of truth, "the API enforces it on every request and clients only use it to decide what to show".
2. **Pure logic lives in `packages/domain`.** State machines, threshold evaluation, unit normalisation, OCR structuring and UI profile resolution have no I/O and are unit-tested in isolation.
3. **Defence in depth for tenancy.** Role checks in handlers, site scoping in queries, and row-level security in the database. Any one layer failing does not expose data.
4. **One ingestion path.** Gateways, manual entry, laboratory entry and the simulator all go through `ingest()` in `modules/telemetry.ts`. Alarms cannot be raised by a side door.
5. **Runtime portability.** The same `buildApp()` (`apps/api/src/app.ts`) runs under `server.ts` (long-lived), `serverless.ts` (Vercel) and the test harness.
6. **Events are hints, the database is truth.** Realtime events tell clients what to refetch. Losing or duplicating one never corrupts state.

3.2 System context

3.2.1 Context diagram

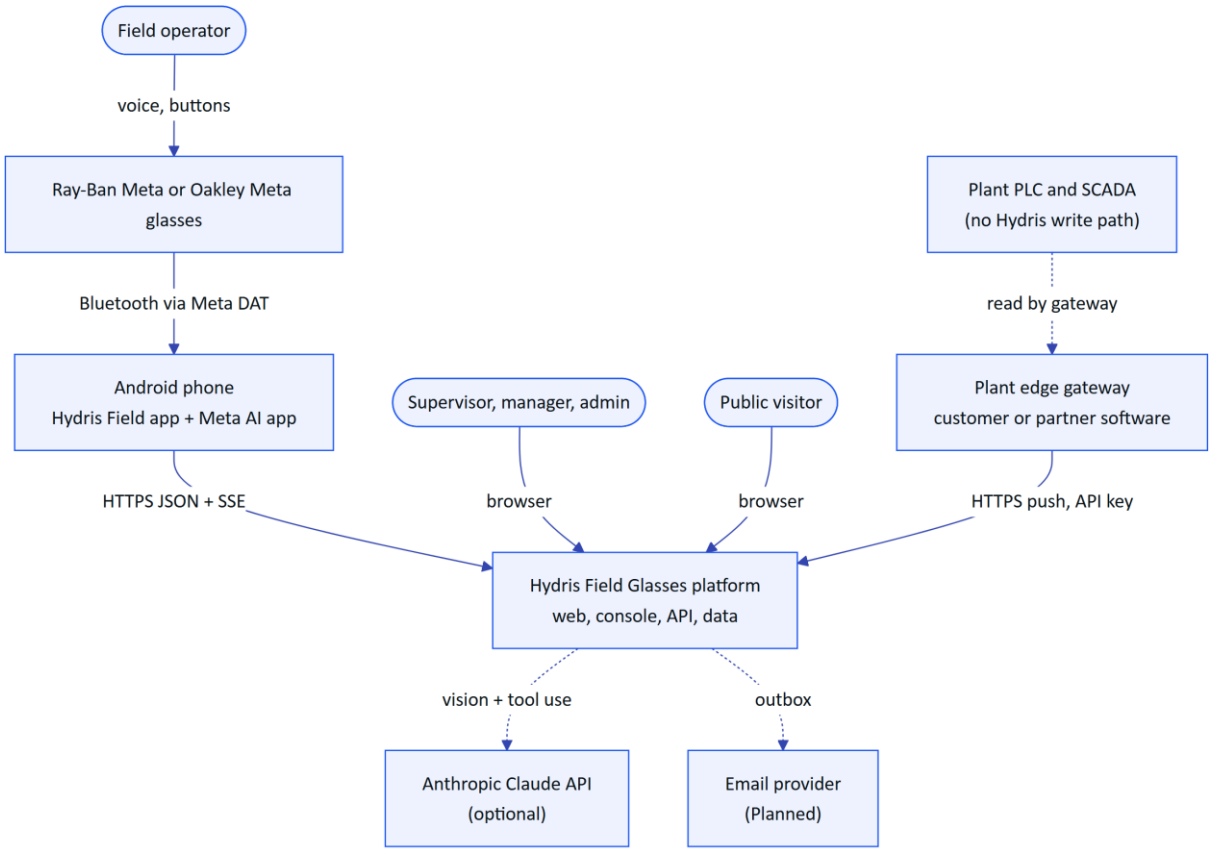

Figure 50. *System context (C4 level 1). Hydris sits beside the plant control system, never inside it.*

3.2.2 Actors and external systems

Table 40. Actors and external systems

Element	Kind	Interaction	Status
Field operator	Person	Glasses HUD, Android app, voice	Built (web HUD, Android app). Partial on real glasses hardware
Supervisor, maintenance engineer, manager, org admin	Person	Operator console (`/console`)	Built
Platform admin	Person	`/console/platform`, `/api/platform/organizations`	Built
Public visitor	Person	Website, live glasses demo, contact form	Built
Meta glasses	Device	Camera, microphone, capture button through the Meta Wearables Device Access Toolkit (DAT) 1.0.1	Partial (code complete, limited hardware verification)
Plant edge gateway	System	Pushes readings with a `hgw_` key	Built (endpoint). Gateway software itself is Planned
Anthropic Claude API	System	Document vision extraction and voice assistant tool use	Built, optional (enabled by `ANTHROPIC_API_KEY`)

Element	Kind	Interaction	Status
Email provider	System	Invitations and password reset	Planned. Messages are written to `mail_outbox` (`core/services.ts`)

> **IMPORTANT**
>
> Hydris never connects to a PLC, SCADA or DCS and has no endpoint that writes to one. Ingestion is outbound HTTPS from the plant to Hydris. This keeps Hydris outside IEC 62443 control zones and means a compromise of Hydris cannot change the physical process.

3.3 Containers

3.3.1 Container diagram

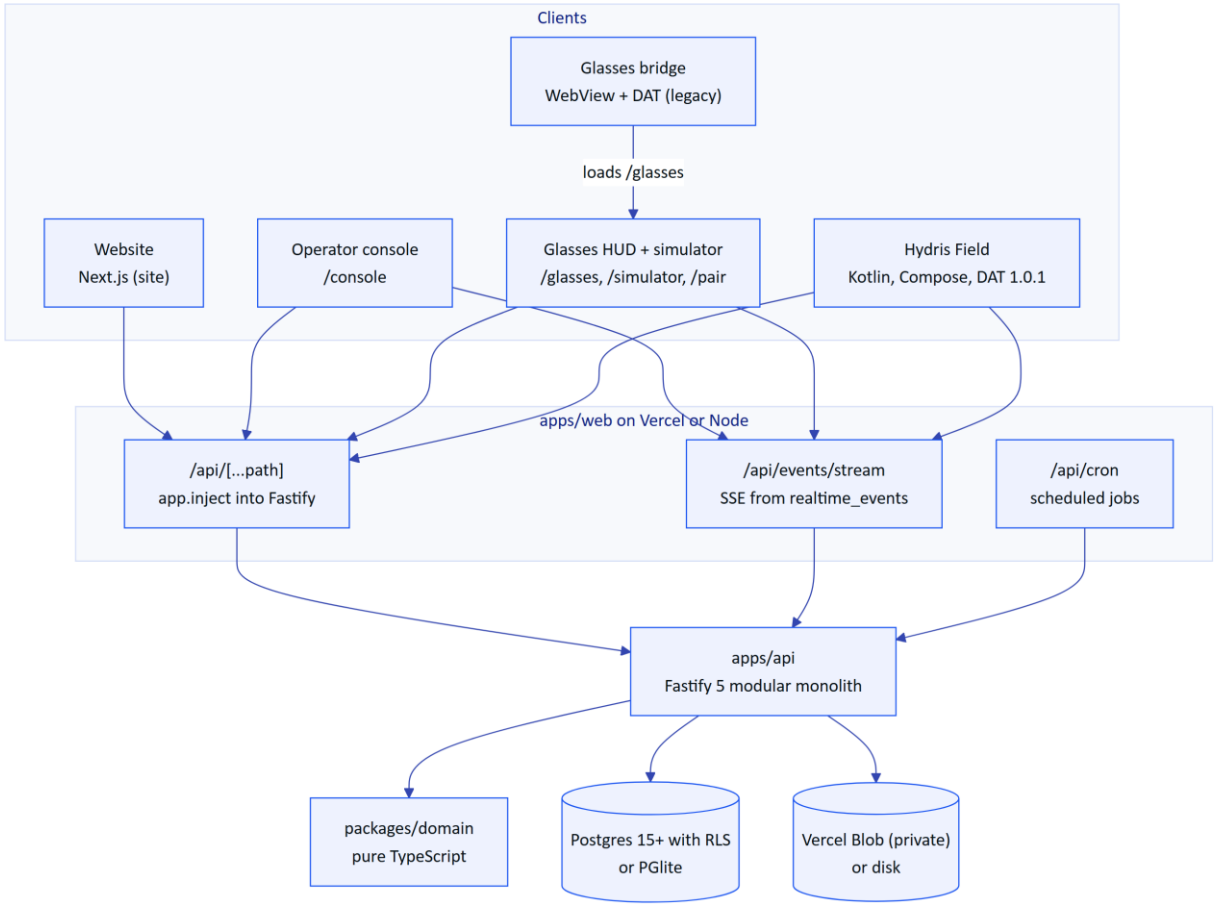

Figure 51. *Containers (C4 level 2). The API can run inside the web deployment or as its own container.*

Table 41. Containers and technology

Container	Technology (versions from `package.json` and `build.gradle.kts`)	Key files	Status
API	Fastify 5.12.7, zod 4.6.7, Drizzle ORM 0.45.4, `pg` 8.23.1, PGlite 0.5.9, `@fastify/jwt`, `@fastify/rate-limit`, `@node-rs/argon2`, `pdf-lib`, `@anthropic-ai/sdk` 0.137.0	`apps/api/src/app.ts`, `modules/*`, `core/*`	Built
Web	Next.js 16.4, React 19.3, TanStack Query 5, Tailwind 4, Radix UI, Recharts, Tesseract.js 7	`apps/web/src/**`	Built
Domain	TypeScript, no runtime dependencies	`packages/domain/src/*`	Built
Contracts	Placeholder package (`export {}`)	`packages/contracts/src/index.ts`	Planned
Hydris Field (Android)	Kotlin, Jetpack Compose (BOM 2026.05.01), OkHttp 4.12, CameraX 1.4.2, ML Kit text recognition 16.0.1, DAT 1.0.1 (core, camera, speech, inputs)	`apps/android/app/src/main/java/ai/hydris/field/**`	Built (Partial on hardware)
Glasses bridge	Kotlin, Android WebView, DAT 1.0.1	`apps/glasses-bridge-android/.../MainActivity.kt`	Partial, superseded by Hydris Field
Database	Postgres 15+ (Neon, Render, Supabase, self-hosted Postgres 17 in Compose) or PGlite in-process	`apps/api/drizzle/0000` to `0008`	Built
File storage	Vercel Blob private store or local disk	`core/services.ts` (`createStorage`)	Built

3.3.2 Repository layout

The repository is an npm workspaces monorepo (`package.json`, `"workspaces": ["packages/*", "apps/*"]`, Node 22.x). The web app depends on the API package directly (`"@hydris/api": "0.2.0"`) and imports `@hydris/api/serverless`, which is how the API runs inside Next.js.

Table 42. Workspaces

Workspace	Package name	Role
`packages/domain`	`@hydris/domain`	Roles, permissions, measurands, alarm and workflow state machines, device lifecycle, UI profiles, audit document structuring, voice parsing, simulation scenarios
`packages/contracts`	`@hydris/contracts`	Reserved for shared request and response schemas (empty today)
`apps/api`	`@hydris/api`	HTTP API, database schema and migrations, seeds, jobs, demo runtime
`apps/web`	`@hydris/web`	Website, console, glasses HUD, simulator, pairing page, built-in API route handlers

Workspace	Package name	Role
`apps/android`	Gradle project	Native operator app with glasses integration (not an npm workspace member in practice: no `package.json`)
`apps/glasses-bridge-android`	Gradle project	WebView companion that bridges DAT into the web HUD

DECISION

One monorepo, one language on the server and web. The API, web and domain packages share TypeScript types without a publishing step, and a change to a state machine lands in one commit across server and clients. The Android app is the exception: it mirrors the JSON contract by hand in `Models.kt` until `packages/contracts` produces a schema that Kotlin can consume (see the technical debt register).

3.4 API: a modular monolith

3.4.1 Component view

`buildApp()` in `apps/api/src/app.ts` creates one Fastify instance, registers shared plugins (helmet, cookie, global rate limit, JWT, multipart), installs the error handler and the glasses UI profile hook, then registers fifteen route modules under the `/api` prefix. Every module receives the same `Ctx` (`apps/api/src/context.ts`): database handle, event bus, configuration, storage adapter and the live-device map.

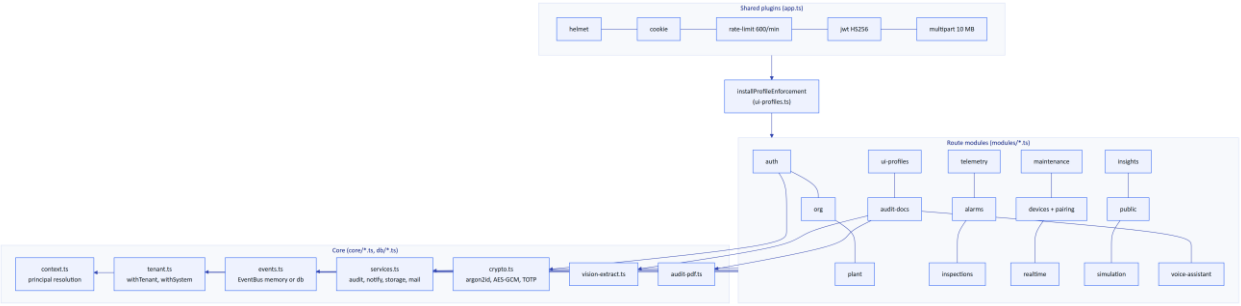

Figure 52. *API components (C4 level 3). Modules share the context object and the core services, never each other's tables directly except through exported functions.*

Table 43. API modules

Module	Routes	Responsibility	Notable rules
`auth.ts`	17	Login, demo guest login, MFA (TOTP), refresh rotation, logout, invitations, password reset, sessions, org switch	Lockout after 5 failures for 15 min, refresh reuse detection with a 30 s grace
`org.ts`	13	Organization, members, invitations, notifications, audit log, platform organizations	No role escalation (`assignableRoles`)
`plant.ts`	28	Sites, facilities, process areas, assets, sensors, gateways, documents, attachments, hierarchy, history	Attachments only served through the authorised route
`telemetry.ts`	6	Gateway ingestion, manual and lab entry, downsampled history, CSV export, corrections	Single `ingest()` path, idempotent on `(sensor_id, ts)`

Module	Routes	Responsibility	Notable rules
`alarms.ts`	9	Alarm rules, alarm list and detail, transitions, stale-data evaluation	Deadband and on-delay, note required to resolve
`inspections.ts`	15	Templates, routes, inspections, step responses, submit and review	Offline replay by `clientOpId`, `revision` counter
`maintenance.ts`	7	Work orders, checklists, activity	Transition table in `domain/workflows.ts`
`devices.ts`	16	Pairing challenges, approval, device refresh, device context, fleet admin	RFC 8628 shape, session version, refresh family revocation
`realtime.ts`	1	`GET /api/events/stream` (SSE)	Per-connection `canReceive` authorisation
`insights.ts`	4	Overview KPIs, grounded document assist, search, reports	Read-only
`public.ts`	1	Website contact and demo requests	5 requests per 10 min
`simulation.ts`	3	Start, stop and inspect demo scenarios	Synthetic data rejected for non-demo tenants
`ui-profiles.ts`	13	Glasses UI profiles, versions, assignments, effective profile	Enforced on device requests by an `onRoute` hook
`audit-docs.ts`	12	Scanned audit documents: capture, enhance, review, verify, approve, PDF	Four-eyes approval, optimistic concurrency
`voice-assistant.ts`	1	`POST /api/assistant/ask`	Claude tool use or deterministic rules, 60 per min

Route counts come from the `app.get/post/put/patch/delete` registrations; `devices.ts` registers `suspend`, `reactivate` and `revoke` in one loop and is counted as three.

3.4.2 Request lifecycle

Every authenticated request follows the same path. In the built-in API mode on Vercel, the path starts in a Next.js route handler that forwards the request into Fastify with `app.inject` (`apps/web/src/app/api/[...path]/route.ts`), so there is no network hop between the web and the API.

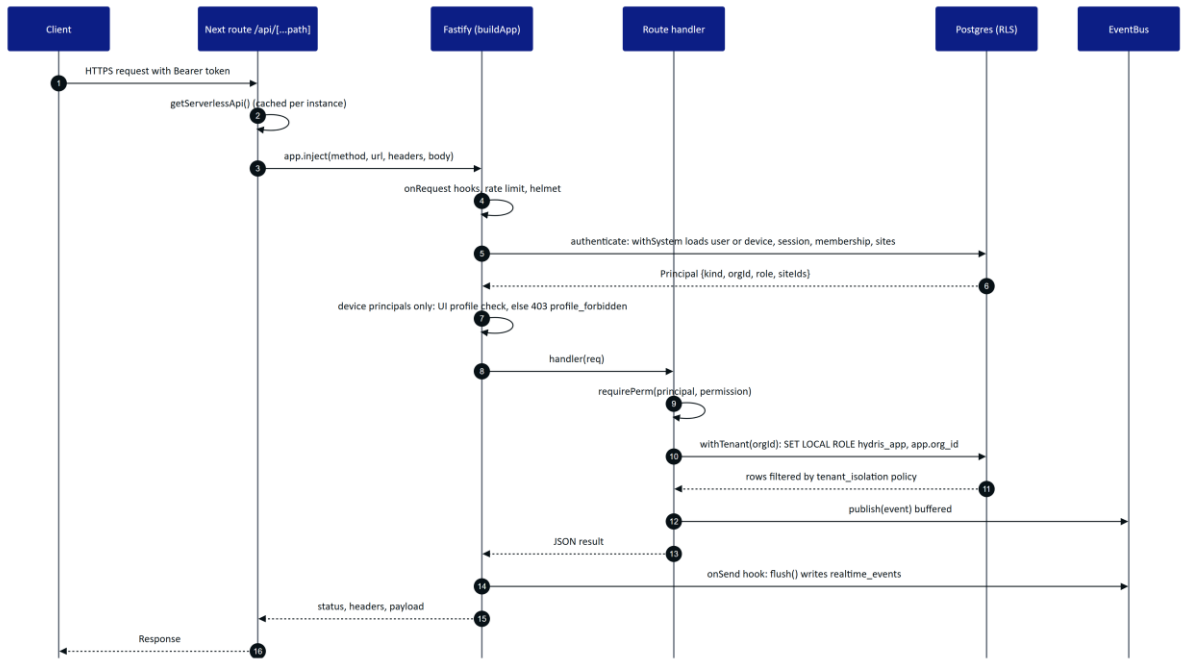

Figure 53. *Request lifecycle for an authenticated call in the built-in API mode, from the Next.js route to the RLS-protected query and the event flush.*

The order matters in three places:

- **Principal resolution re-checks revocation on every request.** `resolveTokenPrincipal()` loads the user status, confirms the session family in `auth_sessions` has not been revoked, and reloads the membership and site list. A device token is rejected if the device is not active, its `session_version` changed, or it has no current assignment. Logout and revocation therefore take effect on the next request, not when the 15 minute access token expires.
- **The UI profile hook runs after authentication.** `installProfileEnforcement()` is registered before the modules so that its `onRoute` hook can append a check to the matching routes. It only applies to device principals: a pair of glasses whose profile hides the alarms screen gets `403 profile_forbidden` from `GET /api/alarms` even if the operator's role allows it.
- **Events are written after the handler returns.** In `db` realtime mode the bus buffers publishes and the `onSend` hook calls `flush()`, so event rows are inserted outside the handler's transaction (`core/events.ts`).

3.4.3 Error contract

All errors use one shape, produced by the error handler in `app.ts` and the helpers in `core/errors.ts`:

```
{ "error": { "code": "validation_failed", "message": "Some fields are invalid", "details": [{ "path": "readings.0.ts", "message": "Invalid ISO datetime" }] } }
```

Table 44. Error mapping

Source	Status	Code
`HttpError` thrown by a handler	As thrown (400, 401, 403, 404, 409)	As thrown, for example `note_required`, `invalid_transition`, `stale`
`ZodError` from request parsing	400	`validation_failed`, with `details[]` per field
Rate limiter	429	`rate_limited`
Anything else	500	`internal_error`, message masked as "Unexpected server error", stack logged

Source	Status	Code
API failed to start (route handler)	503	`service_unavailable` with a short detail
SSE route in the wrong realtime mode	503	`realtime_misconfigured`

Records outside the caller's site scope return **404, not 403** (`requireSite()` in `core/principal.ts`), so a user cannot probe for the existence of another site's assets or alarms.

3.5 Domain model

3.5.1 Key domain types

`packages/domain` is the vocabulary of the system. It has no I/O and is imported by the API, the web app and (by convention, mirrored by hand) the Android app.

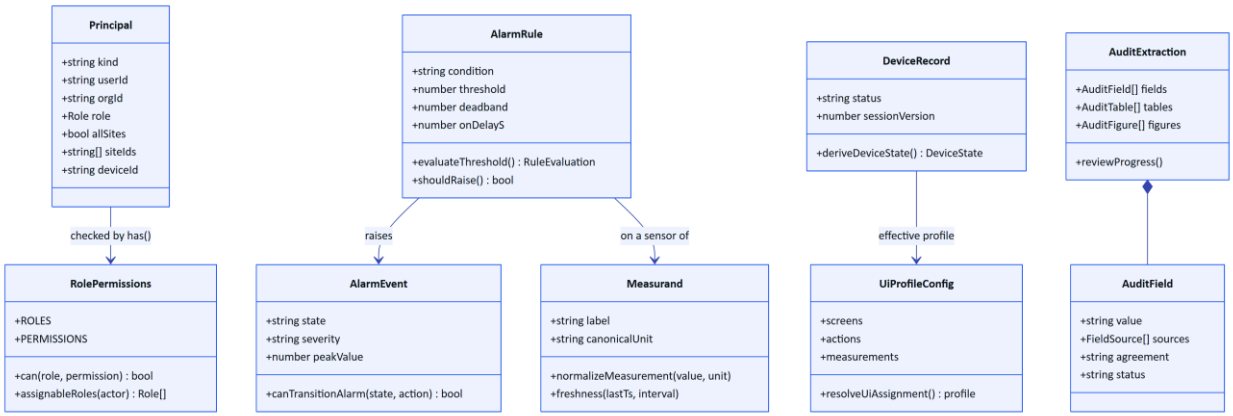

Figure 54. *Key domain types and the functions that own their rules (packages/domain/src and apps/api/src/core/principal.ts).*

Table 45. Domain modules

File	Owns	Used by
`roles.ts`	6 roles, the permission list, `ROLE_PERMISSIONS`, `assignableRoles`, `SITE_SCOPED_BY_DEFAULT`	API authorisation, console navigation, Android visibility
`measurands.ts`	Measurand definitions (label, units, conversions, typical range, decimals), `normalizeMeasurement`	Ingestion, charts, voice answers
`freshness.ts`	`live`, `delayed`, `stale`, `no_data` from last timestamp and expected interval	Console, HUD, stale-data alarms
`alarms.ts`	Severities, conditions, states, `ALARM_TRANSITIONS`, `evaluateThreshold` (deadband), `shouldRaise` (on-delay)	Alarm engine, console, HUD
`workflows.ts`	Inspection and maintenance state machines, step kinds, progress	Inspections and maintenance modules
`devices.ts`	Device states, `deriveDeviceState`, pairing alphabet (30 characters), length 8, TTL 300 s, poll interval 3 s	Pairing, fleet views
`uiprofile.ts`	Screens, widgets, actions, text scales, precedence device then user then role then org	Profile enforcement, HUD

File	Owns	Used by
auditdoc.ts	OCR types, key/value and table structuring, cross-check, plausibility, agreement score	Audit scan pipeline on client and server
voice.ts	Wake phrase regex (WAKE_RE), utterance normalisation, measurand detection	Web voice, rules engine
scenarios.ts, random.ts, catalog.ts	Simulation scenarios with deterministic noise, enumerations	Simulator, seeds

3.5.2 Alarm state machine

Alarm transitions are data, not code paths: ALARM_TRANSITIONS in packages/domain/src/alarms.ts lists for each action the allowed source states, the target state, the permission and whether a note is required. transitionAlarm() in modules/alarms.ts applies it, writes alarm_event_history, appends an audit event and publishes alarm.updated. The transitions are: acknowledge (active to acknowledged), investigate (active or acknowledged to investigating), resolve (acknowledged or investigating to resolved, note required) and close (resolved to closed). The condition clearing (clearedAt) is separate from the human workflow: an alarm whose value returned to normal stays open until someone acknowledges and resolves it.

3.6 Data architecture and tenancy

3.6.1 Schema overview

apps/api/src/db/schema.ts defines 42 tables. DATA_MODEL.md documents each column; this chapter covers the structural decisions.

Table 46. Table groups

Group	Tables	Notes
Identity and tenancy	users, organizations, memberships, membership_sites, invitations, auth_sessions, auth_tokens, mail_outbox	users is global; membership is per organization
Plant model	sites, facilities, process_areas, assets, gateways, sensors, documents, attachments	Hierarchy site, facility, process area, asset, sensor
Telemetry and alarms	telemetry_readings, telemetry_corrections, alarm_rules, alarm_events, alarm_event_history	Readings are immutable; corrections supersede for display
Work	inspection_templates, inspection_routes, inspections, inspection_responses, maintenance_tasks, maintenance_activity	inspections.revision for optimistic concurrency
Devices	devices, device_assignments, pairing_challenges, device_sessions	Assignment history is kept (unassigned_at)
Glasses UI	ui_profiles, ui_profile_versions, ui_profile_assignments	Versioned configuration
Audit documents	audit_documents, audit_document_pages, audit_document_events	Page images stored in the database as base64 text
Platform	notifications, audit_events, inquiries, simulation_runs, realtime_events	realtime_events is system-only

telemetry_readings uses a bigint identity key, a unique index on (sensor_id, ts) that makes ingestion idempotent, a descending (sensor_id, ts) index for latest-value and history reads, and an (org_id, ts) index. There is no partitioning or retention policy yet.

3.6.2 Row-level security

Tenant isolation is enforced by Postgres, not only by application code.

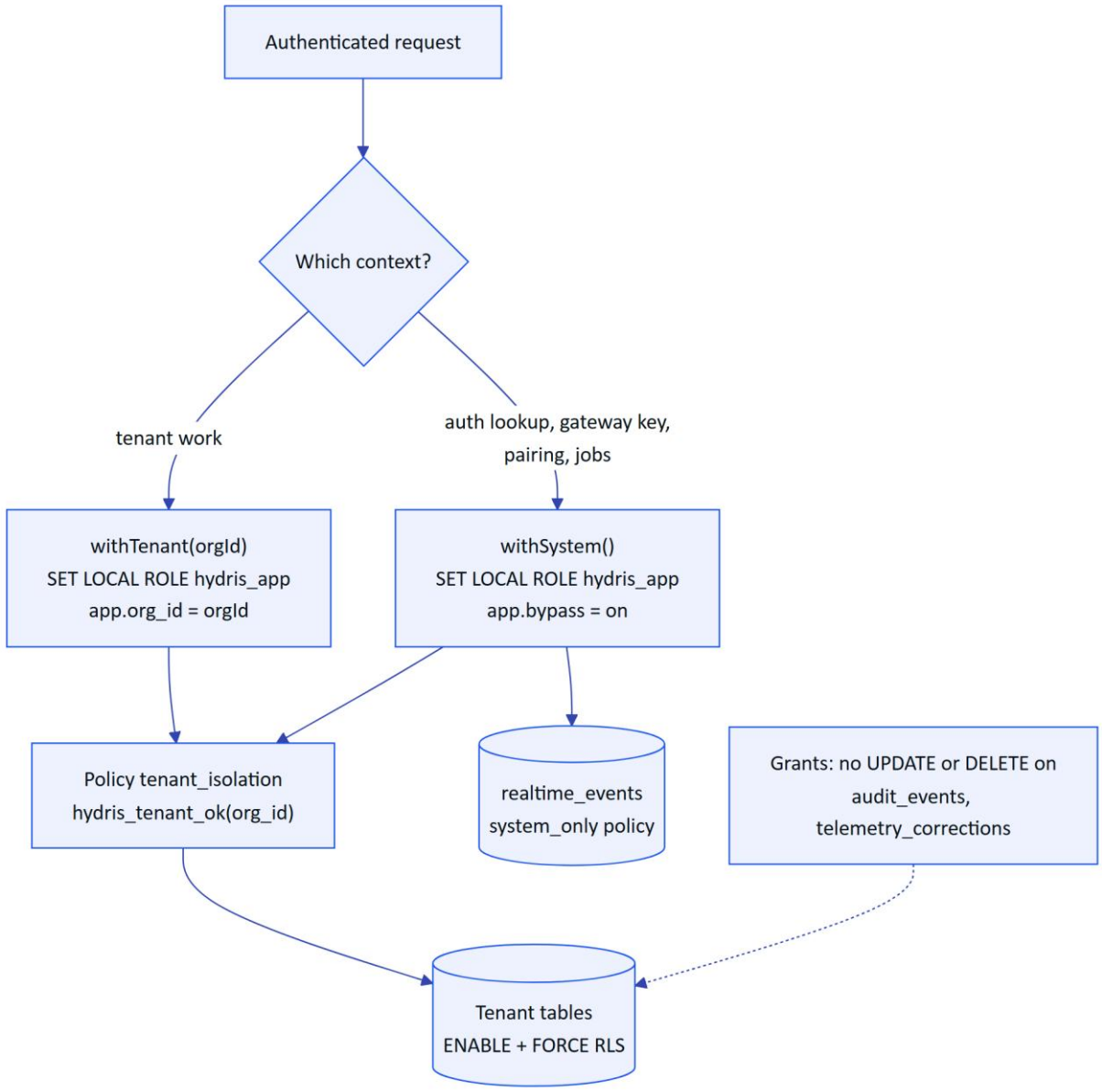

Figure 55. *How a request reaches tenant rows. Both contexts run as the non-privileged hydris_app role; only the session settings differ.*

The mechanics, from `drizzle/0001_rls.sql`, `0003`, `0006`, `0008` and `db/tenant.ts`:

- `hydris_app` is created `NOLOGIN` and without `BYPASSRLS`. Migration `0004_app_role_membership.sql` grants the connecting owner role membership `WITH SET TRUE` so that managed Postgres (PG16+) allows `SET ROLE`.
- `hydris_tenant_ok(row_org)` returns true when `app.bypass` is `on` or when `row_org` equals `app.org_id`. Every tenant table has a `tenant_isolation` policy `USING` and `WITH CHECK` that function, so both reads and writes are checked.
- `withTenant()` and `withSystem()` each open a transaction and use `SET LOCAL` and `set_config(..., true)`, so settings cannot leak to the next request on a pooled connection.

- `audit_events` and `telemetry_corrections` have `UPDATE` and `DELETE` revoked from `hydris_app`, and `audit_document_events` has only `SELECT` and `INSERT`. Even system context cannot rewrite history.
- `realtime_events` uses a `system_only` policy: clients never read it directly; the stream reads it in system context and filters per connection.

> **DECISION**
>
> **Postgres RLS as the last line of tenant isolation.** The application already filters by organization and site. RLS makes a forgotten `WHERE` clause a non-event instead of a data breach, and it lets a security reviewer verify isolation by reading 46 lines of SQL instead of every query. The cost is that every request runs in a transaction, and system work must be explicit (`withSystem`), which is a useful signal in code review.

3.6.3 Append-only records and provenance

- A manual or lab reading is stored as an ordinary immutable reading; the audit row (`telemetry.manual_entry`) is the provenance record of who entered it (`modules/telemetry.ts`).
- Corrections never overwrite `telemetry_readings`. A `telemetry_corrections` row supersedes the value for display and aggregation, and history queries compute effective values with a lateral join.
- Audit document reviews keep the first extracted value in `extracted` and write every confirm, correct, flag, verify and approve action to `audit_document_events`.
- Synthetic data is marked (`synthetic = true`) and `ingest()` refuses simulator data for any tenant that is not a demo organization (`synthetic_not_allowed`).

3.6.4 Migrations

Nine SQL migrations (`apps/api/drizzle/0000_init.sql` to `0008_audit_documents_rls.sql`) are generated by drizzle-kit and applied by the Drizzle migrator. On Postgres, `createDatabase().migrate()` takes a session-level advisory lock (key `4872917633`) on a dedicated connection, so several serverless instances cold-starting at once, or a deploy and an instance, cannot run migrations concurrently (`db/client.ts`). On Vercel the migrations folder is shipped with the functions through `outputFileTracingIncludes` in `apps/web/next.config.ts` and located at runtime by `findMigrationsFolder()`.

3.7 Identity, sessions and tokens

3.7.1 Token model

Table 47. Credentials

Credential	Format and lifetime	Held by	Revocation
User access token	JWT HS256 `{typ: user, sub, org, sid}`, 15 min (12 h in demo mode)	Client memory only	`sid` is the refresh family; revoking the family invalidates access tokens on the next request
User refresh token	48 random bytes, SHA-256 stored in `auth_sessions`, 7 days, cookie `hg_refresh` (httpOnly, SameSite=Strict, path `/api/auth`, Secure in production)	Browser cookie jar; Android encrypted cookie store	Rotation on every use; replay after 30 s revokes the family
MFA challenge	JWT `{typ: mfa}`, 5 min	Client	Single step
Device access token	JWT `{typ: device, sub, org, sv}`, 15 min	Device memory	`devices.session_version` bump, suspend, revoke, unassign

Credential	Format and lifetime	Held by	Revocation
Device refresh token	`hgd_` plus 40 random bytes, `device_sessions`, 30 days, rotating	Android Keystore-encrypted store; web `localStorage`	Reuse revokes the family and is audited (`device.refresh_reuse_detected`)
Gateway key	`hgw_` prefix, SHA-256 in `gateways.key_hash`	Gateway	`POST /api/gateways/:id/revoke`
Demo session cookie	Signed JWT `hg_demo`	Browser	Demo mode only
Demo device refresh	`hgd_z` plus HMAC-signed claims (org, device, user, site, version)	Device	Demo mode only

Passwords are hashed with argon2id (memory 19456 KiB, time cost 2, parallelism 1) and a dummy verification burns equal time for unknown emails (`core/crypto.ts`). TOTP secrets are encrypted at rest with AES-256-GCM under `DATA_KEY`.

3.7.2 Refresh rotation and reuse detection

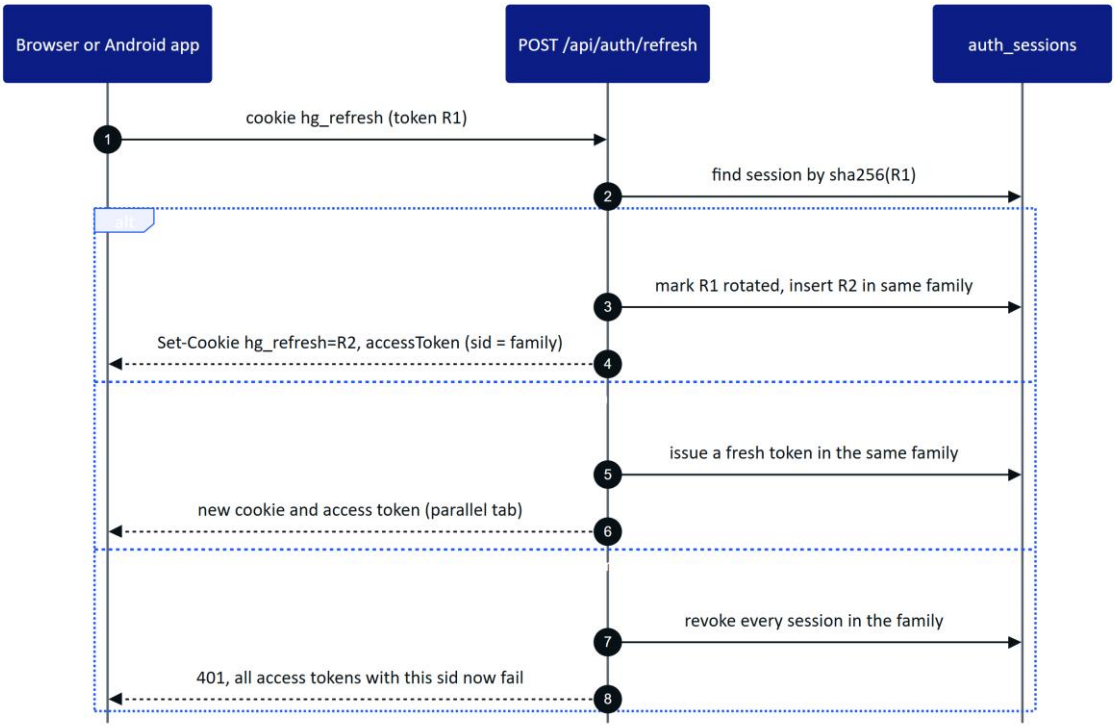

Figure 56. *User refresh rotation with family revocation (modules/auth.ts). The 30 s grace absorbs parallel tabs; a later replay is treated as theft.*

Clients refresh with a **single-flight** guard so parallel 401s trigger one refresh: `refreshSession()` in `apps/web/src/lib/api.ts`, `refresh()` in `apps/web/src/glasses/link.ts`, and a coroutine `Mutex` in `HydrisApi.kt` on Android.

3.7.3 Session and token states

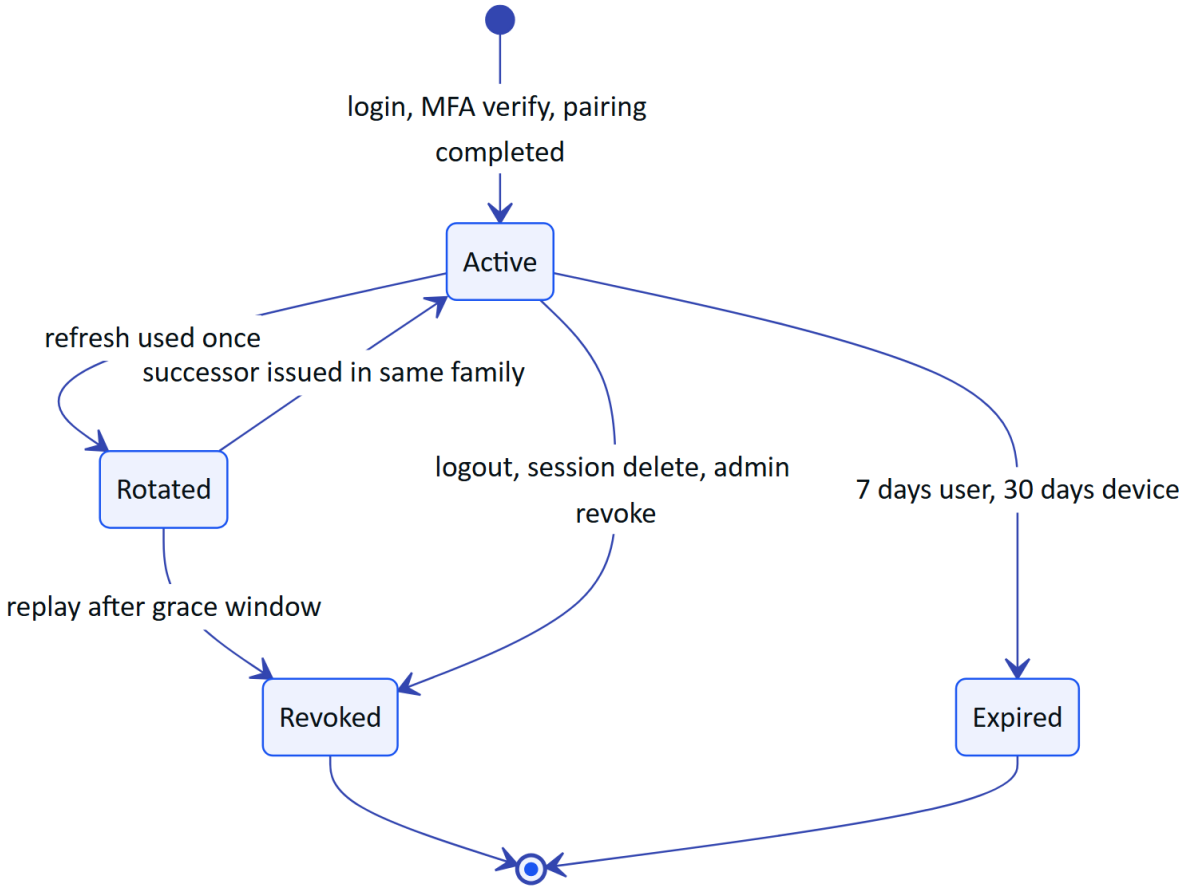

Figure 57. *Lifecycle of a refresh-token family (user sessions in auth_sessions, device sessions in device_sessions).*

3.7.4 Principals and permissions

A verified token becomes a `Principal` (`core/principal.ts`). Devices act **on behalf of their assigned operator**: a device principal carries the operator's `userId`, role and sites, and `has()` additionally requires the permission to be in `DEVICE_PERMISSIONS`, a fixed set of 13 permissions (read site, asset, telemetry, alarm, inspection, maintenance, document and audit document data; acknowledge alarms; execute inspections and maintenance; request maintenance; capture audit documents). Glasses therefore can never verify or approve a document, manage members or edit rules, whatever the operator's role. Audit document verification and approval add an explicit `p.kind !== 'user'` check on top.

3.8 Devices and pairing

3.8.1 Pairing flow

Pairing follows the shape of the OAuth 2.0 Device Authorization Grant (RFC 8628). The glasses or the app requests a challenge, shows a short code and a QR link, and a signed-in person approves it. The QR encodes only the verification URL and the short code, never a credential.

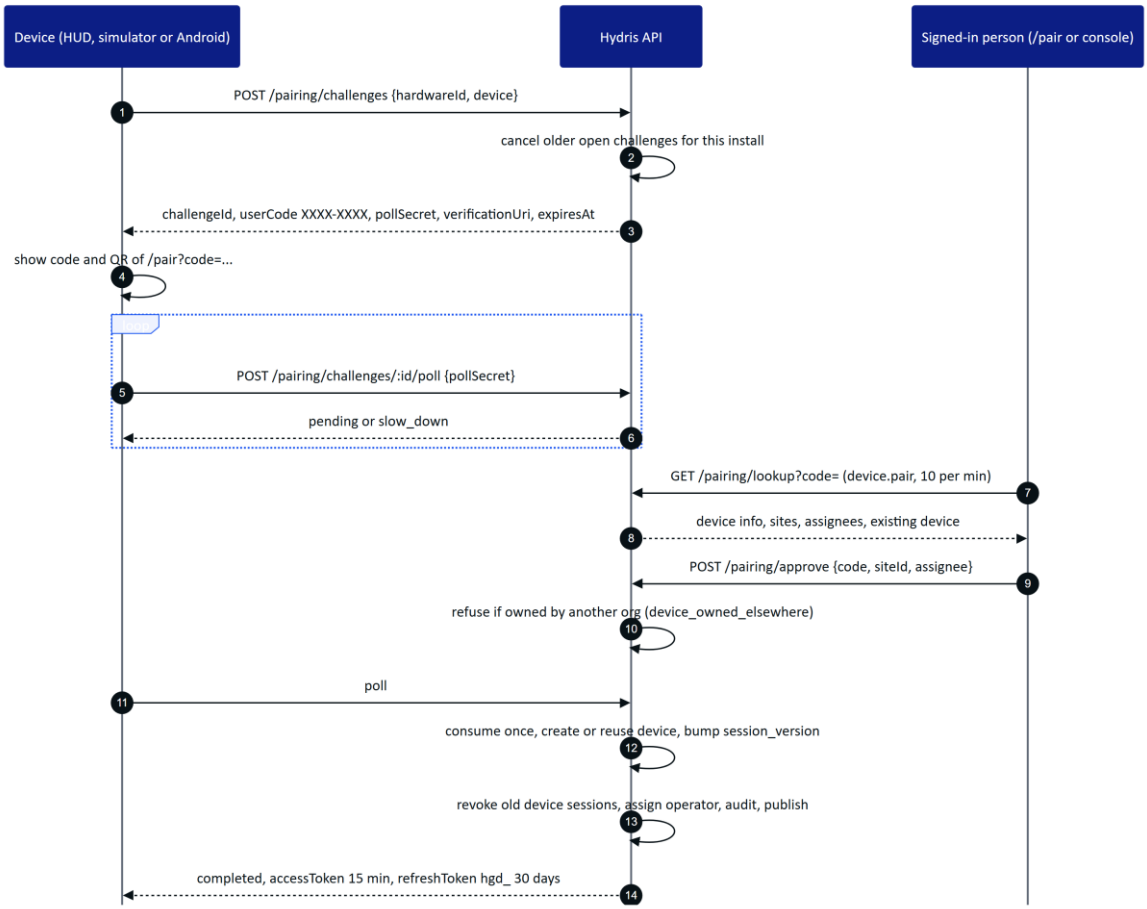

Figure 58. *Device pairing (modules/devices.ts). The poll secret is never displayed; the code is single use and expires after 300 s.*

The Android app uses the same endpoints but approves its own challenge with the signed-in user's session: `Repository.registerGlasses()` creates the challenge with the DAT hardware id, looks the code up, approves it for the first site the user can see, and polls for the tokens. The web HUD, the simulator and the companion bridge use the two-device flow shown above.

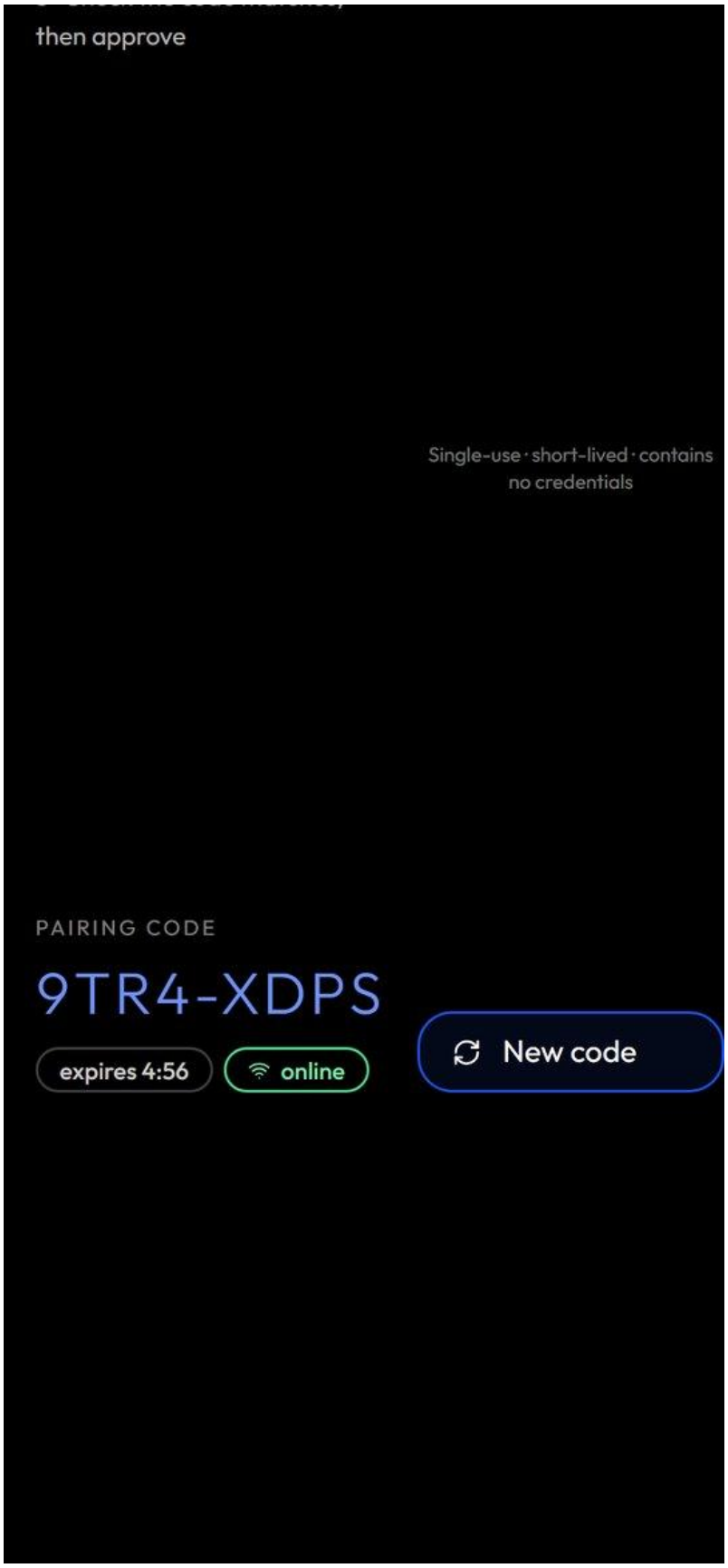

Figure 59. *The pairing page on a phone: the operator enters or scans the short code and picks the site and assignee.*

3.8.2 Device lifecycle

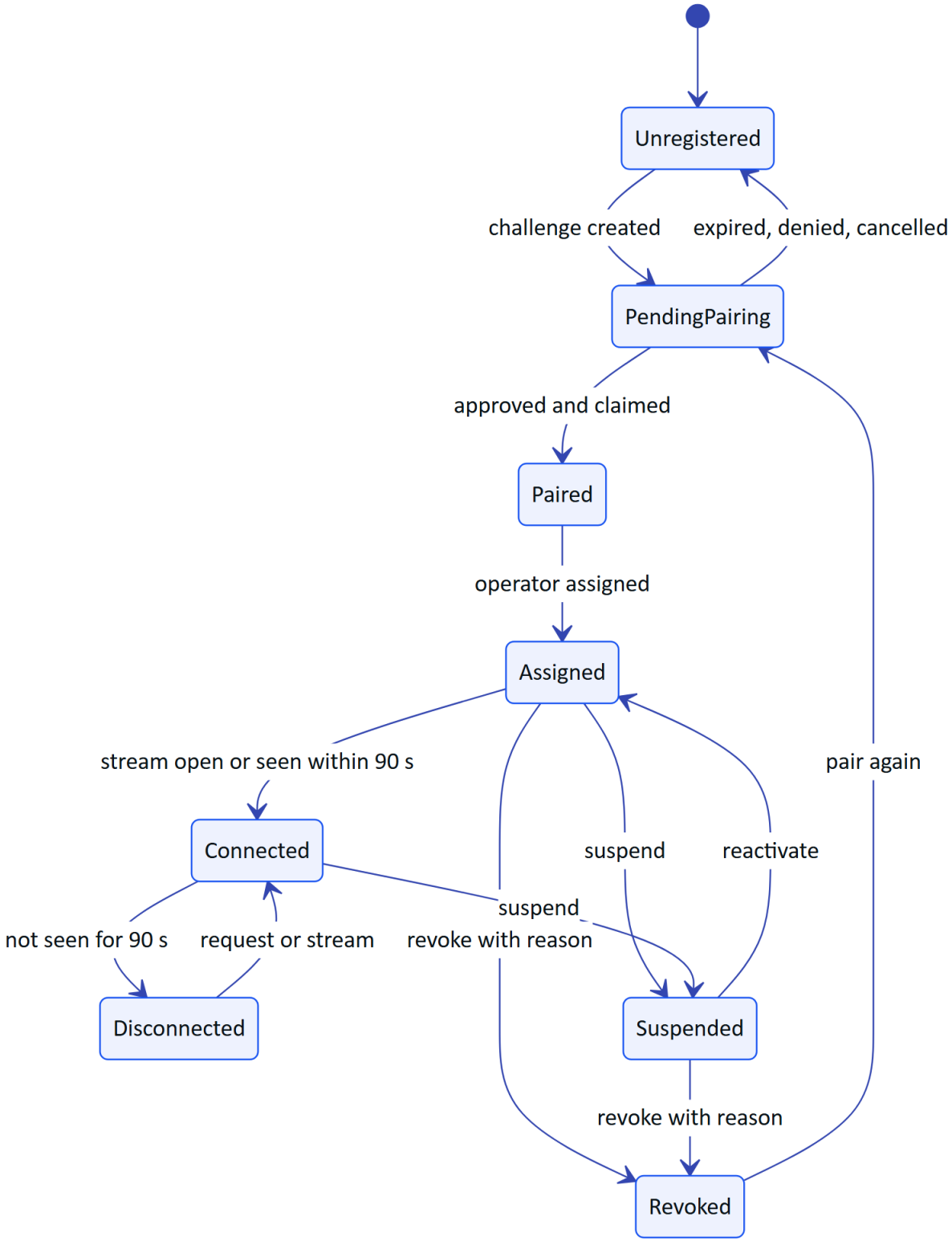

Figure 60. *Device lifecycle as shown to administrators (deriveDeviceState in packages/domain/src/devices.ts). Online means an open stream or seen within 90 s.*

Suspension and revocation bump nothing in the token itself; they change `devices.status`, which `resolveTokenPrincipal()` reads on every request. Revocation also revokes all `device_sessions`, closes the assignment and publishes `device.revoked` to that device only.

3.8.3 Glasses UI profiles

Administrators decide what each role or device sees on glasses without shipping an app. A profile (`ui_profiles`, versioned in `ui_profile_versions`) lists enabled screens, widgets, allowed actions, measurand filters and text scale. Assignments resolve with the precedence device, then user, then role, then organization (`UI_SCOPE_PRECEDENCE` in `packages/domain/src/uiprofile.ts`). The API enforces the effective profile on device requests through the route rule table in `modules/ui-profiles.ts` and caches each device's effective profile for 5 s. A change invalidates the cache and publishes `device.profile_changed` to the affected devices, which reload their context.

3.9 Telemetry and alarms

3.9.1 Ingestion and alarm raising

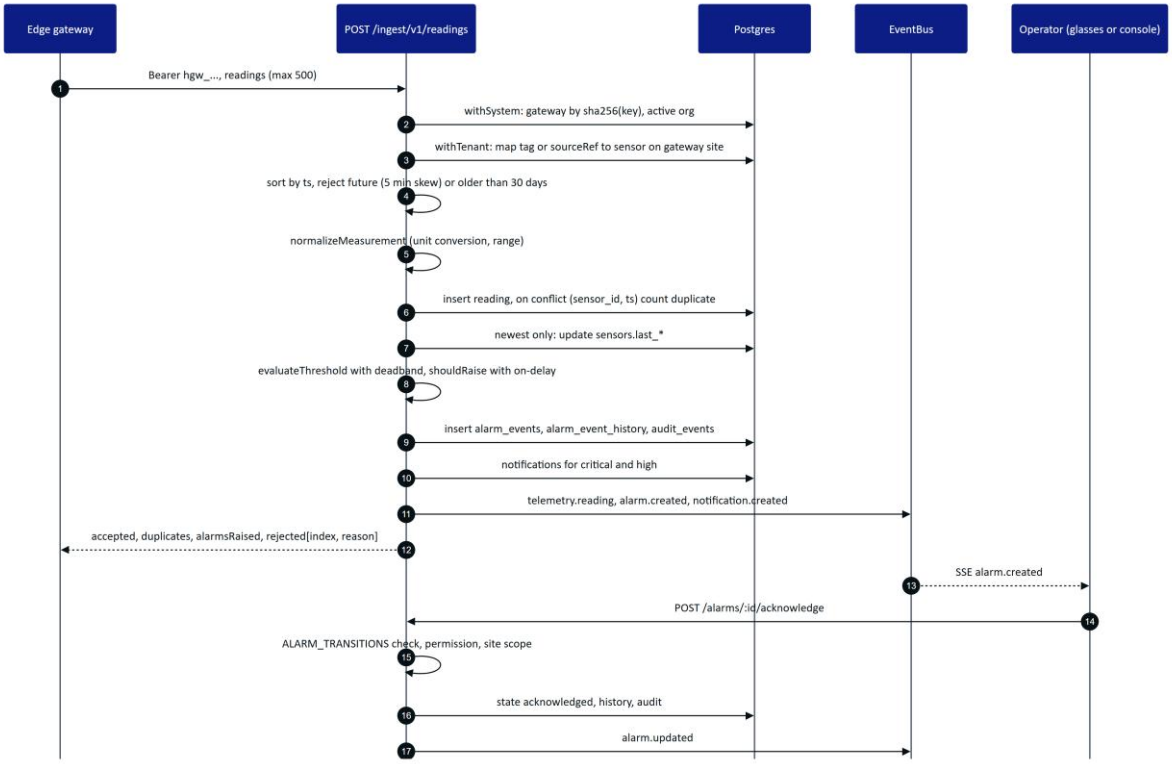

Figure 61. *Telemetry ingestion, alarm raising and acknowledgement. Every source shares ingest() in modules/telemetry.ts.*

Design points worth knowing:

- **Out-of-order data** is stored for history but does not overwrite a newer latest value or re-trigger alarm evaluation.
- **Hysteresis**: once in alarm, a value inside the deadband reports `holding`, not `normal`, so a noisy signal does not flap. A breach must persist for `onDelayS` (tracked in `alarm_rules.breach_since`) before an alarm is raised.
- **One open event per rule**: a continuing breach only updates `peak_value`. When the value returns to normal the event gets `cleared_at` but stays open for people to acknowledge and resolve.
- **Bad quality readings** update the latest value but skip alarm evaluation.
- **Partial success**: the gateway receives per-reading rejections with the original index, so it can retry or drop individual readings.

3.9.2 Stale-data alarms and background jobs

"No fresh data" rules cannot be evaluated on ingestion, because the problem is the absence of readings. `evaluateStaleRules()` scans all enabled stale rules across tenants in system context. `jobs.ts` schedules it and three other jobs.

Table 48. Background jobs

Job	Long-running server (`startJobs`)	Vercel (`/api/cron` via `runScheduledJobs`)	Demo mode
Stale-data rules	Every 30 s	Once per cron run: `vercel.json` schedules `0 3 * * *` (daily)	Not ticked by `DemoTicker`
Pairing expiry	Every 60 s	Once per cron run	Challenges also expire by `expiresAt` checks
Realtime event pruning (older than 1 day)	Hourly in `db` mode	Once per cron run	Per instance memory
Simulation steps	`setInterval` per run, 5 s	Once per cron run	`DemoTicker.poke()` on API activity, at most every 5 s

> **WARNING**
> In the all-Vercel topology the cron runs once a day, so stale-data alarms are raised at most daily. A production plant on Vercel needs a more frequent scheduler (a higher cron tier or an external trigger calling `/api/cron` with `CRON_SECRET`) or the API in a container where `startJobs` runs every 30 s. This is tracked as TD-02.

3.10 Realtime

3.10.1 Server-Sent Events

Clients receive domain events over Server-Sent Events. The connection is made with `fetch()`, not the browser `EventSource`, so the bearer token travels in the `Authorization` header and never in a URL (`apps/web/src/lib/realtime.ts`). The Android app does the same with OkHttp (`HydrisApi.events()`).

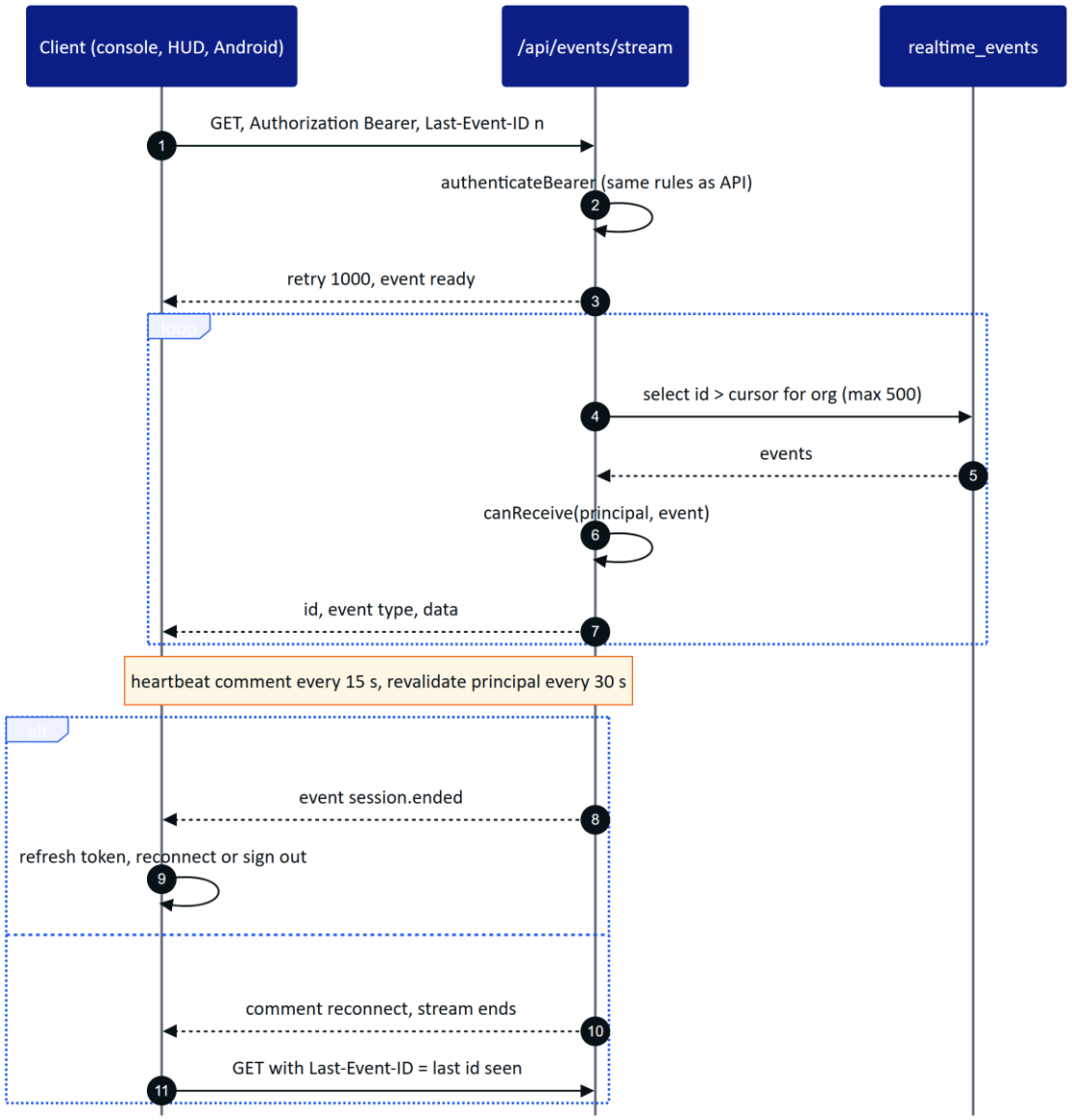

Figure 62. *SSE in db mode on Vercel. Each invocation streams for about 50 s, then the client reconnects with Last-Event-ID and resumes without gaps.*

Table 49. Realtime modes (REALTIME_MODE, core/events.ts)

Aspect	`memory`	`db`
Transport inside the API	In-process `EventEmitter`	Buffered rows in `realtime_events`, flushed in `onSend`
Fan-out across instances	No (single instance)	Yes, every instance polls the same table
Replay on reconnect	Ring buffer of the last 1,000 events	Any event id still in the table (pruned after 1 day)
Latency	Immediate	Up to the 1.5 s poll interval
Device presence	`ctx.liveDevices` counts open streams	`devices.last_seen_at`
Used by	`server.ts` default, Docker Compose, Render	Vercel, demo mode, any multi-instance deployment

Authorisation is per connection and per event (`canReceive` in `modules/realtime.ts`): the event's organization must match, device-targeted events go only to those devices, user-targeted events only to those users, and

site and permission constraints apply. Events carry identifiers and small payloads; clients refetch through the normal API, which applies RLS again.

The console batches invalidations: `Shell.tsx` maps event types to API path prefixes (`EVENT_PATHS`) and invalidates matching TanStack Query entries every 1.5 s, while critical and high `alarm.created` events raise a toast immediately.

Table 50. Event types published today

Area	Events
Telemetry and alarms	`telemetry.reading`, `telemetry.corrected`, `sensor.updated`, `alarm.created`, `alarm.updated`
Work	`inspection.updated`, `maintenance.updated`, `notification.created`
Plant	`asset.updated`, `simulation.step`
Devices	`pairing.completed`, `device.paired`, `device.updated`, `device.context_changed`, `device.profile_changed`, `device.revoked`
Documents and UI	`auditdoc.updated`, `ui_profile.updated`

DECISION

SSE instead of WebSockets. All realtime traffic is server to client; client actions are ordinary authenticated HTTP calls that go through the same validation, RBAC and RLS. SSE works through the Vercel function model (a streamed response with a time budget), through corporate proxies, and with plain `fetch` on every client. WebSockets would need a separate stateful service on Vercel. The price is polling in `db` mode and a reconnect every 50 s, both acceptable at pilot scale.

3.11 Audit document scanning

3.11.1 Pipeline

Paper logbooks, lab sheets and permits become verified digital records through a pipeline that combines on-device OCR, server structuring, an optional vision model and two human steps.

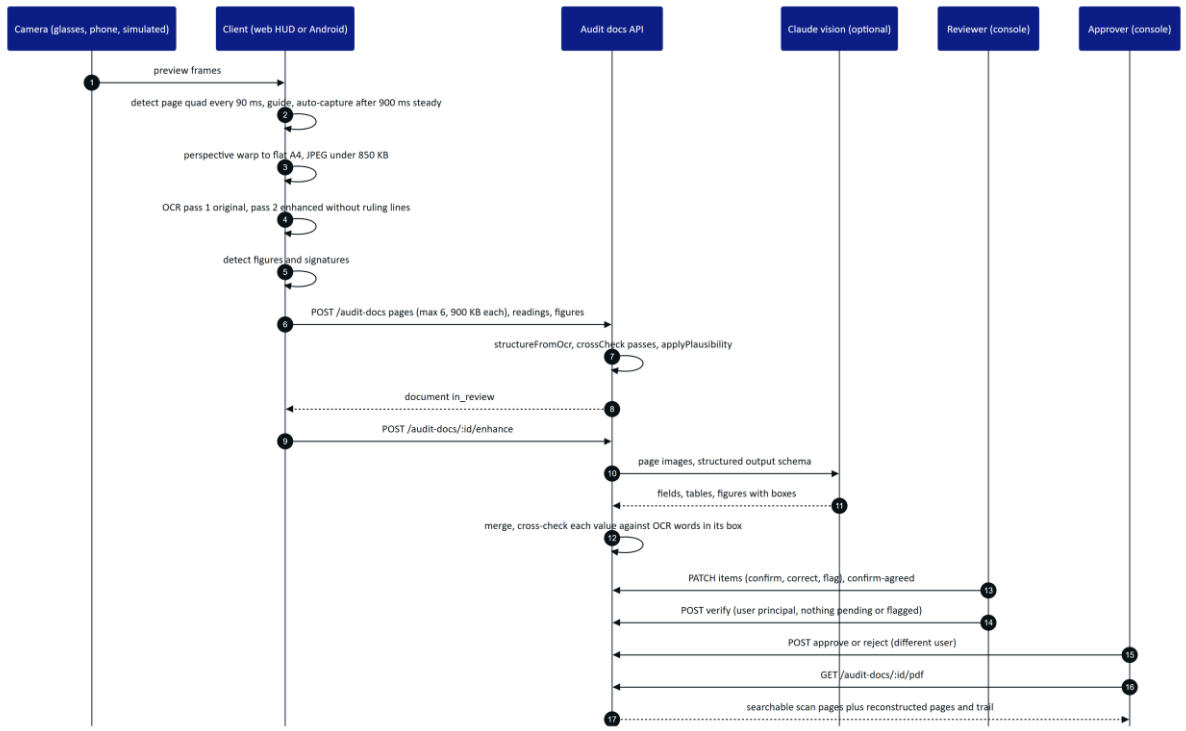

Figure 63. *Audit scan pipeline from camera to approved PDF (lib/docdetect.ts, lib/ocr.ts, modules/audit-docs.ts, core/vision-extract.ts, core/audit-pdf.ts).*

Key properties:

- **Two independent machine readings.** Each page is read twice on the device, once as captured (`ocr:original`) and once contrast-normalised with ruling lines removed (`ocr:enhanced`). The web uses Tesseract.js in WebAssembly (`apps/web/src/lib/ocr.ts`); Android uses ML Kit (`scan/OcrEngine.kt`) and emits exactly the same JSON shape. Disagreement between passes is a signal for the reviewer, which is why the server stores per-field `sources` and an `agreement` value (`match`, `close`, `mismatch`, `single`).
- **No image leaves the device before upload.** Detection, warping and OCR run locally. The web warp produces a 1654 px wide A4 page (`warpToA4` default; the file header comment still says 1240 by 1754).
- **Vision is additive.** If no Anthropic credential is configured, `enhance` returns `aiStatus: unavailable` and the OCR result stands. If the model fails, `aiStatus` becomes `failed` with a short reason and the OCR result stands. Vision cannot replace an extraction once review has started (`409 review_started`).
- **Dual human verification.** Verification and approval require a user principal signed in to the console, and approval must come from a different person than the verifier. Demo tenants may self-approve, and the event records it as such.
- **Optimistic concurrency.** Every write is `UPDATE ... WHERE revision = n`; a lost race returns `409 stale` and the client reloads.

3.11.2 Document states

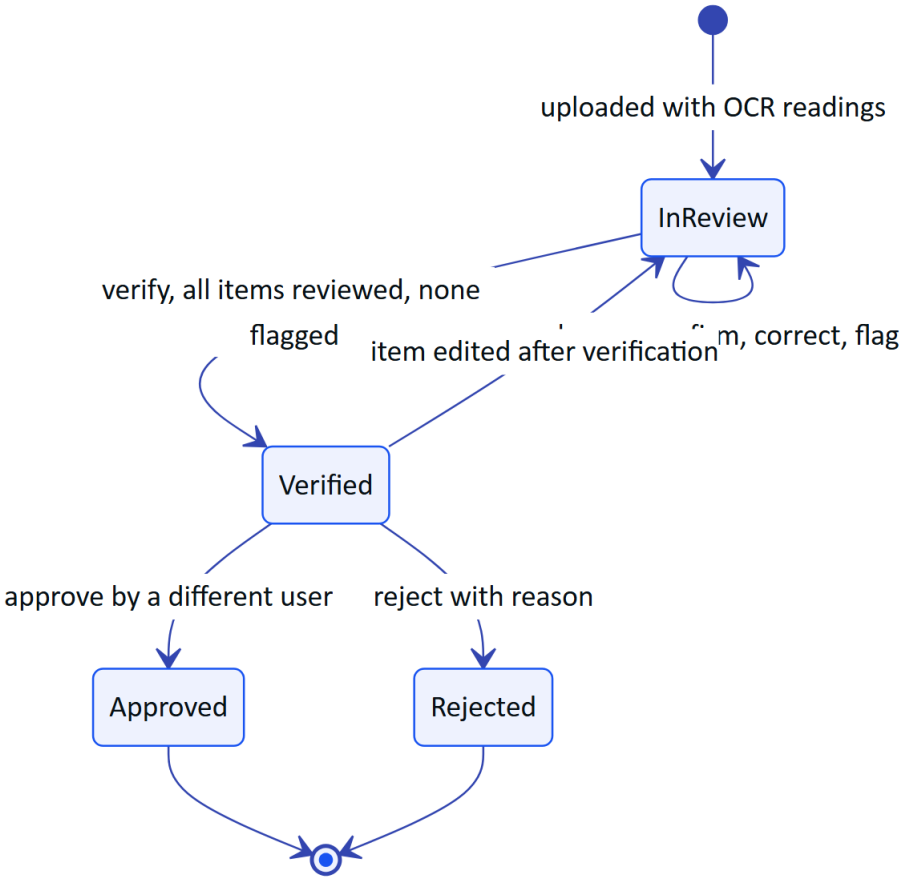

Figure 64. *Audit document states (AuditDocStatus in packages/domain/src/auditdoc.ts). Editing a verified item reopens review.*

The schema also defines a `processing` status for server-side processing; the current upload path writes `in_review` directly because structuring is synchronous.

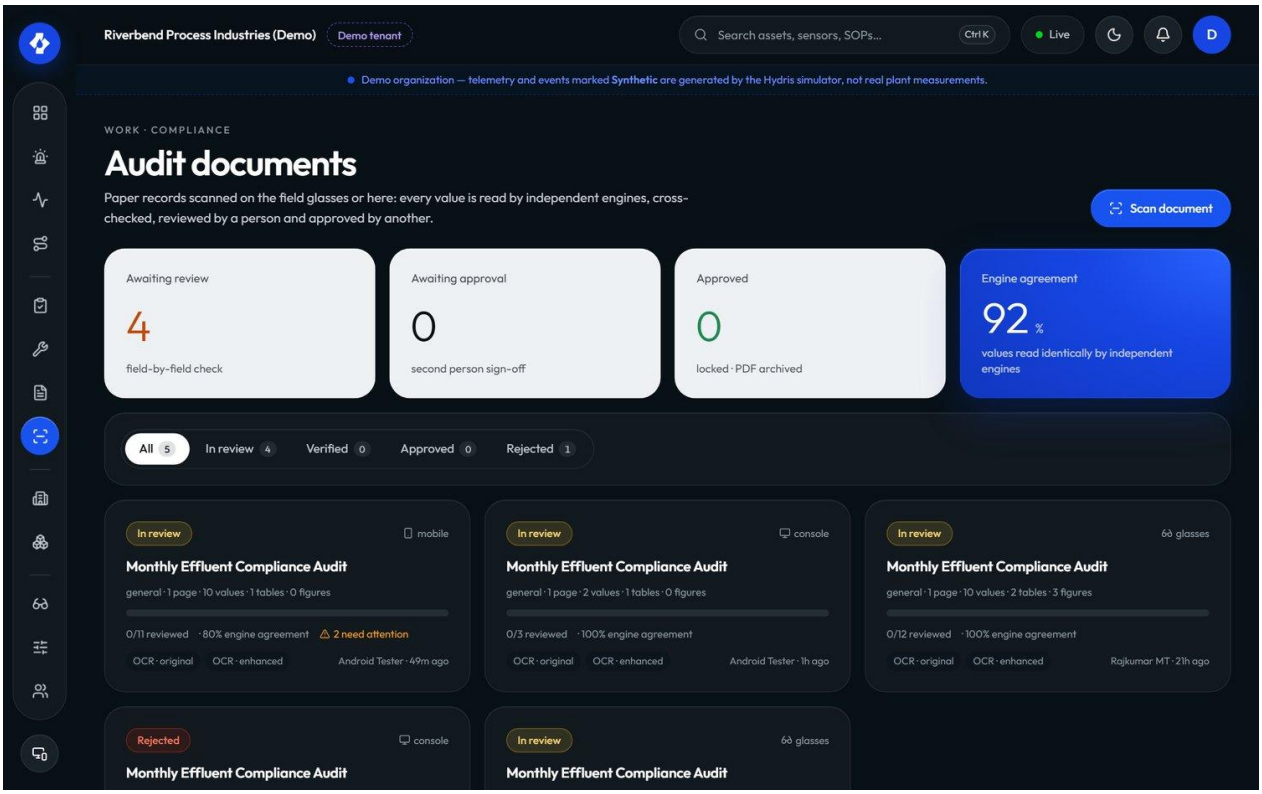

Figure 65. *Audit documents in the console: agreement score, items needing attention and review progress per scan.*

3.12 Voice assistant

"Hey Hydris" answers spoken questions with data from the caller's own account. The wake phrase is matched on the client (`WAKE_RE` in `packages/domain/src/voice.ts`, which tolerates common mis-transcriptions such as "hydras" or "idris"), and the question after the wake phrase goes to `POST /api/assistant/ask`.

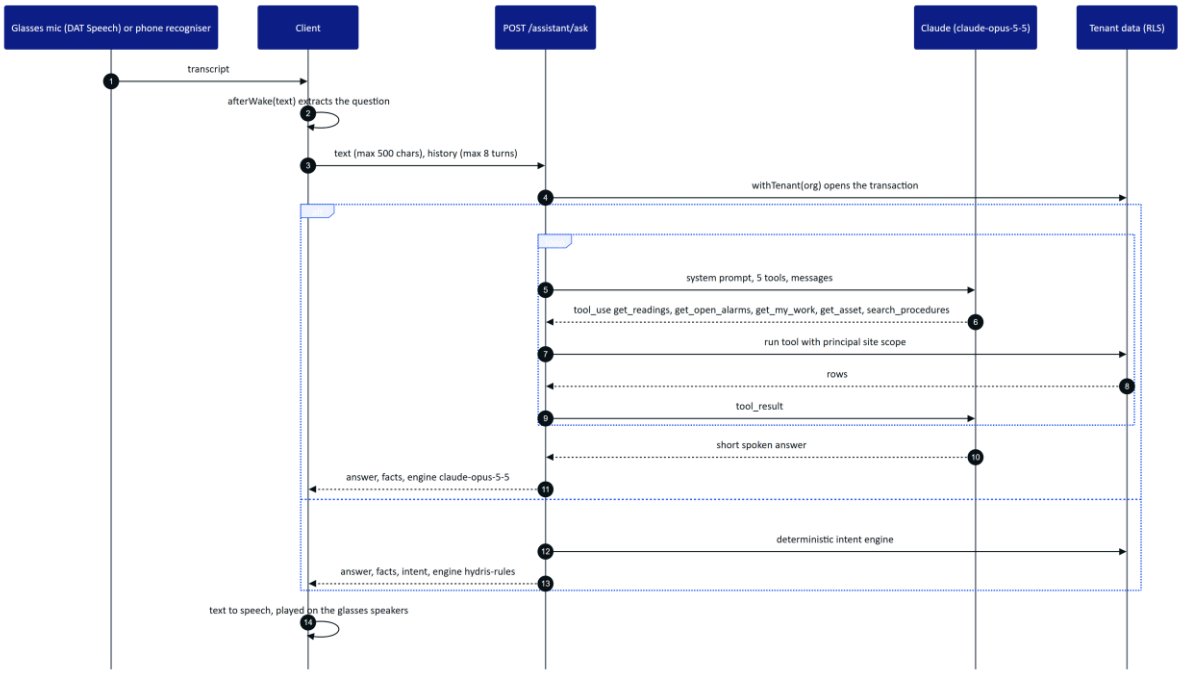

Figure 66. *Voice assistant with tool use (modules/voice-assistant.ts). Tools run inside the caller's tenant transaction, so RLS and site scope apply to every lookup.*

The system prompt forbids estimating or inventing values, asks for the age of each reading, and states that Hydris never operates equipment. The rules engine never guesses: an unrecognised question gets a plain "I can't answer that". The response's `facts[]` list the readings used (label, value, unit, time, and whether they are synthetic), so the client can show what an answer was based on. The endpoint is rate limited to 60 calls per minute.

> **DECISION**
>
> **Grounded tool use with a deterministic fallback.** The model never sees the database; it can only call five read-only tools that execute inside the caller's tenant transaction. The deterministic engine keeps the feature working (with narrower coverage) when no credential is configured, the provider is unavailable, or a customer forbids external AI processing.

3.13 Client applications

3.13.1 Web application

One Next.js application serves four very different experiences from one deployment.

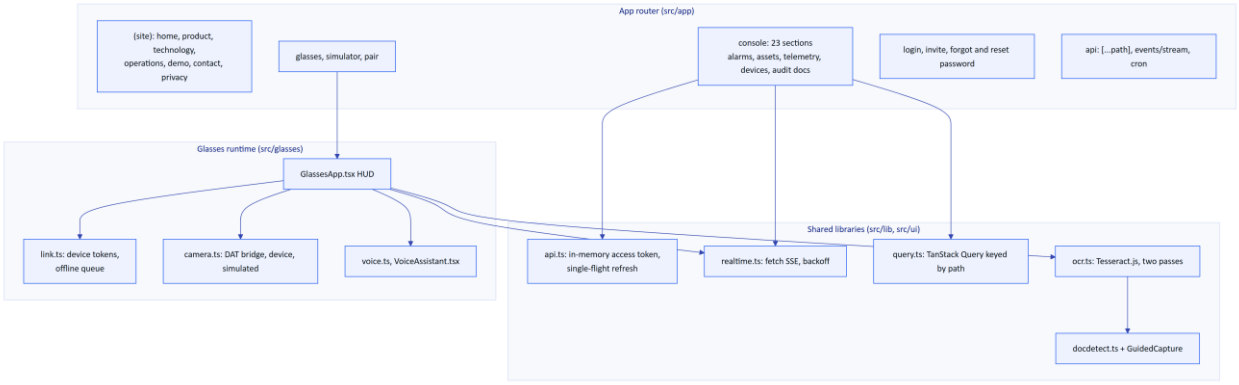

Figure 67. *Web application components (apps/web/src). The glasses HUD and the console share the API client and realtime client but not their session model.*

- **Console** sessions keep the access token in memory and the refresh token in the httpOnly cookie. Data is fetched with TanStack Query, where the API path is the cache key so that realtime events can invalidate by prefix (`lib/query.ts`).
- **Glasses HUD** sessions are device sessions. `link.ts` stores the install id and device refresh token behind a `SecureStore` interface (web implementation: `localStorage`), and keeps an **offline queue** of inspection step responses. Each queued operation carries a client id, and the server deduplicates on `(inspection_id, client_op_id)`, so replays after a network drop are safe.
- **Simulator** (`/simulator`) renders the 600 by 600 additive HUD (`GLASSES_DISPLAY` in `packages/domain/src/devices.ts`) in a browser, with a simulated camera that films a sample A4 sheet so the whole scan pipeline can be demonstrated without hardware.
- **Security headers** come from `next.config.ts`: `X-Frame-Options: DENY` everywhere except `/glasses` and `/simulator` (`SAMEORIGIN`, so the website can embed the live demo), `nosniff`, a strict referrer policy and a permissions policy that allows only the camera for the same origin. Helmet runs in the API with its content security policy disabled.

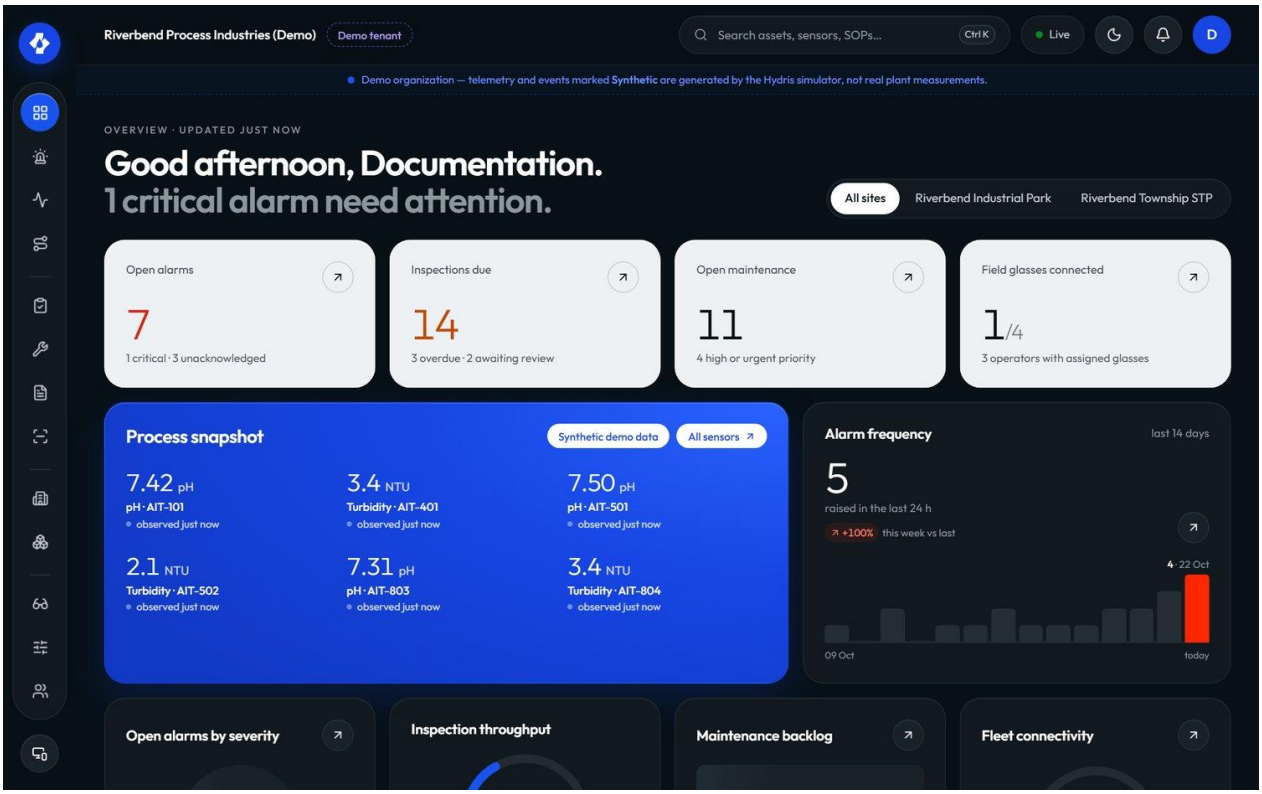

Figure 68. *Operator console overview: KPIs, open alarms by severity and live instruments for the selected site.*

3.13.2 Android app: Hydris Field

Hydris Field (`apps/android`, package `ai.hydris.field`, minSdk 31 to match DAT, targetSdk 36) is the operator's phone app and the bridge to the glasses.

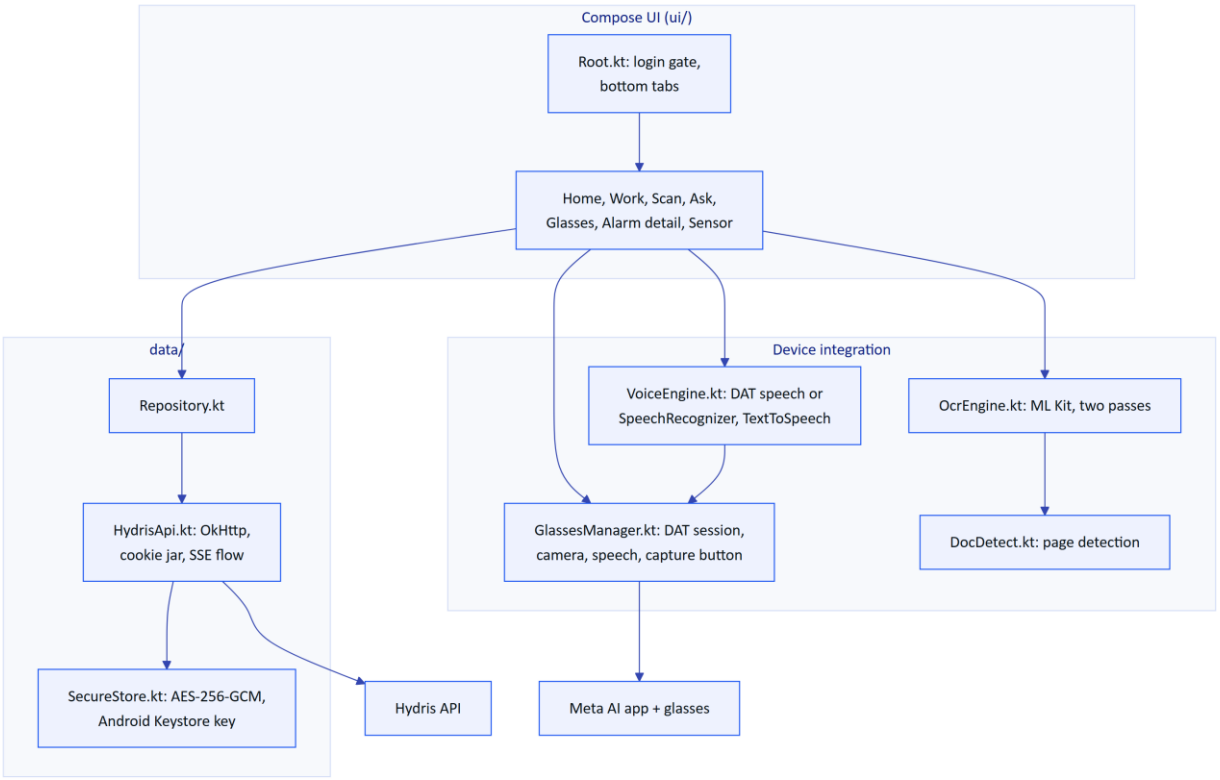

Figure 69. *Hydris Field components. One API client carries two credential channels: the operator session and the paired glasses' device token.*

- **Sessions** follow the web contract: an access token in memory and the rotating `hg_refresh` cookie (plus `hg_demo` in demo mode) persisted only as ciphertext in `SecureStore`. 401 responses refresh once under a `Mutex` and retry.
- **Glasses** are reached through the Meta Wearables Device Access Toolkit 1.0.1: `Wearables.initialize`, registration with the Meta AI app, an auto-selected `DeviceSession`, and the camera (preview stream and `capturePhoto`), speech (`addSpeech`) and inputs (capture button) capabilities. A debug-only mock device kit dependency supports development without hardware.
- **Scans taken with the glasses are attributed to the glasses**: `Repository.uploadScan()` uses the device channel (`deviceCall`) when the glasses are registered, so the console shows `capturedVia: glasses` and the device id.
- **Server URL** defaults to `https://hydrisview.vercel.app` and can be overridden in `local.properties` (`hydris_url`), as can the DAT application id and client token for production builds.

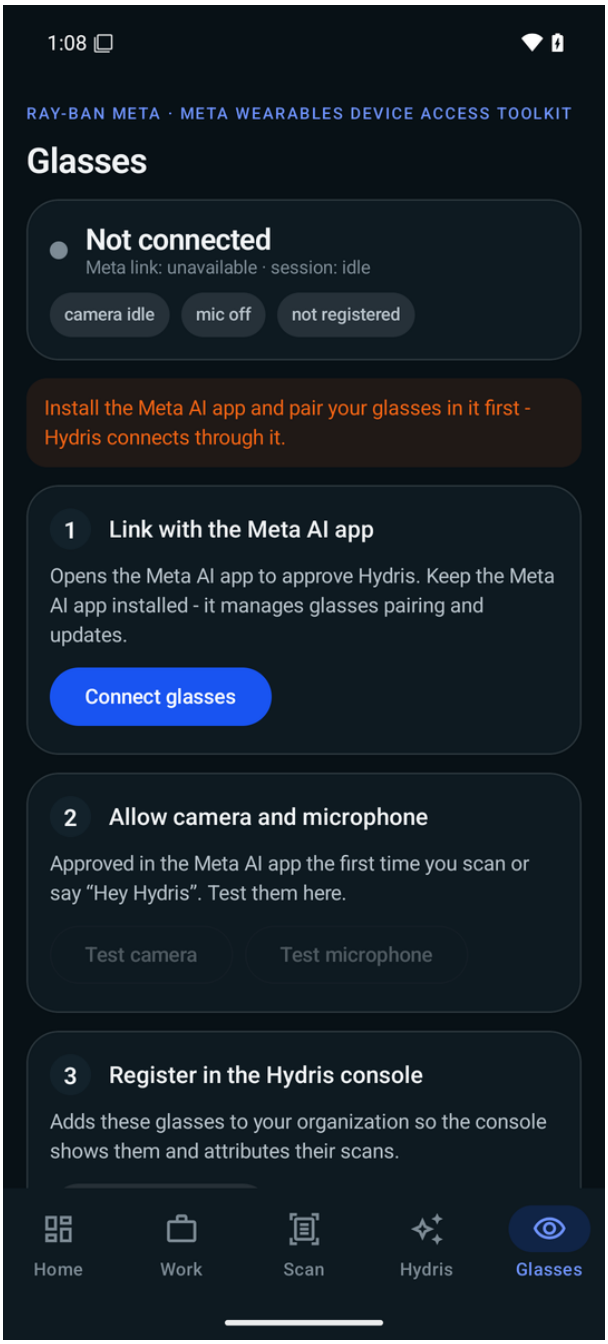

Figure 70. *Hydris Field glasses screen: Meta AI registration, connected glasses and the capture and speech capabilities.*

> **DECISION**
>
> **DAT through a native app, not a WebView bridge.** The first integration (`apps/glasses-bridge-android`) hosted the web HUD in a WebView and exposed DAT through `window.HydrisNative`. It works, but camera frames had to be encoded and passed across the JavaScript bridge, permissions and lifecycle were fragile, and operators need a real phone app anyway. Hydris Field calls DAT directly from Kotlin, runs OCR with ML Kit on the phone, and keeps the web HUD for the simulator, the website demo and future display glasses. The bridge remains in the repository as a fallback.

3.14 Deployment topologies

The same API code runs in four topologies. Configuration is a zod-validated environment (`apps/api/src/config.ts`): `DATABASE_URL` selects Postgres or PGlite, `REALTIME_MODE` selects the event bus, `BLOB_READ_WRITE_TOKEN` selects Vercel Blob over disk, and `API_URL` in the web app selects the external API over the built-in one.

3.14.1 All on Vercel (serverless)

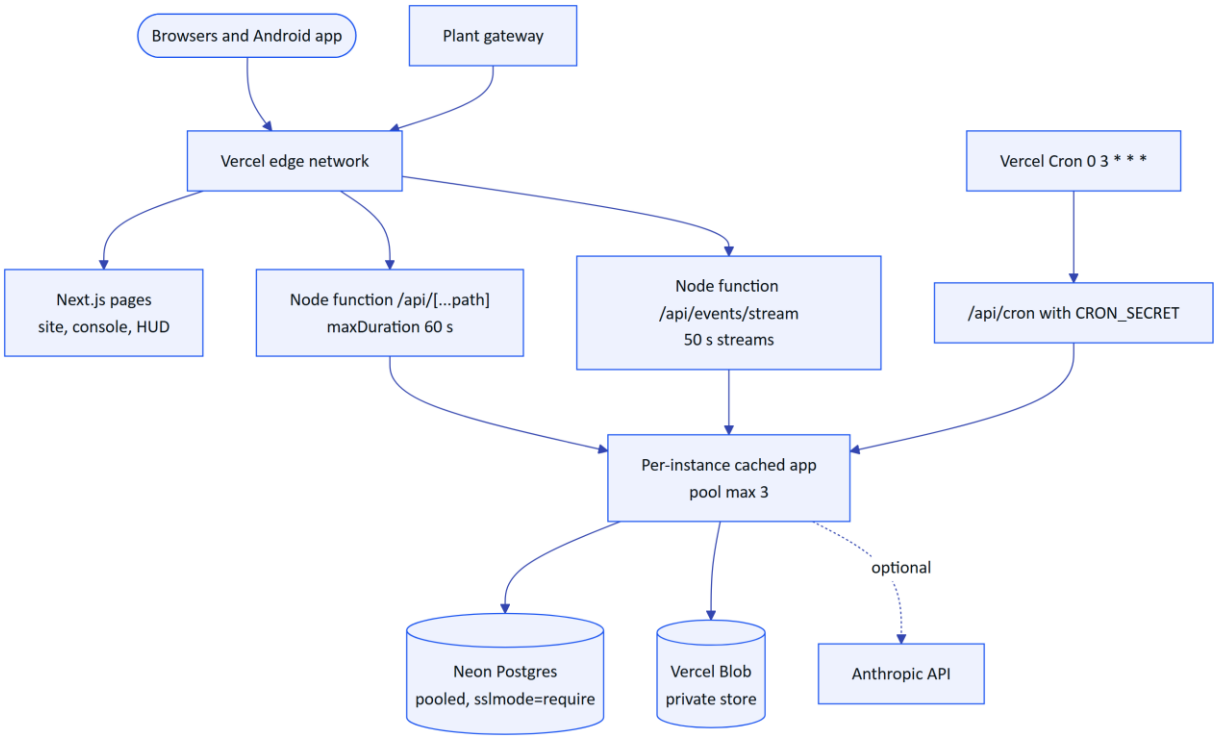

Figure 71. *All-in-Vercel topology (Built, live). Pages and the API run in the same Next.js deployment; each function instance builds the Fastify app once.*

- `getServerlessApi()` (`apps/api/src/serverless.ts`) builds the app on first use, caches the promise for the life of the instance, and clears it on failure so the next request retries.
- Native and WebAssembly packages (`@node-rs/argon2`, `pg`, PGlite, Fastify plugins, `@vercel/blob`) are declared `serverExternalPackages` so they load from `node_modules` instead of being bundled.
- Migrations run on cold start under the advisory lock unless `MIGRATE_ON_START=false`. The root `vercel-build` script would migrate during the build, but `vercel.json` sets `buildCommand` to `npm run build`, so cold-start migration is what runs today.

- `REALTIME_MODE=db` is required; the SSE route refuses to start otherwise.

3.14.2 Self-hosted and plant edge

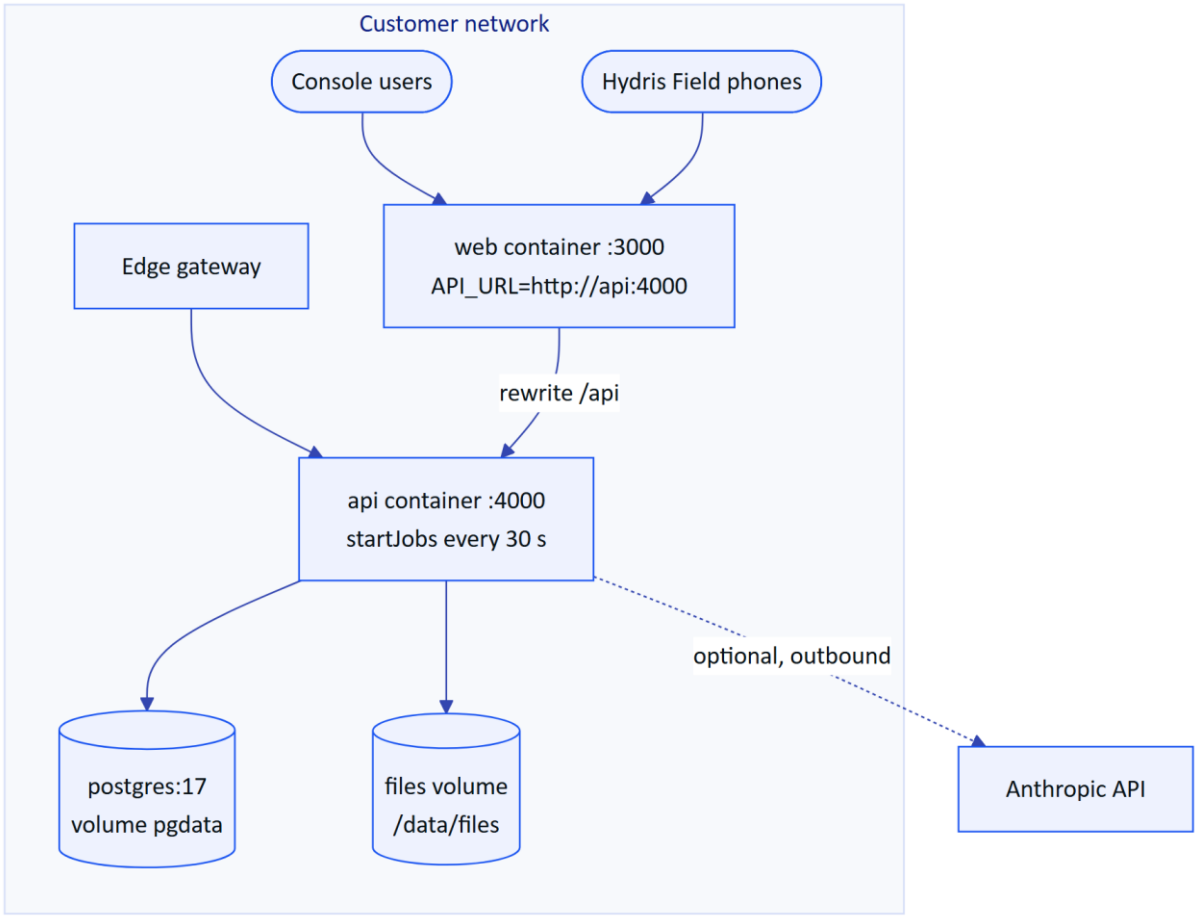

Figure 72. *Self-hosted single host with Docker Compose (Built), suitable for an on-premises or plant-edge server.*

`docker-compose.yml` runs Postgres 17, the API (`apps/api/Dockerfile`) and the web app (`apps/web/Dockerfile`, built with `API_URL=http://api:4000`). The API runs migrations as the owner and then works as `hydris_app` through `SET LOCAL ROLE`, so RLS applies. Memory realtime mode is enough on a single host, and the long-running jobs give 30 s stale-data detection. With no Anthropic key the system is fully functional offline from the internet except for vision enhancement and the language model assistant, which fall back as described.

3.14.3 Hybrid

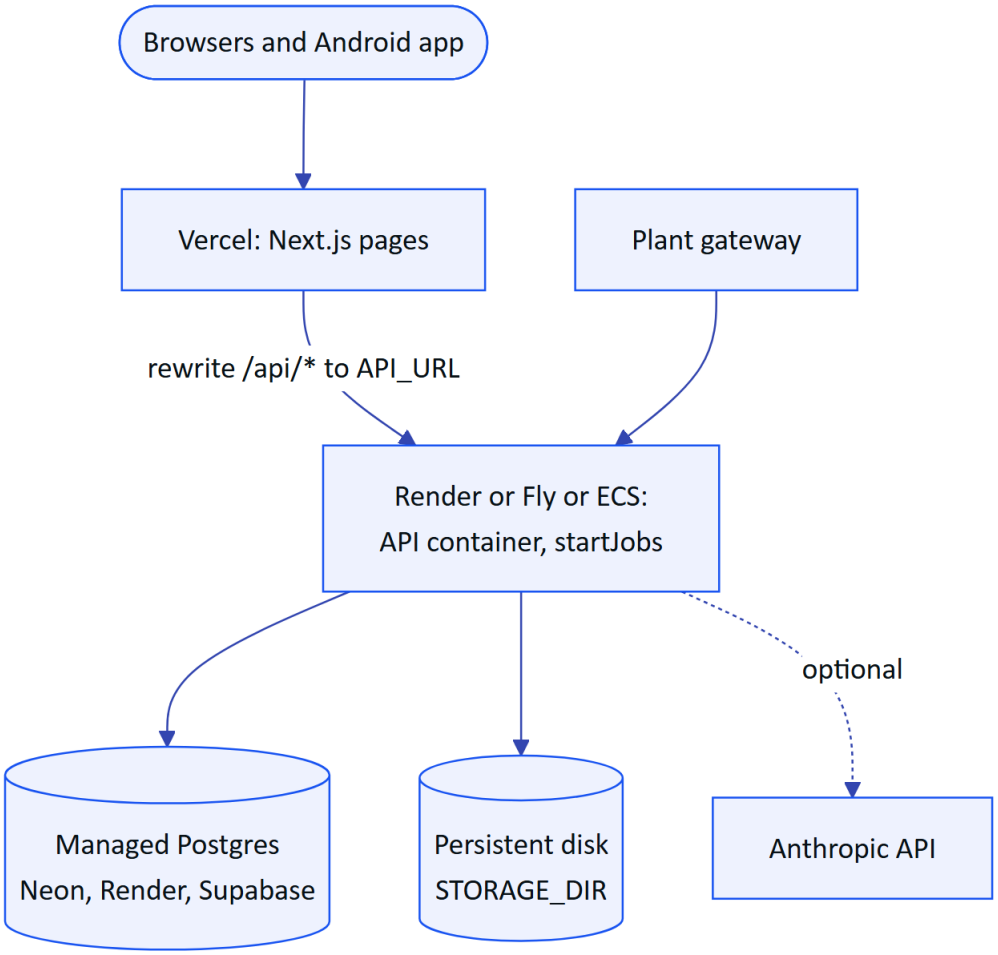

Figure 73. *Hybrid topology (Partial: render.yaml exists, not the live setup). Web on Vercel, API as a long-running container next to managed Postgres.*

When `API_URL` is set, `next.config.ts` adds a `beforeFiles` rewrite that proxies `/api/*` to the external API, and the in-app route handlers are bypassed. `render.yaml` describes the API service (Docker, health check `/api/health`, 5 GB disk at `/data/files`, generated `JWT_SECRET`, simulator off). This topology removes the serverless limits: memory or db realtime, 30 s jobs, no cold-start migrations, long SSE connections.

Table 51. Topology comparison

Concern	All Vercel	Hybrid	Docker Compose	Demo mode
Status	Built, live	Partial	Built	Built, live
Database	Neon Postgres	Managed Postgres	Postgres 17 container	PGlite in memory per instance
Realtime	`db`, 1.5 s poll, 50 s streams	`memory` (one instance) or `db`	`memory`	`db` per instance
Jobs	Cron (daily today)	Timers (30 s)	Timers (30 s)	Lazy ticker on activity
Files	Vercel Blob private	Disk volume	Disk volume	Disk in function temp
Best for	Pilots, demos, low ops	Production cloud	On-premises, air-gapped plants	Sales and evaluation

3.14.4 Demo mode and cross-instance state

Demo mode turns on automatically on Vercel when no `DATABASE_URL` is set (`isDemoEnv()` in `config.ts`). Each serverless instance migrates an in-memory PGlite database and seeds the same deterministic tenant (`seed/showcase.ts`). Because instances share nothing, the state that must survive a hop between instances goes through a small shared key/value store (`core/demo-shared.ts`).

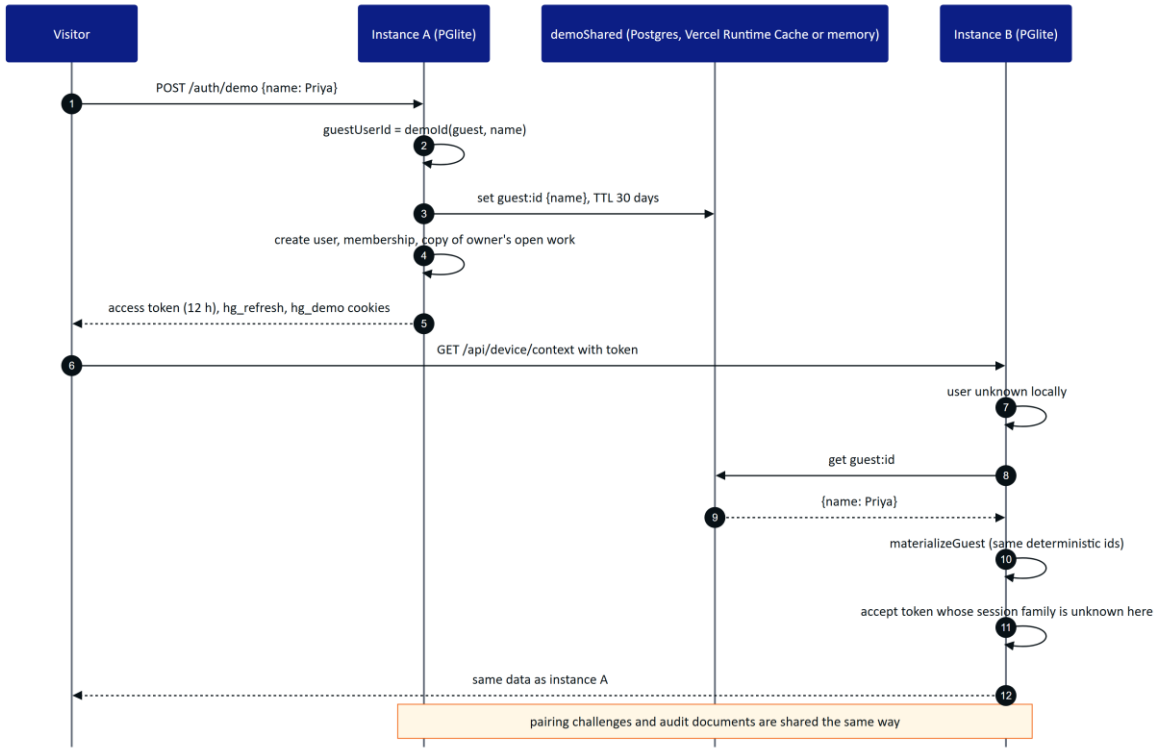

Figure 74. *Demo mode across serverless instances. A guest created on instance A is rebuilt on instance B from the shared store and deterministic ids.*

The shared store prefers Postgres (one table, `hydris_demo_kv`) when a Postgres URL is present, then the Vercel Runtime Cache, then process memory, and a `Resilient` wrapper falls back to memory on any error so the demo never fails because of the store. Device tokens in demo mode carry the operator and site, and demo device refresh tokens are HMAC-signed and self-contained, so any instance can rebuild a paired device. Audit documents are snapshotted into the store after each write and hydrated by revision on read.

> **CAUTION**
> Demo mode relaxes session checks (it accepts a valid token whose session family this instance has never seen) and uses public, fixed secrets. It is selected only when no database is configured or `DEMO_MODE=true`, logs a warning, and must never hold real plant data.

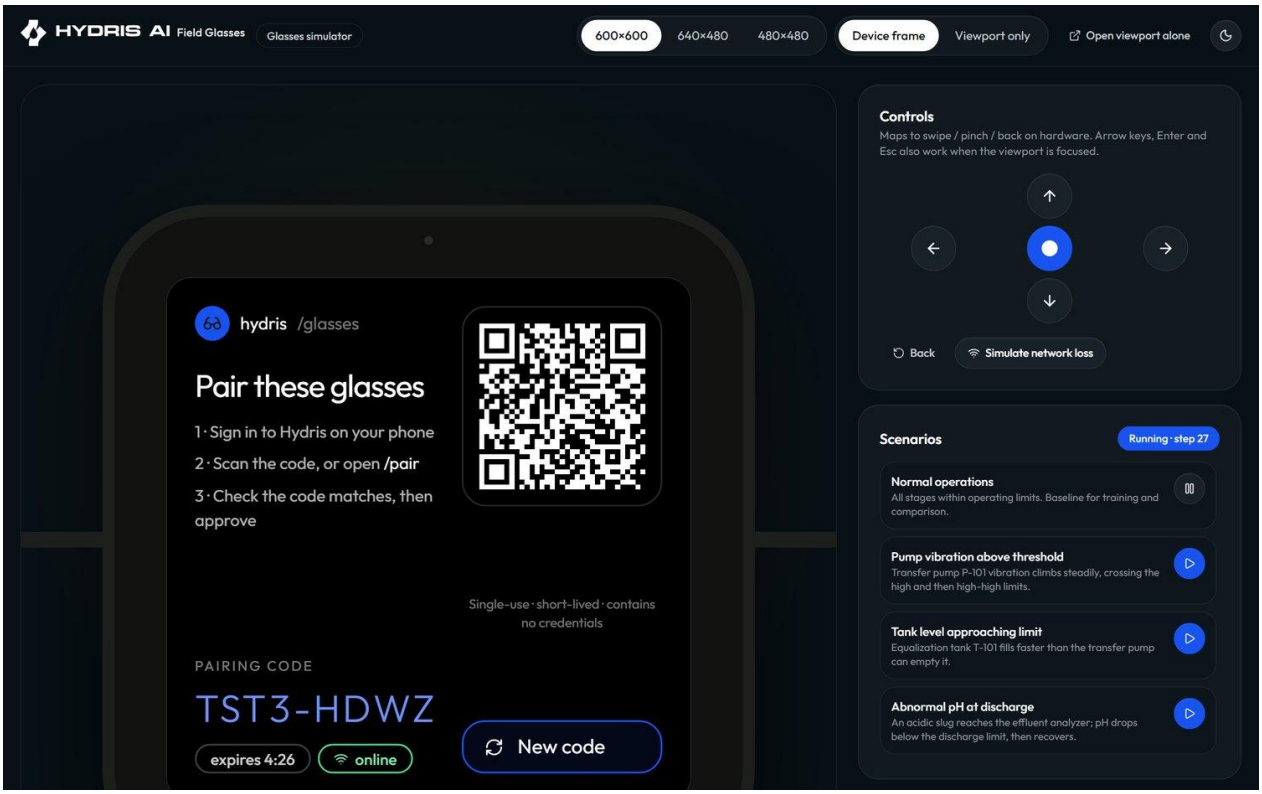

Figure 75. *The browser simulator: the glasses HUD, the simulated camera and scenario controls running against the demo tenant.*

3.15 Cross-cutting concerns

3.15.1 Caching and concurrency

Table 52. Caches

Cache	Scope and lifetime	Invalidation	File
Serverless API instance	Per function instance, until recycled	Cleared if the build fails	`apps/api/src/serverless.ts`
Effective glasses UI profile	Per API instance, 5 s	Cleared on any profile or assignment change on that instance	`modules/ui-profiles.ts`
Console query cache	Per browser tab	Realtime events by path prefix, batched every 1.5 s; mutations	`lib/query.ts`, `console/Shell.tsx`
OCR engine	Per page, worker created once; language data in browser cache	Never	`lib/ocr.ts`
Demo shared store	Postgres table, Vercel Runtime Cache (per region) or memory, with TTLs	TTL	`core/demo-shared.ts`

Concurrency is handled with database constraints and explicit versions rather than locks held across requests:

- **Idempotent writes**: readings are unique on `(sensor_id, ts)`; inspection responses on `(inspection_id, client_op_id)`; pairing code allocation retries up to five times on a unique violation.
- **Optimistic concurrency**: `inspections.revision` and `audit_documents.revision`. Audit document updates include `WHERE revision = n` and return `409 stale` on a lost race.

- **Single-flight**: token refresh on every client; `DemoTicker` allows one step in flight and at most one step per 5 s; `EventBus.flush()` chains writes so batches are inserted in order.
- **Serialized migrations**: Postgres advisory lock.
- **Connection pools**: 3 connections per serverless instance and 10 for a long-running server (`serverless.ts`, `db/client.ts`).

3.15.2 Error handling and resilience

Table 53. Failure modes and responses

Failure	Behaviour	Where
API fails to build on a cold start	503 `service_unavailable` with a short detail; next request retries the build	`app/api/[...path]/route.ts`, `serverless.ts`
Network drop on the glasses	Inspection responses queue locally and replay with client ids	`glasses/link.ts`
SSE disconnect	Reconnect with backoff 0.5 s doubling to a 15 s cap, resume from `Last-Event-ID`	`lib/realtime.ts`
Session revoked during a stream	`session.ended`, client tries one refresh, then signs out	`modules/realtime.ts`
Access token expired	One transparent refresh and retry	`lib/api.ts`, `link.ts`, `HydrisApi.kt`
Vision engine error or missing key	OCR extraction kept, `aiStatus` failed or unavailable with reason	`modules/audit-docs.ts`
Language model error or missing key	Deterministic rules engine answers	`modules/voice-assistant.ts`
One background job fails	Others still run; the cron report lists each job's result	`jobs.ts`
Demo shared store unreachable	Falls back to instance memory	`core/demo-shared.ts`
Event written for a rolled-back request	Harmless refetch hint	`core/events.ts`
Gateway sends bad readings	Per-reading rejection with index and reason; the rest are accepted	`modules/telemetry.ts`

3.15.3 Observability

Observability is basic today. **Built**: Fastify request logging at `info` level on the long-running server (`server.ts`) and opt-in on serverless (`API_LOG=true`); 5xx errors are logged with the stack; `GET /api/health` for load balancers and Render; the cron endpoint returns a JSON report with per-job status and duration; `audit_events` records security-relevant and business actions (logins, device lifecycle, alarm transitions, exports, verifications) with actor, device and IP; Vercel runtime logs capture `console.error` from route handlers and jobs. **Planned**: OpenTelemetry traces across the route handler and Fastify, metrics for ingestion rate, rejection reasons, alarm latency, SSE connections and OCR agreement, error tracking, and alerting on stale gateways.

3.15.4 Security architecture summary

Security is covered in depth in `SECURITY_PRIVACY.md`. Architecturally, the controls are layered:

Table 54. Security layers

Layer	Control	File
Transport and headers	HTTPS, helmet, frame and permissions policies	`app.ts`, `next.config.ts`
Abuse	Global 600 requests per minute plus stricter per-route limits (login 10, refresh 60, pairing 10, assistant 60, ingestion 1,200, inquiries 5 per 10 min)	`modules/*.ts`
Authentication	argon2id, TOTP MFA, lockout, short access tokens, rotating refresh families	`modules/auth.ts`, `core/crypto.ts`
Authorisation	Role permissions, device permission cap, site scope with 404, UI profile hook	`core/principal.ts`, `modules/ui-profiles.ts`
Data isolation	RLS with `FORCE`, non-privileged role, append-only grants	`drizzle/0001_rls.sql` and later
Secrets at rest	MFA secrets AES-256-GCM; token and key hashes only (SHA-256)	`core/crypto.ts`
Files	Random keys, private Blob store, served only through `/api/attachments/:id`	`core/services.ts`
Mobile	Keystore-backed AES-GCM for cookies and device tokens	`SecureStore.kt`

3.16 Architecture decision records

The decisions below are recorded here until they move to individual ADR files. Each lists the alternatives that were considered.

Table 55. Architecture decision records

ID	Decision	Alternatives considered	Consequences	Status
ADR-001	npm workspaces monorepo for API, web, domain and Android	Separate repositories; Nx or Turborepo	Atomic cross-cutting changes; no build cache yet	Accepted
ADR-002	Fastify modular monolith with one module per bounded area	Microservices; Next.js route handlers only	One deployable, in-process calls between modules; modules must stay disciplined	Accepted
ADR-003	Run Fastify inside a Next.js catch-all route with `app.inject`	Separate API deployment only; rewrite all routes to Next handlers	One Vercel project, no network hop, same code for container mode; cold start includes app build and migration	Accepted
ADR-004	Postgres row-level security plus application RBAC and site scope	Application filtering only; schema or database per tenant	Strong isolation with one schema; every request is a transaction	Accepted
ADR-005	PGlite for development, tests and the demo	SQLite; Docker Postgres for tests	Real Postgres semantics (RLS, enums, `date_bin`) with no external service; per-instance memory in demo	Accepted
ADR-006	Drizzle ORM with generated SQL migrations	Prisma; hand-written SQL only	Typed queries, readable SQL migrations, raw SQL where needed	Accepted

ID	Decision	Alternatives considered	Consequences	Status
ADR-007	Server-Sent Events over `fetch` for realtime	WebSockets; polling only; managed pub/sub	Works on serverless and through proxies, token stays out of URLs; polling in `db` mode	Accepted
ADR-008	Two event bus modes, `memory` and `db`	Redis pub/sub; Postgres `LISTEN/NOTIFY`	No extra infrastructure; 1.5 s latency in `db` mode	Accepted, revisit at scale
ADR-009	On-device OCR with two passes, vision model optional	Server-side OCR; vision model only	Works offline and without AI processing; vision improves layout when allowed	Accepted
ADR-010	Meta DAT through the native Hydris Field app	WebView bridge (built first); waiting for display glasses SDK	Reliable camera, speech and button access; Android first, iOS later	Accepted
ADR-011	RFC 8628-style pairing with short codes and poll secrets	QR containing a token; manual device enrolment by admins	No credential on screen or in a QR; works on devices without keyboards	Accepted
ADR-012	Push-only HTTPS ingestion from an edge gateway	Hydris polling OPC UA or Modbus directly	Hydris stays outside control networks; customers or partners run the gateway	Accepted
ADR-013	Short JWT access tokens plus rotating opaque refresh tokens checked on every request	Long-lived JWTs; server sessions only	Revocation is immediate; one database lookup per request	Accepted
ADR-014	Demo mode as in-memory tenants with a small shared store	One shared demo database	Zero configuration, no cleanup; cross-instance logic confined to demo code paths	Accepted
ADR-015	LLM features grounded through tenant-scoped tools, with deterministic fallbacks	Retrieval over exported data; no AI	Answers cite real readings; feature degrades gracefully	Accepted
ADR-016	Pure domain package shared by server and web	Logic duplicated per client	One definition of state machines and rules; Android mirrors by hand for now	Accepted

DECISION

Fastify inside a Next.js route (ADR-003). The built-in API mode lets a pilot run as one Vercel project with one set of environment variables, and the same Fastify application runs unchanged as a container when a customer needs long-lived connections, frequent jobs or on-premises hosting. The trade-offs are explicit: a cold start builds the app and may run migrations, streams must end within the function budget, and in-memory state (rate limits, profile cache) is per instance.

DECISION

PGlite for development, tests and demo (ADR-005). Tests run against real Postgres semantics, including RLS policies and the `hydris_app` role, without Docker. The same property makes the public demo free to operate: each instance seeds its own database in memory. The cost is the cross-instance code in demo mode, which is deliberately isolated in `demo.ts`, `core/demo-auth.ts`, `core/demo-guests.ts` and `core/demo-shared.ts`.

DECISION

On-device OCR first (ADR-009). Paper records often contain personal names and signatures. Reading them on the device means the platform works without sending images to any AI provider, and the two independent passes give the reviewer a machine cross-check for free. A vision model is an opt-in enhancement, and its values are cross-checked against the OCR words inside each returned box rather than trusted on their own.

3.17 Quality attributes

Table 56. Quality attributes, tactics and evidence

Attribute	Scenario	Tactics	Evidence or target
Tenant isolation	A query in a handler forgets the organization filter	RLS `FORCE`, non-privileged role, `SET LOCAL` per transaction	`drizzle/0001_rls.sql`; covered by `apps/api/test/platform.test.ts`
Security	A refresh token is stolen and replayed	Rotation, family revocation, per-request family check	`modules/auth.ts`
Availability	One serverless instance fails to start	Lazy build with retry; 503 with detail; stateless instances	`serverless.ts`
Reliability of records	Network drops during an inspection	Offline queue, client operation ids, unique constraints	`glasses/link.ts`, `modules/inspections.ts`
Data integrity	Two reviewers edit one audit document	Revision check, `409 stale`	`modules/audit-docs.ts`
Performance	An operator opens a 24 h chart	Server-side downsampling with `date_bin` to at most 2,000 points (default 240)	`modules/telemetry.ts`
Responsiveness	A critical alarm is raised	Event published in the same request; console toast on receipt	Target: under 3 s end to end in `db` mode (not yet measured)
Modifiability	A new screen is hidden for a role	Data-driven UI profiles, no app release	`modules/ui-profiles.ts`
Auditability	A regulator asks who changed a value	Append-only audit and corrections, actor and device recorded	`audit_events`, `telemetry_corrections`
Usability hands-free	Operator wearing gloves needs a value	Wake phrase, spoken short answers, capture button	`voice.ts`, `GlassesManager.kt`
Portability	Customer requires on-premises hosting	Same code in Compose; no cloud-only dependency on the critical path	`docker-compose.yml`
Testability	Change to alarm hysteresis	Pure functions in `packages/domain` with unit tests	`packages/domain/test/domain.test.ts`

Automated tests today: four domain test files and six API test files (about 69 test cases by count of `test(` calls), run against PGlite with `npm test`, plus a Playwright journey suite in `tests/e2e/journeys.spec.ts`. There is no CI pipeline yet.

3.18 Scalability path

3.18.1 Capacity model

The numbers below are **planning estimates, not measurements**. They assume one sensor reports every 60 s, a site has about 200 sensors and 10 concurrent console or glasses users, and the gateway batches up to 500 readings per request.

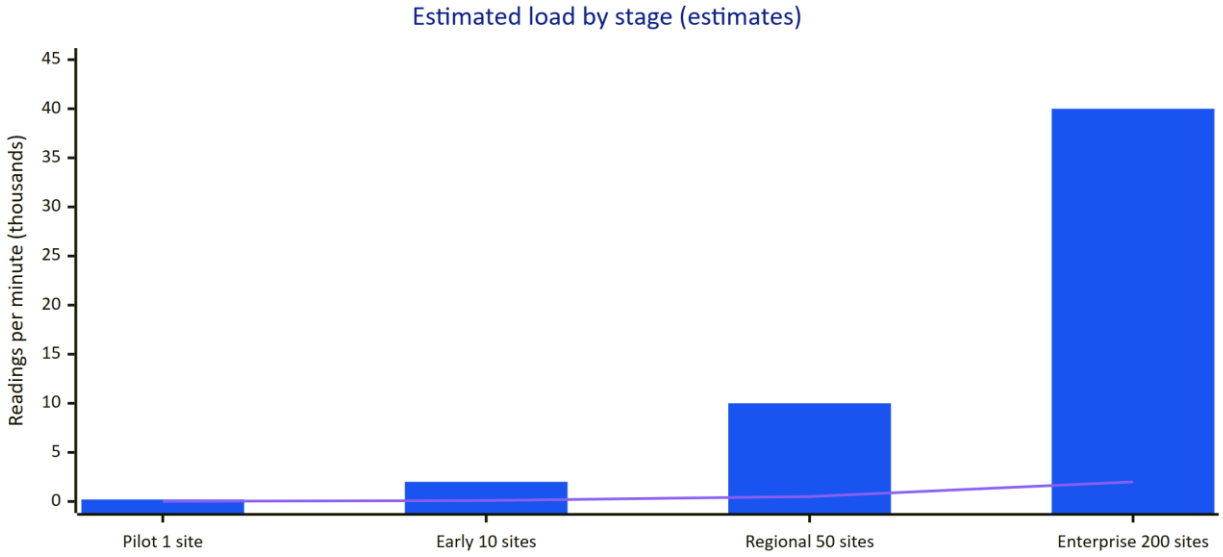

Figure 76. *Estimated telemetry load and open SSE connections by deployment stage (planning estimates, not measured).*

The bars are readings per minute in thousands (200, 2,000, 10,000 and 40,000). The line is open realtime connections in thousands (10, 100, 500 and 2,000). At 200 sensors per site, one site produces about 288,000 readings per day and about 105 million per year.

Table 57. Scaling stages

Stage	Load (estimate)	What holds	What must change
Pilot, 1 to 3 sites	Under 1,000 readings per minute, tens of users	Any topology. All-Vercel is fine if stale-data checks run more often than daily	Fix TD-02 (job frequency) and TD-01 (Android source in git)
Early fleet, about 10 sites	About 2,000 readings per minute, about 100 streams	Hybrid or Compose; Postgres with the current indexes	Shared rate-limit store (TD-03); move audit page images to object storage (TD-04); CI (TD-06)
Regional, about 50 sites	About 10,000 readings per minute, about 500 streams	Hybrid with 2 to 4 API instances in `db` realtime mode	Batch inserts and rule evaluation in `ingest()` (TD-11); time-partitioned telemetry and retention (TD-10); replace polling with `LISTEN/NOTIFY` or a broker (TD-07)
Enterprise, 200 sites and more	About 40,000 readings per minute, about 2,000 streams	Separate ingestion workers from the interactive API	Edge gateway with store and forward, MQTT ingestion, read replicas, regional deployments

The main bottlenecks, in the order they will be hit:

1. **Ingestion is row by row.** `ingest()` inserts each reading and, for the newest reading per sensor, loads the sensor's rules and the open alarm for each rule. This is correct and simple but costs several queries per reading.
2. `db` **realtime polls per connection.** Each open stream runs one query every 1.5 s. 500 streams are about 330 queries per second of polling on top of real work.
3. **Telemetry table growth.** One table with three indexes and no partitioning will need time partitioning and retention well before a billion rows.
4. **Audit page images in Postgres.** Up to 6 pages of up to 900 KB each per document, stored as base64 text, inflate the database and its backups.

3.19 Technical debt register

Table 58. Technical debt register

ID	Item	Impact	Severity	Proposed fix
TD-01	The root `.gitignore` rule `data` also ignores `apps/android/app/src/main/java/ai/hydris/field/data/` (`HydrisApi.kt`, `Models.kt`, `Repository.kt`, `SecureStore.kt`)	A fresh clone of the Android app does not compile	High	Anchor the rule (`/data`, `apps/api/data`) and commit the package
TD-02	Vercel Cron runs `/api/cron` daily (`0 3 * * *`)	Stale-data alarms and pairing expiry run once a day on the all-Vercel topology	High	More frequent schedule or external trigger; or hybrid topology
TD-03	Rate limits use the plugin's in-memory store	Limits are per instance on serverless and multi-instance deployments	Medium	Shared store (Redis or Postgres) or edge firewall rules
TD-04	Audit page images stored as base64 `text` in `audit_document_pages`	Database and backup growth; large rows in API responses	Medium	Store in the existing `Storage` adapter, keep keys in the table
TD-05	`packages/contracts` is empty; Android `Models.kt` mirrors JSON by hand	Contract drift between web, API and Android	Medium	Generate JSON Schema or OpenAPI from zod schemas; generate Kotlin models
TD-06	No CI pipeline	Regressions found late; no automatic migration check	Medium	GitHub Actions: typecheck, unit tests on PGlite, build, e2e on preview
TD-07	`db` realtime mode polls every 1.5 s per stream	Database load grows with connections	Medium at scale	`LISTEN/NOTIFY` fan-out per instance or a managed pub/sub
TD-08	Email transport not wired (`mail_outbox` only)	Invitations and password resets need manual delivery	Medium	Provider adapter with outbox worker and retries
TD-09	Events from rolled-back requests may be written in `db` mode	Spurious refetches	Low	Accepted by design; revisit with transactional outbox

ID	Item	Impact	Severity	Proposed fix
TD-10	`telemetry_readings` has no partitioning or retention	Index and vacuum cost at large volumes	Low now, High later	Monthly partitions or TimescaleDB, retention per tenant contract
TD-11	Ingestion evaluates readings one by one with per-rule queries	Throughput ceiling per instance	Low now, High later	Batch insert, load rules and open alarms once per batch
TD-12	Helmet content security policy disabled in the API; no CSP set by Next.js	Weaker defence against injected scripts	Medium	Nonce-based CSP for pages; strict CSP for API responses
TD-13	Web HUD keeps the device refresh token in `localStorage`	Token exposed to any script on the origin	Medium	Native runtimes implement `SecureStore`; shorten web device sessions
TD-14	UI profile cache invalidation is local to one instance	Other instances may apply an old profile for up to 5 s	Low	Accepted; or invalidate through realtime events
TD-15	Android release build is signed with the debug keystore	Not distributable through managed app stores	Medium before rollout	Release keystore and Play App Signing
TD-16	Comment in `lib/docdetect.ts` says the warp is 1240 by 1754; the default is 1654 px wide	Misleading documentation	Low	Align comment or default
TD-17	`vercel-build` script is unused because `vercel.json` sets `buildCommand`	Migrations happen on cold start instead of at deploy	Low	Choose one: build-time migration with `MIGRATE_ON_START=false`, or remove the script
TD-18	Observability limited to logs and the audit table	Slow diagnosis in production	Medium	OpenTelemetry, metrics, error tracking

RISK

TD-01 and TD-02 should be fixed before any pilot plant goes live: the first blocks anyone else from building the Android app, and the second means a silent gateway would go unnoticed for up to a day on the all-Vercel deployment.

3.20 Future architecture

3.20.1 Target picture

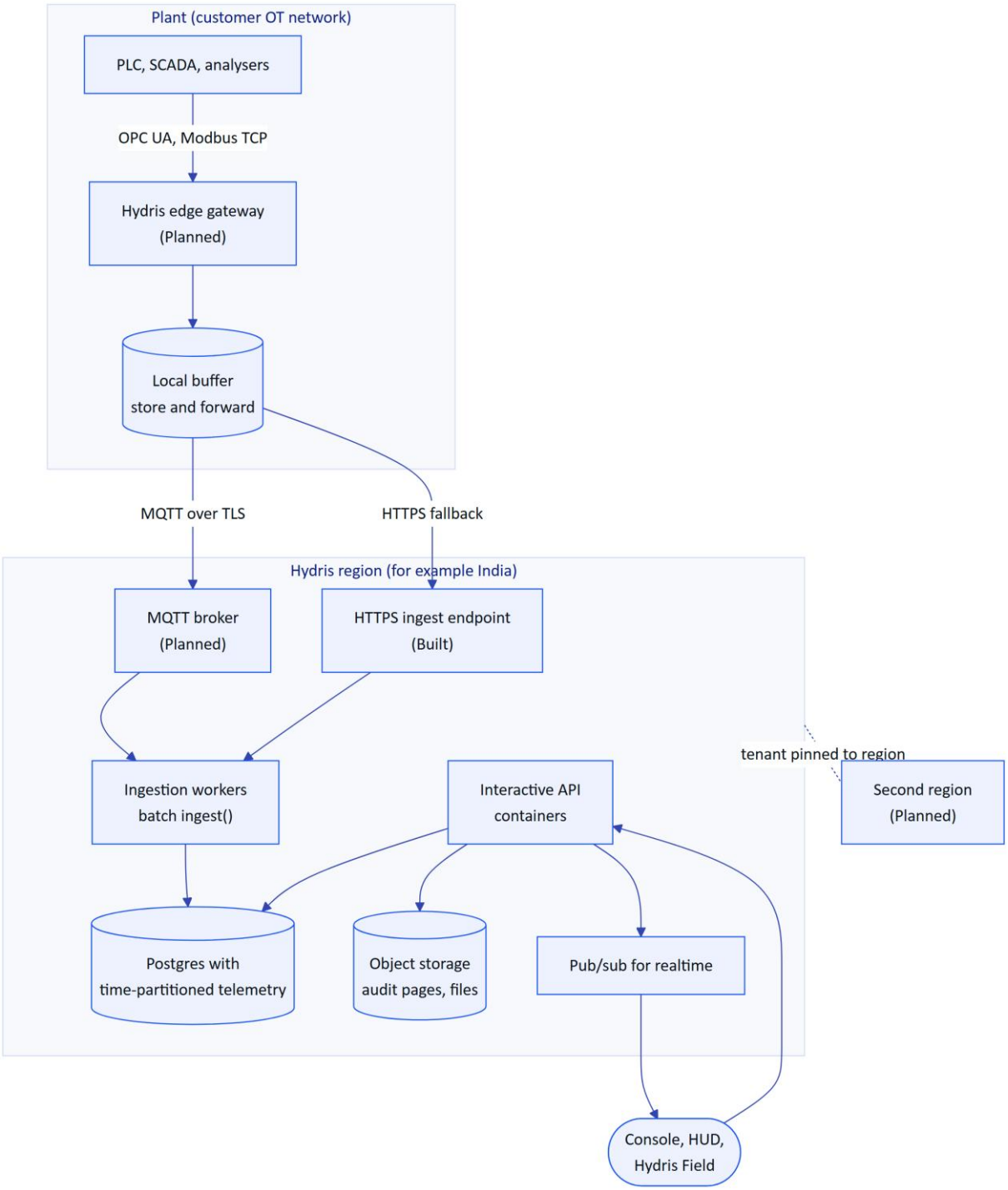

Figure 77. *Target architecture (Planned). Edge collection with store and forward, a broker for high-volume ingestion, and regional deployments for data residency.*

3.20.2 Planned building blocks

Table 59. Future architecture items

Item	Purpose	Builds on	Status
Hydris edge gateway	Read OPC UA and Modbus from the plant side, map tags to `sourceRef`, buffer during outages, push outbound only	`POST /api/ingest/v1/readings`, `gateways.key_hash`, `sensors.source_ref`	Planned
MQTT ingestion	Efficient high-frequency transport from many gateways	Same `ingest()` path behind workers	Planned
Batch ingestion workers	Separate write-heavy ingestion from the interactive API	TD-11	Planned
Telemetry partitioning and retention	Keep queries fast and storage bounded	TD-10	Planned
Realtime pub/sub	Remove polling, sub-second delivery across instances	`EventBus` interface already isolates the transport	Planned
Object storage for audit pages	Smaller database, cheaper backups	`Storage` adapter	Planned
Contracts package	One schema for API, web and Android	`packages/contracts`	Planned
Multi-region	Data residency (for example India for CPCB-regulated plants), lower latency	Stateless API, tenant pinned to one region's database	Planned
iOS app	Operators on iPhone	DAT iOS SDK, same API	Planned
Display glasses rendering	Native HUD on Meta Ray-Ban Display	Web HUD layouts, `GLASSES_DISPLAY`, UI profiles	Planned
Observability stack	Traces, metrics and alerts	TD-18	Planned

The current design keeps these doors open on purpose. Ingestion has one entry point and an idempotency key, so a broker and workers can be added behind it. The event bus is an interface with two implementations, so a third (pub/sub) is a contained change. Storage is an adapter. Tenancy is enforced in the database, so splitting tenants across regions does not change application code. The API is stateless apart from caches that are already safe to lose.

4 Data Model

> **NOTE**
>
> **HFG-DAT v2.0** · Draft for review · Owner: Engineering, Hydris. Every table, column, state, JSON shape and tenancy rule behind Hydris Field Glasses, and how the data should evolve

This document describes the persistent data of Hydris Field Glasses: what is stored, where, in which shape, who may read it, how long it lives and how it is expected to grow. It is written for engineers who change the schema, for data and analytics people who query it, and for security reviewers who need to know which tables hold personal or sensitive data.

Everything about the current product is taken from the code on `main`:

- Schema: `apps/api/src/db/schema.ts` (Drizzle ORM, one file, 42 tables).
- Migrations: `apps/api/drizzle/0000_init.sql` to `0008_audit_documents_rls.sql`, with snapshots in `apps/api/drizzle/meta/`.
- Tenancy helpers: `apps/api/src/db/tenant.ts` and `apps/api/src/context.ts`.
- Shared types and enumerations: `packages/domain/src/*` (`catalog.ts`, `alarms.ts`, `workflows.ts`, `devices.ts`, `measurands.ts`, `uiprofile.ts`, `auditdoc.ts`, `roles.ts`).
- Seed and demo data: `apps/api/src/seed/demo.ts`, `apps/api/src/seed/showcase.ts`, `apps/api/src/core/demo-shared.ts`.
- Client-side storage: `apps/android/app/src/main/java/ai/hydris/field/data/SecureStore.kt` and `apps/web/src/glasses/link.ts`.

Statements are marked **Built** (on `main`), **Partial** or **Planned**. Volume and storage figures are estimates and are labelled as such.

42
Tables in the Drizzle schema
plus 1 runtime table in demo mode

548
Columns
from the 0008 snapshot

23
Postgres enum types
22 in 0000, 1 in 0007

9
SQL migrations
0000 to 0008

71
Indexes
23 unique, 4 of them partial

130
Foreign keys
cascade from organizations

36
Tables with row-level security
6 system-only tables without

23
JSONB columns
typed in TypeScript, not in SQL

4.1 Data principles

The schema follows a small set of rules. New tables and columns are expected to follow them too; a review that breaks one should say why.

Table 60. Data principles and where the code enforces them

#	Principle	How it shows up in the code
P1	**Every tenant row carries `org_id`.** Tenancy is a column, not a database or schema per customer.	`orgRef()` helper in `schema.ts` adds `org_id uuid not null references organizations(id) on delete cascade` to every tenant table.
P2	**Isolation is enforced by the database, not only by queries.** A missing `WHERE org_id = ...` must not leak data.	Row-level security policy `tenant_isolation` on 35 tables; requests run as the non-privileged role `hydris_app` (`0001_rls.sql`, `tenant.ts`).
P3	**Provenance is explicit.** Every measurement says where it came from and whether it is synthetic.	`telemetry_readings.source`, `quality`, `synthetic` plus CHECK `telemetry_synthetic_source_ck`; `synthetic` flags on alarms, inspections and maintenance tasks.
P4	**History is appended, never overwritten.** Corrections, reviews and decisions add rows.	`telemetry_corrections`, `alarm_event_history`, `maintenance_activity`, `audit_document_events`, `audit_events` (UPDATE and DELETE revoked on three of them).
P5	**Missing data is not zero.** A value without a timestamp is meaningless.	`sensors.last_value` and `last_ts` are nullable; `freshness()` in `packages/domain/src/freshness.ts` returns `no_data` instead of 0.
P6	**One canonical unit per measurand.** Conversion happens at ingestion.	`normalizeMeasurement()` in `measurands.ts`; `sensors.unit` stores the canonical unit.
P7	**Snapshots protect history from later edits.**	`inspections.steps` copies the template steps at assignment time; `AuditField.extracted` keeps the first extracted value.
P8	**Offline clients replay safely.**	Unique `(inspection_id, client_op_id)` on `inspection_responses`; unique `(sensor_id, ts)` on `telemetry_readings`; `revision` counters for optimistic locking.
P9	**Secrets are stored as hashes or ciphertext.**	`password_hash` (argon2id), `token_hash`, `key_hash`, `code_hash`, `poll_secret_hash` (SHA-256), `mfa_secret_enc` (AES-256-GCM with `DATA_KEY`).
P10	**Times are UTC `timestamptz`.** Site and organization time zones are display settings.	`ts()` helper uses `timestamp with time zone`; `sites.timezone` and `organizations.timezone` default to `UTC`.

DECISION

Shared-schema multi-tenancy with RLS was chosen over a database per tenant. It keeps one migration path, one connection pool and cheap tenant creation, and moves the isolation guarantee into Postgres. The cost is that every new tenant table must be added to an RLS migration; section "Tenancy and row-level security" lists the checklist.

4.2 Domain overview

The 42 tables fall into ten domains. Each domain is owned by one API module in `apps/api/src/modules/`.

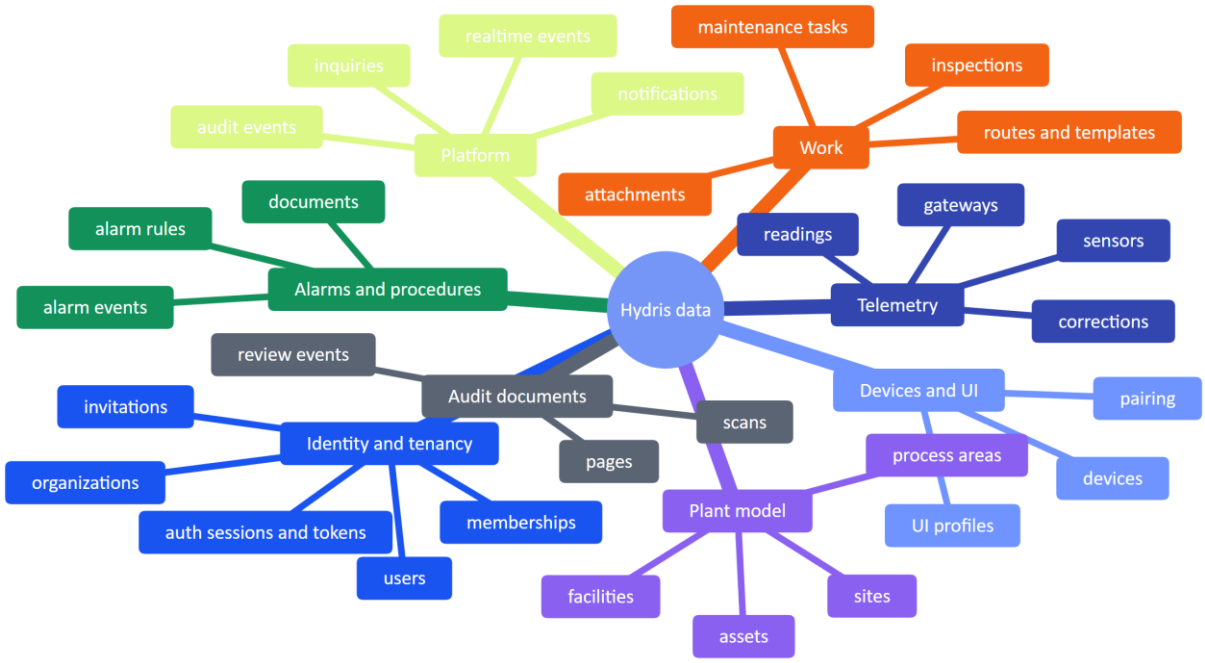

Figure 78. *Data domains of Hydris Field Glasses and their main tables*

Table 61. Domains, tables and owning modules

Domain	Tables	Owning module(s)	Status
Identity and tenancy	`users`, `organizations`, `memberships`, `membership_sites`, `invitations`, `auth_sessions`, `auth_tokens`, `mail_outbox`	`auth.ts`, `org.ts`	**Built**
Plant model	`sites`, `facilities`, `process_areas`, `assets`	`plant.ts`	**Built**
Telemetry	`gateways`, `sensors`, `telemetry_readings`, `telemetry_corrections`	`telemetry.ts`, `plant.ts`	**Built**
Alarms and procedures	`alarm_rules`, `alarm_events`, `alarm_event_history`, `documents`	`alarms.ts`, `plant.ts`, `insights.ts`	**Built**
Work	`inspection_templates`, `inspection_routes`, `inspections`, `inspection_responses`, `maintenance_tasks`, `maintenance_activity`, `attachments`	`inspections.ts`, `maintenance.ts`, `plant.ts`	**Built**
Devices and UI profiles	`devices`, `device_assignments`, `pairing_challenges`, `device_sessions`, `ui_profiles`, `ui_profile_versions`, `ui_profile_assignments`	`devices.ts`, `ui-profiles.ts`	**Built**
Audit documents	`audit_documents`, `audit_document_pages`, `audit_document_events`	`audit-docs.ts`	**Built**
Platform	`audit_events`, `notifications`, `realtime_events`, `inquiries`, `simulation_runs`	`core/services.ts`, `core/events.ts`, `public.ts`, `simulation.ts`	**Built**
Demo key/value	`hydris_demo_kv` (created at runtime, not in migrations)	`core/demo-shared.ts`	**Built** (demo mode only)

4.2.1 Conventions

- **Primary keys** are `uuid` with `DEFAULT gen_random_uuid()` (helper `pk()`), except the two high-volume append logs `telemetry_readings` and `realtime_events`, which use `bigint GENERATED ALWAYS AS IDENTITY`, and `membership_sites`, which has a composite key.
- **Timestamps**: `created_at` and `updated_at` are `timestamptz not null default now()`. `updated_at` is set by the application on every update (there are no triggers).
- **Soft delete**: plant objects and UI profiles use `archived_at`. Hard deletes are rare and guarded (an asset with sensors, alarms, inspections or tasks returns `in_use`; an alarm rule with events is disabled instead of deleted).
- **Text-typed states**: some small state sets are `text` columns typed with `$type<...>()` in TypeScript rather than Postgres enums (for example `users.status`, `devices.kind`, `audit_documents.ai_status`). Section "Enumerations" lists both kinds.
- **Naming**: tables are plural snake case; foreign keys are `<entity>_id`; actor columns are `<verb>_by` (user) and `<verb>_at` (time).

4.3 Entity relationship model

The full model is too large for one readable diagram, so it is split by domain. Shared entities (`organizations`, `users`, `sites`, `assets`, `devices`) appear as stubs with only their key where they are referenced from another domain. Every tenant table also has `org_id` referencing `organizations.id` with `ON DELETE CASCADE`; that edge is drawn only in the identity diagram to keep the others legible. Column types use Postgres names (`timestamptz`, `float8` for `double precision`).

4.3.1 Identity and tenancy

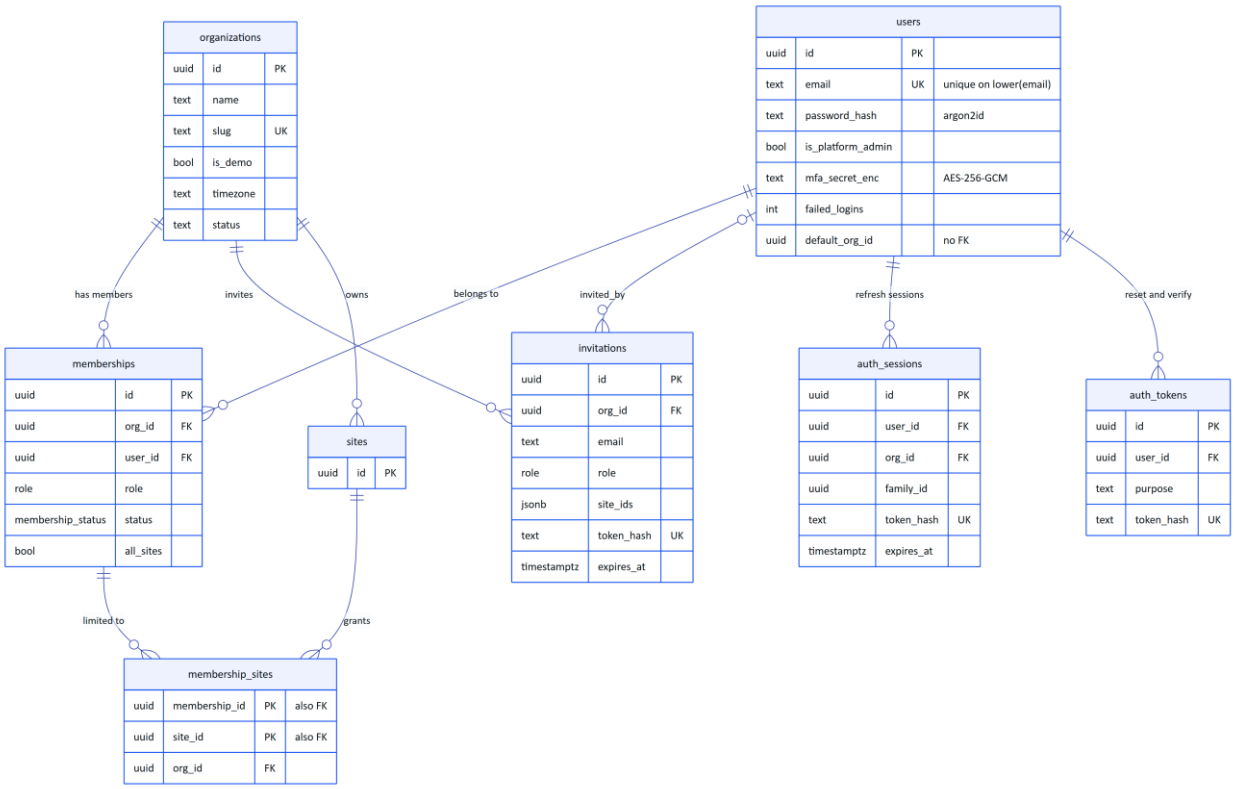

Figure 79. *Identity and tenancy: users join organizations through memberships, optionally limited to sites*

`users` is global: one person can belong to several organizations with a different role in each. The active organization is carried in the access token (`org` claim) and resolved to a membership on every request. `mail_outbox` stands alone (no foreign keys) and is not drawn.

4.3.2 Plant model

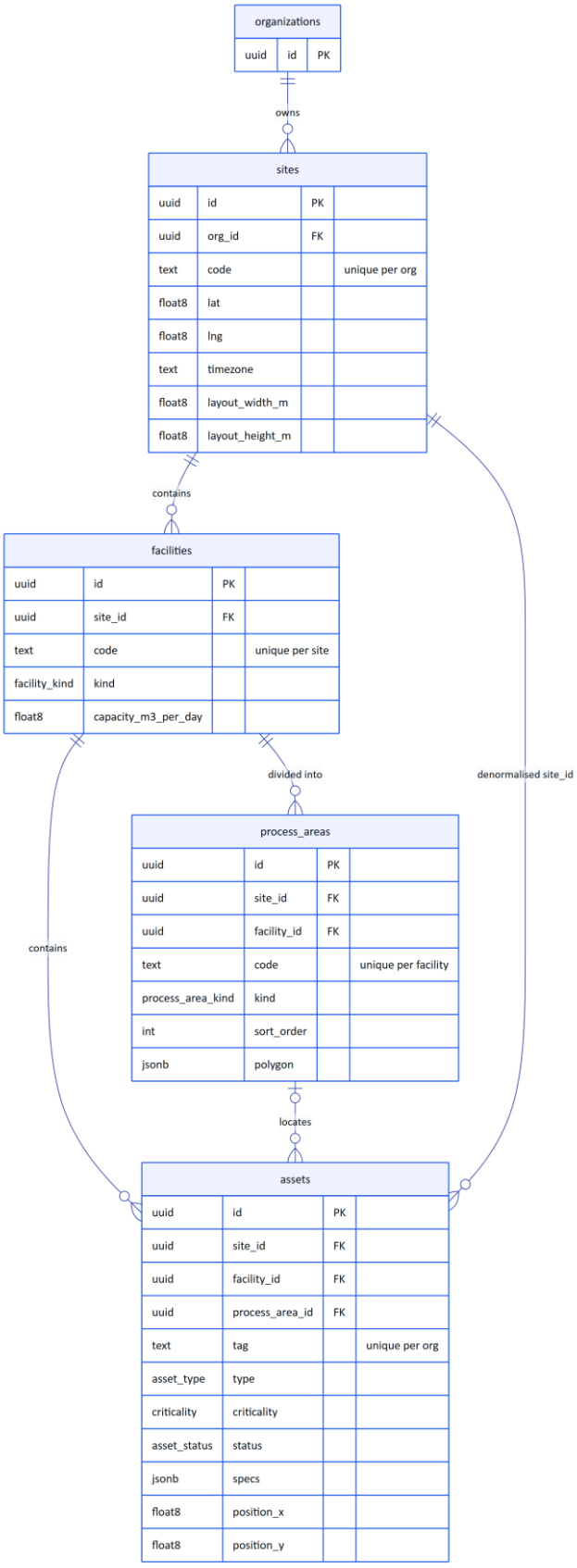

Figure 80. *Plant model: site, facility, process area and asset hierarchy*

`assets.site_id` duplicates what can be derived through `facility_id`; it is kept so that site-scoped queries and site permission checks need no join.

4.3.3 Telemetry

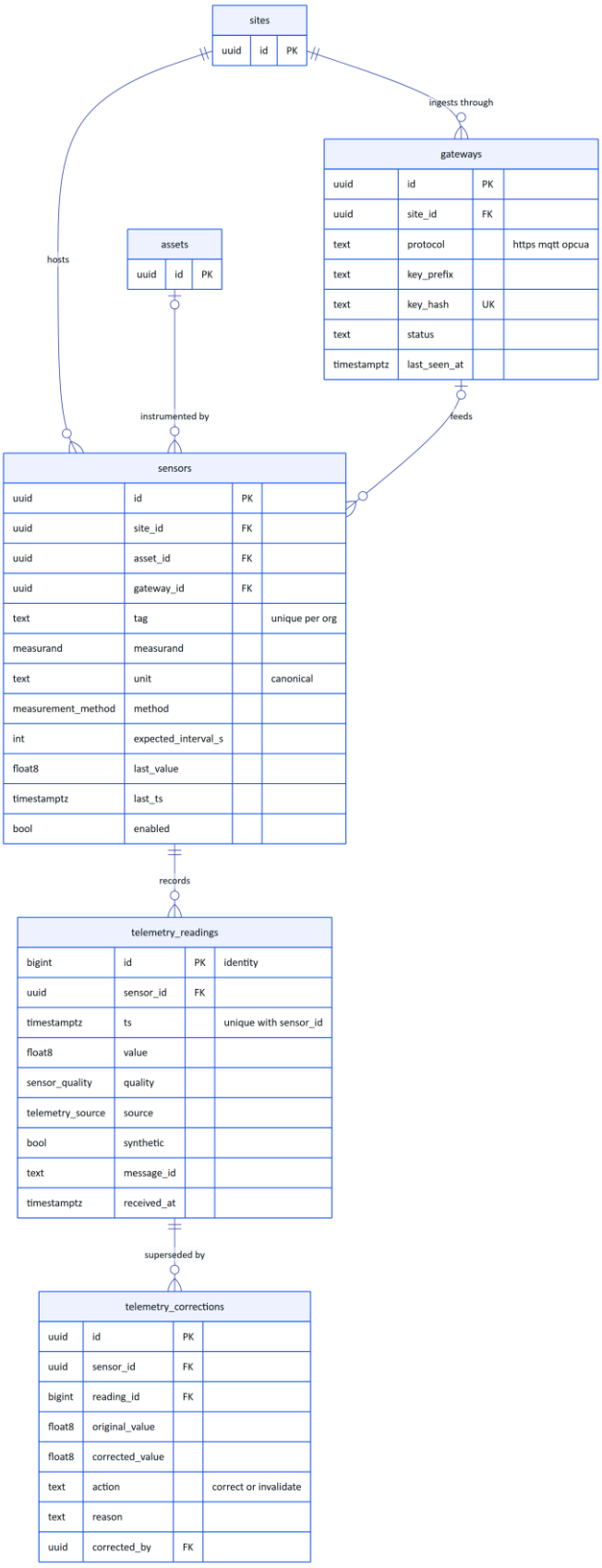

Figure 81. *Telemetry: gateways feed sensors, sensors record readings, corrections supersede readings*

4.3.4 Alarms and procedures

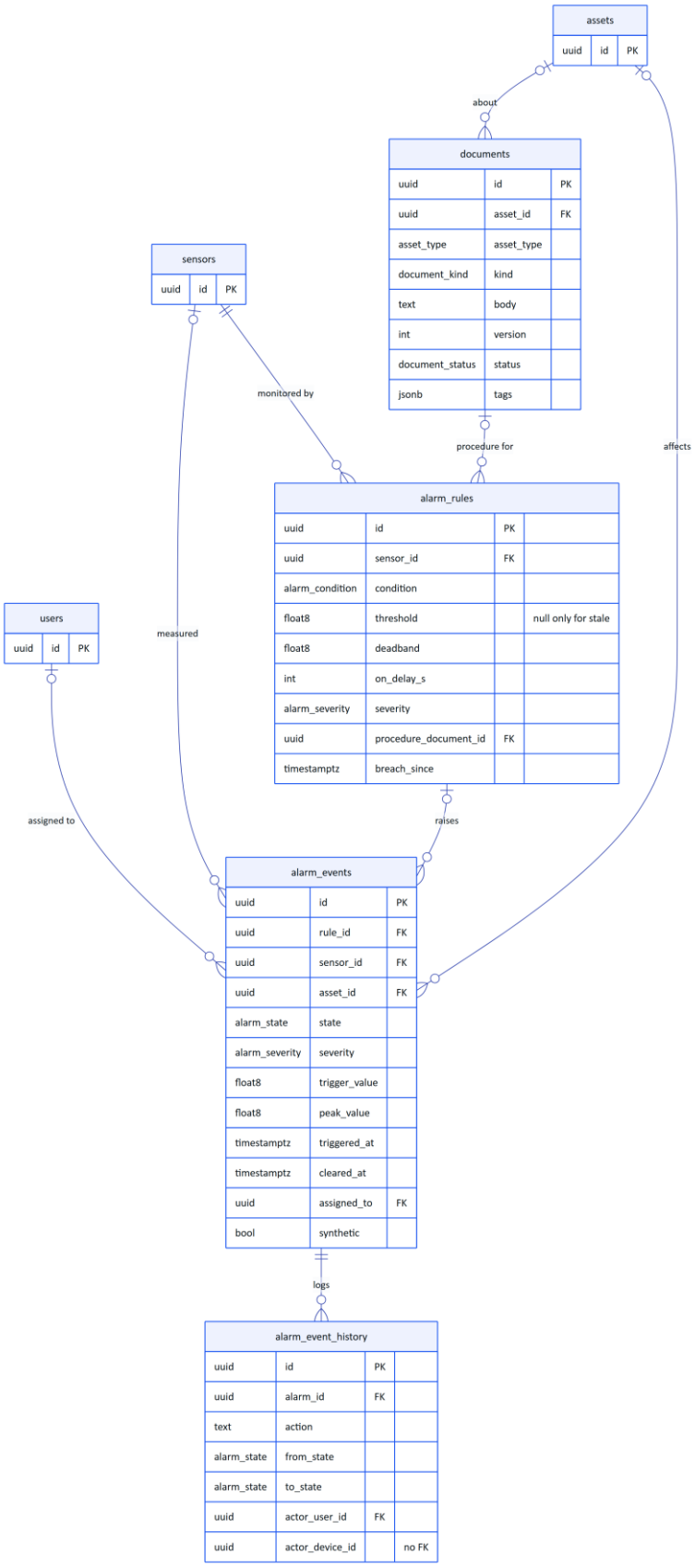

Figure 82. *Alarms and procedures: rules watch sensors, events carry the human lifecycle, documents hold SOPs*

4.3.5 Inspections and maintenance

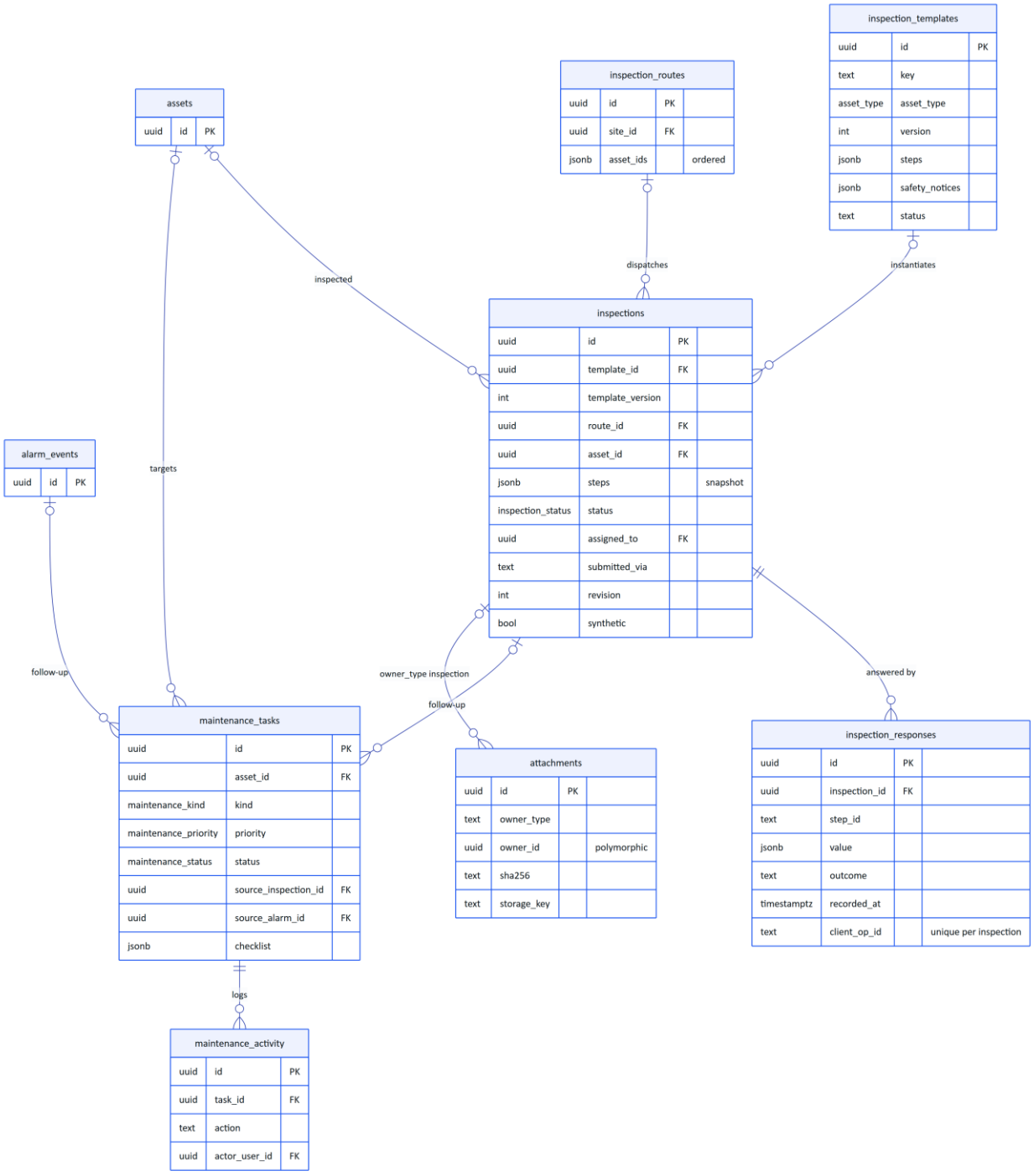

Figure 83. *Work: templates and routes produce inspections, abnormal results and alarms produce maintenance tasks*

`attachments.owner_id` is polymorphic: it points at an inspection, maintenance task, asset or alarm depending on `owner_type`, and therefore has no foreign key. The upload route validates the owner inside the tenant transaction before inserting (`apps/api/src/modules/plant.ts`).

4.3.6 Devices and UI profiles

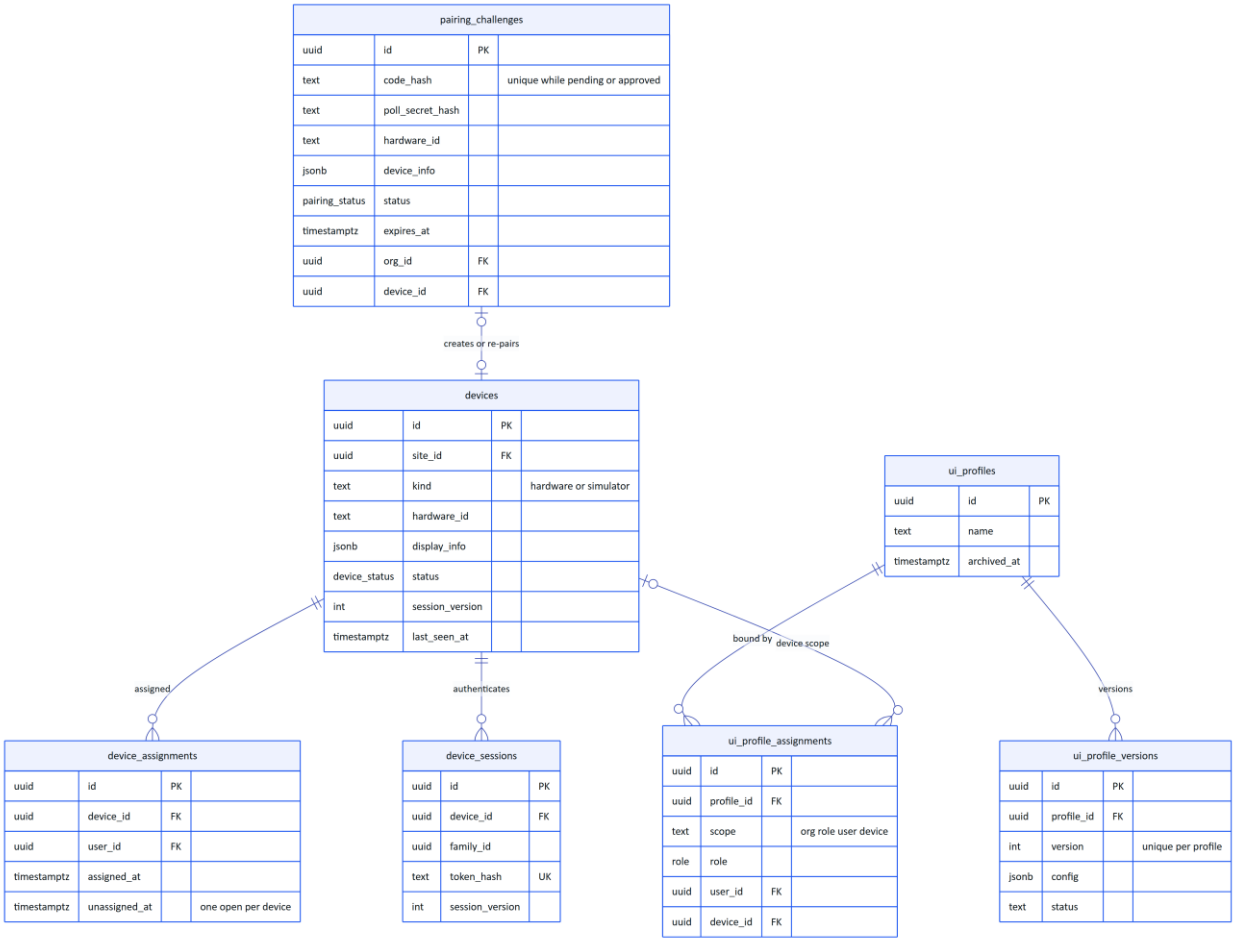

Figure 84. *Devices and UI profiles: pairing creates devices, assignments bind operators, profiles shape the glasses UI*

4.3.7 Audit documents

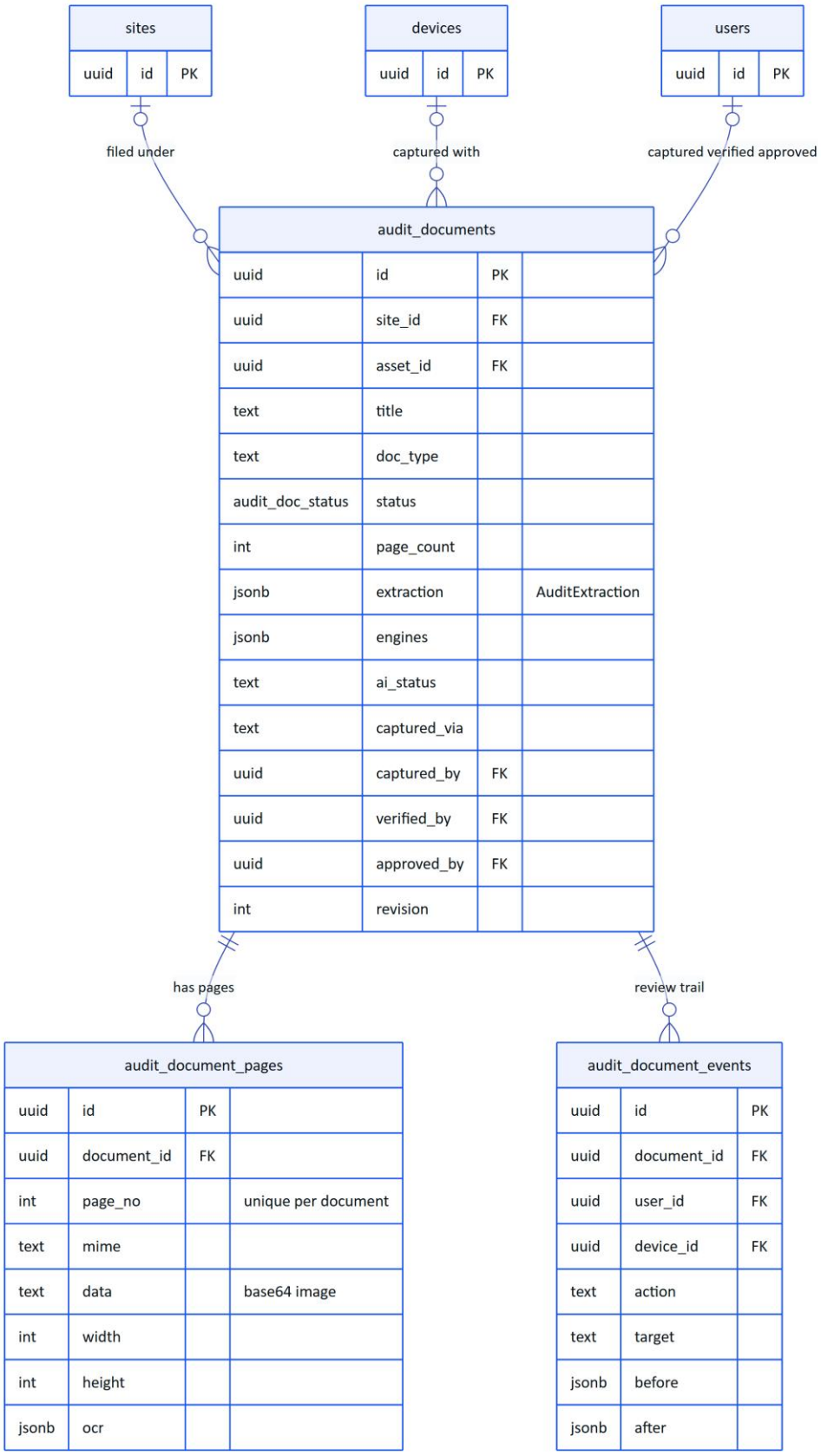

Figure 85. *Audit documents: a scan has pages with images and OCR, and an append-only review trail*

4.3.8 Platform and demo storage

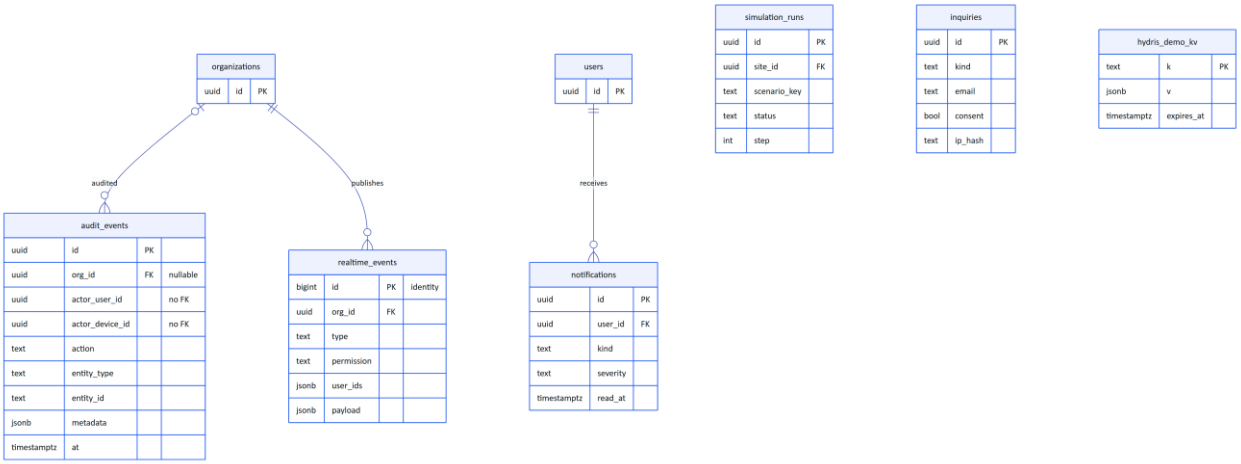

Figure 86. *Platform tables: audit trail, notifications, realtime log, simulations, leads and the demo key/value store*

`inquiries` and `hydris_demo_kv` have no relationships: inquiries are platform-owned leads from the public website, and the demo key/value store is infrastructure for demo mode only.

4.4 Data dictionary

One table per database table, listing every column. "Null" is **No** for `NOT NULL` columns. Defaults are the SQL defaults from the migrations; application-level defaults are noted in the description. Foreign keys give the target and the `ON DELETE` action (`cascade`, `set null`, or `no action` when omitted). Every `id` without a note is `uuid` with default `gen_random_uuid()`, and every `org_id` is `uuid not null`, FK `organizations.id`, cascade.

4.4.1 Identity and tenancy tables

users

Table 62. **users (global identities, not tenant-scoped, no RLS)**

Column	Type	Null	Default	Description
`id`	uuid	No	gen_random_uuid()	Primary key.
`email`	text	No		Sign-in email. Unique case-insensitively.
`name`	text	No		Display name.
`password_hash`	text	Yes		argon2id hash (`core/crypto.ts`). Null for accounts without a password.
`is_platform_admin`	boolean	No	false	Hydris staff flag; enables platform administration routes.
`status`	text	No	'active'	`active` or `disabled`. Disabled users cannot authenticate.
`email_verified_at`	timestamptz	Yes		Set on verification or first password reset.
`mfa_secret_enc`	text	Yes		TOTP secret, AES-256-GCM encrypted with `DATA_KEY`.
`mfa_enabled`	boolean	No	false	TOTP required at sign-in when true.
`failed_logins`	integer	No	0	Consecutive failed sign-ins; reset on success.
`locked_until`	timestamptz	Yes		Set for 15 minutes after 5 failures (`TTL` in `config.ts`).

Column	Type	Null	Default	Description
last_login_at	timestamptz	Yes		Last successful sign-in.
default_org_id	uuid	Yes		Preferred organization at sign-in. No FK.
created_at	timestamptz	No	now()	
updated_at	timestamptz	No	now()	

Indexes and constraints: users_email_lower_uq unique on lower(email).

organizations

Table 63. organizations (tenants)

Column	Type	Null	Default	Description
id	uuid	No	gen_random_uuid()	Primary key; the tenant id used in app.org_id.
name	text	No		Display name.
slug	text	No		URL-safe unique key (demo: riverbend-demo).
is_demo	boolean	No	false	Demo tenants may receive simulator telemetry; customer tenants never do.
region	text	Yes		Free-text region label.
timezone	text	No	'UTC'	IANA zone for display and reports.
status	text	No	'active'	active or suspended.
created_at	timestamptz	No	now()	
updated_at	timestamptz	No	now()	

Indexes and constraints: organizations_slug_uq unique on slug. RLS policy uses id instead of org_id.

memberships

Table 64. memberships (user role in an organization)

Column	Type	Null	Default	Description
id	uuid	No	gen_random_uuid()	Primary key.
org_id	uuid	No		Tenant.
user_id	uuid	No		FK users.id, cascade.
role	role	No		One of the six roles in roles.ts.
status	membership_status	No	'active'	invited, active, deactivated.
all_sites	boolean	No	true	When false, access is limited to membership_sites.
title	text	Yes		Job title shown in the UI.
created_at	timestamptz	No	now()	
updated_at	timestamptz	No	now()	
deactivated_at	timestamptz	Yes		When access was removed.

Indexes and constraints: `memberships_org_user_uq` unique `(org_id, user_id)`; `memberships_user_idx` on `user_id`.

membership_sites

Table 65. membership_sites (site scope for restricted memberships)

Column	Type	Null	Default	Description
`membership_id`	uuid	No		FK `memberships.id`, cascade. Part of the primary key.
`site_id`	uuid	No		FK `sites.id`, cascade. Part of the primary key.
`org_id`	uuid	No		Tenant.

Indexes and constraints: primary key `(membership_id, site_id)`.

invitations

Table 66. invitations (pending invites to an organization)

Column	Type	Null	Default	Description
`id`	uuid	No	gen_random_uuid()	Primary key.
`org_id`	uuid	No		Tenant.
`email`	text	No		Invitee address.
`name`	text	Yes		Invitee name if known.
`role`	role	No		Role granted on acceptance.
`site_ids`	jsonb	No	'[]'	Array of site uuids when `all_sites` is false.
`all_sites`	boolean	No	true	Site scope on acceptance.
`token_hash`	text	No		SHA-256 of the invitation token.
`expires_at`	timestamptz	No		Seven days after creation (`TTL.inviteMs`).
`accepted_at`	timestamptz	Yes		Set on acceptance.
`revoked_at`	timestamptz	Yes		Set when an admin revokes.
`invited_by`	uuid	Yes		FK `users.id`.
`created_at`	timestamptz	No	now()	

Indexes and constraints: `invitations_token_uq` unique on `token_hash`; `invitations_org_idx` on `org_id`.

auth_sessions

Table 67. auth_sessions (rotating refresh-token sessions for people; no RLS)

Column	Type	Null	Default	Description
`id`	uuid	No	gen_random_uuid()	Primary key.
`user_id`	uuid	No		FK `users.id`, cascade.

Column	Type	Null	Default	Description
org_id	uuid	Yes		FK organizations.id, cascade. Organization of the session.
family_id	uuid	No		Rotation family; the access token sid claim. Revoking a family ends all its tokens.
token_hash	text	No		SHA-256 of the refresh token.
expires_at	timestamptz	No		Seven days after issue (TTL.refreshMs).
revoked_at	timestamptz	Yes		Logout or reuse detection.
rotated_at	timestamptz	Yes		Set when the token was exchanged for a new one.
user_agent	text	Yes		Client user agent.
ip	text	Yes		Client IP at issue.
created_at	timestamptz	No	now()	
last_used_at	timestamptz	Yes		Last refresh.

Indexes and constraints: auth_sessions_token_uq unique on token_hash; auth_sessions_user_idx; auth_sessions_family_idx.

auth_tokens

Table 68. auth_tokens (single-use password reset and email verification tokens; no RLS)

Column	Type	Null	Default	Description
id	uuid	No	gen_random_uuid()	Primary key.
purpose	text	No		password_reset or email_verify.
user_id	uuid	No		FK users.id, cascade.
token_hash	text	No		SHA-256 of the emailed token.
expires_at	timestamptz	No		30 minutes for resets (TTL.resetMs).
used_at	timestamptz	Yes		Set when consumed.
created_at	timestamptz	No	now()	

Indexes and constraints: auth_tokens_token_uq unique on token_hash.

mail_outbox

Table 69. mail_outbox (recorded outgoing mail; no transport yet; no RLS)

Column	Type	Null	Default	Description
id	uuid	No	gen_random_uuid()	Primary key.
to	text	No		Recipient address.
subject	text	No		Subject line.
body_text	text	No		Plain-text body. Can contain reset and invitation links.
created_at	timestamptz	No	now()	

Column	Type	Null	Default	Description
sent_at	timestamptz	Yes		Reserved for a mail transport; never set today.

Indexes and constraints: none.

> **RISK**
> mail_outbox.body_text holds live password-reset and invitation links in clear text and is never purged. Until a mail transport exists, restrict database access to this table and add the purge job proposed in "Data lifecycle and retention".

4.4.2 Plant model tables

sites

Table 70. sites (physical plant locations)

Column	Type	Null	Default	Description
id	uuid	No	gen_random_uuid()	Primary key.
org_id	uuid	No		Tenant.
name	text	No		Display name.
code	text	No		Short code, unique per organization (demo: RIP, RTS).
address	text	Yes		Street address.
city	text	Yes		City.
country	text	Yes		Country.
lat	double precision	Yes		Latitude, WGS84.
lng	double precision	Yes		Longitude, WGS84.
timezone	text	No	'UTC'	IANA zone.
layout_width_m	double precision	No	100	Plant layout canvas width in metres (map and route planner).
layout_height_m	double precision	No	80	Canvas height in metres.
archived_at	timestamptz	Yes		Soft delete.
created_at	timestamptz	No	now()	
updated_at	timestamptz	No	now()	

Indexes and constraints: sites_org_code_uq unique (org_id, code); sites_org_idx on org_id.

facilities

Table 71. facilities (treatment plants within a site)

Column	Type	Null	Default	Description
id	uuid	No	gen_random_uuid()	Primary key.
org_id	uuid	No		Tenant.
site_id	uuid	No		FK sites.id, cascade.
name	text	No		Display name.
code	text	No		Unique per site (demo: ETP-01).
kind	facility_kind	No		ETP, STP, WTP, CETP or other.
capacity_m3_per_day	double precision	Yes		Design capacity.
description	text	Yes		Free text.
archived_at	timestamptz	Yes		Soft delete.
created_at	timestamptz	No	now()	
updated_at	timestamptz	No	now()	

Indexes and constraints: facilities_site_code_uq unique (site_id, code); facilities_org_idx.

process_areas

Table 72. process_areas (treatment stages inside a facility)

Column	Type	Null	Default	Description
id	uuid	No	gen_random_uuid()	Primary key.
org_id	uuid	No		Tenant.
site_id	uuid	No		FK sites.id, cascade.
facility_id	uuid	No		FK facilities.id, cascade.
name	text	No		Display name.
code	text	No		Unique per facility.
kind	process_area_kind	No		Treatment stage (13 values).
sort_order	integer	No	0	Display order along the treatment train.
polygon	jsonb	Yes		Outline on the site layout, metres: [[x, y], ...].
archived_at	timestamptz	Yes		Soft delete.
created_at	timestamptz	No	now()	
updated_at	timestamptz	No	now()	

Indexes and constraints: process_areas_facility_code_uq unique (facility_id, code); process_areas_org_idx.

assets

Table 73. assets (equipment)

Column	Type	Null	Default	Description
id	uuid	No	gen_random_uuid()	Primary key.
org_id	uuid	No		Tenant.
site_id	uuid	No		FK sites.id, cascade.
facility_id	uuid	No		FK facilities.id, cascade.
process_area_id	uuid	Yes		FK process_areas.id, set null.
tag	text	No		Plant tag (P-101), unique per organization.
name	text	No		Display name.
type	asset_type	No		Equipment class (14 values).
manufacturer	text	Yes		
model	text	Yes		
serial_number	text	Yes		
installed_on	text	Yes		Installation date as entered (ISO date string by convention).
criticality	criticality	No	'medium'	low, medium, high.
status	asset_status	No	'operational'	Operational state (6 values).
specs	jsonb	No	'{}'	Name to value map of nameplate data.
notes	text	Yes		Free text.
position_x	double precision	Yes		Map position in metres from the layout origin.
position_y	double precision	Yes		Map position in metres.
archived_at	timestamptz	Yes		Soft delete.
created_at	timestamptz	No	now()	
updated_at	timestamptz	No	now()	

Indexes and constraints: assets_org_tag_uq unique (org_id, tag); assets_site_idx (org_id, site_id); assets_area_idx on process_area_id.

4.4.3 Telemetry tables

gateways

Table 74. gateways (ingestion credentials per site)

Column	Type	Null	Default	Description
id	uuid	No	gen_random_uuid()	Primary key.
org_id	uuid	No		Tenant.

Column	Type	Null	Default	Description
site_id	uuid	No		FK sites.id, cascade. A gateway may only write sensors on its own site.
name	text	No		Display name.
protocol	text	No	'https'	https, mqtt or opcua (only HTTPS ingestion is implemented).
key_prefix	text	No		Visible prefix of the API key for identification.
key_hash	text	No		SHA-256 of the full key.
status	text	No	'active'	active or revoked.
last_seen_at	timestamptz	Yes		Updated on each authenticated ingest call.
created_by	uuid	Yes		FK users.id.
created_at	timestamptz	No	now()	
revoked_at	timestamptz	Yes		

Indexes and constraints: gateways_key_uq unique on key_hash; gateways_org_idx.

sensors

Table 75. sensors (measurement points, with the latest value cached)

Column	Type	Null	Default	Description
id	uuid	No	gen_random_uuid()	Primary key.
org_id	uuid	No		Tenant.
site_id	uuid	No		FK sites.id, cascade.
asset_id	uuid	Yes		FK assets.id, cascade.
process_area_id	uuid	Yes		FK process_areas.id, set null.
gateway_id	uuid	Yes		FK gateways.id, set null.
tag	text	No		Instrument tag (VT-101), unique per organization.
name	text	No		Display name.
measurand	measurand	No		What is measured (16 values).
unit	text	No		Canonical unit from MEASURAND_DEFS.
method	measurement_method	No	'continuous'	continuous, lab or manual.
expected_interval_s	integer	No	60	Expected reporting interval; drives freshness and stale alarms.
source_ref	text	Yes		Gateway-side address (demo: ETP01.VT-101).
last_value	double precision	Yes		Cached effective latest value.
last_ts	timestamptz	Yes		Observation time of last_value.
last_quality	sensor_quality	Yes		Quality of the latest reading.

Column	Type	Null	Default	Description
last_source	telemetry_source	Yes		Source of the latest reading.
last_received_at	timestamptz	Yes		Server receive time of the latest reading.
enabled	boolean	No	true	Disabled sensors reject ingestion.
archived_at	timestamptz	Yes		Soft delete; archived sensors reject ingestion.
created_at	timestamptz	No	now()	
updated_at	timestamptz	No	now()	

Indexes and constraints: sensors_org_tag_uq unique (org_id, tag); sensors_asset_idx; sensors_site_idx (org_id, site_id).

telemetry_readings

Table 76. telemetry_readings (time series of measured values)

Column	Type	Null	Default	Description
id	bigint	No	identity	Primary key, GENERATED ALWAYS AS IDENTITY.
org_id	uuid	No		Tenant.
sensor_id	uuid	No		FK sensors.id, cascade.
ts	timestamptz	No		Observation time from the source.
value	double precision	No		Value in the canonical unit.
quality	sensor_quality	No	'good'	good, uncertain, bad. Bad readings never raise alarms.
source	telemetry_source	No		gateway, manual, lab or simulator.
synthetic	boolean	No	false	True if and only if source = 'simulator'.
message_id	text	Yes		Source message id for tracing (demo: demo:VT-101:<ms>).
received_at	timestamptz	No	now()	Server receive time.

Indexes and constraints: telemetry_sensor_ts_uq unique (sensor_id, ts) (idempotent ingestion); telemetry_sensor_ts_idx (sensor_id, ts DESC); telemetry_org_ts_idx (org_id, ts); CHECK telemetry_synthetic_source_ck: synthetic = (source = 'simulator').

telemetry_corrections

Table 77. telemetry_corrections (append-only corrections; UPDATE and DELETE revoked)

Column	Type	Null	Default	Description
id	uuid	No	gen_random_uuid()	Primary key.
org_id	uuid	No		Tenant.
sensor_id	uuid	No		FK sensors.id, cascade.
reading_id	bigint	No		FK telemetry_readings.id, cascade.
original_value	double precision	No		Value of the reading at correction time.

Column	Type	Null	Default	Description
corrected_value	double precision	Yes		New value (canonical unit) for correct; null for invalidate.
action	text	No		correct replaces the value; invalidate excludes the reading.
reason	text	No		Required explanation, 5 to 1,000 characters.
corrected_by	uuid	Yes		FK users.id.
created_at	timestamptz	No	now()	The newest correction per reading wins.

Indexes and constraints: telemetry_corrections_reading_idx (reading_id, created_at); telemetry_corrections_sensor_idx.

4.4.4 Alarm and procedure tables

alarm_rules

Table 78. alarm_rules (threshold and data-freshness rules)

Column	Type	Null	Default	Description
id	uuid	No	gen_random_uuid()	Primary key.
org_id	uuid	No		Tenant.
site_id	uuid	No		FK sites.id, cascade. Copied from the sensor.
sensor_id	uuid	No		FK sensors.id, cascade.
name	text	No		Display name (demo: VT-101 high-high).
condition	alarm_condition	No		high_high, high, low, low_low or stale.
threshold	double precision	Yes		Limit in the canonical unit; required unless stale.
deadband	double precision	No	0	Hysteresis band before an alarm counts as cleared.
on_delay_s	integer	No	0	Seconds a breach must persist before raising.
severity	alarm_severity	No		critical, high, medium, low.
enabled	boolean	No	true	Rules with history are disabled instead of deleted.
recommended_action	text	Yes		First action shown to the operator.
procedure_document_id	uuid	Yes		FK documents.id, set null. Linked SOP.
breach_since	timestamptz	Yes		Evaluation state: start of the current breach (on-delay).
created_by	uuid	Yes		FK users.id.
created_at	timestamptz	No	now()	
updated_at	timestamptz	No	now()	

Indexes and constraints: `alarm_rules_sensor_idx`; CHECK `alarm_rules_threshold_ck`: `condition = 'stale' OR threshold IS NOT NULL`.

alarm_events

Table 79. alarm_events (alarm instances and their human lifecycle)

Column	Type	Null	Default	Description
`id`	uuid	No	gen_random_uuid()	Primary key.
`org_id`	uuid	No		Tenant.
`site_id`	uuid	No		FK `sites.id`, cascade.
`asset_id`	uuid	Yes		FK `assets.id`, set null.
`sensor_id`	uuid	Yes		FK `sensors.id`, set null.
`rule_id`	uuid	Yes		FK `alarm_rules.id`, set null.
`severity`	alarm_severity	No		Copied from the rule at trigger time.
`condition`	alarm_condition	No		Copied from the rule.
`state`	alarm_state	No	'active'	Human lifecycle state.
`title`	text	No		Headline.
`description`	text	Yes		Breach description (`describeBreach()`).
`recommended_action`	text	Yes		Copied from the rule.
`trigger_value`	double precision	Yes		Value that raised the alarm.
`threshold`	double precision	Yes		Limit at trigger time.
`unit`	text	Yes		Unit of the values.
`peak_value`	double precision	Yes		Worst value seen while the condition persisted.
`triggered_at`	timestamptz	No		Observation time of the triggering reading.
`cleared_at`	timestamptz	Yes		When the measured condition returned to normal; independent of `state`.
`assigned_to`	uuid	Yes		FK `users.id`.
`acknowledged_by`	uuid	Yes		FK `users.id`.
`acknowledged_at`	timestamptz	Yes		
`resolved_by`	uuid	Yes		FK `users.id`.
`resolved_at`	timestamptz	Yes		
`resolution_note`	text	Yes		Required by the `resolve` transition.
`closed_by`	uuid	Yes		FK `users.id`.
`closed_at`	timestamptz	Yes		
`synthetic`	boolean	No	false	Raised from simulator data.
`updated_at`	timestamptz	No	now()	There is no `created_at`; `triggered_at` serves.

Indexes and constraints: `alarm_events_org_state_idx (org_id, state, triggered_at)`; `alarm_events_rule_idx (rule_id, state)`; `alarm_events_asset_idx`.

alarm_event_history

Table 80. alarm_event_history (transition and note log per alarm)

Column	Type	Null	Default	Description
`id`	uuid	No	gen_random_uuid()	Primary key.
`org_id`	uuid	No		Tenant.
`alarm_id`	uuid	No		FK `alarm_events.id`, cascade.
`action`	text	No		`triggered`, `condition_cleared`, `acknowledge`, `investigate`, `resolve`, `close`, `assigned`, `note`.
`from_state`	alarm_state	Yes		State before a transition.
`to_state`	alarm_state	Yes		State after a transition.
`actor_user_id`	uuid	Yes		FK `users.id`; null for the alarm engine.
`actor_device_id`	uuid	Yes		Glasses that performed the action. No FK.
`note`	text	Yes		Operator note or engine message.
`at`	timestamptz	No	now()	

Indexes and constraints: `alarm_history_alarm_idx (alarm_id, at)`.

documents

Table 81. documents (SOPs, manuals and safety notices; full-text searchable)

Column	Type	Null	Default	Description
`id`	uuid	No	gen_random_uuid()	Primary key.
`org_id`	uuid	No		Tenant.
`site_id`	uuid	Yes		FK `sites.id`, cascade. Null means organization-wide.
`asset_id`	uuid	Yes		FK `assets.id`, set null. Specific asset.
`asset_type`	asset_type	Yes		Applies to every asset of this type.
`title`	text	No		Title (demo: `SOP-PMP-07 Pump vibration response`).
`kind`	document_kind	No		sop, manual, datasheet, safety, troubleshooting, other.
`body`	text	No		Plain text body; searched with `to_tsvector('english', ...)`.
`version`	integer	No	1	Document revision number.
`status`	document_status	No	'draft'	Only `approved` documents reach the glasses and the assistant.
`tags`	jsonb	No	'[]'	Array of strings.
`approved_by`	uuid	Yes		FK `users.id`.
`approved_at`	timestamptz	Yes		

Column	Type	Null	Default	Description
created_by	uuid	Yes		FK users.id.
created_at	timestamptz	No	now()	
updated_at	timestamptz	No	now()	

Indexes and constraints: documents_org_idx (org_id, status); documents_asset_idx. No full-text index (see "Indexing and query patterns").

4.4.5 Work tables

inspection_templates

Table 82. inspection_templates (versioned checklists)

Column	Type	Null	Default	Description
id	uuid	No	gen_random_uuid()	Primary key.
org_id	uuid	No		Tenant.
key	text	Yes		Stable key across versions (demo: pump_routine). Not unique.
name	text	No		Display name.
description	text	Yes		
asset_type	asset_type	Yes		Asset class the template is for.
version	integer	No	1	Template revision.
steps	jsonb	No		InspectionStep[] (see "JSON column schemas").
safety_notices	jsonb	No	'[]'	Array of strings shown before the first step.
estimated_minutes	integer	Yes		Planning estimate.
status	text	No	'published'	draft, published or archived.
created_by	uuid	Yes		FK users.id.
created_at	timestamptz	No	now()	
updated_at	timestamptz	No	now()	

Indexes and constraints: inspection_templates_org_idx.

inspection_routes

Table 83. inspection_routes (ordered rounds through assets on a site)

Column	Type	Null	Default	Description
id	uuid	No	gen_random_uuid()	Primary key.
org_id	uuid	No		Tenant.
site_id	uuid	No		FK sites.id, cascade.
name	text	No		Display name (demo: Morning round · ETP-01).

Column	Type	Null	Default	Description
description	text	Yes		
asset_ids	jsonb	No	'[]'	Ordered array of asset uuids. Not foreign keys.
created_by	uuid	Yes		FK users.id.
created_at	timestamptz	No	now()	
updated_at	timestamptz	No	now()	

Indexes and constraints: inspection_routes_site_idx (org_id, site_id).

inspections

Table 84. inspections (assigned inspection instances)

Column	Type	Null	Default	Description
id	uuid	No	gen_random_uuid()	Primary key.
org_id	uuid	No		Tenant.
site_id	uuid	No		FK sites.id, cascade.
asset_id	uuid	Yes		FK assets.id, set null.
template_id	uuid	Yes		FK inspection_templates.id, set null.
template_version	integer	Yes		Template version at assignment.
route_id	uuid	Yes		FK inspection_routes.id, set null.
title	text	No		Display title.
steps	jsonb	No		Snapshot of the template steps at assignment time.
safety_notices	jsonb	No	'[]'	Snapshot of the safety notices.
status	inspection_status	No	'scheduled'	Lifecycle state.
priority	text	No	'normal'	normal or high.
assigned_to	uuid	Yes		FK users.id. Only this person may record responses.
assigned_by	uuid	Yes		FK users.id. Notified on submit.
due_at	timestamptz	Yes		
started_at	timestamptz	Yes		First response or explicit start.
paused_at	timestamptz	Yes		
submitted_at	timestamptz	Yes		
submitted_via	text	Yes		console or glasses.
submitted_device_id	uuid	Yes		Glasses that submitted. No FK.
reviewed_by	uuid	Yes		FK users.id.
reviewed_at	timestamptz	Yes		
review_note	text	Yes		Reason when returned.

Column	Type	Null	Default	Description
revision	integer	No	1	Incremented on every change; optimistic lock for offline clients.
synthetic	boolean	No	false	Created by a simulation.
created_at	timestamptz	No	now()	
updated_at	timestamptz	No	now()	

Indexes and constraints: inspections_org_status_idx (org_id, status); inspections_assignee_idx (assigned_to, status); inspections_asset_idx.

inspection_responses

Table 85. inspection_responses (step answers; idempotent for offline replay)

Column	Type	Null	Default	Description
id	uuid	No	gen_random_uuid()	Primary key.
org_id	uuid	No		Tenant.
inspection_id	uuid	No		FK inspections.id, cascade.
step_id	text	No		InspectionStep.id within the snapshot.
value	jsonb	No		StepValue (see "JSON column schemas").
outcome	text	No		ok, abnormal or na, derived by stepOutcome().
note	text	Yes		Operator note.
recorded_at	timestamptz	No		Client time of the answer; at most 5 minutes in the future.
received_at	timestamptz	No	now()	Server receive time.
recorded_by	uuid	Yes		FK users.id.
device_id	uuid	Yes		Glasses that recorded the answer. No FK.
client_op_id	text	No		Client-generated idempotency key.

Indexes and constraints: inspection_responses_op_uq unique (inspection_id, client_op_id); inspection_responses_step_idx (inspection_id, step_id, recorded_at). A step may have several responses; the latest recorded_at is current.

attachments

Table 86. attachments (file metadata; bytes live in object storage)

Column	Type	Null	Default	Description
id	uuid	No	gen_random_uuid()	Primary key.
org_id	uuid	No		Tenant.
owner_type	text	No		inspection, maintenance, asset, document, alarm (upload route accepts all but document).
owner_id	uuid	No		Owner row id. Polymorphic, no FK.

Column	Type	Null	Default	Description
filename	text	No		Sanitised original name, at most 120 characters.
mime	text	No		JPEG, PNG, WebP, PDF or plain text; magic bytes checked.
size_bytes	integer	No		Size in bytes.
sha256	text	No		Content hash.
storage_key	text	No		Disk path <org>/<uuid> or private Vercel Blob key attachments/<org>/<uuid>.
uploaded_by	uuid	Yes		FK users.id.
device_id	uuid	Yes		Glasses that uploaded. No FK.
created_at	timestamptz	No	now()	

Indexes and constraints: attachments_owner_idx (owner_type, owner_id).

maintenance_tasks

Table 87. maintenance_tasks (work orders)

Column	Type	Null	Default	Description
id	uuid	No	gen_random_uuid()	Primary key.
org_id	uuid	No		Tenant.
site_id	uuid	No		FK sites.id, cascade.
asset_id	uuid	Yes		FK assets.id, set null.
title	text	No		
description	text	Yes		
kind	maintenance_kind	No	'corrective'	corrective, preventive, inspection_followup, alarm_followup.
priority	maintenance_priority	No	'medium'	low, medium, high, urgent.
status	maintenance_status	No	'open'	Lifecycle state.
assigned_to	uuid	Yes		FK users.id.
requested_by	uuid	Yes		FK users.id.
source_inspection_id	uuid	Yes		FK inspections.id, set null. Set for automatic follow-ups.
source_alarm_id	uuid	Yes		FK alarm_events.id, set null.
checklist	jsonb	No	'[]'	Checklist items (see "JSON column schemas").
due_at	timestamptz	Yes		
started_at	timestamptz	Yes		
completed_at	timestamptz	Yes		
completion_note	text	Yes		
synthetic	boolean	No	false	Created by a simulation.

Column	Type	Null	Default	Description
created_at	timestamptz	No	now()	
updated_at	timestamptz	No	now()	

Indexes and constraints: maintenance_org_status_idx (org_id, status); maintenance_assignee_idx (assigned_to, status); maintenance_asset_idx.

maintenance_activity

Table 88. maintenance_activity (task log)

Column	Type	Null	Default	Description
id	uuid	No	gen_random_uuid()	Primary key.
org_id	uuid	No		Tenant.
task_id	uuid	No		FK maintenance_tasks.id, cascade.
action	text	No		Transition or event name (start, hold, complete, note, assigned, ...).
note	text	Yes		
actor_user_id	uuid	Yes		FK users.id.
at	timestamptz	No	now()	

Indexes and constraints: maintenance_activity_task_idx (task_id, at).

4.4.6 Device and UI profile tables

devices

Table 89. devices (paired glasses and simulators)

Column	Type	Null	Default	Description
id	uuid	No	gen_random_uuid()	Primary key.
org_id	uuid	No		Tenant.
site_id	uuid	Yes		FK sites.id, set null. Home site.
name	text	No		Display name.
kind	text	No		hardware or simulator.
model	text	Yes		Model string (demo: Vuzix M400).
hardware_id	text	No		Install identifier generated on the device; stable across re-pairing.
firmware_version	text	Yes		
app_version	text	Yes		
display_info	jsonb	Yes		{ "width": n, "height": n }.
status	device_status	No	'active'	active, suspended, revoked.

Column	Type	Null	Default	Description
session_version	integer	No	1	Bumped on revoke, suspension or reassignment to invalidate tokens.
paired_at	timestamptz	No	now()	
paired_by	uuid	Yes		FK users.id.
last_seen_at	timestamptz	Yes		Updated on every authenticated device request.
last_ip	text	Yes		IP of the last device request.
suspended_at	timestamptz	Yes		
revoked_at	timestamptz	Yes		
revoked_by	uuid	Yes		FK users.id.
revoke_reason	text	Yes		
created_at	timestamptz	No	now()	
updated_at	timestamptz	No	now()	

Indexes and constraints: devices_org_idx (org_id, status); devices_hw_idx on hardware_id.

device_assignments

Table 90. device_assignments (which operator wears which device)

Column	Type	Null	Default	Description
id	uuid	No	gen_random_uuid()	Primary key.
org_id	uuid	No		Tenant.
device_id	uuid	No		FK devices.id, cascade.
user_id	uuid	No		FK users.id, cascade.
assigned_by	uuid	Yes		FK users.id.
assigned_at	timestamptz	No	now()	
unassigned_at	timestamptz	Yes		Null while the assignment is current.
reason	text	Yes		

Indexes and constraints: device_assignments_device_idx (device_id, assigned_at); device_assignments_active_uq unique (device_id) WHERE unassigned_at IS NULL.

pairing_challenges

Table 91. pairing_challenges (device authorization flow state; pre-tenant, no RLS)

Column	Type	Null	Default	Description
id	uuid	No	gen_random_uuid()	Primary key; the challengeId.
code_hash	text	No		SHA-256 of the 8-character user code.
poll_secret_hash	text	No		SHA-256 of the secret only the device holds.

Column	Type	Null	Default	Description
hardware_id	text	No		Requesting install.
device_info	jsonb	No		Self-description sent by the device.
status	pairing_status	No	'pending'	Flow state.
expires_at	timestamptz	No		300 seconds after creation (PAIRING_TTL_S).
last_polled_at	timestamptz	Yes		Enforces the 3 second poll interval.
approved_by	uuid	Yes		FK users.id.
approved_at	timestamptz	Yes		
org_id	uuid	Yes		FK organizations.id, cascade. Set on approval.
site_id	uuid	Yes		FK sites.id, set null.
assignee_user_id	uuid	Yes		FK users.id. Operator to assign.
device_id	uuid	Yes		FK devices.id, set null. Device created or re-paired.
completed_at	timestamptz	Yes		
requester_ip	text	Yes		
created_at	timestamptz	No	now()	

Indexes and constraints: pairing_code_active_uq unique (code_hash) WHERE status IN ('pending','approved'); pairing_expires_idx on expires_at.

device_sessions

Table 92. device_sessions (rotating device refresh tokens)

Column	Type	Null	Default	Description
id	uuid	No	gen_random_uuid()	Primary key.
org_id	uuid	No		Tenant.
device_id	uuid	No		FK devices.id, cascade.
family_id	uuid	No		Rotation family.
token_hash	text	No		SHA-256 of the refresh token.
session_version	integer	No		Device session_version at issue; older versions are rejected.
expires_at	timestamptz	No		30 days after issue (TTL.deviceRefreshMs).
revoked_at	timestamptz	Yes		
rotated_at	timestamptz	Yes		
created_at	timestamptz	No	now()	
last_used_at	timestamptz	Yes		

Indexes and constraints: device_sessions_token_uq unique on token_hash; device_sessions_device_idx.

ui_profiles

Table 93. ui_profiles (containers for versioned glasses UI configuration)

Column	Type	Null	Default	Description
id	uuid	No	gen_random_uuid()	Primary key.
org_id	uuid	No		Tenant.
name	text	No		Display name.
description	text	Yes		
created_by	uuid	Yes		FK users.id.
archived_at	timestamptz	Yes		Archived profiles are skipped during resolution.
created_at	timestamptz	No	now()	
updated_at	timestamptz	No	now()	

Indexes and constraints: ui_profiles_org_idx.

ui_profile_versions

Table 94. ui_profile_versions (immutable published configurations)

Column	Type	Null	Default	Description
id	uuid	No	gen_random_uuid()	Primary key.
org_id	uuid	No		Tenant.
profile_id	uuid	No		FK ui_profiles.id, cascade.
version	integer	No		Monotonic per profile. Rollback creates a new draft from an old version.
config	jsonb	No		UiProfileConfig, normalised by normalizeUiProfileConfig().
status	text	No	'draft'	draft, published or superseded; one published version per profile.
notes	text	Yes		Change note.
created_by	uuid	Yes		FK users.id.
created_at	timestamptz	No	now()	
published_by	uuid	Yes		FK users.id.
published_at	timestamptz	Yes		

Indexes and constraints: ui_profile_versions_uq unique (profile_id, version); ui_profile_versions_status_idx (profile_id, status).

ui_profile_assignments

Table 95. ui_profile_assignments (scope bindings; device beats user beats role beats org)

Column	Type	Null	Default	Description
id	uuid	No	gen_random_uuid()	Primary key.

Column	Type	Null	Default	Description
org_id	uuid	No		Tenant.
profile_id	uuid	No		FK ui_profiles.id, cascade.
scope	text	No		org, role, user or device.
role	role	Yes		Set when scope = 'role'.
user_id	uuid	Yes		FK users.id, cascade. Set when scope = 'user'.
device_id	uuid	Yes		FK devices.id, cascade. Set when scope = 'device'.
created_by	uuid	Yes		FK users.id.
created_at	timestamptz	No	now()	

Indexes and constraints: four partial unique indexes, one binding per subject: ui_assign_org_uq (org_id) WHERE scope='org', ui_assign_role_uq (org_id, role) WHERE scope='role', ui_assign_user_uq (org_id, user_id) WHERE scope='user', ui_assign_device_uq (org_id, device_id) WHERE scope='device'.

4.4.7 Audit document tables

audit_documents

Table 96. audit_documents (scanned paper records and their extraction)

Column	Type	Null	Default	Description
id	uuid	No	gen_random_uuid()	Primary key.
org_id	uuid	No		Tenant.
site_id	uuid	Yes		FK sites.id, set null. Defaults to the first permitted site.
asset_id	uuid	Yes		FK assets.id, set null.
title	text	No		Given, guessed from page 1 (guessTitle()), or Audit scan <date>.
doc_type	text	No	'general'	Document class; may be refined by the vision engine.
status	audit_doc_status	No	'in_review'	Review workflow state.
page_count	integer	No	0	Number of pages (1 to 6).
extraction	jsonb	No		AuditExtraction: fields, tables, figures, page text, warnings.
engines	jsonb	No	'[]'	Engines that contributed, for example ["mlkit", "vision:<model>"].
ai_status	text	No	'none'	none, pending, done, failed, unavailable.
ai_error	text	Yes		Last vision-engine error.
captured_by	uuid	Yes		FK users.id.
captured_via	text	No		glasses, console or mobile.
device_id	uuid	Yes		FK devices.id, set null.
verified_by	uuid	Yes		FK users.id. Field reviewer.

Column	Type	Null	Default	Description
verified_at	timestamptz	Yes		
approved_by	uuid	Yes		FK users.id. Must differ from verified_by outside demo tenants.
approved_at	timestamptz	Yes		Also set on rejection.
rejected_reason	text	Yes		Required on rejection.
revision	integer	No	1	Optimistic lock; every write checks and increments it.
created_at	timestamptz	No	now()	
updated_at	timestamptz	No	now()	

Indexes and constraints: audit_documents_org_idx (org_id, created_at).

audit_document_pages

Table 97. audit_document_pages (page images and raw OCR)

Column	Type	Null	Default	Description
id	uuid	No	gen_random_uuid()	Primary key.
org_id	uuid	No		Tenant.
document_id	uuid	No		FK audit_documents.id, cascade.
page_no	integer	No		1-based page number.
mime	text	No		image/jpeg or image/png; magic bytes checked.
data	text	No		Base64 image, under 900 KB decoded per page.
width	integer	No		Pixels, 50 to 6,000.
height	integer	No		Pixels, 50 to 6,000.
ocr	jsonb	Yes		{ "readings": OcrReading[] }, one to three device OCR passes.
created_at	timestamptz	No	now()	

Indexes and constraints: audit_document_pages_uq unique (document_id, page_no).

audit_document_events

Table 98. audit_document_events (append-only review trail; SELECT and INSERT only)

Column	Type	Null	Default	Description
id	uuid	No	gen_random_uuid()	Primary key.
org_id	uuid	No		Tenant.
document_id	uuid	No		FK audit_documents.id, cascade.
user_id	uuid	Yes		FK users.id.
device_id	uuid	Yes		FK devices.id, set null.

Column	Type	Null	Default	Description
action	text	No		captured, enhanced, item.confirm, item.correct, item.flag, item.reset, items.confirm_agreed, review.reopened, verified, approved, rejected, renamed.
target	text	Yes		Field or table id for item actions.
before	jsonb	Yes		Item or title before the change.
after	jsonb	Yes		Item, title or summary after the change.
note	text	Yes		Reviewer note.
created_at	timestamptz	No	now()	

Indexes and constraints: audit_document_events_doc_idx (document_id, created_at).

4.4.8 Platform tables

audit_events

Table 99. audit_events (append-only audit trail; UPDATE and DELETE revoked)

Column	Type	Null	Default	Description
id	uuid	No	gen_random_uuid()	Primary key.
org_id	uuid	Yes		FK organizations.id, cascade. Null for platform-level events.
actor_user_id	uuid	Yes		Acting person. No FK, so the trail survives user deletion.
actor_device_id	uuid	Yes		Acting glasses. No FK.
actor_label	text	Yes		Non-person actor (alarm engine, glasses).
action	text	No		Dotted verb, for example alarm.triggered, auditdoc.verified, device.revoke.
entity_type	text	Yes		Affected entity class.
entity_id	text	Yes		Affected id (text, so non-uuid ids fit).
summary	text	Yes		Human-readable one-liner.
metadata	jsonb	Yes		Structured detail, for example field-level changes.
ip	text	Yes		Request IP.
at	timestamptz	No	now()	

Indexes and constraints: audit_org_at_idx (org_id, at); audit_entity_idx (entity_type, entity_id).

notifications

Table 100. notifications (per-user in-app notifications)

Column	Type	Null	Default	Description
id	uuid	No	gen_random_uuid()	Primary key.
org_id	uuid	No		Tenant.

Column	Type	Null	Default	Description
user_id	uuid	No		FK users.id, cascade. Recipient.
kind	text	No		inspection_assigned, inspection_submitted, and similar.
severity	text	No	'info'	info, success, warning, critical.
title	text	No		
body	text	Yes		
link	text	Yes		Console path.
entity_type	text	Yes		
entity_id	uuid	Yes		No FK.
read_at	timestamptz	Yes		Null while unread.
created_at	timestamptz	No	now()	

Indexes and constraints: notifications_user_idx (user_id, read_at, created_at).

realtime_events

Table 101. realtime_events (durable SSE event log for serverless mode; system context only)

Column	Type	Null	Default	Description
id	bigint	No	identity	Primary key and SSE cursor (Last-Event-ID).
org_id	uuid	No		FK organizations.id, cascade.
type	text	No		Event type, for example telemetry.reading, alarm.updated, auditdoc.updated.
site_id	uuid	Yes		Site filter. No FK.
permission	text	Yes		Permission a subscriber needs to receive the event.
user_ids	jsonb	Yes		Array of recipient user ids, or null for everyone permitted.
device_ids	jsonb	Yes		Array of recipient device ids.
payload	jsonb	Yes		Event body.
created_at	timestamptz	No	now()	Pruned after one day.

Indexes and constraints: realtime_events_created_idx on created_at. Written only when REALTIME_MODE=db (always in demo mode on serverless).

simulation_runs

Table 102. simulation_runs (demo scenario runs)

Column	Type	Null	Default	Description
id	uuid	No	gen_random_uuid()	Primary key.
org_id	uuid	No		Tenant (demo organizations only).
site_id	uuid	No		FK sites.id, cascade.

Column	Type	Null	Default	Description
scenario_key	text	No		Scenario from packages/domain/src/scenarios.ts (for example normal_operations).
status	text	No	'running'	running, stopped, completed.
step	integer	No	0	Last executed step; runs resume from step + 1 after a restart.
started_by	uuid	Yes		FK users.id.
started_at	timestamptz	No	now()	
stopped_at	timestamptz	Yes		

Indexes and constraints: simulation_runs_org_idx (org_id, status).

inquiries

Table 103. inquiries (public website contact and demo requests; platform-owned, no RLS)

Column	Type	Null	Default	Description
id	uuid	No	gen_random_uuid()	Primary key.
kind	text	No		contact or demo.
name	text	No		
email	text	No		
company	text	Yes		
job_title	text	Yes		
phone	text	Yes		
site_count	text	Yes		Self-reported range.
message	text	Yes		
consent	boolean	No		Must be true to submit.
consent_version	text	No		Version of the consent text shown.
ip_hash	text	Yes		SHA-256 of a secret prefix and the IP; the raw IP is not stored.
user_agent	text	Yes		Truncated to 200 characters.
status	text	No	'new'	new, contacted, closed, spam.
created_at	timestamptz	No	now()	

Indexes and constraints: none.

hydris_demo_kv

Table 104. hydris_demo_kv (demo-mode shared key/value store; created at runtime)

Column	Type	Null	Default	Description
k	text	No		Primary key. Keys: auditdoc:index:<org>, auditdoc:doc:<id>, pair:id:<id>, pair:code:<hash>, guest:<userId>.

Column	Type	Null	Default	Description
v	jsonb	No		Value, for example a full audit document snapshot with pages and events.
expires_at	timestamptz	No		Rows past this time are ignored on read; never deleted.

This table is created with `create table if not exists` by `PgShared` in `apps/api/src/core/demo-shared.ts` only when demo mode runs against a Postgres `DATABASE_URL`. It is not part of the Drizzle schema or the migrations and has no RLS. On Vercel without Postgres the same interface uses the Vercel Runtime Cache, and locally it uses process memory.

> **NOTE**
> In demo mode every serverless instance runs its own in-memory PGlite database seeded with identical deterministic ids. `hydris_demo_kv` (or the runtime cache) is how a pairing approved on one instance, a guest account, or a scanned audit document becomes visible on another. It is a demo convenience, not a production pattern.

4.5 Enumerations

4.5.1 Postgres enum types

All 23 enum types are generated from the arrays in `packages/domain/src`, so TypeScript and SQL share one list. Adding a value needs a migration (`ALTER TYPE ... ADD VALUE`); removing one needs a type rebuild.

Table 105. Postgres enum types, values and source

Enum type	Values	Used by	Source
role	org_admin, factory_manager, supervisor, field_operator, maintenance_engineer, viewer	memberships, invitations, ui_profile_assignments	roles.ts
membership_status	invited, active, deactivated	memberships	catalog.ts
facility_kind	etp, stp, wtp, cetp, other	facilities	catalog.ts
process_area_kind	intake, screening, equalization, primary_treatment, aeration, clarification, filtration, disinfection, sludge_handling, chemical_dosing, discharge, utilities, other	process_areas	catalog.ts
asset_type	pump, blower, tank, valve, mixer, clarifier, filter, dosing_pump, analyzer, motor, screen, diffuser, uv_unit, other	assets, documents, inspection_templates	catalog.ts
asset_status	operational, degraded, down, maintenance, standby, decommissioned	assets	catalog.ts
criticality	low, medium, high	assets	catalog.ts

Enum type	Values	Used by	Source
measurand	ph, temperature, dissolved_oxygen, turbidity, conductivity, flow_rate, tank_level, pressure, motor_current, vibration, runtime_hours, tss, cod, bod, ammonia, residual_chlorine	sensors	measurands.ts
measurement_method	continuous, lab, manual	sensors	schema.ts
sensor_quality	good, uncertain, bad	telemetry_readings, sensors	catalog.ts
telemetry_source	gateway, manual, lab, simulator	telemetry_readings, sensors	catalog.ts
alarm_severity	critical, high, medium, low	alarm_rules, alarm_events	alarms.ts
alarm_condition	high_high, high, low, low_low, stale	alarm_rules, alarm_events	alarms.ts
alarm_state	active, acknowledged, investigating, resolved, closed	alarm_events, alarm_event_history	alarms.ts
inspection_status	scheduled, in_progress, paused, submitted, approved, rejected, cancelled	inspections	workflows.ts
maintenance_status	open, assigned, in_progress, on_hold, completed, cancelled	maintenance_tasks	workflows.ts
maintenance_priority	low, medium, high, urgent	maintenance_tasks	workflows.ts
maintenance_kind	corrective, preventive, inspection_followup, alarm_followup	maintenance_tasks	workflows.ts
device_status	active, suspended, revoked	devices	devices.ts
pairing_status	pending, approved, completed, denied, expired, cancelled	pairing_challenges	devices.ts
document_kind	sop, manual, datasheet, safety, troubleshooting, other	documents	catalog.ts
document_status	draft, approved, archived	documents	catalog.ts
audit_doc_status	processing, in_review, verified, approved, rejected	audit_documents	schema.ts (mirrors AuditDocStatus in auditdoc.ts)

4.5.2 Text-typed value sets

These columns are text in SQL and constrained only by the TypeScript type and request validation (Zod). A direct SQL write can store any string.

Table 106. Text columns with a closed value set

Column	Allowed values
users.status	active, disabled
organizations.status	active, suspended
auth_tokens.purpose	password_reset, email_verify

Column	Allowed values
gateways.protocol	https, mqtt, opcua
gateways.status	active, revoked
telemetry_corrections.action	correct, invalidate
inspection_templates.status	draft, published, archived
inspections.priority	normal, high
inspections.submitted_via	console, glasses
inspection_responses.outcome	ok, abnormal, na
attachments.owner_type	inspection, maintenance, asset, document, alarm
devices.kind	hardware, simulator
notifications.severity	info, success, warning, critical
inquiries.kind / inquiries.status	contact, demo / new, contacted, closed, spam
simulation_runs.status	running, stopped, completed
ui_profile_versions.status	draft, published, superseded
ui_profile_assignments.scope	org, role, user, device
audit_documents.ai_status	none, pending, done, failed, unavailable
audit_documents.captured_via	glasses, console, mobile

> **RISK**
> Text-typed sets drift silently: a typo in a script or a manual fix is accepted by Postgres. Planned change: add CHECK constraints for each set above in one migration (cheap, non-blocking with `NOT VALID` then `VALIDATE`).

4.6 JSON column schemas

The 23 JSONB columns are typed in TypeScript (`$type<...>()`) and validated with Zod at the API boundary, but not in SQL. This section documents their shapes with realistic examples taken from the seed data and the domain code.

4.6.1 Inspection steps and step values

`inspection_templates.steps` and the snapshot in `inspections.steps` hold `InspectionStep[]` (`packages/domain/src/workflows.ts`). Step kinds are `check`, `reading`, `choice`, `text` and `photo`.

```
[
  { "id": "guard", "kind": "check", "title": "Coupling guard fitted and secure", "required": true,
    "caution": "Never remove guards from running equipment." },
  { "id": "leak", "kind": "choice", "title": "Mechanical seal condition", "required": true,
    "options": ["No leakage", "Weeping", "Active leak"], "abnormalOptions": ["Active leak"] },
  { "id": "vib", "kind": "reading", "title": "Bearing vibration (handheld)", "measurand": "vibration",
    "unit": "mm/s", "required": true, "expectedMax": 4.5,
    "instruction": "Measure at the drive-end bearing, horizontal direction." }
]
```

`inspection_responses.value` holds one `StepValue`, discriminated by `kind`, which must match the step kind:

```
{ "kind": "check", "checked": true }
{ "kind": "reading", "value": 5.2 }
{ "kind": "choice", "option": "Weeping" }
{ "kind": "text", "text": "Slight odour near the seal" }
{ "kind": "photo", "attachmentId": "7f3c1b2e-5d4a-4c61-9a0e-2b8f6d1c9e44" }
```

`stepOutcome()` derives `outcome`: a reading outside `expectedMin`/`expectedMax`, an option in `abnormalOptions`, or an unchecked (or explicitly abnormal) check is `abnormal`. The reading above (5.2 against `expectedMax` 4.5) is abnormal, and submitting an inspection with abnormal outcomes creates an `inspection_followup` maintenance task.

4.6.2 Maintenance checklist

```
[
  { "id": "a", "label": "Isolate blower (SAFE-01)", "done": true, "doneAt": "2026-10-21T06:12:00Z",
"doneBy": "<user uuid>" },
  { "id": "b", "label": "Replace filter element", "done": false },
  { "id": "c", "label": "Record differential after restart", "done": false }
]
```

4.6.3 Audit extraction

`audit_documents.extraction` holds an `AuditExtraction` (`packages/domain/src/auditdoc.ts`). Bounding boxes are normalised `[x, y, width, height]` in 0..1 with the origin at the top left. Each field keeps every engine reading in `sources`, the computed `agreement`, the review `status`, and the first extracted value in `extracted`, so a correction never overwrites history.

```
{
  "title": "ETP daily log sheet",
  "docType": "general",
  "summary": "",
  "pages": [{ "page": 1, "text": "ETP DAILY LOG\nOutlet pH: 7.4\n...", "engine": "mlkit" }],
  "fields": [{
    "id": "p1-outlet-ph", "label": "Outlet pH", "value": "7.4", "unit": null, "type": "number",
    "page": 1, "bbox": [0.05, 0.13, 0.31, 0.03], "conf": 0.93,
    "sources": [
      { "engine": "mlkit", "value": "7.4", "conf": 0.93 },
      { "engine": "mlkit-contrast", "value": "7.4", "conf": 0.9 }
    ],
    "agreement": "match", "status": "confirmed", "extracted": "7.4",
    "reviewedBy": "<user uuid>", "reviewedAt": "2026-10-21T09:40:00Z", "note": null
  }],
  "tables": [{
    "id": "p1-t1", "title": null, "page": 1, "bbox": [0.05, 0.4, 0.9, 0.3],
    "header": ["Parameter", "Inlet", "Outlet"],
    "rows": [["pH", "7.9", "7.4"], ["COD mg/L", "412", "180"], ["Temperature", "31", "71"]],
    "conf": 0.88, "engine": "mlkit", "status": "pending",
    "issues": [{ "row": 2, "col": 2, "message": "Temperature: \"71\" - temperature outside -20..120",
"suggestion": null }]
  }],
  "figures": [{ "id": "p1-g1", "page": 1, "bbox": [0.7, 0.85, 0.2, 0.08], "kind": "signature",
"caption": "Operator signature" }],
  "warnings": []
}
```

Table 107. Value sets inside AuditExtraction

Property	Values	Meaning
`fields[].type`	text, number, date, id, checkbox, signature	Inferred by `inferType()`.
`fields[].agreement`	match, close, mismatch, single	Cross-engine comparison; numbers must match exactly after look-alike folding.
`fields[].status`, `tables[].status`	pending, confirmed, corrected, flagged	Human review state per item.
`figures[].kind`	photo, diagram, chart, signature, stamp, logo, other	Detected regions.

The engine names shown (`mlkit`, `mlkit-contrast`) are illustrative; the stored value is whatever the capturing client sends in `readings[].engine` (at most 40 characters). The vision engine is stored as `vision:<model>` (`VISION_ENGINE` in `apps/api/src/core/vision-extract.ts`).

4.6.4 OCR readings per page

`audit_document_pages.ocr` stores the raw device readings so the server can re-structure or merge them later (the vision merge reads them back). Request validation caps each page at 3 readings, 600 lines per reading and 400 words per line.

```
{
  "readings": [{
    "engine": "mlkit",
    "text": "ETP DAILY LOG\nOutlet pH: 7.4",
    "conf": 91.5,
    "lines": [{
      "text": "Outlet pH: 7.4", "conf": 93, "bbox": [0.05, 0.13, 0.31, 0.03],
      "words": [
        { "text": "Outlet", "conf": 95, "bbox": [0.05, 0.13, 0.1, 0.03] },
        { "text": "pH:", "conf": 92, "bbox": [0.16, 0.13, 0.05, 0.03] },
        { "text": "7.4", "conf": 93, "bbox": [0.22, 0.13, 0.04, 0.03] }
      ]
    }]
  }]
}
```

NOTE

OCR confidences in `ocr` are on a 0 to 100 scale (as engines report them); the structuring code divides by 100, so `conf` inside `extraction` is 0 to 1.

4.6.5 UI profile configuration

`ui_profile_versions.config` holds a `UiProfileConfig` (`packages/domain/src/uiprofile.ts`, schema version 1). Any input is normalised before storage: unknown ids are dropped, missing ids are appended in default order, `home` and `status` screens are always enabled, and `textScale` is clamped to 0.85 to 1.4.

```
{
  "schemaVersion": 1,
  "screens": [
    { "id": "home", "enabled": true }, { "id": "alerts", "enabled": true },
    { "id": "inspections", "enabled": true }, { "id": "map", "enabled": false },
    { "id": "ai", "enabled": true }, { "id": "status", "enabled": true }
  ],
  "widgets": [
    { "id": "alarm_summary", "visible": true }, { "id": "next_inspection", "visible": true },
    { "id": "measurements_glance", "visible": true }, { "id": "task_progress", "visible": false }
  ],
  "measurements": {
    "allow": ["ph", "dissolved_oxygen", "vibration", "tank_level"],
    "orderByAssetType": { "pump": ["vibration", "pressure", "motor_current"] }
  },
  "actions": {
    "acknowledge_alarm": true, "record_inspection": true, "submit_inspection": true,
    "toggle_checklist": true, "ask_ai": true, "view_sops": true, "view_map": false,
    "mark_notifications_read": true
  },
  "layout": { "density": "compact", "textScale": 1.15, "handedness": "right" }
}
```

The example lists only some screens for brevity; normalisation always stores all twelve (`home`, `alerts`, `inspections`, `tasks`, `map`, `measurements`, `assets`, `sops`, `ai`, `scan`, `notifications`, `status`).

4.6.6 Smaller JSON columns

Table 108. Smaller JSONB columns and their shapes

Column	Shape	Example
`invitations.site_ids`	`string[]` of site uuids	`["2b1c...", "9e4f..."]`
`process_areas.polygon`	`[number, number][]`, metres on the site layout	`[[6,10],[44,10],[44,40],[6,40]]`
`assets.specs`	`Record<string, string or number>`	`{"Rated flow": "140 m³/h", "Motor": "15 kW / 28 A"}`
`documents.tags`	`string[]`	`["vib"]`
`inspection_templates.safety_notices`, `inspections.safety_notices`	`string[]`	`["Hearing protection required in the pump house."]`
`inspection_routes.asset_ids`	ordered `string[]` of asset uuids	
`devices.display_info`	`{ width, height }` or null	`{"width": 640, "height": 360}`
`pairing_challenges.device_info`	`{ kind, model?, appVersion?, firmwareVersion?, display?, name? }`	`{"kind": "hardware", "model": "Ray-Ban Meta", "appVersion": "1.4.0"}`
`audit_documents.engines`	`string[]`	`["mlkit", "vision:<model>"]`
`audit_document_events.before` / `after`	the item, title or summary before and after	`{"pages": 2, "fields": 14, "tables": 1}`
`audit_events.metadata`	free-form object	`{"changes": {"status": {"from": "operational", "to": "degraded"}}}`
`realtime_events.user_ids`, `device_ids`	`string[]` or null	
`realtime_events.payload`	event body	`{"sensorId": "...", "value": 2.61, "unit": "mm/s", "ts": "...", "quality": "good", "source": "simulator", "synthetic": true}`

4.7 State models

Status columns are guarded by transition tables in `packages/domain/src` and checked inside the same transaction that writes the new state. Every transition writes a history row.

4.7.1 Alarm events

`alarm_events.state` is the human lifecycle. The measured condition is tracked separately in `cleared_at`, so an alarm whose value has returned to normal stays open until people resolve it (`ALARM_TRANSITIONS` in `alarms.ts`).

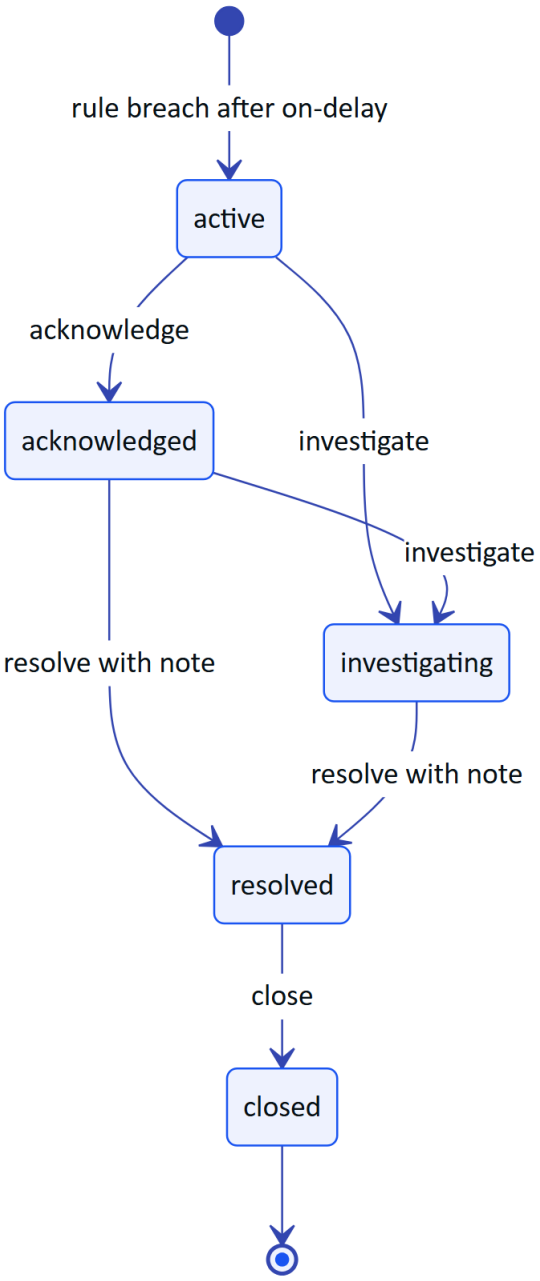

Figure 87. *Alarm event lifecycle (alarm_state); cleared_at records the measured condition independently*

Table 109. Alarm transitions, permissions and side effects

Action	From	To	Permission	Side effects
(engine) trigger	none	active	alarm engine	Insert `alarm_events`, history `triggered`, `audit_events` `alarm.triggered`.
(engine) clear	open states	unchanged	alarm engine	Set `cleared_at`; history `condition_cleared`.
acknowledge	active	acknowledged	`alarm.acknowledge`	Set `acknowledged_by/at`.
investigate	active, acknowledged	investigating	`alarm.investigate`	
resolve	acknowledged, investigating	resolved	`alarm.resolve`	Note required, stored in `resolution_note`.

Action	From	To	Permission	Side effects
close	resolved	closed	`alarm.close`	Set `closed_by/at`.

While an alarm is open and not cleared, a worse value updates `peak_value`. A new alarm for the same rule is only raised after the previous one has cleared.

4.7.2 Inspections

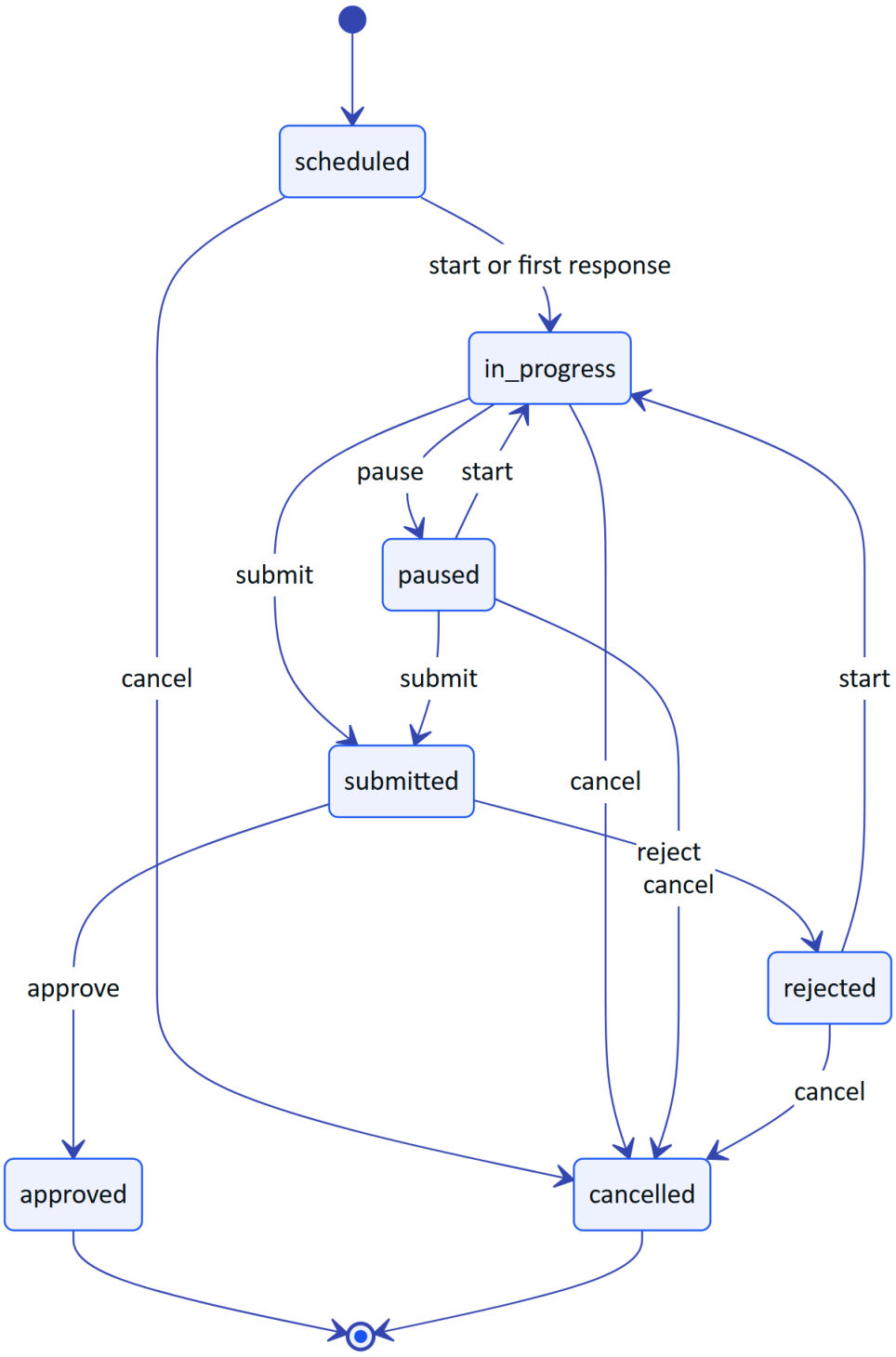

Figure 88. *Inspection lifecycle (inspection_status) from INSPECTION_TRANSITIONS*

Responses are accepted in `scheduled`, `in_progress`, `paused` and `rejected` (`INSPECTION_EDITABLE`). Every state change increments `revision`; a client that sends a stale `expectedRevision` receives `revision_conflict`. The UI labels `rejected` as "Returned".

4.7.3 Maintenance tasks

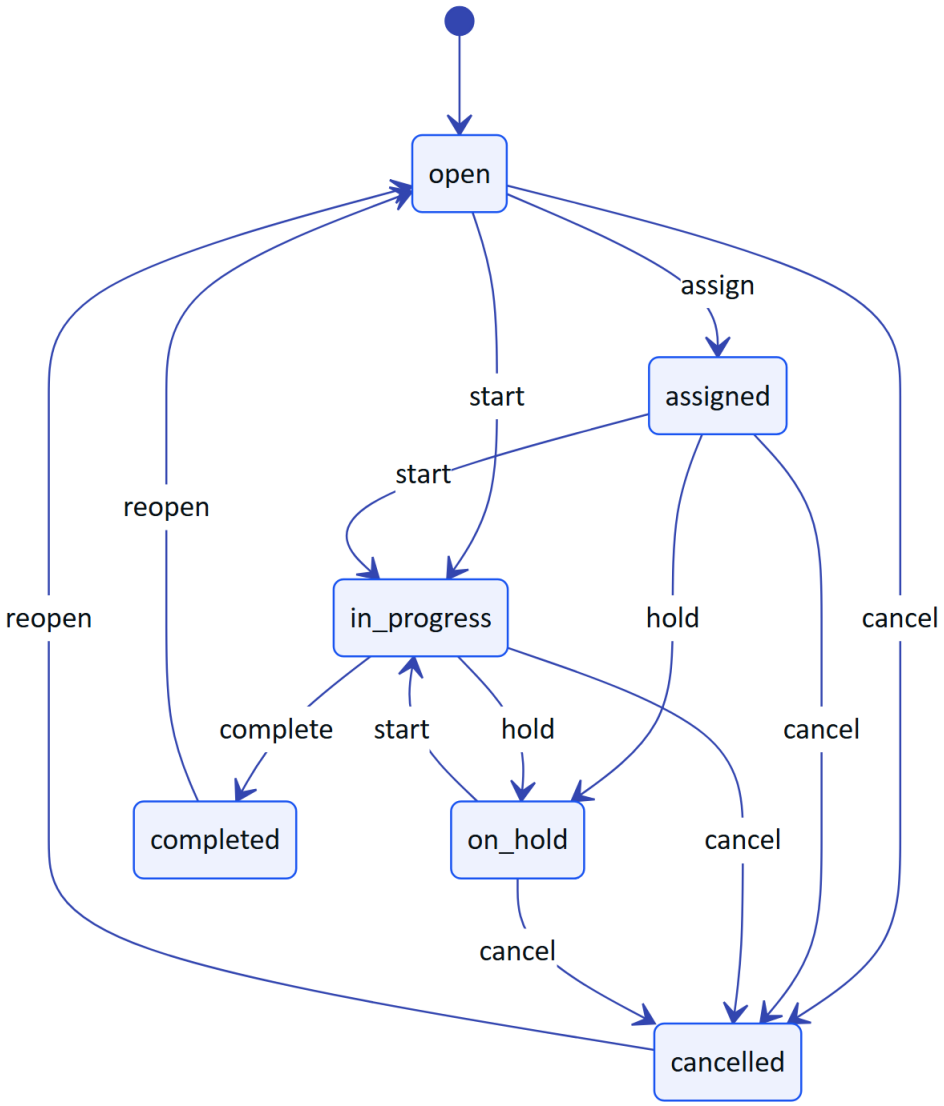

Figure 89. *Maintenance task lifecycle (maintenance_status) from MAINTENANCE_TRANSITIONS*

4.7.4 Audit documents

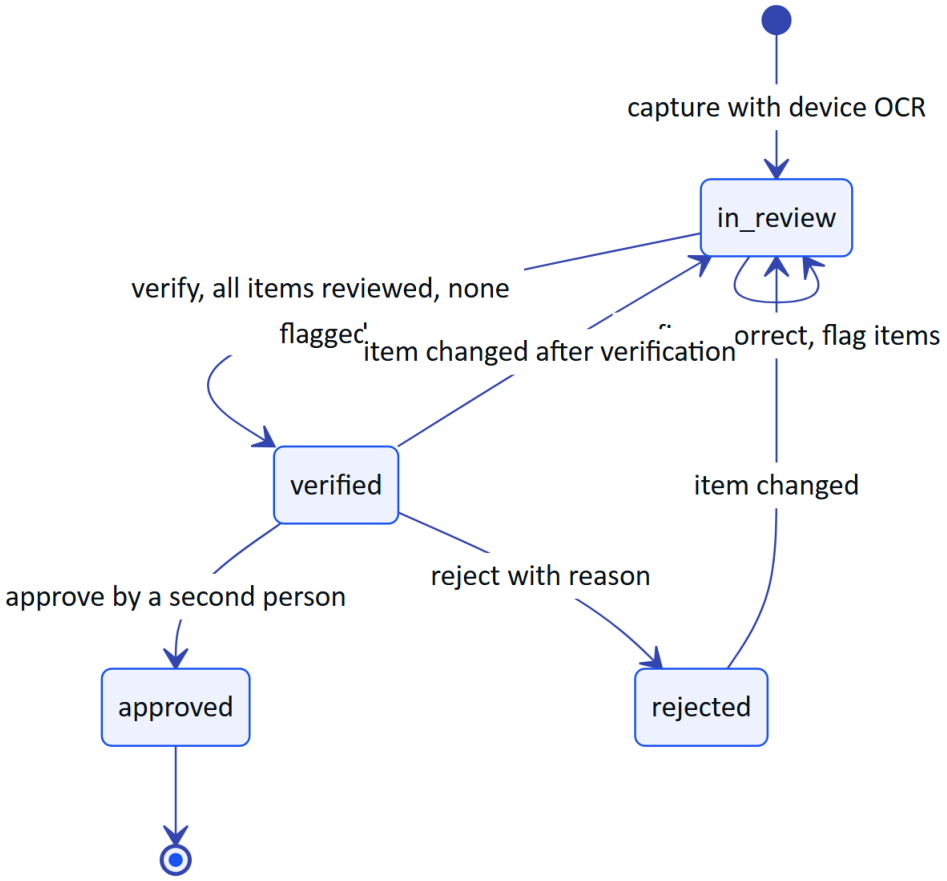

Figure 90. *Audit document review lifecycle (audit_doc_status); processing is reserved and not yet written*

Table 110. Audit document rules enforced in apps/api/src/modules/audit-docs.ts

Rule	Enforcement
Capture starts in `in_review`	Insert sets `status = 'in_review'`. `processing` exists in the enum and the list filter but is never written.
Vision refinement only before review starts	`enhance` returns `review_started` once any item is confirmed or corrected.
Verification needs a complete review	`reviewProgress()` must report no pending and no flagged items.
Verification and approval are console actions	Device principals receive `forbidden`.
Four-eyes approval	`approved_by` must differ from `verified_by`, except in demo tenants (recorded as self-approved in the event note).
Approved documents are locked	Item edits and renames return `locked`.
Editing after verification reopens the review	Status returns to `in_review`; verifier and approver fields are cleared; event `review.reopened`.
Concurrent edits	Every update matches on `revision` and increments it; a mismatch returns `stale`.

4.7.5 Pairing challenges and devices

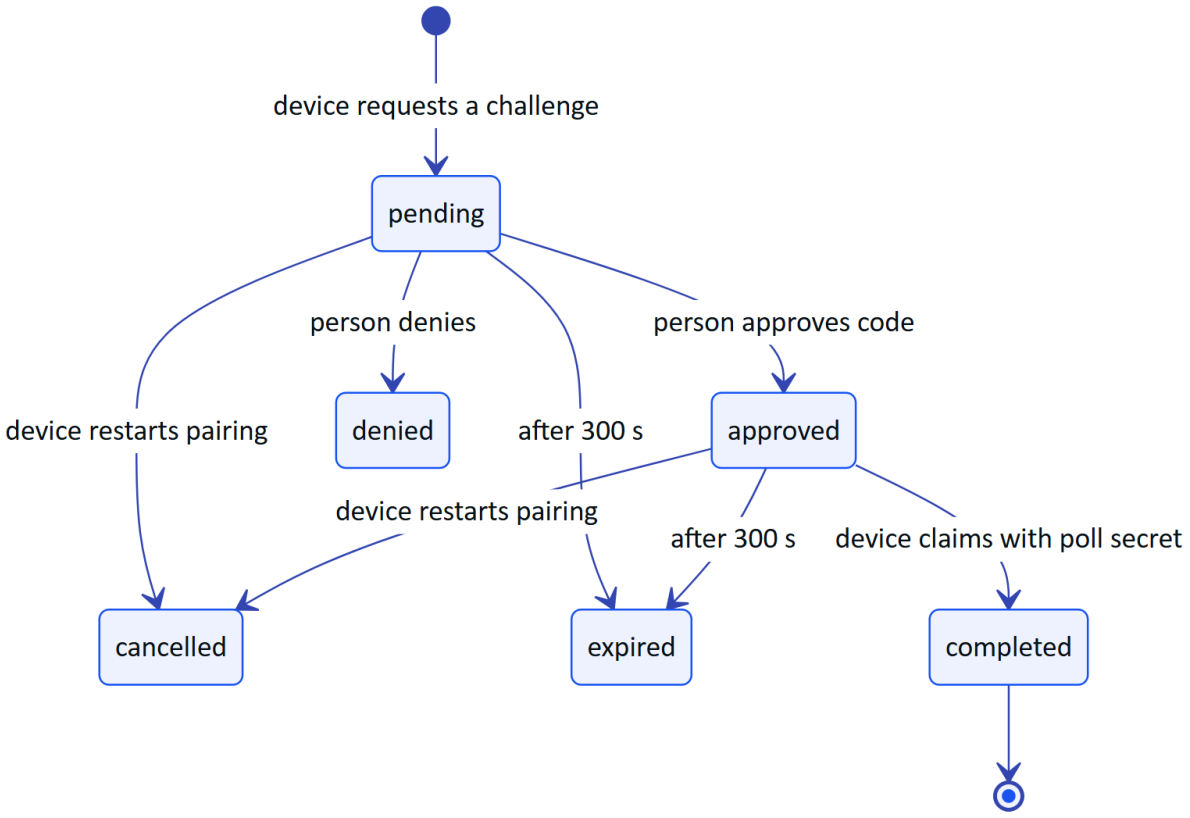

Figure 91. *Pairing challenge lifecycle (pairing_status), modelled on the OAuth device authorization grant*

`devices.status` has only three stored values (active, suspended, revoked). The console shows a richer composite state computed by `deriveDeviceState()` in `packages/domain/src/devices.ts`:

Table 111. Composite device state derived from stored columns

Displayed state	Derived from
revoked	`status = 'revoked'`
suspended	`status = 'suspended'`
connected	active, current assignment, and an open stream or `last_seen_at` within 90 s
paired	active, no current assignment
assigned	active, assigned, never seen
disconnected	active, assigned, last seen more than 90 s ago
unregistered, pending_pairing	client-side states before a `devices` row exists

4.8 Tenancy and row-level security

4.8.1 How organization context is set

Every request runs in a database transaction that first drops to the role `hydris_app` and then sets a transaction-local setting. `withTenant()` sets `app.org_id`; `withSystem()` sets `app.bypass = 'on'` for the few

operations that legitimately span tenants. Both use `set_config(..., true)` and `SET LOCAL`, so the context ends with the transaction and cannot leak to the next request on a pooled connection.

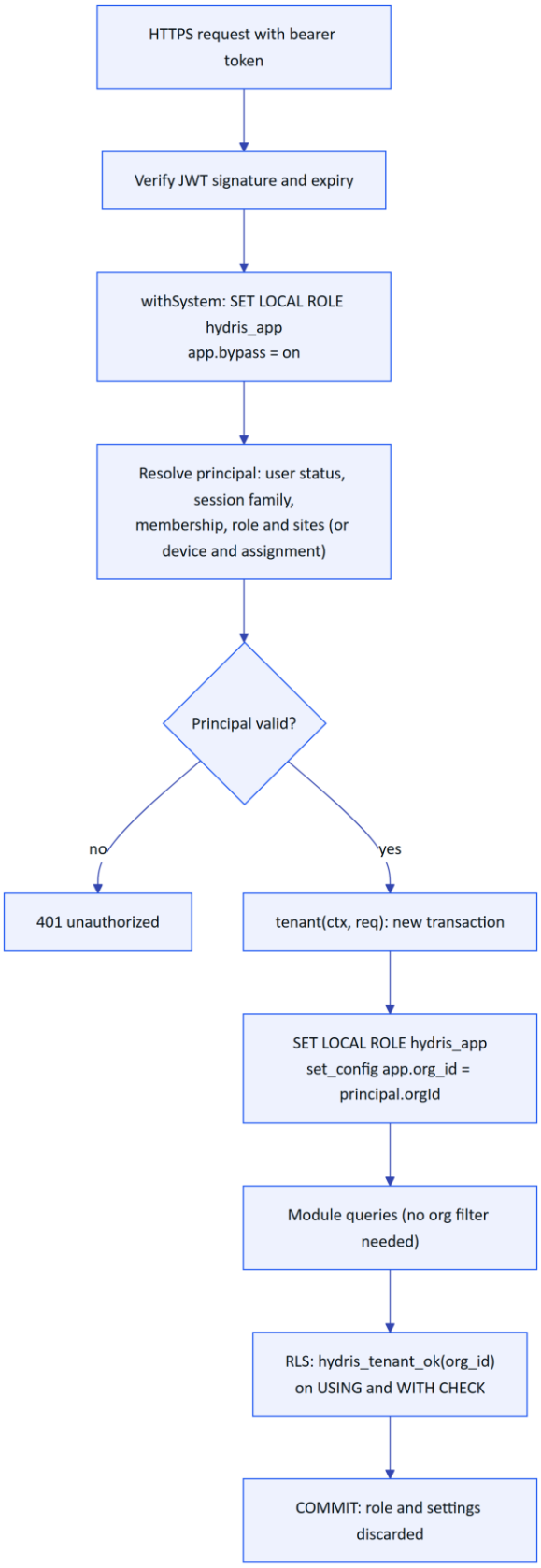

Figure 92. *How the organization context is set for each API request*

The policy function from `0001_rls.sql`:

```
CREATE OR REPLACE FUNCTION hydris_tenant_ok(row_org uuid) RETURNS boolean
LANGUAGE sql STABLE AS $$
  SELECT current_setting('app.bypass', true) = 'on'
      OR row_org = nullif(current_setting('app.org_id', true), '')::uuid
$$;
```

If neither setting is present, `current_setting(..., true)` returns null, the comparison is null, and the row is invisible. A request that forgets to set context therefore sees nothing rather than everything. `FORCE ROW LEVEL SECURITY` makes the policy apply to the table owner as well.

4.8.2 Policy inventory

Table 112. Row-level security and grants per table

Policy or grant	Tables	Migration
`tenant_isolation` on `org_id`	memberships, membership_sites, invitations, sites, facilities, process_areas, assets, gateways, sensors, telemetry_readings, alarm_rules, alarm_events, alarm_event_history, documents, inspection_templates, inspection_routes, inspections, inspection_responses, attachments, maintenance_tasks, maintenance_activity, devices, device_assignments, device_sessions, notifications, audit_events, simulation_runs	0001
`tenant_isolation` on `id`	organizations	0001
`tenant_isolation` on `org_id`	telemetry_corrections, ui_profiles, ui_profile_versions, ui_profile_assignments	0003
`system_only` (`app.bypass = 'on'`)	realtime_events	0006
`tenant_isolation` on `org_id`	audit_documents, audit_document_pages, audit_document_events	0008
No RLS, system context only	users, auth_sessions, auth_tokens, mail_outbox, pairing_challenges, inquiries	n/a
UPDATE and DELETE revoked from `hydris_app`	audit_events (0001), telemetry_corrections (0003)	0001, 0003
SELECT and INSERT only for `hydris_app`	audit_document_events	0008
SELECT, INSERT, DELETE only	realtime_events	0006

Because `withSystem()` also runs as `hydris_app`, the append-only grants hold even for background jobs: no code path in the application can rewrite the audit trail.

> **WARNING**
> Superuser and owner connections bypass RLS. Production must connect as a role that is a member of `hydris_app` with `SET` rights (`0004_app_role_membership.sql` and `0006_realtime_events_rls.sql` grant this on PG16+ managed Postgres). Ad hoc SQL consoles connected as the owner see every tenant.

4.8.3 Checklist for a new tenant table

1. Add `orgId: orgRef()` as a column in `schema.ts`.
2. Generate the table migration with `drizzle-kit generate`.

3. Write a second, hand-written migration that grants `hydris_app` the needed privileges, enables and forces RLS, and creates `tenant_isolation` with `hydris_tenant_ok(org_id)` (copy `0008_audit_documents_rls.sql`).
4. Revoke UPDATE and DELETE if the table is a log.
5. Add a test that reads the table from a second organization and expects zero rows.

4.8.4 Site scoping

RLS isolates organizations, not sites. Site restrictions (`memberships.all_sites = false` plus `membership_sites`) are enforced in application code by `requireSite()`, `siteScope()` and `scopedRow()` in `apps/api/src/core` and the modules. This is **Built** but not defended in depth by the database.

> **DECISION**
>
> Site scoping stays in the application for now: it changes per request and per role, and expressing it in policies would need a per-request site list in a setting. A **Planned** option is an `app.site_ids` setting and a second policy on site-bearing tables for the most sensitive data (telemetry, audit documents).

4.9 Module access patterns

Table 113. Which modules read and write which tables

Module	Writes	Reads	Context
`auth.ts`	users, auth_sessions, auth_tokens, memberships, membership_sites, mail_outbox (via `sendMail`)	users, memberships, organizations	system for sign-in, tenant for org switching
`org.ts`	memberships, membership_sites, invitations, organizations	audit_events, users	tenant
`plant.ts`	sites, facilities, process_areas, assets, gateways, sensors, documents, attachments	all plant tables	tenant
`telemetry.ts`	telemetry_readings, sensors (latest value), telemetry_corrections, gateways (`last_seen_at`)	sensors, readings, corrections	system to resolve gateway keys, then tenant
`alarms.ts`	alarm_rules, alarm_events, alarm_event_history	rules, events, sensors	tenant; system for stale-rule jobs
`inspections.ts`	inspections, inspection_responses, maintenance_tasks (follow-ups), notifications	templates, routes, assets	tenant
`maintenance.ts`	maintenance_tasks, maintenance_activity	tasks, assets	tenant
`devices.ts`	pairing_challenges, devices, device_assignments, device_sessions	memberships, sites	system for pairing, tenant for management
`ui-profiles.ts`	ui_profiles, ui_profile_versions, ui_profile_assignments	same	tenant

Module	Writes	Reads	Context
audit-docs.ts	audit_documents, audit_document_pages, audit_document_events	same, sites, users	tenant; system for demo hydration
insights.ts	none	documents (full text), assets, sensors, users, alarms, inspections	tenant
voice-assistant.ts	none	sensors, alarms, inspections, tasks, documents	tenant
simulation.ts	simulation_runs, telemetry (through ingest), maintenance_tasks, assets	scenarios	tenant
public.ts	inquiries	none	system
core/services.ts	audit_events, notifications, mail_outbox	none	caller's transaction
core/events.ts, realtime.ts	realtime_events	realtime_events	system
jobs.ts	pairing_challenges (expire), alarm tables (stale), realtime_events (prune)	simulation_runs	system

4.10 Indexing and query patterns

The 71 indexes are all B-tree. They follow the main read paths of the console and glasses:

Table 114. Main query patterns and the indexes that serve them

Query pattern	Example route or code	Index
Latest values per site	overview, glasses measurements	sensors_site_idx plus cached sensors.last_* (no time-series scan)
Sensor history for a range	GET /sensors/:id/history with date_bin buckets	telemetry_sensor_ts_uq and telemetry_sensor_ts_idx
Idempotent ingest	INSERT ... ON CONFLICT (sensor_id, ts) DO NOTHING	telemetry_sensor_ts_uq
Effective value with corrections	effectiveReadings() lateral join, newest correction first	telemetry_corrections_reading_idx
Open alarms for an org	alarm list, overview counters	alarm_events_org_state_idx
Open alarm for a rule during evaluation	evaluateSensor()	alarm_events_rule_idx
My inspections and tasks	glasses home, mobile work tab	inspections_assignee_idx, maintenance_assignee_idx
Latest response per step	inspection view	inspection_responses_step_idx
Unread notifications	bell counter	notifications_user_idx
Audit log by time or by entity	console audit view, entity history	audit_org_at_idx, audit_entity_idx
Realtime replay since cursor	SSE id > lastEventId	primary key on realtime_events.id
Token lookups	refresh, invite, reset, device refresh, gateway key	unique token_hash, key_hash, code_hash indexes

Known gaps, all **Planned**:

- **Full-text search** on `documents` computes `to_tsvector('english', title || ' ' || body)` on every query (`insights.ts`). Add a generated `tsvector` column with a GIN index once a tenant holds more than a few hundred documents.
- `audit_document_pages` has no `org_id` index; reads go through `document_id`, which is covered by the unique index, so this is acceptable.
- **Telemetry at scale** needs time partitioning (monthly) or TimescaleDB hypertables, plus BRIN on `ts` for archive partitions. See "Volumes and growth".
- `telemetry_org_ts_idx` serves organization-wide time scans (exports, reports). It costs one index entry per reading; review it once reports move to rollup tables.

> **TIP**
> RLS adds `hydris_tenant_ok(org_id)` to every scan. Indexes that lead with `org_id` (sites, assets, sensors, alarm events, inspections, maintenance, audit) let the planner combine the policy and the filter in one index range. Lead with `org_id` for new tenant indexes unless the first column is already tenant-unique.

4.11 Data quality rules

Data quality is enforced at three points: ingestion, the review workflows, and display. All rules below are **Built** unless marked otherwise.

4.11.1 Telemetry ingestion

`ingest()` in `apps/api/src/modules/telemetry.ts` validates every reading before storage and returns a per-item rejection reason. Accepted gateway batches hold up to 500 readings.

Table 115. Telemetry ingestion checks and rejection reasons

Check	Rule	Rejection reason
Synthetic isolation	`source = 'simulator'` only for `is_demo` organizations; also enforced by CHECK on `synthetic`	`synthetic_not_allowed`
Sensor exists in the organization, not archived	lookup by id within the tenant	`unknown_sensor`
Sensor enabled	`sensors.enabled`	`sensor_disabled`
Gateway site	a gateway may only write sensors on its own site	`sensor_not_on_gateway_site`
Clock skew	`ts` at most 5 minutes in the future	`timestamp_in_future`
Late data	`ts` at most 30 days old, except `lab`	`timestamp_too_old`
Finite value	not NaN or infinite	`not_finite`
Known unit	canonical or a listed alternative unit	`unknown_unit`
Physical plausibility	value within `validRange` after conversion	`out_of_range`
Duplicate	same `(sensor_id, ts)`	counted as duplicate, not an error

Table 116. Physical plausibility ranges in canonical units (MEASURAND_DEFS)

Measurand	Unit	Valid range	Typical band
ph	pH	0 to 14	6.5 to 8.5
temperature	°C	-20 to 120	10 to 35
dissolved_oxygen	mg/L	0 to 25	1.5 to 3
turbidity	NTU	0 to 4,000	0 to 10
conductivity	µS/cm	0 to 200,000	200 to 3,000
flow_rate	m³/h	0 to 100,000	none
tank_level	%	0 to 100	20 to 80
pressure	bar	-1 to 100	none
motor_current	A	0 to 2,000	none
vibration	mm/s	0 to 100	0 to 4.5
runtime_hours	h	0 to 1,000,000	none
tss	mg/L	0 to 50,000	none
cod	mg/L	0 to 100,000	none
bod	mg/L	0 to 50,000	none
ammonia	mg/L	0 to 2,000	0 to 5
residual_chlorine	mg/L	0 to 20	0.2 to 1

Out-of-order readings are stored for history but do not overwrite a newer cached latest value or re-trigger alarms. Readings with quality `bad` are stored but never evaluated against alarm rules.

4.11.2 Freshness

`freshness()` classifies the age of `sensors.last_ts` relative to `expected_interval_s`: **live** up to 2 times the interval, **delayed** up to 5 times, **stale** beyond, and **no_data** without a timestamp. Stale values must be visibly marked and are excluded from health roll-ups. Stale alarm rules (`condition = 'stale'`) are evaluated by a background job every 30 seconds on the long-running server, or per cron pass on serverless.

4.11.3 Corrections

A correction needs a reason of at least 5 characters, the corrected value is normalised and range-checked like an ingested reading, and the original reading row is never modified. The newest correction per reading defines the effective value; `invalidate` excludes the reading from charts, aggregates and the cached latest value (`recomputeLatest()`). CSV exports show original value, effective value, status, reason, author and time.

4.11.4 Inspections

Readings outside `expectedMin`/`expectedMax` and abnormal options set `outcome = 'abnormal'`. Response timestamps more than 5 minutes in the future and values whose kind does not match the step are rejected as conflicts.

4.11.5 Audit documents

Table 117. Audit document quality rules (packages/domain/src/auditdoc.ts)

Rule	Behaviour
Plausibility ranges	pH 0 to 14; dissolved oxygen 0 to 25 mg/L; temperature -20 to 120; percentages 0 to 100; turbidity 0 to 4,000 NTU. Out-of-range fields get confidence capped at 0.4 and a note.
Table cell checks	Numeric cells are range-checked against the row label; when a labelled field with the same digits passes, it is offered as a suggestion (catches a dropped decimal point).
Engine agreement	Numbers must match exactly after folding look-alike characters (O to 0, l to 1, S to 5); text may differ slightly (`close` at similarity 0.85 or more).
Needs attention	`mismatch` or confidence below 0.75 is shown to the reviewer first.
Bulk confirm	Only pending fields with more than one source, `match` agreement and confidence of at least 0.8.
Low-quality capture	A page with OCR confidence below 60 adds a warning to rescan.

RISK

Two engines can agree on a wrong value. Plausibility checks cover the common water-quality parameters only; COD, BOD and flow values in scanned sheets are not range-checked. **Planned**: reuse `MEASURAND_DEFS.validRange` for field labels that map to a measurand.

4.12 Data lifecycle and retention

4.12.1 Current behaviour

Only one table is pruned today: `realtime_events`, older than one day, by `pruneRealtimeEvents()` (hourly on the long-running server in db mode, each cron pass on serverless). Pairing challenges are marked `expired` every minute but not deleted. Everything else is kept indefinitely. Deleting an organization cascades to every tenant row.

4.12.2 Recommended policy

Retention should be configurable per organization, with defaults that meet typical record-keeping obligations for effluent and sewage treatment. The periods below are proposals, not legal advice; confirm each against the customer's consent-to-operate conditions and applicable law before enforcement.

Figure 93. *Recommended data lifecycle by age (Planned, except where marked Built)*

Table 118. Retention by table, current and recommended

Data	Current	Recommended (Planned)
`realtime_events`	Pruned after 1 day (**Built**)	Keep.
`telemetry_readings`	Indefinite	2 years online in monthly partitions, then archive (cold storage or compressed partitions) for at least 5 years; keep hourly rollups online.
`telemetry_corrections`	Indefinite	Same as the readings they correct.
`alarm_events`, `alarm_event_history`	Indefinite	At least 5 years; history follows its event.
`inspections`, `inspection_responses`, `maintenance_*`	Indefinite	At least 5 years, configurable.
`audit_documents`, `audit_document_pages`	Indefinite, images in Postgres	Approved records for the customer's record period (at least 5 years); page images in object storage with lifecycle rules; rejected scans 1 year.
`audit_events`, `audit_document_events`	Indefinite, append-only	At least 7 years, then export to WORM storage before deletion.
`notifications`	Indefinite	Read notifications 1 year.
`auth_sessions`, `device_sessions`, `auth_tokens`	Expired rows remain	Purge 30 days after expiry or revocation.
`pairing_challenges`	Expired rows remain	Purge 30 days after expiry.
`invitations`	Indefinite	Purge 90 days after expiry, acceptance or revocation.
`mail_outbox`	Indefinite, contains live links	Purge after send or 7 days.
`inquiries`	Indefinite	24 months, or on consent withdrawal (DPDP Act 2023, GDPR).
`simulation_runs`	Indefinite	90 days after stop.
`hydris_demo_kv`	Expired rows ignored, never deleted	Delete rows past `expires_at` daily.

> **IMPORTANT**
>
> Retention jobs must run in system context as `hydris_app`. Because UPDATE and DELETE are revoked on the audit logs, purging them needs a dedicated maintenance role and a documented, audited procedure. That is intentional.

4.13 Volumes and growth

4.13.1 Telemetry

Telemetry dominates storage. Rows per day per sensor follow directly from the reporting interval:

Table 119. Telemetry rows per sensor by reporting interval

Interval	Rows per sensor per day	Rows per sensor per year	Typical use
5 s	17,280	6.3 million	Demo simulator (`SIM_STEP_SECONDS = 5`)
10 s	8,640	3.2 million	Fast online analysers
60 s	1,440	525,600	Default `expected_interval_s`
300 s	288	105,120	Slow process values
1 day	1	365	Lab parameters (COD, BOD, TSS)

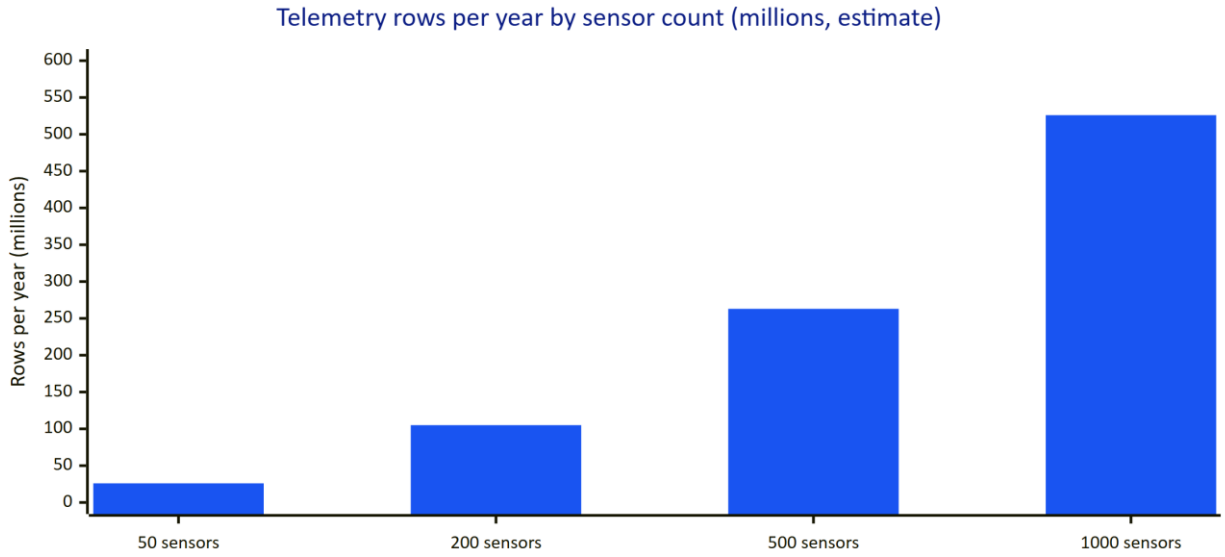

Figure 94. *Estimated telemetry rows per year at the default 60 s interval (estimate, not measured)*

Storage estimate: a reading row is about 100 to 130 bytes of heap (tuple header, `bigint`, two `uuid`, two `timestamptz`, `double`, two enums, `boolean` and a short `message_id`), plus about 40 to 50 bytes in each of the three indexes. Roughly **250 bytes per reading** in total is a reasonable planning figure (estimate). At 60 s that is about 130 MB per sensor-year, so a 200-sensor plant adds about 26 GB per year and 1,000 sensors about 130 GB per year. Dropping `message_id` for gateways that do not need it and moving cold months to compressed partitions are the main levers.

RISK

A single unpartitioned `telemetry_readings` table will slow down vacuum, index maintenance and range scans beyond a few hundred million rows. Partition by month before the first customer passes about 200 sensors at 60 s.

4.13.2 Other tables

Table 120. Growth estimates for one mid-size site (estimate, not measured)

Table	Driver	Estimate per site per year
`alarm_events` and history	5 to 20 alarms a day, 3 to 5 history rows each	2,000 to 7,000 events; 10,000 to 35,000 history rows
`inspections`, `inspection_responses`	10 to 30 inspections a day, 5 to 10 steps	4,000 to 11,000 inspections; 20,000 to 110,000 responses
`maintenance_tasks`	2 to 10 a day	700 to 3,700
`audit_events`	every write action	50,000 to 200,000
`realtime_events`	every published event, pruned daily	bounded at about one day of events
`audit_documents` and pages	5 to 20 scans a day, 1 to 3 pages	2,000 to 7,000 documents; 0.6 to 4 GB of base64 page data in Postgres

4.13.3 Demo tenant

The showcase seed produces a known volume, useful for performance tests: 27 simulated signals at 10-minute resolution over 7 days (27 × 1,008 = 27,216 readings), 30 lab readings, 2 sites, 15 assets and 30 sensors.

4.14 Binary and image storage

Two storage paths exist today:

- **Attachments** (`attachments`) store only metadata in Postgres. Bytes go to local disk (`STORAGE_DIR`) or a private Vercel Blob store (`attachments/<org>/<uuid>`) through `createStorage()` in `apps/api/src/core/services.ts`. Files are only served through the authorised `/api/attachments/:id` route. **Built**.
- **Audit page images** (`audit_document_pages.data`) are stored as base64 text inside Postgres, together with the OCR JSON. The device downscales scans (the schema comment targets about 600 KB) and the API rejects pages over 900 KB decoded and documents over 6 pages. **Built**.

RISK

Base64 in Postgres costs about 37 % more space than the binary, is mostly incompressible (JPEG), inflates backups and logical replication, and every `SELECT *` on pages drags megabytes through the pool. A six-page document can approach 7 MB of row data. In demo mode, the same snapshot is also copied into `hydris_demo_kv`.

4.14.1 Proposed migration to object storage

Planned. Move page images to the same storage adapter as attachments and keep only a reference in Postgres.

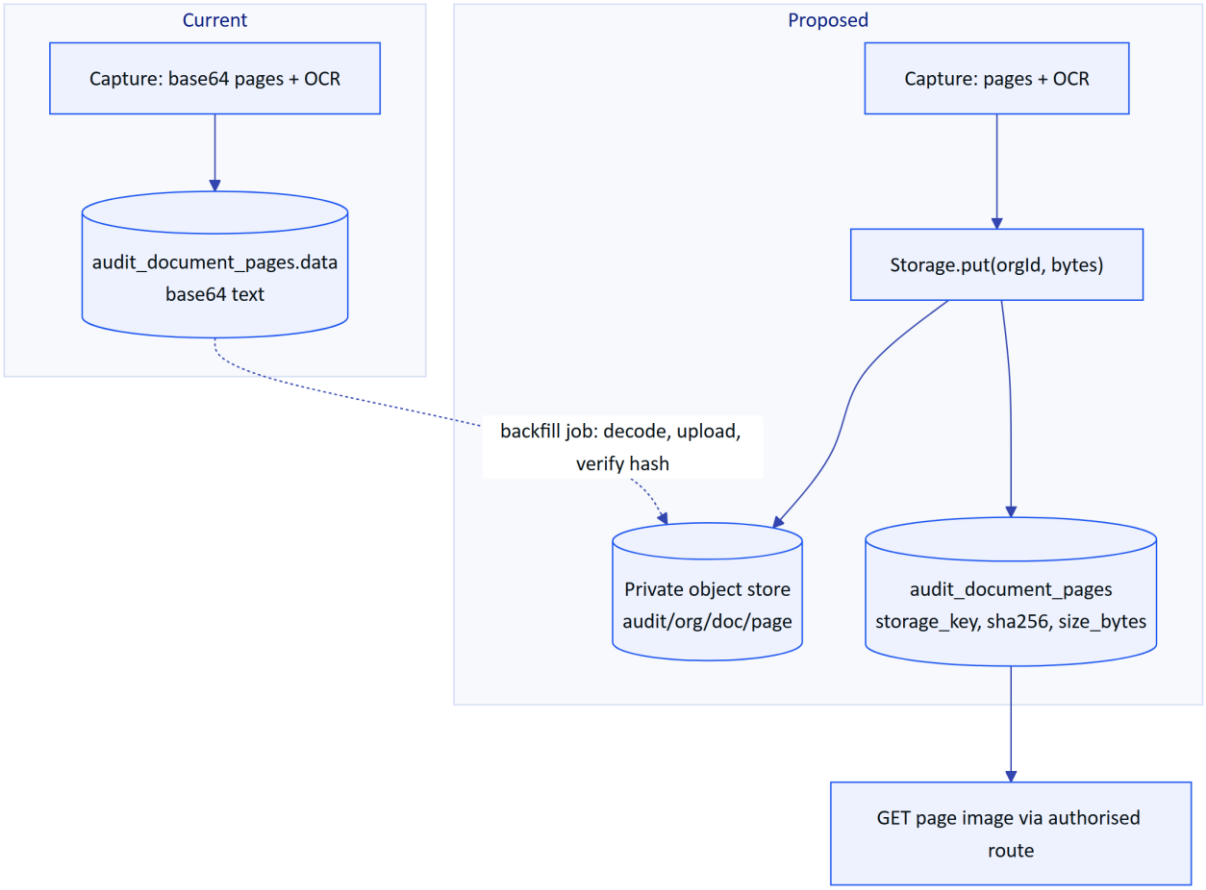

Figure 95. *Proposed migration of audit page images from Postgres to object storage (Planned)*

Migration steps:

1. Add nullable columns `storage_key text`, `sha256 text` and `size_bytes integer` to `audit_document_pages`; make `data` nullable.
2. Write new captures to object storage; keep `ocr` in Postgres (it is queried during review and vision merge).
3. Backfill existing rows in batches in system context: decode, upload, record hash and key, verify by re-reading, then set `data` to null.
4. Serve images through an authorised route that checks tenancy and site scope, like attachments.
5. After a full release cycle with no reads of `data`, drop the column and `VACUUM FULL` (or repack) the table.
6. Apply bucket lifecycle rules that mirror the record retention policy, with object lock for approved records.

4.15 Migrations

4.15.1 Process

Schema changes start in `apps/api/src/db/schema.ts`. `npm run db:generate` (Drizzle Kit, config in `apps/api/drizzle.config.ts`) writes a numbered SQL file and a snapshot into `apps/api/drizzle/`. Policy, grant and role changes are hand-written migrations in the same folder, registered in `meta/_journal.json`. `npm run db:migrate` applies pending migrations; the API also migrates on start-up.

On Postgres, `createDatabase().migrate()` in `apps/api/src/db/client.ts` takes a session-level advisory lock (`pg_advisory_lock(4872917633)`) on one dedicated connection, so several cold-starting serverless instances never migrate concurrently. PGlite (tests, demo) migrates in process.

Table 121. Migration history

Migration	Kind	Content
`0000_init`	generated	22 enums, 34 tables, foreign keys, indexes, 2 CHECK constraints
`0001_rls`	hand-written	`hydris_app` role, grants, `hydris_tenant_ok()`, RLS on 27 tenant tables and organizations, audit_events append-only
`0002_corrections_ui_profiles`	generated	telemetry_corrections, ui_profiles, ui_profile_versions, ui_profile_assignments
`0003_rls_new_tables`	hand-written	Grants and RLS for the four new tables; corrections append-only
`0004_app_role_membership`	hand-written	Grant `hydris_app` to the migrating user (`WITH SET TRUE` on PG16+)
`0005_realtime_events`	generated	realtime_events
`0006_realtime_events_rls`	hand-written	`system_only` policy; re-check SET capability on PG16+
`0007_audit_documents`	generated	audit_doc_status enum, audit_documents, pages, events
`0008_audit_documents_rls`	hand-written	Grants (events insert-only) and RLS for audit tables

4.15.2 Rules for schema changes

- **Expand, then contract.** Add nullable columns or new tables first, deploy code that writes both shapes, backfill, then remove the old shape in a later release. Serverless instances of two versions run side by side during a deploy.
- **No destructive change in a generated migration without review.** Drizzle Kit can emit `DROP COLUMN`; a reviewer must confirm a backfill exists.
- **Every new tenant table gets an RLS migration** (see the checklist above). Tests run on PGlite with the same migrations, so a missing policy fails the cross-tenant test.
- **Enum values are append-only.** Renaming or removing a value needs a new type and a data migration.
- **Large tables** (`telemetry_readings`, `audit_events`) need `CREATE INDEX CONCURRENTLY` in a hand-written migration outside a transaction once they are big.

4.16 Seed and demo data

Table 122. Seed entry points

Entry point	What it creates	Used by
`npm run seed` (`apps/api/src/seed/cli.ts`)	Platform admin from `SEED_ADMIN_EMAIL` and `SEED_ADMIN_PASSWORD` (12+ characters)	New environments
`npm run seed -- --demo`	Plus the Riverbend demo tenant with 7 role accounts at `@demo.hydris.local` (password `SEED_DEMO_PASSWORD`)	Staging, sales environments
`seedDemoShowcase()` (`seed/showcase.ts`) via `bootDemo()`	Full showcase tenant on every instance in demo mode	Public demo, tests

The demo tenant is the organization `Riverbend Process Industries (Demo)` (slug `riverbend-demo`, `is_demo = true`, time zone `Asia/Kolkata`). Every row has a deterministic UUID v5 from `demoId()` in `seed/ids.ts`, so separate in-memory databases on separate serverless instances agree on every identifier.

Table 123. Demo showcase content

Area	Content
Sites	Riverbend Industrial Park (`RIP`, ETP-01, 3,000 m³/day, 5 process areas) and Riverbend Township STP (`RTS`, STP-02, 5,000 m³/day, 4 process areas)
Assets and sensors	7 + 8 assets; 15 + 12 continuous sensors and 3 lab sensors (COD, BOD, TSS)
Telemetry	7 days at 10-minute resolution for 27 signals, `source = 'simulator'`, `synthetic = true`; 10 lab samples per lab parameter over 14 days, `source = 'lab'`, including one COD exceedance (262 mg/L)
Rules and alarms	Threshold rules from `SIM_PLANT.thresholds`, two stale rules, alarm events across all states with history
Documents	Approved SOPs and a lockout/tagout safety notice, linked to rules as procedures
Work	5 inspection templates, routes, inspections in several states with responses, maintenance tasks with activity
Devices	5 devices: two active hardware glasses, a browser simulator, one suspended and one revoked unit
UI profiles	Profiles with published, superseded and draft versions and assignments at several scopes
People and trail	Owner account, role accounts, notifications and audit events

Live data in demo mode comes from `DemoTicker` (`apps/api/src/demo.ts`): at most every 5 seconds a request advances running simulations and feeds the second site's instruments through the normal `ingest()` path, so alarms and realtime events behave exactly as with a gateway.

> **IMPORTANT**
> Synthetic data is labelled at every level: `organizations.is_demo`, `telemetry_readings.synthetic` (tied to `source` by a CHECK constraint), and `synthetic` flags on alarms, inspections and maintenance tasks. Ingestion rejects simulator data for customer tenants. Analytics must filter on these flags.

4.17 Privacy classification

Each table is classified by the most sensitive data it holds. **PII** means personal data about an identifiable person; **Secret** means credentials or material that grants access; **Sensitive** means business or compliance data a customer would treat as confidential; **Operational** means plant data without personal content.

Table 124. Privacy classification per table

Table	Class	Personal or sensitive fields	Notes
`users`	PII, Secret	email, name, password_hash, mfa_secret_enc, last_login_at	Global; erasure must keep audit references (actor ids have no FK).
`memberships`, `membership_sites`	PII	role, title	
`invitations`	PII, Secret	email, name, token_hash	
`auth_sessions`	PII, Secret	ip, user_agent, token_hash	

Table	Class	Personal or sensitive fields	Notes
auth_tokens	Secret	token_hash	
mail_outbox	PII, Secret	recipient, body with live links	Purge (Planned).
inquiries	PII	name, email, phone, company, job_title, message	Consent recorded; IP hashed.
devices	PII (indirect)	last_ip, hardware_id, link to wearer	
device_assignments	PII	who wore which device when	Workforce monitoring sensitivity.
device_sessions, pairing_challenges	Secret	token, code and poll-secret hashes, requester_ip	
notifications	PII	per-person content	
audit_events	PII, Sensitive	actor ids, ip, metadata	Legal record.
alarm_event_history, maintenance_activity	PII	actor ids, notes	
inspections, inspection_responses	PII, Sensitive	assignee, recorded_by, notes, photos via attachments	
attachments	PII possible	photos may show people	Bytes in object storage.
audit_documents, audit_document_pages, audit_document_events	Sensitive, PII possible	scanned names and signatures, compliance values	Page images may contain handwriting and signatures.
telemetry_readings, telemetry_corrections	Sensitive	discharge quality values	Compliance evidence; corrected_by is PII.
alarm_rules, alarm_events	Sensitive	limits and exceedances	assigned_to and actor columns are PII.
documents	Sensitive	procedures	
organizations, sites, facilities, process_areas, assets, gateways, sensors	Operational	site coordinates, gateway key hashes	gateways.key_hash is Secret.
ui_profiles, ui_profile_versions, ui_profile_assignments	Operational		User and device scope rows are PII by link.
realtime_events	Sensitive, transient	payloads	Pruned after 1 day.
simulation_runs, hydris_demo_kv	Operational	demo only	

NOTE

Hydris processes field workers' personal data as a processor for its customers. Data subject requests (access, correction, erasure) are handled per organization; a user erasure should anonymise users and keep the trail rows whose actor ids no longer resolve to a name. The full control mapping is in the Security and Privacy document (HFG-SEC).

4.18 Mobile and client-side data

4.18.1 Android app

The native Android app (`apps/android`) keeps **no local database**. Screens load from the API on demand. The only persisted data is credentials in `SecureStore` (`data/SecureStore.kt`): an encrypted key/value store on top of SharedPreferences file `hydris.secure`.

Table 125. Android SecureStore entries

Key	Content	Written by
`cookies`	Session cookies for the API (refresh cookie), one per line	`HydrisApi` cookie jar
`device.access`	Glasses device access token	Pairing, device refresh
`device.refresh`	Glasses device refresh token (30 days)	Pairing, device refresh
`device.id`, `device.name`	Paired device identity	Pairing

Each value is encrypted with AES-256-GCM using a non-exportable Android Keystore key (alias `hydris.secure.v1`); the stored string is Base64 of a 12-byte IV followed by ciphertext and a 128-bit tag. Nothing secret reaches SharedPreferences in plain text. Scanned pages are held in memory until upload, then sent as base64 with their OCR readings (`ScanUpload` in `data/Models.kt`, `source = "mobile"`). The Kotlin models mirror the API responses and ignore unknown keys, so the server can add fields without breaking the app.

4.18.2 Web glasses app

The browser-based glasses app (`apps/web/src/glasses`) is offline-first for inspections and stores in `localStorage`:

Table 126. Web glasses app local storage

Key	Content
`hydris.glasses.hw`	Install identifier used as `hardware_id` at pairing
`hydris.glasses.refresh`	Device refresh token
`hydris.glasses.queue`	Offline queue of inspection responses `{ inspectionId, op: { opId, stepId, value, note, recordedAt } }`
`hydris.glasses.cache.ctx`, `hydris.glasses.cache.docs`, `hydris.glasses.cache.doc.<id>`	Last good context and SOPs with a timestamp, shown as stale when offline

Queued operations replay idempotently because `opId` becomes `client_op_id` and the server ignores duplicates.

> **RISK**
>
> `localStorage` is readable by any script on the origin. A device refresh token there is acceptable for the browser simulator but not for production hardware; `link.ts` defines a `SecureStore` interface so a native runtime can supply hardware-backed storage. **Planned** for production glasses runtimes.

4.19 Analytics and event model

Planned. There is no analytics pipeline today. Product and operational analytics are currently answered from the transactional tables and `audit_events`. The proposal below keeps analytics out of the request path and reuses events the API already produces.

4.19.1 Principles

- Analytics read from a replica or an export, never from the primary during business hours.
- Every analytic row carries `org_id`, `site_id` where relevant, and the synthetic flag; demo and synthetic data are excluded by default.
- No free text and no names leave the tenant boundary; people are referenced by pseudonymous ids.

4.19.2 Proposed event schema

Table 127. Proposed analytics event envelope

Field	Type	Description
`event_id`	uuid	Unique id for deduplication.
`event_type`	text	`alarm.acknowledged`, `inspection.submitted`, `auditdoc.verified`, `assistant.answered`, `glasses.session_started`, ...
`occurred_at`	timestamptz	When it happened.
`org_id`	uuid	Tenant.
`site_id`	uuid	Optional.
`actor_ref`	text	Pseudonymous actor id (keyed hash of user id per org).
`surface`	text	console, glasses, mobile, gateway, engine.
`entity_type`, `entity_id`	text	Subject of the event.
`properties`	jsonb	Typed per event type, no free text.
`synthetic`	boolean	From the source row.

Sources: `audit_events` already captures most state changes with actor and entity; `realtime_events` captures telemetry and alarm updates. A nightly job (or logical replication) copies them into an analytics store, and rollup tables answer the product questions.

Table 128. Proposed rollups and the questions they answer

Rollup	Grain	Answers
`telemetry_hourly`	sensor, hour: min, max, avg, count, invalidated count	Long-range charts and compliance summaries without scanning raw readings.
`alarm_daily`	site, severity, day: raised, acknowledged, time to acknowledge, time to resolve	Alarm burden and response performance.
`inspection_daily`	site, template, day: due, done on time, abnormal steps	Round completion and findings.
`auditdoc_daily`	site, day: scanned, verified, approved, items corrected, engine agreement	Scanning adoption and OCR quality.
`surface_usage_daily`	org, surface, day: active users and devices	Adoption of glasses vs console vs mobile.

4.20 Future schema changes

All items in this section are **Planned** and listed in priority order. Each one should follow the expand and contract rules in "Migrations".

Table 129. Planned schema changes

#	Change	Why	Size
1	Move `audit_document_pages.data` to object storage (`storage_key`, `sha256`, `size_bytes`)	Database size, backup time, pool pressure	M
2	Partition `telemetry_readings` by month; add `telemetry_hourly` rollup	Scale beyond about 100 million rows	L
3	Retention jobs (sessions, tokens, pairing, mail_outbox, demo kv) and per-org retention settings	Privacy, table hygiene	M
4	CHECK constraints for every text-typed value set	Integrity under manual fixes	S
5	Generated `tsvector` column and GIN index on `documents`	Search latency	S
6	Foreign keys or validation for `actor_device_id`, `submitted_device_id`, `inspection_responses.device_id`, `attachments.device_id` (set null)	Referential integrity	S
7	Document versions table (`document_versions`) so approved SOP text is immutable and inspections can cite the version used	Compliance traceability	M
8	Sample and laboratory model (`samples`, `lab_results` with chain of custody) instead of lab readings in telemetry	Lab provenance, OCEMS reporting	L
9	Compliance limits per discharge point (`consent_limits`) separate from alarm rules	Exceedance reporting	M
10	Voice assistant conversation log (opt-in, `assistant_turns` with grounding references, short retention)	Quality review of answers	M
11	Optional site-level RLS (`app.site_ids`) for telemetry and audit documents	Defence in depth for restricted memberships	M
12	Push notification tokens per device and user (`push_tokens`)	Mobile alerts	S
13	Use `processing` status for asynchronous OCR or vision on the server	Large documents, retries	S
14	Mail transport sets `mail_outbox.sent_at`, then purges	Operational correctness	S

> **DECISION**
> TimescaleDB is attractive for item 2, but managed Postgres offerings used for Hydris deployments may not support the extension. Native declarative partitioning is the default plan; TimescaleDB stays an option per deployment.

4.21 Open questions and unverified facts

Table 130. Items to verify before this document leaves draft

#	Item	Status
1	Retention periods for discharge-monitoring and audit records under each customer's consent to operate (India CPCB and state boards)	Needs legal confirmation
2	Per-row storage figure (about 250 bytes per reading) and per-site growth figures	Estimates; measure on a staging dataset
3	Engine names written by the Android and web OCR clients (`mlkit` and similar in examples)	Illustrative; confirm against `apps/android/.../scan/OcrEngine.kt` and the web scan code
4	Whether production deployments run `REALTIME_MODE=db` (and therefore write `realtime_events`) outside demo mode	Depends on deployment configuration
5	Exact list of `maintenance_activity.action` values written by `maintenance.ts` beyond transitions and notes	Verify in code

5 API Reference

> **NOTE**
>
> **HFG-API v2.0** · Draft for review · Owner: Engineering, Hydris. Every Hydris endpoint, payload, permission, error code and realtime event, as implemented in the code

The Hydris API is the single back end behind every Hydris Field Glasses surface: the operator console, the glasses app, the native Android app and the public website. It is a Fastify 5 application in `apps/api/src`. `app.ts` builds the server, registers global plugins (Helmet, cookies, rate limiting, JWT, multipart) and mounts fifteen route modules from `apps/api/src/modules/*.ts` under the `/api` prefix. Request bodies, query strings and path parameters are validated with zod. Every tenant read and write runs inside a Postgres transaction with row-level security (RLS) bound to the caller's organization.

This reference is generated by reading the code, not the other way round. Where the code and this document disagree, the code wins and this document has a bug. Each chapter names the file it describes. Status labels follow the house convention: **Built** means the behaviour is in the code today, **Partial** means some of it is, and **Planned** means it is not built yet.

148
HTTP endpoints
146 module routes, plus health and cron

15
route modules
registered in apps/api/src/app.ts

18
domain event types
on the SSE stream, plus ready and session.ended

38
permissions
across 6 roles, 13 usable by glasses

600
requests per minute
global per-IP limit (stricter on auth and pairing)

15 min
user access token
12 h in demo mode

7 days
user refresh session
rotating httpOnly cookie

30 days
glasses refresh session
rotating hgd_ token

5.1 How to read this reference

The reference is organized in three layers. The first chapters describe what is common to every call: base URLs, conventions, authentication, authorization, errors, rate limits, idempotency and the realtime stream. The middle chapters cover one route module each, in the order `app.ts` registers them, with a captioned summary table followed by one entry per endpoint. The last chapters cover background jobs, client libraries, the change history and the planned public API.

5.1.1 Endpoint entry format

Each endpoint entry lists the same facts in the same order so that entries can be scanned quickly.

Table 131. Fields of an endpoint entry

Field	Meaning
Auth	Which credential the route accepts: `public` (none), `user` (console session access token), `device` (paired glasses access token), `any` (user or device), `cookie` (refresh cookie), `gateway` (`hgw_` API key), `cron` (`CRON_SECRET`) or `platform` (platform administrator)

Field	Meaning
Permission	The permission from `packages/domain/src/roles.ts` checked by the handler. "Scope only" means no permission is checked beyond site scope
Limit	A route-specific rate limit. Routes without one share the global 600 per minute per IP
Request	Path parameters, query parameters and the JSON body, with the zod rules that apply
Response	Status code and body shape. JSON examples use illustrative values
Errors	The `error.code` values the handler can return, with their HTTP status
Side effects	Audit events written, SSE events published, notifications created

NOTE

JSON examples use realistic but illustrative identifiers and values. Field names, types, status codes and error codes are exact. Response objects that return a full database row include every column of that table; the examples show the fields clients rely on and abbreviate the rest.

5.1.2 Module map

The pie chart shows how the 146 module routes divide across the fifteen modules. Plant hierarchy and records is the largest module because it owns sites, facilities, process areas, assets, sensors, gateways, documents, attachments, history and the hierarchy tree.

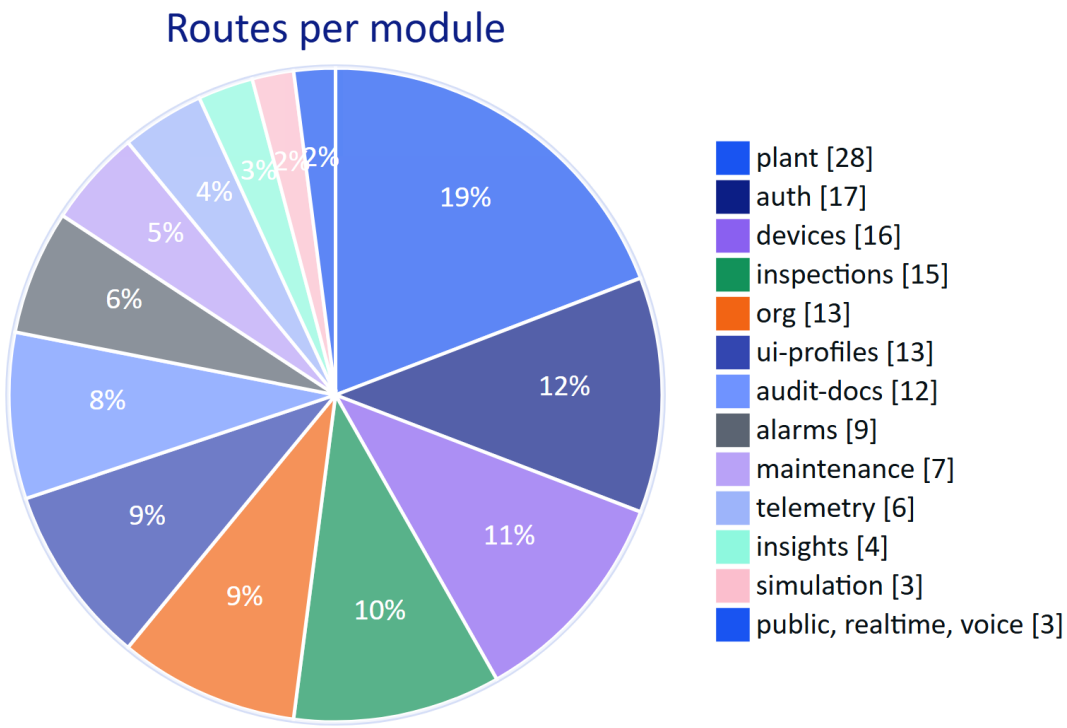

Figure 96. *Routes per API module (146 module routes, counted from apps/api/src/modules)*

5.2 Overview and conventions

5.2.1 Base URLs and deployment shapes

The API is served at the same origin as the console. Browsers, the glasses app and the Android app call relative or absolute `/api/...` paths, never a separate API host.

Table 132. Base URLs by environment

Environment	Base URL	How the API runs
Live demo	`https://hydrisview.vercel.app/api`	Next.js route handler `apps/web/src/app/api/[...path]/route.ts` forwards each request in-process to Fastify with `app.inject` (no network hop). Demo mode: in-memory database per serverless instance
Vercel production	`https://<your-deployment>/api`	Same in-process route handler, Postgres via `DATABASE_URL`, `REALTIME_MODE=db`, Vercel Blob for attachments
Local development	`http://localhost:3000/api`	Next.js proxies `/api` to the Fastify server on port 4000 (`API_PORT`)
Standalone API	`http://<host>:4000/api`	`apps/api/src/server.ts`, long-running process with in-memory event bus and interval jobs

Two paths are served by dedicated Next.js route handlers rather than by the catch-all forwarder: the SSE stream (`apps/web/src/app/api/events/stream/route.ts`) and the cron entry (`apps/web/src/app/api/cron/route.ts`). The Fastify app also registers `GET /api/events/stream` for long-running deployments; both implementations apply the same authorization function `canReceive` from `apps/api/src/modules/realtime.ts`.

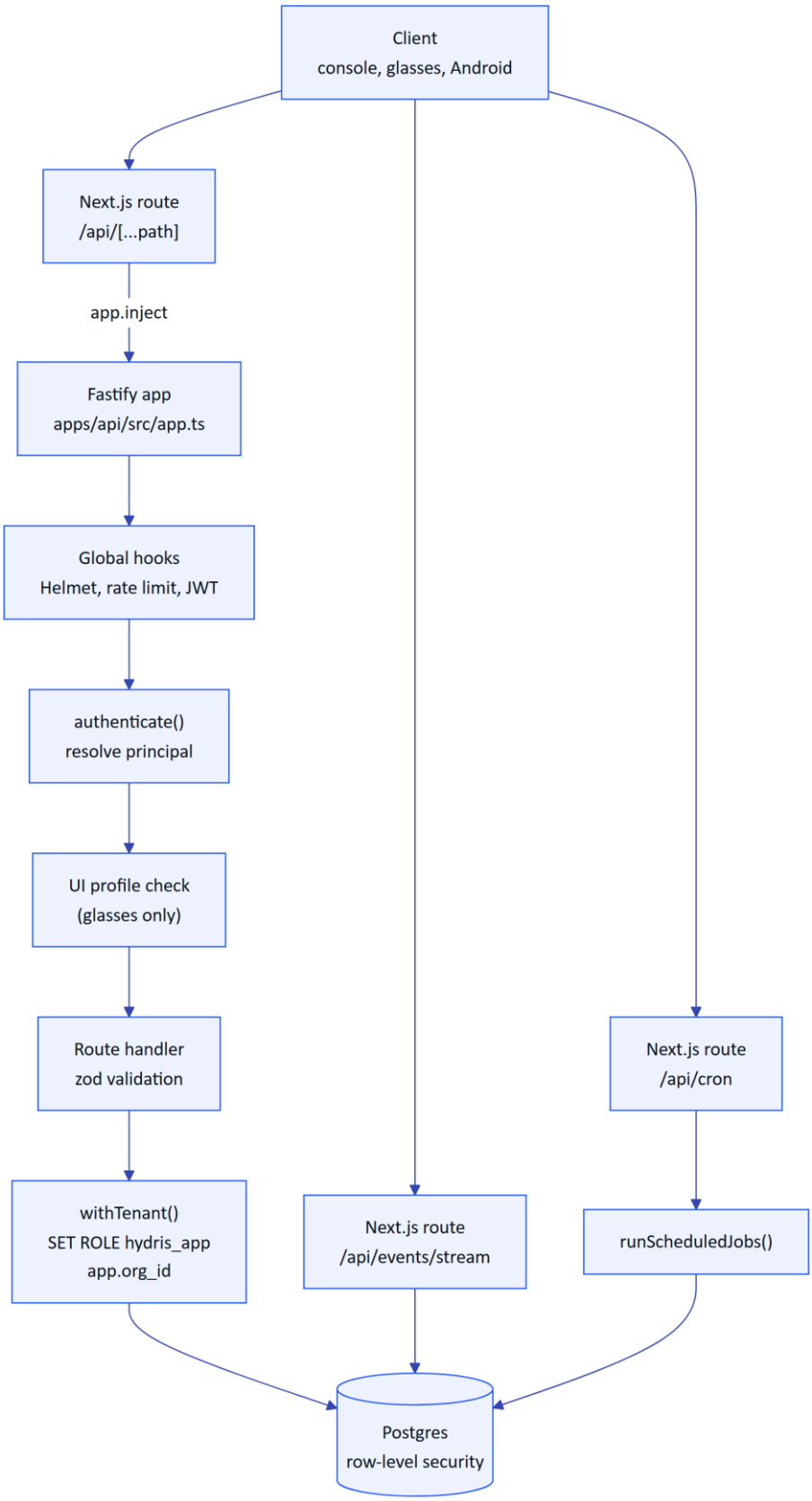

Figure 97. *Request path on Vercel: the Next.js route forwards to Fastify in-process, and every tenant query runs under RLS*

5.2.2 Versioning

The API is versioned as a whole with the product; there is no version segment in most paths. The one exception is the edge ingestion endpoint `POST /api/ingest/v1/readings`, which carries `v1` because gateway firmware is deployed in the field and upgrades slowly. The console, glasses app and Android app ship from the same repository as the API, so breaking changes to unversioned routes are made in the same commit as the client changes. The deprecation policy for routes that third parties may depend on is described in the chapter on change history.

5.2.3 Content types

Table 133. Content types accepted and produced

Direction	Content type	Where
Request	`application/json`	Every JSON route. Global body limit 1 MB (`bodyLimit` in `app.ts`)
Request	`application/json` up to 8 MB	`POST /api/audit-docs` (base64 page images)
Request	`multipart/form-data`	`POST /api/attachments`: one file, at most 10 MB
Response	`application/json`	Default for every route
Response	`text/csv; charset=utf-8`	Sensor export, report exports
Response	`application/pdf`	Searchable audit document PDF
Response	`image/jpeg`, `image/png`, `image/webp`, `text/plain`	Attachment and audit page downloads (stored MIME type)
Response	`text/event-stream; charset=utf-8`	`GET /api/events/stream`

Send `content-type: application/json` with every JSON body. Action endpoints that take an optional body (for example `POST /api/alarms/:id/acknowledge`) accept an empty object `{}`; the clients always send one.

5.2.4 Identifiers

Every entity identifier is a UUID (version 4, generated by Postgres `defaultRandom()`), with three exceptions: telemetry reading ids and realtime event ids are 64-bit integers (`bigint` identity columns), and checklist item ids inside a maintenance task are 8-character strings. Path parameters named `:id` are validated with `z.uuid()` and a malformed id returns `400 validation_failed`, not `404`.

Human-facing codes are separate from ids. Site codes, facility codes, process area codes, asset tags and sensor tags are 1 to 24 characters of letters, digits, dot, dash or underscore, unique within their scope. Pairing codes are eight characters from the alphabet `23456789ABCDEFGHJKMNPQRSTVWXYZ` and are displayed as `XXXX-XXXX`.

5.2.5 Timestamps and time zones

Timestamps in responses are ISO 8601 strings in UTC with millisecond precision, for example `2026-10-22T08:14:03.512Z`. Timestamps in requests are accepted in two forms:

- Fields validated with `z.iso.datetime({ offset: true })` (telemetry `ts`, inspection `recordedAt`) require an ISO 8601 string and accept an explicit offset such as `+05:30`. Timestamps more than 5 minutes in the future are rejected.
- Query parameters and date fields validated with `z.coerce.date()` (`from`, `to`, `dueAt`, `before`) accept anything JavaScript's `Date` parses. Send ISO 8601 to avoid ambiguity.

Sites and organizations carry an IANA `timezone` for display; the API never converts timestamps into local time.

5.2.6 Lists, filtering and pagination

List endpoints return a named array inside an object (`{ "alarms": [...] }`), with one exception: `GET /api/audit-docs` returns a bare array. Most lists are bounded rather than paginated, because operational lists are small and are always filtered by site scope.

Table 134. List bounds and pagination styles

Style	Endpoints	Parameters
Fixed cap	Inspections (300), maintenance (300), audit documents (200), notifications (50), device context sections (20)	None
Limit	Alarms	`limit` 1 to 500, default 200
Limit and time cursor	Audit log	`limit` 1 to 500, default 100, plus `before` (exclusive timestamp cursor)
Limit and offset	Sensor readings	`limit` 1 to 200, default 50, `offset` 0 to 1,000,000, response carries `total`
Downsampling	Sensor history	`points` 10 to 2000, default 240: the server buckets readings with `date_bin`
Range	Reports, CSV export	`from` and `to`. Reports default to the last 30 days and cap at 5000 rows; sensor CSV export spans at most 93 days

Boolean query parameters are strings: `"true"` or `"false"`. Enumerated filters reject unknown values with `400 validation_failed`.

5.2.7 Tenancy and site scope

Every authenticated request carries exactly one organization: the `org` claim of a user token or the organization of the paired device. The handler opens a transaction with `withTenant()` (`apps/api/src/db/tenant.ts`), which switches to the non-privileged `hydris_app` role and sets `app.org_id`, so RLS policies hide every other tenant's rows even if a query forgets a filter.

Inside a tenant, members are either organization-wide (`allSites: true`) or limited to a list of sites. Site scope is enforced in the handlers with `siteScope()`, `requireSite()` and `scopedRow()` from `apps/api/src/core/principal.ts`. A record outside the caller's sites returns **404 not_found**, never 403, so a caller cannot probe which ids exist.

5.3 Authentication and sessions

Hydris has four credential types. Console users hold a short-lived bearer access token plus a rotating refresh cookie. Paired glasses hold a device access token plus a rotating device refresh token. Edge gateways hold a static API key. The Vercel cron caller holds a shared secret. All implementations live in `apps/api/src/modules/auth.ts`, `apps/api/src/modules/devices.ts`, `apps/api/src/context.ts` and `apps/api/src/config.ts`.

Table 135. Credential types

Credential	Format	Lifetime	Transport	Issued by
User access token	JWT, claims `typ: "user"`, `sub` (user id), `org`, `sid` (session family)	15 minutes, 12 hours in demo mode	`Authorization: Bearer`	Login, MFA verify, demo login, refresh, switch org, invitation accept, password change

Credential	Format	Lifetime	Transport	Issued by
User refresh token	48 random bytes, base64url, stored as SHA-256 hash in `auth_sessions`	7 days, rotated on every refresh	Cookie `hg_refresh`, httpOnly, SameSite=Strict, `Secure` in production, path `/api/auth`	Same as above
Demo session cookie	JWT, `typ: "demo_session"`	7 days	Cookie `hg_demo`, same attributes	Demo mode only
MFA challenge token	JWT, `typ: "mfa"`, `sub`	5 minutes	JSON body	`POST /api/auth/login` when MFA is enabled
Device access token	JWT, claims `typ: "device"`, `sub` (device id), `org`, `sv` (session version); demo mode adds `uid` and `site`	15 minutes, 12 hours in demo mode	`Authorization: Bearer`	Pairing completion, device refresh
Device refresh token	`hgd_` plus 40 random bytes (demo mode: `hgd_z` plus an HMAC-signed body)	30 days, rotated on every refresh	JSON body of `POST /api/device/refresh`	Same as above
Gateway API key	`hgw_` plus 32 random bytes, stored as SHA-256 hash with a 10-character prefix	Until revoked	`Authorization: Bearer hgw_...`	`POST /api/gateways` (shown once)
Cron secret	Value of `CRON_SECRET`	Until rotated	`Authorization: Bearer <CRON_SECRET>`	Deployment configuration

Access tokens are signed with `JWT_SECRET` (at least 32 characters) through `@fastify/jwt`. Every request re-checks revocation state in the database: for a user token the session family must still have a non-revoked row and the membership and organization must be active; for a device token the device must be `active`, its `sessionVersion` must equal the token's `sv` claim, and an operator must still be assigned. Revoking a session family or bumping a device's session version therefore invalidates outstanding access tokens immediately, not when they expire.

5.3.1 Password sign-in

`POST /api/auth/login` verifies the password with argon2id (memory 19,456 KiB, 2 iterations). Unknown emails burn the same verification time against a dummy hash so response timing does not reveal which emails exist. Five consecutive failures lock the account for 15 minutes (`423 account_locked`). A successful login picks the user's default organization (or the first active membership), writes an `auth.login` audit event, stores a refresh session and returns the session view.

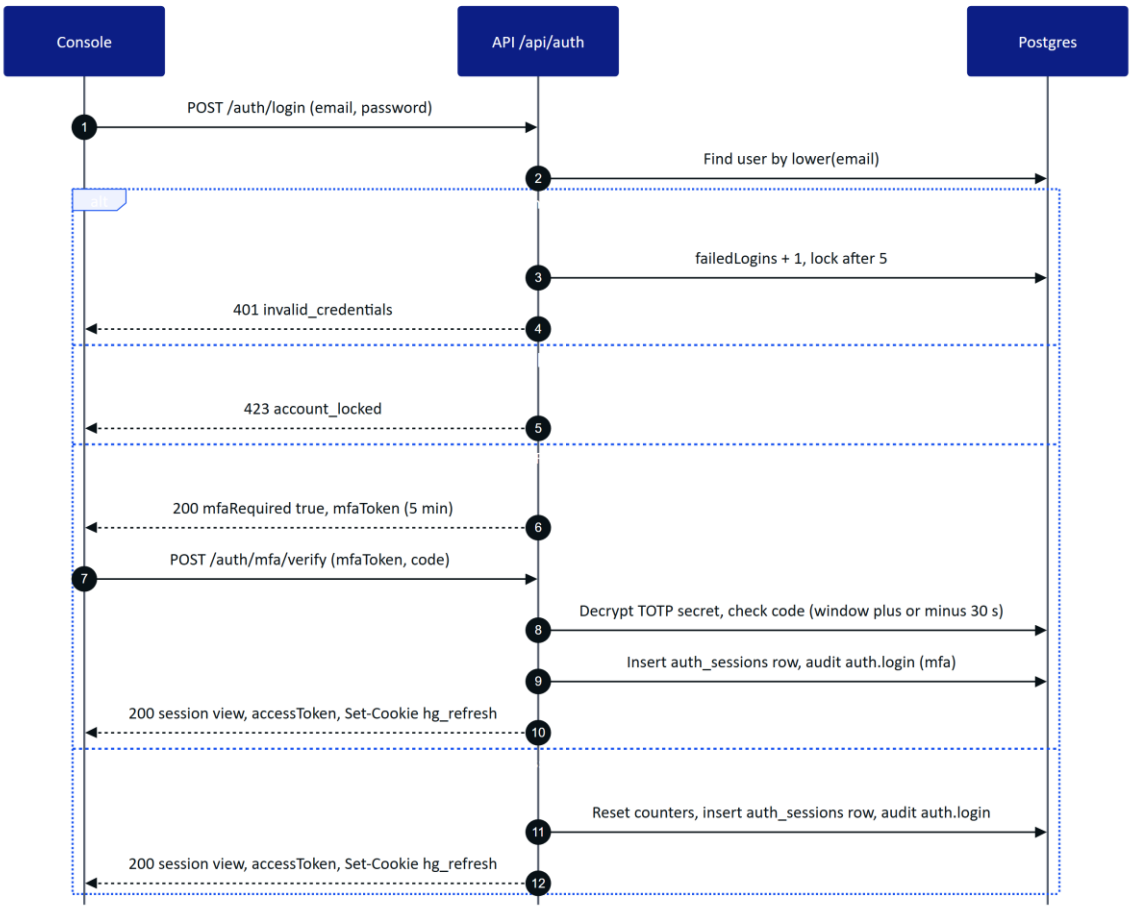

Figure 98. *Password sign-in, with the MFA branch taken when the user has TOTP enabled*

The session view returned by login, MFA verify, demo login, refresh, switch org, password change and invitation accept has the same shape:

```
{
  "accessToken": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...",
  "user": { "id": "7c1e...", "name": "Tarun KM", "email": "tarun@hydris.ai", "isPlatformAdmin": false,
"mfaEnabled": false },
  "memberships": [
    { "orgId": "0b6f...", "orgName": "Hydris Demo Utilities", "role": "org_admin", "isDemo": true,
"allSites": true }
  ],
  "org": { "id": "0b6f...", "name": "Hydris Demo Utilities", "isDemo": true },
  "role": "org_admin",
  "permissions": ["org.manage", "site.read", "site.manage", "..."],
  "siteScope": "all"
}
```

`siteScope` is the string `"all"` for organization-wide members and an array of site ids otherwise. `permissions` is the full list for the active role; clients use it only to decide what to show, the API re-checks on every call.

5.3.2 Multi-factor authentication

MFA is TOTP (RFC 6238: SHA-1, 30-second step, 6 digits, one step of clock tolerance either side). Enrolment is two calls: `POST /api/auth/mfa/setup` generates a 20-byte secret, stores it encrypted with AES-256-GCM under `DATA_KEY` and returns the secret and an `otpauth://` URI for an authenticator app; `POST /api/auth/mfa/enable` turns MFA on once the user proves a valid code. `POST /api/auth/mfa/disable` needs both the password and a current code. When MFA is on, `POST /api/auth/login` returns `{ "mfaRequired": true, "mfaToken": "..." }` instead of a session, and the client completes sign-in with `POST /api/auth/mfa/verify`.

5.3.3 Demo sign-in

The live demo runs in demo mode (`DEMO_MODE=true`, or automatically on Vercel when `DATABASE_URL` is unset). In demo mode `POST /api/auth/demo` with a name creates a named guest account in the synthetic demo tenant. The account id is derived deterministically from the normalized name (`guestUserId()` in `apps/api/src/core/demo-guests.ts`), so the same name always maps to the same account, and the guest receives a personal copy of the demo owner's open inspections and maintenance tasks. Guests are organization administrators of the demo tenant with access to all sites.

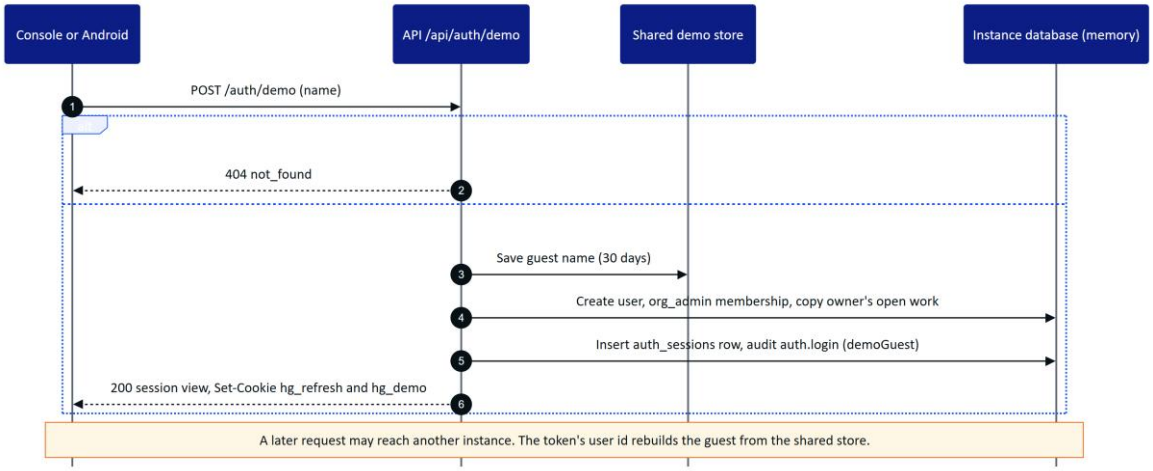

Figure 99. *Demo sign-in creates or reuses a named guest and starts a normal session plus a signed demo cookie*

Because each serverless instance in demo mode has its own in-memory database, demo mode relaxes three stateful checks (documented in `apps/api/src/demo.ts`): a user access token whose session family is unknown on the instance is accepted (a family that is known and revoked is still rejected); `POST /api/auth/refresh` falls back to the signed `hg_demo` cookie and re-creates the session family locally; and device tokens carry the operator and site so any instance can rebuild the device record.

> **WARNING**
>
> Demo mode uses fixed public secrets (`DEMO_SECRETS` in `apps/api/src/config.ts`) when `JWT_SECRET` and `DATA_KEY` are not configured. It exists for the synthetic demo tenant only and must never serve a real plant.

5.3.4 Refresh and rotation

`POST /api/auth/refresh` reads the `hg_refresh` cookie, marks the presented session row as rotated, and issues a new row in the same family with a new cookie and a new access token. Rotation detects theft: if a refresh token that was already rotated is presented again more than 30 seconds after its rotation, the API revokes the whole family, writes `auth.refresh_reuse_detected` and returns `401 session_revoked`. Inside the 30-second grace window a replay issues another successor instead, which keeps several open tabs that refresh at the same moment from logging each other out.

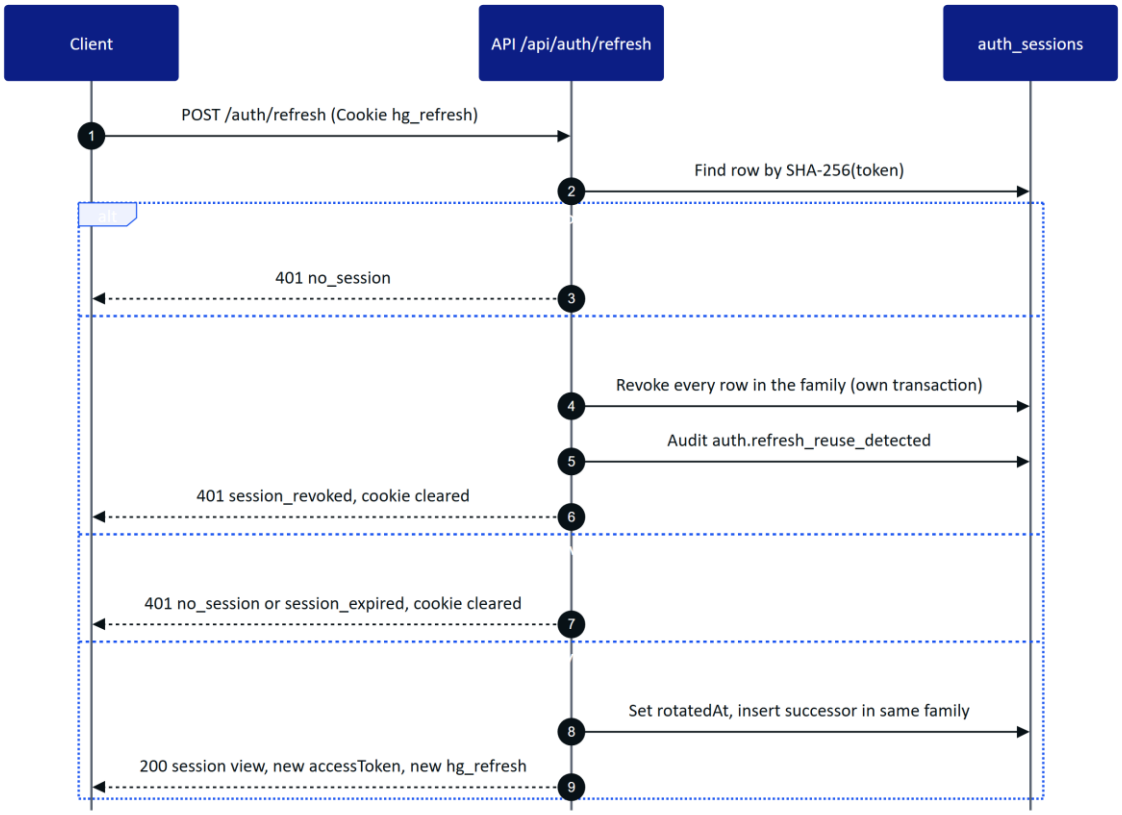

Figure 100. *Refresh token rotation with a 30-second reuse grace window and family-wide revocation on replay*

The web client (`apps/web/src/lib/api.ts`) and the Android client (`HydrisApi.kt`) both serialize refreshes so that a burst of parallel 401 responses triggers one refresh call.

5.3.5 Glasses pairing

Glasses never see a user password. Pairing follows the shape of the OAuth 2.0 Device Authorization Grant (RFC 8628). The glasses request a challenge and display a short user code and a QR code that encodes only the verification URL with that code. A signed-in person approves the code on a phone or console, choosing the site and (with `device.manage`) the operator. The glasses, which hold a separate poll secret that is never displayed, poll until the challenge is approved and then receive device credentials exactly once.

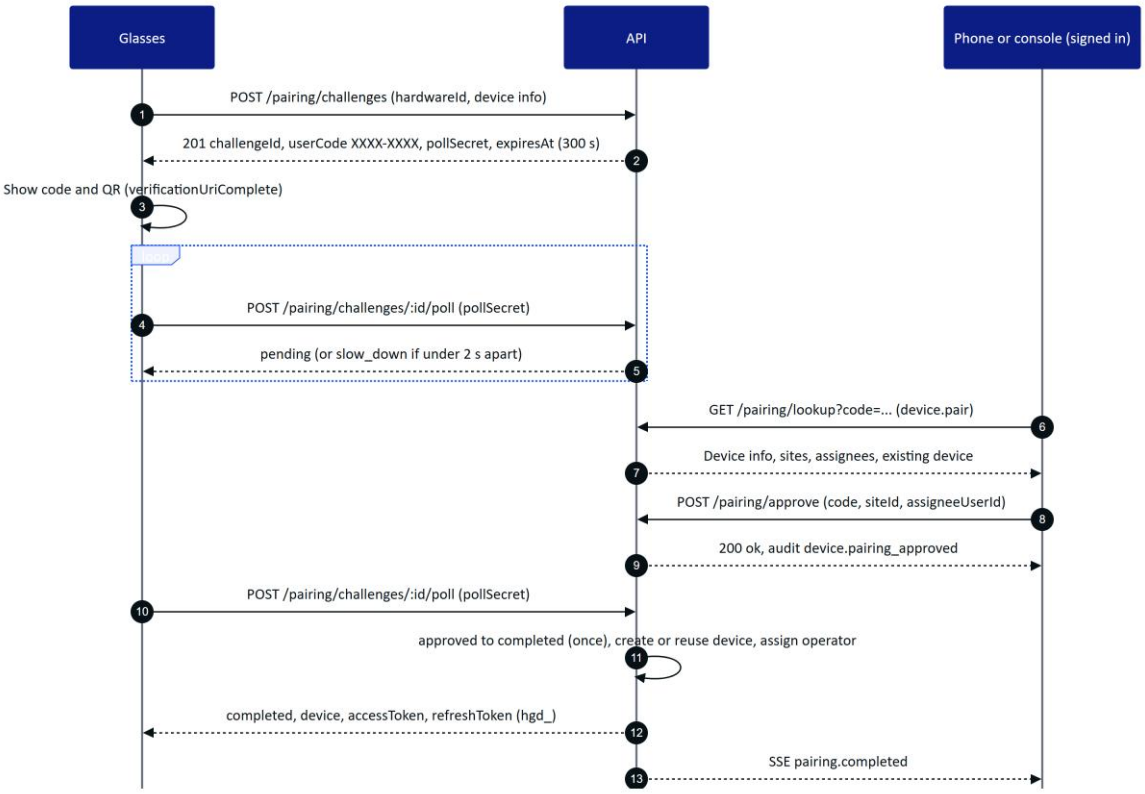

Figure 101. *Glasses pairing in the shape of RFC 8628: code on the glasses, approval by a signed-in person, single-use claim*

Security properties of the flow, all implemented in `apps/api/src/modules/devices.ts`:

- The code is drawn with a CSPRNG from a 30-character unambiguous alphabet, stored only as a SHA-256 hash, valid for 300 seconds and usable once. Code lookups and approvals are limited to 10 per minute per IP.
- The poll secret (32 random bytes) is stored as a hash and compared in constant time; a wrong secret returns 404.
- Requesting a new challenge from the same `hardwareId` cancels any open challenge from that install.
- Glasses registered to another organization cannot be approved until revoked there (`409 device_owned_elsewhere`).
- Re-pairing the same install within the same organization reuses the device record, bumps its session version and revokes its previous device sessions.
- A background job marks pending and approved challenges past their expiry as `expired`.

5.3.6 Device token refresh

`POST /api/device/refresh` exchanges a device refresh token for a new access token and a new refresh token. Unlike user refresh there is no grace window: presenting a device refresh token that was already rotated revokes the entire device session family, writes `device.refresh_reuse_detected` and returns `401 device_session_invalid`. The glasses then need to be paired again.

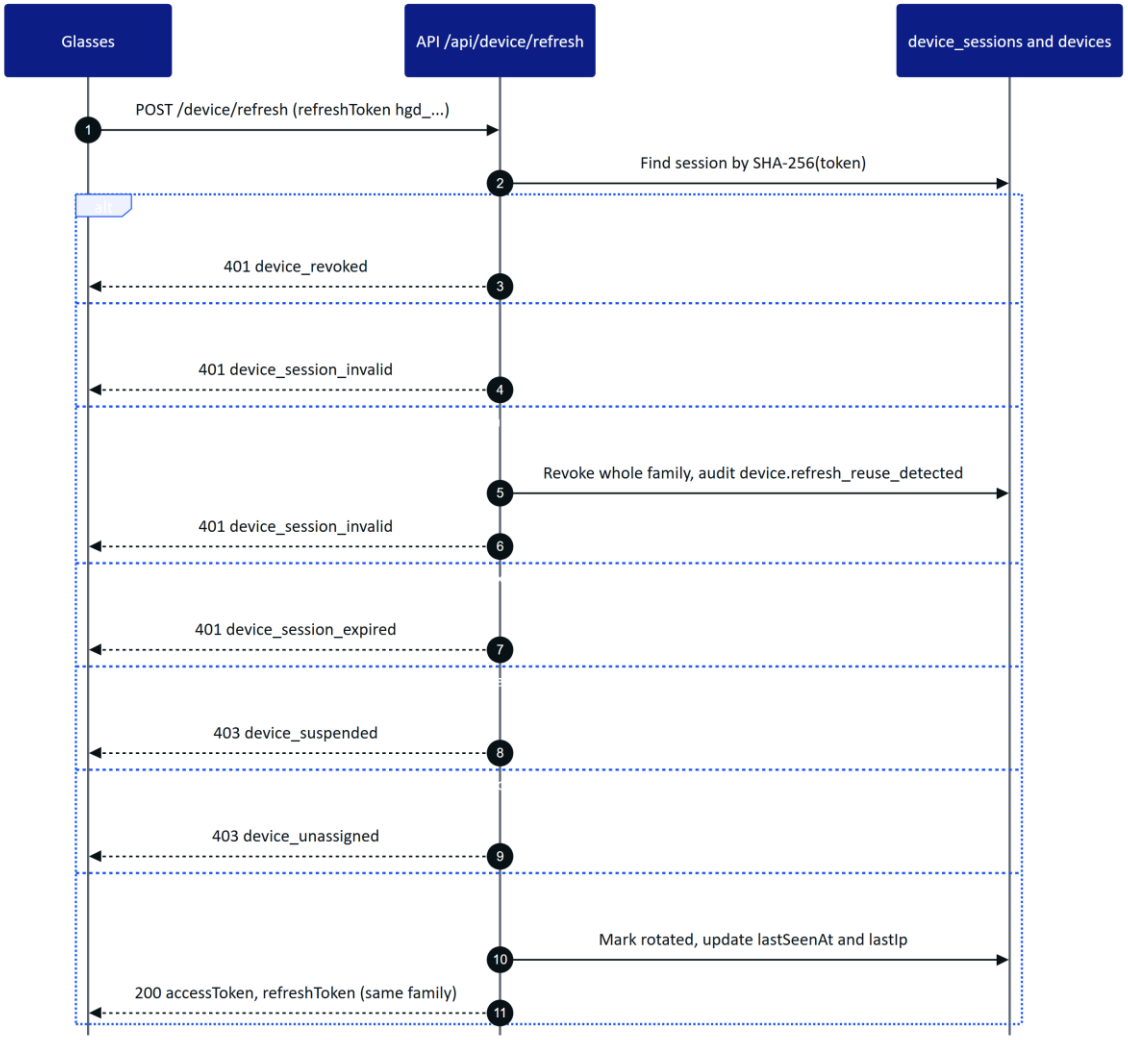

Figure 102. *Device refresh rotation, with revocation, suspension and unassignment checks*

5.3.7 Ending sessions

Sessions end in several ways, and each one is enforced on the very next request because principals are re-resolved from the database on every call and every 30 seconds on open SSE streams.

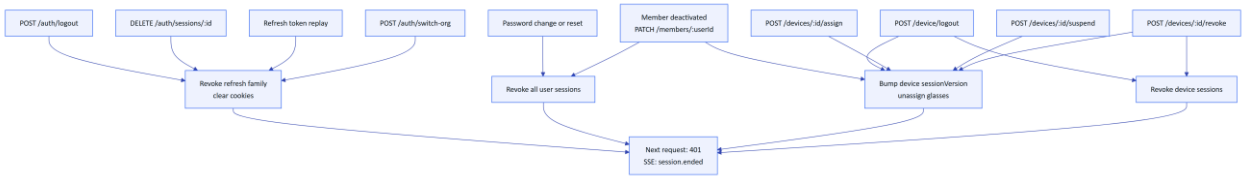

Figure 103. *Every way a user or device session ends, and the mechanism that enforces it*

`POST /api/device/logout` is the glasses-initiated unpair: it revokes the device's refresh sessions and bumps its session version, so the same glasses can be paired again by anyone. Console-initiated revocation (`POST /api/devices/:id/revoke`) is final for that device record: it also releases the operator assignment, and the glasses must pair again to obtain a new credential.

5.4 Authorization

Authorization has four layers, applied in this order on every authenticated route:

1. **Authentication.** `authenticate(ctx)` resolves the bearer token into a principal (user or device) or returns `401 unauthenticated`.
2. **Role permission.** `requirePerm(p, permission)` checks the role matrix in `packages/domain/src/roles.ts`. For device principals the permission must also be in `DEVICE_PERMISSIONS` (`apps/api/src/core/principal.ts`), so a paired glasses device can never do more than its operator, and never anything administrative. Failure returns `403 forbidden`.
3. **Site scope.** Records on sites outside the principal's scope return `404 not_found`.
4. **Glasses UI profile.** For device principals only, a hook installed by `installProfileEnforcement()` (`apps/api/src/modules/ui-profiles.ts`) checks that the device's effective UI profile enables the screen and action the route serves. Failure returns `403 profile_forbidden`.

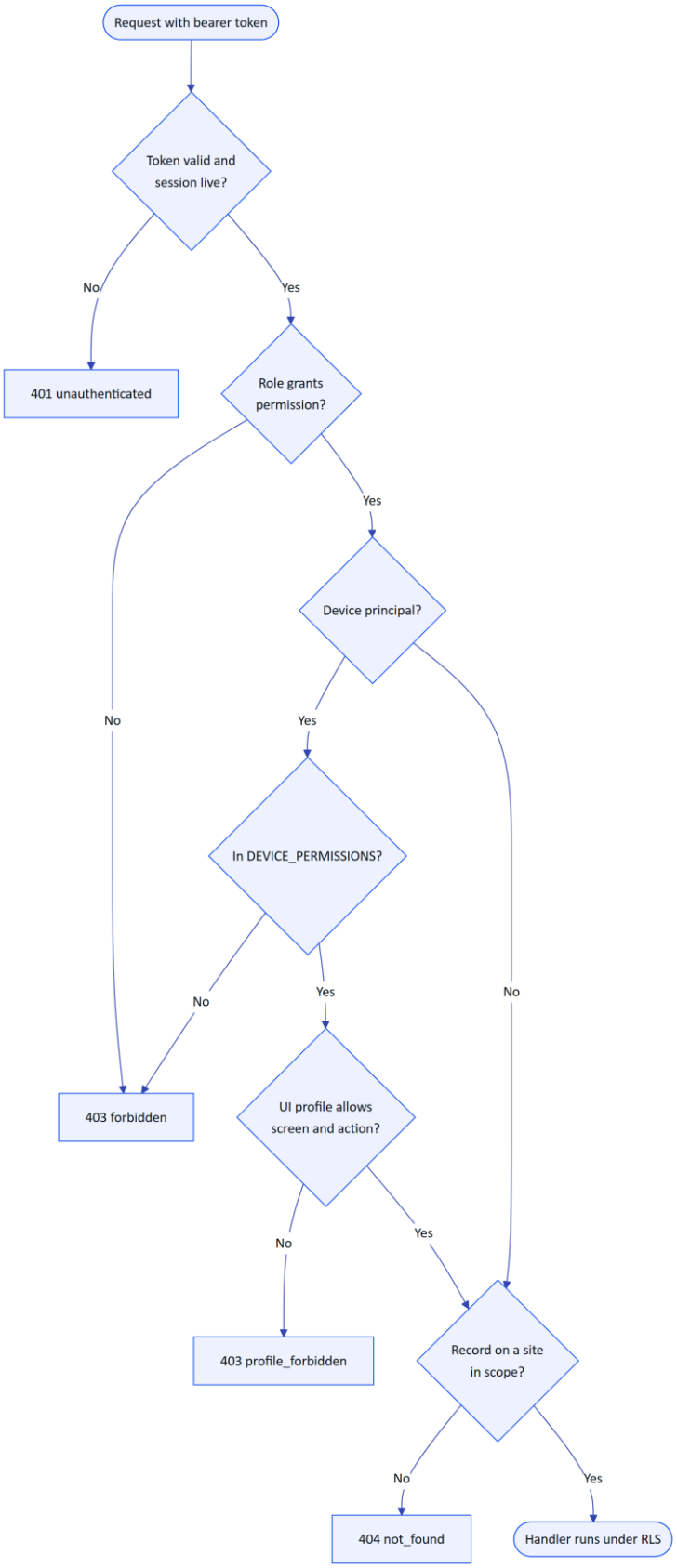

Figure 104. *Authorization decision for an authenticated request*

Some rules are expressed in the handler rather than as a single permission. The most important ones: inspections are recorded only by their assignee and reviewed only by someone else; audit documents are approved by a different person from the verifier (except in demo tenants, where self-approval is recorded as

such); maintenance executors can start, hold and complete only tasks assigned to them; and members can grant only roles listed by `assignableRoles()` (organization administrators grant any role, factory managers grant supervisor, field operator, maintenance engineer and viewer).

5.4.1 Role and permission matrix

Table 136. Permissions by role (Y = granted). The last column marks the 13 permissions a paired glasses device may use on behalf of its operator

Permission	Org admin	Factory manager	Supervisor	Field operator	Maintenance engineer	Viewer	Glasses
`org.manage`	Y						
`site.read`	Y	Y	Y	Y	Y	Y	Y
`site.manage`	Y	Y					
`asset.read`	Y	Y	Y	Y	Y	Y	Y
`asset.manage`	Y	Y					
`sensor.manage`	Y	Y					
`telemetry.read`	Y	Y	Y	Y	Y	Y	Y
`gateway.manage`	Y	Y					
`alarm.read`	Y	Y	Y	Y	Y	Y	Y
`alarm.acknowledge`	Y	Y	Y	Y	Y		Y
`alarm.assign`	Y	Y	Y				
`alarm.investigate`	Y	Y	Y		Y		
`alarm.resolve`	Y	Y	Y		Y		
`alarm.close`	Y	Y	Y				
`alarm_rule.manage`	Y	Y					
`inspection.read`	Y	Y	Y	Y	Y	Y	Y
`inspection.manage`	Y	Y	Y				
`inspection.execute`	Y		Y	Y	Y		Y
`inspection.review`	Y	Y	Y				
`maintenance.read`	Y	Y	Y	Y	Y	Y	Y
`maintenance.request`	Y	Y	Y	Y	Y		Y
`maintenance.manage`	Y	Y	Y				
`maintenance.execute`	Y				Y		Y
`member.read`	Y	Y	Y				
`member.manage`	Y	Y					
`device.read`	Y	Y	Y				
`device.pair`	Y	Y	Y	Y	Y		
`device.manage`	Y	Y					

Permission	Org admin	Factory manager	Supervisor	Field operator	Maintenance engineer	Viewer	Glasses
`document.read`	Y	Y	Y	Y	Y	Y	Y
`document.manage`	Y	Y					
`report.read`	Y	Y	Y			Y	
`report.export`	Y	Y	Y				
`audit.read`	Y	Y					
`simulation.control`	Y	Y					
`auditdoc.read`	Y	Y	Y	Y	Y	Y	Y
`auditdoc.capture`	Y	Y	Y	Y	Y		Y
`auditdoc.verify`	Y	Y	Y				
`auditdoc.approve`	Y	Y					

The glasses column is an upper bound: a device holds the intersection of this column and its operator's role. Glasses assigned to a viewer, for example, cannot acknowledge alarms. Factory managers deliberately lack `inspection.execute` and `maintenance.execute`: they plan and review work rather than record it.

> **NOTE**
> Platform administration (`/api/platform/*`) is outside the role matrix. It requires `users.isPlatformAdmin`, which is resolved from the user token even when the user has no membership in any organization.

5.5 Errors

5.5.1 Error envelope

Every error produced by the API has the same JSON envelope. The error handler is registered in `apps/api/src/app.ts`; handlers throw `HttpError` (`apps/api/src/core/errors.ts`) and zod throws `ZodError`.

```
{
  "error": {
    "code": "validation_failed",
    "message": "Some fields are invalid",
    "details": [
      { "path": "readings.0.ts", "message": "Invalid ISO datetime" }
    ]
  }
}
```

- `code` is stable and machine-readable. Clients branch on `code`, never on `message`.
- `message` is a human-readable sentence suitable for display. It may change between releases.
- `details` is optional. For `validation_failed` it is an array of `{ path, message }` with dotted paths; a few handler errors put a structured object here instead (for example the inspection `progress` on `incomplete`, or the current `state` on `invalid_transition`).

Errors that do not originate in a handler are mapped as follows: a status of 429 becomes `rate_limited`; any status of 500 or more becomes `internal_error` with the message "Unexpected server error" (the real error is

logged, never returned); other Fastify errors keep their own code (for example `FST_ERR_CTP_BODY_TOO_LARGE` with status 413).

> **NOTE**
> Requests to a path that matches no route are answered by Fastify's default not-found handler, which does not use the Hydris envelope. The Next.js route handlers have their own failure codes: `service_unavailable` (503) when the API cannot start, `internal` (500) when forwarding fails, and `unauthorized` (401) on the cron and stream routes.

5.5.2 HTTP status codes

Table 137. HTTP status codes used by the API

Status	Meaning in Hydris
200	Success with a body
201	Resource created (invitation, site, asset, sensor, rule, inspection, task, audit document, pairing challenge, assignment)
204	Success without a body (`POST /api/device/logout`)
400	Validation failure or a business rule the request breaks (missing note, incomplete checklist, weak password)
401	Missing, invalid, expired or revoked credential
403	Authenticated but not permitted (role, device cap, UI profile, four-eyes rule)
404	Not found, or outside the caller's site scope
409	Conflict with current state: invalid transition, stale revision, duplicate, already used
413	Body or file larger than the configured limit
423	Account locked after repeated failed logins
429	Rate limit exceeded
500	Unexpected server error
503	Pairing code allocation failed, or the serverless API could not start

5.5.3 Error code catalogue

The table lists every `error.code` the API emits, collected from the handlers. Codes are grouped by status.

Table 138. Error codes by HTTP status

Status	Code	Raised by	Meaning
400	`validation_failed`	All routes (zod), plus asset, sensor, rule, document and maintenance edits	Body, query or path parameters are invalid. `details` lists each field
400	`weak_password`	Change password, reset, invitation accept	Fewer than 12 characters, more than 200, or one repeated character
400	`wrong_password`	Change password, MFA disable, invitation accept	Current password is incorrect
400	`invalid_token`	Password reset complete	Reset link invalid, used or expired (30 minutes)

Status	Code	Raised by	Meaning
400	`invalid_invitation`	Invitation preview and accept	Invitation invalid, used, revoked or expired (7 days)
400	`invalid_mfa_code`	MFA enable and disable	TOTP code is not valid
400	`mfa_already_enabled`	MFA setup	MFA is already on
400	`sites_required`	Create invitation	`allSites` is false and no site was chosen
400	`invalid_sites`	Create invitation	A site id does not exist in the organization
400	`forbidden_scope`	Create site, document scope	Only organization-wide managers may do this
400	`invalid_process_area`	Create asset	Process area is not in the asset's facility
400	`file_required`	Upload attachment	No file part in the multipart body
400	`unsupported_type`	Upload attachment	MIME type is not JPEG, PNG, WebP, PDF or plain text
400	`content_mismatch`	Upload attachment	File signature does not match the declared MIME type
400	`method_mismatch`	Manual reading	Lab entry against an online (continuous) instrument
400	`reading_rejected`	Manual reading	The ingestion pipeline rejected the value; `details.reason` gives why
400	`invalid_range`	Sensor history, reports	`from` is not before `to`
400	`range_too_large`	Sensor CSV export	More than 93 days requested
400	`note_required`	Alarm resolve, inspection reject, maintenance complete	A note is mandatory for this transition
400	`invalid_assignee`	Alarms, inspections, maintenance, devices, pairing	Assignee is not an active member, or cannot execute inspections
400	`assignee_no_site_access`	Pairing approve	The chosen operator has no access to the chosen site
400	`threshold_required`	Create alarm rule	Non-stale rules need a threshold
400	`invalid_template`	Create inspection	Template missing or not published
400	`invalid_asset`	Create inspection	Asset not found
400	`invalid_assets`	Inspection routes	A route stop is not an asset on the route's site
400	`site_required`	Create inspection, create maintenance task	Neither an asset nor a site was given
400	`incomplete`	Inspection submit	Required steps are unanswered; `details` carries the progress object
400	`invalid_alarm`	Create maintenance task	`sourceAlarmId` does not exist
400	`checklist_incomplete`	Maintenance complete	Checklist items are still open
400	`reason_required`	Device revoke, audit document reject	A reason is mandatory
400	`invalid_code`	Pairing lookup and approve	Code is not 8 valid characters

Status	Code	Raised by	Meaning
400	`page_too_large`	Audit document capture	A page image is over 900 KB
400	`invalid_image`	Audit document capture	Page bytes are not a JPEG or PNG
400	`value_required`	Audit item correct	Corrected field value missing
400	`rows_required`	Audit item correct	Corrected table rows missing
400	`flagged_items`	Audit document verify	Flagged items must be resolved first
400	`review_incomplete`	Audit document verify	Items are still pending review
400	`no_draft`	UI profile publish	There is no draft to publish
400	`cannot_rollback_to_draft`	UI profile rollback	Target version is the open draft
400	`invalid_profile`, `invalid_user`, `invalid_device`	UI profile assignment	Target not found in the organization
400	`subject_required`	Effective UI profile	Neither `deviceId` nor `userId` given
400	`too_fast`	Public enquiry	Form submitted less than 2.5 s after it was opened
401	`unauthenticated`	Every authenticated route	No token, invalid token, or session no longer live
401	`invalid_credentials`	Login	Email or password incorrect (also for inactive accounts)
401	`mfa_expired`	MFA verify	The 5-minute MFA token expired
401	`invalid_mfa_code`	MFA verify	TOTP code is not valid
401	`no_session`	Refresh	No cookie, unknown or revoked session
401	`session_expired`	Refresh	Refresh session older than 7 days
401	`session_revoked`	Refresh	Replay of a rotated token; whole family revoked
401	`device_revoked`	Device refresh	Device revoked by an administrator
401	`device_session_invalid`	Device refresh	Unknown, revoked or replayed device refresh token
401	`device_session_expired`	Device refresh	Device refresh session older than 30 days
401	`invalid_gateway_key`	Edge ingestion	Missing, wrong, revoked key or suspended organization
403	`forbidden`	Every permission check	Role, device cap or handler rule denies the action
403	`device_suspended`	Device refresh	Device is suspended
403	`device_unassigned`	Device refresh	No operator assigned to the device
403	`profile_forbidden`	Glasses routes	The device's UI profile disables this screen or action
404	`not_found`	Every lookup	Record missing or outside site scope; `/api/auth/demo` outside demo mode
409	`slug_taken`	Create organization	Slug already used
409	`already_member`	Create invitation	Email belongs to an active member

Status	Code	Raised by	Meaning
409	`code_taken`	Sites, facilities, process areas	Code already used in scope
409	`tag_taken`	Assets, sensors	Tag already used
409	`source_ref_taken`	Update sensor	Gateway reference already used on the site
409	`in_use`	Delete asset	Asset has sensors, alarms, inspections or tasks: archive instead
409	`duplicate_reading`	Manual reading	A reading already exists at that sensor and timestamp
409	`no_change`	Reading correction, template status	Nothing would change
409	`already_invalidated`	Reading correction	Reading is already invalidated
409	`invalid_transition`	Alarms, inspections, maintenance, devices	Action not allowed from the current state
409	`stale_state`	Alarm transitions, maintenance transitions	State changed between read and write
409	`revision_conflict`	Inspection transitions	`expectedRevision` does not match, or concurrent write
409	`not_editable`	Inspection and maintenance edits	Record is closed
409	`archived`	Template update	Template must be restored before editing
409	`not_running`	Simulation stop	No simulation is running
409	`already_used`	Pairing approve	Code was approved concurrently
409	`device_owned_elsewhere`	Pairing approve	Glasses registered to another organization
409	`device_not_active`	Device assign	Only active devices can be reassigned
409	`device_revoked`	Device suspend, reactivate, revoke	Device is already revoked
409	`review_started`	Audit enhance	Human review has begun
409	`stale`	Audit document writes	Revision changed underneath the caller
409	`locked`	Audit document edits	Approved documents are locked
409	`not_in_review`	Audit confirm-agreed and verify	Document is not in review
409	`not_verified`	Audit approve	Document must be verified first
409	`profile_archived`	UI profile draft and publish	Archived profiles cannot change
409	`profile_assigned`	UI profile archive	Remove assignments first
409	`already_published`	UI profile rollback	Target version is already live
413	`FST_ERR_CTP_BODY_TOO_LARGE`	Any JSON route	Body over 1 MB (8 MB for audit capture)
423	`account_locked`	Login	Five failed attempts; locked 15 minutes
429	`rate_limited`	Any route	Rate limit exceeded
500	`internal_error`	Any route	Unexpected error (logged server-side)
503	`pairing_unavailable`	Pairing challenge	No unique code after five attempts

5.6 Rate limiting

Rate limits use `@fastify/rate-limit`, keyed by client IP. Fastify runs with `trustProxy: true`, and the Vercel route handler passes the first `x-forwarded-for` address as the remote address, so the key is the real client IP rather than the platform's proxy. A global limit of 600 requests per minute applies to every route; the routes below set a stricter limit of their own.

Table 139. Route-specific rate limits

Route	Limit	Window	Reason
`POST /api/auth/login`	10	1 minute	Credential stuffing
`POST /api/auth/mfa/verify`	10	1 minute	TOTP guessing
`POST /api/auth/demo`	20	1 minute	Guest creation
`POST /api/auth/refresh`	60	1 minute	Token churn
`POST /api/auth/password-reset/request`	5	1 minute	Mail abuse, enumeration
`POST /api/auth/password-reset/complete`	10	1 minute	Token guessing
`GET /api/auth/invitations/:token`	30	1 minute	Token probing
`POST /api/auth/invitations/accept`	10	1 minute	Token guessing
`POST /api/pairing/challenges`	10	1 minute	Code exhaustion
`POST /api/pairing/challenges/:id/poll`	40	1 minute	Polling every 3 s with headroom
`POST /api/pairing/challenges/:id/cancel`	20	1 minute	
`GET /api/pairing/lookup`	10	1 minute	Pairing code guessing
`POST /api/pairing/approve`, `POST /api/pairing/deny`	10	1 minute	Pairing code guessing
`POST /api/device/refresh`	30	1 minute	Token churn
`POST /api/assistant/ask`	60	1 minute	Language model cost
`POST /api/ingest/v1/readings`	1200	1 minute	Gateway throughput (each call carries up to 500 readings)
`POST /api/public/inquiries`	5	10 minutes	Website form spam
All other routes	600	1 minute	Global ceiling

When a limit is exceeded the API returns `429 rate_limited`. The plugin also sets its standard rate limit headers on responses (limit, remaining and reset, plus `retry-after` on 429); clients should back off for the indicated time. Tests build the app with `relaxedLimits`, which raises every limit except the assistant's.

> **WARNING**
>
> The rate limit store is the plugin's in-process memory store. On Vercel each serverless instance counts separately, so the effective limit across a burst that fans out to several instances is a multiple of the configured value. A shared store (Redis or Postgres) is needed before the limits can be treated as hard guarantees.

The pairing poll endpoint has a second, application-level throttle: polling the same challenge again less than two seconds after the previous poll returns `{ "status": "slow_down", "pollIntervalS": 3 }` without consuming the approval.

5.7 Idempotency and concurrency

Hydris does not use a generic `Idempotency-Key` header. Instead each write that can be retried by an intermittently connected client is idempotent by design, and each write that races with other people uses optimistic concurrency.

Table 140. Idempotency and concurrency mechanisms

Operation	Mechanism	Replay or race outcome
Edge ingestion	Unique `(sensor_id, ts)` on `telemetry_readings`	A repeated reading counts in `duplicates`, nothing is stored twice
Manual reading	Same unique key	`409 duplicate_reading`: correct the existing reading instead
Inspection step responses	Client-generated `opId` (8 to 80 characters), unique per inspection	Replayed ops count in `duplicates`
Inspection transitions	`revision` counter, optional `expectedRevision` in the body	`409 revision_conflict`
Alarm transitions	`WHERE state = <state read>` on update	`409 stale_state`
Maintenance transitions	`WHERE status = <status read>` on update	`409 stale_state`
Audit document writes	`revision` counter, optional `expectedRevision`	`409 stale`
Pairing claim	`approved` to `completed` update guarded by status	Second claim returns `completed` with `error: "already_claimed"` and no tokens
Pairing approve	`pending` to `approved` update guarded by status	`409 already_used`
UI profile assignment	One assignment per org, role, user or device	Re-posting replaces the previous assignment (200 instead of 201)
Device assign	Same operator already assigned	`200 { "ok": true, "unchanged": true }`

The glasses app keeps an offline queue of inspection step operations in `apps/web/src/glasses/link.ts` and replays them grouped by inspection when connectivity returns. Because each operation carries its `opId`, replays are safe. Operations for an inspection that was closed in the meantime come back in `conflicts` with a reason such as `inspection_submitted`.

5.8 Realtime events

The API pushes domain events over Server-Sent Events at `GET /api/events/stream`. Events are refetch hints: they tell a client that something changed and carry enough data to update a list row, but the REST endpoints remain the source of truth.

5.8.1 Connecting

Clients connect with `fetch()` rather than `EventSource` so the access token travels in the `Authorization` header and never in a URL. Resumption uses the standard `Last-Event-ID` request header (a `lastEventId` query parameter is also accepted). A fresh connection with no last id starts at the current head of the log.

```
curl -N https://hydrisview.vercel.app/api/events/stream \
  -H "authorization: Bearer $TOKEN" \
  -H "accept: text/event-stream" \
  -H "last-event-id: 1842"
```

The stream begins with a `ready` event and a `retry` hint, then delivers events as they occur, with a comment line heartbeat every 15 seconds.

```
retry: 1000
event: ready
data: {"kind":"user","serverTime":"2026-10-22T08:14:03.512Z"}

id: 1843
event: alarm.created
data: {"id":"5f0c...","severity":"high","state":"active","title":"High Vibration · P-101","_at":"2026-
10-22T08:14:05.101Z"}

: hb 1761120860000
```

Each event's `data` is its payload object with an `_at` field (the server time the event was published) added.

5.8.2 Delivery modes

Table 141. Realtime delivery modes (REALTIME_MODE)

Mode	Used by	Mechanism	Replay
`memory`	Long-running API (`server.ts`)	In-process `EventEmitter`, pushed immediately	Ring buffer of the last 1000 events
`db`	Vercel and any multi-instance deployment, always in demo mode	Events appended to `realtime_events` after the request finishes (`onSend` hook); streams poll every 1.5 s	Everything still in the table (pruned after 24 hours)

On Vercel the stream route ends each response after about 50 seconds with a `: reconnect` comment, so every invocation finishes inside the 60-second function limit. The `retry: 1000` hint makes clients reconnect within a second, and `Last-Event-ID` resumes without gaps. Both client libraries reconnect automatically with backoff.

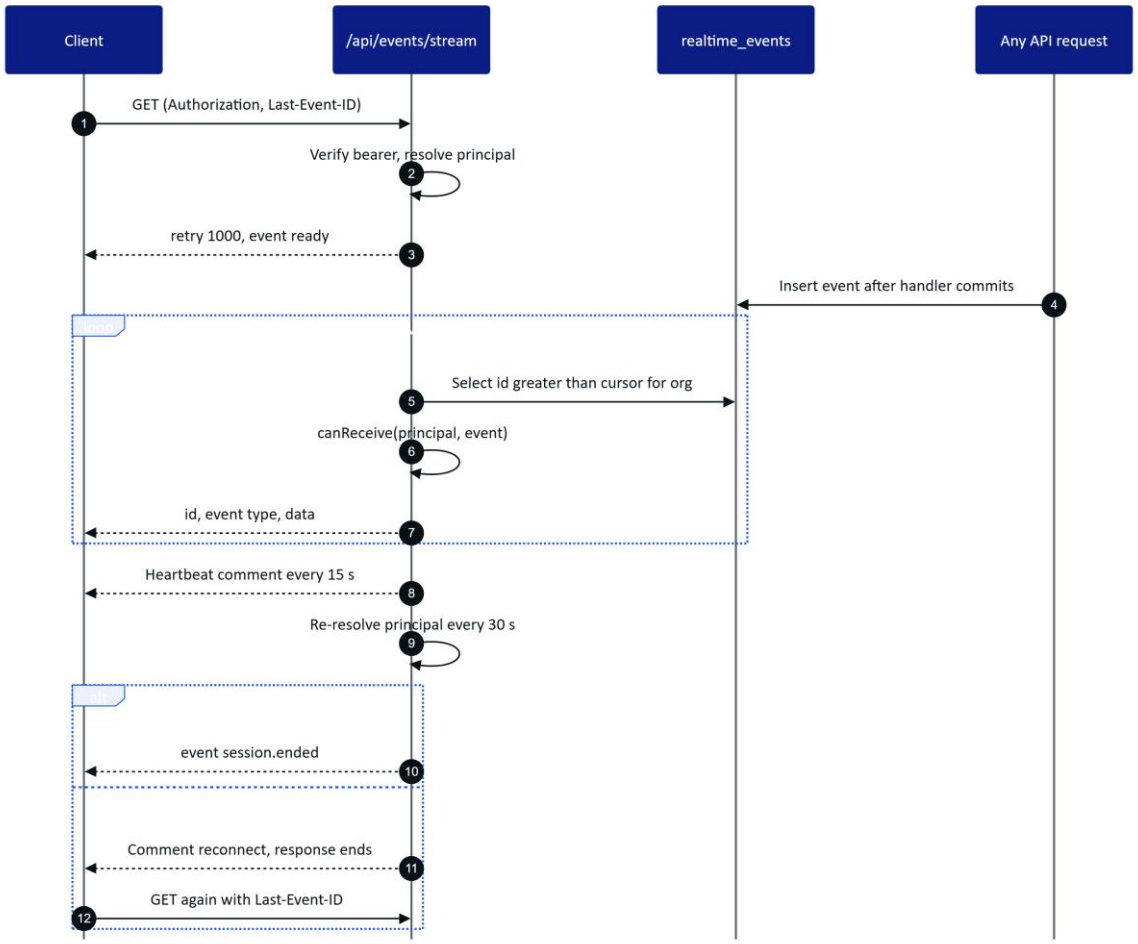

Figure 105. *SSE stream lifecycle in db mode: authorize per event, heartbeat, revalidate, reconnect before the function limit*

5.8.3 Per-connection authorization

Every event names its organization and optionally a site, a permission, a list of user ids or a list of device ids. `canReceive()` in `apps/api/src/modules/realtime.ts` delivers an event to a connection only when all of these hold:

- the event's organization is the principal's organization;
- if the event names devices, the connection is one of those devices (device-targeted events go to nobody else);
- if the event names users, the principal is one of them;
- if the event names a site, the site is in the principal's scope;
- if the event names a permission, the principal holds it (with the device cap applied for glasses).

The stream re-resolves the principal every 30 seconds and sends `session.ended` when the user or device session is no longer valid, so a revoked device stops receiving data within half a minute even if its stream stays open.

5.8.4 Event catalogue

Table 142. SSE event types, their audience and payload

Event	Published by	Audience filter	Payload
`ready`	Stream start	The connection	`kind` (`user` or `device`), `serverTime`
`session.ended`	Stream revalidation	The connection	`{}`

Event	Published by	Audience filter	Payload
`alarm.created`	Alarm engine on breach or stale data	Site, `alarm.read`	Full alarm event row
`alarm.updated`	Transitions, assignment, condition cleared, peak update	Site, `alarm.read`	Full row, or `{ id, clearedAt }` when the condition clears
`telemetry.reading`	Ingestion (newest reading only)	Site, `telemetry.read`	`sensorId`, `assetId`, `tag`, `value`, `unit`, `ts`, `quality`, `source`, `synthetic`
`telemetry.corrected`	Reading correction	Site, `telemetry.read`	`sensorId`, `readingId`, `action`
`sensor.updated`	Sensor edit	Site, `telemetry.read`	`id`
`asset.updated`	Asset edit, simulator offline event	Site, `asset.read`	`id`, `status`
`inspection.updated`	Create, edit, responses, transitions	Site, `inspection.read`	`id`, `status`, `assignedTo` or `progressChanged`
`maintenance.updated`	Create, edit, checklist, transitions, inspection follow-up	Site, `maintenance.read`	`id`, `status`
`auditdoc.updated`	Capture, enhance, review, verify, approve, rename	Site, `auditdoc.read`	`id`, `status`, `title`
`notification.created`	Any personal notification	That user	Full notification row
`device.paired`	Pairing completion	Site, `device.read`	`id`, `name`
`device.updated`	Assign, suspend, reactivate, revoke	Site, `device.read`	`id`, optionally `status`
`device.context_changed`	Reassign, site change, suspend, reactivate, operator deactivated	That device	`reason`
`device.revoked`	Device revoke	That device	`reason: "revoke"`
`device.profile_changed`	UI profile publish, rollback, assignment change	Affected devices	`profileId`, `version` or `reason`
`ui_profile.updated`	Same as above	`device.read`	`profileId`, `reason`, `affectedDevices`
`pairing.completed`	Pairing completion	The approving user	`challengeId`, `deviceId`
`simulation.step`	Demo simulator	Site, `telemetry.read`	`runId`, `scenario`, `step`, `events`

Glasses react to `device.context_changed`, `device.revoked` and `device.profile_changed` by refetching `GET /api/device/context` or returning to the pairing screen. The console invalidates its query cache by event type.

> **NOTE**
> A code comment in the Android client names an `alarm.raised` event. The server publishes `alarm.created`; there is no `alarm.raised` event.

5.9 Health

`app.ts` registers one route outside the modules.

GET /api/health

Liveness probe. Does not touch the database.

- **Auth:** public. **Permission:** none.
- **Response:** `200 { "ok": true, "time": "2026-10-22T08:14:03.512Z" }`.

5.10 Authentication module

File: `apps/api/src/modules/auth.ts`. All routes run in system context (`withSystem`) because they act on users and sessions, which are not tenant-partitioned. The refresh and demo cookies are scoped to the path `/api/auth`, so they are sent only to this module.

Table 143. Authentication endpoints

Method	Path	Auth	Limit	Purpose
POST	`/api/auth/demo`	public (demo mode)	20/min	One-click sign-in as a named guest
POST	`/api/auth/login`	public	10/min	Email and password sign-in
POST	`/api/auth/mfa/verify`	MFA token	10/min	Complete sign-in with a TOTP code
POST	`/api/auth/refresh`	cookie	60/min	Rotate the refresh cookie, new access token
POST	`/api/auth/logout`	cookie	global	End the session family
GET	`/api/auth/me`	user or platform admin	global	Current session view
POST	`/api/auth/switch-org`	user	global	Start a session in another organization
POST	`/api/auth/change-password`	user	global	Change password, end other sessions
POST	`/api/auth/password-reset/request`	public	5/min	Email a reset link
POST	`/api/auth/password-reset/complete`	public	10/min	Set a new password from a reset link
GET	`/api/auth/invitations/:token`	public	30/min	Preview an invitation
POST	`/api/auth/invitations/accept`	public	10/min	Accept an invitation, create or join account
POST	`/api/auth/mfa/setup`	user	global	Generate a TOTP secret
POST	`/api/auth/mfa/enable`	user	global	Turn MFA on
POST	`/api/auth/mfa/disable`	user	global	Turn MFA off
GET	`/api/auth/sessions`	user	global	List active sessions
DELETE	`/api/auth/sessions/:id`	user	global	Revoke one session

POST /api/auth/demo

Creates (or reuses) a named guest account in the demo tenant and signs it in. Available only when the server runs in demo mode; otherwise the route answers `404 not_found` as if it did not exist.

- **Auth:** public. **Limit:** 20 per minute.
- **Request:** `name`, 2 to 40 characters after trimming. Whitespace is collapsed and the name is NFKC-normalized.
- **Response:** `200` session view (see the authentication chapter) plus `Set-Cookie: hg_refresh` and `hg_demo`.
- **Errors:** `404 not_found` (demo mode off), `400 validation_failed` ("Enter your name").

- **Side effects:** audit `auth.login` with `metadata.demoGuest: true`; the guest receives copies of the demo owner's scheduled inspections and open maintenance tasks.

```
curl -s -c cookies.txt -X POST https://hydrisview.vercel.app/api/auth/demo \
  -H "content-type: application/json" \
  -d '{"name":"Priya Raman"}' | jq -r .accessToken
```

POST /api/auth/login

Signs in with email and password. Returns either a session or an MFA challenge.

- **Auth:** public. **Limit:** 10 per minute.
- **Request:** `email` (trimmed, lower-cased, valid email), `password` (1 to 200 characters).
- **Response:** `200` session view with `Set-Cookie: hg_refresh`, or `200 { "mfaRequired": true, "mfaToken": "..." }`.
- **Errors:** `401 invalid_credentials` (unknown email, wrong password, inactive account), `423 account_locked`.
- **Side effects:** audit `auth.login` or `auth.login_failed`; failure counter and lockout (5 failures, 15 minutes).

```
{ "email": "tarun@hydris.ai", "password": "correct horse battery staple" }
```

POST /api/auth/mfa/verify

Completes a sign-in that returned `mfaRequired`.

- **Auth:** the `mfaToken` from login, in the body. **Limit:** 10 per minute.
- **Request:** `{ "mfaToken": "...", "code": "123456" }`.
- **Response:** `200` session view with `Set-Cookie: hg_refresh`.
- **Errors:** `401 mfa_expired` (token older than 5 minutes or invalid), `401 unauthenticated` (wrong token type), `401 invalid_mfa_code`.
- **Side effects:** audit `auth.login` with `metadata.mfa: true`.

POST /api/auth/refresh

Rotates the refresh session and issues a new access token. Send it with credentials (cookies) included and no body.

- **Auth:** `hg_refresh` cookie (demo mode also accepts `hg_demo`). **Limit:** 60 per minute.
- **Response:** `200` session view with a new access token and new cookies.
- **Errors:** `401 no_session`, `401 session_expired`, `401 session_revoked`. Every 401 clears the cookies.
- **Side effects:** on replay outside the 30-second grace window, revokes the family and writes `auth.refresh_reuse_detected`.

POST /api/auth/logout

Revokes the session family of the presented refresh cookie and clears the cookies. Always succeeds.

- **Auth:** `hg_refresh` cookie (optional). **Response:** `200 { "ok": true }`.
- **Side effects:** audit `auth.logout` when a session was found.

GET /api/auth/me

Returns the session view for the bearer token. Unlike other user routes it also answers for platform administrators who have no organization membership (with `org: null`, `role: null` and empty permissions).

- **Auth:** user access token. **Response:** `200` session view without `accessToken`.
- **Errors:** `401 unauthenticated`.

POST /api/auth/switch-org

Moves the caller to another organization they belong to. The current session family is revoked and a new one is started; the target becomes the user's default organization.

- **Auth:** user. **Request:** `{ "orgId": "<uuid>" }`.
- **Response:** `200` session view for the new organization, new cookie.
- **Errors:** `403 forbidden` ("Not a member of that organization").

POST /api/auth/change-password

- **Auth:** user. **Request:** `{ "currentPassword": "...", "newPassword": "..." }`. The new password must be 12 to 200 characters and not one repeated character.
- **Response:** `200` session view. Every other session of the user is revoked; this call starts a fresh one.
- **Errors:** `400 weak_password`, `400 wrong_password`.
- **Side effects:** audit `auth.password_changed`.

POST /api/auth/password-reset/request

Sends a reset link valid for 30 minutes. The response is identical whether or not the account exists.

- **Auth:** public. **Limit:** 5 per minute. **Request:** `{ "email": "..." }`.
- **Response:** `200 { "ok": true }`.
- **Side effects:** a message in `mail_outbox` (the development mail transport), audit `auth.password_reset_requested` for existing active accounts.

POST /api/auth/password-reset/complete

- **Auth:** public. **Limit:** 10 per minute. **Request:** `{ "token": "...", "password": "..." }` (token at least 10 characters).
- **Response:** `200 { "ok": true }`. Clears the lockout, marks the email verified and revokes every session.
- **Errors:** `400 weak_password`, `400 invalid_token`.
- **Side effects:** audit `auth.password_reset`.

GET /api/auth/invitations/:token

Previews an invitation before the person accepts it.

- **Auth:** public. **Limit:** 30 per minute.
- **Response:** `200 { "email", "name", "role", "orgName", "existingAccount" }`. `existingAccount` tells the page to ask for the existing password instead of a new one.
- **Errors:** `400 invalid_invitation`.

POST /api/auth/invitations/accept

Accepts an invitation. Accepting doubles as email verification. If an account with the invited email exists, its password is required; otherwise a new account is created with the given password.

- **Auth:** public. **Limit:** 10 per minute.
- **Request:** `{ "token": "...", "name": "Priya Raman", "password": "..." }` (`name` optional, 2 to 120 characters).
- **Response:** `200` session view in the inviting organization.
- **Errors:** `400 invalid_invitation`, `400 wrong_password`, `400 weak_password`.
- **Side effects:** membership created or reactivated with the invited role and site scope; audit `member.invitation_accepted`.

POST /api/auth/mfa/setup

- **Auth:** user. **Response:** `200 { "secret": "JBSWY3DPEHPK3PXP...", "otpauthUrl": "otpauth://totp/Hydris:..." }`.
- **Errors:** `400 mfa_already_enabled`.
- **Notes:** the secret is stored encrypted but MFA is not on until `mfa/enable` succeeds.

POST /api/auth/mfa/enable

- **Auth:** user. **Request:** `{ "code": "123456" }`. **Response:** `200 { "ok": true }`.
- **Errors:** `400 invalid_mfa_code`. **Side effects:** audit `auth.mfa_enabled`.

POST /api/auth/mfa/disable

- **Auth:** user. **Request:** `{ "password": "...", "code": "123456" }`. **Response:** `200 { "ok": true }`.
- **Errors:** `400 wrong_password`, `400 invalid_mfa_code`. **Side effects:** secret deleted, audit `auth.mfa_disabled`.

GET /api/auth/sessions

Lists the caller's live sessions (not revoked, not rotated, not expired), newest first.

- **Auth:** user.
- **Response:** `200 { "sessions": [{ "id", "userAgent", "ip", "createdAt", "current" }] }`. `id` is the session family id; `current` marks the caller's own session.

DELETE /api/auth/sessions/:id

Revokes one session family of the caller (for example "sign out my tablet").

- **Auth:** user. **Response:** `200 { "ok": true }`. **Side effects:** audit `auth.session_revoked`.

5.11 Organization module

File: `apps/api/src/modules/org.ts`. Covers platform administration, organization settings, members and invitations, personal notifications and the audit log.

Table 144. Organization endpoints

Method	Path	Auth	Permission	Purpose
GET	`/api/platform/organizations`	platform	platform admin	List all tenants
POST	`/api/platform/organizations`	platform	platform admin	Create a tenant and invite its first admin
POST	`/api/platform/organizations/:id/status`	platform	platform admin	Suspend or reactivate a tenant
GET	`/api/org`	any	none	Current organization
PATCH	`/api/org`	user	`org.manage`	Rename, region, timezone
GET	`/api/members`	user	`member.read`	Members, pending invitations, assignable roles
GET	`/api/members/:userId`	user	`member.read`	Member detail with devices and work
POST	`/api/invitations`	user	`member.manage`	Invite a person
DELETE	`/api/invitations/:id`	user	`member.manage`	Revoke an invitation
PATCH	`/api/members/:userId`	user	`member.manage`	Role, title, site scope, status

Method	Path	Auth	Permission	Purpose
GET	`/api/notifications`	any	none	Own notifications
POST	`/api/notifications/read`	any	none	Mark own notifications read
GET	`/api/audit`	user	`audit.read`	Organization audit log

5.11.1 Platform administration

GET /api/platform/organizations

- **Auth:** user token of a platform administrator (membership not required). **Errors:** `401 unauthenticated`.
- **Response:** `200 { "organizations": [{ "id", "name", "slug", "isDemo", "status", "region", "createdAt", "members", "sites" }] }`.

POST /api/platform/organizations

Creates an organization and an invitation for its first organization administrator.

- **Auth:** platform administrator.
- **Request:**

```
{
  "name": "Coimbatore Municipal STP",
  "slug": "cbe-stp",
  "region": "Tamil Nadu",
  "timezone": "Asia/Kolkata",
  "isDemo": false,
  "admin": { "name": "Anand Kumar", "email": "anand@example.org" }
}
```

- **Response:** `201 { "organization": { ... }, "adminInvitationLink": "https://.../invite/<token>" }`.
- **Errors:** `409 slug_taken`, `400 validation_failed` (slug must match `^[a-z0-9-]{2,40}$`).
- **Side effects:** invitation email to `mail_outbox`, audit `organization.created`.

POST /api/platform/organizations/:id/status

- **Auth:** platform administrator. **Request:** `{ "status": "active" | "suspended" }`.
- **Response:** `200 { "organization": { ... } }`. **Errors:** `404 not_found`.
- **Notes:** a suspended organization fails principal resolution for every member and device, and its gateway keys stop working. Audit `organization.suspended` or `organization.reactivated`.

5.11.2 Organization settings

GET /api/org

- **Auth:** any. **Response:** `200 { "organization": { "id", "name", "slug", "isDemo", "status", "region", "timezone", ... } }`.

PATCH /api/org

- **Auth:** user. **Permission:** `org.manage`.
- **Request:** any of `name` (2 to 120), `region` (up to 80, nullable), `timezone` (up to 60).
- **Response:** `200 { "organization": { ... } }`. **Side effects:** audit `organization.updated`.

5.11.3 Members and invitations

GET /api/members

- **Auth:** user. **Permission:** `member.read`.
- **Response:** `200 { "members": [...], "invitations": [...], "assignableRoles": [...] }`. Each member carries `membershipId`, `userId`, `name`, `email`, `role`, `status`, `allSites`, `siteIds`, `title`, `lastLoginAt`, `mfaEnabled`, `devices` (active assigned glasses), `openInspections` and `openTasks`. Pending invitations are included only for organization administrators and factory managers.

GET /api/members/:userId

- **Auth:** user. **Permission:** `member.read`.
- **Response:** `200 { "member", "devices", "inspections", "tasks", "sessions" }` with the ten most recent inspections and tasks. `sessions` (live session families) is filled only for organization administrators.
- **Errors:** `404 not_found`.

POST /api/invitations

Invites a person by email. The link is also returned so administrators can share it when mail delivery is not configured.

- **Auth:** user. **Permission:** `member.manage`.
- **Request:**

```
{ "email": "karthik@example.org", "name": "Karthik S", "role": "field_operator", "allSites": false,
"siteIds": ["2d4c..."] }
```

- **Response:** `201 { "invitationLink": "https://.../invite/<token>" }`.
- **Errors:** `403 forbidden` (role not assignable by the caller, or sites outside the caller's scope), `400 sites_required`, `400 invalid_sites`, `409 already_member`.
- **Notes:** any open invitation for the same email is revoked first. Invitations expire after 7 days. Audit `member.invited`.

DELETE /api/invitations/:id

- **Auth:** user. **Permission:** `member.manage`. **Response:** `200 { "ok": true }`.
- **Errors:** `404 not_found` (unknown or already accepted). **Side effects:** audit `member.invitation_revoked`.

PATCH /api/members/:userId

Changes a member's role, title, site scope or status.

- **Auth:** user. **Permission:** `member.manage`.
- **Request:** any of `role`, `title` (up to 80, nullable), `allSites`, `siteIds` (replaces the list), `status` (`active` or `deactivated`).
- **Response:** `200 { "ok": true }`.
- **Errors:** `403 forbidden` (changing your own role or status, managing a role you cannot assign, granting sites outside your scope), `404 not_found`.
- **Side effects:** deactivation takes effect at once: the member's sessions in this organization are revoked, their glasses are unassigned and each device's session version is bumped, with `device.context_changed` (reason `operator_deactivated`) sent to each device. Audit `member.deactivated` or `member.updated`.

5.11.4 Notifications

GET /api/notifications

Returns the caller's 50 most recent notifications and the unread count. Glasses need the `notifications` screen enabled in their UI profile.

- **Auth:** any. **Response:** `200 { "notifications": [{ "id", "kind", "title", "body", "link", "severity", "entityType", "entityId", "readAt", "createdAt" }], "unread": 3 }`.

POST /api/notifications/read

Marks notifications read. Without `ids`, marks all of the caller's unread notifications.

- **Auth:** any. **Request:** `{ "ids": ["<uuid>", ...] }` (optional, up to 200). **Response:** `200 { "ok": true }`.
- **Notes:** glasses need the `notifications` screen and the `mark_notifications_read` action.

5.11.5 Audit log

GET /api/audit

Searches the append-only audit trail of the organization, newest first.

- **Auth:** user. **Permission:** `audit.read`.
- **Query:** `action` (prefix match, for example `alarm.`), `entityType`, `q` (summary or actor name contains), `from`, `to`, `limit` (1 to 500, default 100), `before` (exclusive cursor: pass the `at` of the last row).
- **Response:** `200 { "events": [{ "id", "action", "entityType", "entityId", "summary", "metadata", "ip", "at", "actorUserId", "actorDeviceId", "actorName" }] }`. `actorName` falls back to the actor label (`alarm engine`, `simulator`, `glasses`) for non-human actors.

5.12 Plant hierarchy and records module

File: `apps/api/src/modules/plant.ts`. The plant is modelled as site, facility, process area, asset and sensor. This module also owns edge gateway credentials, documents (SOPs, manuals, datasheets), file attachments, per-record change history and the navigator tree. Codes and tags (site code, facility code, process area code, asset tag, sensor tag) are 1 to 24 characters from `[A-Za-z0-9._-]`. Delete is offered only for assets without history; everything else is archived with an `archived` flag so operational history stays intact.

Table 145. Plant endpoints

Method	Path	Auth	Permission	Purpose
GET	`/api/sites`	any	scope only	Sites with counts
POST	`/api/sites`	user	`site.manage` (organization-wide)	Create a site
GET	`/api/sites/:id`	any	scope only	Site with facilities, areas, assets, gateways
PATCH	`/api/sites/:id`	user	`site.manage`	Edit a site
POST	`/api/sites/:id/archive`	user	`site.manage`	Archive or restore
POST	`/api/facilities`	user	`site.manage`	Create a facility
PATCH	`/api/facilities/:id`	user	`site.manage`	Edit, archive or restore
POST	`/api/process-areas`	user	`site.manage`	Create a process area
PATCH	`/api/process-areas/:id`	user	`site.manage`	Edit, archive or restore
GET	`/api/assets`	any	scope only	Filtered asset list with health

Method	Path	Auth	Permission	Purpose
POST	`/api/assets`	user	`asset.manage`	Create an asset
GET	`/api/assets/:id`	any	scope only	Asset detail with sensors, alarms, work, documents, files
PATCH	`/api/assets/:id`	any	`asset.manage`, or `inspection.execute` for status and notes only	Edit, archive or restore
DELETE	`/api/assets/:id`	user	`asset.manage`	Delete an asset without history
GET	`/api/sensors`	any	scope only	Sensors with freshness
POST	`/api/sensors`	user	`sensor.manage`	Create a sensor
GET	`/api/sensors/:id`	any	`telemetry.read`	Sensor detail, including archived
PATCH	`/api/sensors/:id`	user	`sensor.manage`	Edit, bind gateway, archive or restore
POST	`/api/gateways`	user	`gateway.manage`	Create an edge gateway, returns its key once
POST	`/api/gateways/:id/revoke`	user	`gateway.manage`	Revoke a gateway key
GET	`/api/documents`	any	`document.read`	Search documents
GET	`/api/documents/:id`	any	`document.read`	One document
POST	`/api/documents`	user	`document.manage`	Create a document
PATCH	`/api/documents/:id`	user	`document.manage`	Edit, approve, archive
POST	`/api/attachments`	any	depends on owner type	Upload a file
GET	`/api/attachments/:id`	any	tenant	Download a file
GET	`/api/history/:entityType/:entityId`	any	depends on entity type	Change history of one record
GET	`/api/hierarchy`	any	scope only	Site to sensor navigator tree

5.12.1 Sites

GET /api/sites

- **Auth:** any. **Permission:** site scope only.
- **Query:** `archived` (`"true"` includes archived sites; default `"false"`).
- **Response:** `200 { "sites": [{ ...site, "facilities": 2, "assets": 31, "openAlarms": 4, "criticalAlarms": 1 }] }`. Site fields include `id`, `name`, `code`, `address`, `city`, `country`, `lat`, `lng`, `timezone`, `layoutWidthM`, `layoutHeightM`, `archivedAt`.

POST /api/sites

- **Auth:** user. **Permission:** `site.manage`, and the caller must be organization-wide.
- **Request:**

```
{ "name": "Ukkadam STP", "code": "UKD", "city": "Coimbatore", "country": "India", "lat": 10.99, "lng": 76.96, "timezone": "Asia/Kolkata", "layoutWidthM": 240, "layoutHeightM": 160 }
```

- **Response:** `201 { "site": { ... } }`.

- **Errors:** `400 forbidden_scope`, `409 code_taken`. **Side effects:** audit `site.created`.
- **Notes:** `layoutWidthM` and `layoutHeightM` (10 to 5000 metres) size the plant map shown on the glasses.

GET /api/sites/:id

- **Auth:** any. **Permission:** site scope.
- **Response:** `200 { "site", "facilities", "processAreas", "assets", "gateways" }`. Assets carry the same health summary as `GET /api/assets`. Gateways show `keyPrefix` but never the key.
- **Errors:** `404 not_found`.

PATCH /api/sites/:id

- **Auth:** user. **Permission:** `site.manage` and site scope. **Request:** any site field.
- **Response:** `200 { "site": { ... } }`. **Side effects:** audit `site.updated`.

POST /api/sites/:id/archive

- **Auth:** user. **Permission:** `site.manage`. **Request:** `{ "archived": true }` or `false` to restore.
- **Response:** `200 { "site": { ... } }`. **Side effects:** audit `site.archived` or `site.restored`.

5.12.2 Facilities and process areas

POST /api/facilities

- **Auth:** user. **Permission:** `site.manage` and scope over `siteId`.
- **Request:** `siteId`, `name`, `code`, `kind` (`etp`, `stp`, `wtp`, `cetp`, `other`), optional `capacityM3PerDay` and `description`.
- **Response:** `201 { "facility": { ... } }`. **Errors:** `404 not_found` (site), `409 code_taken`.
- **Side effects:** audit `facility.created`.

PATCH /api/facilities/:id

- **Auth:** user. **Permission:** `site.manage`. **Request:** any facility field except `siteId`, plus `archived`.
- **Response:** `200 { "facility": { ... } }`. **Side effects:** audit `facility.updated`.

POST /api/process-areas

- **Auth:** user. **Permission:** `site.manage`.
- **Request:** `facilityId`, `name`, `code`, `kind` (intake, screening, equalization, primary_treatment, aeration, clarification, filtration, disinfection, sludge_handling, chemical_dosing, discharge, utilities, other), optional `sortOrder` and `polygon` (3 to 64 `[x, y]` points in metres on the site layout).
- **Response:** `201 { "processArea": { ... } }`. The site is taken from the facility.
- **Errors:** `404 not_found`, `409 code_taken`. **Side effects:** audit `process_area.created`.

PATCH /api/process-areas/:id

- **Auth:** user. **Permission:** `site.manage`. **Request:** any field except `facilityId`, plus `archived`.
- **Response:** `200 { "processArea": { ... } }`. **Side effects:** audit `process_area.updated`.

5.12.3 Assets

GET /api/assets

- **Auth:** any. **Permission:** site scope.
- **Query:** `siteId`, `facilityId`, `processAreaId`, `type`, `status`, `q` (tag, name or manufacturer contains), `archived`.

- **Response:** `200 { "assets": [...] }`, ordered by tag. Each asset adds `areaName`, `facilityName`, `siteName`, `openAlarms`, `worstSeverity`, `sensorCount` and `openTasks` to its row.

```
{
  "assets": [
    {
      "id": "a91f...", "tag": "P-101", "name": "Raw sewage lift pump 1", "type": "pump", "status":
"operational",
      "criticality": "high", "siteName": "Ukkadam STP", "facilityName": "STP 1", "areaName": "Intake",
      "openAlarms": 1, "worstSeverity": "high", "sensorCount": 3, "openTasks": 0
    }
  ]
}
```

POST /api/assets

- **Auth:** user. **Permission:** `asset.manage`.
- **Request:** `facilityId`, `tag`, `name`, `type` (pump, blower, tank, valve, mixer, clarifier, filter, dosing_pump, analyzer, motor, screen, diffuser, uv_unit, other), optional `processAreaId`, `manufacturer`, `model`, `serialNumber`, `installedOn`, `criticality` (`low`, `medium`, `high`), `status`, `specs` (string to string or number map), `notes`, `positionX`, `positionY` (metres on the site layout).
- **Response:** `201 { "asset": { ... } }`.
- **Errors:** `404 not_found` (facility), `400 invalid_process_area`, `409 tag_taken`.
- **Side effects:** audit `asset.created`.

GET /api/assets/:id

- **Auth:** any. **Permission:** site scope.
- **Response:** `200 { "asset", "sensors", "alarms", "maintenance", "inspections", "documents", "attachments" }`. Sensors carry `freshness` (`live`, `delayed`, `stale`, `no_data`). Alarms, tasks and inspections are the 20 most recent. Documents are approved ones linked to the asset or to its asset type.
- **Errors:** `404 not_found`.

PATCH /api/assets/:id

- **Auth:** any. **Permission:** `asset.manage`; a body containing only `status` and `notes` needs only `inspection.execute`, so operators can record an asset as degraded from the field.
- **Request:** any asset field except `facilityId`, plus `archived`.
- **Response:** `200 { "asset": { ... } }`.
- **Errors:** `400 validation_failed` (process area in another facility), `409 tag_taken`.
- **Side effects:** audit `asset.updated`, `asset.archived` or `asset.restored` with field-level `changes`; SSE `asset.updated`.

DELETE /api/assets/:id

- **Auth:** user. **Permission:** `asset.manage`. **Response:** `200 { "ok": true }`.
- **Errors:** `409 in_use` when sensors, alarms, inspections or maintenance tasks reference the asset.
- **Side effects:** audit `asset.deleted`.

5.12.4 Sensors

GET /api/sensors

- **Auth:** any. **Permission:** site scope. Glasses need the `measurements` or `assets` screen.
- **Query:** `siteId`, `assetId`, `measurand`, `q`, `archived` (`"false"` default, `"true"` all, `"only"` archived).

- **Response:** `200 { "sensors": [...] }`. Each sensor adds `assetTag`, `assetName`, `openAlarms`, `rules` (enabled rule count) and `freshness`; the row carries `lastValue`, `lastTs`, `lastQuality`, `lastSource`, `unit`, `method` and `expectedIntervalS`.

POST /api/sensors

- **Auth:** user. **Permission:** `sensor.manage`.
- **Request:** `assetId`, `tag`, `name`, `measurand` (16 values: ph, temperature, dissolved_oxygen, turbidity, conductivity, flow_rate, tank_level, pressure, motor_current, vibration, runtime_hours, tss, cod, bod, ammonia, residual_chlorine), optional `method` (`continuous`, `lab`, `manual`), `expectedIntervalS` (1 s to 30 days), `gatewayId`, `sourceRef`, `enabled`.
- **Response:** `201 { "sensor": { ... } }`. The unit comes from the measurand definition; method and interval default to the measurand's (60 seconds for continuous, one day otherwise).
- **Errors:** `404 not_found` (asset), `409 tag_taken`. **Side effects:** audit `sensor.created`.

GET /api/sensors/:id

Returns one sensor, including archived ones, with what the edit form needs.

- **Auth:** any. **Permission:** `telemetry.read` and site scope.
- **Response:** `200 { "sensor", "asset", "gateway", "gateways", "counts" }`. `sensor` adds `freshness`, `label` and `decimals`; `gateways` lists the site's active gateways; `counts` gives `rules`, `readings` and `corrections`.

PATCH /api/sensors/:id

- **Auth:** user. **Permission:** `sensor.manage`. **Request:** any field except `assetId` and `measurand`, plus `archived`.
- **Response:** `200 { "sensor": { ... } }`.
- **Errors:** `400 validation_failed` (gateway on another site or revoked), `409 source_ref_taken`, `409 tag_taken`.
- **Side effects:** audit `sensor.updated`, `sensor.archived` or `sensor.restored`; SSE `sensor.updated`.
- **Notes:** `sourceRef` is the identifier a gateway sends instead of the tag (for example a PLC register name); it must be unique on the site against both tags and other source references.

5.12.5 Gateways

POST /api/gateways

Creates an edge gateway credential for one site.

- **Auth:** user. **Permission:** `gateway.manage` and site scope.
- **Request:** `{ "siteId": "<uuid>", "name": "STP-1 edge box", "protocol": "https" }`. `protocol` is `https` (default), `mqtt` or `opcua`; only HTTPS ingestion exists today (see the planned API chapter).
- **Response:**

```
{ "gateway": { "id": "e3b7...", "name": "STP-1 edge box", "keyPrefix": "hgw_Zk3pQ" }, "apiKey":
"hgw_Zk3pQ...full key..." }
```

- **Side effects:** audit `gateway.created`.

> **IMPORTANT**
>
> `apiKey` is returned exactly once. Only its SHA-256 hash and a 10-character prefix are stored. A lost key cannot be recovered: revoke the gateway and create a new one.

POST /api/gateways/:id/revoke

- **Auth:** user. **Permission:** `gateway.manage`. **Response:** `200 { "ok": true }`.
- **Side effects:** the key stops working on the next request; audit `gateway.revoked`.

5.12.6 Documents

Documents are SOPs, manuals, datasheets, safety notes and troubleshooting guides. They are versioned: editing the body of an approved document increments `version` and returns it to `draft` until approved again. Field operators, viewers and glasses see approved documents only.

GET /api/documents

- **Auth:** any. **Permission:** `document.read`. Glasses need the `sops` screen and `view_sops` action.
- **Query:** `q` (title contains or full-text match on the body), `kind`, `assetId`, `status` (`draft`, `approved`, `archived`).
- **Response:** `200 { "documents": [{ "id", "title", "kind", "status", "version", "assetId", "assetType", "siteId", "tags", "updatedAt", "approvedAt" }] }`. Organization-wide documents (no site) are visible to every site.

GET /api/documents/:id

- **Auth:** any. **Permission:** `document.read`. Glasses need `view_sops`.
- **Response:** `200 { "document": { ..., "body": "markdown" } }`.
- **Errors:** `404 not_found` (missing, out of scope, or not approved for operators, viewers and devices).

POST /api/documents

- **Auth:** user. **Permission:** `document.manage`.
- **Request:** `title` (3 to 160), `kind`, `body` (up to 100,000 characters), optional `siteId`, `assetId`, `assetType`, `tags` (up to 20).
- **Response:** `201 { "document": { ... } }` in `draft`. **Side effects:** audit `document.created`.

PATCH /api/documents/:id

- **Auth:** user. **Permission:** `document.manage`.
- **Request:** any document field plus `status`. Setting `status: "approved"` records the caller as approver.
- **Response:** `200 { "document": { ... } }`.
- **Errors:** `404 not_found`, `400 forbidden_scope` (making a document organization-wide without organization-wide scope), `400 validation_failed` (asset on another site).
- **Side effects:** audit `document.approved`, `document.archived`, `document.restored` or `document.updated` with field changes, `contentChanged` and the new version.

5.12.7 Attachments

POST /api/attachments

Uploads one evidence file and links it to an inspection, maintenance task, asset or alarm.

- **Auth:** any. **Content type:** `multipart/form-data` with fields `ownerType` (`inspection`, `maintenance`, `asset`, `alarm`), `ownerId` and one file (at most 10 MB).
- **Permission:** by owner type: `inspection.execute`, `maintenance.request`, `asset.manage` or `alarm.acknowledge`; the owner record must be in scope.
- **Response:** `201 { "attachment": { "id", "ownerType", "ownerId", "filename", "mime", "sizeBytes", "sha256", "uploadedBy", "deviceId", "createdAt" } }`.

- **Errors:** `400 file_required`, `400 unsupported_type`, `400 content_mismatch`, `404 not_found`.
- **Notes:** allowed types are JPEG, PNG, WebP, PDF and plain text, verified against the file's magic bytes. Files are stored under a random key on disk or in a private Vercel Blob store and are never exposed as public URLs. Audit `attachment.uploaded`.

```
curl -X POST https://hydrisview.vercel.app/api/attachments \
  -H "authorization: Bearer $TOKEN" \
  -F ownerType=maintenance -F ownerId=4be1... -F file=@seal-leak.jpg
```

GET /api/attachments/:id

Streams a stored file with its MIME type, `content-disposition: inline` and `x-content-type-options: nosniff`.

- **Auth:** any. **Errors:** `404 not_found`.
- **Notes:** the lookup runs under the tenant's RLS. The handler does not re-check the site scope of the owning record, so any authenticated member of the organization who knows an attachment id can download it. Clients use the authenticated blob helper so the token never appears in a URL.

5.12.8 History and hierarchy

GET /api/history/:entityType/:entityId

Returns the audit events of one record, newest first, for the "History" panels in the console.

- **Auth:** any. **Path:** `entityType` is one of `asset`, `sensor`, `alarm_rule`, `maintenance`, `document`, `inspection_template`.
- **Permission:** `asset.read`, `telemetry.read`, `alarm.read`, `maintenance.read`, `document.read` or `inspection.read` respectively; the record must be in scope. Non-approved documents need `document.manage`.
- **Query:** `limit` (1 to 500, default 100).
- **Response:** `200 { "events": [{ "id", "action", "summary", "metadata", "at", "actor": { "id", "name" } }] }`. Actors are users, devices (named "Device" or their label) or the system.

GET /api/hierarchy

Returns the full navigator tree for the caller's active sites in one call: sites, facilities, process areas, assets (with `openAlarms` and `worstSeverity`), each asset's sensors (with `lastValue`, `lastTs` and `freshness`), and per facility the `unassignedAssets` that have no process area.

- **Auth:** any. **Permission:** site scope.
- **Response:** `200 { "sites": [{ "id", "name", "code", "facilities": [{ ..., "processAreas": [{ ..., "assets": [...] }], "unassignedAssets": [...] }] }] }`.

5.13 Telemetry module

File: `apps/api/src/modules/telemetry.ts`. Every reading, whatever its source, goes through one function, `ingest()`: validation, unit normalization, idempotent storage, out-of-order handling, alarm evaluation and realtime fan-out. The sources are edge gateways (`gateway`), people entering values in the console (`manual` and `lab`) and the demo simulator (`simulator`). Stored readings are never modified; corrections are appended in a separate table and the effective value of a reading is its most recent correction.

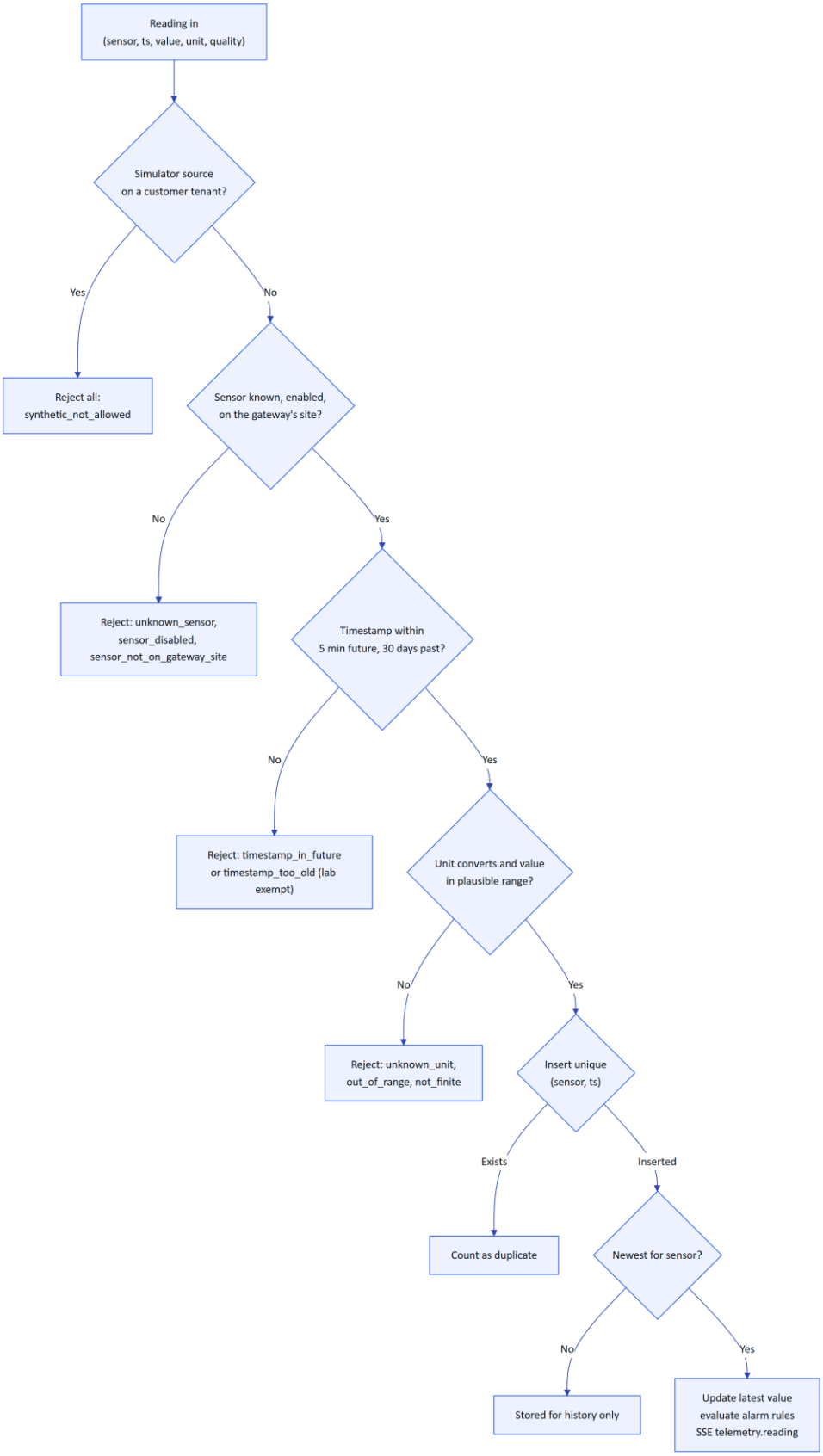

Figure 106. *The single ingestion pipeline shared by gateways, manual and lab entry, and the demo simulator*

Readings are processed in timestamp order within a batch. A reading older than the sensor's latest is stored for history and exports but does not overwrite the latest value or re-trigger alarms. Readings with quality `bad` update the latest value but are not evaluated against alarm rules.

Table 146. Telemetry endpoints

Method	Path	Auth	Permission	Purpose
POST	`/api/ingest/v1/readings`	gateway	gateway key	Batch ingestion from an edge gateway
POST	`/api/telemetry/manual`	user or device	`inspection.execute` (manual) or `sensor.manage` (lab)	Record one reading by hand
GET	`/api/telemetry/sensors/:id/history`	any	`telemetry.read`	Downsampled chart series
GET	`/api/telemetry/sensors/:id/export.csv`	any	`report.export`	Raw readings as CSV
GET	`/api/telemetry/sensors/:id/readings`	any	`telemetry.read`	Paged raw readings with provenance
POST	`/api/telemetry/readings/:readingId/corrections`	user	`sensor.manage`	Correct or invalidate a reading

POST /api/ingest/v1/readings

Accepts a batch of up to 500 readings from an edge gateway. The gateway identifies sensors by tag or by `sourceRef`, and only sensors on the gateway's own site are accepted.

- **Auth:** `Authorization: Bearer hgw_...`. The key must be active and its organization active.
- **Limit:** 1200 requests per minute per IP.
- **Request:**

```
{
  "readings": [
    { "sensor": "PT-101-VIB", "ts": "2026-10-22T13:44:00+05:30", "value": 7.4, "unit": "mm/s",
"quality": "good", "messageId": "plc1-000918" },
    { "sensor": "40001", "ts": "2026-10-22T13:44:00+05:30", "value": 6.92 }
  ]
}
```

- **Response:** `200`. `rejected[].index` refers to the position in the request array.

```
{ "accepted": 1, "duplicates": 0, "alarmsRaised": 1, "rejected": [{ "index": 1, "reason":
"unknown_sensor" }] }
```

- **Errors:** `401 invalid_gateway_key`, `400 validation_failed` (`ts` must be ISO 8601 with offset, 1 to 500 readings).
- **Side effects:** gateway `lastSeenAt` updated; sensor latest values; alarm events and notifications; SSE `telemetry.reading` and `alarm.created`.
- **Notes:** a partially bad batch still returns 200; inspect `rejected`. Rejection reasons are `unknown_sensor`, `sensor_disabled`, `sensor_not_on_gateway_site`, `timestamp_in_future`, `timestamp_too_old`, `unknown_unit`, `out_of_range` and `not_finite`. `messageId` is stored for traceability; deduplication is by sensor and timestamp.

POST /api/telemetry/manual

Records one reading entered by a person, for example a lab COD result or a hand-held pH reading.

- **Auth:** user or device. **Permission:** `inspection.execute` for `source: "manual"`, `sensor.manage` for `source: "lab"`; the sensor must be in scope.

- **Request:** `{ "sensorId": "<uuid>", "value": 182, "unit": "mg/L", "ts": "2026-10-22T09:00:00+05:30", "source": "lab" }` (`ts` defaults to now, `source` to `manual`).
- **Response:** `201 { "accepted": 1, "duplicates": 0, "alarmsRaised": 0, "rejected": [], "readingId": 99812, "recordedBy": { "id": "..." } }`.
- **Errors:** `400 method_mismatch`, `400 reading_rejected` (with `details.reason`), `409 duplicate_reading`.
- **Side effects:** audit `telemetry.manual_entry` (the provenance record of who entered the value).

GET /api/telemetry/sensors/:id/history

Returns a chart-ready series. When the range holds more readings than `points`, the server averages them into equal time buckets and returns min, max and count per bucket; otherwise it returns the raw readings. Invalidated readings are excluded and corrected readings use their corrected value.

- **Auth:** any. **Permission:** `telemetry.read`. Glasses need the `measurements` or `assets` screen.
- **Query:** `from` (default `to` minus 24 hours), `to` (default now), `points` (10 to 2000, default 240).
- **Response:**

```
{
  "sensor": { "id": "c0d1...", "tag": "PT-101-VIB", "unit": "mm/s", "freshness": "live", "label":
"Vibration", "decimals": 1, "typicalRange": [0, 4.5] },
  "from": "2026-10-21T08:00:00.000Z", "to": "2026-10-22T08:00:00.000Z",
  "bucketSeconds": 360, "totalReadings": 1440,
  "points": [{ "t": "2026-10-21T08:00:00.000Z", "v": 3.12, "min": 2.9, "max": 3.4, "n": 6 }],
  "corrections": { "corrected": 1, "invalidated": 2 }
}
```

- **Errors:** `400 invalid_range`, `404 not_found`. `bucketSeconds` is `null` when raw readings are returned.

GET /api/telemetry/sensors/:id/export.csv

Exports raw readings with original and effective values and the latest correction of each.

- **Auth:** any. **Permission:** `report.export`.
- **Query:** `from` and `to` (both required, at most 93 days apart).
- **Response:** `200 text/csv`, file name `sensor-<id>.csv`, columns `timestamp_utc, sensor_tag, original_value, effective_value, unit, quality, source, synthetic, status, correction_reason, corrected_by, corrected_at_utc`. Cells that start with `=`, `+`, `-` or `@` and are not numbers are prefixed with a quote to defuse spreadsheet formula injection.
- **Errors:** `400 range_too_large`. **Side effects:** audit `report.exported`.

GET /api/telemetry/sensors/:id/readings

Pages through raw readings newest first, with who recorded manual values and the full correction trail.

- **Auth:** any. **Permission:** `telemetry.read`.
- **Query:** `limit` (1 to 200, default 50), `offset`, `from`, `to`, `source` (`gateway`, `manual`, `lab`, `simulator`), `status` (`all`, `corrected`, `invalidated`).
- **Response:** `200 { "sensor", "total", "limit", "offset", "readings": [...] }`. Each reading has `id` (integer), `ts`, `receivedAt`, `quality`, `source`, `synthetic`, `originalValue`, `effectiveValue` (null when invalidated), `status` (`original`, `corrected`, `invalidated`), `recordedBy` and `corrections[]`.

POST /api/telemetry/readings/:readingId/corrections

Appends a correction or invalidation to one stored reading. The original row is never changed.

- **Auth:** user. **Permission:** `sensor.manage` and scope over the sensor.
- **Request:** `{ "action": "correct", "correctedValue": 7.1, "unit": "pH", "reason": "Probe recalibrated after drift" }` or `{ "action": "invalidate", "reason": "Sensor was out of the`

`tank during cleaning" }`. `reason` is 5 to 1000 characters; `correctedValue` is required for `correct` and forbidden for `invalidate`.

- **Response:** `201 { "correction": { ... }, "effectiveValue": 7.1 }`.
- **Errors:** `404 not_found`, `400 validation_failed` (value implausible for the measurand), `409 no_change`, `409 already_invalidated`.
- **Side effects:** audit `telemetry.reading_corrected` or `telemetry.reading_invalidated`; the sensor's latest value is recomputed if this reading was the latest; SSE `telemetry.corrected`.

5.14 Alarms module

File: `apps/api/src/modules/alarms.ts`. Alarm rules are thresholds on one sensor: `high_high`, `high`, `low`, `low_low` with a deadband and an on-delay, or `stale` (no fresh data for five expected intervals). The engine raises one alarm event per breach and tracks two independent things: the measured condition (`clearedAt` is set when the value returns to normal) and the human lifecycle below. A cleared alarm stays open until people resolve it.

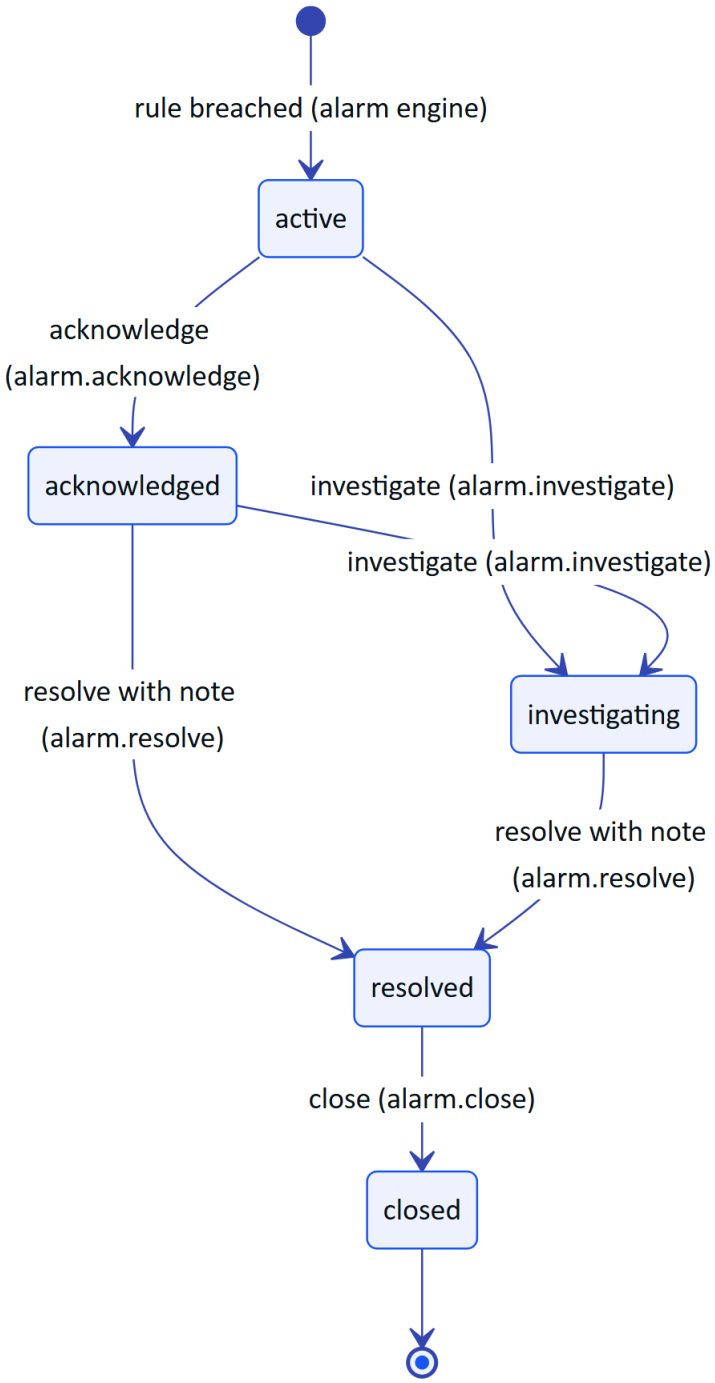

Figure 107. *Alarm lifecycle (ALARM_TRANSITIONS in packages/domain/src/alarms.ts) with the permission each action needs*

Critical and high alarms notify every active member with an operational role who has access to the site. Assignment notifies the assignee.

Table 147. Alarm endpoints

Method	Path	Auth	Permission	Purpose
GET	`/api/alarms`	any	`alarm.read`	Filtered alarm list, most urgent first
GET	`/api/alarms/:id`	any	`alarm.read`	Alarm with history, rule, procedure, follow-ups, allowed actions
POST	`/api/alarms/:id/:action`	any	per action	acknowledge, investigate, resolve, close

Method	Path	Auth	Permission	Purpose
POST	`/api/alarms/:id/assign`	user	`alarm.assign`	Assign or unassign
POST	`/api/alarms/:id/notes`	any	`alarm.acknowledge`	Add a note to the history
GET	`/api/alarm-rules`	any	`alarm.read`	List rules
POST	`/api/alarm-rules`	user	`alarm_rule.manage`	Create a rule
PATCH	`/api/alarm-rules/:id`	user	`alarm_rule.manage`	Edit, enable or disable a rule
DELETE	`/api/alarm-rules/:id`	user	`alarm_rule.manage`	Delete, or disable if it has history

GET /api/alarms

- **Auth:** any. **Permission:** `alarm.read`. Glasses need the `alerts` screen.
- **Query:** `state` (one of the five states, or `open` for active, acknowledged and investigating), `severity` (`critical`, `high`, `medium`, `low`), `siteId`, `assetId`, `assignedToMe`, `from`, `to` (on `triggeredAt`), `limit` (1 to 500, default 200).
- **Response:** `200 { "alarms": [...] }` ordered by state (active first), then severity, then newest. Each alarm adds `assetTag`, `assetName`, `sensorTag` and `assigneeName`.

```
curl -s "https://hydrisview.vercel.app/api/alarms?state=open&severity=critical" \
  -H "authorization: Bearer $TOKEN" | jq '.alarms[] | {id, title, state, triggerValue, unit}'

{
  "alarms": [
    {
      "id": "5f0c2a7e-...", "siteId": "2d4c...", "assetId": "a91f...", "sensorId": "c0d1...", "ruleId":
"77aa...",
      "severity": "critical", "condition": "high_high", "state": "active",
      "title": "High-high Vibration · P-101", "description": "Vibration 9.8 mm/s is above the high-high
limit 9 mm/s",
      "recommendedAction": "Stop the pump and check the coupling", "triggerValue": 9.8, "threshold": 9,
"unit": "mm/s",
      "peakValue": 10.2, "triggeredAt": "2026-10-22T08:10:00.000Z", "clearedAt": null, "assignedTo":
null,
      "synthetic": false, "assetTag": "P-101", "assetName": "Raw sewage lift pump 1", "sensorTag": "PT-
101-VIB", "assigneeName": null
    }
  ]
}
```

GET /api/alarms/:id

- **Auth:** any. **Permission:** `alarm.read` and site scope. Glasses need the `alerts` screen.
- **Response:** `200 { "alarm", "history", "rule", "procedure", "followUps", "assignee", "allowedActions", "canAssign" }`. `history` lists triggered, condition_cleared, transitions, assignments and notes with actor names (`Hydris` for the engine). `procedure` is the SOP linked to the rule. `allowedActions` lists the transitions valid from the current state that the caller holds the permission for, so clients render exactly the right buttons.
- **Errors:** `404 not_found`.

POST /api/alarms/:id/:action

Moves an alarm through its lifecycle. `action` is `acknowledge`, `investigate`, `resolve` or `close`.

- **Auth:** any. **Permission:** `alarm.acknowledge`, `alarm.investigate`, `alarm.resolve` or `alarm.close`. Glasses need the `alerts` screen, and for acknowledge also the `acknowledge_alarm` action.
- **Request:** `{ "note": "..." }` (optional, up to 2000; required for `resolve`).
- **Response:** `200 { "alarm": { ...updated row } }`.

- **Errors:** `409 invalid_transition` (with `details.state`), `400 note_required`, `409 stale_state`, `404 not_found`.
- **Side effects:** acknowledge records `acknowledgedBy` and `acknowledgedAt`; investigate self-assigns if unassigned; resolve records the resolution note; history row; audit `alarm.<action>` with `via: user | device`; SSE `alarm.updated`.

```
curl -s -X POST https://hydrisview.vercel.app/api/alarms/5f0c2a7e-.../acknowledge \
  -H "authorization: Bearer $TOKEN" -H "content-type: application/json" -d '{}'
```

POST /api/alarms/:id/assign

- **Auth:** user. **Permission:** `alarm.assign`.
- **Request:** `{ "userId": "<uuid>" | null, "note": "..." }`.
- **Response:** `200 { "alarm": { ... } }`.
- **Errors:** `409 invalid_transition` (closed alarms), `400 invalid_assignee`, `404 not_found`.
- **Side effects:** history `assigned`, audit `alarm.assigned`, notification `alarm_assigned`, SSE `alarm.updated`.

POST /api/alarms/:id/notes

- **Auth:** any. **Permission:** `alarm.acknowledge`. **Request:** `{ "note": "Coupling guard loose, tightened" }` (1 to 2000). **Response:** `200 { "ok": true }`.
- **Notes:** adds a `note` history row attributed to the user and, from glasses, the device. No audit event.

GET /api/alarm-rules

- **Auth:** any. **Permission:** `alarm.read`. **Query:** `sensorId`, `siteId`.
- **Response:** `200 { "rules": [{ ...rule, "sensorTag", "sensorName", "unit", "assetTag" }] }`. Rule fields: `id`, `sensorId`, `siteId`, `name`, `condition`, `threshold`, `deadband`, `onDelayS`, `severity`, `enabled`, `recommendedAction`, `procedureDocumentId`, `breachSince`.

POST /api/alarm-rules

- **Auth:** user. **Permission:** `alarm_rule.manage`.
- **Request:**

```
{ "sensorId": "c0d1...", "name": "Pump vibration high-high", "condition": "high_high", "threshold": 9,
"deadband": 0.5, "onDelayS": 30, "severity": "critical", "recommendedAction": "Stop the pump and check
the coupling", "procedureDocumentId": "d3e4..." }
```

- **Response:** `201 { "rule": { ... } }`.
- **Errors:** `400 threshold_required`, `404 not_found` (sensor).
- **Side effects:** audit `alarm_rule.created` and `sensor.threshold_added` (so the sensor's own history shows it).
- **Notes:** `onDelayS` (0 to 86,400) is how long a breach must persist before an alarm is raised. `deadband` is the hysteresis before a breach counts as cleared.

PATCH /api/alarm-rules/:id

- **Auth:** user. **Permission:** `alarm_rule.manage`. **Request:** any rule field except `sensorId`.
- **Response:** `200 { "rule": { ... } }`. Any pending breach timer is reset.
- **Errors:** `400 validation_failed` (missing threshold, unknown procedure document), `404 not_found`.
- **Side effects:** audit `alarm_rule.updated`, `alarm_rule.enabled` or `alarm_rule.disabled` with before and after, plus `sensor.threshold_changed`.

DELETE /api/alarm-rules/:id

- **Auth:** user. **Permission:** `alarm_rule.manage`.

- **Response:** `200 { "ok": true, "disabled": false }`. A rule that has raised alarms is disabled instead of deleted (`"disabled": true`) so past alarm events keep their context.
- **Side effects:** audit `alarm_rule.deleted` or `alarm_rule.disabled`.

5.15 Inspections module

File: `apps/api/src/modules/inspections.ts`. Templates define the steps of an inspection; routes group assets into a round; inspections are the assigned work items. Each inspection stores a snapshot of its template's steps and safety notices at creation, so editing a template never changes work in progress. Step kinds are `check`, `reading`, `choice`, `text` and `photo`; a response is `abnormal` when a check fails or is flagged, a reading falls outside `expectedMin` to `expectedMax`, or a choice is in `abnormalOptions`.

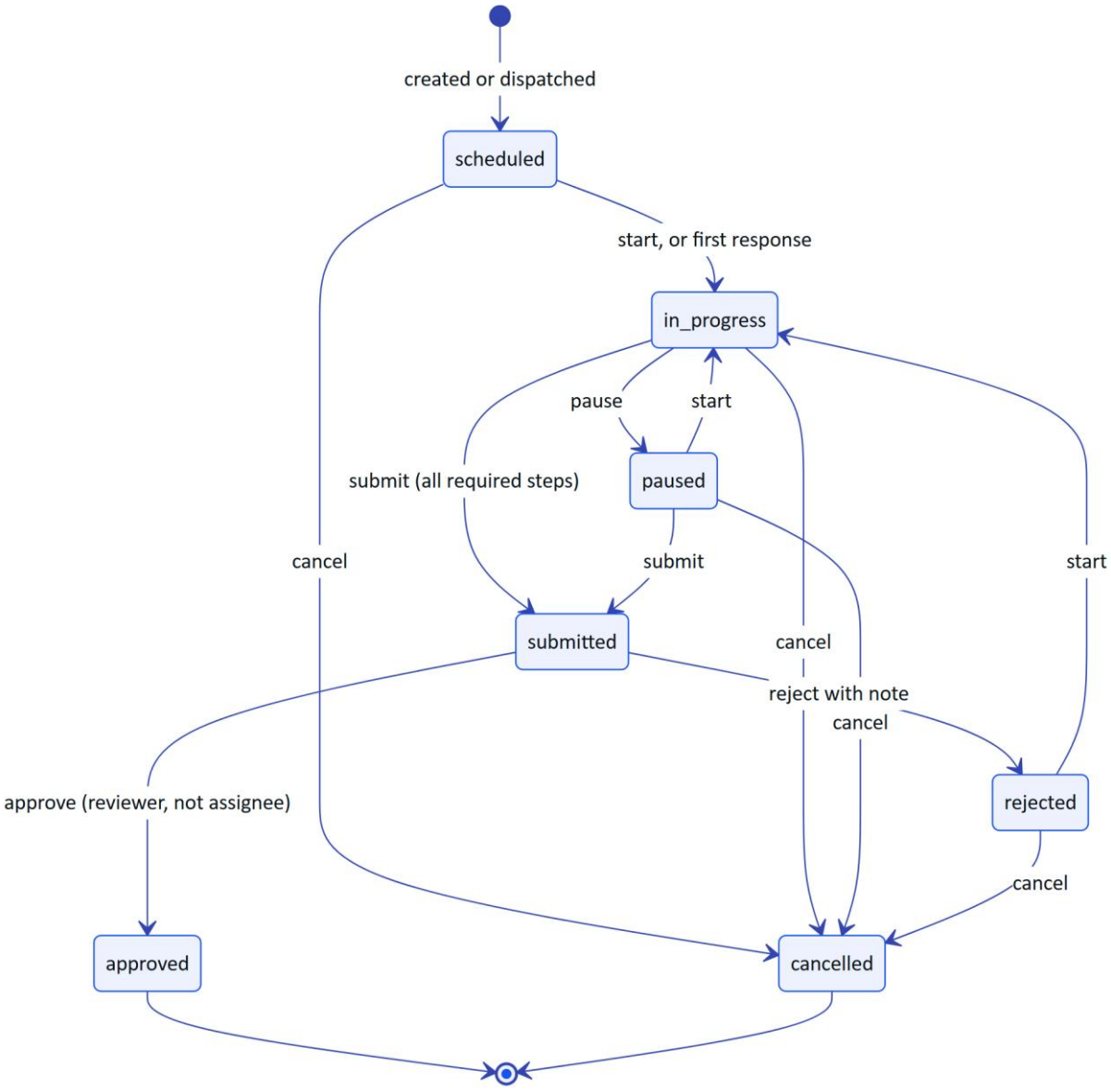

Figure 108. *Inspection lifecycle (INSPECTION_TRANSITIONS in packages/domain/src/workflows.ts)*

Submitting an inspection with abnormal findings automatically opens an `inspection_followup` maintenance task (priority `high` for more than one finding, otherwise `medium`) unless the client sends `createFollowUp: false`.

Table 148. Inspection endpoints

Method	Path	Auth	Permission	Purpose
GET	`/api/inspection-templates`	any	`inspection.read`	List templates
POST	`/api/inspection-templates`	user	`inspection.manage`	Create a template
PUT	`/api/inspection-templates/:id`	user	`inspection.manage`	Replace a template (versioned)
GET	`/api/inspection-templates/:id`	any	`inspection.read`	Template with usage counts
POST	`/api/inspection-templates/:id/status`	user	`inspection.manage`	Publish, unpublish, archive, restore
GET	`/api/inspection-routes`	any	`inspection.read`	List routes
POST	`/api/inspection-routes`	user	`inspection.manage`	Create a route
PUT	`/api/inspection-routes/:id`	user	`inspection.manage`	Replace a route
POST	`/api/inspection-routes/:id/dispatch`	user	`inspection.manage`	Create one inspection per stop
GET	`/api/inspections`	any	`inspection.read`	List inspections
POST	`/api/inspections`	user	`inspection.manage`	Create an inspection
GET	`/api/inspections/:id`	any	`inspection.read`	Inspection with responses and progress
PATCH	`/api/inspections/:id`	user	`inspection.manage`	Reassign, due date, priority
POST	`/api/inspections/:id/responses`	any	`inspection.execute` (assignee)	Sync step responses
POST	`/api/inspections/:id/:action`	any	per action	start, pause, submit, approve, reject, cancel

5.15.1 Templates

GET /api/inspection-templates

- **Auth:** any. **Permission:** `inspection.read`. **Response:** `200 { "templates": [...] }` ordered by name.

POST /api/inspection-templates

- **Auth:** user. **Permission:** `inspection.manage`.
- **Request:**

```
{
  "name": "Lift pump daily check",
  "assetType": "pump",
  "estimatedMinutes": 10,
  "safetyNotices": ["Do not open the guard while the pump is running"],
  "status": "draft",
  "steps": [
    { "id": "noise", "kind": "check", "title": "No abnormal noise", "required": true },
    { "id": "vib", "kind": "reading", "title": "Vibration at drive end", "required": true, "measurand":
"vibration", "unit": "mm/s", "expectedMin": 0, "expectedMax": 4.5 },
    { "id": "seal", "kind": "choice", "title": "Seal condition", "required": true, "options": ["Dry",
"Weeping", "Leaking"], "abnormalOptions": ["Leaking"] },
    { "id": "photo", "kind": "photo", "title": "Photo of the gland", "required": false }
  ]
}
```

- **Response:** `201 { "template": { ... } }` with `version: 1`.
- **Errors:** `400 validation_failed` (1 to 60 steps, step ids unique and up to 40 characters, up to 10 safety notices).

- **Side effects:** audit `inspection_template.created`.

PUT /api/inspection-templates/:id

Replaces the template with the same body as create. The content `version` increments only when steps or safety notices change.

- **Auth:** user. **Permission:** `inspection.manage`. **Response:** `200 { "template": { ... } }`.
- **Errors:** `404 not_found`, `409 archived` (archived templates must be restored first unless the body sets `status` to `draft` or `published`).
- **Side effects:** audit `inspection_template.updated` with field changes, `stepsChanged` and step counts.

GET /api/inspection-templates/:id

- **Auth:** any. **Permission:** `inspection.read`.
- **Response:** `200 { "template", "usage": { "total": 42, "open": 3 } }`.

POST /api/inspection-templates/:id/status

- **Auth:** user. **Permission:** `inspection.manage`. **Request:** `{ "status": "draft" | "published" | "archived" }`.
- **Response:** `200 { "template": { ... } }`. Only published templates can be used for new inspections.
- **Errors:** `409 no_change`, `404 not_found`.
- **Side effects:** audit `inspection_template.published`, `.archived`, `.restored` or `.unpublished`.

5.15.2 Routes

GET /api/inspection-routes

- **Auth:** any. **Permission:** `inspection.read`. **Query:** `siteId`. **Response:** `200 { "routes": [...] }`.

POST /api/inspection-routes

- **Auth:** user. **Permission:** `inspection.manage` and site scope.
- **Request:** `{ "siteId": "...", "name": "Morning round STP-1", "description": "...", "assetIds": ["...", "..."] }` (1 to 100 stops, in walking order).
- **Response:** `201 { "route": { ... } }`. **Errors:** `400 invalid_assets`. **Side effects:** audit `inspection_route.created`.

PUT /api/inspection-routes/:id

- **Auth:** user. **Permission:** `inspection.manage`. **Request:** as create without `siteId`.
- **Response:** `200 { "route": { ... } }`. **Side effects:** audit `inspection_route.updated`.

POST /api/inspection-routes/:id/dispatch

Creates one inspection per stop for one person. Each stop uses the published template whose `assetType` matches the asset's type, falling back to `fallbackTemplateId`.

- **Auth:** user. **Permission:** `inspection.manage`.
- **Request:** `{ "assignedTo": "<uuid>", "dueAt": "2026-10-22T11:00:00+05:30", "fallbackTemplateId": "<uuid>" }`.
- **Response:** `201 { "created": 6, "inspectionIds": ["...", "..."] }`.
- **Side effects:** per inspection: audit `inspection.created`, notification `inspection_assigned`, SSE `inspection.updated`.

5.15.3 Inspections

GET /api/inspections

- **Auth:** any. **Permission:** `inspection.read`. Glasses need the `inspections` screen.
- **Query:** `status` (a status, `open` for editable ones, or `review` for submitted), `mine`, `siteId`, `assetId`, `overdue`.
- **Response:** `200 { "inspections": [{ "id", "title", "status", "priority", "dueAt", "siteId", "assetId", "assignedTo", "startedAt", "submittedAt", "submittedVia", "routeId", "synthetic", "revision", "stepCount", "answered", "assetTag", "assetName", "assigneeName", "siteName" }] }`, up to 300, work in progress first.
- **Notes:** callers without `inspection.manage` or `inspection.review` see only inspections assigned to them.

POST /api/inspections

- **Auth:** user. **Permission:** `inspection.manage`.
- **Request:** `templateId`, plus `assetId` or `siteId`, optional `assignedTo`, `dueAt`, `priority` (`normal`, `high`), `title` (defaults to template name and asset tag).
- **Response:** `201 { "inspection": { ... } }` in `scheduled`.
- **Errors:** `400 invalid_template`, `400 invalid_asset`, `400 site_required`, `400 invalid_assignee`.
- **Side effects:** audit `inspection.created`, notification `inspection_assigned`, SSE `inspection.updated`.

GET /api/inspections/:id

- **Auth:** any. **Permission:** `inspection.read`. Glasses need the `inspections` screen.
- **Response:**

```
{
  "inspection": {
    "id": "91b2...", "title": "Lift pump daily check · P-101", "status": "in_progress", "revision": 4,
"priority": "normal",
    "steps": [{ "id": "noise", "kind": "check", "title": "No abnormal noise", "required": true }],
    "safetyNotices": ["Do not open the guard while the pump is running"],
    "asset": { "id": "a91f...", "tag": "P-101", "name": "Raw sewage lift pump 1", "type": "pump",
"status": "operational", "positionX": 42, "positionY": 18 },
    "assignee": { "id": "7c1e...", "name": "Karthik S" },
    "progress": { "total": 4, "required": 3, "answered": 2, "requiredAnswered": 2, "abnormal": 0,
"complete": false, "percent": 50 },
    "responses": { "noise": { "value": { "kind": "check", "checked": true }, "outcome": "ok", "note":
null, "recordedAt": "2026-10-22T08:02:11.000Z", "deviceId": "e0f1..." } },
    "attachments": []
  },
  "followUps": [],
  "permissions": { "execute": true, "manage": false, "review": false }
}
```

- **Errors:** `404 not_found` (also when an executor asks for someone else's inspection).

PATCH /api/inspections/:id

- **Auth:** user. **Permission:** `inspection.manage`.
- **Request:** any of `assignedTo` (nullable), `dueAt` (nullable), `priority`.
- **Response:** `200 { "inspection": { ... } }` with `revision` incremented.
- **Errors:** `409 not_editable` (only scheduled, in progress, paused or returned inspections).
- **Side effects:** audit `inspection.updated`, notification to a new assignee, SSE `inspection.updated`.

POST /api/inspections/:id/responses

Batch sync of step responses, designed for glasses that work offline. Each operation carries a client-generated id and the time it was recorded on the device. The latest response per step (by `recordedAt`) wins.

- **Auth:** any. **Permission:** `inspection.execute`, and the caller must be the assignee. Glasses need the `inspections` screen and the `record_inspection` action.
- **Request:**

```
{
  "ops": [
    { "opId": "op-1761120131-01", "stepId": "noise", "value": { "kind": "check", "checked": true },
"recordedAt": "2026-10-22T13:32:11+05:30" },
    { "opId": "op-1761120131-02", "stepId": "vib", "value": { "kind": "reading", "value": 5.2 },
"note": "Rising since last week", "recordedAt": "2026-10-22T13:33:40+05:30" }
  ]
}
```

- **Response:** `200 { "applied": 2, "duplicates": 0, "conflicts": [], "inspection": { ...view } }`. Conflict reasons: `unknown_step`, `kind_mismatch`, `timestamp_in_future`, or `inspection_<status>` when the inspection is no longer editable (all ops are then returned as conflicts).
- **Errors:** `403 forbidden` (not the assignee).
- **Side effects:** the first applied response moves a scheduled, paused or returned inspection to `in_progress`; SSE `inspection.updated` with `progressChanged`.

POST /api/inspections/:id/:action

`action` is `start`, `pause`, `submit`, `approve`, `reject` or `cancel`.

- **Auth:** any. **Permission:** `start`, `pause` and `submit` need `inspection.execute` and the assignee; `approve` and `reject` need `inspection.review` and someone other than the assignee; `cancel` needs `inspection.manage`. Glasses need the `inspections` screen and `submit_inspection` (for submit) or `record_inspection` (other actions).
- **Request:** `{ "note": "...", "expectedRevision": 4, "createFollowUp": true }`, all optional. `note` is required for `reject`.
- **Response:** `200 { "inspection": { ...view }, "followUpId": "<uuid>" | null }`.
- **Errors:** `403 forbidden`, `409 revision_conflict`, `409 invalid_transition`, `400 note_required`, `400 incomplete` (with the progress object in `details`).
- **Side effects:** audit `inspection.start`, `.pause`, `.submitted`, `.approved`, `.returned` or `.cancel`; submit records `submittedVia` (`glasses` or `console`), may create the follow-up task (audit `maintenance.created`, SSE `maintenance.updated`) and notifies whoever assigned the work; approve and reject notify the assignee; SSE `inspection.updated`.

5.16 Maintenance module

File: `apps/api/src/modules/maintenance.ts`. Maintenance tasks are corrective, preventive, inspection follow-up or alarm follow-up work with an optional checklist. Anyone with `maintenance.request` can raise a task; planners (`maintenance.manage`) assign and edit; the assigned engineer (`maintenance.execute`) starts, holds and completes.

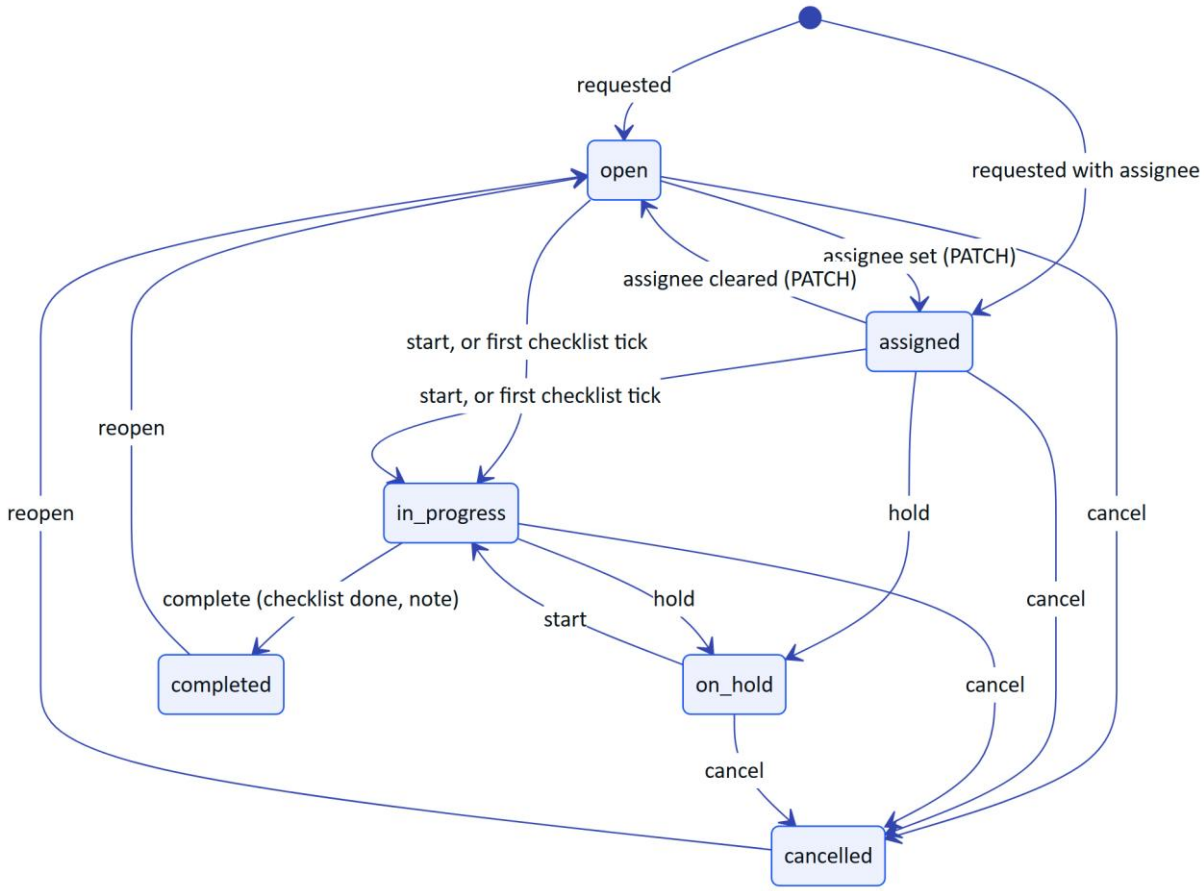

Figure 109. *Maintenance task lifecycle (MAINTENANCE_TRANSITIONS in packages/domain/src/workflows.ts)*

Reopening returns a task to `assigned` instead of `open` when it still has an assignee.

Table 149. Maintenance endpoints

Method	Path	Auth	Permission	Purpose
GET	`/api/maintenance`	any	`maintenance.read`	List tasks
POST	`/api/maintenance`	any	`maintenance.request`	Raise a task
GET	`/api/maintenance/:id`	any	`maintenance.read`	Task with activity log
PATCH	`/api/maintenance/:id`	user	`maintenance.manage`	Edit, assign, edit checklist
POST	`/api/maintenance/:id/checklist/:itemId`	any	manager, or assignee with execute rights	Tick or untick an item
POST	`/api/maintenance/:id/notes`	any	`maintenance.read` plus involvement	Add a note
POST	`/api/maintenance/:id/:action`	any	`maintenance.manage`, or assignee for start, hold, complete	Transition

GET /api/maintenance

- **Auth:** any. **Permission:** `maintenance.read`. Glasses need the `tasks` screen.
- **Query:** `status` (a status or `open_all`), `priority` (`low`, `medium`, `high`, `urgent`), `siteId`, `assetId`, `mine`.

- **Response:** `200 { "tasks": [{ ...task, "assetTag", "assetName", "assigneeName", "siteName" }] }`, up to 300, in-progress and urgent first.

POST /api/maintenance

- **Auth:** any. **Permission:** `maintenance.request`; setting `assignedTo` needs `maintenance.manage`.
- **Request:**

```
{
  "title": "Replace gland packing on P-101",
  "description": "Seal leaking during morning round",
  "assetId": "a91f...",
  "kind": "corrective",
  "priority": "high",
  "assignedTo": "b2c3...",
  "dueAt": "2026-10-23T12:00:00+05:30",
  "sourceAlarmId": null,
  "checklist": ["Isolate and lock out", "Replace packing", "Run and check for leaks"]
}
```

- **Response:** `201 { "task": { ... } }`. Status is `assigned` with an assignee, otherwise `open`. A `sourceAlarmId` forces `kind: "alarm_followup"`. Checklist items (up to 40) get 8-character ids.
- **Errors:** `400 site_required`, `403 forbidden` ("Only planners can assign maintenance"), `400 invalid_assignee`, `400 invalid_alarm`, `404 not_found`.
- **Side effects:** activity `created`, audit `maintenance.created`, notification `maintenance_assigned` (severity `critical` for urgent), SSE `maintenance.updated`.

GET /api/maintenance/:id

- **Auth:** any. **Permission:** `maintenance.read`. Glasses need the `tasks` screen.
- **Response:** `200 { "task": { ..., "asset", "assignee", "attachments" }, "activity": [{ "action", "note", "actorName", "at" }], "permissions": { "manage", "execute" } }`.

PATCH /api/maintenance/:id

- **Auth:** user. **Permission:** `maintenance.manage`.
- **Request:** any of `title`, `description`, `priority`, `assignedTo`, `dueAt`, `kind`, `assetId`, `checklist`. `checklist` is the full list: items with an existing `id` keep their completion state, items without an id are added, missing ids are removed.
- **Response:** `200 { "task": { ... } }`.
- **Errors:** `409 not_editable` (completed or cancelled), `400 validation_failed` (inactive assignee, asset on another site, unknown checklist id).
- **Side effects:** activity `assigned`, `updated` and `checklist_edited`; audit `maintenance.updated` with changes; notification to a new assignee; SSE `maintenance.updated`.

POST /api/maintenance/:id/checklist/:itemId

- **Auth:** any. **Permission:** `maintenance.manage`, or the assignee holding `maintenance.execute` or `inspection.execute`. Glasses need the `tasks` screen and `toggle_checklist` action.
- **Request:** `{ "done": true }`.
- **Response:** `200 { "task": { ... } }`. The first tick moves an open or assigned task to `in_progress`.
- **Errors:** `403 forbidden`, `409 not_editable`, `404 not_found` (item).
- **Side effects:** activity `checklist_done` or `checklist_undone`; SSE `maintenance.updated`.

POST /api/maintenance/:id/notes

- **Auth:** any. **Permission:** `maintenance.read`, and the caller must manage maintenance, be the assignee or be the requester. **Request:** `{ "note": "Waiting for packing from stores" }`. **Response:** `200 { "ok": true }`.

POST /api/maintenance/:id/:action

`action` is `start`, `hold`, `complete`, `cancel` or `reopen`.

- **Auth:** any. **Permission:** `maintenance.manage` for every action; the assignee with `maintenance.execute` may also `start`, `hold` and `complete`.
- **Request:** `{ "note": "..." }`; required for `complete`.
- **Response:** `200 { "task": { ... } }`.
- **Errors:** `403 forbidden`, `409 invalid_transition`, `400 checklist_incomplete`, `400 note_required`, `409 stale_state`.
- **Side effects:** activity row, audit `maintenance.<action>`, notification `maintenance_completed` to the requester, SSE `maintenance.updated`.

5.17 Devices and pairing module

File: `apps/api/src/modules/devices.ts`. The pairing flow and device token rotation are described in the authentication chapter; this chapter lists the endpoints. Device records have three stored statuses (`active`, `suspended`, `revoked`); the console shows a derived state (`paired`, `assigned`, `connected`, `disconnected`, `suspended`, `revoked`) computed from status, assignment, `lastSeenAt` (90-second window) and open SSE streams.

Table 150. Device and pairing endpoints

Method	Path	Auth	Permission	Limit	Purpose
POST	`/api/pairing/challenges`	public	none	10/min	Glasses request a pairing code
POST	`/api/pairing/challenges/:id/poll`	poll secret	none	40/min	Glasses poll for approval and claim tokens
POST	`/api/pairing/challenges/:id/cancel`	poll secret	none	20/min	Glasses cancel a challenge
GET	`/api/pairing/lookup`	user	`device.pair`	10/min	Look up a code before approving
POST	`/api/pairing/approve`	user	`device.pair` (`device.manage` for others)	10/min	Approve a code
POST	`/api/pairing/deny`	user	`device.pair`	10/min	Deny a code
POST	`/api/device/refresh`	device refresh token	none	30/min	Rotate device tokens
POST	`/api/device/logout`	device	none	global	Unpair from the glasses
GET	`/api/device/context`	device	none	global	Glasses home screen data
GET	`/api/devices`	user	`device.read`	global	Fleet list

Method	Path	Auth	Permission	Limit	Purpose
GET	`/api/devices/:id`	user	`device.read`	global	Device detail and diagnostics
PATCH	`/api/devices/:id`	user	`device.manage`	global	Rename or move site
POST	`/api/devices/:id/assign`	user	`device.manage`	global	Reassign operator
POST	`/api/devices/:id/suspend`	user	`device.manage`	global	Suspend
POST	`/api/devices/:id/reactivate`	user	`device.manage`	global	Reactivate a suspended device
POST	`/api/devices/:id/revoke`	user	`device.manage`	global	Revoke permanently

POST /api/pairing/challenges

- **Auth:** public. **Request:** `hardwareId` (16 to 80 characters of `[A-Za-z0-9_-]`, stable per install) and `device` with `kind` (`hardware` or `simulator`), optional `model`, `name`, `appVersion`, `firmwareVersion`, `display` (`width`, `height`).
- **Response:**

```
{
  "challengeId": "3a9e...", "userCode": "K7RM-4QXT", "pollSecret": "q2Vt...", "expiresAt": "2026-10-22T08:19:03.000Z",
  "pollIntervalS": 3, "verificationUri": "https://hydrisview.vercel.app/pair",
  "verificationUriComplete": "https://hydrisview.vercel.app/pair?code=K7RM-4QXT"
}
```

- **Errors:** `503 pairing_unavailable`. **Notes:** open challenges of the same `hardwareId` are cancelled.

POST /api/pairing/challenges/:id/poll

- **Auth:** `{ "pollSecret": "..." }` in the body.
- **Response:** `{ "status": "pending", "expiresAt" }`, `{ "status": "slow_down", "pollIntervalS": 3 }`, `{ "status": "expired" | "denied" | "cancelled" }`, or on approval `{ "status": "completed", "device": { "id", "name" }, "accessToken", "refreshToken" }`. A second claim returns `{ "status": "completed", "error": "already_claimed" }` without tokens.
- **Errors:** `404 not_found` (unknown id or wrong secret).
- **Side effects:** creates or reuses the device, assigns the operator, audit `device.paired`, SSE `device.paired` and `pairing.completed`.

POST /api/pairing/challenges/:id/cancel

- **Auth:** poll secret. **Response:** `200 { "ok": true }`. **Errors:** `404 not_found`.

GET /api/pairing/lookup

- **Auth:** user. **Permission:** `device.pair`. **Query:** `code` (with or without the dash).
- **Response:** `200 { "challenge": { "code", "device", "requestedAt", "expiresAt" }, "organization", "sites", "assignees", "canAssignOthers", "existingDevice" }`. Without `device.manage`, `assignees` contains only the caller.
- **Errors:** `400 invalid_code`, `404 not_found` (expired or used).

POST /api/pairing/approve

- **Auth:** user. **Permission:** `device.pair`; assigning someone else needs `device.manage`.
- **Request:** `{ "code": "K7RM-4QXT", "siteId": "<uuid>", "assigneeUserId": "<uuid>", "deviceName": "Karthik's glasses" }`.

- **Response:** `200 { "ok": true, "challengeId": "..." }`.
- **Errors:** `403 forbidden`, `400 invalid_assignee`, `400 assignee_no_site_access`, `404 not_found`, `409 device_owned_elsewhere`, `409 already_used`. **Side effects:** audit `device.pairing_approved`.

POST /api/pairing/deny

- **Auth:** user. **Permission:** `device.pair`. **Request:** `{ "code": "..." }`. **Response:** `200 { "ok": true }`.
- **Side effects:** audit `device.pairing_denied`; the glasses' next poll returns `denied`.

POST /api/device/refresh

- **Auth:** `{ "refreshToken": "hgd_..." }` (up to 120 characters).
- **Response:** `200 { "accessToken", "refreshToken" }`.
- **Errors:** `401 device_revoked`, `401 device_session_invalid`, `401 device_session_expired`, `403 device_suspended`, `403 device_unassigned`.

POST /api/device/logout

- **Auth:** device (a user token returns `403 forbidden`). **Response:** `204`.
- **Side effects:** device sessions revoked and session version bumped.

GET /api/device/context

Everything the glasses need for the home screen in one call, filtered by the device's effective UI profile: sections for disabled screens are omitted on the server, not just hidden.

- **Auth:** device.
- **Response:** `200 { "device", "operator", "organization", "site", "role", "profile", "inspections", "alarms", "tasks", "map", "assets", "measurements", "serverTime" }`. `inspections`, `alarms` and `tasks` hold up to 20 open items of the operator and site; `map` gives the site layout in metres, process area polygons and asset pins; each measurement carries `freshness` and `threshold` (`alarm`, `out_of_range`, `normal`).

GET /api/devices

- **Auth:** user. **Permission:** `device.read`.
- **Response:** `200 { "devices": [{ ...device, "siteName", "assignee": { "id", "name", "since" }, "streamOpen", "state" }] }`. `hardwareId` is never returned.

GET /api/devices/:id

- **Auth:** user. **Permission:** `device.read`.
- **Response:** `200 { "device", "assignments", "activeSessions", "events", "diagnostics", "canManage", "effectiveProfile" }`. `diagnostics.battery` is `null`: the current integration does not expose hardware sensors.

PATCH /api/devices/:id

- **Auth:** user. **Permission:** `device.manage`. **Request:** `name` (2 to 60), `siteId`.
- **Response:** `200 { "device" }`. **Side effects:** audit `device.updated`; a site change sends `device.context_changed` (`site_changed`).

POST /api/devices/:id/assign

- **Auth:** user. **Permission:** `device.manage`. **Request:** `{ "userId": "<uuid>", "reason": "Shift handover" }`.
- **Response:** `200 { "ok": true }` or `{ "ok": true, "unchanged": true }`.

- **Errors:** `409 device_not_active`, `400 invalid_assignee`, `404 not_found`.
- **Side effects:** session version bumped (the glasses refresh into the new operator), audit `device.reassigned`, SSE `device.context_changed` and `device.updated`.

POST /api/devices/:id/suspend, /reactivate, /revoke

- **Auth:** user. **Permission:** `device.manage`. **Request:** `{ "reason": "Lost on site" }`; required for revoke.
- **Response:** `200 { "ok": true, "status": "suspended" | "active" | "revoked" }`.
- **Errors:** `409 device_revoked`, `409 invalid_transition` (reactivate only from suspended), `400 reason_required`.
- **Side effects:** session version bumped; revoke also ends device sessions and the assignment; audit `device.suspended`, `device.reactivated` or `device.revoked`; SSE `device.revoked` or `device.context_changed`, and `device.updated`.

5.18 Realtime module

File: `apps/api/src/modules/realtime.ts`, with the serverless variant in `apps/web/src/app/api/events/stream/route.ts`.

GET /api/events/stream

- **Auth:** any, as `Authorization: Bearer`. **Request:** optional `Last-Event-ID` header or `lastEventId` query.
- **Response:** `200 text/event-stream`, described in the realtime chapter.
- **Errors:** `401 unauthorized`, `503 realtime_misconfigured` (serverless route without `REALTIME_MODE=db`).

5.19 Insights module

File: `apps/api/src/modules/insights.ts`. Dashboards, reports, grounded procedure retrieval and search.

Table 151. Insights endpoints

Method	Path	Auth	Permission	Purpose
GET	`/api/overview`	any	scope only	Overview dashboard
GET	`/api/reports/:kind`	any	`report.read`, `report.export` for CSV	Reports
GET	`/api/assist`	any	`document.read`	Approved procedure passages and live asset data
GET	`/api/search`	any	scope only	Command palette search

GET /api/overview

- **Query:** `siteId` (optional).
- **Response:** `200 { "generatedAt", "alarms": { "active", "critical", "open", "last24h" }, "inspections": { "due", "overdue", "review", "done7d" }, "maintenance": { "open", "urgent" }, "devices": { "registered", "connected" }, "activeOperators", "sites", "assetHealth", "metrics", "recentAlarms", "activity", "alarmTrend" }`. `alarmTrend` covers 14 days; `metrics` are up to 12 continuous pH, DO, turbidity, flow, level and vibration sensors.

GET /api/reports/:kind

- **Path:** `kind` is `inspections`, `alarms`, `maintenance`, `devices` or `operators`.
- **Query:** `from`, `to` (default last 30 days), `siteId`, `assetId`, `userId`, `severity`, `format` (`json` or `csv`).

- **Response:** JSON `{ "kind", "from", "to", "rows", "summary" }` (up to 5000 rows) or a CSV attachment. Summaries include median minutes to acknowledge (alarms), completion and on-time rates (inspections) and median hours to complete (maintenance).
- **Errors:** `400 invalid_range`. **Side effects:** CSV writes audit `report.exported`.

GET /api/assist

Grounded retrieval only: it never generates procedures or issues commands. Glasses need `ask_ai` or `view_sops`.

- **Query:** `q` (2 to 200), `assetId` (optional).
- **Response:** `200 { "query", "sources": [{ "id", "title", "kind", "version", "rank", "passage" }], "asset", "telemetry", "glossary", "notice" }`, up to five approved documents ranked by Postgres full-text search, with a safety notice.

GET /api/search

- **Query:** `q` (1 to 80). **Response:** `200 { "assets", "sensors", "documents", "people" }`, each with `id`, `label`, `sub`.

5.20 Voice assistant module

File: `apps/api/src/modules/voice-assistant.ts`. "Hey Hydris" answers spoken questions from the caller's own data. With an Anthropic credential configured it uses Claude (`claude-opus-5-5`) with five read-only tools (readings, open alarms, my work, asset status, procedure search), all scoped by RLS and site scope. Otherwise, or if the model call fails, a deterministic intent engine answers. Neither engine can change data.

POST /api/assistant/ask

- **Auth:** any. **Limit:** 60 per minute.
- **Request:** `{ "text": "what is the current pH", "history": [{ "role": "user", "text": "..." }] }` (text 1 to 500, up to 8 history turns).
- **Response:**

```
{
  "answer": "Current pH at Aeration tank 1 is 7.12, measured 40 seconds ago, within the typical
range.",
  "facts": [{ "label": "pH · Aeration tank 1", "value": "7.12", "unit": "pH", "at": "2026-10-
22T08:13:20.000Z", "source": "sensor", "ref": "AT-101-PH" }],
  "intent": "reading",
  "engine": "hydris-rules"
}
```

- **Notes:** `engine` is `claude-opus-5-5` or `hydris-rules`; `intent` is `llm` for model answers, otherwise `alarms`, `my_work`, `reading`, `asset`, `procedure`, `help` or `unknown`.

```
curl -s -X POST https://hydrisview.vercel.app/api/assistant/ask \
  -H "authorization: Bearer $TOKEN" -H "content-type: application/json" \
  -d '{"text":"Do I have any critical alarms?"}'
```

5.21 Audit documents module

File: `apps/api/src/modules/audit-docs.ts`. Paper records (lab sheets, logs, permits) are captured as page images with on-device OCR, optionally refined by a vision model, cross-checked value by value, reviewed by a person, verified and then approved by a different person, and exported as a searchable PDF.

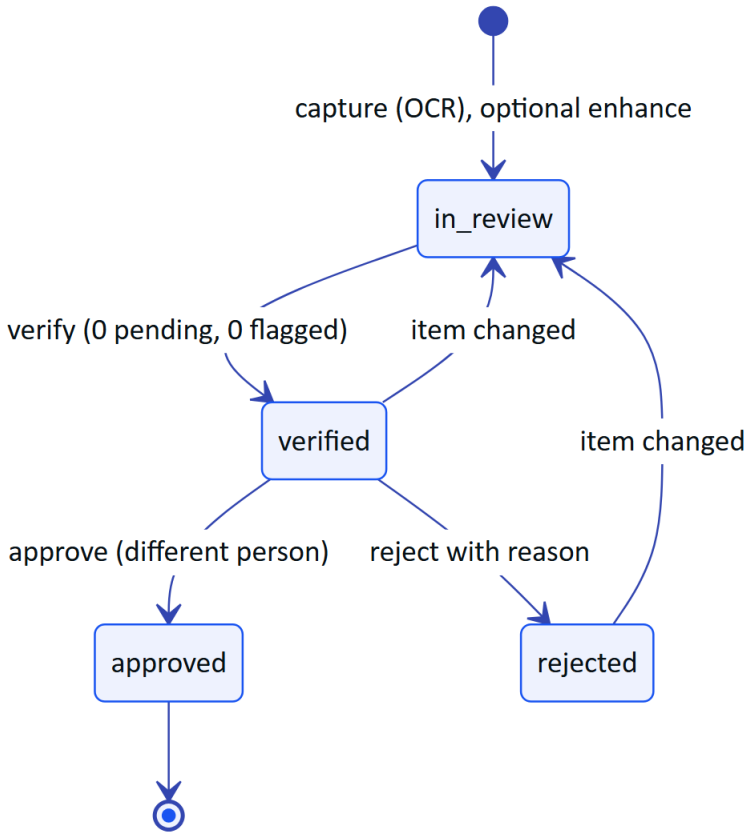

Figure 110. *Audit document lifecycle: capture, review, verification and independent approval*

Table 152. Audit document endpoints

Method	Path	Auth	Permission	Purpose
GET	`/api/audit-docs/capabilities`	any	none	Engine availability and limits
GET	`/api/audit-docs`	any	`auditdoc.read`	List (bare array)
GET	`/api/audit-docs/:id`	any	`auditdoc.read`	Detail with extraction and events
GET	`/api/audit-docs/:id/pages/:pageNo`	any	`auditdoc.read`	Page image
GET	`/api/audit-docs/:id/pdf`	any	`auditdoc.read`	Searchable PDF
POST	`/api/audit-docs`	any	`auditdoc.capture`	Capture
POST	`/api/audit-docs/:id/enhance`	any	`auditdoc.capture`	Run the vision engine
PATCH	`/api/audit-docs/:id/items/:itemId`	any	`auditdoc.verify` or `auditdoc.capture`	Review one field or table
POST	`/api/audit-docs/:id/confirm-agreed`	any	`auditdoc.verify` or `auditdoc.capture`	Bulk-confirm agreed values
POST	`/api/audit-docs/:id/verify`	user	`auditdoc.verify`	Sign off the review
POST	`/api/audit-docs/:id/approve`	user	`auditdoc.approve`	Approve or reject
PATCH	`/api/audit-docs/:id`	any	`auditdoc.verify` or `auditdoc.capture`	Rename

GET /api/audit-docs/capabilities

- **Response:** `200 { "vision": true, "visionEngine": "vision:claude-opus-5-5", "maxPages": 6, "maxPageBytes": 921600 }`.

GET /api/audit-docs

- **Query:** `status` (`processing`, `in_review`, `verified`, `approved`, `rejected`), `mine`.
- **Response:** `200 [{ "id", "title", "docType", "status", "siteName", "pageCount", "capturedVia", "capturedBy", "engines", "aiStatus", "agreement", "fields", "tables", "figures", "review", "attention", ... }]`, up to 200.

GET /api/audit-docs/:id

- **Response:** the summary plus `extraction`, `revision`, `aiError`, `verifiedBy`, `approvedBy`, `rejectedReason`, `pages`, `events` (with `viaGlasses`) and `permissions` (`review`, `verify`, `approve`).

GET /api/audit-docs/:id/pages/:pageNo and GET /api/audit-docs/:id/pdf

- **Response:** the page image (`private, max-age=3600`) or `application/pdf` attachment with OCR text, extracted values and sign-off record.

POST /api/audit-docs

- **Body limit:** 8 MB. **Request:** `title`, `siteId`, `assetId` (optional), `source` (`mobile`, optional) and `pages` (1 to 6): each with `mime` (`image/jpeg` or `image/png`), base64 `data` (at most 900 KB decoded), `width`, `height`, `readings` (1 to 3 OCR passes with engine, text, confidence and word boxes) and optional `figures`.

```
curl -s -X POST https://hydrisview.vercel.app/api/audit-docs \
  -H "authorization: Bearer $TOKEN" -H "content-type: application/json" \
  --data-binary @scan.json
```

- **Response:** `201` summary with `status: "in_review"` and `aiStatus` `pending` or `unavailable`.
- **Errors:** `400 page_too_large`, `400 invalid_image`, `404 not_found` (site).
- **Side effects:** `capturedVia` is `glasses`, `mobile` or `console`; audit `auditdoc.captured`; SSE `auditdoc.updated`.

POST /api/audit-docs/:id/enhance

- **Response:** `200` summary, or `{ "aiStatus": "unavailable" | "failed", "message" }`.
- **Errors:** `409 review_started`, `409 stale`.

PATCH /api/audit-docs/:id/items/:itemId

- **Request:** `action` (`confirm`, `correct`, `flag`, `reset`), `value` and `unit` (fields), `header` and `rows` (tables), `note`, `expectedRevision`.
- **Response:** `200 { "revision", "status", "review", "item" }`. The first extracted value is kept in `extracted`.
- **Errors:** `409 locked`, `409 stale`, `400 value_required`, `400 rows_required`, `404 not_found`.
- **Notes:** editing a verified or rejected document reopens the review.

POST /api/audit-docs/:id/confirm-agreed

Confirms pending fields read by more than one engine that agree with confidence of at least 0.8.

- **Response:** `200 { "revision", "review" }`. **Errors:** `409 not_in_review`, `409 stale`.

POST /api/audit-docs/:id/verify

- **Auth:** user only. **Request:** `{ "note": "..." }` optional. **Response:** `200 { "status": "verified", "revision" }`.

- **Errors:** `409 not_in_review`, `400 flagged_items`, `400 review_incomplete`. **Side effects:** audit `auditdoc.verified`.

POST /api/audit-docs/:id/approve

- **Auth:** user only. **Request:** `{ "decision": "approve" | "reject", "note": "..." }` (note required to reject).
- **Response:** `200 { "status", "revision" }`.
- **Errors:** `409 not_verified`, `403 forbidden` (same person as verifier, outside demo tenants), `400 reason_required`. **Side effects:** audit `auditdoc.approved` or `auditdoc.rejected`.

PATCH /api/audit-docs/:id

- **Request:** `{ "title": "..." }` (2 to 160). **Response:** `200 { "title", "revision" }`. **Errors:** `409 locked`.

5.22 Glasses UI profiles module

File: `apps/api/src/modules/ui-profiles.ts`. UI profiles control which screens, widgets, measurements and actions the glasses show. Profiles are versioned (one draft, one published, superseded history) and assigned by organization, role, user or device; the most specific assignment wins (device, user, role, organization, built-in default). Reading requires an organization administrator or factory manager; editing requires an organization administrator.

Table 153. UI profile endpoints

Method	Path	Access	Purpose
GET	`/api/ui-profiles`	read	Profiles with published version and draft flag
POST	`/api/ui-profiles`	edit	Create (optionally copy another)
GET	`/api/ui-profiles/:id`	read	Versions with change lists, assignments
PATCH	`/api/ui-profiles/:id`	edit	Rename, describe
POST	`/api/ui-profiles/:id/archive`	edit	Archive an unassigned profile
PUT	`/api/ui-profiles/:id/draft`	edit	Save the draft config
DELETE	`/api/ui-profiles/:id/draft`	edit	Discard the draft
POST	`/api/ui-profiles/:id/publish`	edit	Publish the draft
POST	`/api/ui-profiles/:id/rollback`	edit	Republish an earlier version as a new version
GET	`/api/ui-profile-assignments`	read	List assignments
POST	`/api/ui-profile-assignments`	edit	Create or replace an assignment
DELETE	`/api/ui-profile-assignments/:id`	edit	Remove an assignment
GET	`/api/ui-profiles-effective`	read	Resolve the profile for a device or user

PUT /api/ui-profiles/:id/draft

- **Request:**

```
{
  "config": {
    "schemaVersion": 1,
    "screens": [{ "id": "map", "enabled": false }],
    "widgets": [{ "id": "alarm_summary", "visible": true }],
    "measurements": { "allow": ["ph", "dissolved_oxygen", "vibration"], "orderByAssetType": { "pump":
["vibration", "motor_current"] } },
    "actions": { "acknowledge_alarm": true, "record_inspection": true, "submit_inspection": true,
"toggle_checklist": true, "ask_ai": true, "view_sops": true, "view_map": false,
"mark_notifications_read": true },
    "layout": { "density": "compact", "textScale": 1.1, "handedness": "right" }
  },
  "notes": "Hide the map for STP-1 operators"
}
```

- **Response:** `200 { "draft": { ... } }`. **Errors:** `409 profile_archived`.
- **Notes:** screens are `home`, `alerts`, `inspections`, `tasks`, `map`, `measurements`, `assets`, `sops`, `ai`, `scan`, `notifications`, `status`.

POST /api/ui-profiles/:id/publish and /rollback

- **Request:** publish `{ "notes": "..." }`; rollback `{ "versionId": "<uuid>", "notes": "..." }`.
- **Response:** `200 { "version", "affectedDevices": 4 }`.
- **Errors:** `400 no_draft`, `409 profile_archived`, `400 cannot_rollback_to_draft`, `409 already_published`.
- **Side effects:** audit `ui_profile.published` or `ui_profile.rolled_back`; SSE `device.profile_changed` to affected glasses and `ui_profile.updated`.

POST /api/ui-profile-assignments

- **Request:** `{ "scope": "org" | "role" | "user" | "device", "profileId", "role" | "userId" | "deviceId" }`.
- **Response:** `201` (new) or `200` (replaced) `{ "assignment", "replaced", "affectedDevices" }`.
- **Errors:** `400 invalid_profile`, `400 invalid_user`, `400 invalid_device`.

Other UI profile routes

`POST /api/ui-profiles` takes `{ name, description?, copyFromProfileId? }` and returns `201 { profile }` with a v1 draft. `PATCH` takes `{ name?, description? }`. Archive returns `409 profile_assigned` while assignments exist. `DELETE .../draft` returns `404 not_found` without a draft. `GET /api/ui-profiles-effective?deviceId=&userId=` returns `{ subject, effective: { profileId, name, version, source, config } }` or `400 subject_required`. Every write is audited under `ui_profile.*`.

5.23 Simulation module

File: `apps/api/src/modules/simulation.ts`. Demo tenants only: deterministic scenarios feed synthetic readings through the normal ingestion pipeline, which refuses `simulator` data for customer tenants.

GET /api/simulation

- **Auth:** any. **Response:** `200 { "available", "canControl", "scenarios", "stepSeconds": 5, "running" }`. Scenarios: `normal_operations`, `pump_vibration`, `tank_high_level`, `abnormal_ph`, `scheduled_inspection`, `equipment_offline`, `glasses_network_loss`, `high_priority_alert`, `maintenance_checklist`, `device_reassignment`.

POST /api/simulation/start

- **Permission:** `simulation.control`. **Request:** `{ "scenario": "pump_vibration", "siteId": "<uuid>" }`.
- **Response:** `201 { "run" }`. Runs on the same site are stopped first.
- **Errors:** `403 forbidden` (not a demo organization, simulator disabled), `404 not_found`.

POST /api/simulation/stop

- **Permission:** `simulation.control`. **Response:** `200 { "ok": true }`. **Errors:** `409 not_running`.

5.24 Public website module

File: `apps/api/src/modules/public.ts`.

POST /api/public/inquiries

- **Auth:** public. **Limit:** 5 per 10 minutes.
- **Request:** `kind` (`contact` or `demo`), `name`, `email`, optional `company`, `jobTitle`, `phone`, `siteCount` (`1`, `2-5`, `6-20`, `20+`), `message`, `consent: true` (required), `website` (honeypot, must be empty), `startedAt` (epoch milliseconds when the form opened).
- **Response:** `201 { "ok": true }`. **Errors:** `400 too_fast`, `400 validation_failed`.
- **Notes:** the IP is stored only as a salted hash; consent version `2026-10` is recorded.

5.25 Background jobs and cron

File: `apps/api/src/jobs.ts`. Hydris has no outbound webhooks today (see the planned API chapter). Background work runs inside the API.

Table 154. Background jobs

Job	Long-running server	Serverless (Vercel)
Stale-data alarm rules	Every 30 s	Each cron call
Expire pairing challenges	Every 60 s	Each cron call
Prune `realtime_events` older than 24 h	Hourly (db mode)	Each cron call
Advance running simulations	Timers every 5 s	Each cron call, plus lazily on API activity in demo mode

GET /api/cron

- **Auth:** `Authorization: Bearer <CRON_SECRET>`, compared in constant time. The root `vercel.json` schedules it daily at 03:00 UTC (`0 3 * * *`, a Hobby plan constraint; Pro deployments can run it every minute).
- **Response:** `200 { "ok": true, "ms": 412, "report": { "staleRules": "ok", "pairingExpiry": "ok", "realtimePruned": 120, "simulationSteps": 1 } }`.
- **Errors:** `401 unauthorized`.

5.26 Client libraries

5.26.1 Web console fetch wrapper

`apps/web/src/lib/api.ts`. The access token lives only in memory; the refresh token is the httpOnly cookie, which page scripts cannot read. `api()` adds the bearer header, and on a 401 performs one single-flight `POST /auth/refresh` and retries once; if refresh fails it calls the session-lost handler. Errors become `ApiError(status, code, message, details)` from the envelope. `apiBlob()` and `download()` fetch images, PDFs and CSVs with the header so tokens never appear in URLs. `apps/web/src/lib/realtime.ts` reads the SSE stream through `fetch`, tracks `Last-Event-ID`, refreshes on 401 or `session.ended`, and reconnects with backoff capped at 15 seconds.

5.26.2 Glasses device link

`apps/web/src/glasses/link.ts` and `data.ts`. `DeviceLink` owns the install `hardwareId`, the pairing flow, the device refresh token (in `localStorage` in the browser runtime; a native runtime should supply a hardware-backed `SecureStore`), single-flight refresh, one retry on 401, and the offline queue of inspection operations. Responses for read screens are cached so the glasses keep showing the last known data offline.

5.26.3 Android OkHttp client

`apps/android/app/src/main/java/ai/hydris/field/data/HydrisApi.kt` and `Repository.kt`. One `OkHttpClient` with a persistent, encrypted cookie jar for `hg_refresh` and `hg_demo` (15 s connect, 90 s read timeouts for uploads). A mutex-serialized `refresh()` handles 401s, except on login and demo calls. A second credential channel, `deviceCall()`, uses stored device tokens and `/device/refresh`, so scans taken with paired Ray-Ban Meta glasses are attributed to the device. `events()` reads SSE on a separate client without read timeout and resumes with `Last-Event-ID`.

5.27 Change history and deprecation policy

Table 155. API change history (from the git log)

Date	Change
2026-10-15	Whole stack on Vercel: in-process API route, DB-backed realtime, cron, Blob storage. Demo mode with `POST /api/auth/demo`
2026-10-21	Named demo guests; pairing shared across instances; `POST /api/device/logout`
2026-10-21	Audit documents module (12 routes); voice assistant `POST /api/assistant/ask`
2026-10-22	Android client; audit capture accepts `source: "mobile"`

Deprecation policy for this API:

- Additive changes (new routes, optional fields, new event types, new error codes) ship without notice. Clients must ignore unknown fields and events.
- Removing or renaming a route, field or error code used by a shipped client needs a release in which old and new coexist, and a changelog entry here.
- Versioned paths (`/api/ingest/v1`) are kept for at least 12 months after a successor is published, because gateway firmware updates slowly.
- The future public integration API will carry a version segment and a published sunset header policy.

5.28 Planned API

> **NOTE**
> Everything in this chapter is **Planned**. None of it exists in the code today.

- **Public REST API for integrations.** A versioned `/api/v1` surface for CMMS, ERP and historian integrations with organization-scoped API clients (client credentials), read access to sites, assets, readings, alarms and work, and scoped writes for work orders.
- **OpenAPI generation.** Generate an OpenAPI 3.1 document from the existing zod schemas and publish it with the public API, so integrators get typed clients and this reference can be checked against it automatically.

- **MQTT and OPC UA ingestion.** Gateways already record `protocol` (`https`, `mqtt`, `opcua`), but only HTTPS ingestion exists. Planned: an MQTT bridge and an OPC UA client that feed the same `ingest()` pipeline.
- **Outbound webhooks.** Signed callbacks for alarm and work events, with retries and a delivery log.
- **Shared rate-limit store.** Redis or Postgres-backed limits so serverless instances enforce one shared budget.

6 Mobile App Specification

> **NOTE**
>
> **HFG-MOB v2.0** · Draft for review · Owner: Mobile engineering, Hydris. What the native Hydris Field Android app does today, how it is built, and what it must do before pilot rollout

Hydris Field is the native Android app that field operators carry on shift. It signs in with the same Hydris account as the web console, shows the plant dashboard, alarms, work and live sensor charts, scans paper audit records into verified digital documents, answers spoken questions ("Hey Hydris") from live plant data, and connects Ray-Ban Meta glasses through Meta's Wearables Device Access Toolkit (DAT). This specification describes the app as it exists in `apps/android` on `main`, and marks every capability **Built**, **Partial** or **Planned**.

The previous edition of this document (version 1) described a target architecture before any code existed. Version 2 replaces it: the app is now written, and where the code differs from the earlier plan (no Hilt, no Room, no ViewModels yet, a single Gradle module), this document follows the code and records the earlier plan as Planned work.

5
Bottom tabs
Home, Work, Scan, Hydris, Glasses

7
Navigation routes
`Root.kt` NavHost

2
OCR passes per page
original and enhanced, on device

31
Minimum SDK
Android 12

> **NOTE**
>
> Source of truth for this document: `apps/android/app/build.gradle.kts`, `AndroidManifest.xml` and the Kotlin sources under `apps/android/app/src/main/java/ai/hydris/field/`. File names are given relative to that package unless stated otherwise. Screenshots in this document were taken from a debug build on an Android emulator against the demo deployment, so the Glasses screen shows the "Meta AI app not installed" state.

6.1 Product summary and goals

6.1.1 Who the app is for

The primary user is the **field operator** (role `field_operator`): the person walking rounds in an effluent or sewage treatment plant, reading gauges, acknowledging alarms and filling paper logs. Secondary users are **maintenance engineers** (work orders, checklists) and **supervisors** who want the plant status in their pocket. All of them already have a Hydris console account; the app adds no new identity.

6.1.2 Goals

ID	Goal	Status	Evidence
MOB-G01	One account everywhere: sign in with the console credentials and see the same organization, sites and data	Built	`HydrisApi.login`, `Repository` calls the shared `/api`

ID	Goal	Status	Evidence
MOB-G02	Plant status at a glance: open alarms, inspections due, maintenance, live measurements, alarm trend, asset health	Built	`HomeScreen.kt` over `GET /overview`
MOB-G03	Act on alarms from the field: acknowledge, investigate, resolve, close	Built	`AlarmDetailScreen` in `WorkScreen.kt`
MOB-G04	Turn a paper record into a verified digital document in under a minute	Built	`ScanScreen.kt`, `scan/DocDetect.kt`, `scan/OcrEngine.kt`
MOB-G05	Hands-free questions answered from live data, spoken back	Built	`AskScreen.kt`, `voice/VoiceEngine.kt`
MOB-G06	Glasses are optional: every glasses feature has a phone fallback	Built	Phone camera, phone microphone and phone speaker paths in `ScanScreen.kt` and `VoiceEngine.kt`
MOB-G07	Connect Ray-Ban Meta glasses and register them as a Hydris device so scans are attributed	Built (not verified on hardware)	`glasses/GlassesManager.kt`, `Repository.registerGlasses`
MOB-G08	Critical and high alarms reach the operator's phone in real time	Partial (foreground process only)	SSE in `Root.kt`, channel in `HydrisApp.kt`
MOB-G09	Work offline on a round and sync later	Planned	No local cache or outbox yet

6.1.3 Non-goals

- **Replacing the Meta AI app.** It is impossible under DAT and is not attempted (see the next chapter).
- **Equipment control.** The app never writes set points or commands to plant equipment. It reads telemetry and records human actions.
- **Administration.** Members, roles, sites, alarm rules, UI profiles and device revocation stay in the web console.
- **Audit document review and approval.** Reviewers confirm each value against the paper in the console. The app captures and sends; it does not approve.
- **iOS in this release.** The web glasses viewport already supports an iOS host through `window.HydrisWearables`, but there is no native iOS app.
- **Rendering a HUD on the glasses.** Ray-Ban Meta and Oakley Meta glasses have no display. On them the phone is the screen and the glasses are a camera, microphone, speaker and button.

6.1.4 Success measures (targets, not measured results)

Measure	Target	How it will be measured
Time from opening Scan to "Sent" for a one-page A4 record	Under 60 s on a mid-range phone	Pilot telemetry (Planned event logging) and stopwatch tests in QA
Share of scans that auto-capture without "Capture now"	Above 70 %	Planned client event `scan.autocapture`
Critical alarm to phone notification	Under 5 s while the app process is alive	QA with the alarm simulator
Spoken answer latency after the question ends	Under 3 s on 4G	QA script with fixed questions
Crash-free sessions in pilot	Above 99.5 %	Play Console vitals (Planned)

6.2 Relationship to the Meta AI app

6.2.1 What Meta requires

Ray-Ban Meta glasses are paired to a phone in Meta's **Meta AI** app (Android package `com.facebook.stella`). Meta's DAT FAQ states that "the Meta AI app must be used to pair your glasses", including for apps that process data elsewhere. Developer Mode, which lets an unpublished app connect to the glasses, is also switched on inside the Meta AI app. Meta's setup guide describes the toggle as hidden: Settings, App Info, then tap the app version number five times. Glasses firmware is updated through the Meta AI app, and DAT publishes a version dependency table that the Meta AI app and firmware must satisfy.

When Hydris Field calls `Wearables.startRegistration(activity)`, the Meta AI app opens so the wearer can approve Hydris, then returns to Hydris Field. The same happens when Hydris asks for a glasses capability permission (camera or microphone) through `Wearables.RequestPermissionContract()`.

6.2.2 Product position

> **DECISION**
>
> Hydris Field does not replace the Meta AI app. The Meta AI app stays installed and signed in on every operator phone that uses glasses. It pairs the glasses, approves Hydris as a connected app, grants camera and microphone access and updates firmware. After that one-time setup, operators open only Hydris Field; the Meta AI app works in the background as the connection broker.

The app makes this dependency visible instead of hiding it:

- `GlassesManager.metaAiInstalled()` checks for `com.facebook.stella` through the package manager. The manifest declares a `<queries>` entry for that package so the check works on Android 11 and later.
- If the Meta AI app is missing, "Connect glasses" does not call DAT. It shows: "Install the Meta AI app and pair your glasses in it first - Hydris connects through it." (see the Glasses screen chapter).
- Step 1 on the Glasses screen is titled "Link with the Meta AI app" and tells the user to keep it installed.

Table 156. Which app owns which responsibility

Responsibility	Meta AI app	Hydris Field
Bluetooth pairing of the glasses to the phone	Owns	Never touches
Glasses firmware updates	Owns	Shows errors reported by DAT
Developer Mode toggle	Owns	Documents the step
Approving Hydris as a connected app (registration)	Shows the prompt	Starts it with `startRegistration`
Camera and microphone permission for Hydris	Shows the prompt	Requests it with `RequestPermissionContract`
Camera stream, photo capture, capture button, speech	Brokers the link	Consumes through DAT sessions
Hydris account, plant data, scans, alarms, voice answers	Not involved	Owns
Registering the glasses as a Hydris device	Not involved	Owns (pairing endpoints)

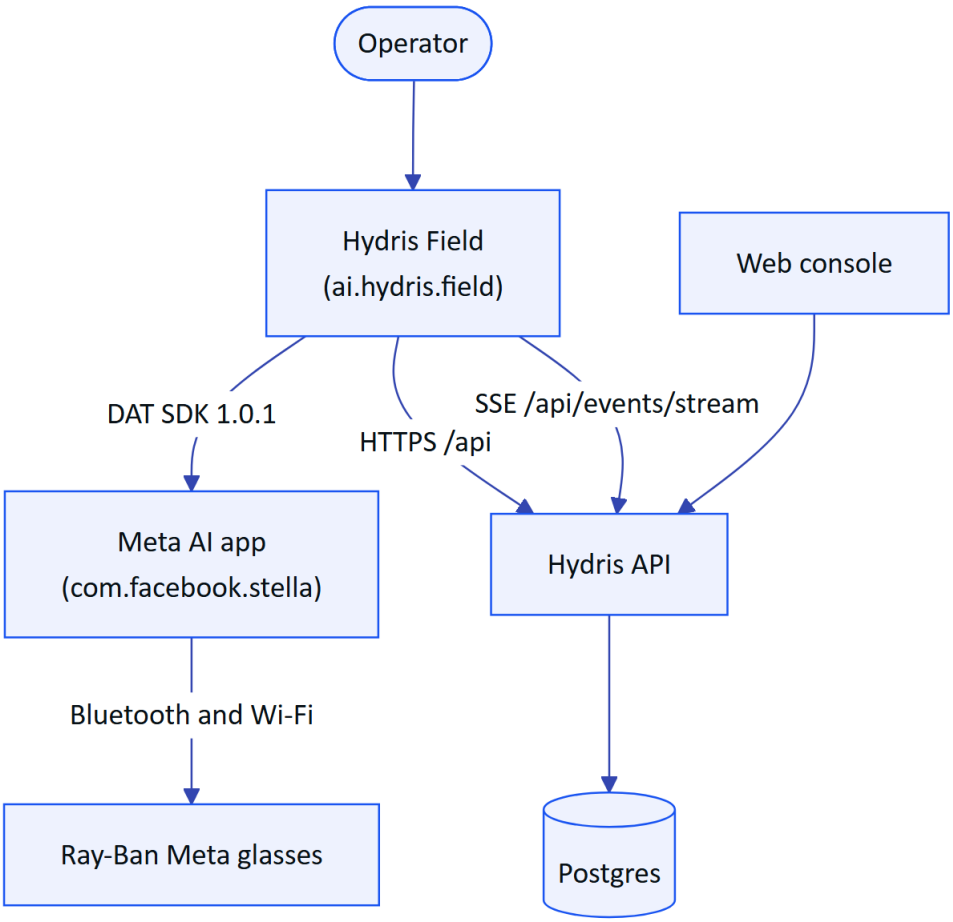

Figure 111. *Where Hydris Field sits between the operator, Meta's stack and the Hydris platform*

6.3 Architecture

6.3.1 Technology stack

Table 157. Technology choices as built (apps/android/app/build.gradle.kts, settings.gradle.kts)

Concern	Choice	Version	Status
Language	Kotlin, JVM target 17	2.2.21	Built
Build	Android Gradle Plugin, Gradle wrapper	AGP 8.11.1, Gradle 8.14.3	Built
SDK levels	`minSdk` 31 (Android 12), `compileSdk` and `targetSdk` 36		Built
UI	Jetpack Compose, Material 3, Material icons extended	Compose BOM 2026.05.01	Built
Navigation	`navigation-compose`, single activity	2.9.5	Built
Concurrency	Kotlin coroutines and Flow	1.10.2	Built
Serialization	`kotlinx-serialization-json`	1.9.0	Built
HTTP and SSE	OkHttp (one client, one SSE client without read timeout)	4.12.0	Built

Concern	Choice	Version	Status
Secrets	Android Keystore AES-256-GCM over SharedPreferences (`data/SecureStore.kt`)	Platform	Built
Preferences	`datastore-preferences`	1.1.7	Partial: declared, not used yet
Camera	CameraX `camera2`, `lifecycle`, `view`	1.4.2	Built
OCR	ML Kit Text Recognition (Latin)	16.0.1	Built
Image metadata	`androidx.exifinterface`	1.4.2	Built
Glasses	Meta DAT `mwdat-core`, `mwdat-camera`, `mwdat-speech`, `mwdat-inputs`; `mwdat-mockdevice` in debug	1.0.1	Built (not hardware-verified)
Charts	Hand-drawn Compose `Canvas` (`Sparkline`, `LineChart`, `Bars`, `Donut`)		Built
Dependency injection	None: app-wide singletons on `HydrisApp`		Built (Hilt is Planned)
Local database	None		Planned (Room)
Push	None (SSE only)		Planned (FCM)

NOTE

Meta's setup guide lists Android 10 as the DAT minimum. Hydris Field sets `minSdk = 31` (Android 12) on purpose: it matches the DAT sample apps and the existing bridge app, and Android 12 introduced the `BLUETOOTH_CONNECT` runtime permission the app is designed around.

6.3.2 Layers

The app is one Gradle module (`:app`) organized in packages. Screens are composable functions that hold their own UI state and call the `Repository` through a small `rememberLoad` helper. There is no ViewModel layer yet; long-lived state (session, glasses link, voice) lives in singletons created once in `HydrisApp.onCreate`.

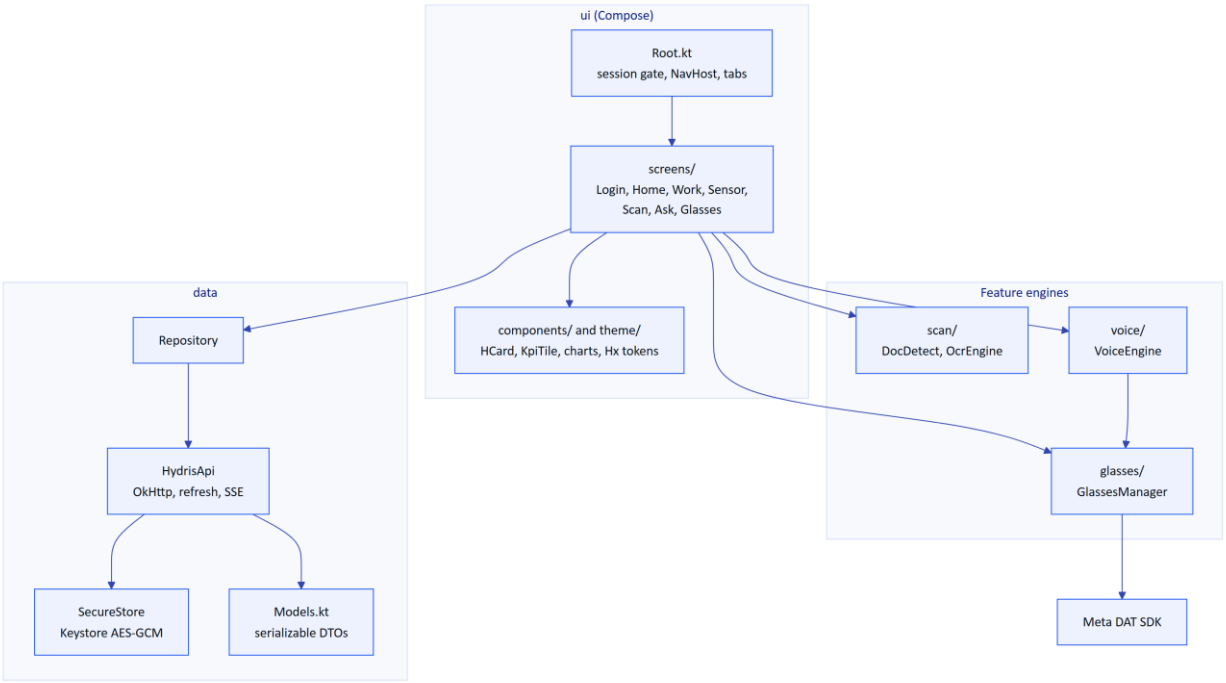

Figure 112. *Layers and packages in apps/android*

6.3.3 Package and file inventory

Table 158. Kotlin sources and their responsibilities

File	Lines (approx.)	Responsibility
`MainActivity.kt`	64	Single activity. Edge-to-edge dark system bars. Launches Bluetooth and notification permissions on every `onStart`. Bridges DAT permission requests (Meta AI app round trip) and phone camera or microphone permissions into suspend functions
`HydrisApp.kt`	31	`Application`. Creates `HydrisApi`, `Repository`, `GlassesManager`, `VoiceEngine`; registers the `alarms` notification channel
`data/HydrisApi.kt`	179	HTTP client, session refresh, encrypted cookie jar, device token channel, SSE event stream
`data/Repository.kt`	71	Every API call the screens use; scan upload; glasses registration flow
`data/Models.kt`	119	`@Serializable` DTOs mirroring `apps/api/src/modules`
`data/SecureStore.kt`	53	Encrypted key/value store
`scan/DocDetect.kt`	226	A4 page detection, guidance, motion, homography warp (port of `apps/web/src/lib/docdetect.ts`)
`scan/OcrEngine.kt`	115	Upload sizing, two ML Kit passes, image enhancement
`glasses/GlassesManager.kt`	305	DAT initialization, registration, session, camera, photo, capture button, speech
`voice/VoiceEngine.kt`	132	Speech input (glasses or phone) and TextToSpeech output
`ui/Root.kt`	142	Session restore, login gate, bottom navigation, realtime alarm notifications
`ui/theme/Theme.kt`	58	`Hx` colour tokens, Material colour scheme, typography
`ui/components/Components.kt`	208	Cards, chips, KPI tiles, progress bar, charts, time formatting
`ui/screens/*.kt`	1,300	Seven screens plus `Common.kt` (`rememberLoad`)

6.3.4 Core classes

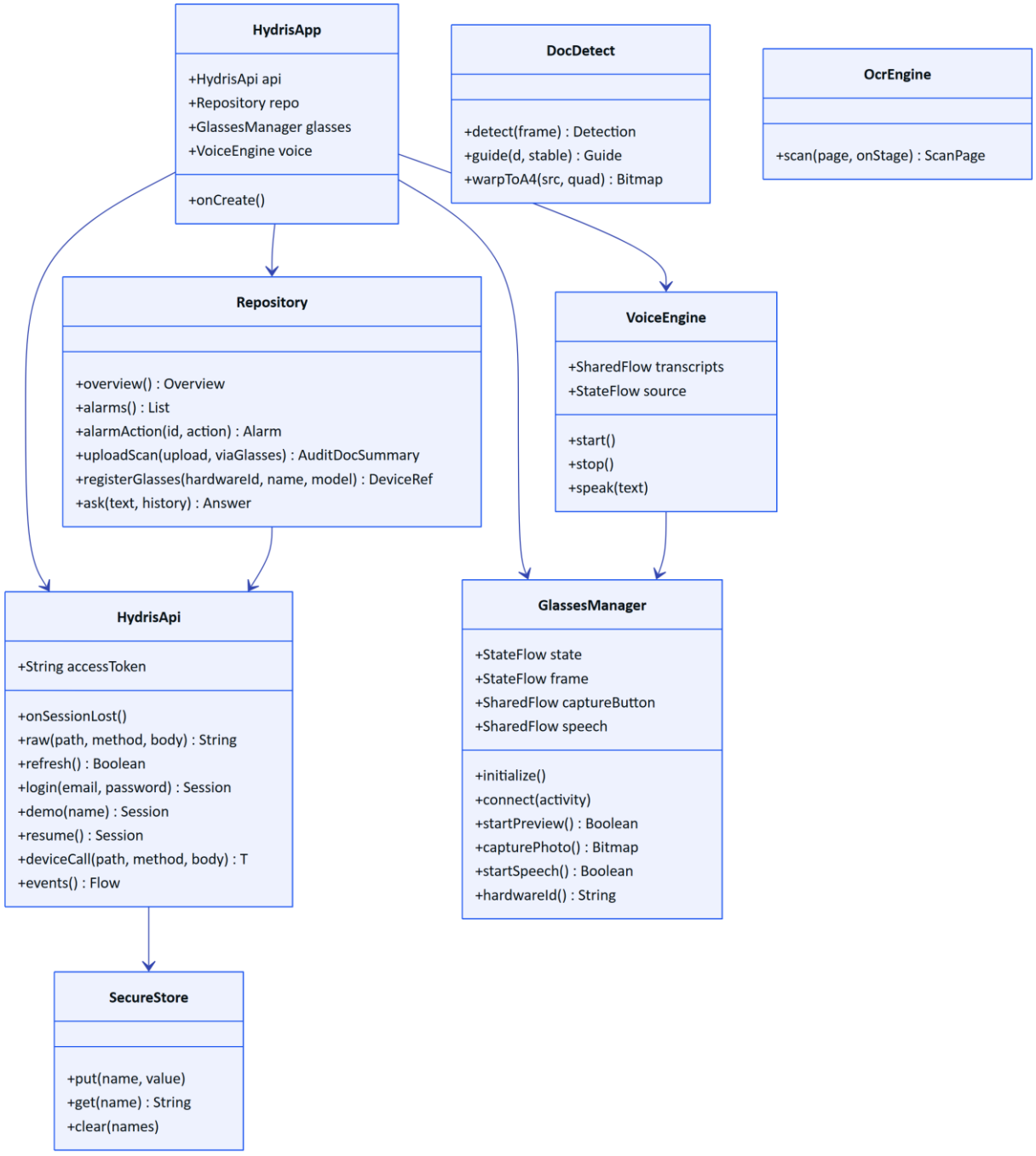

Figure 113. *Main classes and how they depend on each other*

6.3.5 State management pattern

Screens load data through `rememberLoad` in `ui/screens/Common.kt`. It returns a `Loadable` with three Compose states (`data`, `error`, `nonce`) and runs the loader in a `LaunchedEffect` keyed on the caller's keys and the nonce. When `refreshMs` is set, it loops with `delay(refreshMs)` for as long as the screen is composed, which is how live screens poll. `reload()` increments the nonce to force a reload, for example after an alarm action or a checklist toggle.

Table 159. Polling intervals per screen (from the refreshMs arguments)

Screen or tab	Loader	Refresh
Home	`overview()`	15 s
Work, Alarms tab	`alarms()` without closed alarms	15 s
Work, Inspections tab	`inspections()`	On open only
Work, Maintenance tab	`tasks()` without completed and cancelled	On open and after each toggle
Work, Sensors tab	`sensors()`	10 s
Work, Audit docs tab	`auditDocs()`	20 s
Sensor detail	`history(id, hours)`	15 s
Alarm detail	`alarm(id)`	On open and after each action
Scan menu, recent scans	`auditDocs().take(4)`	On open only

DECISION

Polling plus SSE is deliberate for the pilot. It keeps every screen correct after network drops without a local database, at the cost of extra requests while a screen is visible. The Planned offline work (Room cache and outbox) will move these loaders behind ViewModels and repositories that serve cached data first.

6.4 Navigation

6.4.1 Navigation map

`Root.kt` decides between three top-level states: restoring a session, signed out (Login) and signed in (`Main`). `Main` hosts a `Scaffold` with a Material 3 `NavigationBar` and a `NavHost` whose start destination is `home`. Tab navigation pops back to `home` with saved and restored state, so each tab keeps its scroll position.

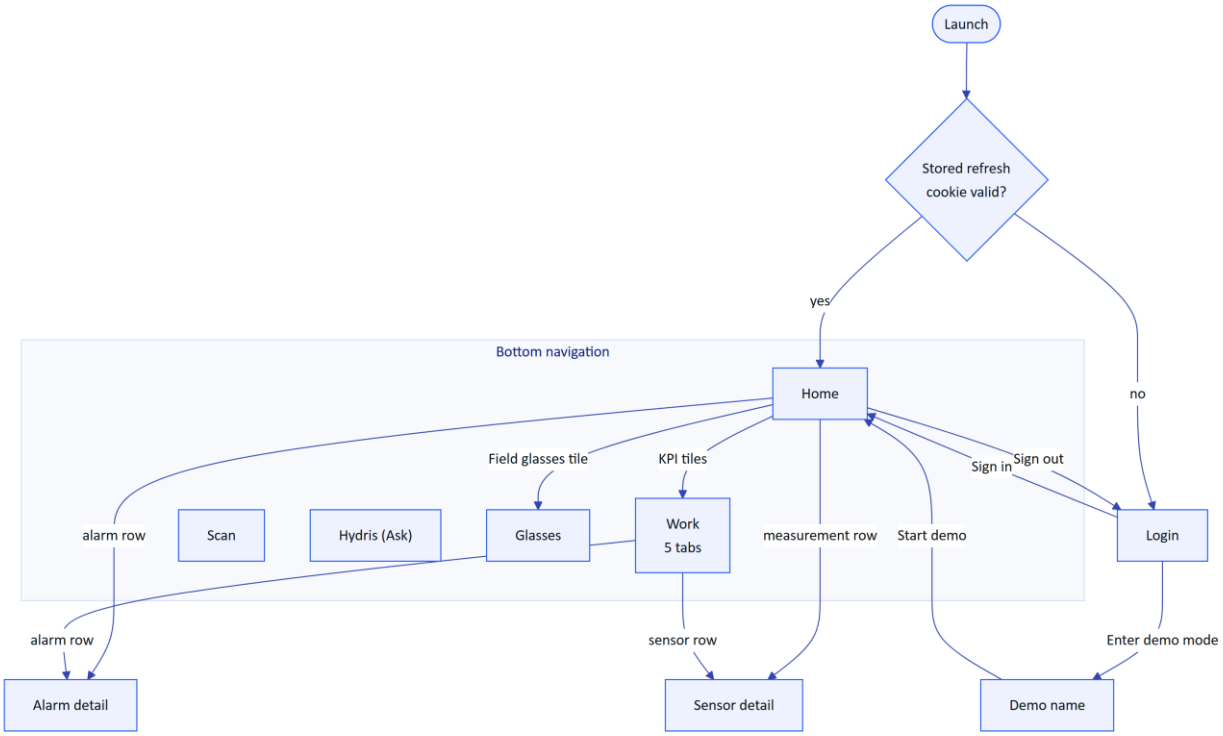

Figure 114. *Navigation map of the app as built*

Table 160. Routes registered in the NavHost

Route	Screen	Arguments	Entry points
`home`	`HomeScreen`	none	Start destination, Home tab
`work`	`WorkScreen`	none	Work tab, Home KPI tiles, Home "All sensors" link
`scan`	`ScanScreen`	none	Scan tab
`ask`	`AskScreen`	none	Hydris tab
`glasses`	`GlassesScreen`	none	Glasses tab, Home "Field glasses" tile
`alarm/{id}`	`AlarmDetailScreen`	alarm UUID	Home recent alarms, Work alarms tab
`sensor/{id}`	`SensorScreen`	sensor UUID	Home live measurements, Work sensors tab

WARNING

Known gap: the Home "All sensors" link navigates to `work`, which opens on the Alarms tab, not the Sensors tab. The fix is to pass the tab index as a route argument (`work?tab=3`). Planned for the next build.

6.4.2 Session gate

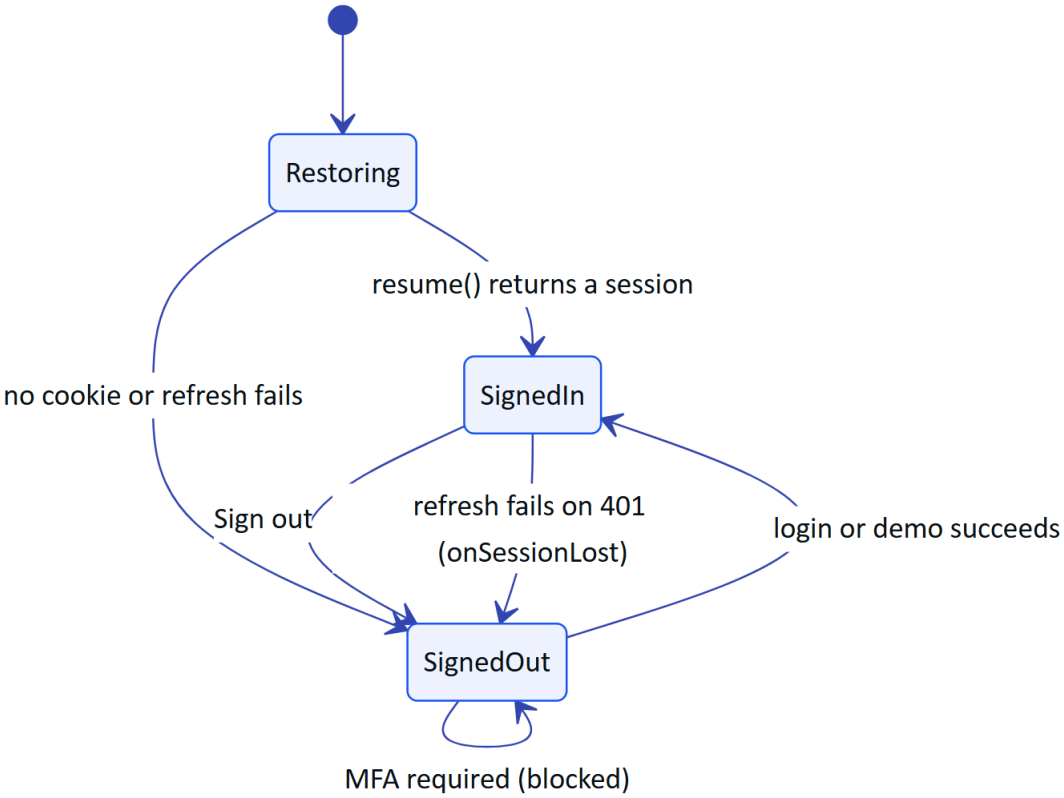

Figure 115. *Top-level session states in Root.kt*

While `Restoring`, the app shows a centred progress indicator on the Deep Midnight background. `HydrisApi.resume()` calls `refresh()` (which sends the stored `hg_refresh` cookie) and then `GET /auth/me`. If either fails, the user lands on Login. `onSessionLost` is installed after restore, so any later 401 that cannot be refreshed returns the user to Login.

6.5 Screen specifications

Each screen below lists its purpose, layout, data, state model, actions and status. Screenshots are from the demo deployment (`hydrisview.vercel.app`).

6.5.1 Login

Purpose. Sign in with the console account, or start a personal demo account.

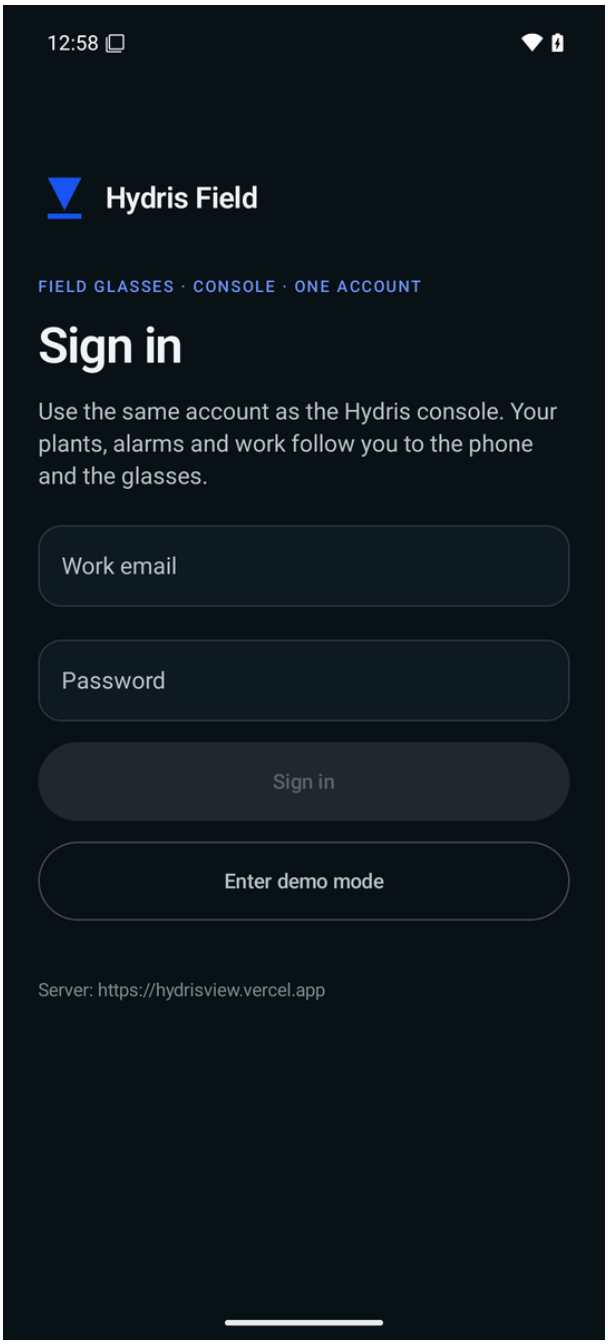

Figure 116. *Login screen with work email and password*

Layout. Hydris mark and "Hydris Field" title, eyebrow "Field glasses · console · one account", "Sign in" headline and a one-line explanation. Email field (email keyboard), password field (masked), primary button "Sign in" (54 dp high, Signal Blue), outlined button "Enter demo mode". The configured server URL (`BuildConfig.HYDRIS_URL`) is printed at the bottom so pilots can see which deployment the build points at.

Demo mode. Tapping "Enter demo mode" swaps the form for a single "Your name" field with the explanation "Demo mode creates a personal account with synthetic plant data, your own inspections and tasks." "Start demo" is enabled when the trimmed name has at least two characters and calls `POST /auth/demo {name}`.

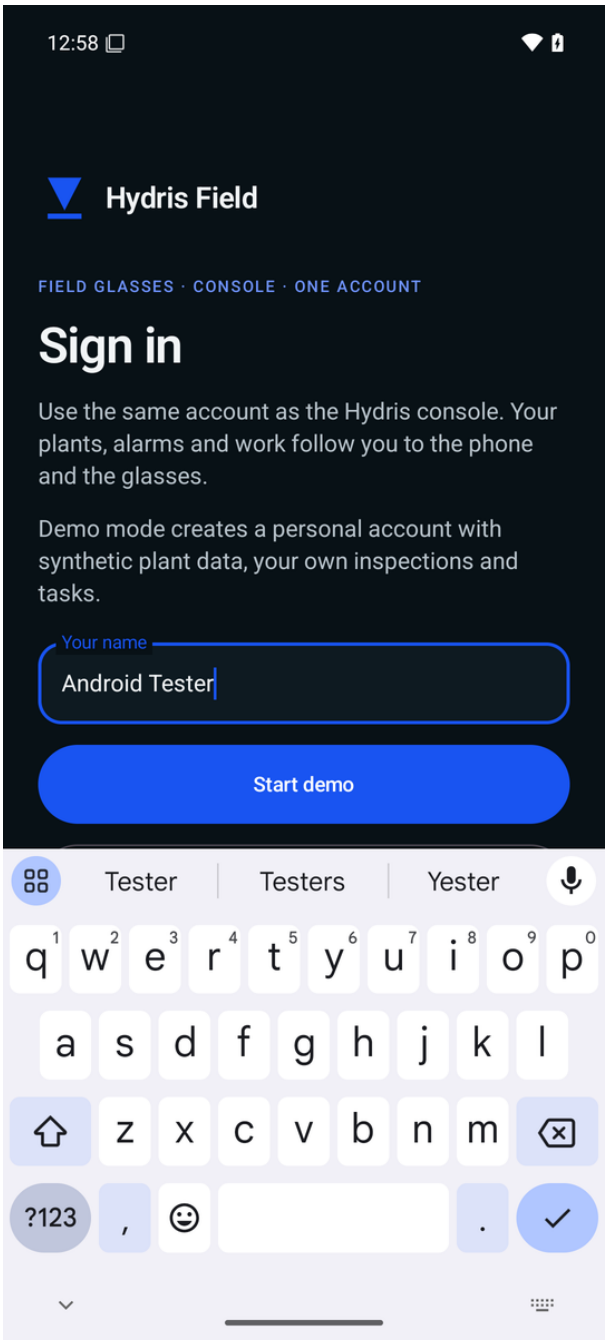

Figure 117. *Demo mode asks only for a name*

Table 161. Login states

State	Trigger	UI
Idle	Screen opens	Buttons enabled when email is not blank and password is not empty
Busy	Sign in or Start demo tapped	Button text "Signing in…" or "Starting…", buttons disabled
Error	API error	`ErrorNote` with the server message (for example invalid credentials or lockout)
MFA required	`Session.mfaRequired = true`	Error text: sign in on the web console first
Signed in	Session returned	`Root` switches to `Main`

> **IMPORTANT**
>
> In-app MFA is **Planned**. The API supports `POST /auth/mfa/verify`, and `Session` already carries `mfaRequired` and `mfaToken`, but the app does not yet show a code entry step. Accounts with TOTP enabled cannot sign in on the phone today. This blocks pilots where MFA is mandatory and is the first item in the release checklist.

Also Planned on this screen: organization switching for users with several memberships (`POST /auth/switch-org`), a server URL picker for multi-deployment pilots (today the URL is fixed at build time), and password reset links.

6.5.2 Home

Purpose. The operator's dashboard: what needs attention now on the sites they can see.

Figure 118. *Home dashboard with KPI tiles, alarm trend and asset health*

Data. `GET /overview`, refreshed every 15 s, plus the glasses link state from `GlassesManager.state`.

Layout, top to bottom.

1. Header: organization name eyebrow, time-of-day greeting with the first name ("Good morning, Meera."), a "Demo" chip for demo organizations and a sign-out icon.
2. Status line: "N critical alarms need attention." in Alert Orange, or "N open alarms, none critical.", or "All quiet on your sites."
3. Four KPI tiles in two rows: Open alarms (critical and unacknowledged counts, orange when any are critical), Inspections (due, overdue, in review; amber when overdue), Maintenance (open, high or urgent), Field glasses (connected over registered across the organization, with "Ray-Ban Meta linked" when this phone's glasses are registered with DAT).

4. Alarm trend: total for the last 14 days, count in the last 24 h, and a bar chart where the latest day is full Signal Blue.
5. Asset health: donut of asset statuses (operational, degraded, down, maintenance, standby) with a legend.
6. Live measurements: one card per metric with label, asset tag, freshness dot and age, "synthetic" when the source is the simulator, and the value with unit.
7. Recent alarms: severity dot, title, state and age, severity chip; critical alarms get an orange card border.
8. Activity: the last eight audit events with actor and age.

Table 162. Home interactions

Element	Action
Open alarms, Inspections, Maintenance tiles	Navigate to Work
Field glasses tile	Navigate to Glasses
Live measurement card	Open Sensor detail
Recent alarm card	Open Alarm detail
Sign-out icon	Stop voice, `POST /auth/logout`, clear cookies, return to Login

States. Loading (spinner) until the first overview arrives; error note if it fails with no cached value; content otherwise. A failed refresh keeps the last overview on screen and only shows an error when nothing has loaded yet.

6.5.3 Hydris assistant

Purpose. Ask questions about readings, alarms, work or an asset, by typing or by voice, and get answers grounded in live plant data.

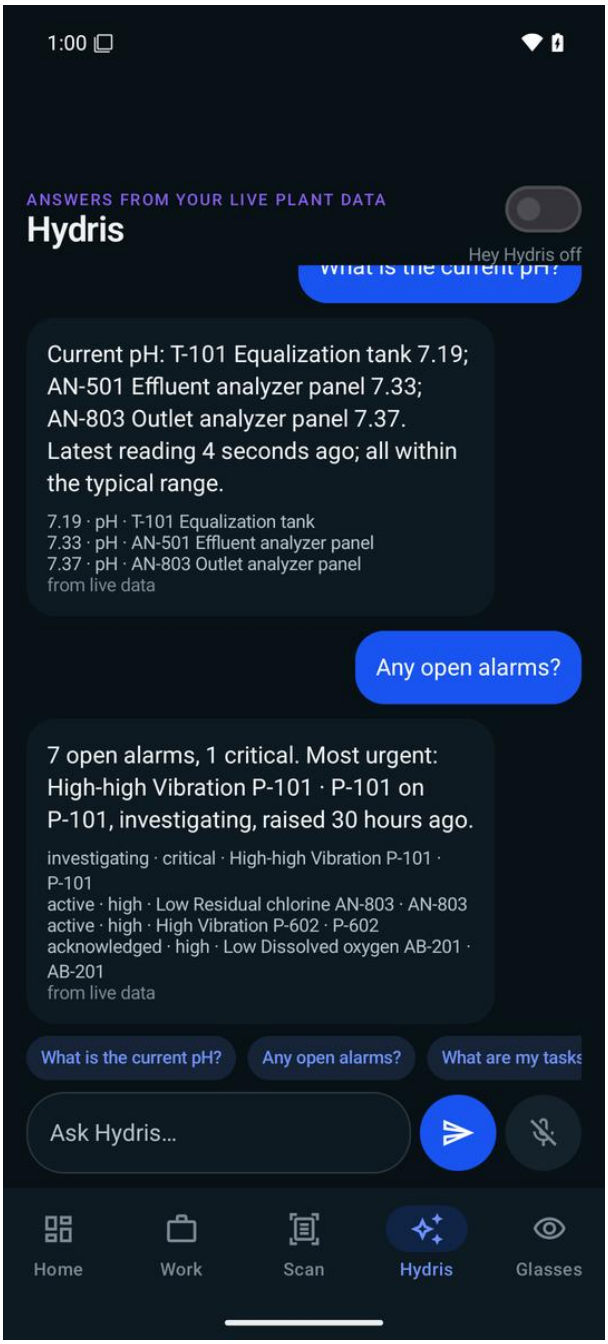

Figure 119. *Hydris assistant answering with facts from live data*

Layout. Eyebrow "Answers from your live plant data" in Insight Purple, a "Hey Hydris" switch with a status line ("Hey Hydris off", "listening · glasses mic", "listening · phone mic" or "starting…"), the conversation list, a row of suggestion chips ("What is the current pH?", "Any open alarms?", "What are my tasks?", "Dissolved oxygen", "Status of P-101"), a text field with a Send button and a Talk button.

Answers. `POST /assistant/ask {text, history}` with the last six turns. The answer bubble shows the text, up to four facts ("7.3 · Outlet pH"), and the engine label ("from live data" for the rules engine `hydris-rules`, otherwise the engine name). If the call fails, the bubble says "Sorry, I couldn't get that answer" with the error message.

Voice behaviour. The wake phrase regex is the same as `WAKE_RE` in `packages/domain/src/voice.ts`: "hey", "hi", "hay", "ok", "okay" or "a", followed by one of the common recognitions of "Hydris" ("hydris", "hydras", "hydrus", "hydrous", "high dris", "hi dris", "hydra", "idris"). A stop phrase ("stop", "cancel", "thanks", "thank

you", "never mind", "that's all", "bye") ends the follow-up window. Answers are spoken only when Hey Hydris is on.

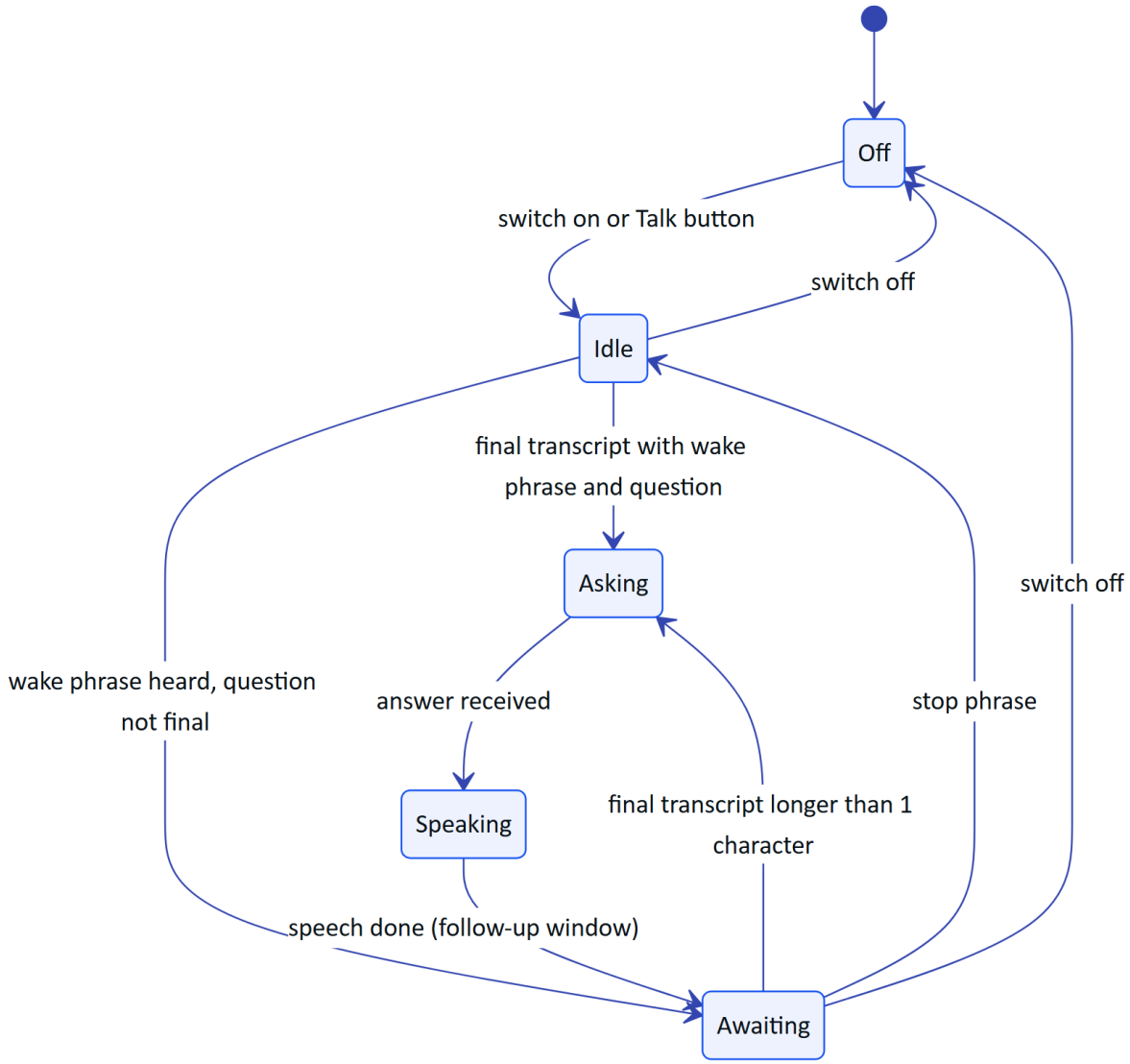

Figure 120. *Wake-phrase state machine in AskScreen.kt*

> **NOTE**
> The native app keeps the follow-up window open until a stop phrase or the switch is turned off. The glasses web viewport (`apps/web/src/glasses/VoiceAssistant.tsx`) closes it after 8 s of silence. Aligning the two (an 8 s timeout on the phone) is Planned.

6.5.4 Work

Purpose. Everything assigned or open, in five tabs: Alarms, Inspections, Maintenance, Sensors and Audit docs (`PrimaryScrollableTabRow`).

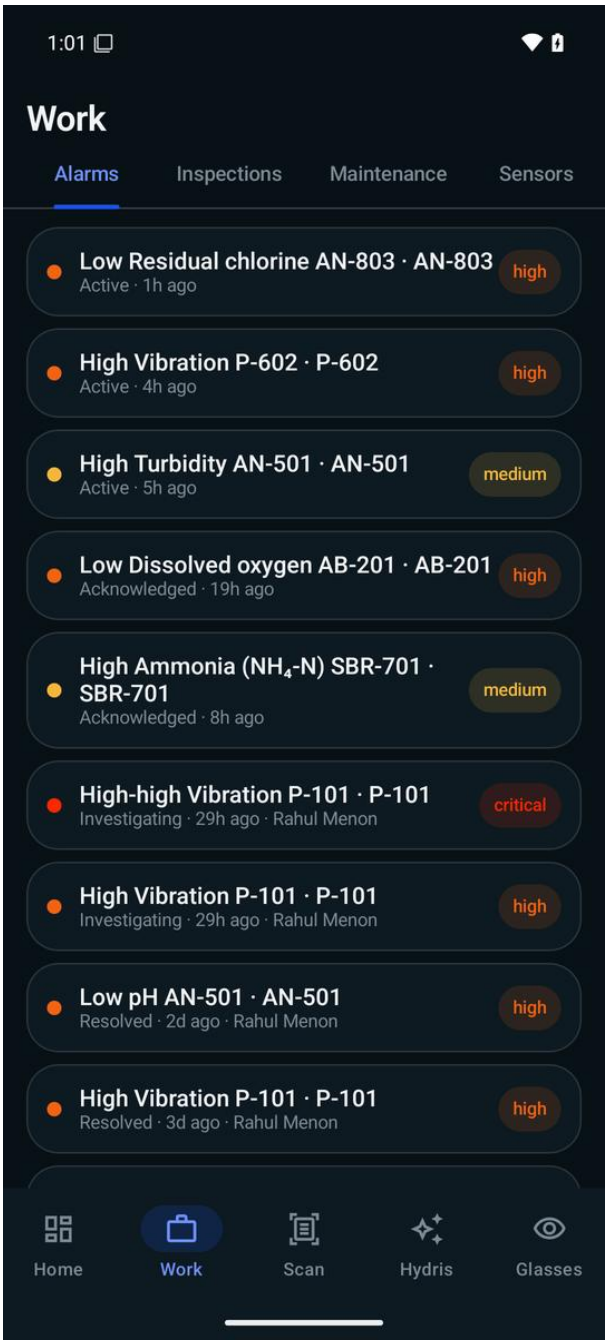

Figure 121. *Work screen on the Alarms tab*

Table 163. Work tabs

Tab	Data	Row content	Actions	Status
Alarms	`GET /alarms`, closed alarms hidden, 15 s refresh	Severity dot, title, state, age, assignee, severity chip; orange border when critical and active	Open Alarm detail	Built
Inspections	`GET /inspections`	Title, status chip, assignee, site, overdue marker, progress bar of answered over total steps	None (read-only)	Partial: running an inspection on the phone is Planned

Tab	Data	Row content	Actions	Status
Maintenance	`GET /maintenance`, completed and cancelled hidden	Title, priority chip, status, assignee, checks done; expands to description and checklist	Toggle checklist items (`POST /maintenance/{id}/checklist/{itemId}`)	Built; start, hold and complete are Planned
Sensors	`GET /sensors`, 10 s refresh	Name, freshness dot, tag, age, open alarm count, value in orange when alarmed	Open Sensor detail	Built
Audit docs	`GET /audit-docs`, 20 s refresh	Title, status chip, values, tables, capture route, author, age, review progress and engine agreement	None (review stays in the console)	Built

Empty states are explicit: "No open alarms.", "No inspections.", "No open maintenance tasks.", "No sensors.", and "No audit documents yet - scan one from the Scan tab."

6.5.5 Alarm detail

Purpose. Understand one alarm and move it through its lifecycle.

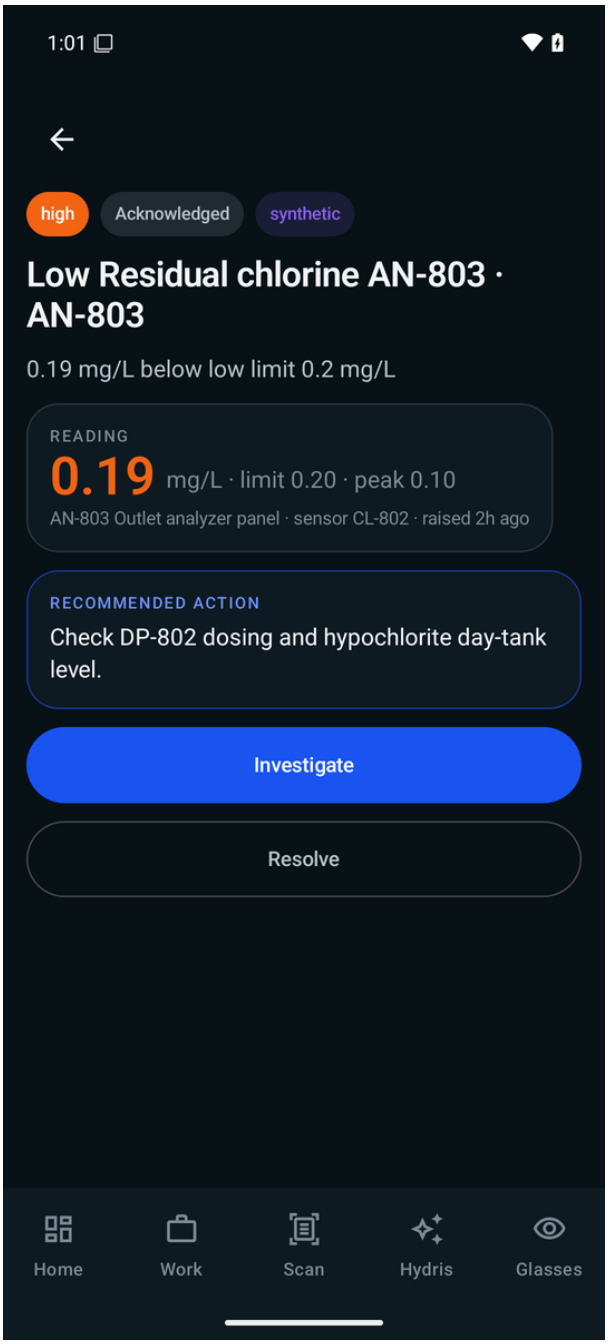

Figure 122. *Alarm detail with reading, limit, recommended action and actions*

Layout. Back button; severity chip (filled), state chip and "synthetic" chip for simulator alarms; title and description; a Reading card with the trigger value in the severity colour, unit, limit and peak, asset, sensor tag and age; a Recommended action card; then action buttons for the current state. The first action is a filled button, the second an outlined button.

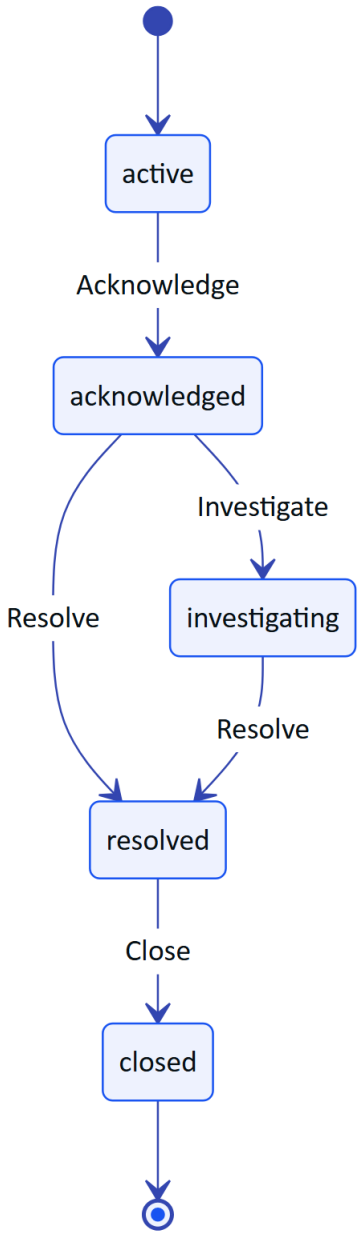

Figure 123. *Alarm actions offered by state (AlarmDetailScreen)*

Each button calls `POST /alarms/{id}/{action}` and reloads the alarm. Server rules still apply: if the user lacks the permission for an action, or another user already moved the alarm, the API error message is shown in an `ErrorNote` and the reloaded alarm shows the real state.

Planned on this screen: resolution notes (the repository already accepts a `note`), the alarm history timeline, the SOP link, assignment, and a "pending" marker for actions taken offline.

6.5.6 Sensor detail

Purpose. Trend of one sensor with its typical range.

Data. `GET /telemetry/sensors/{id}/history?from=…&points=144` with `from` set to 6, 24 or 168 hours ago (filter chips "6 h", "24 h", "7 d"), refreshed every 15 s.

Layout. Tag and measurand label, sensor name, the last value in 44 sp, freshness and the number of readings, the range chips, a 220 dp line chart with a gradient fill, the typical range shaded in green when the API

provides `typicalRange`, min, typical and max labels, and a "What this measures" card when the API returns an explanation.

Status. Built. Thresholds overlay, CSV export and pinch zoom are Planned.

6.5.7 Scan

The Scan tab is the app's most complex screen. Its pipeline is described in detail in the "Audit scan pipeline" chapter; this section covers the UI.

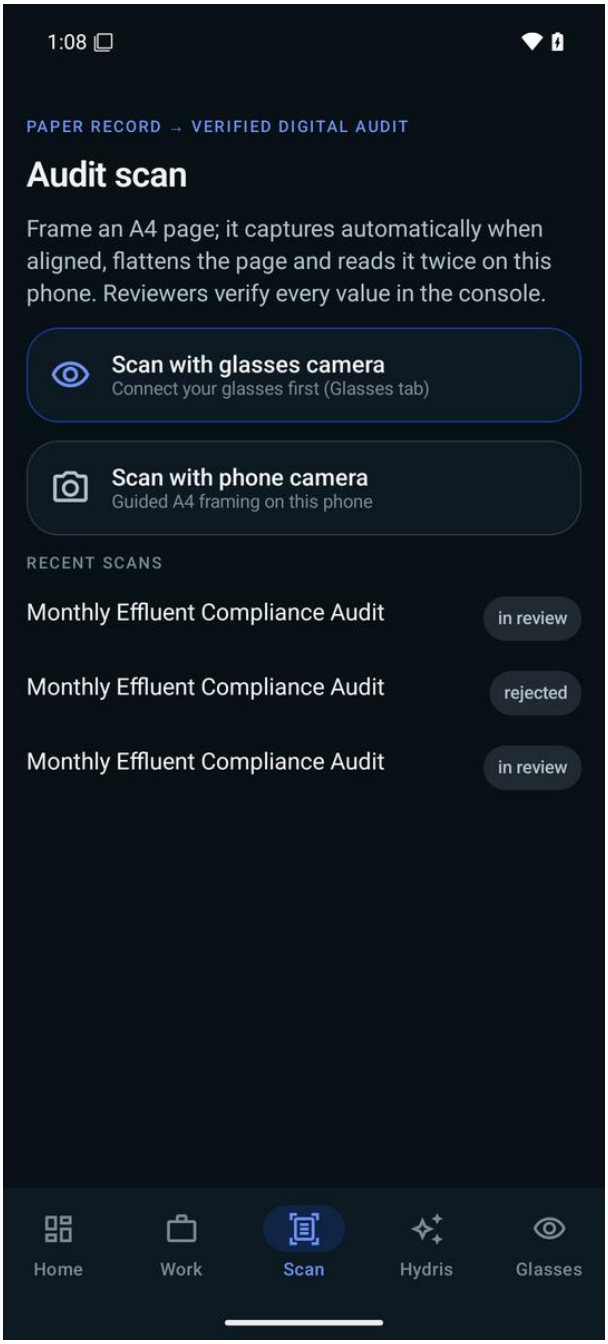

Figure 124. *Scan menu with glasses, phone and import options*

Menu. Eyebrow "Paper record → verified digital audit", title "Audit scan", explanation, and three source cards:

- **Scan with glasses camera** (Signal Blue border). Subtitle shows the glasses name and "or press the capture button" when DAT registration is complete, otherwise "Connect your glasses first (Glasses tab)".

- **Scan with phone camera**: guided A4 framing with CameraX. Asks for the camera permission in context.
- **Import a photo**: Android photo picker (`PickVisualMedia`, images only). Also the path on devices without a usable camera.

Below the cards, up to four recent scans with their status.

Guided capture. A status bar shows the guidance message with a coloured dot (green when ready, grey with no document, orange otherwise) and a source chip ("phone" or "Ray-Ban Meta"). The preview fills the remaining height with an A4 guide rectangle (86 % of the frame height), the detected page outline, corner dots and a green progress bar while the auto-capture timer runs. "Capture now" forces a capture; "Cancel" returns to the menu or to the review.

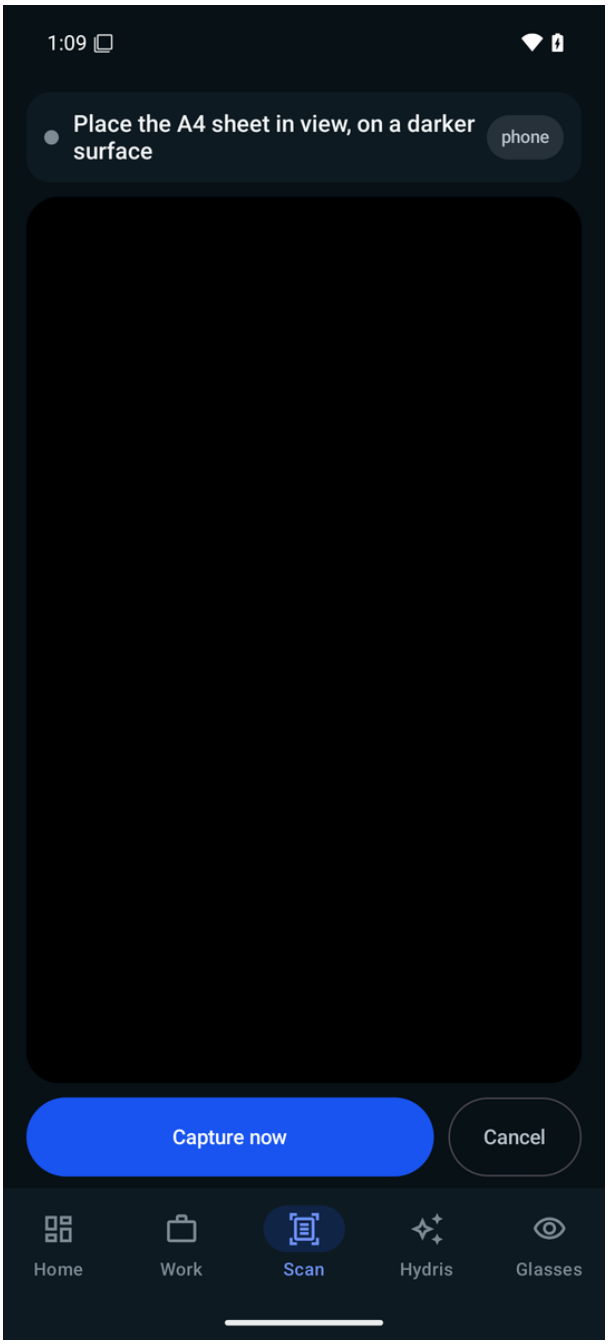

Figure 125. *Guided capture waiting for an A4 sheet*

Review. "N page(s) read on this phone", thumbnails of every page with the first pass's mean confidence, and a "Lines read (pass 1)" card with up to eight label and value pairs from the latest page. Buttons: "Send for verification", "Add page" (disabled at six pages) and "Discard".

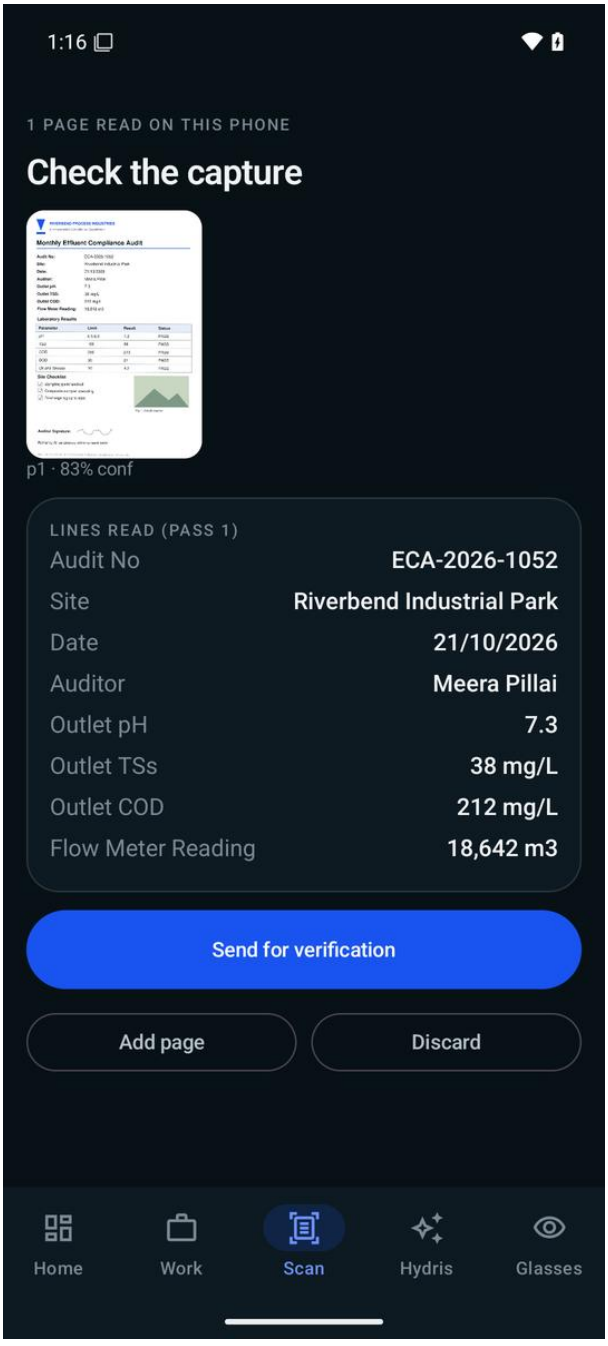

Figure 126. *Review of a captured page with label and value pairs*

Sent. Eyebrow "Sent · awaiting dual verification", the document title detected by the server, chips for values, tables, engine agreement and items to check, the capture route, and "Scan another document".

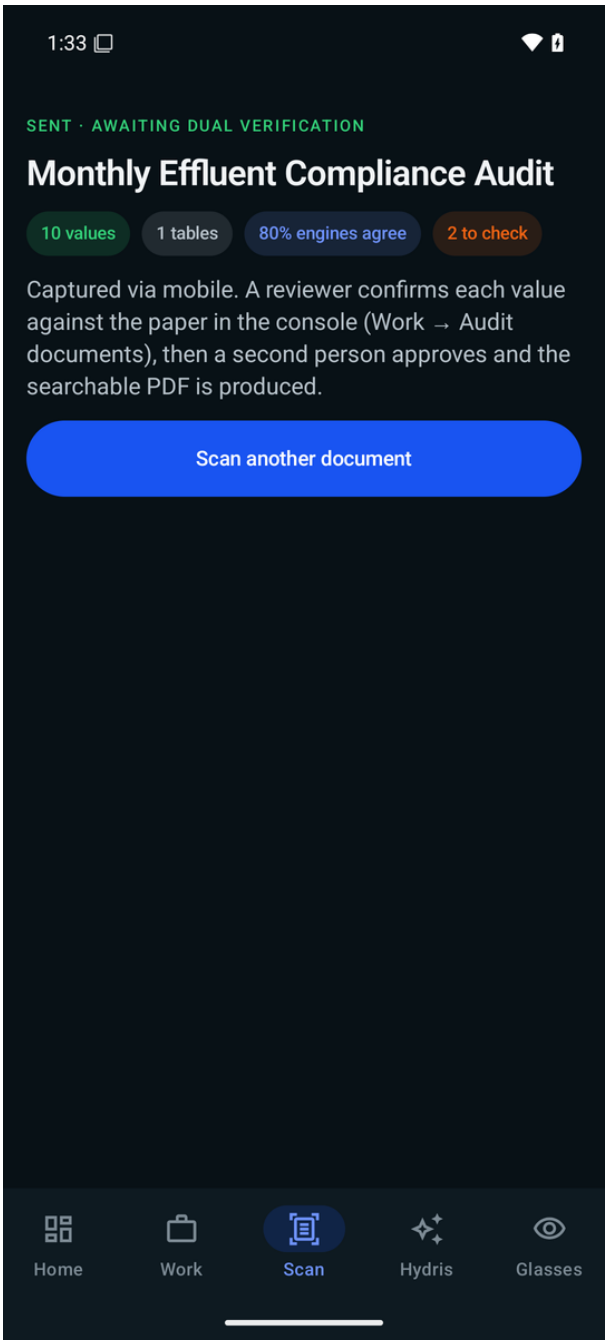

Figure 127. *Scan sent for dual verification*

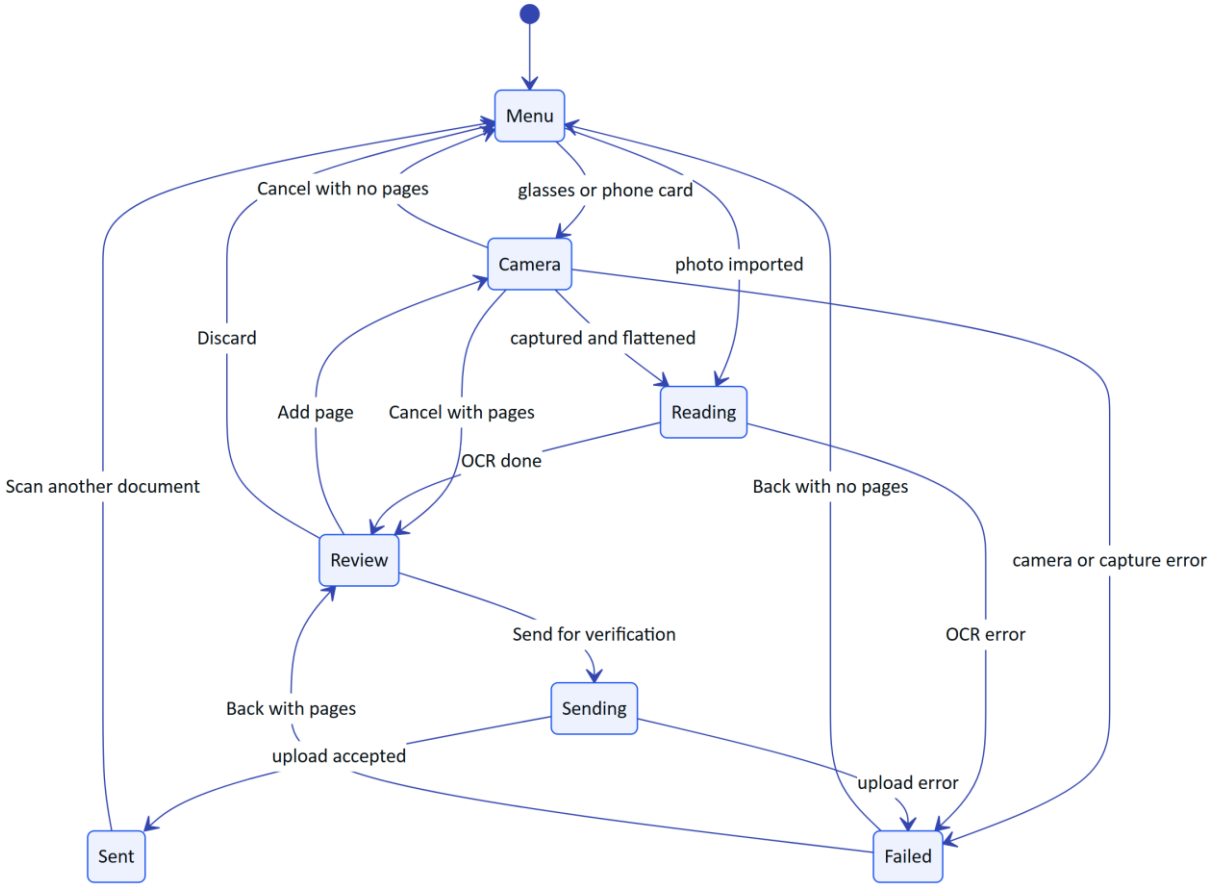

Figure 128. *Scan screen stages (sealed interface Stage in ScanScreen.kt)*

Capture button. While the menu or review is showing, a short press of the glasses capture button starts a glasses scan. During guided capture with the glasses source, it takes the photo.

6.5.8 Glasses

Purpose. One place to connect Ray-Ban Meta glasses, test them and register them with Hydris.

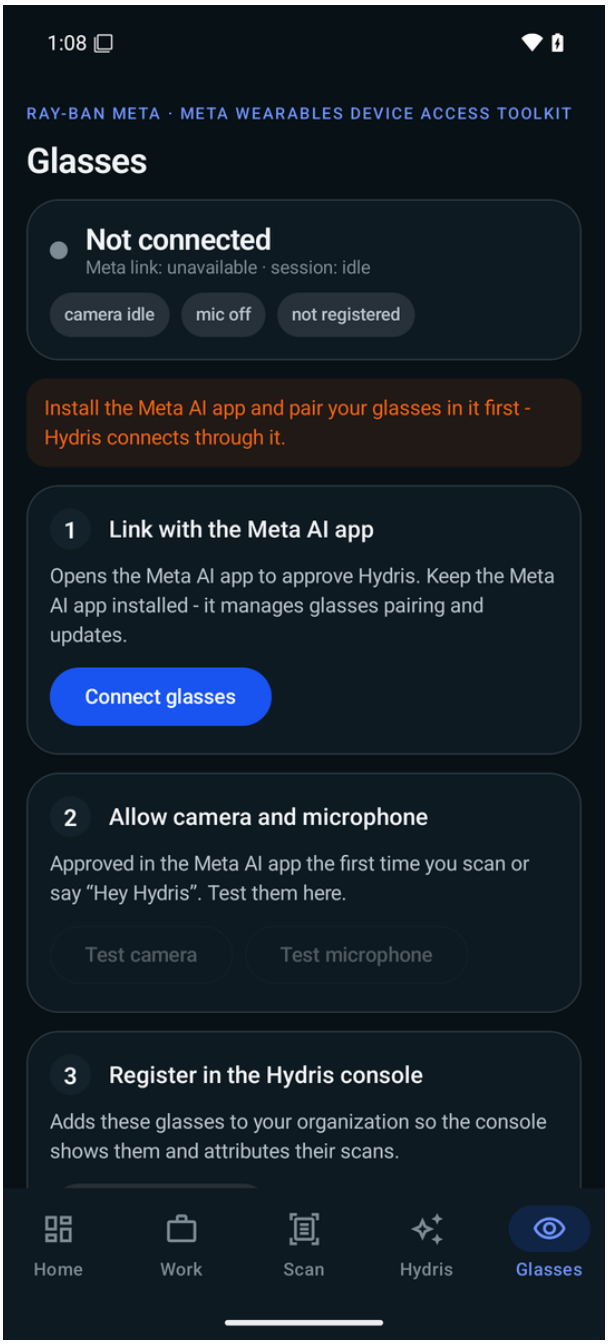

Figure 129. *Glasses screen before the Meta AI app is installed*

Status card. Glasses name (or "Waiting for glasses" or "Not connected"), status dot (green when registered and a device is visible, amber when registered with no device, grey otherwise), "Meta link" registration state and "session" state, and three chips: camera live or idle, mic state, and "in Hydris console" or "not registered".

Table 164. Glasses setup steps

Step	Done when	Action	Notes
1 Link with the Meta AI app	DAT `registrationState` is `REGISTERED`	"Connect glasses" initializes DAT if needed and calls `startRegistration`; "Disconnect" calls `startUnregistration`	Fails early with a clear message if the Meta AI app is missing
2 Allow camera and microphone	Camera streaming or speech listening	"Test camera" starts and stops a preview; "Test microphone" starts and stops speech	Each test triggers the Meta AI permission prompt the first time

Step	Done when	Action	Notes
3 Register in the Hydris console	A device refresh token is stored	"Register glasses" runs the pairing flow; "Unregister" logs the device out and forgets its tokens	Requires the `device.pair` permission on the server

Below the steps, a card lists what the glasses do in Hydris (audit scan, capture button, Hey Hydris, alarm notifications) and a "Connection log" card shows the latest eight of up to 30 DAT log lines kept in memory.

The DAT flows behind this screen are specified in the Glasses Integration document (HFG-GLS). The registration with Hydris is described under "Data layer and networking" below.

6.6 Data layer and networking

6.6.1 API client

`HydrisApi` talks to `${BuildConfig.HYDRIS_URL}/api`. `HYDRIS_URL` comes from `hydris_url` in `local.properties` and defaults to `https://hydrisview.vercel.app`. Every request sends `accept: application/json`; JSON bodies use `application/json`; non-GET requests without a body send `{}`.

Table 165. HTTP client configuration

Setting	Value	Why
Connect timeout	15 s	Fail fast on dead networks
Read timeout	90 s	OCR uploads with server-side vision extraction can take a while
SSE read timeout	0 (none)	The event stream is long-lived; the server sends heartbeats
JSON	`ignoreUnknownKeys`, `explicitNulls = false`, `encodeDefaults`	The API can add fields without breaking older app builds
Errors	`ApiException(status, code, message)` parsed from `{error:{code,message}}`	Screens show the server's message

6.6.2 Session model

The app follows the web session contract exactly, so the server needed no mobile-specific auth:

- A short-lived **access token** returned by `/auth/login`, `/auth/demo` and `/auth/refresh` is kept in memory only (`@Volatile accessToken`).
- A rotating **refresh cookie** (`hg_refresh`, path `/api/auth`) and, for demo accounts, the demo session cookie are stored by an OkHttp `CookieJar` whose backing store is the encrypted `SecureStore` entry `cookies`.
- The cookie jar only sends cookies to the configured host and drops expired cookies on load and on save.

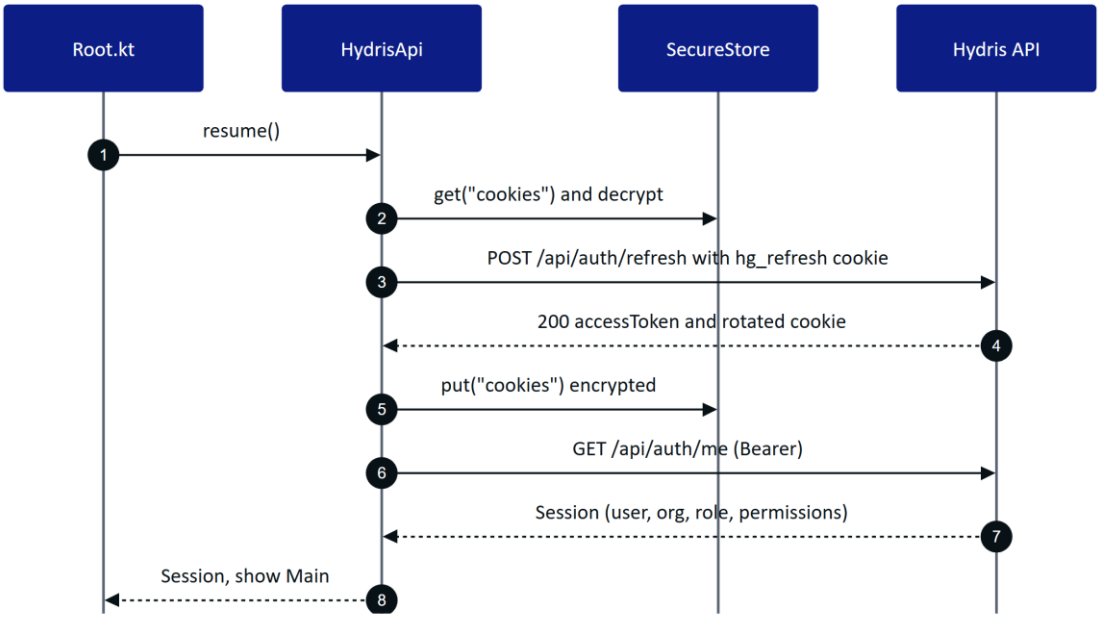

Figure 130. *Session restore on app start*

6.6.3 Refresh on 401 with a mutex

`raw()` retries a 401 once after a successful refresh, except for `/auth/login` and `/auth/demo`. `refresh()` is wrapped in a coroutine `Mutex`, so when several screens hit a 401 at the same moment the refresh calls run one after another and the server sees each rotated cookie in order. This matters because the API revokes the whole refresh family when a cookie is reused outside its grace window.

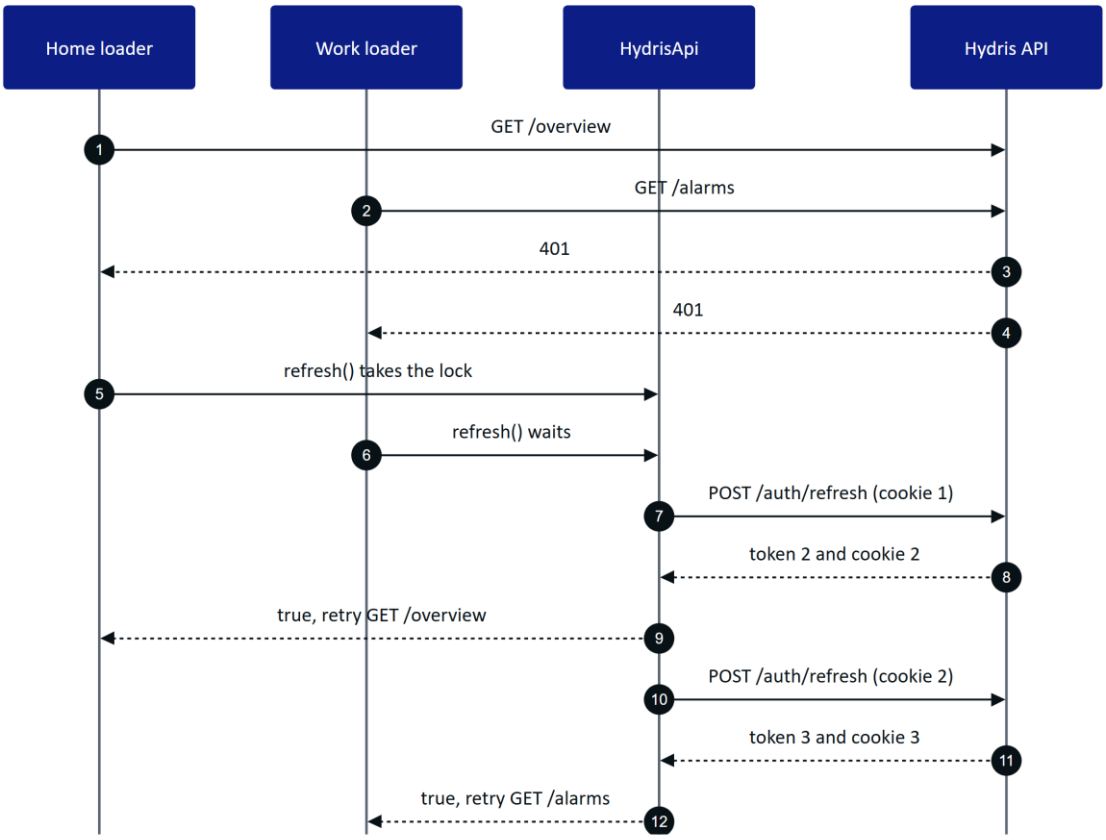

Figure 131. *Parallel 401s share one refresh at a time*

> **NOTE**
> The second waiter performs its own refresh rather than reusing the first result. That is correct but costs one extra rotation. A Planned improvement is to skip the refresh when the access token changed while waiting for the lock.

If the retry still returns 401, or the refresh fails, `onSessionLost` fires and the user returns to Login with "Session ended. Please sign in again."

6.6.4 Device credential channel

When the glasses are registered, a second credential set is stored in `SecureStore`: `device.access`, `device.refresh`, `device.name`. `deviceCall()` sends a request with the device access token and, on 401, exchanges the device refresh token at `POST /device/refresh` for a new pair and retries once. Scans captured with the glasses are uploaded through this channel so the console attributes them to the device (`capturedVia` set by the server from the principal), while every other call uses the user's token.

> **RISK**
> The device refresh in `deviceCall()` is not serialized by a mutex. Two concurrent glasses uploads could both rotate the device token and trigger reuse detection. Today only one upload runs at a time (the Scan screen is modal), so the risk is low; adding a lock is Planned.

6.6.5 Repository

Table 166. Repository functions and endpoints

Function	Method and path	Used by
`overview()`	`GET /overview`	Home
`alarms()`	`GET /alarms`	Work alarms tab
`alarm(id)`	`GET /alarms/{id}`	Alarm detail
`alarmAction(id, action, note)`	`POST /alarms/{id}/{action}`	Alarm detail
`inspections()`	`GET /inspections`	Work inspections tab
`tasks()`	`GET /maintenance`	Work maintenance tab
`toggleChecklist(task, item, done)`	`POST /maintenance/{id}/checklist/{itemId}`	Work maintenance tab
`sensors()`	`GET /sensors`	Work sensors tab
`history(id, hours)`	`GET /telemetry/sensors/{id}/history`	Sensor detail
`notifications()`	`GET /notifications`	Not used by a screen yet
`ask(text, history)`	`POST /assistant/ask`	Hydris assistant
`auditDocs()`	`GET /audit-docs`	Work audit docs tab, Scan menu
`uploadScan(upload, viaGlasses)`	`POST /audit-docs`, then `POST /audit-docs/{id}/enhance` when `aiStatus` is `pending`	Scan
`registerGlasses(hw, name, model)`	`POST /pairing/challenges`, `GET /pairing/lookup`, `POST /pairing/approve`, `POST /pairing/challenges/{id}/poll`	Glasses

Function	Method and path	Used by
`unregisterGlasses()`	`POST /device/logout` (device token)	Glasses

6.6.6 Registering glasses as a Hydris device

The console already pairs display glasses with an RFC 8628-style device authorization flow: the device asks for a challenge and shows a code, a signed-in user approves the code on `/pair`, and the device polls until it receives tokens. The phone plays both roles in one go: it creates the challenge for the glasses and immediately approves it as the signed-in user.

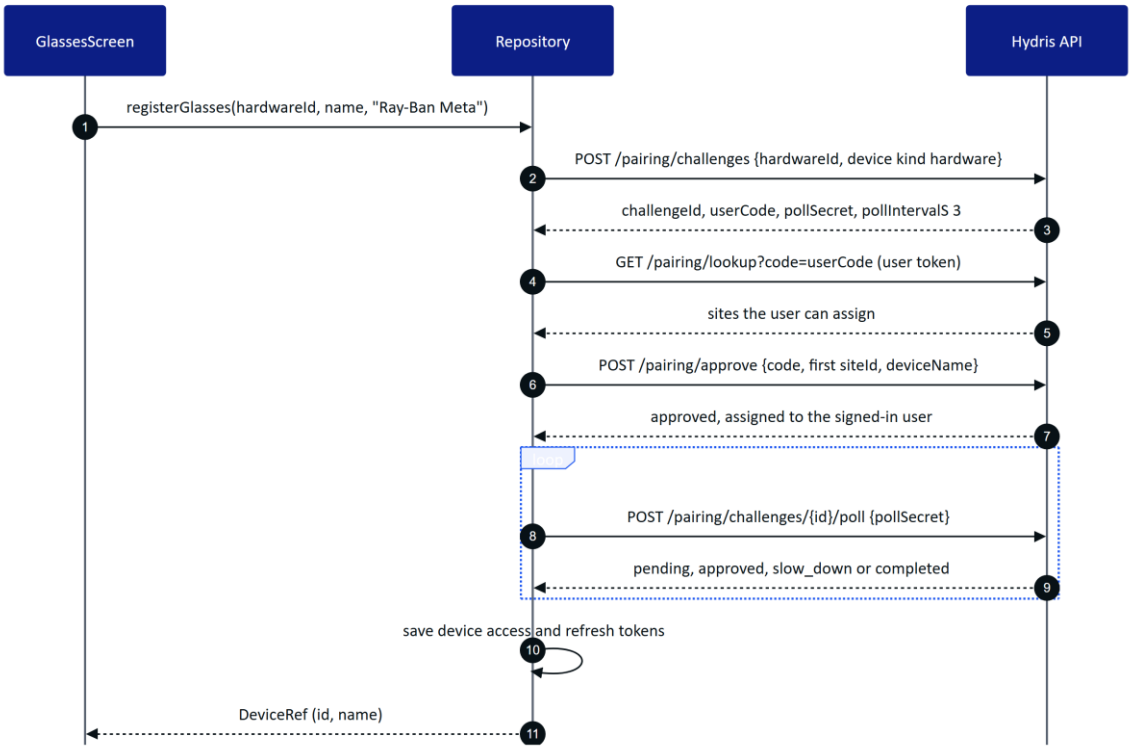

Figure 132. *Self-approved registration of the glasses (Repository.registerGlasses)*

Rules enforced by the server (`apps/api/src/modules/devices.ts`): the approver needs `device.pair`; pairing to another user needs `device.manage`; glasses registered to another organization must be revoked there first (`device_owned_elsewhere`); challenges expire after 300 s; polling faster than every 3 s returns `slow_down`. The hardware ID is `rbm-` followed by the first 16 bytes (hex) of SHA-256 over `hydris-glasses:` plus the DAT device identifier, so the raw Meta identifier never leaves the phone and re-registering the same glasses reuses the console's device record.

> **WARNING**
>
> The app assigns the glasses to the **first** site returned by `/pairing/lookup`. Users with access to several sites cannot choose. A site picker is Planned before multi-site pilots.

6.6.7 Realtime events (SSE)

`HydrisApi.events()` returns a cold `Flow<Pair<event, data>>` backed by a daemon thread. It connects to `GET /api/events/stream` with the bearer token and `accept: text/event-stream`, and sends `last-event-id` after

the first event so the server can replay missed events. The parser handles `event:`, `data:` and `id:` lines and emits on each blank line.

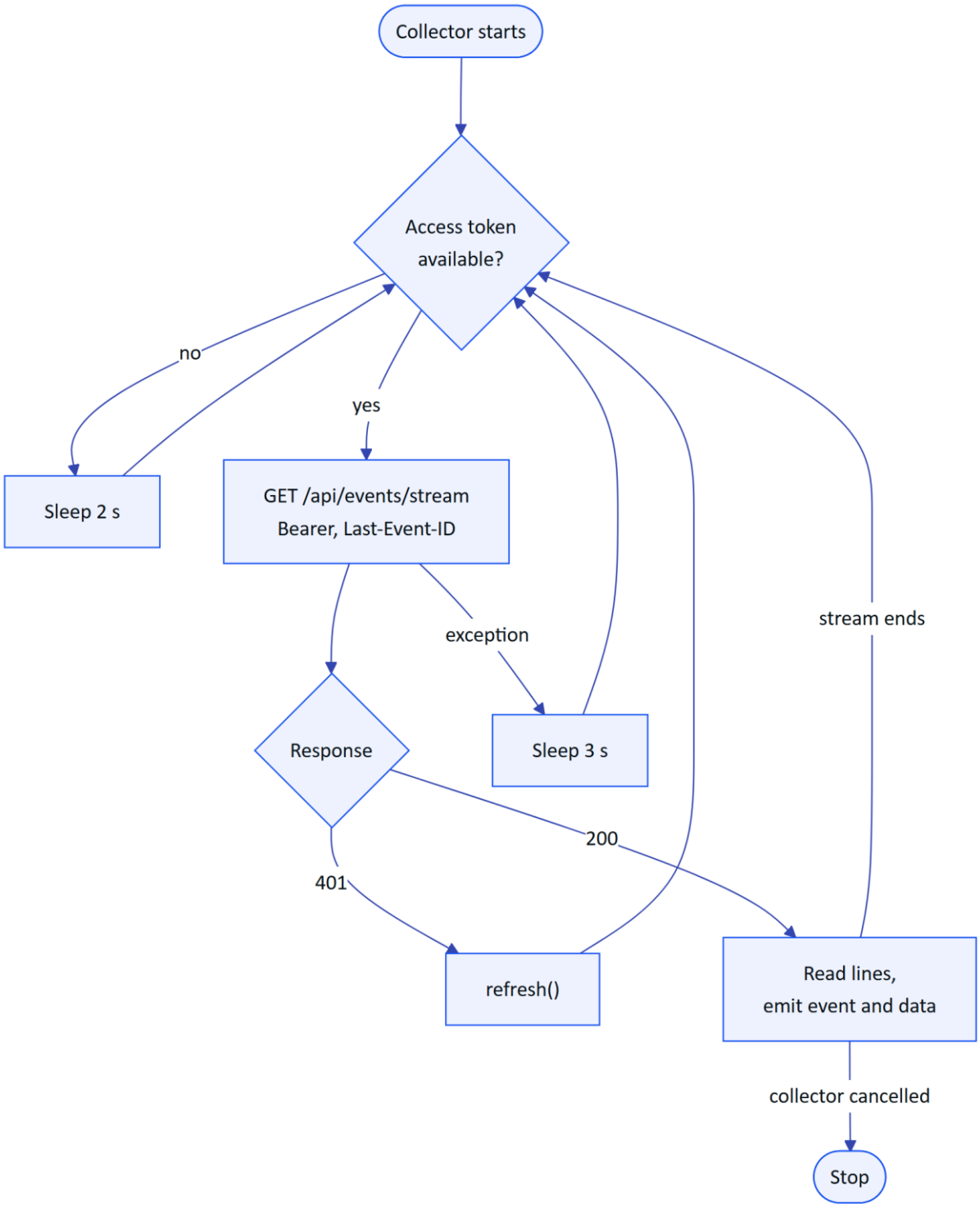

Figure 133. *SSE connection loop in HydrisApi.events()*

The server (`apps/api/src/modules/realtime.ts`) sends a `ready` event with `retry: 3000`, comment heartbeats every 15 s, and ends long-lived streams cleanly so clients reconnect with `Last-Event-ID`.

6.7 Audit scan pipeline

6.7.1 End-to-end flow

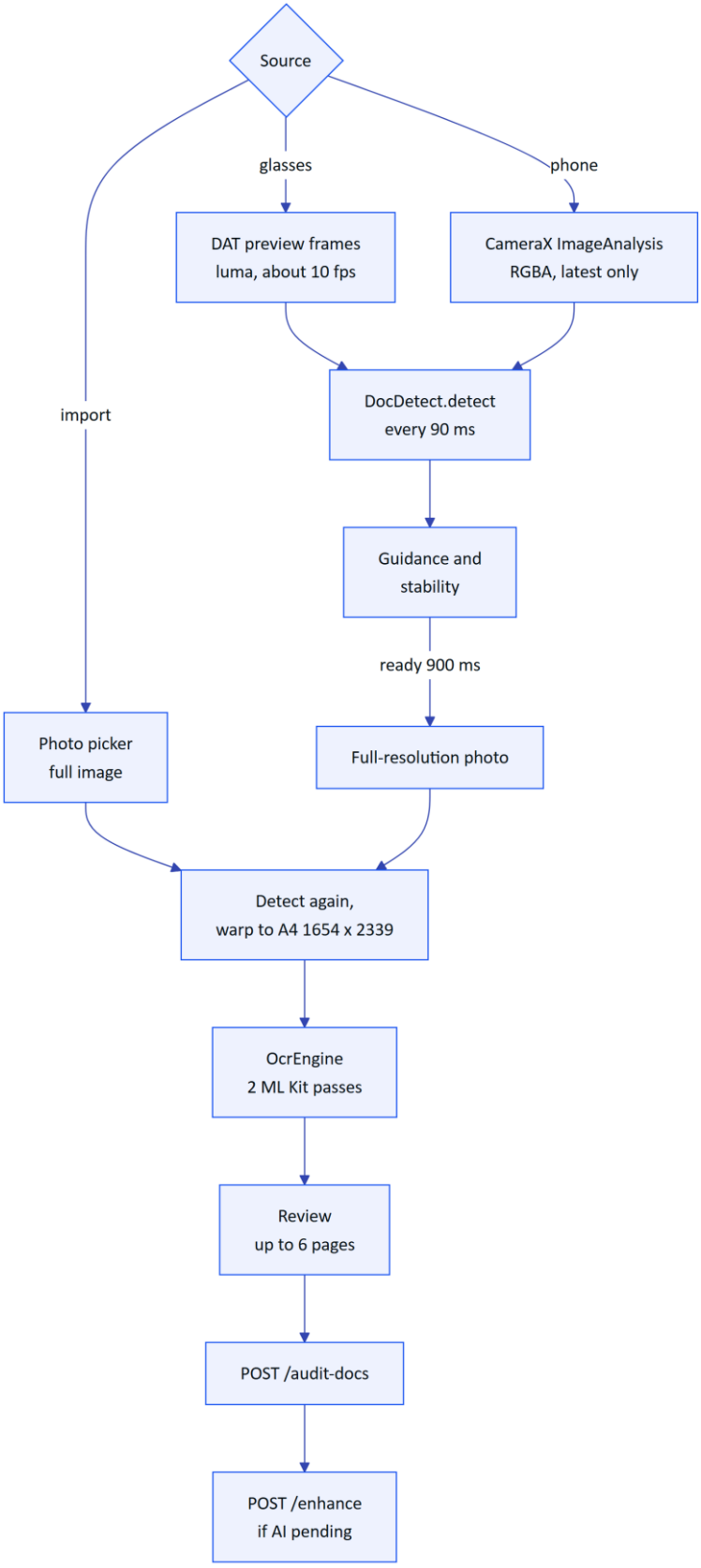

Figure 134. *Audit scan pipeline on the phone*

6.7.2 Page detection (DocDetect)

`scan/DocDetect.kt` is a line-by-line Kotlin port of `apps/web/src/lib/docdetect.ts` with the same thresholds, so the phone, the glasses web viewport and the console guide the wearer identically.

Table 167. Detection algorithm

Step	What it does	Constant
Downscale	Scale the frame to 200 px wide, keep aspect	`DW = 200`
Grayscale	Luma with integer weights 299, 587, 114 per 1000	
Threshold	Otsu's threshold on the histogram, floored	at least 90
Region	4-connected flood fill of bright pixels; keep the largest region	Region must cover at least 6 % of pixels
Corners	Extremes of x+y (top-left, bottom-right) and x−y (top-right, bottom-left)	
Coverage	Quad area over frame area	
Aspect	Longer over shorter mean side	A4 is √2 (1.414)
Skew	Largest deviation of a corner angle from 90°	
Tilt	Angle of the top edge	
Edge contact	Region pixels on the frame border over the perimeter	More than 8 % means cut off
Sharpness	Variance of the Laplacian inside the quad's bounding box	

6.7.3 Guidance

`DocDetect.guide()` checks conditions in a fixed order and returns the first message that applies. The order matters: there is no point asking the wearer to straighten a page that is half out of frame.

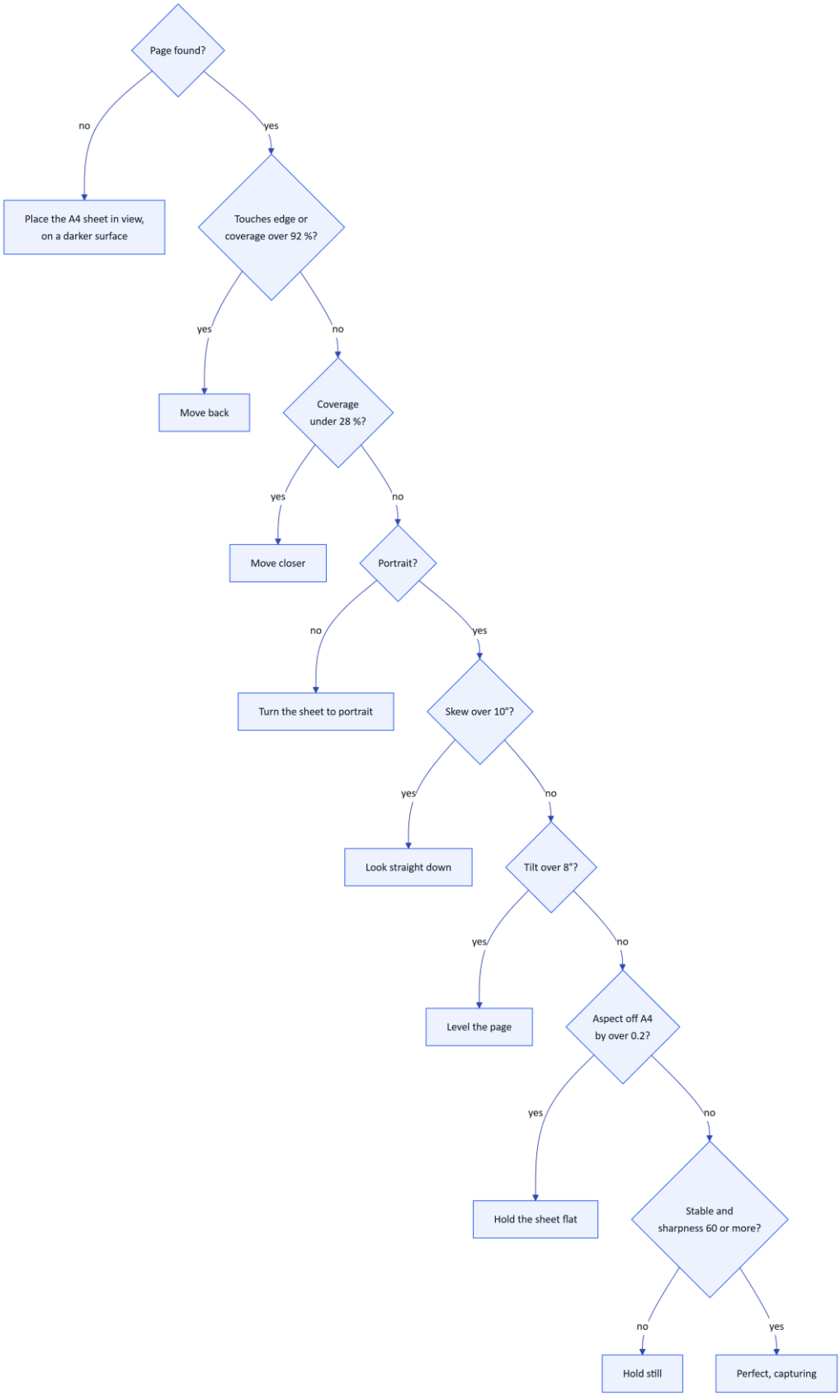

Figure 135. *Guidance decision order (DocDetect.guide)*

6.7.4 Stability and auto-capture

`GuidedCapture` runs detection at most every 90 ms. A frame is "still" when every corner moved less than 0.012 of the frame size since the previous detection (`DocDetect.motion`). Four still detections in a row make the page stable. When the guidance is "Ready", a timer starts; after **900 ms** of continuous readiness the app captures automatically. Any non-ready frame resets the timer. The green progress bar shows the timer.

6.7.5 Capture and flattening

On capture the app takes a full-resolution photo: CameraX `ImageCapture` in `CAPTURE_MODE_MAXIMIZE_QUALITY` (rotated by the reported rotation degrees), or `GlassesManager.capturePhoto()` for the glasses (EXIF orientation applied). Detection runs again on the full photo. If it finds a page with coverage over 20 % that does not touch the edges, that quad is used; otherwise the last quad from the preview is used; if there is none, the photo is kept as is.

`warpToA4` maps the unit square onto the quad with a projective homography (`squareToQuad`), insets the quad by 0.4 % to drop the page border, and samples the source bilinearly into a **1654 x 2339** bitmap: A4 at 200 dpi. Pixels outside the source are white.

> **NOTE**
> A comment in `apps/web/src/lib/docdetect.ts` still mentions 1240 x 1754 at 150 dpi, but the code in both the web and Android versions defaults to `outW = 1654`. The comment is stale.

6.7.6 On-device OCR (OcrEngine)

`scan/OcrEngine.kt` reads each page twice with ML Kit Text Recognition on the device. No image leaves the phone until the user taps "Send for verification".

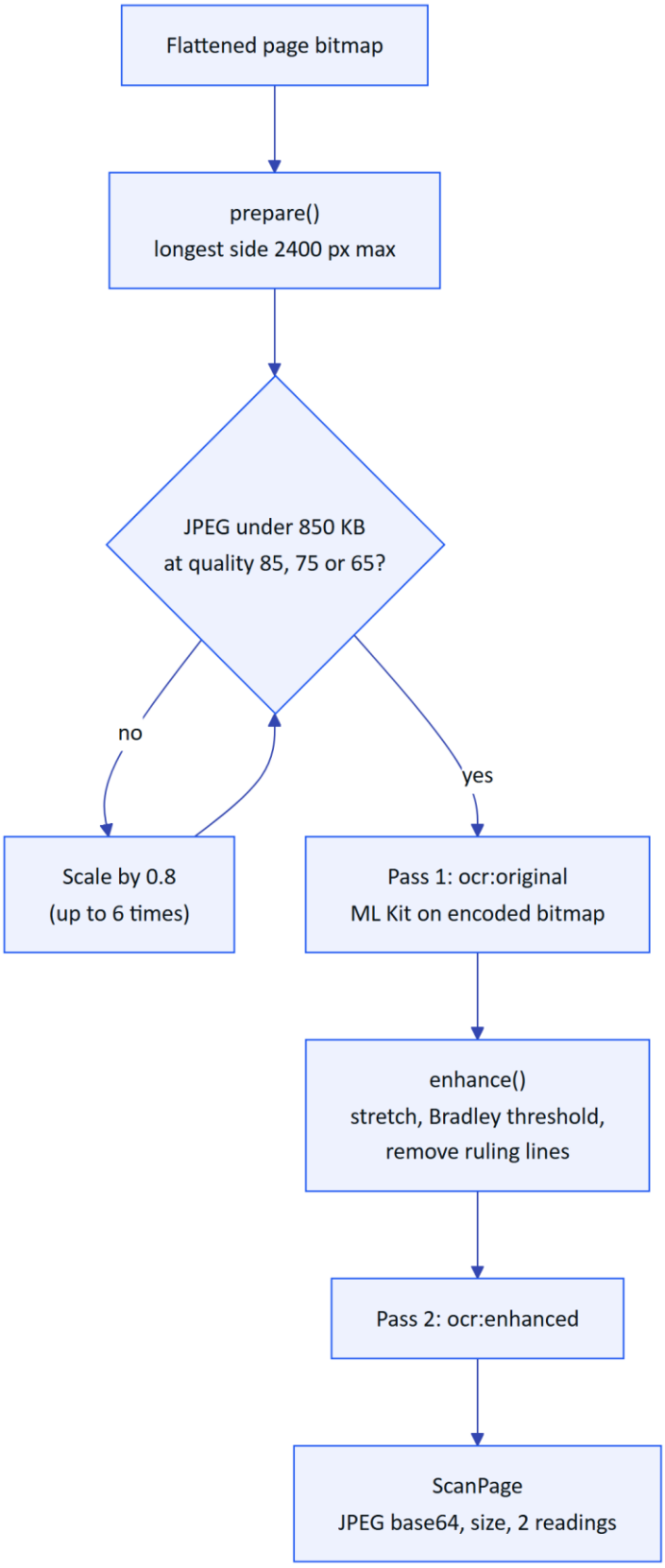

Figure 136. *Two-pass OCR and upload sizing in OcrEngine.scan*

Table 168. OCR details

Item	Behaviour
Size budget	Longest side at most 2400 px; JPEG at quality 85, then 75, then 65; if still over 850 KB, scale by 0.8 and retry, up to six times, else fail with "This image is too large to upload"
Pass 1	`ocr:original`: the encoded bitmap as captured
Pass 2	`ocr:enhanced`: grayscale, 1 % contrast stretch at both ends, Bradley adaptive threshold (window radius `max(8, min(W,H)/32)`, factor 0.85), then removal of horizontal black runs of at least 20 % of the width and vertical runs of at least 2.8 % of the height, so table borders and underlines do not merge cells
Output format	`OcrReading{engine, text, conf, lines[]}` with words; boxes normalized to `[x, y, w, h]` in 0..1; confidences scaled to 0..100 to match the Tesseract-compatible web format in `apps/web/src/lib/ocr.ts`
Line order	Sorted by top edge, then left edge
Progress	"Preparing image" 5 %, "Reading text (pass 1 of 2)" 15 %, "Reading text (pass 2 of 2)" 55 %, "Done" 100 %

The two readings are the machine half of Hydris's dual verification: the server compares them field by field (`agreement`) and flags disagreements for the reviewer.

6.7.7 Label and value pairing on the review card

ML Kit often returns a form's label column and value column as separate lines. `labelValues()` takes every line that contains a colon. If text follows the colon, that is the value. Otherwise it looks for the nearest line to the right (starting beyond 90 % of the label's width) whose vertical centre is within 60 % of the label's height, and uses it as the value. The card shows the first eight pairs. This is display only; the server does its own structuring.

6.7.8 Upload limits

Table 169. Upload limits (apps/api/src/modules/audit-docs.ts and OcrEngine.kt)

Limit	Server	App
Pages per document	6 (`MAX_PAGES`)	"Add page" disabled at 6
Bytes per page image	900 KB (`MAX_PAGE_BYTES`)	Encodes under 850 KB to leave headroom
Request body	8 MB	6 pages of up to 850 KB in base64 fit
Image types	JPEG or PNG, checked by magic bytes	Always JPEG
Source tag	`source` field	`"mobile"`

After a successful upload, if the server reports `aiStatus = "pending"`, the app calls `POST /audit-docs/{id}/enhance` to start the optional vision extraction. Failures of that call are ignored; the document is already saved.

6.8 Voice

6.8.1 Speech sources

`voice/VoiceEngine.kt` provides one transcript stream to the assistant regardless of where the audio comes from.

Table 170. Speech engines

Source	Recognition	Restart behaviour	Self-hearing protection
Glasses	DAT Speech capability on the glasses microphone; partial and final transcripts	`GlassesManager` restarts the speech capability when it reports `STOPPED` after having started, while listening is wanted	Transcripts are dropped while TTS is speaking
Phone	Android `SpeechRecognizer`, free-form model, partial results on, device locale	Restarts 150 ms after results, no match or speech timeout; 1200 ms after other errors	Recognizer is cancelled during TTS and restarted after
None	Not available	Not applicable	Not applicable

6.8.2 Speech output

Answers are spoken with Android `TextToSpeech` in the device locale, `QUEUE_FLUSH`, with a 30 s safety timeout. When the glasses are the active Bluetooth audio output, Android routes the speech to their open-ear speakers; the app does not route audio itself. `speak()` waits up to 1.5 s for the TTS engine to initialize the first time.

> **NOTE**
> The phone microphone needs `RECORD_AUDIO`, which the assistant requests when Hey Hydris is turned on. The glasses microphone needs the DAT `MICROPHONE` permission, approved in the Meta AI app.

6.8.3 Language

Recognition and speech use `Locale.getDefault()`. The assistant's answers come from the server in English. Hindi and other Indian languages for questions and answers are Planned and depend on server-side support in `voice-assistant.ts`.

6.9 Notifications

Built. `HydrisApp` creates a notification channel `alarms` ("Plant alarms", importance high, description "Critical and high alarms on your sites"). While the signed-in UI is composed, `Root.kt` collects the SSE stream and, for every `alarm.created` event whose `state` is `active` and whose severity is `critical` or `high`, posts a notification titled "Critical alarm" or "High alarm" with the alarm title, using the Hydris mark as the small icon and auto-cancel on tap.

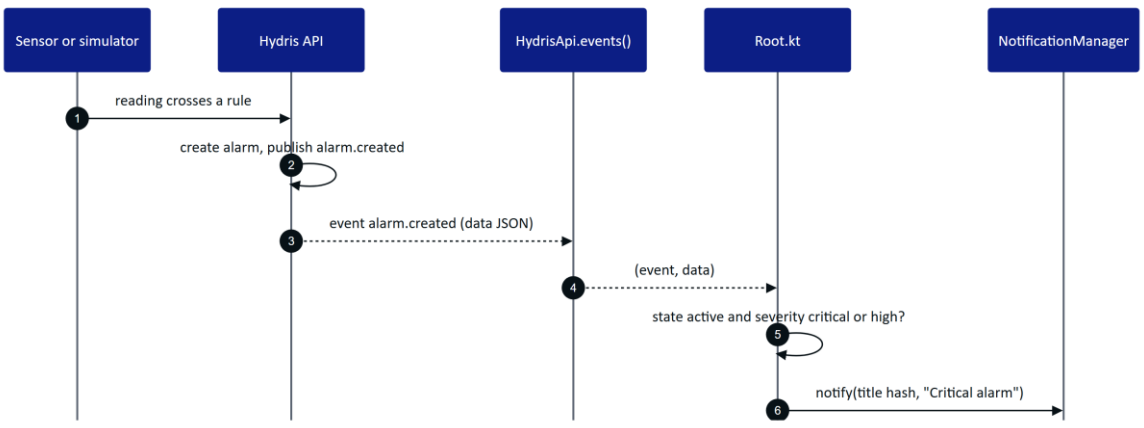

Figure 137. *Alarm notification path*

Table 171. Notification gaps

Gap	Effect	Plan
No push service	Notifications arrive only while the app process and signed-in UI are alive	FCM with a server-side sender (Planned)
Notification ID is the title's hash	Two alarms with the same title replace each other	Use the alarm ID (Planned)
Tap opens the app, not the alarm	Operator must find the alarm	Deep link `alarm/{id}` in the pending intent (Planned)
Permission asked on every start	Users who deny it are asked again on each `onStart`	Ask once, explain in Settings (Planned)
`GET /notifications` not shown	The in-app inbox is console only	Notifications screen (Planned)

6.10 Permissions

6.10.1 Permission matrix

Table 172. Android and DAT permissions

Permission	Kind	Declared in	Requested when	Used for	If denied
`BLUETOOTH`	Normal (install time)	Manifest	Always granted	DAT link	Not applicable
`BLUETOOTH_CONNECT`	Runtime (Android 12+)	Manifest	Every `MainActivity.onStart`, with the other two	DAT reaches the glasses through the Meta AI app	DAT is not initialized; glasses features unavailable, phone features work
`POST_NOTIFICATIONS`	Runtime (Android 13+)	Manifest	Every `onStart`	Alarm notifications	No notifications
`INTERNET`	Normal	Manifest	Always granted	API and SSE	Not applicable
`CAMERA`	Runtime	Manifest	Tapping "Scan with phone camera"	CameraX scan	"Camera permission is needed to scan with the phone"; import still works
`RECORD_AUDIO`	Runtime	Manifest	Turning Hey Hydris on	Phone microphone	Switch turns back off; typing still works
DAT `CAMERA`	Glasses capability	DAT	First glasses preview or photo	Glasses camera	"Camera permission denied in Meta AI"
DAT `MICROPHONE`	Glasses capability	DAT	First glasses speech start	Glasses microphone	"Microphone permission denied in Meta AI"; falls back to phone microphone

The manifest marks `android.hardware.camera` as not required, so the app installs on devices without a camera.

6.10.2 Glasses permission round trip

`MainActivity` registers `Wearables.RequestPermissionContract()` and exposes `requestGlassesPermission(permission)` as a suspend function guarded by a `Mutex`, so only one Meta AI prompt is open at a time. `GlassesManager.ensurePermission()` first calls `Wearables.checkPermissionStatus()` and only launches the prompt when the permission is not granted. The full sequence is in the Glasses Integration document.

6.11 Design system

6.11.1 Colour tokens

The `Hx` object in `ui/theme/Theme.kt` holds the brand palette shared with the website, console and glasses HUD. The app is dark only.

Table 173. Hx colour tokens

Token	Hex	Use
`Hx.Midnight` (Deep Midnight)	`#081116`	App and window background, status bar
`Hx.Surface`	`#0E1A21`	Cards, navigation bar, text fields
`Hx.Surface2`	`#13222B`	Raised surfaces, step numbers, Talk button
`Hx.Line`	white at 12 % alpha	Card borders, dividers, chart grid
`Hx.Signal` (Signal Blue)	`#1954F1`	Primary buttons, chart lines, progress
`Hx.Core` (Core Blue)	`#1746E4`	Primary container
`Hx.SignalText`	`#6F93FF`	Blue text and icons on dark, selected tab
`Hx.Alert` (Alert Orange)	`#FF2800`	Critical severity, errors
`Hx.High`	`#F26414`	High severity, stale data, warnings
`Hx.Medium`	`#F5B83D`	Medium severity, delayed data
`Hx.Ok` (Active Green)	`#00FE21`	Ready states, done steps (on dark only)
`Hx.OkSoft`	`#34D27A`	Live data, success chips, checklist done
`Hx.Insight` (Insight Purple)	`#8A60F0`	AI surfaces, synthetic markers, maintenance health
`Hx.Text`	`#F2F5F7`	Primary text
`Hx.Text2`	`#B7C2CA`	Secondary text
`Hx.Muted`	`#7D8B95`	Captions, inactive icons

Table 174. Semantic colour mappings

Mapping	Values
`Hx.severity()`	critical: Alert, high: High, medium: Medium, anything else: SignalText
`Hx.freshness()`	live: OkSoft, delayed: Medium, stale: High, anything else: Muted
Asset health (Home)	operational: OkSoft, degraded: High, down: Alert, maintenance: Insight, standby: Muted
Audit doc status	approved: OkSoft, rejected: Alert, verified: SignalText, others: Medium

6.11.2 Typography, shape and spacing

Table 175. Type scale (Material 3 Typography overrides)

Style	Size	Weight	Use
`displaySmall`	34 sp	SemiBold	"Sign in"
`headlineSmall`	24 sp	SemiBold	Screen titles
`titleLarge`	20 sp	SemiBold	Section titles
`titleMedium`	16 sp	Medium	Card titles
`bodyLarge`	16 sp	Regular	Long text
`bodyMedium`	14 sp	Regular	Body
`bodySmall`	12.5 sp	Regular	Captions
`labelSmall`	11 sp	Medium, 1.4 sp tracking	Uppercase eyebrows

Cards use a 22 dp corner radius and a 1 dp border; chips are fully rounded; text fields use 16 dp corners; primary buttons are 52 to 54 dp tall. KPI numbers are 34 sp. The app uses the system font; bundling Outfit to match the web is Planned.

6.11.3 Components

Table 176. Shared components (ui/components/Components.kt)

Component	Purpose
`HCard`	Rounded card with optional accent border and click
`Eyebrow`	Uppercase small label
`SectionTitle`	Section heading with optional action link
`Chip`	Rounded label, tinted or filled
`Dot`	Status dot
`KpiTile`	Label, large value, sub line, optional icon
`ProgressBar`	6 dp rounded bar
`Loading`, `ErrorNote`	Standard loading and error states
`Sparkline`, `LineChart`, `Bars`, `Donut`	Canvas charts with no chart library
`ago()`, `fmt()`, `label()`	Relative time, number and label formatting

6.12 Accessibility

Field use means gloves, glare, noise and one free hand. The requirements below are written against that context.

Table 177. Accessibility requirements

Requirement	Status	Notes
Text scales with the system font size	Built	All text uses `sp`
Primary actions at least 48 dp tall	Built	Buttons are 52 to 54 dp; tabs use Material defaults

Requirement	Status	Notes
Touch targets of 56 dp for gloved use on primary flows	Partial	Main buttons meet it; chips, checklist rows and the sign-out icon do not
Content descriptions on actionable icons	Partial	Back, Sign out, Send and Talk have labels; source cards rely on their text
Colour is never the only signal	Partial	Severity has text chips; freshness also has text; scan guidance has text; the alarm card border is colour only
Contrast on Deep Midnight	Built for text	`Hx.Text` and `Hx.Text2` are high contrast; `Hx.Muted` captions should be checked at 4.5:1
Voice alternative for the main question flow	Built	Hey Hydris and Talk button
Screen reader pass (TalkBack)	Planned	Not yet audited
Reduced motion	Not applicable	The app uses no decorative animation
Light theme for office use	Planned	Dark only today

6.13 Performance budgets

> **IMPORTANT**
>
> The numbers below are **targets** for pilot acceptance. None has been measured yet on reference hardware. QA will record results in the QA test plan (HFG-QA) on a mid-range reference phone.

Table 178. Performance targets

Metric	Target	Notes
Cold start to Home with a valid session	Under 2.0 s	Includes refresh and `/auth/me`
Home overview refresh	Under 800 ms on 4G	One request every 15 s
SSE alarm to notification	Under 2 s after the server publishes	Foreground process
Detection per preview frame	Under 25 ms	200 px frame on the main thread; moving it off the main thread is Planned
Warp to A4	Under 1.5 s	3.9 MP output with bilinear sampling in Kotlin
OCR, two passes, one A4 page	Under 6 s	ML Kit on device
Upload, one page	Under 900 KB	Enforced by `OcrEngine`
Peak memory during a six-page scan	Under 300 MB	Pages are kept as full bitmaps for thumbnails; downsampling thumbnails is Planned
Glasses camera on	Only while the scan camera is open	See the battery gap below
APK size (release, minified)	Under 45 MB	ML Kit bundled model and DAT included

RISK

Battery: `GlassesManager.stopPreview()` stops forwarding frames but does not stop the DAT camera stream; the stream ends only when the session stops. On hardware this may keep the glasses camera and LED on after leaving the scan screen. Stopping the stream on `stopPreview` is a Planned fix that must be verified on real glasses.

6.14 Offline behaviour and sync

6.14.1 Today

Table 179. Offline behaviour as built

Situation	Behaviour
Network lost on a loaded screen	The last loaded data stays on screen; the next refresh fails silently because `error` is only shown when nothing has loaded
Network lost on a new screen	Loading spinner, then the error message
App restarted offline	Session restore fails and the user sees Login (the cookie is kept, so sign-in works once online again after restart)
Alarm action offline	Fails with the network error; nothing is queued
Checklist toggle offline	Fails silently (`runCatching`), the list reloads and shows the real state
Scan offline	Capture and OCR work fully offline; upload fails and the pages stay in the review until the user retries or discards, but are lost if the app process ends
Voice offline	Phone recognition may still work (device dependent); the answer fails with an error bubble

6.14.2 Planned

- **Room cache** of assigned inspections with step snapshots, tasks, sensor last values and the device context, with "last updated" shown on every cached screen.
- **Outbox** for inspection responses (with `clientOpId`, made idempotent by the server's unique index), checklist toggles, alarm acknowledgements, notes and scans (pages written to app-private storage until accepted).
- **WorkManager** replay with exponential backoff when connectivity returns. Conflicts (`409`, invalid transition) shown as "updated by someone else"; local edits never silently dropped. Offline alarm acknowledgements shown as "pending" until confirmed.
- The glasses web viewport already implements the inspection response queue (`apps/web/src/glasses/link.ts`, `flush()`), which is the reference behaviour for the native outbox.

6.15 Security

Table 180. Security controls

Control	Status	Detail
Access tokens in memory only	Built	`HydrisApi.accessToken`, never persisted

Control	Status	Detail
Refresh cookies and device tokens encrypted at rest	Built	`SecureStore`: AES-256-GCM, non-exportable Android Keystore key alias `hydris.secure.v1`, random 12-byte IV stored before the ciphertext, 128-bit tag; only ciphertext in SharedPreferences `hydris.secure`
Undecryptable values treated as absent	Built	`get()` returns null if decryption fails (for example after a key reset), which forces a new sign-in
Backups disabled	Built	`android:allowBackup="false"`
HTTPS only	Built by default	No `usesCleartextTraffic`; the default for target SDK 28 and later blocks cleartext
Cookies scoped to the Hydris host	Built	Cookie jar ignores other hosts
Raw glasses identifier not sent	Built	SHA-256 derived `rbm-` hardware ID
Images kept in memory, never written to the gallery	Built	Pages are bitmaps in the Scan screen state
Sign out revokes the server session	Built	`POST /auth/logout`, then local cookies cleared
`FLAG_SECURE` on scan review	Planned	Prevents screenshots and recents thumbnails of documents
Certificate pinning with a backup pin	Planned	For production hosts
Release signing key	Planned	Release builds are currently signed with the debug key
Biometric re-entry after long inactivity	Planned	
Managed configuration (MDM) for server URL	Planned	`app_restrictions.xml`
Device integrity signals	Planned	Informational only for pilots

CAUTION

Until the release signing configuration is replaced, any build produced with `assembleRelease` is signed with a debug key and must not be distributed outside the team.

The Hydris-wide threat model, data classification and retention are in the Security and Privacy document (HFG-SEC).

6.16 Build and release

6.16.1 Project setup

Table 181. Gradle setup

Item	Value
Root project	`hydris-field`, one module `:app`
Namespace and application ID	`ai.hydris.field`
Version	`versionName 1.0.0`, `versionCode 1`
Repositories	Google (Android and Google groups only), Maven Central, Gradle Plugin Portal; `FAIL_ON_PROJECT_REPOS`
Gradle JVM	`-Xmx4g`

Item	Value
Java and Kotlin target	17
Build features	Compose, BuildConfig
Packaging	Excludes `META-INF/AL2.0` and `LGPL2.1`
Release build	R8 minify on, `proguard-android-optimize.txt` plus `proguard-rules.pro` (kotlinx.serialization keep rules; DAT and ML Kit ship consumer rules)

Table 182. Build-time configuration in local.properties (not committed)

Key	Default	Purpose
`sdk.dir`	none	Android SDK path
`hydris_url`	`https://hydrisview.vercel.app`	API base URL compiled into `BuildConfig.HYDRIS_URL`
`mwdat_application_id`	`0`	DAT application ID manifest placeholder
`mwdat_client_token`	`0`	DAT client token manifest placeholder

```
cd apps/android
./gradlew assembleDebug
adb install app/build/outputs/apk/debug/app-debug.apk
```

NOTE

DAT artifacts (`com.meta.wearable:mwdat-*`) are not on Maven Central or Google Maven. The bridge README and Meta's distribution use a GitHub Packages Maven repository that needs a token with `read:packages`. The `settings.gradle.kts` in `apps/android` lists only Google and Maven Central, so the build relies on that repository being configured in the developer's Gradle user settings or on a local Maven cache. Adding the repository declaration explicitly is Planned.

6.16.2 DAT credentials and Developer Mode

With the placeholders set to `0`, the app runs against glasses only when **Developer Mode** is enabled in the Meta AI app. DAT logs that it read no credentials; that is expected in Developer Mode. For production, the Wearables Developer Center issues an application ID and client token, which go into `local.properties` and the manifest placeholders.

6.16.3 Release channels

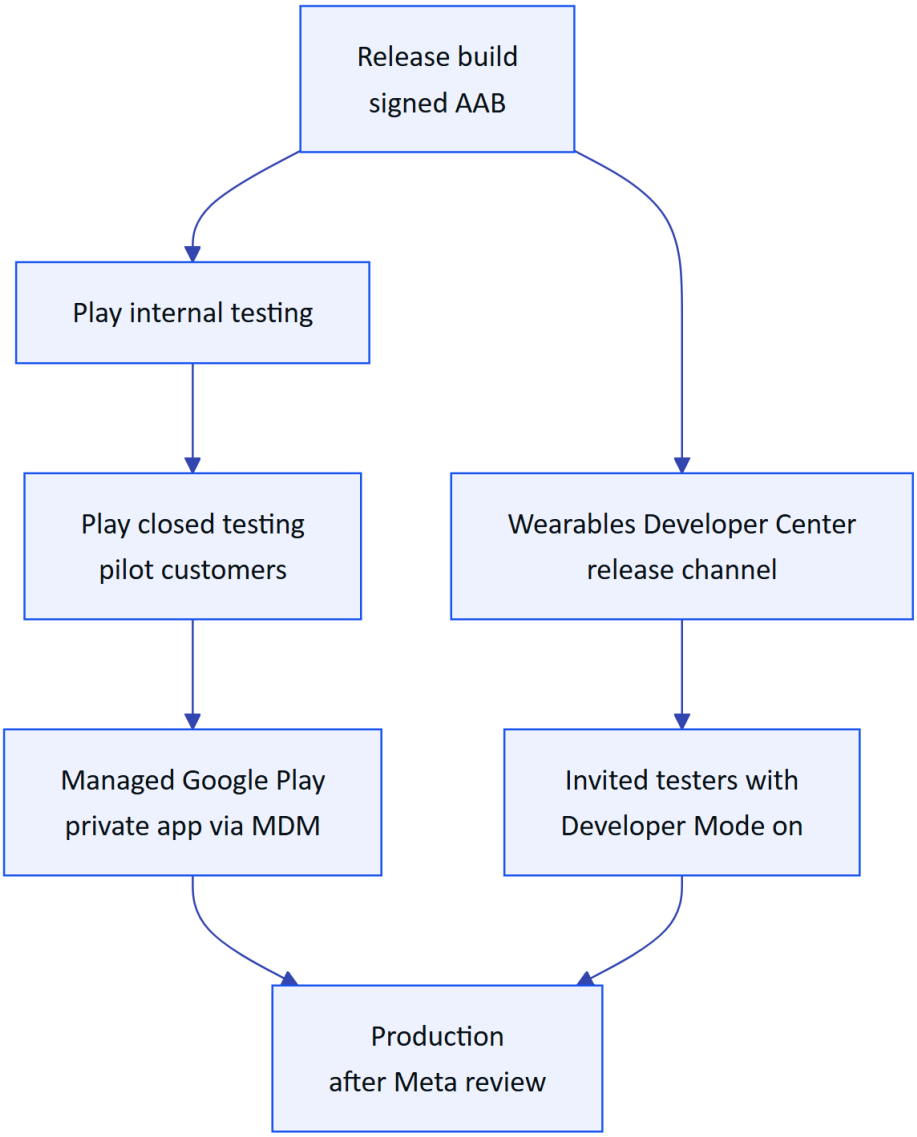

Figure 138. *Release path from build to operators*

Table 183. Distribution status

Channel	What it covers	Status
Direct APK (sideload)	Team devices and demos	In use
Google Play internal and closed testing	Phone features for pilot users	Planned (needs release signing and the Play data safety form)
Managed Google Play (private app)	Enterprise rollout through the customer's MDM	Planned
Wearables Developer Center release channels	Invite-only testing of the glasses features without publishing review	Planned (needs a Developer Center organization and app)
Public listing with DAT features	General availability	Blocked: Meta's FAQ says submissions are not open yet and will open "soon"

> **RISK**
> Full Wearables Developer Center access is limited to countries where Meta AI glasses are supported. Eligibility for Hydris's Indian pilots must be confirmed with Meta before planning a production glasses rollout. Phone features are not affected.

The Play data safety declaration must cover: camera images (processed on device, uploaded when the user sends a scan), audio (recognized on device or by the system recognizer; transcripts sent to Hydris), account information, and crash diagnostics once added.

6.17 Testing

6.17.1 Current state

There are no automated tests in `apps/android` yet (no `src/test` or `src/androidTest`). The scanning algorithm is shared with the web, where the same thresholds are exercised by the web test suite and the simulated glasses camera in `apps/web/src/glasses/camera.ts`. The bridge app README states that the page side was tested end to end with a stand-in bridge, and that DAT calls are type-checked but not yet run on physical glasses.

6.17.2 Test plan

Table 184. Planned test layers

Layer	Scope	Tooling
Unit	`DocDetect` (guidance, warp corners, parity with web fixtures), `OcrEngine.prepare` sizing, `labelValues`, wake and stop regexes, cookie jar expiry	JUnit, Robolectric for `Bitmap`
Networking	Refresh on 401, mutex ordering, session lost, device token refresh, SSE parsing and Last-Event-ID	OkHttp MockWebServer, Turbine
UI	Login states, alarm actions per state, scan stages, glasses steps	Compose UI tests
Screenshots	Every screen at font scale 1.0 and 1.3	Roborazzi or Paparazzi
Glasses without hardware	Registration, permissions, camera stream, photo, capture button	DAT Mock Device Kit (`mwdat-mockdevice`, already a debug dependency)
Hardware	Ray-Ban Meta Gen 2 and Oakley Meta, phone matrix	Manual, per the QA test plan (HFG-QA)
End to end	Demo deployment, scripted round	Manual for pilot, automated later

> **NOTE**
> Meta's FAQ states that the Mock Device Kit does not support display glasses. That does not affect Hydris Field, which targets camera glasses.

6.18 Feature status and roadmap

6.18.1 Status summary

Table 185. Feature status

Feature	Status
Sign in, demo mode, session restore, sign out	Built
MFA, org switch, server picker	Planned
Home dashboard	Built
Alarms list, detail and lifecycle actions	Built
Alarm notes, history, SOP links	Planned
Inspections list	Built (read-only)
Running inspections on the phone	Planned
Maintenance list and checklist toggles	Built
Maintenance start, hold, complete	Planned
Sensors list and history chart	Built
Audit scan: phone camera, glasses camera, import, guided capture, two-pass OCR, upload	Built
Audit document list	Built
Audit review and approval in the app	Planned (console only)
Hydris assistant: text, Hey Hydris on phone and glasses, spoken answers	Built
Voice Invocations ("Hey Meta, start Hydris") in the native app	Planned (implemented in the bridge app only)
Glasses connect, test, register, unregister	Built, not verified on hardware
Realtime alarm notifications while open	Built
Push notifications when closed	Planned
Offline cache and outbox	Planned
Automated tests	Planned
Release signing and Play distribution	Planned

Figure 139. *Mobile roadmap (indicative quarters)*

6.18.2 Open questions

- Should the phone app become the only companion for display glasses as well, replacing `apps/glasses-bridge-android`, or should the WebView bridge remain for Meta Ray-Ban Display web apps? (See the Glasses Integration document.)
- Which customer MDMs must be supported at pilot (Intune, Workspace ONE, Google endpoint management)?
- Is MFA mandatory for field operators at the first pilot site? If so, in-app MFA moves to the top of the plan.

7 Glasses Integration

> **NOTE**
>
> **HFG-GLS v2.0** · Draft for review · Owner: Engineering, Hydris. How Hydris connects to Ray-Ban Meta and other smart glasses through Meta's Device Access Toolkit, the WebView bridge and the glasses web viewport

Hydris puts live plant data, guided paper capture and a voice assistant in front of field operators who need their hands free. This document explains how Hydris talks to smart glasses: which glasses, which capabilities, which software stack, and what is built, partial or planned. It covers three runtimes that share one backend and one account model:

- **Hydris Field** (`apps/android`), the native Android app, which uses Meta's Wearables Device Access Toolkit (DAT) directly.
- **Hydris Glasses bridge** (`apps/glasses-bridge-android`), a WebView host that loads the glasses web app and exposes DAT to it through a JavaScript bridge.
- **The glasses web viewport** (`apps/web/src/glasses/*`, served at `/glasses`), which runs as a heads-up display (HUD) in the browser simulator and as a full-screen phone layout inside the bridge.

4
DAT modules used
core, camera, speech, inputs (1.0.1)

0
Displays on Ray-Ban Meta
the phone is the screen

3
Camera bridge shapes
JS interface, injected API, postMessage

300 s
Pairing code lifetime
`PAIRING_TTL_S`

> **IMPORTANT**
>
> No Hydris build has yet been run against physical Meta glasses. All DAT calls compile and are type-checked against DAT 1.0.1, and the page side of the bridge was tested end to end with a stand-in bridge (`apps/glasses-bridge-android/README.md`). Every hardware behaviour in this document is therefore **Built, not hardware-verified** unless stated otherwise. The first hardware run is the top item on the roadmap.

7.1 Scope and status

Table 186. Integration status at a glance

Capability	Native app	Bridge app	Web viewport alone
Register with the Meta AI app	Built	Built	Not possible
Glasses camera preview for guided A4 capture	Built	Built	Not possible
Full-resolution glasses photo	Built	Built	Not possible
Capture button	Built	Built	Keyboard and simulator only
Glasses microphone ("Hey Hydris")	Built	Built	Browser microphone

Capability	Native app	Bridge app	Web viewport alone
Spoken answers on the glasses speakers	Built (phone TTS over Bluetooth)	Built	Browser speech synthesis
Voice Invocations ("Hey Meta, start Hydris")	Planned	Built	Not possible
Register glasses as a Hydris device	Built (self-approved)	Built (code and QR via the viewport)	Built (code and QR)
HUD on glasses with a display	Not applicable	Not applicable	Built in the simulator; on-glasses Planned

7.1.1 Terminology

Table 187. Terms used in this document

Term	Meaning
DAT	Meta Wearables Device Access Toolkit: the Android and iOS SDK that lets phone apps use supported Meta glasses
Meta AI app	Meta's phone app (Android package `com.facebook.stella`) that pairs the glasses, updates firmware and approves third-party apps
Registration	DAT's approval that Hydris may connect to the glasses, confirmed in the Meta AI app
Session	A DAT `DeviceSession` with one device; camera, speech and inputs are capabilities added to it
Pairing	Hydris's own device authorization flow that turns the glasses into a Hydris device record
HUD	The 600 x 600 display layout of the web viewport
Phone mode	The full-screen phone layout of the web viewport used inside the bridge

7.2 Hardware and capabilities

7.2.1 Supported glasses

Meta's DAT FAQ lists Ray-Ban Meta (Gen 1 and Gen 2), Meta Ray-Ban Display, Oakley Meta HSTN and Oakley Meta Vanguard as supported, and says capability availability varies by device. Meta's setup guide additionally lists Ray-Ban Meta Optics and "Meta glasses". Hydris's primary target is Ray-Ban Meta Gen 2.

Table 188. Capability matrix by glasses family

Capability	Ray-Ban Meta (Gen 1, Gen 2)	Oakley Meta (HSTN, Vanguard)	Meta Ray-Ban Display	Browser simulator
Supported by DAT	Yes (Meta FAQ)	Yes (Meta FAQ)	Yes (Meta FAQ)	Not applicable
Camera stream and photo to the phone app	Yes via DAT	Yes via DAT, unverified by Hydris	Yes via DAT, unverified by Hydris	Simulated camera
Microphone and speech	Yes via DAT Speech	Unverified by Hydris	Unverified by Hydris	Web Speech API
Open-ear speakers	Phone TTS over Bluetooth	Phone TTS over Bluetooth	Phone TTS over Bluetooth	`speechSynthesis`
Capture button input	DAT Inputs	Unverified by Hydris	Unverified by Hydris	Keyboard, `hydris:input`

Capability	Ray-Ban Meta (Gen 1, Gen 2)	Oakley Meta (HSTN, Vanguard)	Meta Ray-Ban Display	Browser simulator
Display	**None**	**None**	Yes	600 x 600 HUD
Hydris UI	On the phone	On the phone	On the phone today; on-glasses Planned	HUD
Hydris status	Primary target, not yet run on hardware	Expected to work, untested	Phone path expected to work; display Planned	Built

> **NOTE**
> Vendor hardware figures (camera megapixels, battery life, microphone count) are deliberately not quoted here. Take them from Meta's current specification sheets when talking to customers.

7.2.2 Display glasses from other vendors

The console's demo data (`apps/api/src/seed/showcase.ts`) contains device records for **Vuzix M400** and **RealWear Navigator 520** units, plus a browser simulator. These records show how a fleet of display glasses appears in the console (status, firmware, app version, assignment). Hydris has no native app for Vuzix or RealWear yet. The web viewport is designed for such devices: a 600 x 600 additive display, large type and directional input. The simulator offers 600 x 600, 640 x 480 and 480 x 480 viewports (`apps/web/src/app/simulator/page.tsx`).

Table 189. Other display glasses

Device	Where it appears	Status
Vuzix M400	Demo device records	Planned: run `/glasses` in the device browser or a thin Android host; not tested
RealWear Navigator 520	Demo device records	Planned: as above, with voice-first navigation; not tested
Browser simulator	`/simulator`, device kind `simulator`	Built

7.3 Runtimes and architecture

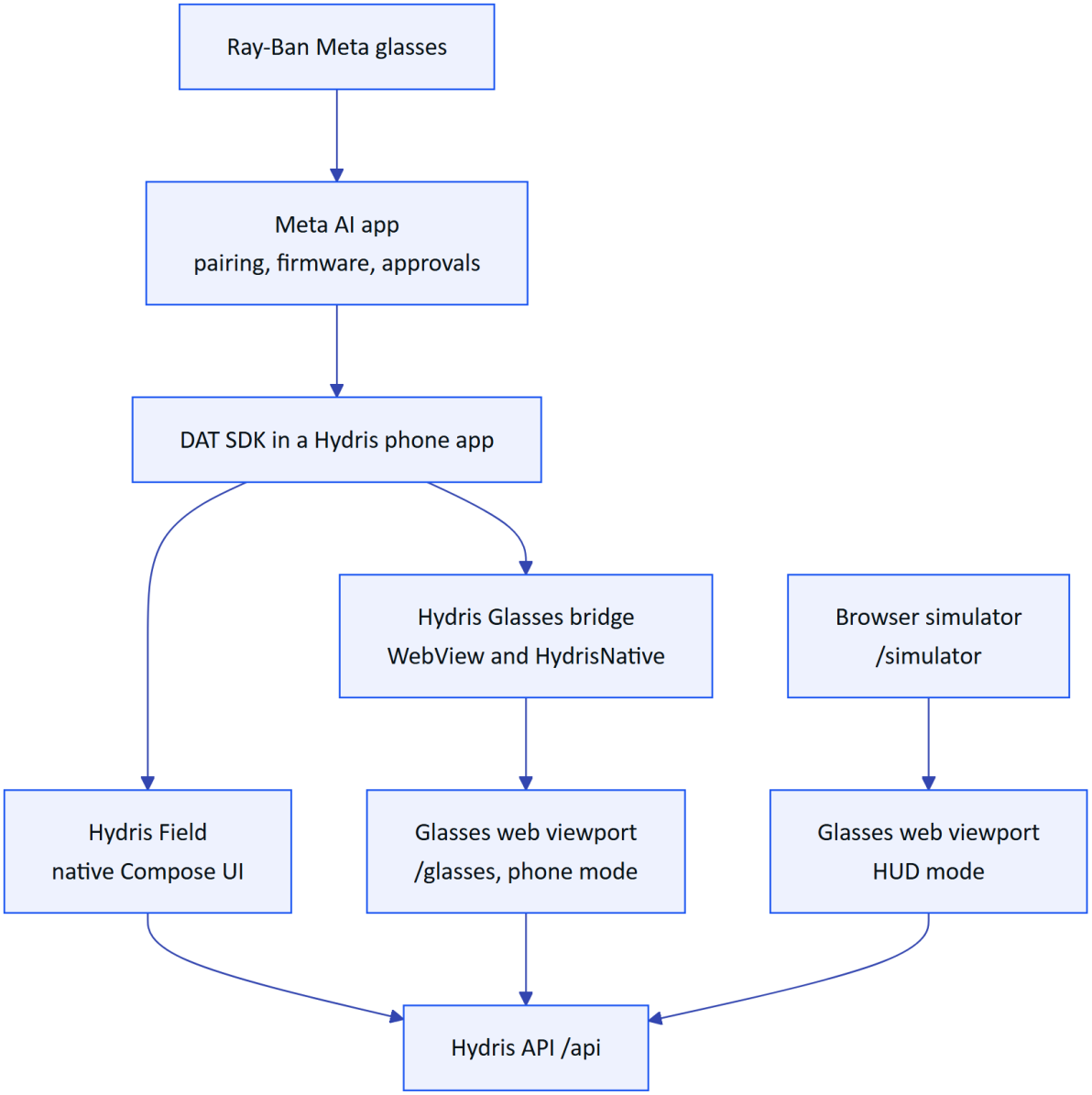

Figure 140. *The three glasses runtimes and the shared Hydris backend*

7.3.1 Meta's stack

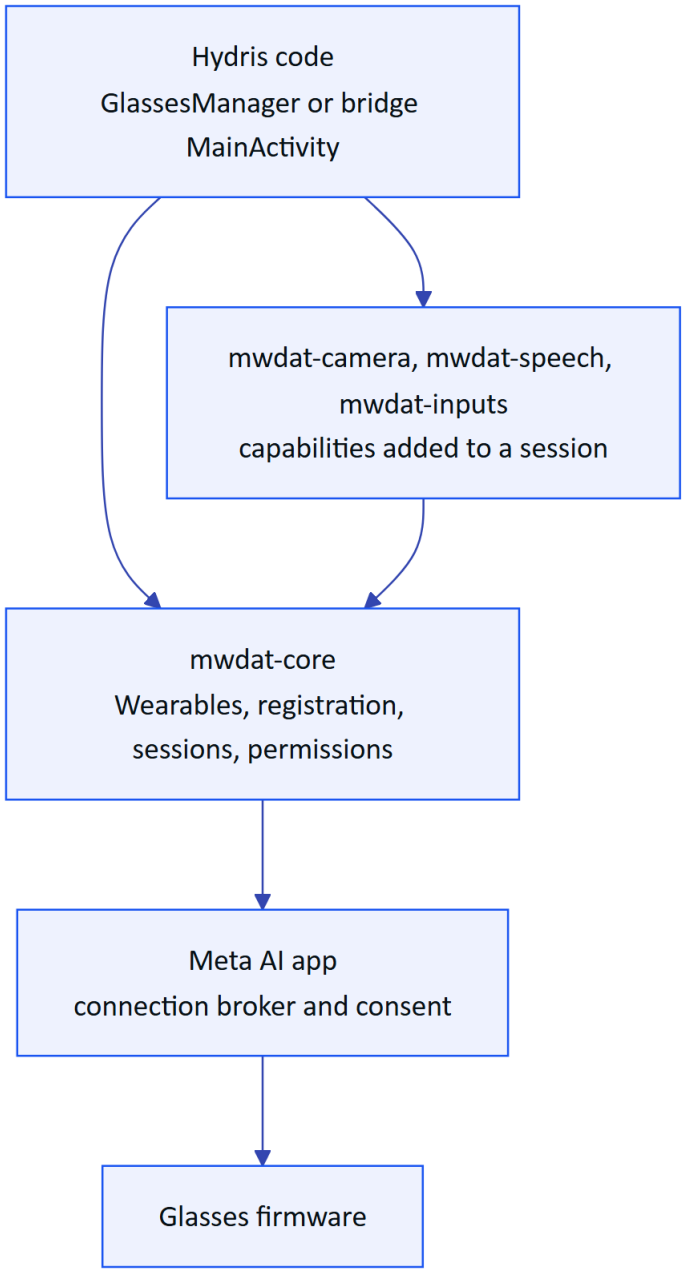

Figure 141. *Layers between Hydris code and the glasses*

Table 190. DAT artifacts used (com.meta.wearable, version 1.0.1)

Artifact	What Hydris uses
`mwdat-core`	`Wearables.initialize`, `registrationState`, `registrationErrorStream`, `startRegistration`, `startUnregistration`, `devices`, `devicesMetadata`, `checkPermissionStatus`, `RequestPermissionContract`, `createSession(AutoDeviceSelector())`, `isVoiceInvocationsIntent`
`mwdat-camera`	`session.addCamera(StreamConfiguration(VideoQuality.MEDIUM, 24))`, `stream.videoStream`, `stream.start`, `stream.state`, `stream.capturePhoto`, `PhotoData.Bitmap` and `PhotoData.HEIC`
`mwdat-speech`	`session.addSpeech()`, `transcriptions`, `state`, `errors`, `start`, `stop`
`mwdat-inputs`	`session.addInputs(InputsConfiguration(CAPTURE_BUTTON))`, `InputEvent.Capture` with `CapturePressType.SHORT_PRESS`

Artifact	What Hydris uses
`mwdat-mockdevice`	Debug dependency only; not yet used by tests

The bridge README notes that Speech, Inputs and Voice Invocations are experimental in DAT and must be enabled for the app in the Wearables Developer Center. Hydris treats them as optional at runtime: failures are logged and the phone fallbacks remain.

7.4 The Meta AI app dependency

Meta's documentation is explicit: "the Meta AI app must be used to pair your glasses". Developer Mode, which lets an unpublished app connect, is enabled in the Meta AI app (Settings, App Info, tap the app version five times, then turn on Developer Mode); connected apps then appear under Meta AI settings, App connections, Developer mode apps. Firmware versions must meet DAT's version dependency table, and firmware is updated through the Meta AI app.

> **DECISION**
> Hydris does not attempt to replace the Meta AI app. It must stay installed and signed in. Hydris checks for it (`GlassesManager.metaAiInstalled()` looks up `com.facebook.stella`, declared in the manifest `<queries>`) and tells the user to install it and pair the glasses there first.

Developer Mode builds use the manifest placeholders `mwdat_application_id = "0"` and `mwdat_client_token = "0"` (both apps' `build.gradle.kts`). DAT logs that it read no credentials, which is expected. Production builds put the values issued by the Wearables Developer Center in `local.properties`.

7.5 DAT lifecycle

The native `GlassesManager` and the bridge's `MainActivity` implement the same lifecycle. Differences are noted where they matter.

7.5.1 Initialization

DAT is initialized only after Android grants the Bluetooth permissions. The native app requests `BLUETOOTH`, `BLUETOOTH_CONNECT` and `POST_NOTIFICATIONS` on every `onStart` and initializes when `BLUETOOTH_CONNECT` is not denied. The bridge requests `BLUETOOTH`, `BLUETOOTH_CONNECT` and `INTERNET` and initializes only if all are granted.

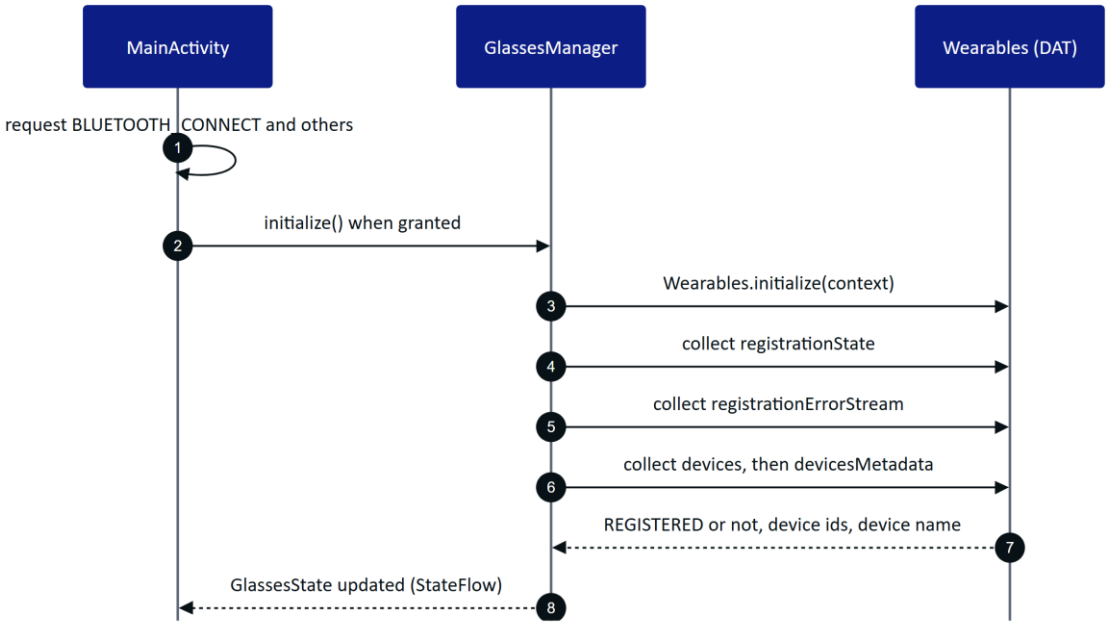

Figure 142. *DAT initialization in the native app*

The device name comes from `devicesMetadata` and falls back to "Ray-Ban Meta" when empty. The state holds the registration state, the device list, the session state, camera streaming, speech state, the last error and a log of the latest 30 messages.

7.5.2 Registration through the Meta AI app

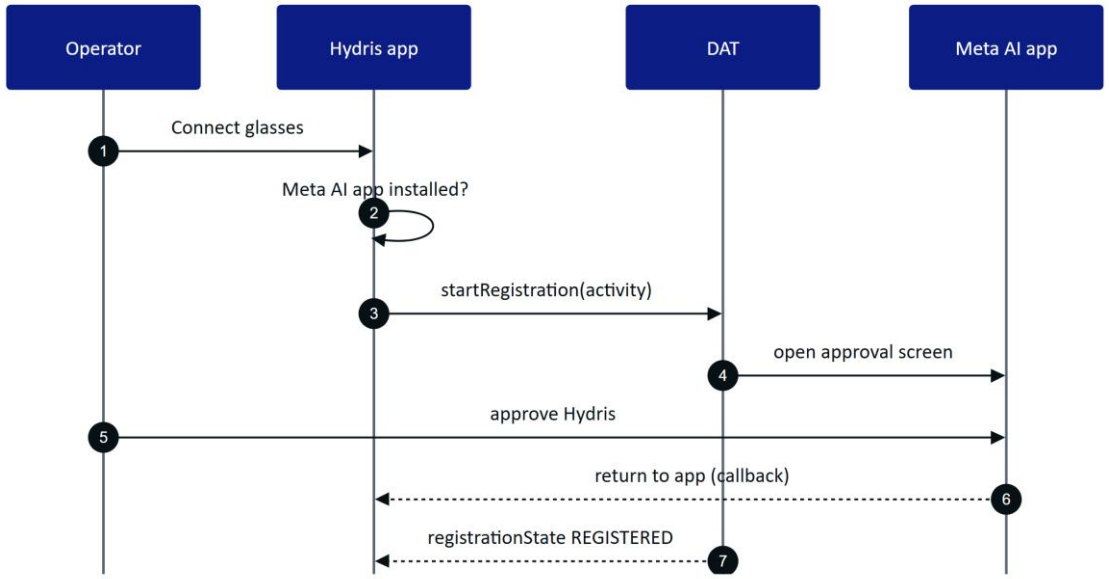

Figure 143. *Registration hands off to the Meta AI app and returns*

Each app declares a callback URL scheme in its manifest for the return from the Meta AI app: `hydrisfield` for the native app and `hydrisglasses` for the bridge. The native app starts registration only from the Glasses screen. The bridge also starts it automatically when the page needs a session and the app is not registered, and exposes `HydrisNative.connectGlasses()`. Unregistration uses `startUnregistration` (native "Disconnect").

7.5.3 Permissions

Camera and microphone access on the glasses are DAT permissions, separate from Android permissions, and are approved in the Meta AI app.

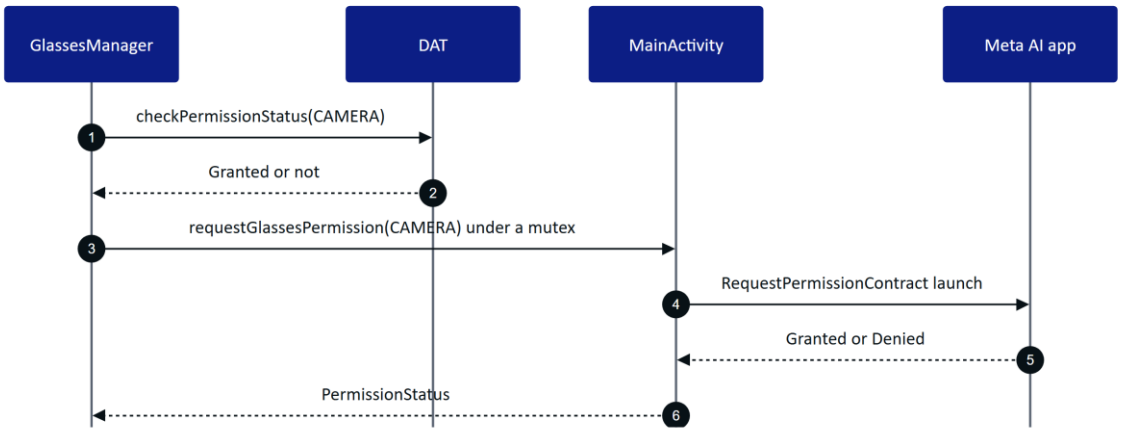

Figure 144. *Requesting a glasses permission*

The mutex ensures only one prompt is open at a time. A cancelled or failed result is treated as `Denied`.

7.5.4 Session

One `DeviceSession` is shared by camera, speech and inputs. It is created lazily with `createSession(AutoDeviceSelector())` the first time a capability is needed, started, and reused while its state is `STARTED`. A `CompletableDeferred` lets concurrent callers wait for the same session.

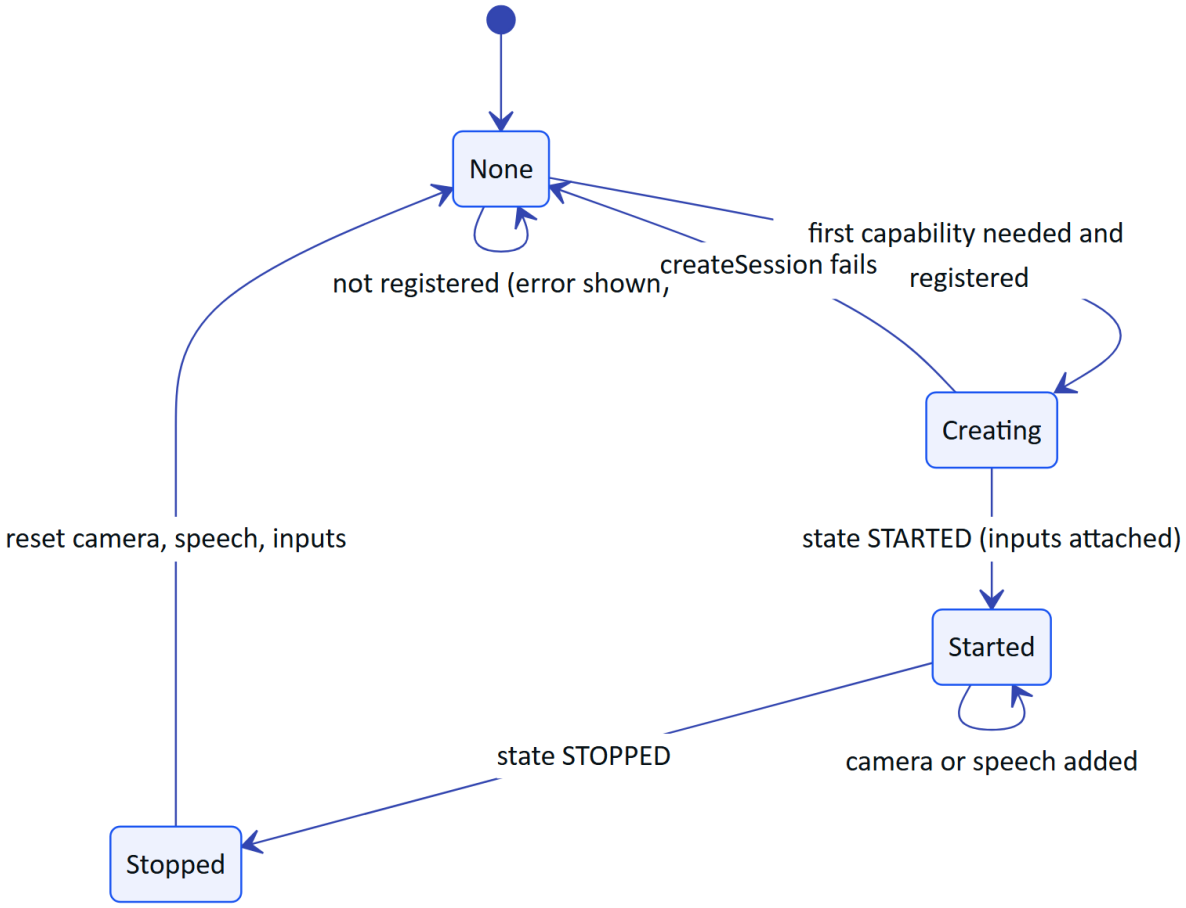

Figure 145. *Device session lifecycle*

On `STOPPED` both apps drop the camera, speech and inputs objects so the next request creates a fresh session. Session errors are logged ("Session: ...").

7.5.5 Camera stream for guided capture

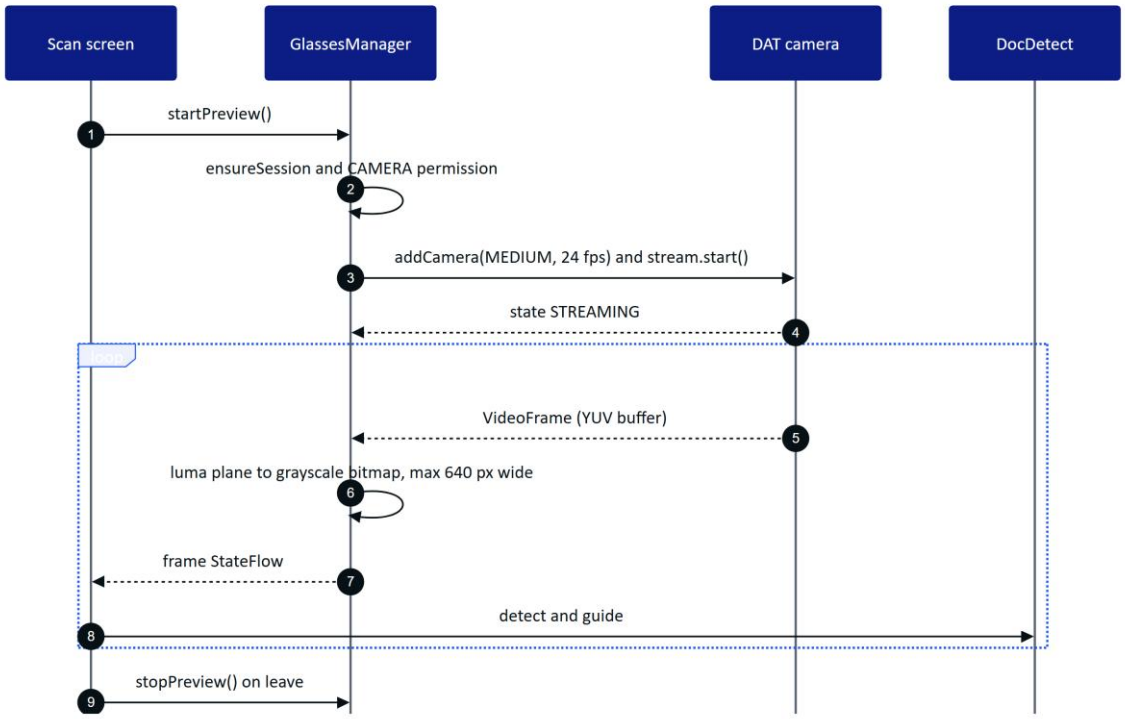

Figure 146. *Glasses preview frames feeding page detection*

DAT documents raw frames as YUV pixel data without fixing the chroma layout. Every planar or semi-planar 4:2:0 layout starts with the full-resolution luma plane, so Hydris reads luma only. The row stride is inferred from the buffer size (total bytes over height times 1.5), and frames are subsampled to at most 640 px wide. Compressed frames are skipped. The native app forwards at most one frame per 100 ms; the bridge encodes one JPEG (quality 70) per 120 ms and calls `window.__hydrisCameraFrame`.

> **RISK**
>
> Two parts of this path are unverified on hardware: stride inference, and the battery cost of the stream. `stopPreview()` stops forwarding frames but does not stop the DAT stream, which ends only with the session. Stopping the stream explicitly is Planned and must be checked against the capture LED behaviour.

7.5.6 Photo capture

`capturePhoto()` works only on an active stream, so both apps wait until the stream reports `STREAMING` (or stops) before capturing.

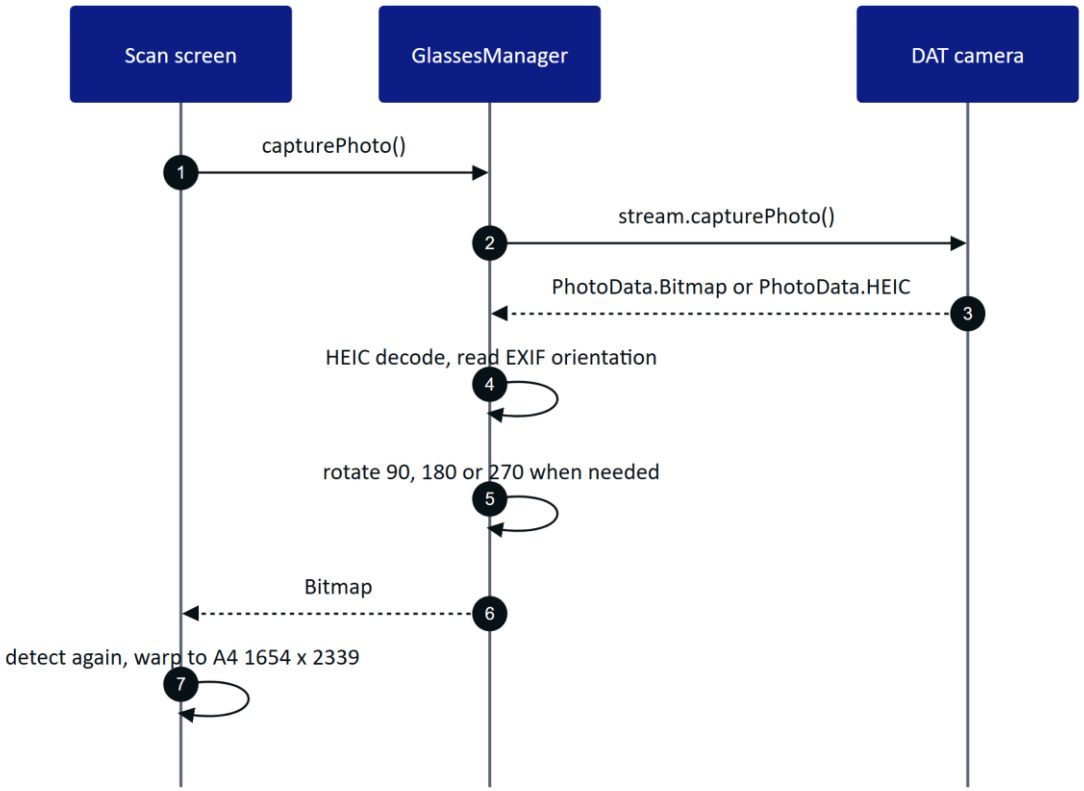

Figure 147. *Full-resolution photo from the glasses*

The bridge also handles EXIF flips and transposes, then encodes the photo as JPEG at quality 92 and calls `window.__hydrisCameraPhoto(requestId, base64)`. The native app handles rotations only; adding flips is Planned.

7.5.7 Capture button

When a session starts, both apps add the Inputs capability with `sources = {CAPTURE_BUTTON}` and `consumeBack = false`. Only `InputEvent.Capture` with `SHORT_PRESS` is used.

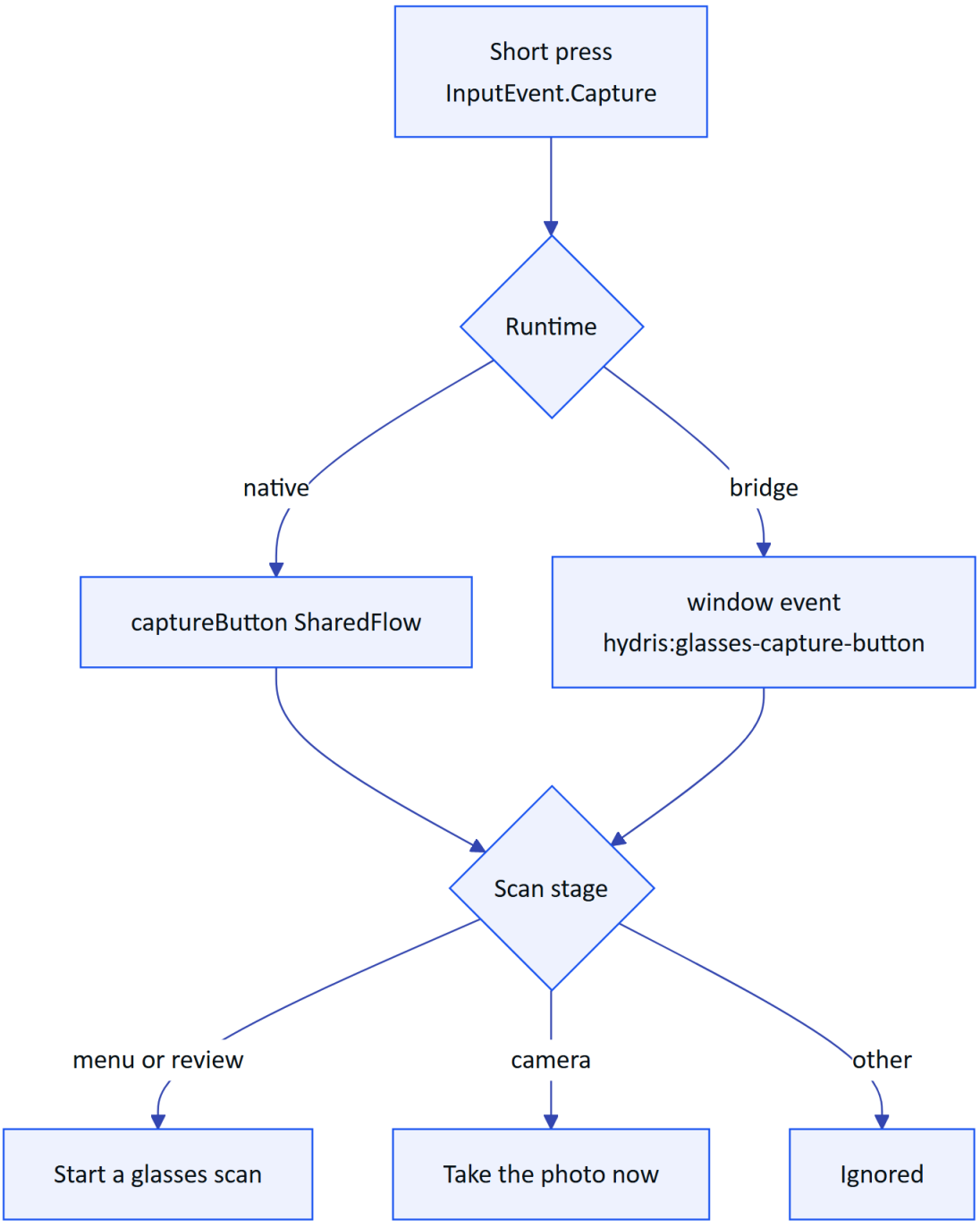

Figure 148. *What a short press of the capture button does*

The capture press decomposition (which presses DAT reports as short) is unverified on hardware.

7.5.8 Speech

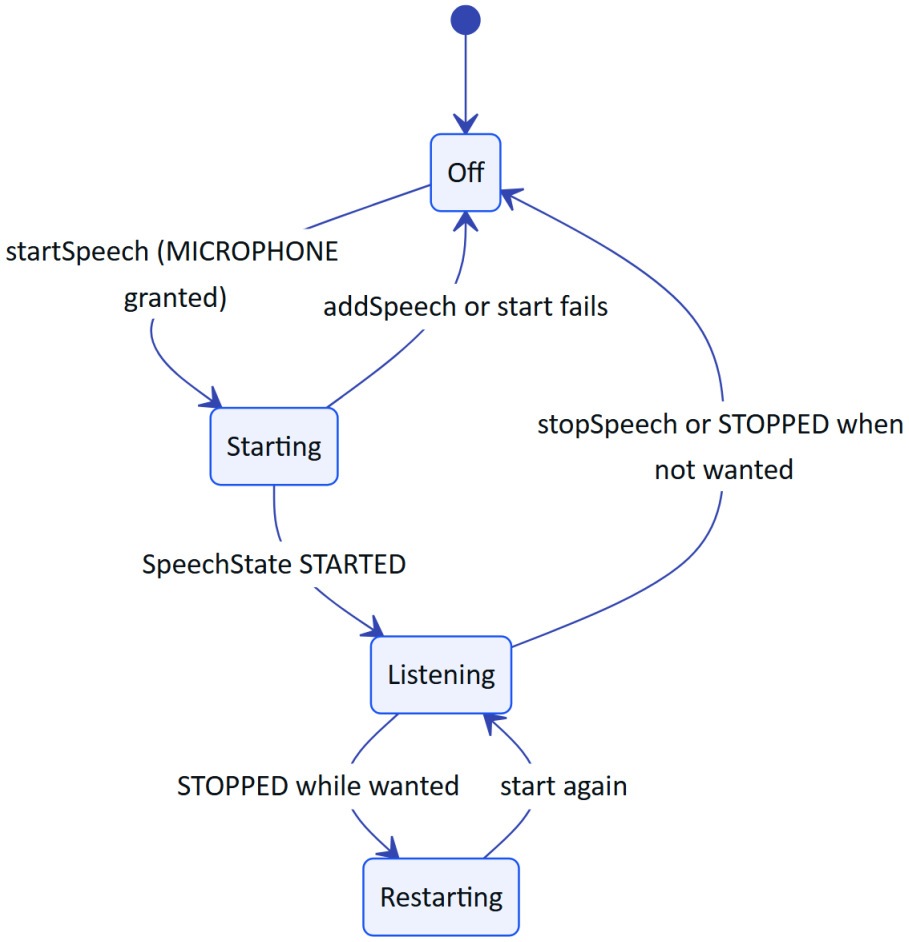

Figure 149. *Always-on glasses speech with restart after silence*

Transcriptions arrive as text with an `isFinal` flag. The native `VoiceEngine` drops them while text-to-speech is playing. The bridge forwards them with `window.__hydrisSpeech(text, isFinal)` and reports states with `window.__hydrisSpeechState(state, detail)`. Answers are spoken by the phone's `TextToSpeech`; Android plays them on the glasses when they are the Bluetooth audio output.

7.5.9 Voice Invocations

The bridge checks every launch and new intent with `isVoiceInvocationsIntent(intent)` and, when it matches, dispatches `hydris:voice-activate` to the page, which opens the assistant. The native app does not handle Voice Invocations yet (Planned).

7.5.10 Errors

Table 191. Error sources and handling

Source	Example message	Handling
Registration	"Meta AI: ..."	Shown on the Glasses screen
Missing Meta AI app	"Install the Meta AI app and pair your glasses in it first"	Registration not started
Not registered	"Connect Hydris in the Meta AI app first"	No session created

Source	Example message	Handling
Session	"No glasses session: ..."	Deferred fails, next call retries
Camera permission	"Camera permission denied in Meta AI"	Preview and capture fail; phone camera still available
Camera stream	"Stream failed: ..."	Logged; scan shows the error
Microphone permission	"Microphone permission denied in Meta AI"	Phone microphone used instead
Inputs	"Capture button unavailable: ..."	Logged only

7.6 Web bridge contract

The bridge app (`ai.hydris.glassesbridge`, version 0.2.0, minSdk 31) loads `${HYDRIS_URL}/glasses` in a WebView with JavaScript and DOM storage enabled and adds a `HydrisNative` JavaScript interface. The page detects the bridge and switches to phone mode.

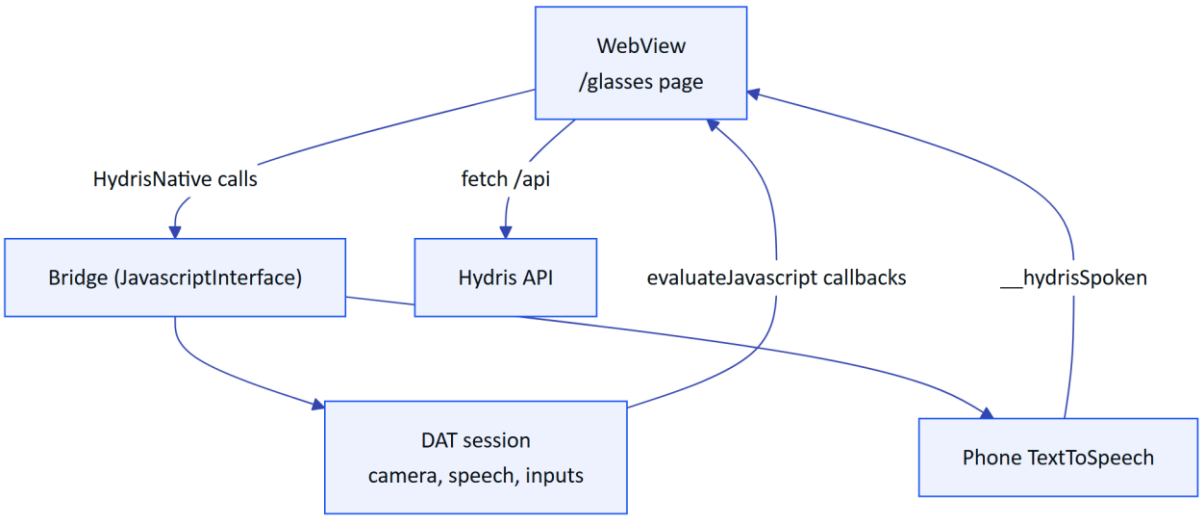

Figure 150. *Bridge app structure*

7.6.1 Page to host

Table 192. window.HydrisNative methods

Method	Effect
`startPreview()`	Turn preview forwarding on and ensure the camera stream
`stopPreview()`	Stop forwarding preview frames
`capturePhoto(requestId)`	Take a full-resolution photo and answer with the same ID
`deviceName()`	Returns "Ray-Ban Meta" (constant today)
`startSpeech()`	Start always-on glasses speech
`stopSpeech()`	Stop speech
`speak(text)`	Speak with phone TTS
`connectGlasses()`	Start DAT registration
`status()`	DAT registration state as text

7.6.2 Host to page

Table 193. Callbacks and events the host calls

Callback or event	Arguments	Meaning
`window.__hydrisCameraFrame`	JPEG base64, width, height	Preview frame (about 8 fps)
`window.__hydrisCameraPhoto`	request ID, JPEG base64	Photo result
`window.__hydrisCameraError`	request ID, message	Photo failed
`window.__hydrisSpeech`	text, isFinal	Transcript
`window.__hydrisSpeechState`	state, detail	`listening`, `idle` or `error`
`window.__hydrisSpoken`	none	TTS finished
`hydris:glasses-capture-button`	CustomEvent	Capture button short press
`hydris:voice-activate`	CustomEvent, optional `question`	Open the assistant

The page rejects a photo request after 20 s ("The glasses did not return a photo"). For speech output it resolves after `__hydrisSpoken` or a timeout of 1.5 s plus 70 ms per character (at most 20 s).

7.6.3 Other host shapes

`apps/web/src/glasses/camera.ts` also supports an injected promise API for iOS WKWebView hosts, `window.HydrisWearables` (`capturePhoto(): Promise<{dataUrl}>`, optional `startPreview(onFrame)`, `stopPreview()`, `deviceName`), and a postMessage protocol for any embedding host (React Native WebView or a parent frame).

Table 194. postMessage protocol

Direction	Message	Fields
Host to page	`hydris:camera-ready`	`deviceName` (announces a bridged camera)
Page to host	`hydris:camera-start`	none
Host to page	`hydris:camera-frame`	`dataUrl`
Page to host	`hydris:camera-capture`	`requestId`
Host to page	`hydris:camera-photo`	`requestId`, `dataUrl`
Host to page	`hydris:camera-error`	`requestId`, `message`
Page to host	`hydris:camera-stop`	none
Simulator to page	`hydris:input`	`key`: up, down, left, right, select, back (same origin only)
Simulator to page	`hydris:network`	`online` (simulated loss)
Console to page	`hydris:profile`	`config` (demo preview only)
Page to console	`hydris:preview-ready`	none
Page to simulator	`hydris:event`	`event`, `detail`, `at` (event log)

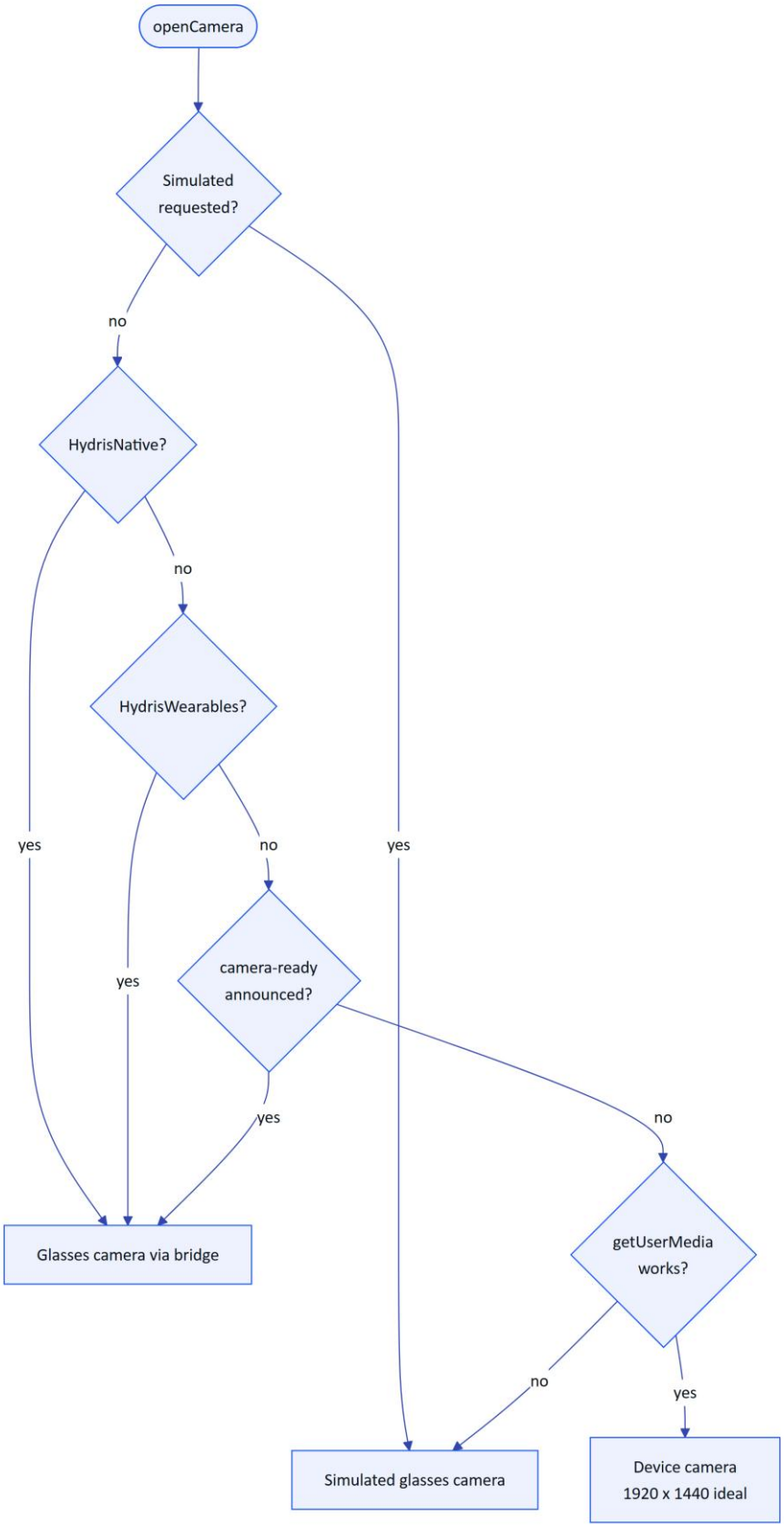

Figure 151. *Camera source selection in openCamera()*

The simulated camera draws the sample A4 audit sheet moving through too close, too far, tilted and finally square, so every guidance state is exercised without hardware. Its stills are 2016 x 1512.

> **WARNING**
> Bridge hardening is Planned: the WebView grants every page permission request (`onPermissionRequest` grants all resources) and does not restrict navigation away from `HYDRIS_URL`. The JavaScript interface would be exposed to any page loaded. Restrict navigation to the Hydris origin before distributing the bridge outside the team.

7.7 Glasses web viewport

7.7.1 HUD and phone modes

`useViewportLayout` in `apps/web/src/glasses/hud.tsx` picks the layout.

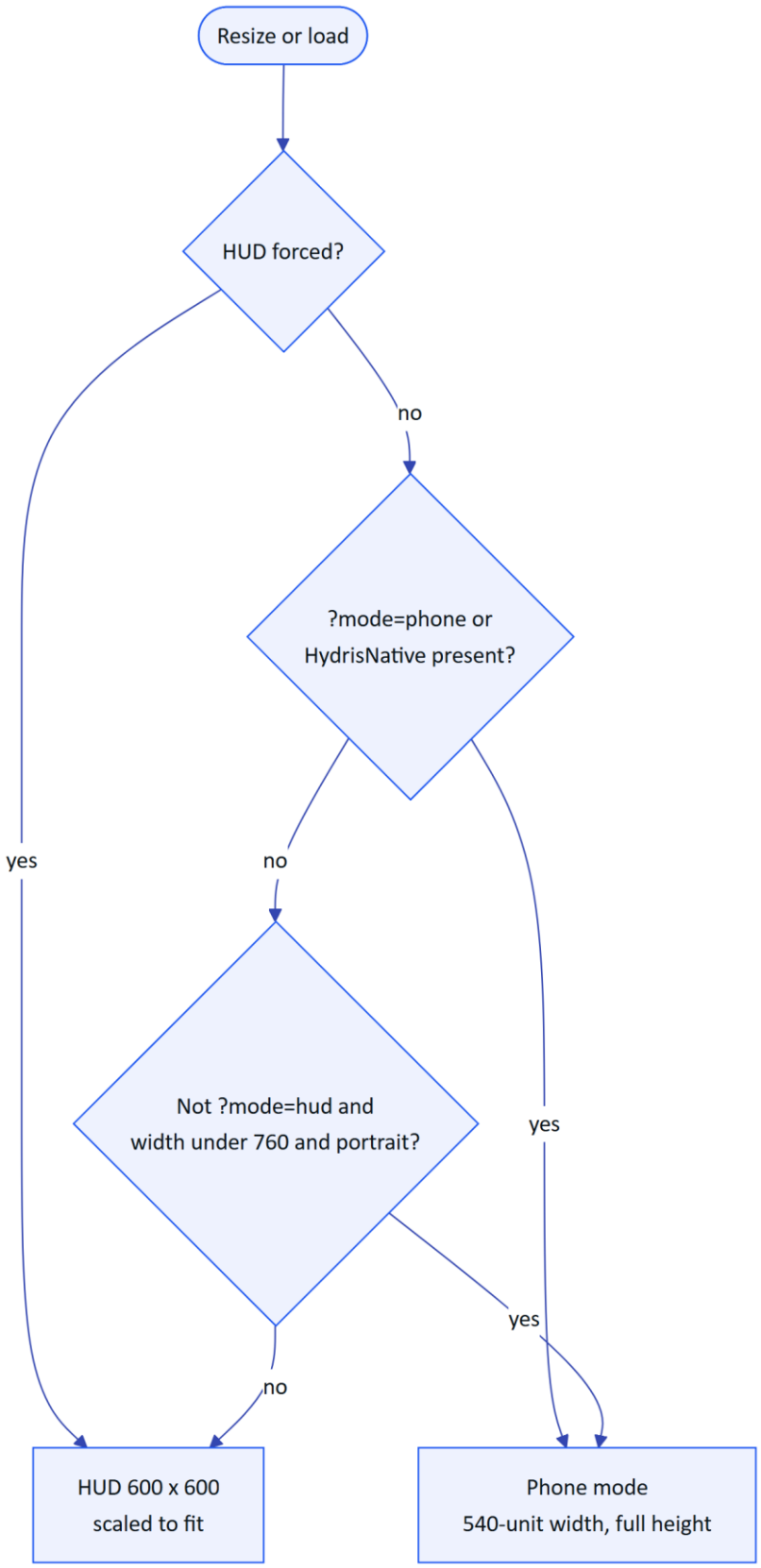

Figure 152. *Choosing HUD or phone layout*

Table 195. HUD versus phone mode

Aspect	HUD mode	Phone mode
Canvas	600 x 600, scaled to the window	540 design units wide, full device height
Target	Display glasses, simulator	Bridge app, portrait phones
Navigation	Directional keys, swipes, `hydris:input`; hint bar for the configured hand	Bottom tab bar: Home or Back, Scan, Hey Hydris, Alerts
Design rules	Additive display: black is transparent, light strokes, type at 20 px or more, a lime focus ring, one primary action per screen	Same components, safe-area insets

7.7.2 Phases and screens

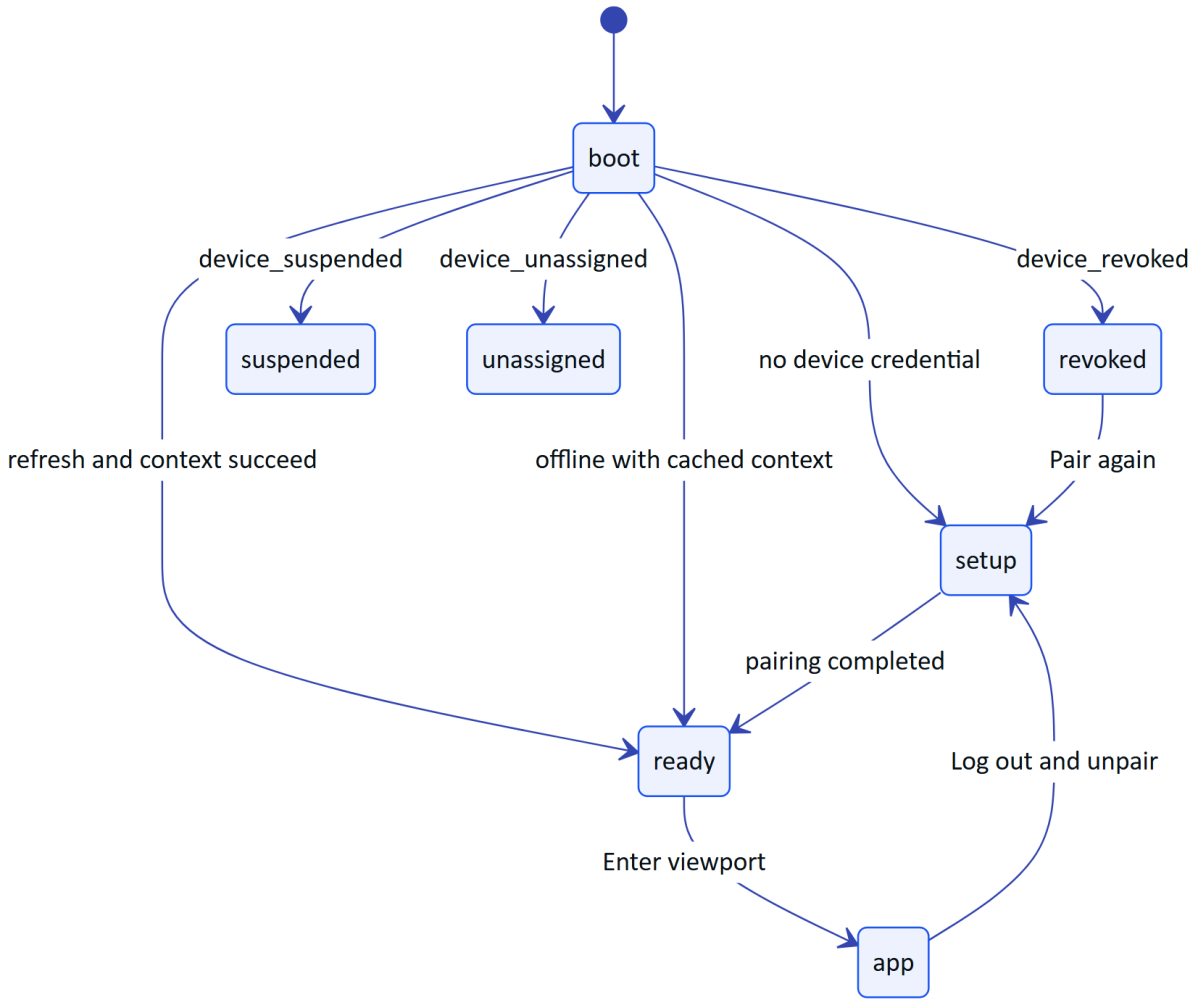

Figure 153. *Viewport phases (GlassesApp.tsx)*

Screens: home, alerts and alert detail, inspections and the inspection runner, tasks and task detail, asset context, assist, status, notifications, Hydris AI, plant map, audit scan, measurements, assets, SOP library and reader. Each screen is gated by the organization's UI profile (`screenPermitted`), so administrators decide in the console what each role sees on the glasses.

7.7.3 Voice assistant on the viewport

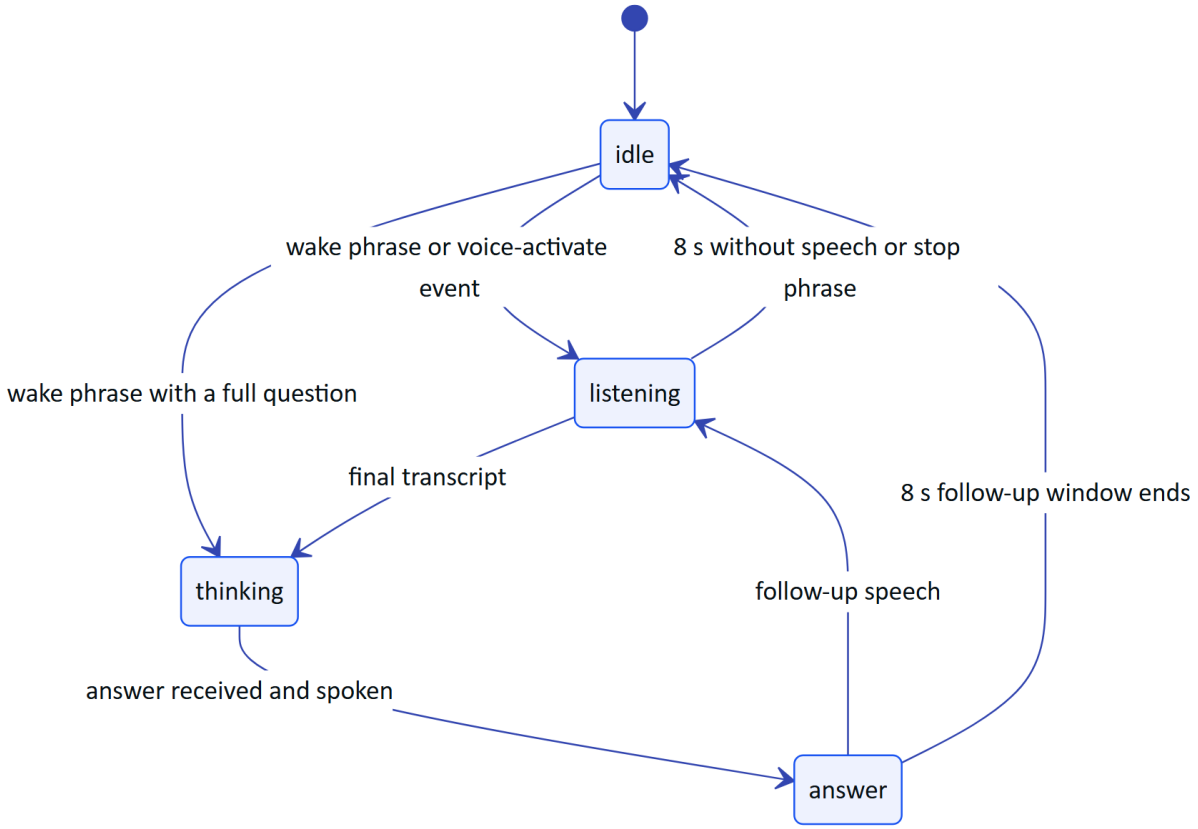

Figure 154. *Hey Hydris on the viewport (VoiceAssistant.tsx)*

The engine order is the bridge (`meta-glasses`), then the browser Web Speech API, then none (typed questions). Offline, demo and cached answers come from `answerFromContext` using the same rules as the API.

7.7.4 Audit scan on the viewport

`AuditScan.tsx` follows the same steps as the native app: guided capture (`apps/web/src/lib/GuidedCapture.tsx`, 900 ms ready time, stability threshold 0.012), A4 warp to 1654 px wide, two OCR passes in the browser, review of up to six pages and upload. The capture button shoots while framing and starts a scan from the menu or review.

7.8 Pairing with the Hydris account

Glasses become Hydris devices through an RFC 8628-style device authorization flow (`apps/api/src/modules/devices.ts`). Two variants exist.

7.8.1 Code and QR (viewport and bridge)

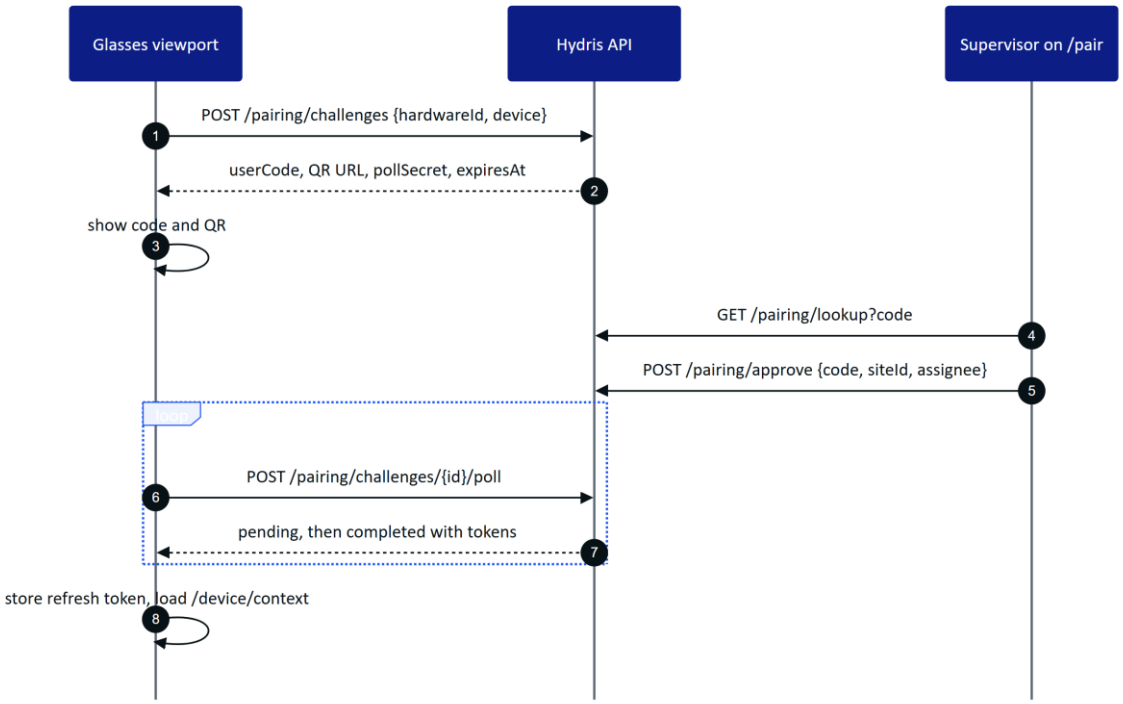

Figure 155. *Pairing from the glasses viewport*

The viewport's hardware ID is `hg-` plus a random UUID kept in local storage; the refresh token is stored in local storage too (`link.ts`), which the code marks as the integration boundary for a hardware-backed store.

7.8.2 Self-approval (native app)

The native app creates the challenge and approves it as the signed-in user in one step, assigning the glasses to that user and the first site the user can see. Its hardware ID is `rbm-` plus a SHA-256 digest of the DAT device identifier. Details are in the Mobile App Specification (HFG-MOB).

Table 196. Pairing rules and constants

Rule	Value
Code length and alphabet	8 characters from `23456789ABCDEFGHJKMNPQRSTVWXYZ`
Challenge lifetime	300 s
Poll interval	3 s; faster polls return `slow_down`
Statuses	pending, approved, completed, denied, expired, cancelled
Approver permission	`device.pair`; pairing to another person needs `device.manage`
Cross-organization	`device_owned_elsewhere` until revoked in the other organization
A new challenge for the same hardware	Cancels older pending or approved ones
Device states seen by the viewport	active, suspended, revoked, unassigned

7.9 Setup guide

7.9.1 Developer setup

1. Create an organization and app in the Wearables Developer Center (confirm country eligibility).

2. Install the Meta AI app on an Android 12 or later phone, sign in, pair the glasses and update the firmware.
3. In the Meta AI app, open Settings, App Info, tap the version five times and turn on Developer Mode.
4. Configure the GitHub Packages Maven repository for `com.meta.wearable` with a token that has `read:packages`.
5. Set `sdk.dir` and `hydris_url` in `local.properties`; leave the DAT placeholders at `0` for Developer Mode.
6. Build and install: `./gradlew assembleDebug`, then `adb install`.
7. Enable the experimental Speech, Inputs and Voice Invocations capabilities for the app in the Developer Center.

7.9.2 Operator setup (native app)

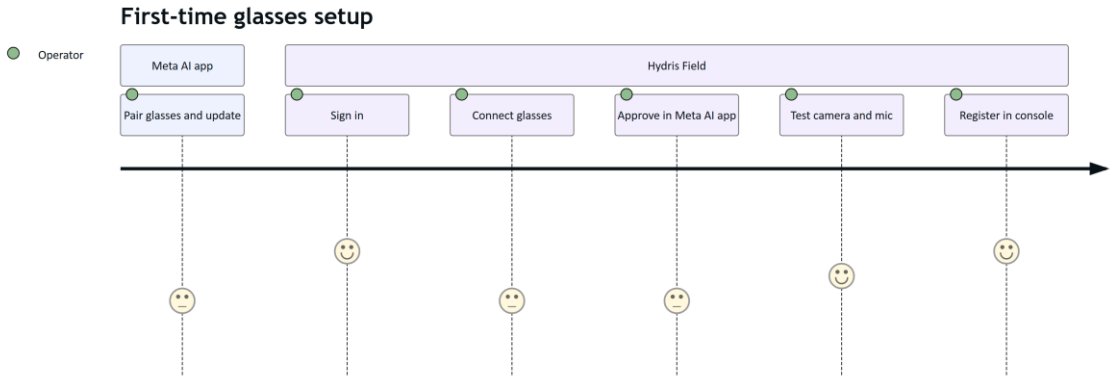

Figure 156. *Operator setup journey*

1. Open Hydris Field, sign in and allow Bluetooth and notifications.
2. Glasses tab, step 1: Connect glasses, approve Hydris in the Meta AI app, return.
3. Step 2: Test camera and Test microphone, approving each in the Meta AI app the first time.
4. Step 3: Register glasses. The glasses now appear in the console under Field glasses.
5. Scan tab: Scan with glasses camera, or press the capture button.

7.10 Troubleshooting

Table 197. Troubleshooting

Symptom	Likely cause	Fix
"Install the Meta AI app and pair your glasses in it first"	Meta AI app missing	Install it, sign in, pair the glasses
Meta link stays "unavailable"	DAT not initialized (Bluetooth permission denied)	Allow Nearby devices for Hydris in Android settings, reopen the app
Connect does nothing or returns unregistered	Developer Mode off, or firmware below DAT requirements	Enable Developer Mode, update firmware in the Meta AI app
"Camera permission denied in Meta AI"	Prompt declined	Run Test camera again and approve, or change it in Meta AI app connections
"No glasses session"	Glasses off, out of range or connected elsewhere	Wake the glasses, keep them near the phone, retry

Symptom	Likely cause	Fix
Preview is black or garbled	Stride inference wrong on this firmware	Report the device and firmware; use the phone camera meanwhile
Capture button ignored	Inputs not enabled for the app, or not on the scan screen	Enable Inputs in the Developer Center; open Scan
Hey Hydris uses the phone microphone	Glasses not registered or speech failed	Check step 1 and the microphone permission
Answers play on the phone speaker	Glasses not the Bluetooth audio output	Reconnect audio in Android Bluetooth settings
"Your account has no site to assign the glasses to"	User has no site access	Ask an admin for site access
`device_owned_elsewhere`	Glasses registered in another organization	Revoke them there first
Viewport shows "Device revoked"	Admin revoked the device	Pair again

7.11 Limitations and known gaps

- Not yet run on physical glasses; preview stride, capture press decomposition and speech restart are unverified.
- The DAT camera stream is not stopped when the preview ends (battery and capture LED impact).
- Preview frames reach the phone over the wireless link, so preview resolution and frame rate are far below on-glasses recording; detection runs on about 200 px frames and the photo is full resolution.
- The native app assigns registered glasses to the first available site.
- Voice Invocations are handled only by the bridge.
- `deviceName()` in the bridge is a constant.
- The bridge WebView needs navigation and permission hardening.
- Hardware gestures (Neural Band, temple touchpad) are not delivered to the web runtime; `input.ts` keeps a placeholder.
- The Mock Device Kit does not support display glasses (Meta FAQ), and Hydris has no automated DAT tests yet.
- Public publishing of DAT apps is not open yet (Meta FAQ); pilots rely on Developer Mode and release channels.
- Full Developer Center access is limited to countries where Meta AI glasses are supported.

7.12 Glasses state model and console view

7.12.1 GlassesState fields

`GlassesManager.state` is the single source of truth the native UI reads. Every field is written by DAT callbacks or by Hydris's own calls, never by the screens.

Table 198. GlassesState fields (glasses/GlassesManager.kt)

Field	Type	Written by	Shown as
`initialized`	Boolean	`initialize()`	Gates the Connect button (initializes on demand)

Field	Type	Written by	Shown as
`registration`	Text	`registrationState` collector	"Meta link: ..." on the status card
`registered`	Boolean	`registrationState` is `REGISTERED`	Step 1 done, Home tile text, Scan menu subtitle
`devices`	List of IDs	`Wearables.devices`	Status dot colour; input to the hardware ID
`deviceName`	Text	`devicesMetadata` (fallback "Ray-Ban Meta")	Card title, scan chip, registration name
`session`	Text	Session state collector	"session: ..." on the status card
`cameraStreaming`	Boolean	Camera stream state after start	"camera live" chip, step 2 done
`speech`	Text	Speech state collector	"mic listening" chip, step 2 done
`lastError`	Text	`fail()`	Error note on the Glasses screen
`log`	List, latest 30	`log()` and `fail()`	Connection log card (latest 8)

Separate flows carry the data path: `frame` (latest preview bitmap), `captureButton` (short presses) and `speech` (transcripts with the final flag). The shared flows have small extra buffers (4 presses, 16 transcripts) so a slow collector drops nothing in normal use.

7.12.2 What the console sees

Once registered, the glasses are an ordinary Hydris device record. The console's Field glasses page lists them with status, assignment, site, last seen time and app version; supervisors can assign, suspend or revoke them. Scans uploaded with the device token are attributed to the device. Domain events `device.paired` and `device.updated` are published on the realtime bus with the `device.read` permission, so open consoles update live.

Table 199. Device records created by each runtime

Runtime	Kind	Model	Hardware ID	App version reported
Native app	hardware	"Ray-Ban Meta"	`rbm-` plus SHA-256 prefix	`android-1.0.0`
Bridge viewport	hardware	"Web viewport"	`hg-` plus random UUID	`0.2.0`
Simulator viewport	simulator	"Hydris browser simulator"	`hg-` plus random UUID	`0.2.0`

> **NOTE**
> The same physical glasses paired once through the native app and once through the bridge produce two device records, because the hardware IDs are derived differently. Pilots should use one runtime per pair of glasses. Unifying the ID is Planned.

7.13 Data flows and network use

Table 200. What crosses which link

Data	Glasses to phone	Phone to Hydris	Notes
Preview frames	Raw YUV over the Meta link	Never	Luma only, processed in memory

Data	Glasses to phone	Phone to Hydris	Notes
Page photo	Bitmap or HEIC	JPEG under 850 KB per page, only on "Send"	Two OCR readings attached
Speech	Transcripts from DAT	Question text only	Audio is not sent to Hydris
Spoken answers	Phone TTS over Bluetooth audio	Answer text received from `/assistant/ask`	
Capture button	Input events	Never	
Device identifier	DAT device ID	SHA-256 derived hardware ID	Native app only

Table 201. Timing targets for the first hardware run (targets, not measured)

Path	Target
First preview frame after opening the glasses scan	Under 3 s with an existing session
Preview rate reaching detection	8 to 10 frames per second
Photo returned after capture	Under 4 s
Wake phrase to listening card	Under 1 s after the transcript arrives
Speech restart gap after silence	Under 1 s

Results will be recorded in the QA test plan (HFG-QA).

7.14 Hardware acceptance tests

The first hardware run must answer the open questions in this document. Failures become issues against `GlassesManager.kt` or the bridge.

Table 202. Hardware test checklist

ID	Test	Expected result
GLS-T01	Connect glasses with the Meta AI app installed and Developer Mode on	Meta AI app opens, approval returns to Hydris, Meta link "registered"
GLS-T02	Connect with Developer Mode off	Clear error from DAT shown, no crash
GLS-T03	Test camera, first time	Meta AI permission prompt, then "Glasses camera is streaming"
GLS-T04	Guided scan of an A4 sheet on a dark desk	Preview correct (not sheared), guidance changes, auto-capture within 10 s of a steady pose
GLS-T05	Leave the scan screen	Preview stops; record whether the capture LED and stream stay on
GLS-T06	Capture photo in different head poses	Bitmap upright after EXIF rotation
GLS-T07	Short press of the capture button on the Scan menu	Glasses scan starts
GLS-T08	Long press of the capture button	Ignored by Hydris; record what DAT reports
GLS-T09	Hey Hydris through the glasses microphone for 10 minutes	Listening resumes after each silence; no transcripts of Hydris's own answers
GLS-T10	Answer playback	Speech heard on the open-ear speakers
GLS-T11	Register glasses	Device appears in the console with the right name and site

ID	Test	Expected result
GLS-T12	Revoke the device in the console, then upload a glasses scan	Upload fails with a clear message; Unregister clears local tokens
GLS-T13	Turn the glasses off mid-session	Session stops, state resets, next scan recreates the session
GLS-T14	Bridge: tests T03 to T10 inside the WebView	Same results through the JavaScript bridge
GLS-T15	Battery over a simulated four-hour shift with periodic scans and Hey Hydris on	Recorded for phone and glasses

7.15 Frequently asked questions

Can an operator use Hydris without the Meta AI app? Yes, without glasses. Every glasses feature has a phone fallback: the phone camera, microphone and speaker. With Ray-Ban Meta or Oakley Meta glasses, the Meta AI app is mandatory.

Why does Hydris need its own pairing when the glasses are already paired in the Meta AI app? The Meta AI app pairs the glasses to the phone. Hydris pairing makes the glasses a managed device in the customer's organization: assigned to a person and a site, visible to supervisors, revocable, and credited for the scans it captures.

Why not run Hydris as a Web App on Meta Ray-Ban Display? Meta's Web Apps cannot use the camera or capture microphone audio, so audit scans and Hey Hydris on the glasses microphone need a phone app with DAT. A Web App could still show the HUD, which is a Planned option.

Does Hydris record video? No. It uses preview frames in memory for framing and takes single photos of documents.

Which runtime should a pilot use? The native Hydris Field app. The bridge exists for the web viewport and for development comparison; it is not planned for general distribution.

Can one phone serve several pairs of glasses? DAT selects a device with `AutoDeviceSelector`. Hydris assumes one pair per phone and uses the first device it sees.

7.16 Privacy

> **IMPORTANT**
> Glasses cameras raise bystander concerns in plants. Hydris keeps capture deliberate, local and visible.

- **Deliberate capture.** The glasses camera runs only on the Scan screen or the camera test. Photos are taken by auto-capture after the page is aligned, by "Capture now" or by the capture button. There is no background recording and no video upload.
- **Capture LED.** Meta glasses show a capture LED when the camera is in use; Hydris cannot and does not try to control it. Whether the LED is lit during DAT preview streaming is to be confirmed on hardware.
- **On-device processing.** Page detection and both OCR passes run on the phone. Images leave the phone only when the operator taps "Send for verification".
- **Minimal identifiers.** The raw DAT device identifier is hashed before it reaches Hydris.
- **Audio.** Glasses speech is recognized through DAT; Hydris receives transcripts, not audio, and sends only the question text to `/assistant/ask`.

- **Bystanders.** Pilot sites should brief staff, mark scan points, and avoid scanning near people; scans target A4 documents and the warp crops to the page.
- **Retention and access.** Scans follow the audit document retention and permissions described in the Security and Privacy document (HFG-SEC).

7.17 Roadmap

Figure 157. *Glasses integration roadmap (indicative)*

Table 203. Planned display work

Item	Status	Basis
Use the display of Meta Ray-Ban Display through DAT	Planned, unverified	Meta's documentation index says DAT apps can use "the display on Meta Ray-Ban Display glasses"; the API details are not yet reviewed
Run the viewport as a Web App on Meta Ray-Ban Display	Planned, unverified	Meta Web Apps run in a Chromium-based browser on the glasses over HTTPS; they cannot use the camera or capture microphone audio, and receive speech only as dictated text or WebMCP tool calls
Native device adapter for display glasses with neural band input	Planned	Listed on the Hydris website roadmap (`apps/web/src/site/content.ts`)
Vuzix M400 and RealWear Navigator 520	Planned	Device records in demo data only

Audit scans from glasses will always need a phone app with DAT, because Web Apps cannot use the camera.

8 Design System and UX Guidelines

> **NOTE**
>
> **HFG-DSN v1.0** · Draft for review · Owner: Design, Hydris. The tokens, components, layouts and interaction rules that make every Hydris surface safe, glanceable and consistent in the field

Hydris Field Glasses is used by people standing next to running pumps, aeration basins and chemical dosing skids, often wearing gloves, ear defenders and safety glasses, sometimes in bright sun and sometimes in a dark pump room. The design system exists so that every surface (the marketing website, the operator console, the glasses heads-up app, the browser simulator and the native Android app) shows plant state the same way, uses colour for the same meaning, and never asks an operator to read more than they can take in at a glance.

This document describes the system **as it is built today**, with every token and component traced to the file that defines it, and it marks the gaps honestly. Where the code and the intended design differ, the difference is called out as a finding with a recommended fix. Status labels follow the house convention: **Built** (in the code and working), **Partial** (in the code with known gaps), **Planned** (designed or intended, not yet in the code).

5	3	4	19.05:1
surfaces website, console, glasses HUD, simulator, Android app	**token sources** globals.css, site.css brand layer, Android Theme.kt	**severity levels** critical, high, medium, low, one colour each	**primary text contrast** white on Deep Midnight (WCAG AAA)

8.1 Design principles

The principles below are ranked. When two of them conflict, the one higher in the list wins. Every design review and every QA pass (see the design QA checklist chapter) checks a screen against them in this order.

8.1.1 Safety first

Hydris is an information and records tool. It does not operate equipment, and the interface must never suggest that it does. That rule is visible in the product today:

- The glasses task screen states "Records only" and that Hydris does not operate equipment (`apps/web/src/glasses/GlassesApp.tsx`, `TaskDetail`). **Built.**
- The glasses Acknowledge action carries the inline hint "does not change equipment" (`AlertDetail`). **Built.**
- Recommended actions on alarms are labelled "Next step (approved procedure)" so the operator knows the text comes from an approved SOP, not from a model (`AlertDetail`). **Built.**
- Inspection runs show safety notices **before** the first step and render step cautions in the medium (amber) tone with a shield icon (`InspectionRunner`). **Built.**

> **IMPORTANT**
>
> No Hydris screen may present a control that looks like it starts, stops, opens, closes or sets plant equipment. Buttons are verbs about records: Acknowledge, Record, Submit, Resolve, Send for verification.

8.1.2 The one-glance rule

An operator should get the answer to "is anything wrong, and what do I do next?" from a single glance of under two seconds. In practice that means:

- One primary action per HUD screen and one focus indicator (`apps/web/src/glasses/hud.tsx` header comment). **Built.**
- Numbers are shown large, with their unit and their freshness, never as a bare value.
- Severity is carried by colour **and** by a word (Critical, High) and, on the HUD, by a border. Colour alone is never the only signal.
- Lists on the HUD are one column, short labels, with secondary meta in a smaller, dimmer line.

8.1.3 Advisory only

The voice assistant and the audit-scan OCR are advisory. Answers from "Hey Hydris" are grounded in the account's live data and shown with the facts (value, unit, label) they used (`apps/android/.../ui/screens/AskScreen.kt` shows up to four facts per answer). Scanned audit values always go to a human reviewer and then to a second approver in the console; the Android scan flow says so on its confirmation screen (`ScanScreen.kt`). The design must keep that distinction: model output is shown in Insight Purple contexts or with explicit "verify" copy, and is never styled as a confirmed record.

8.1.4 Never hide state

Every value has a state, and the state is always visible:

- **Freshness.** A reading is only meaningful together with when it was observed. The domain defines `live`, `delayed`, `stale` and `no_data` (`packages/domain/src/freshness.ts`), and missing data is reported as `no_data`, never as zero. The console renders this with `FreshnessBadge` (`apps/web/src/ui/index.tsx`). **Built.**
- **Connectivity.** The glasses app shows an offline strip ("Offline · showing data cached ...") and a queued-steps counter in the status bar (`GlassesApp.tsx`, `StatusBar`). **Built.**
- **Synthetic data.** Simulator-generated values carry a dashed purple "Synthetic" chip on every surface (`SyntheticBadge` in `ui/index.tsx`, the `synthetic` chip on Android alarm detail, "· synthetic" on the HUD), and the console shows a demo banner for demo tenants (`console/Shell.tsx`). **Built.**
- **Permissions.** When a UI profile blocks an action, the glasses app shows a dashed "not permitted by your profile" note instead of silently removing the control (`Blocked` in `GlassesApp.tsx`). **Built.**

8.1.5 Dark first for the glasses

Display glasses use an additive display: pure black renders as transparent. The HUD therefore uses a black canvas, light strokes, large type (at least 20 px for body content) and no large filled areas (`hud.tsx`). The glasses route is forced dark regardless of the user's theme (`FORCED_DARK_RE` in `apps/web/src/ui/theme-script.ts`). The console and website default to dark (`THEME_DEFAULT_MODE = 'dark'`) and offer a full light theme; the Android app is dark only today.

8.1.6 Consistency across surfaces

The same alarm should look like the same alarm on the console, the phone and the glasses: same severity colour family, same state words, same order of information (title, value and unit, asset, time, recommended action). The token chapter below shows where the surfaces currently drift and how to close the gaps.

8.2 Design system architecture

The system has four layers. Brand decisions flow down into tokens; tokens are implemented per platform; components consume only tokens; screens compose components.

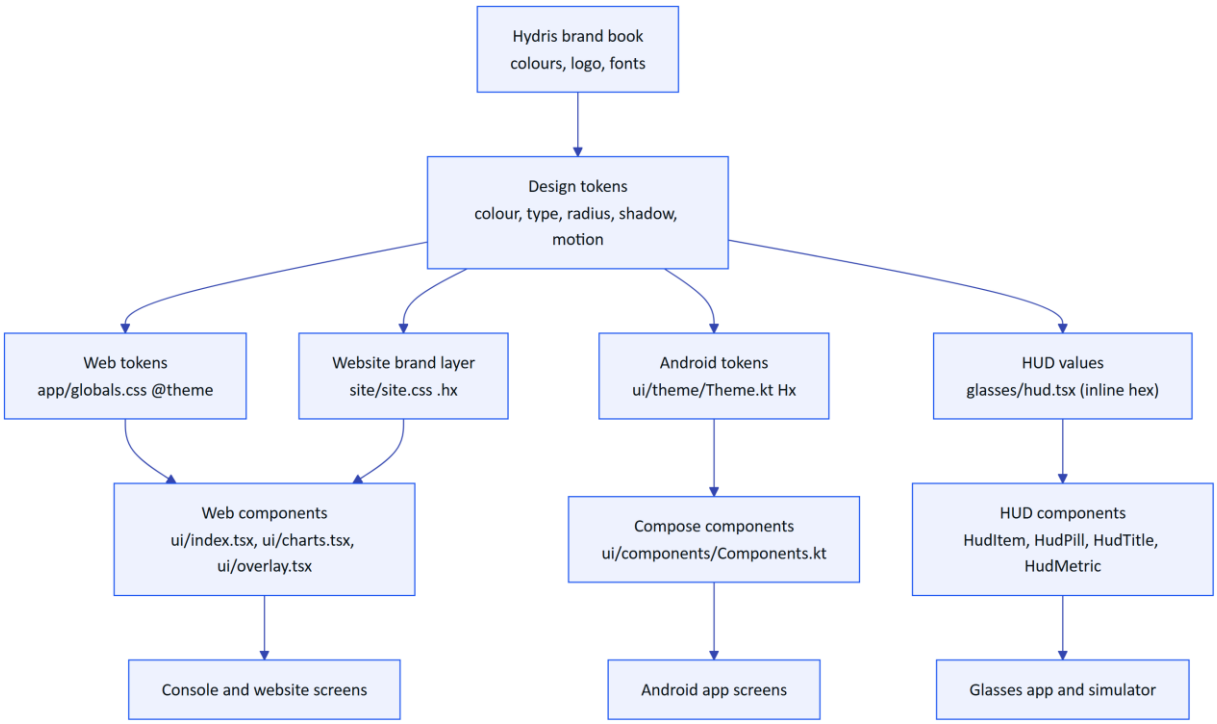

Figure 158. *Design system layers, from brand book to screens*

Table 204. Where the design system lives in the code

Layer	Web console and website	Glasses HUD and phone mode	Android app
Colour, radius, shadow, motion tokens	`apps/web/src/app/globals.css` (`@theme`, `:root`, `:root[data-theme='light']`)	Inherits globals.css; severity tints hard-coded in `apps/web/src/glasses/hud.tsx` and `GlassesApp.tsx`	`apps/android/app/src/main/java/ai/hydris/field/ui/ther` (`object Hx`)
Website brand overrides	`apps/web/src/site/site.css` (`.hx` scope)	Not used	Not used
Typography roles	`.t-display` to `.t-kpi-sm` in globals.css; Outfit via `next/font` in `app/layout.tsx`	Tailwind pixel sizes in `hud.tsx`	Material 3 `Typography` in Theme.kt
Components	`apps/web/src/ui/index.tsx`, `ui/charts.tsx`, `ui/overlay.tsx`, `ui/theme.tsx`	`hud.tsx` primitives plus screens in `GlassesApp.tsx`	`ui/components/Components.kt`
Icons	`lucide-react` 1.57.0	`lucide-react`	`material-icons-extended` (Outlined set)
Theme switching	`ui/theme.tsx`, `ui/theme-script.ts`	Forced dark	Dark only
Shared token package	None	None	None

RISK

Tokens are defined three times and the HUD uses inline hex values. The values were aligned by hand, and several have drifted (see the severity mapping table). Until a single token source generates all three platforms (Planned, see the future chapter), every colour change must be made in all three files in the same pull request.

8.3 Brand and colour system

8.3.1 Brand palette

The brand palette comes from the Hydris AI brand book and is quoted in the header comments of `globals.css` and `site.css`. Core Blue is the dominant brand colour on the website; Signal Blue is the action colour in product surfaces. Deep Intelligence Blue (navy) is used for pressed states and deep brand panels.

Table 205. Brand palette

Name	Hex	Role	Where defined
Hydris Core Blue	`#1746E4`	Brand fills, hover on primary, chart series 1, website accent	`--color-brand` (globals.css), `--hx-blue` (site.css), `Hx.Core`
Deep Intelligence Blue	`#0C1E85`	Pressed primary, deep brand panels, logo navy	`--color-brand-deep`, `--hx-deep`
Signal Blue	`#1954F1`	Primary buttons, focus fills, selected state, links on paper	`--color-signal`, `--color-lime`, `Hx.Signal`
Signal tint (accent text)	`#6F93FF`	Accent **text** and focus ring on dark backgrounds	`--color-accent`, `Hx.SignalText`
Deep Midnight	`#081116`	App background (dark)	`--color-bg`, `Hx.Midnight`
Graphite Black	`#222222`	Print and brand graphite	`--color-graphite`
Pure White	`#FFFFFF`	Primary text on dark, ink on blue	`--color-text`
Alert Orange	`#FF2800`	Critical severity only	`--color-critical`, `--color-orange`, `Hx.Alert`
Active Green	`#00FE21`	OK and live: dots and text only, never large fills	`--color-ok`, `Hx.Ok`
Hydris Insight Purple	`#8A60F0`	AI, information, secondary data series, synthetic data	`--color-purple`, `--color-cobalt`, `Hx.Insight`

NOTE

The token names `lime`, `lime-2`, `lime-3` and `cobalt` are historical. In the code today `lime` means the brand blue role with white ink, and `cobalt` means Insight Purple (globals.css header comment). New code should use `signal`, `brand`, `brand-deep` and `purple`. Renaming the legacy roles is part of the token pipeline work.

8.3.2 Web neutral and surface tokens (dark and light)

The console defines every semantic token for both themes. The dark values live in `@theme`; the light values in `:root[data-theme='light']`. Components refer only to the semantic names, so a theme switch needs no JavaScript.

Table 206. Web surface and text tokens (apps/web/src/app/globals.css)

Token	Dark	Light	Semantic role
`--color-bg`	`#081116`	`#F6F8FC`	Page background
`--color-bg-2`	`#0C161C`	`#EEF2F8`	Inset areas, input wells
`--color-surface`	`#121A20`	`#FFFFFF`	Cards, menus, dialogs
`--color-surface-2`	`#1A2229`	`#F2F5FA`	Secondary buttons, chips, hover
`--color-surface-3`	`#252E35`	`#E5EAF2`	Pressed, disabled primary, progress track
`--color-line`	white 7.5 %	slate 8.5 %	Hairline borders
`--color-line-2`	white 14 %	slate 15 %	Strong borders, inputs
`--color-text`	`#FFFFFF`	`#1B2129`	Primary text
`--color-text-2`	`#C5CCD2`	`#46505B`	Secondary text
`--color-muted`	`#8D979F`	`#66717D`	Labels, meta, captions
`--color-paper`	`#EEF1F4`	`#FFFFFF`	KPI paper cards (light island on dark page)
`--color-ink` / `--color-ink-2`	`#081116` / `#4C5862`	`#1B2129` / `#545F6B`	Text on paper
`--color-accent`	`#6F93FF`	`#1746E4`	Accent text, focus ring
`--color-scrim`	midnight 72 %	slate 38 %	Modal backdrop
`--color-field`	`#0C161C`	`#FFFFFF`	Input background
`--color-placeholder`	`#67727B`	`#8A949F`	Input placeholder
`--color-chart-grid`	white 7 %	slate 7 %	Chart gridlines

8.3.3 Android tokens

The Android app maps the same palette into a Material 3 dark colour scheme (`Theme.kt`). `primary` is Signal Blue, `secondary` is Insight Purple, `tertiary` is a softer green (`OkSoft`), `error` is Alert Orange.

Table 207. Android Hx tokens (apps/android/.../ui/theme/Theme.kt)

Token	Hex	Material role	Use
`Hx.Midnight`	`#081116`	`background`, `surfaceContainerLow`	Screen background
`Hx.Surface`	`#0E1A21`	`surface`, `surfaceContainer`	Cards, navigation bar
`Hx.Surface2`	`#13222B`	`surfaceVariant`, `surfaceContainerHigh`	Raised containers
`Hx.Line`	white 12 %	`outline`	Card borders, progress track
`Hx.Signal`	`#1954F1`	`primary`	Primary buttons, progress, spinner
`Hx.Core`	`#1746E4`	`primaryContainer`	Pressed and selected
`Hx.SignalText`	`#6F93FF`	(direct)	Accent text, selected tab
`Hx.Alert`	`#FF2800`	`error`	Critical severity
`Hx.High`	`#F26414`	(direct)	High severity, stale data, error notes
`Hx.Medium`	`#F5B83D`	(direct)	Medium severity, delayed data

Token	Hex	Material role	Use
`Hx.Ok`	`#00FE21`	(direct)	Active Green (reserved)
`Hx.OkSoft`	`#34D27A`	`tertiary`	Live data, typical-range band
`Hx.Insight`	`#8A60F0`	`secondary`	AI, synthetic chip
`Hx.Text` / `Hx.Text2` / `Hx.Muted`	`#F2F5F7` / `#B7C2CA` / `#7D8B95`	`onBackground`, `onSurfaceVariant`	Text hierarchy

8.3.4 HUD colour values

The HUD inherits `globals.css`, so `lime` (focus border, selected pill) is Signal Blue `#1954F1`. Severity tints on the HUD are lighter than the console's, because they must read as thin strokes and text on an additive display. They are currently inline hex values in `hud.tsx` and `GlassesApp.tsx`.

Table 208. HUD colour values (apps/web/src/glasses/hud.tsx, GlassesApp.tsx)

Use	Border	Text	Notes
Critical	`#FF5A4E`	`#FF8A80`	`HudPill tone="critical"`, `AlertInterrupt` border
High	`#F26414`	`#FF9A5C`	Matches Android `Hx.High` border
Medium	`#F5B83D`	`#F7C96A`	Step cautions, safety notices icon
OK	`#4ED98E`	`#7FE6AE`	Softer than Active Green to avoid glare
Neutral line	white 25 %	white 80 %	Default pill
Focus / selected	`lime` (`#1954F1`)	`#141714` on fill	See the contrast findings below
Critical banner background	n/a	n/a	`#2A0D0B` (critical), `#2A1608` (high and offline strip)

8.3.5 Severity and status colour mapping

Severity colours are semantic and are **never** reused as decoration (globals.css header). The table shows how each surface renders each severity today, which is where the drift is easiest to see.

Table 209. Severity colour mapping by surface

Severity	Web console fill / text (dark)	Web light text	Website `.hx`	Android `Hx.severity()`	HUD border / text
Critical	`#FF2800` / `#FF7A5C`	`#C41F00`	`#FF2800`	`#FF2800`	`#FF5A4E` / `#FF8A80`
High	`#FF9F1C` / `#FFB547`	`#A35200`	`#FF5A1F`	`#F26414`	`#F26414` / `#FF9A5C`
Medium	`#F5C542` / `#F7D36E`	`#7F6000`	(inherits)	`#F5B83D`	`#F5B83D` / `#F7C96A`
Low	`#8FB0FF` / `#B3C9FF`	`#1746E4`	(inherits)	`#6F93FF`	neutral line
OK / live	`#00FE21` / `#00FE21`	`#0B7A1F`	`#00FE21`	`#34D27A` (live), `#00FE21`	`#4ED98E` / `#7FE6AE`

DECISION

The canonical High colour is **#F26414** (the brand-adjacent orange already used on Android and the HUD). The console's amber #FF9F1C and the website's #FF5A1F will move to it, with a lighter text tint for small text on dark. Critical stays Alert Orange #FF2800 everywhere, with surface-specific text tints for legibility. Status: **Planned** (token alignment).

The same colours drive object states through `StateChip` (`ui/index.tsx`). The mapping rule: red family for "needs attention now", amber for "seen, not finished", purple for "someone is working on it", green for "done or healthy", outline for "neutral or historical".

Table 210. State to tone mapping used by StateChip (apps/web/src/ui/index.tsx)

Tone	States
Critical	`active` (alarm), `down`, `revoked`, `urgent`
Medium	`acknowledged`, `paused`, `on_hold`, `degraded`, `disconnected`
Cobalt (Insight Purple)	`investigating`, `in_progress`, `assigned`, `maintenance`
OK	`resolved`, `approved`, `completed`, `operational`, `connected`
High	`rejected`, `suspended`
Brand blue (lime)	`submitted`
Outline	`closed`, `scheduled`, `cancelled`, `open`, `standby`, `draft`, `archived` and other neutral states

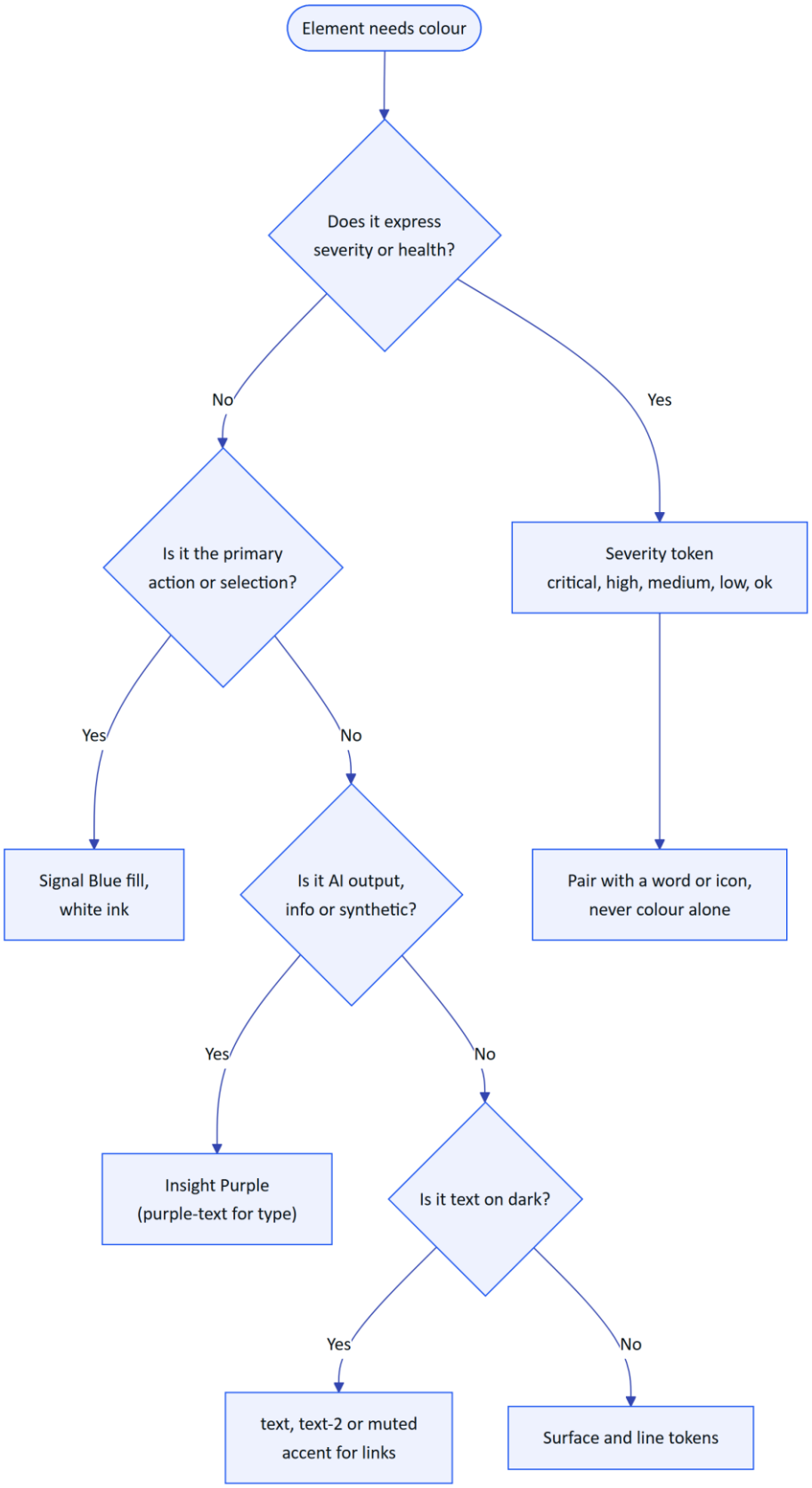

Figure 159. *Choosing a colour for a value or element*

8.3.6 Contrast audit

Contrast ratios below were computed with the WCAG 2.x relative luminance formula from the hex values in the code (script run during authoring; alpha tints were not composited and are excluded). The target for all product text is WCAG 2.2 **AA**: 4.5:1 for normal text and 3:1 for large text (24 px regular or 18.66 px bold and above) and for non-text UI components.

Table 211. Contrast ratios for key colour pairs (computed from code values)

Foreground	Background	Ratio	Result	Where used
White `#FFFFFF`	Midnight `#081116`	19.05:1	AAA	Primary text, console and HUD
text-2 `#C5CCD2`	surface `#121A20`	10.84:1	AAA	Secondary text in cards
muted `#8D979F`	surface `#121A20`	5.91:1	AA	Labels, meta
muted `#8D979F`	bg `#081116`	6.41:1	AA	Captions on page
Signal `#1954F1`	Midnight `#081116`	3.25:1	Large text only	Why accent text uses `#6F93FF`
Accent `#6F93FF`	Midnight `#081116`	6.62:1	AA	Links, selected tab, focus ring
White	Signal `#1954F1`	5.87:1	AA	Primary button label
White	Core Blue `#1746E4`	6.92:1	AA	Hover primary, brand panel
White	Deep Blue `#0C1E85`	13.64:1	AAA	Pressed primary
White	Danger solid `#E02400`	4.75:1	AA	`btn-danger-solid`
critical-text `#FF7A5C`	surface `#121A20`	6.86:1	AA	Critical chip text
high-text `#FFB547`	surface	10.01:1	AAA	High chip text
ok-text `#00FE21`	surface	12.73:1	AAA	OK chip text
purple-text `#B69CFF`	surface	7.69:1	AAA	Insight and synthetic chips
Ink `#081116`	paper `#EEF1F4`	16.81:1	AAA	KPI paper cards
Light text `#1B2129`	White	16.20:1	AAA	Light theme body
Light muted `#66717D`	bg `#F6F8FC`	4.68:1	AA	Light theme labels
Light critical `#C41F00`	White	5.93:1	AA	Light theme critical text
Light high `#A35200`	White	5.58:1	AA	Light theme high text
Light ok `#0B7A1F`	White	5.50:1	AA	Light theme OK text
Android Text2 `#B7C2CA`	Surface `#0E1A21`	9.75:1	AAA	Secondary text
Android Muted `#7D8B95`	Surface `#0E1A21`	5.05:1	AA	Meta lines
Android Insight `#8A60F0`	Surface `#0E1A21`	4.22:1	Fails AA (normal)	Synthetic chip text
White	Android Alert `#FF2800`	3.78:1	Fails AA (normal)	Filled critical chip, 12 sp
White	Android High `#F26414`	3.18:1	Fails AA (normal)	Filled high chip, 12 sp
HUD critical `#FF8A80`	Black	9.20:1	AAA	HUD critical text
HUD white 55 % (about `#8C8C8C`)	Black	6.25:1	AA	HUD meta lines
HUD ink `#141714`	Signal `#1954F1`	3.08:1	Large text only	Selected pill, View button, digit entry

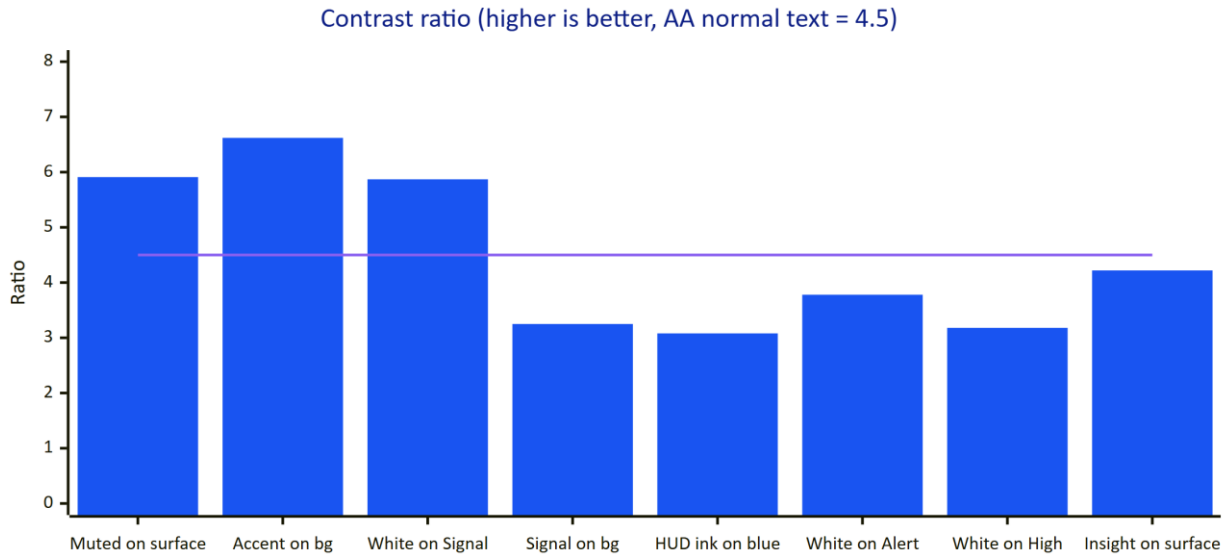

Figure 160. *Contrast of selected text pairs against the 4.5:1 AA threshold*

WARNING

Three contrast failures exist in shipped code. (1) **HUD selected pill and View button**: `#141714` ink on Signal Blue is 3.08:1. The ink colour is a leftover from when the `lime` role was a light lime green. Fix: white ink on the blue fill (5.87:1) in `HudPill tone="lime"`, `AlertInterrupt` and the digit-entry cell in `GlassesApp.tsx`. (2) **Android filled severity chips**: white on `#FF2800` (3.78:1) and on `#F26414` (3.18:1) at 12 sp. Fix: use the tinted chip style (coloured text on a 14 % fill) or dark ink on the fill. (3) **Android synthetic chip**: Insight `#8A60F0` text on Surface is 4.22:1. Fix: add an `Hx.InsightText` tint equivalent to the web `#B69CFF`. Status: **Planned** fixes, tracked in the QA checklist.

8.4 Typography

8.4.1 Typefaces

Table 212. **Typefaces by surface**

Surface	Typeface in code	Source	Status
Console, website, glasses web app, simulator	Outfit (300, 400, 500, 600)	`next/font/google` in `apps/web/src/app/layout.tsx`, exposed as `--font-outfit`	**Built**
Monospace (IDs, codes)	System UI monospace stack	`--font-mono` in globals.css	**Built**
Android app	Platform default (Roboto on most devices)	Material 3 `Typography` without a custom `FontFamily`	**Built**
Word and PDF product documents	Arial and Calibri	`docs/product/tools/build.js` theme	**Built**
Brand typefaces Unison Pro (display) and Eloquia Text (body)	Not in any product surface	Brand book	**Planned** (licensing for web and app embedding)

DECISION

Product UI keeps Outfit and the platform font until web and app embedding licences for Unison Pro and Eloquia Text are confirmed. When they are, Unison Pro is limited to display roles (`t-display`, `t-hero`, website headings) and Eloquia Text replaces Outfit for body roles. Numerals in KPIs keep tabular figures (`.num`), whichever face is used.

All numeric values use tabular, lining figures (`.num` in globals.css: `font-variant-numeric: tabular-nums lining-nums`) so live values do not jitter as digits change. HUD metrics and digit-entry cells apply `.num` too.

8.4.2 Web type scale

Table 213. Web typography roles (apps/web/src/app/globals.css)

Role class	Size	Line height	Tracking	Weight	Use
`t-display`	clamp 44 to 92 px	1.02	-0.04 em	600	Website hero
`t-hero`	clamp 36 to 64 px	1.04	-0.035 em	600	Website section heroes
`t-page`	clamp 30 to 44 px	1.06	-0.03 em	600	Console page titles
`t-section`	21 px	1.25	-0.02 em	600	Card groups, dialog titles
`t-card`	16.5 px	1.3	-0.012 em	600	Card titles
body (`.input`, table)	14 to 14.5 px	1.5	normal	400	Body text, tables, inputs
`t-label`	13 px	1.35	0.005 em	400	Field labels, KPI labels
`t-eyebrow`	11.5 px	1.4	0.16 em, uppercase	500	Section eyebrows
`t-kpi`	clamp 38 to 54 px	0.95	-0.045 em	300	KPI numerals
`t-kpi-sm`	28 px	1.05	-0.035 em	300	Secondary KPI numerals
Table header	11.5 px	n/a	0.08 em, uppercase	500	`.table th`

8.4.3 Android type scale

Table 214. Android typography (apps/android/.../ui/theme/Theme.kt)

Material role	Size	Weight	Tracking	Use
`displaySmall`	34 sp / 38 sp	SemiBold	-0.8 sp	Large greetings
`headlineSmall`	24 sp	SemiBold	-0.4 sp	Screen titles ("Audit scan", "Glasses")
`titleLarge`	20 sp	SemiBold	-0.2 sp	Section titles
`titleMedium`	16 sp	Medium	0	List item titles, option rows
`bodyLarge`	16 sp / 22 sp	Regular	0	Primary body
`bodyMedium`	14 sp / 20 sp	Regular	0	Secondary body, KPI labels
`bodySmall`	12.5 sp / 17 sp	Regular	0	Meta lines
`labelSmall`	11 sp	Medium	1.4 sp	Eyebrows (uppercase)
KPI value (inline)	34 sp	SemiBold	-1 sp	`KpiTile` value
Alarm reading (inline)	36 sp	SemiBold	0	Alarm detail reading

Sizes are in sp, so the app follows the operator's system font-size setting. **Built.**

8.4.4 HUD type scale

The HUD is designed on a 600 by 600 canvas. All sizes below are canvas units, scaled to the display.

Table 215. HUD typography (apps/web/src/glasses/hud.tsx, GlassesApp.tsx)

Role	Size	Weight	Use
HudTitle	38	Regular, -0.03 em	Screen title
HudMetric value	52	Light, tabular	Key reading
Digit entry cell	56	Light, tabular	Inspection numeric input
Alert interrupt title	34	Regular	Full-screen alarm
HudItem row	23	Regular	List rows, actions
Body and instructions	19 to 22	Regular	Step instructions, SOP text
Status bar	18	Regular	Connection, queue, operator
HudPill	18	Medium	Status pills
Eyebrow	17	Uppercase, 0.14 em	Section and step labels
Meta line	17	Regular, white 55 %	Secondary row text
Hint bar	16	Regular, white 45 %	Control hints

> **TIP**
> The HUD comment sets the floor at 20 px for content. Meta lines, eyebrows and hints at 16 to 18 are allowed because they are secondary and never carry the only copy of a safety-relevant value. A UI profile can raise everything with layout.textScale (0.9, 1, 1.15, 1.3; clamped to 0.85 to 1.4) in packages/domain/src/uiprofile.ts; the glasses app applies it with CSS zoom in GlassesApp.tsx. **Built.**

8.5 Spacing, radius and elevation

8.5.1 Spacing

Web layouts use the Tailwind 4 spacing scale (4 px base) without custom spacing tokens. The recurring values are documented here so new screens stay on the rhythm.

Table 216. Spacing rhythm in use

Context	Web console	Android	HUD
Page gutter	16 px mobile, 32 px from md (px-4 md:px-8)	16 dp (padding(horizontal = 16.dp))	32 units HUD, 24 units phone mode (px-8 / px-6)
Card padding	24 px (.card), 18 px tight, 20 to 24 px KPI	16 dp default, 14 dp list cards	16 to 24 units
Gap between cards	12 to 16 px	12 dp (Row2, spacedBy(12.dp))	12 units
Section top spacing	32 px (pt-8)	20 dp (SectionTitle)	20 units (HudTitle margin)

Context	Web console	Android	HUD
Max content width	1440 px (`max-w-[1440px]`)	Full width	600 units (HUD) / 540 units (phone)

8.5.2 Radius

Table 217. Radius tokens

Token	Value	Use
`--radius-xs`	6 px	Small tags, kbd
`--radius-sm`	10 px	Skeletons
`--radius-md`	14 px	Inputs, selects
`--radius-lg`	18 px	Inset cards, menus
`--radius-xl`	24 px	Cards, panels, empty states
`--radius-2xl`	32 px	Centered dialogs
Pill	9999 px	Buttons, chips, capsules, icon buttons
Android `CardShape`	22 dp	`HCard`
Android chip / progress	50 %	`Chip`, `ProgressBar`
HUD row	26 units	`HudItem`
HUD alert	34 units	`AlertInterrupt`

8.5.3 Elevation

Elevation on dark surfaces comes from a top-edge sheen and graded surface colours more than from shadows.

Table 218. Elevation tokens (globals.css)

Token	Dark	Light	Use
`--sheen`	inset 1 px white 4.5 %	soft 1 to 2 px shadow	Every card and capsule
`--shadow-1`	1 px plus 8 px blur, black 20 to 30 %	slate 5 %	Resting raised elements
`--shadow-2`	6 px plus 28 px, black 30 to 35 %	slate 6 to 9 %	Hover lift, glass panels
`--shadow-3`	12 px plus 72 px, black 35 to 55 %	slate 8 to 16 %	Menus, dialogs, mobile tab bar
Brand panel glow	18 px 48 px Core Blue 22 %	18 %	`.card-lime` focal panel

Android cards are flat (`tonalElevation = 0` on the navigation bar, no shadow on `HCard`), separated by a 1 dp `Hx.Line` border. The HUD uses no shadows at all, since shadows are invisible on an additive display.

8.6 Iconography

Table 219. Icon systems

Surface	Library	Style	Default sizes	Status
Console and website	`lucide-react` 1.57.0	1.5 to 2 px stroke, rounded	19 px navigation, 16 to 18 px buttons, 14 px inline	**Built**
Glasses HUD and phone mode	`lucide-react`	Same	22 to 28 units	**Built**
Android	`androidx.compose.material:material-icons-extended`	Outlined set (`Icons.Outlined.*`, `Icons.AutoMirrored.Outlined.*`)	18 to 24 dp	**Built**

The two libraries are visually close (outlined, geometric), which keeps the surfaces consistent without sharing assets. Use the same concept for the same meaning:

Table 220. Icon concept mapping across surfaces

Concept	Lucide (web, HUD)	Material Outlined (Android)
Home / overview	`LayoutGrid`, `Glasses` (HUD home tab)	`Dashboard`
Alarms	`Siren` (console nav), `AlertTriangle` (HUD)	`Warning`
Inspections	`ClipboardCheck`	`FactCheck`
Maintenance	`Wrench`	`Build`
Audit scan	`ScanLine`	`DocumentScanner`
Voice / Hydris AI	`Mic`, `Sparkles`	`AutoAwesome`, `Mic`, `MicOff`
Glasses / devices	`Glasses`	`Visibility`
Back	`ChevronLeft`	`AutoMirrored.Outlined.ArrowBack`
Camera	n/a	`PhotoCamera`
Safety notice	`ShieldAlert`	n/a
Not permitted	`Ban`	n/a

Rules: icons never stand alone for an action on the HUD or phone (always with a label); icon-only buttons on the web carry an `aria-label` and a `Tooltip` (`IconButton` in `ui/index.tsx`); decorative icons are `aria-hidden`.

8.7 Component library

8.7.1 Component inventory

Table 221. Component inventory

Component	Purpose	Surfaces	States	Status
`Button`, `ButtonLink`	Actions: primary, secondary, ghost, light, cobalt, danger, danger-solid	Web	default, hover, active, focus, disabled, busy (`aria-busy`)	**Built**

Component	Purpose	Surfaces	States	Status
`IconButton`	Circular icon actions with label and badge	Web	default, light, lime, ghost, outline, on-paper, danger; sm, md, lg	**Built**
Material `Button`, `OutlinedButton`	Primary and secondary actions, 52 to 54 dp tall	Android	enabled, disabled (busy)	**Built**
`Card`	Section container (dark, paper, lime focal)	Web	default, tight, hover lift	**Built**
`HCard`	Card with optional click and accent border	Android	default, clickable, accent	**Built**
`KpiCard`	KPI island: label, value plus unit, sub line, open link	Web	paper, dark, lime; critical, high, ok accent	**Built**
`KpiTile`	KPI tile: label, 34 sp value, sub line, icon	Android	accent colour, clickable	**Built**
`HudMetric`	Large reading with unit and label	HUD	tone colour	**Built**
`Chip`, `SeverityChip`, `StateChip`	Status and category labels	Web	10 tones including synthetic	**Built**
`Chip` (Compose)	Status label	Android	tinted, filled	**Partial** (filled contrast)
`HudPill`	Status pill on HUD	HUD	line, lime, critical, high, ok, medium	**Partial** (lime contrast)
`FreshnessBadge`	Live, delayed, stale, no data with time	Web	4 freshness states, pulsing live dot	**Built**
`SyntheticBadge`	Marks simulator data	Web, HUD text, Android chip	n/a	**Built**
`Capsule`	Segmented control and filters	Web	selected, hover, counts, lime variant	**Built**
`Input`, `PasswordInput`, `Select`, `Textarea`, `Field`	Forms with label, hint, error	Web	hover, focus, invalid, disabled	**Built**
`Switch`	Boolean settings	Web	on, off, disabled	**Built**
`DataTable`	Sortable, clickable tables	Web	hover, focus, selected, dense, empty	**Built**
`Timeline`	Event history	Web	lime, critical, ok, muted, cobalt	**Built**
`KeyValues`	Definition grid	Web	1 to 3 columns	**Built**
`Progress`	Progress bar with ARIA	Web	lime, ok, critical	**Built**
`ProgressBar`	6 dp progress bar	Android	colour	**Built**
`Dialog`, `ConfirmDialog`	Modal, side panel, bottom sheet; confirmation with optional required note	Web	open, destructive, note required	**Built**
Toasts (`ToastProvider`, `useToast`)	Transient confirmations with timer	Web	info, error	**Built**

Component	Purpose	Surfaces	States	Status
`EmptyState`, `ErrorState`, `LoadingBlock`, `Skeleton`, `QueryView`	Data states	Web	empty, not found, forbidden, error with retry, loading	**Built**
`Loading`, `ErrorNote`	Data states	Android	spinner, error note	**Built**
`HudItem`	Focusable HUD row	HUD	active (focus), disabled, tone	**Built**
`HudTitle`	Screen title with eyebrow	HUD	n/a	**Built**
`PhoneTabBar`	Bottom tab bar in phone mode	Glasses phone mode	active, back, badge	**Built**
`HintBar`	Control hints aligned to handedness	HUD	left, right	**Built**
`AlertInterrupt`	Full-screen critical or high alarm	HUD	critical, high, synthetic	**Partial** (button contrast)
`GuidedCapture`	Live A4 framing with guidance and auto-capture	Glasses, web	8 guidance codes, ready progress	**Built**
Web charts (`ui/charts.tsx`)	Sparkline, Sparkbars, BarStrip, Gauge, RadialGauge, Donut, StackedBar, BarList, Heatmap, BulletBar, RangeBar, Delta, MiniTrend, Funnel, StatusStrip, TrendChart	Web	empty state, screen-reader table	**Built**
Android charts	`Sparkline`, `LineChart`, `Bars`, `Donut` (Canvas)	Android	fewer than 2 points draws nothing	**Partial** (no accessible description)
`ThemeToggle`	System, light, dark	Web	checked, compact	**Built**
Haptic feedback	Confirm critical actions by vibration	Android, glasses	n/a	**Planned**

8.7.2 Cards

Web `Card` (`ui/index.tsx`, `.card` in globals.css): surface background, 1 px line border, 24 px radius, 24 px padding, sheen. Three tones: default dark; `paper` (light island on a dark page, used for KPIs so numbers pop); `lime`, the focal brand panel, a radial gradient of Core Blue with white type. Use at most **one** focal panel per view. Interactive cards opt into `.lift` (2 px rise on hover, settle on press).

Android `HCard` (`Components.kt`): `Hx.Surface`, 22 dp radius, 1 dp `Hx.Line` border, 16 dp padding. Passing `accent` tints the border at 55 % alpha; the home screen uses this for critical alarm cards and the alarm detail uses a Signal accent for the recommended action card.

Figure 161. *Android home: KPI tiles, accent-bordered alarm cards and sparkline rows*

8.7.3 KPI tiles

Table 222. KPI tile specification

Property	Web `kpiCard`	Android `kpiTile`	HUD `hudMetric`
Minimum height	124 px, 148 px from `sm`	Content height	Content height
Label	13.5 px, 75 % opacity, top left	`bodyMedium`, Text2, single line	17, white 60 %, below value
Value	`t-kpi` (38 to 54 px, light 300)	34 sp SemiBold	52 light
Unit	15 px, 60 % opacity, baseline aligned	Inline in value or sub	20, white 60 %
Accent	`kpi-accent-critical`, `-high`, `-ok`; deepened on paper for AA	`accent` colours the value	`tone` colours the value

Property	Web `kpiCard`	Android `pillRow`	HUD `hudMetric`
Link	`OpenLink` arrow, top right	Whole tile clickable	Not interactive
Sub line	13 px, 70 % opacity	`bodySmall`, Muted	n/a

Rules: a KPI value is a fact with a unit and a time context ("in the last 24 h"); never show a KPI computed from stale data without saying so; never use the accent colour unless the number itself is the alert.

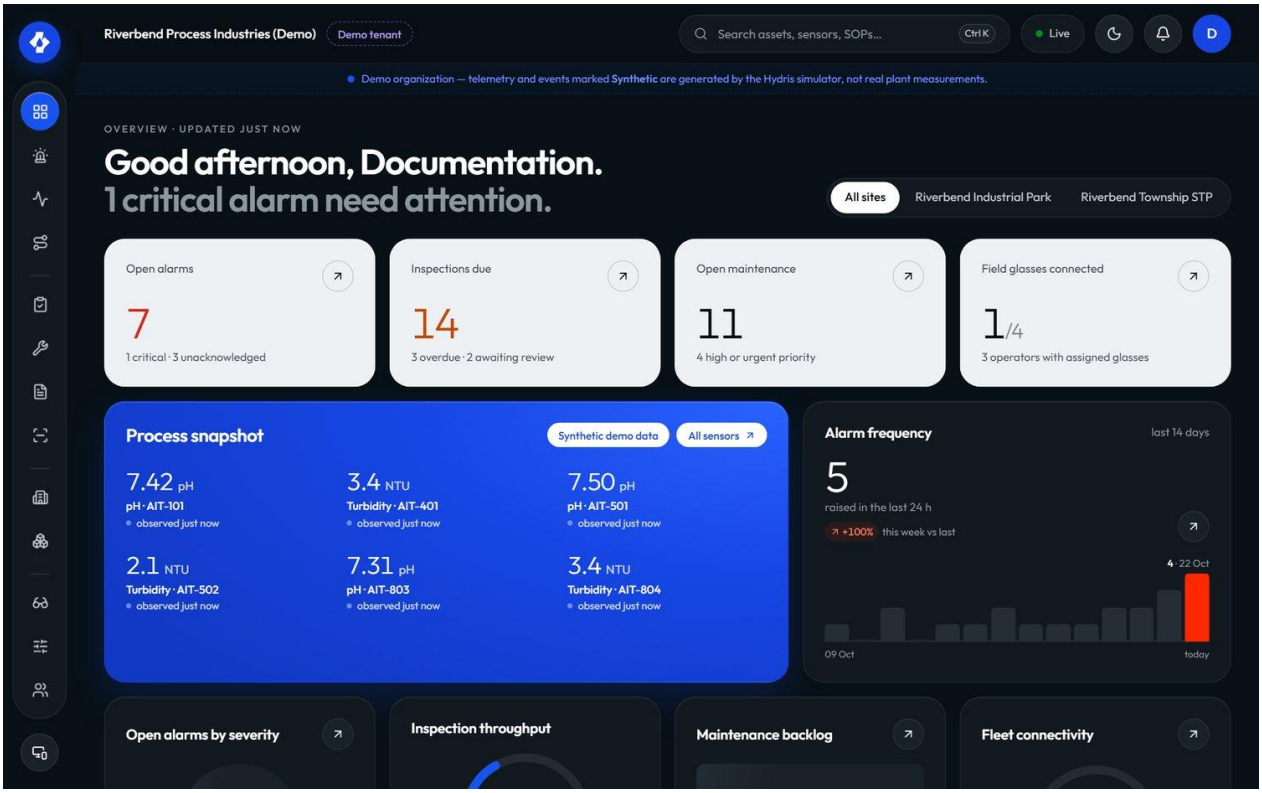

Figure 162. *Console overview: paper KPI islands, focal blue panel and live charts*

8.7.4 Chips and badges

Web chips are 28 px tall pills, 12.5 px medium text, with a 1 px tinted border and a 12 to 15 % tinted fill per tone. Clickable chips get a 44 px hit area through an `::after` pseudo-element. `SeverityChip` adds a dot in the severity colour plus the word. `chip-synthetic` is a dashed purple outline so simulated data is recognisable even in greyscale.

Android `Chip` is 12 sp medium text with 10 by 4 dp padding on a 14 % tint, or a solid fill when `filled = true` (alarm severity on the alarm detail screen). The HUD `HudPill` is 34 units tall with a 2 unit border and 18 unit text; on the HUD, outlines are preferred to fills.

8.7.5 Buttons

Table 223. Button specification (globals.css)

Variant	Fill	Text	Hover	Use
`btn-primary`	Signal `#1954F1`	White	Core Blue plus blue glow; pressed Deep Blue	One per view: the main record action
`btn-secondary`	surface-2	text	surface-3	Secondary actions
`btn-ghost`	transparent	text-2	surface-2	Tertiary, toolbars

Variant	Fill	Text	Hover	Use
`btn-light`	White	ink	paper	On the blue focal panel
`btn-cobalt`	Core Blue	White	Deep Blue	Brand emphasis on the website
`btn-danger`	transparent, orange border	critical-text	12 % orange fill	Reversible destructive actions
`btn-danger-solid`	`#E02400`	White	`#C41F00`	Irreversible destructive confirmation

Sizes: `btn-sm` 36 px (with an invisible 44 px hit area), default 44 px, `btn-lg` 52 px. All buttons scale to 0.97 on press, show 50 % opacity when disabled (the disabled primary turns into a neutral surface so it reads as "not yet" rather than a faded blue), and set `aria-busy` while an action is in flight. Android primary actions are Material `Button` at 52 to 54 dp height and full width in task flows (alarm detail, scan review), with `OutlinedButton` for secondary actions.

8.7.6 Inputs and forms

Inputs are at least 46 px tall, 14 px radius, 14.5 px text, on `--color-field` with a `line-2` border. Focus shows a Signal Blue border plus a 4 px Signal glow at 22 %; invalid inputs show a critical border and a `field-error` line prefixed by a small critical dot (so the error is not colour-only). `Field` always renders a visible label, an optional hint and an error, linked by `id`. Glove-friendly rules: never rely on small inline controls in the field apps; on Android use full-width text fields (the Ask screen uses a 26 dp radius pill field) and full-width buttons.

8.7.7 Tables

`.table` (globals.css) and `DataTable` (`ui/index.tsx`): sticky uppercase 11.5 px headers, 13 px row padding (10 px dense), hover wash, a 2 px Signal inset on the first cell of clickable rows, keyboard focus outline, and a 14 % Signal fill for the selected row. Sorting uses `aria-sort` on the header. Tables belong to the console only; phones and the HUD use lists.

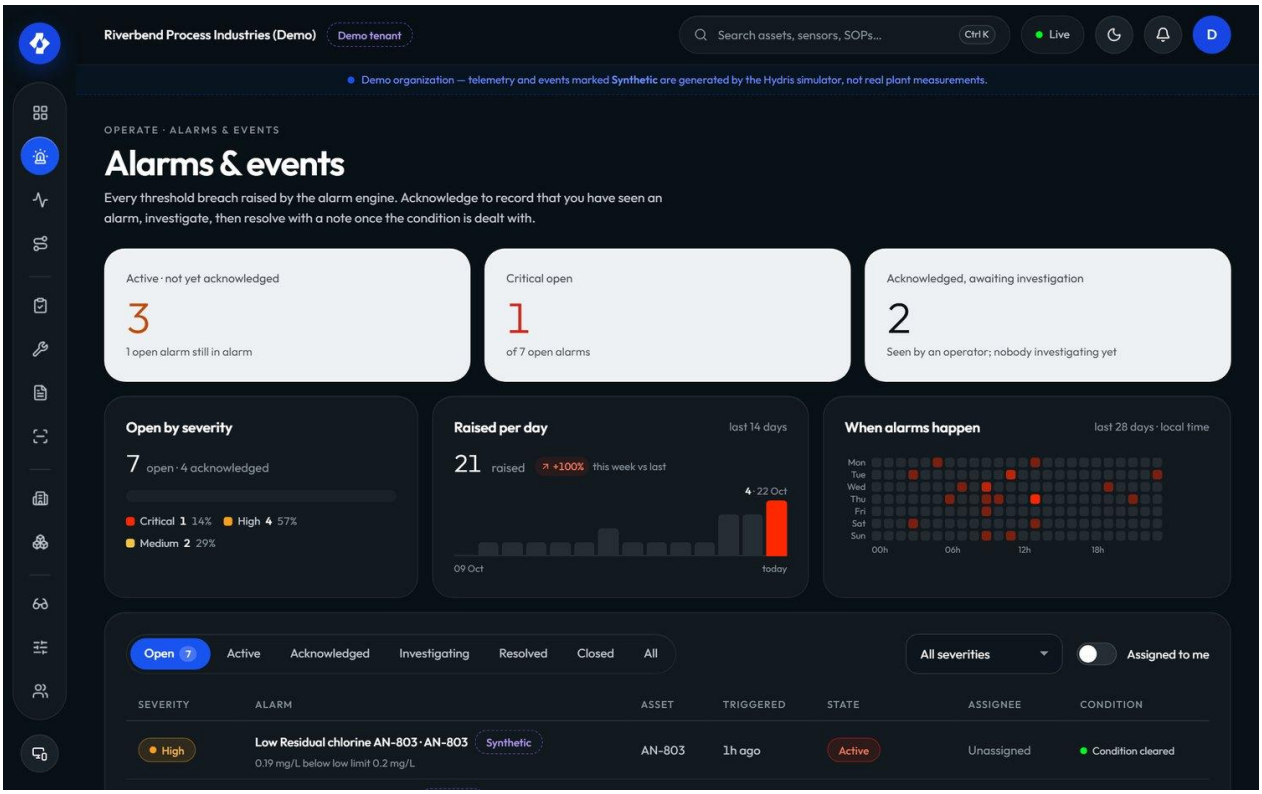

Figure 163. *Console alarms list: severity chips, state chips and clickable rows*

8.7.8 Alarm rows

An alarm row shows, in this order: severity (chip or coloured border plus word), title, value with unit, asset tag, state, and time since it was raised. Synthetic alarms add the synthetic marker.

Table 224. Alarm row anatomy by surface

Element	Console table row	Android list card	HUD `HudItem`
Severity	`SeverityChip` (dot plus word)	Accent border (critical) plus severity colour	`tone` colours the text, pill on detail
Title	Primary text	`titleMedium`	23 unit row text
Value and unit	Column	Meta line	Detail screen, 24 unit
State	`StateChip`	Chip on detail	Meta line ("active · 3m ago")
Time	Relative	`ago()` (just now, 5m ago)	`timeAgo`
Synthetic	`SyntheticBadge`	Purple chip	"· synthetic"
Open	Row click	Card click to `alarm/{id}`	Select to `alert` screen

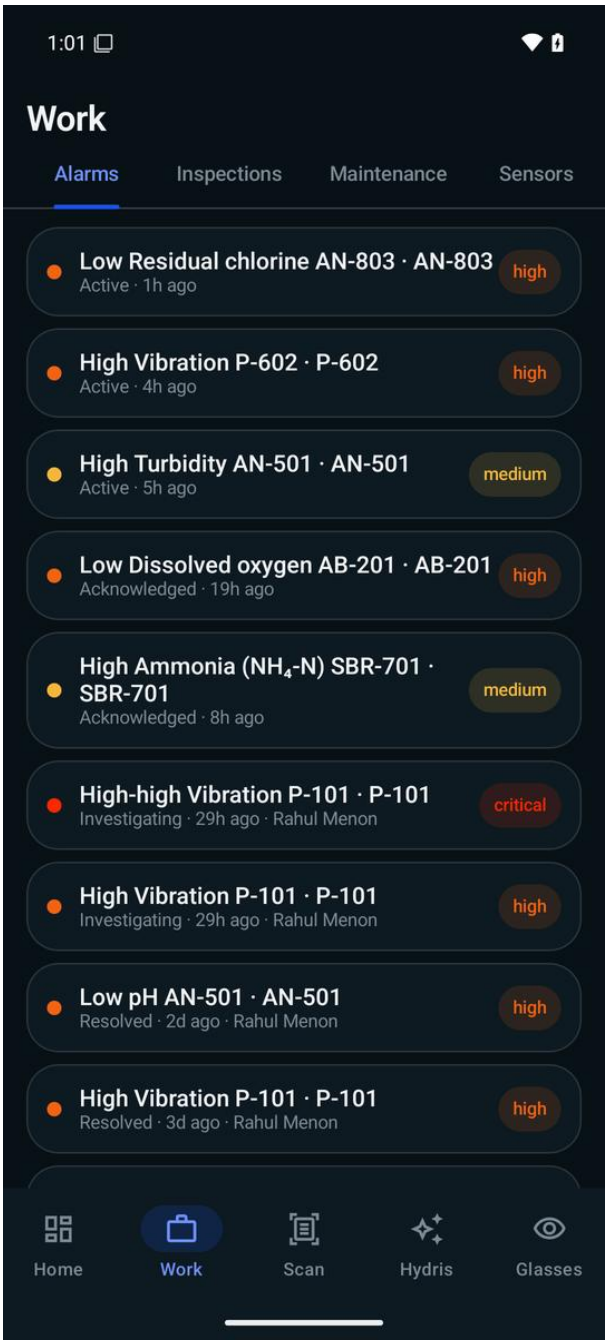

Figure 164. *Android work list with alarms grouped by severity*

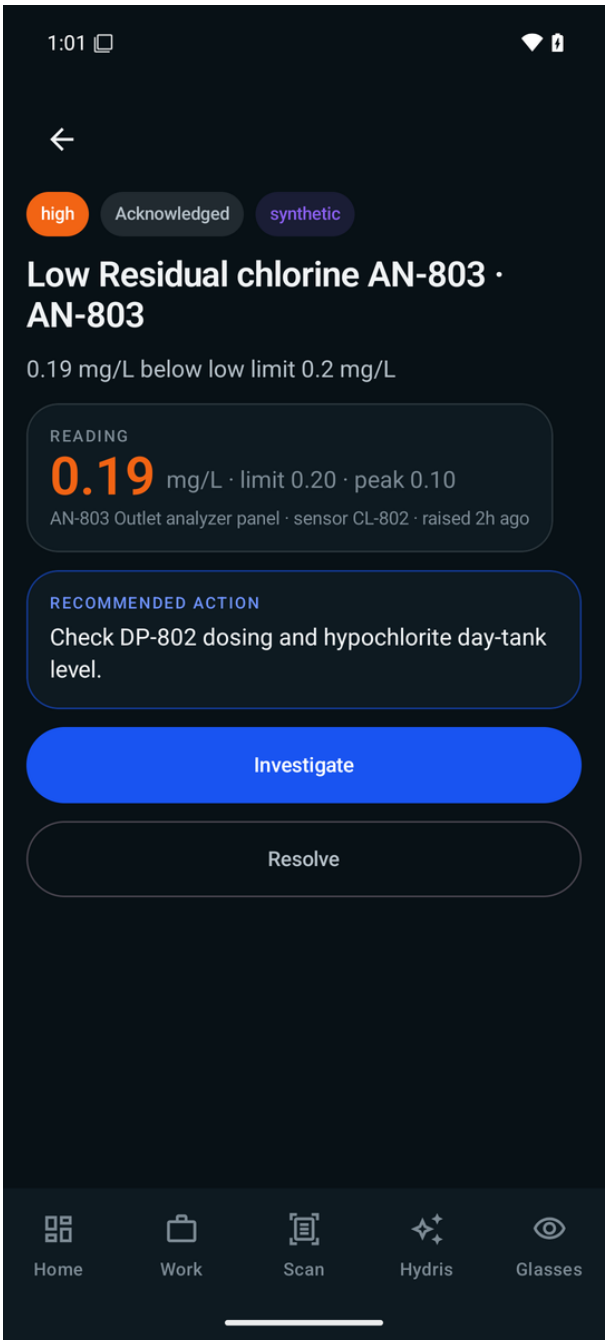

Figure 165. *Android alarm detail: reading, limit, recommended action and the next lifecycle action*

8.7.9 Charts

Web charts (`apps/web/src/ui/charts.tsx`) are hand-built SVG microcharts, plus `TrendChart` built on Recharts `AreaChart`. Every chart reads colours from CSS variables, is an accessible image (`role="img"` with `aria-label`), can render a screen-reader data table (`SrTable`), and shows an explicit empty state instead of inventing numbers. Android charts are drawn on Compose `Canvas` without a chart library (`Components.kt`; the mobile spec's Vico choice was not adopted).

Table 225. Chart specifications

Chart	Web	Android	Specification
Sparkline	`Sparkline`: default 40 px tall, stroke `--color-lime`	`Sparkline`: 2 dp round stroke, range padded 7.5 % top and bottom	No axes; trend only; always next to the current value

Chart	Web	Android	Specification
Line / trend	`TrendChart` (Recharts area, 260 px, thresholds as reference lines, typical band); `MiniTrend` with band and limits	`LineChart`: typical range shaded in `OkSoft` 8 %, 3 gridlines, 28 % gradient fill, end-point dot	Thresholds and typical range always drawn when known
Bars	`Sparkbars` (peak emphasis), `BarStrip`, `BarList`, `StackedBar`	`Bars`: 4 dp gaps, 4 dp radius, last bar full colour, others 45 %	Highlight only the bar that matters (latest or peak)
Donut	`Donut`: 156 px, 14 px ring, centre value, legend with percentages	`Donut`: 14 dp stroke, 2 degree gaps, starts at 12 o'clock, centre slot	Compositions only; at most 5 segments
Gauges	`Gauge`, `RadialGauge` with marks and band	n/a	Single value against a range
Heatmap	`Heatmap` (week by hour)	n/a	Event density
Progress	`Progress`, `BulletBar` (target)	`ProgressBar`	Fraction of a known total

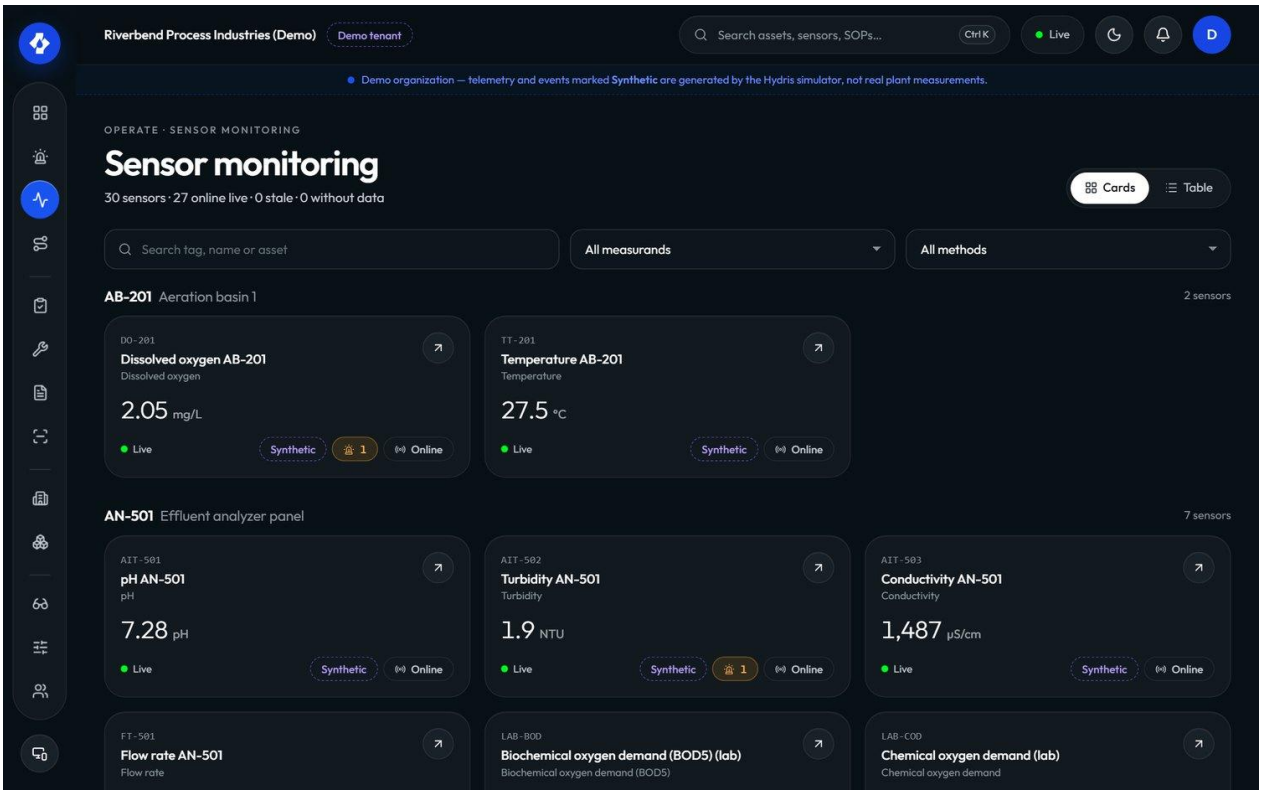

Figure 166. *Console telemetry: trend chart with thresholds and freshness*

8.7.10 Empty, loading and error states

Table 226. Data state patterns

State	Web	Android	HUD
Loading	`LoadingBlock` skeleton rows with shimmer, `role="status"`; `QueryView` keeps last good data while refetching	`Loading`: Signal Blue spinner	"Generating...", "Context will load when the connection returns"
Empty	`EmptyState`: dashed border, concentric art, title, body, action	Inline text ("No ..."), per screen	Plain 22 unit sentence ("This alert is no longer open.")
Not found	`ErrorState` turns 404 and invalid ids into "We couldn't find that record" with a back link	n/a	n/a

State	Web	Android	HUD
Forbidden	"You do not have access to this" with admin guidance	Server error text	`NotPermitted` screen and `Blocked` notes
Error	Red-tinted `role="alert"` panel with message and Retry	`ErrorNote`: High tint panel with message	Banner (`role="status"`) in critical or high tone
Offline	Realtime status in shell	Error notes	Offline strip plus queued counter

8.8 Data visualisation guidelines

1. **Show the value, its unit and its time together.** A number without freshness is not shown. Use `FreshnessBadge` (web) or `Hx.freshness()` (Android: live `OkSoft`, delayed `Medium`, stale `High`, no data `Muted`).
2. **Never plot invented data.** Missing readings are gaps, not zeros (`freshness.ts`); charts render their empty state ("No data", "No records").
3. **Limits and typical ranges are context, not decoration.** Draw thresholds as reference lines and typical ranges as a faint band (Android `LineChart`, web `MiniTrend` and `TrendChart`).
4. **One highlight per chart.** Emphasise the latest or peak bar; mute the rest (`Sparkbars emphasise`, Android `Bars` 45 % alpha).
5. **Severity colours only for severity.** Series colours come from the brand set (Signal, Core Blue, Insight Purple, text-2) in that order; Alert Orange is reserved for critical.
6. **Every chart has a text equivalent.** Web: `aria-label` plus optional `SrTable`. Android: **Planned** (`Modifier.semantics { contentDescription }` on chart canvases).
7. **Tabular figures** for every numeric axis, legend and value.
8. **HUD: no charts beyond a single sparkline.** On the glasses, a reading and its state word beat any chart.

8.9 Layout grids

8.9.1 Console desktop

The console shell (`apps/web/src/console/Shell.tsx`) has a fixed 84 px icon rail on the left from the `md` breakpoint, a sticky blurred header with the search trigger (Ctrl or Cmd K command palette), theme toggle and account menu, an optional demo banner, and a main column capped at 1440 px with 32 px gutters. Below `md` the rail becomes a floating bottom capsule bar with four primary items (Overview, Alarms, Sensors, Inspections) and More, which opens an "All modules" bottom sheet.

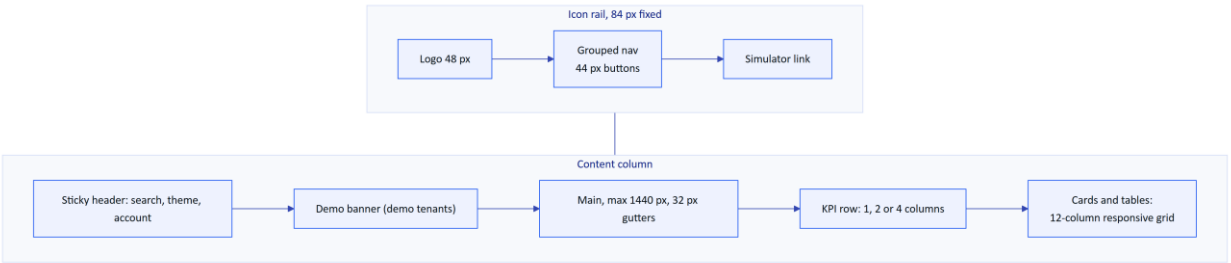

Figure 167. *Console desktop layout regions (md breakpoint and wider)*

Table 227. Console breakpoints and grid behaviour

Breakpoint (Tailwind)	Navigation	KPI row	Content grid
Below `sm` (640 px)	Bottom capsule bar plus More sheet	1 to 2 columns	Single column; dialogs become bottom sheets
`sm` to `md`	Bottom capsule bar	2 columns	Single column; centred dialogs
`md` (768 px) and up	84 px rail	2 columns	Two-column card layouts begin
`lg` (1024 px)	Rail	2 to 3 columns	Main plus side column
`xl` (1280 px) and up	Rail	4 columns (`xl:grid-cols-4`)	Full dashboard grid, max 1440 px

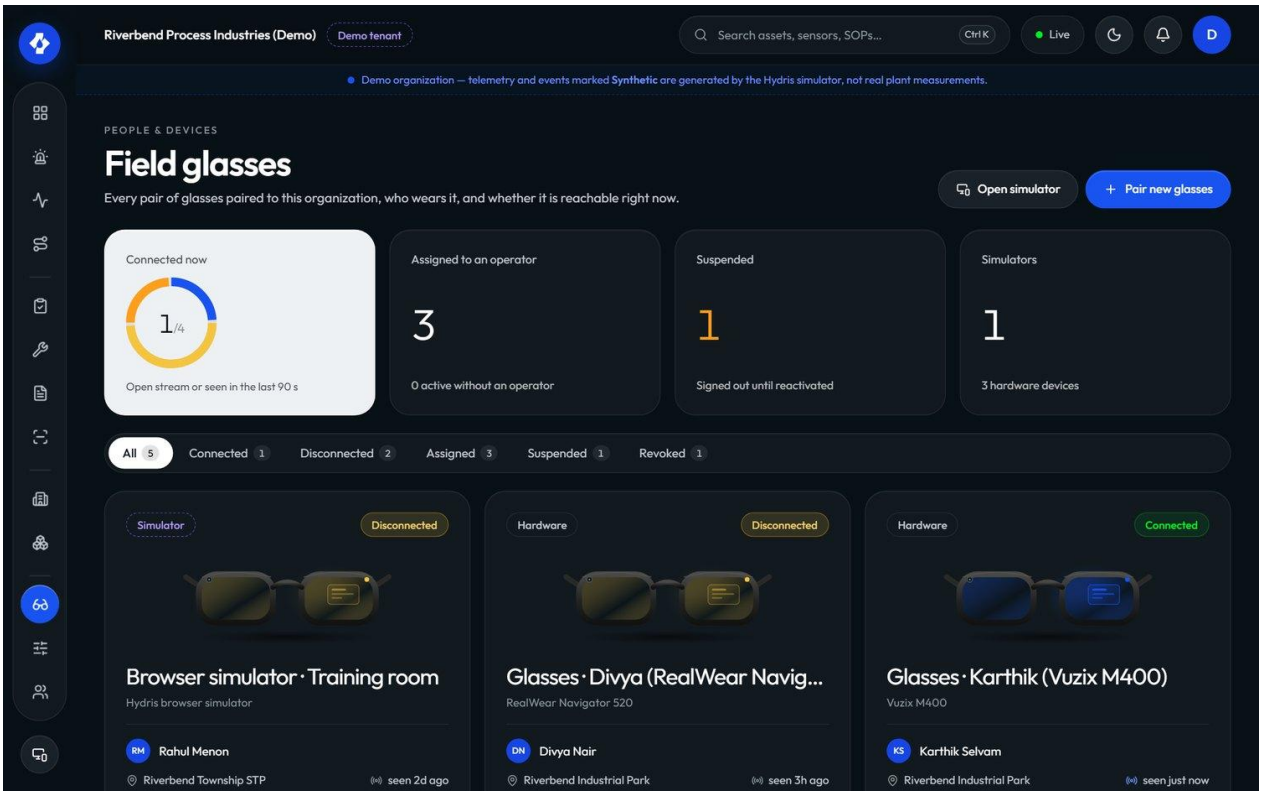

Figure 168. *Console devices: table layout inside the 1440 px content column*

8.9.2 Phone (glasses phone mode and Android)

The glasses web app switches to **phone mode** for portrait phones, the Ray-Ban Meta companion app and any page with `?mode=phone` (`useViewportLayout` in `hud.tsx`). Phone mode keeps a 540-unit design width (`PHONE_DESIGN_WIDTH`), scales by `window width / 540`, and grows to the full device height so screens fill the phone instead of sitting in a letterboxed square. Safe areas are respected with `env(safe-area-inset-top)` on the status bar and `env(safe-area-inset-bottom)` on the tab bar.

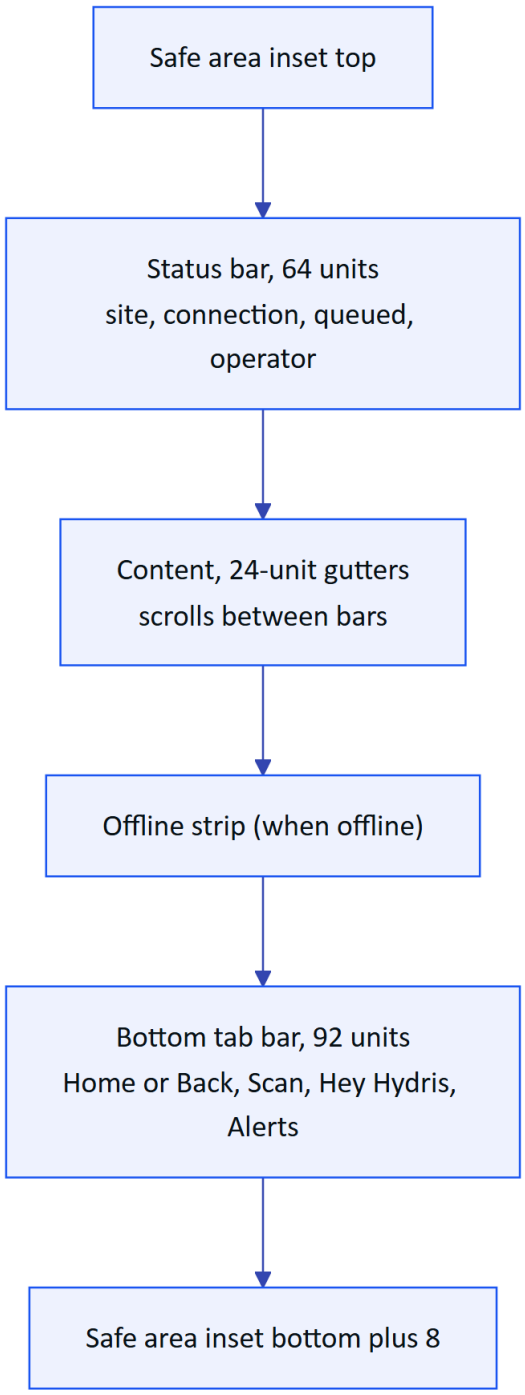

Figure 169. *Phone mode layout, 540-unit design width with safe areas*

The Android app follows the same structure natively: `statusBarsPadding()`, 16 dp horizontal padding, a `LazyColumn` of cards with 12 dp spacing, and a Material `NavigationBar` with five tabs (Home, Work, Scan, Hydris, Glasses) on `Hx.Surface`, selected items in `SignalText` with an 18 % Signal indicator (`ui/Root.kt`).

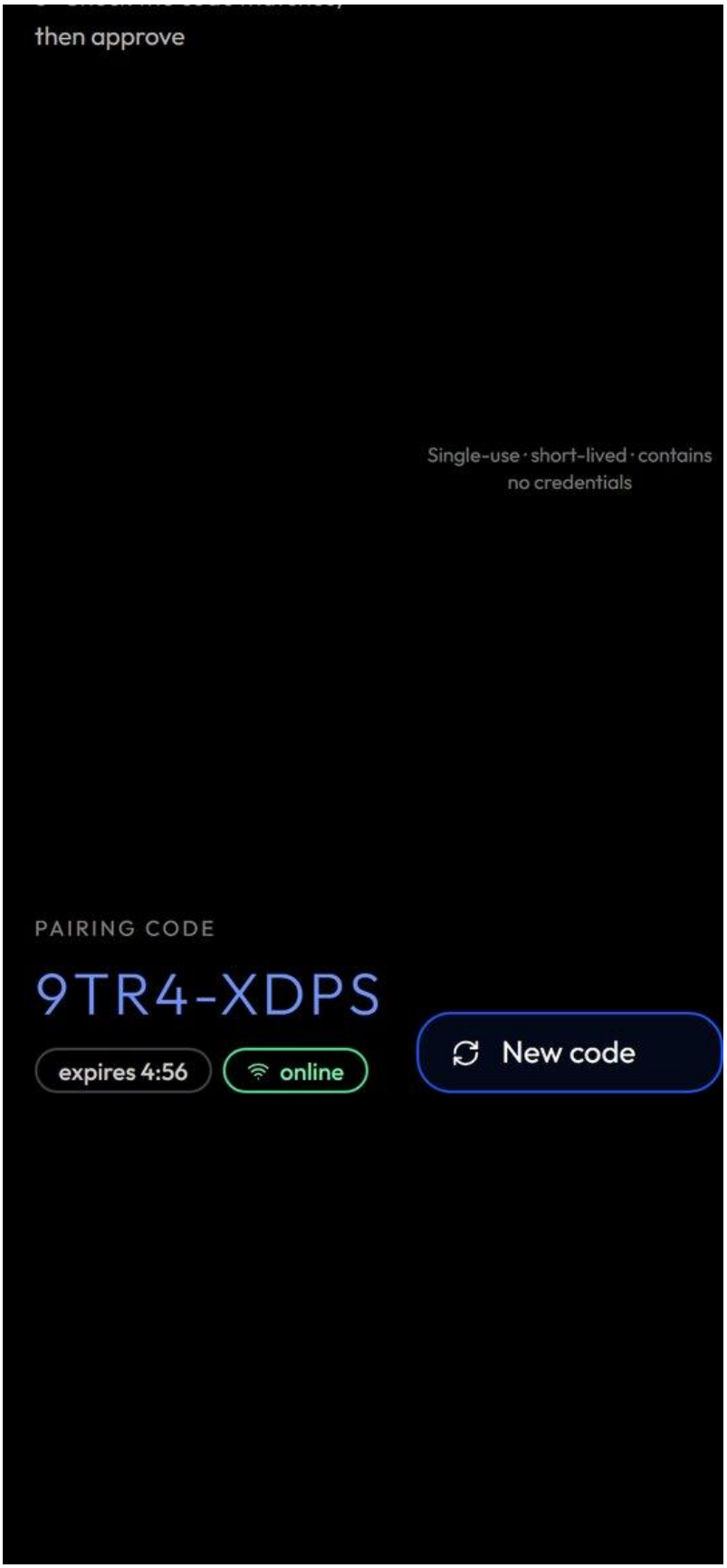

Figure 170. *Glasses app in phone mode: pairing screen on a phone*

8.9.3 HUD 600 by 600

The HUD canvas is 600 by 600 units scaled to fit (`useFitScale`, `useViewportLayout`). The simulator also offers 640 by 480 and 480 by 480 viewports (`apps/web/src/app/simulator/page.tsx`) to test other display shapes.

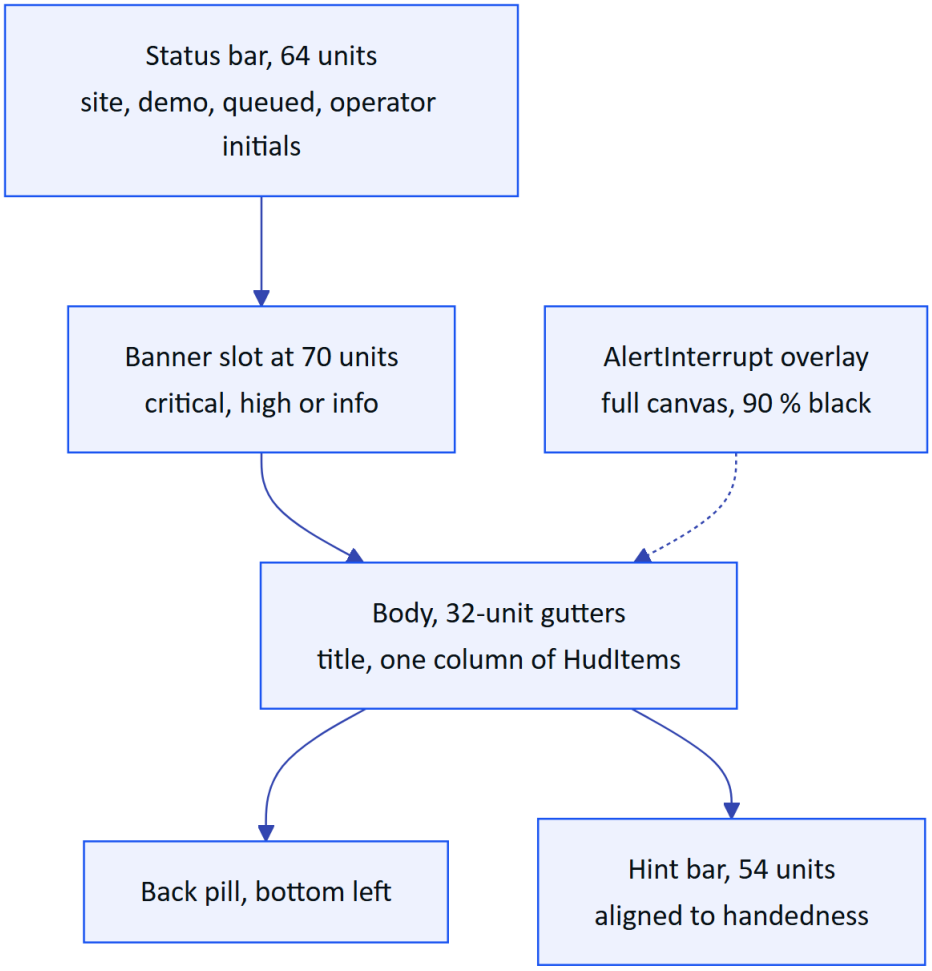

Figure 171. *HUD 600 by 600 layout regions*

Table 228. HUD layout rules

Rule	Value	Source
Canvas	600 by 600 units, black (renders transparent)	`hud.tsx`
Gutters	32 units (HUD), 24 units (phone)	`GlassesApp.tsx`
Columns	One	`hud.tsx` header comment
Focus	One focused `HudItem` with Signal border and 10 % Signal fill	`HudItem`
Row height	14 units vertical padding (comfortable), 9 units (compact)	`HudItem`, `layout.density`
Primary actions	One per screen	`hud.tsx` header comment
Hint bar	Right or left aligned per `layout.handedness`	`HintBar`
Text scale	`layout.textScale` 0.85 to 1.4	`uiprofile.ts`

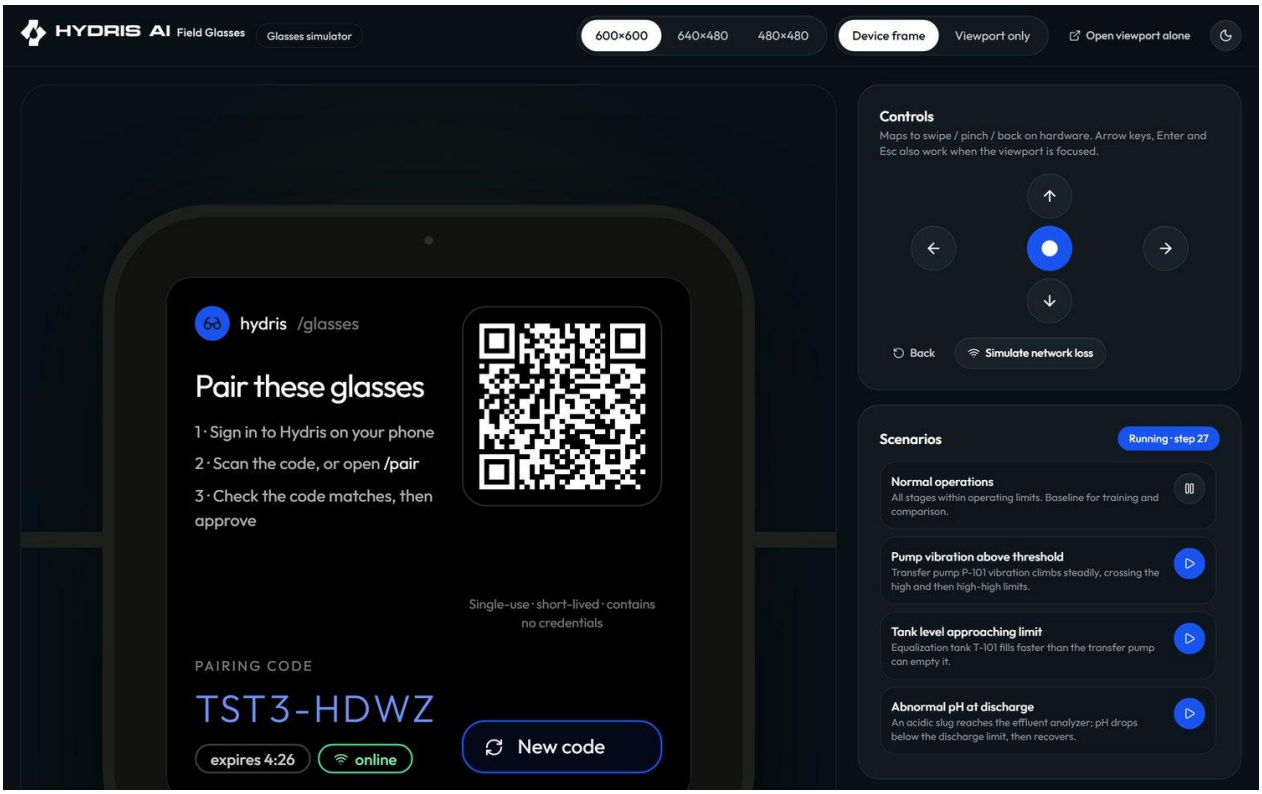

Figure 172. *Glasses simulator: HUD viewport, device frame, input pad and event console*

8.9.4 UI profiles shape the layout

Administrators control what the glasses show through versioned UI profiles (`packages/domain/src/uiprofile.ts`): 12 screens (Home and Device and session can never be disabled), 4 home widgets, 8 actions, a measurement allow-list and per-asset-type order, plus layout density, text scale and handedness. The most specific published assignment wins entirely: device, then user, then role, then organisation, then the built-in default. Designers must design every glasses screen for both densities, all four text scales and both hands.

8.10 Navigation patterns

8.10.1 Console information architecture

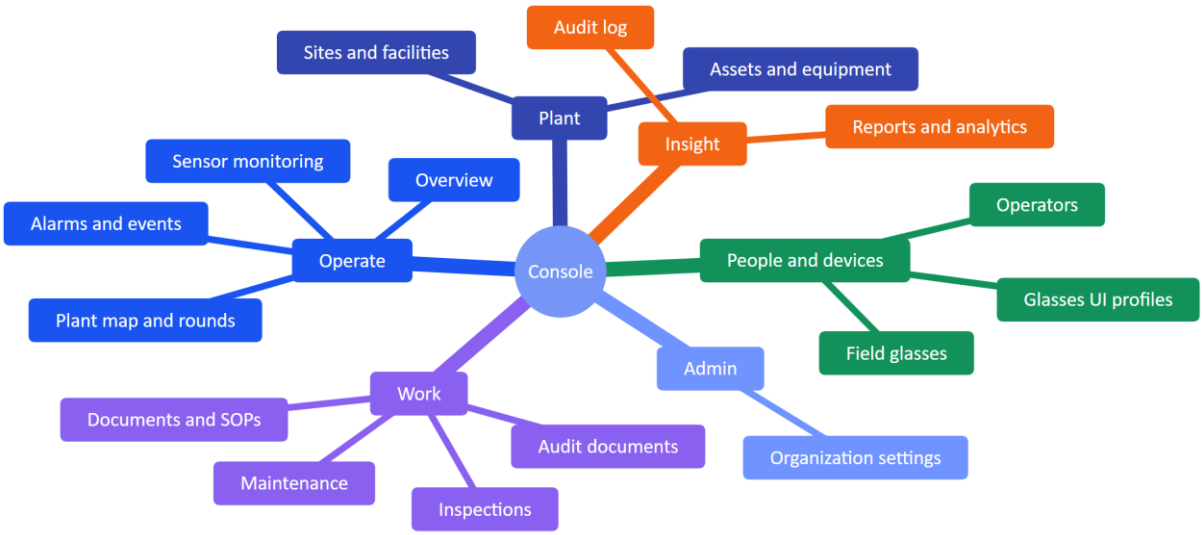

Figure 173. *Console information architecture (navigation groups in Shell.tsx)*

Navigation items are filtered by permission (`perm` on each `NavItem`), so an operator never sees a module they cannot open. The command palette searches assets, sensors, documents and people from every page.

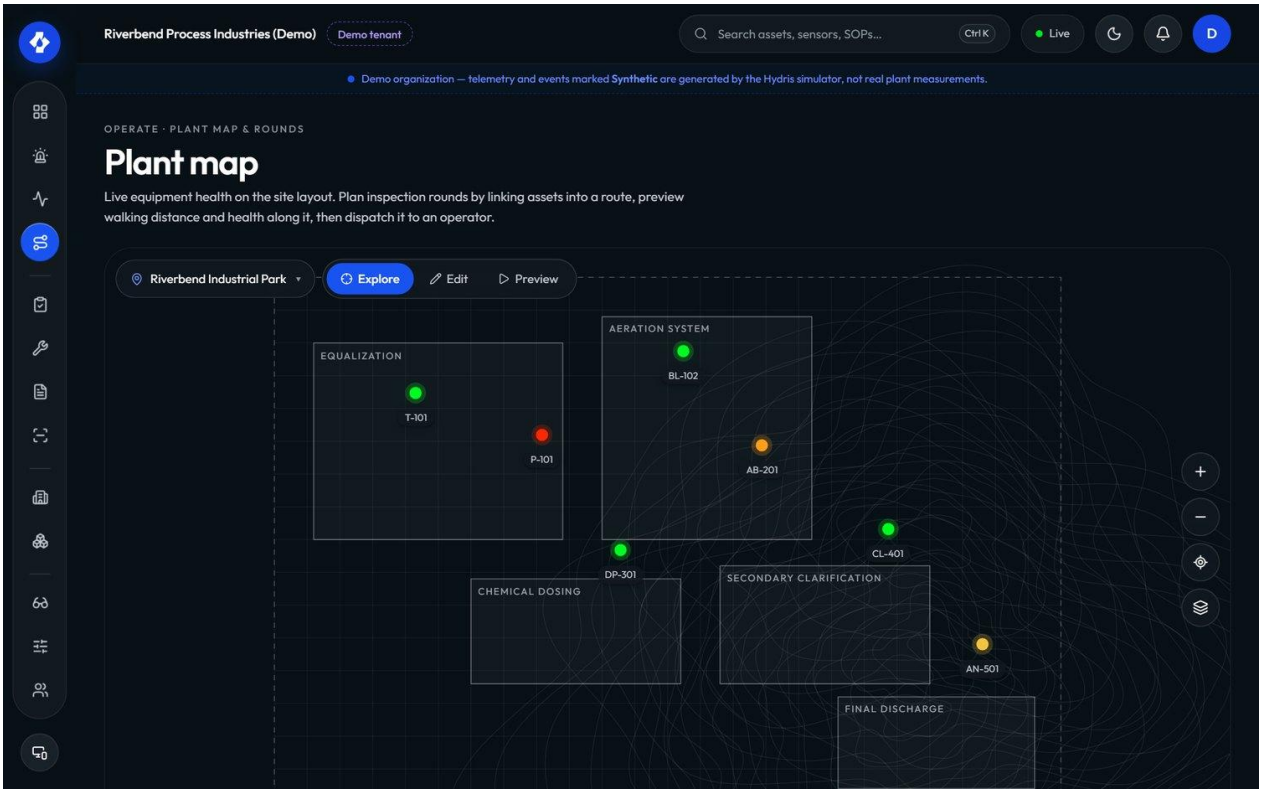

Figure 174. *Console plant map and rounds*

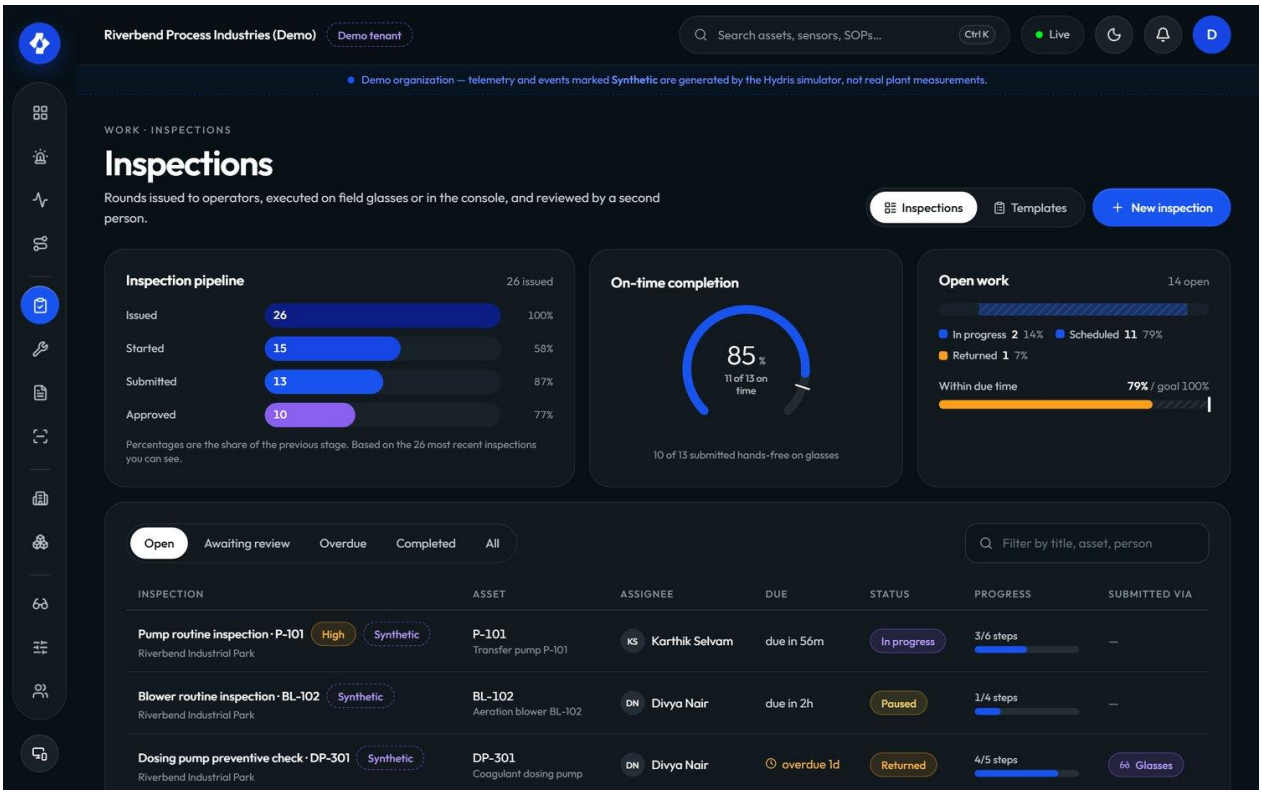

Figure 175. *Console inspections*

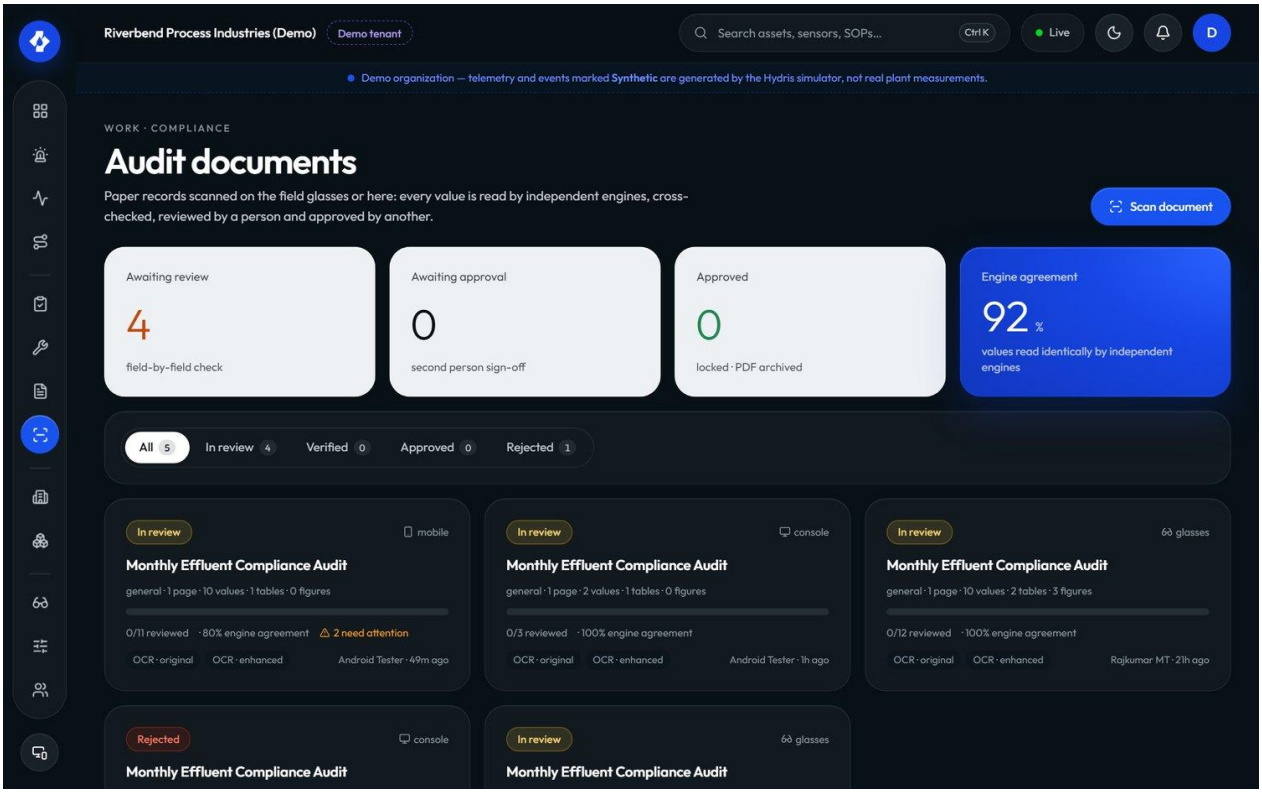

Figure 176. *Console audit documents: review and approval of scanned records*

8.10.2 Glasses app information architecture

The glasses app is a stack: Home is the root, every selection pushes a screen, and Back pops. Screens are gated by the UI profile (`screenAllowed` logic at the top of `GlassesApp.tsx`).

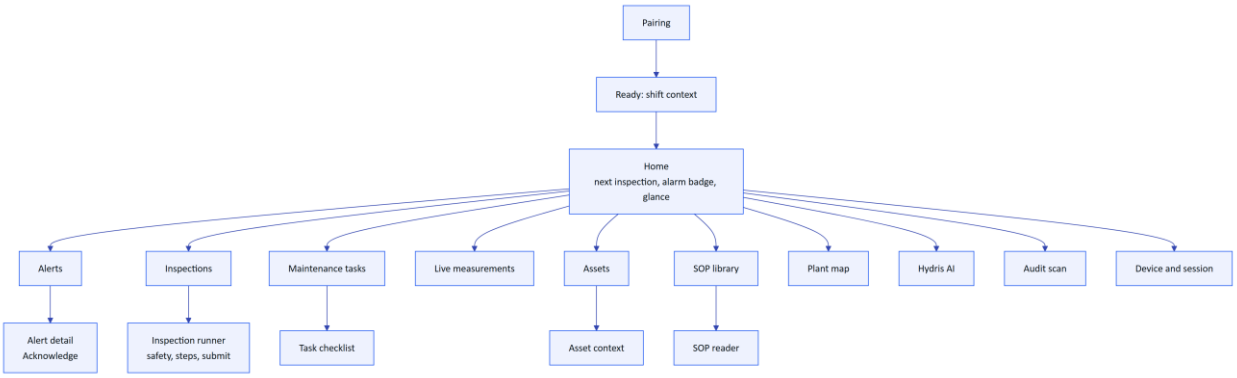

Figure 177. *Glasses app navigation stack (profile-gated screens)*

8.10.3 Android app navigation

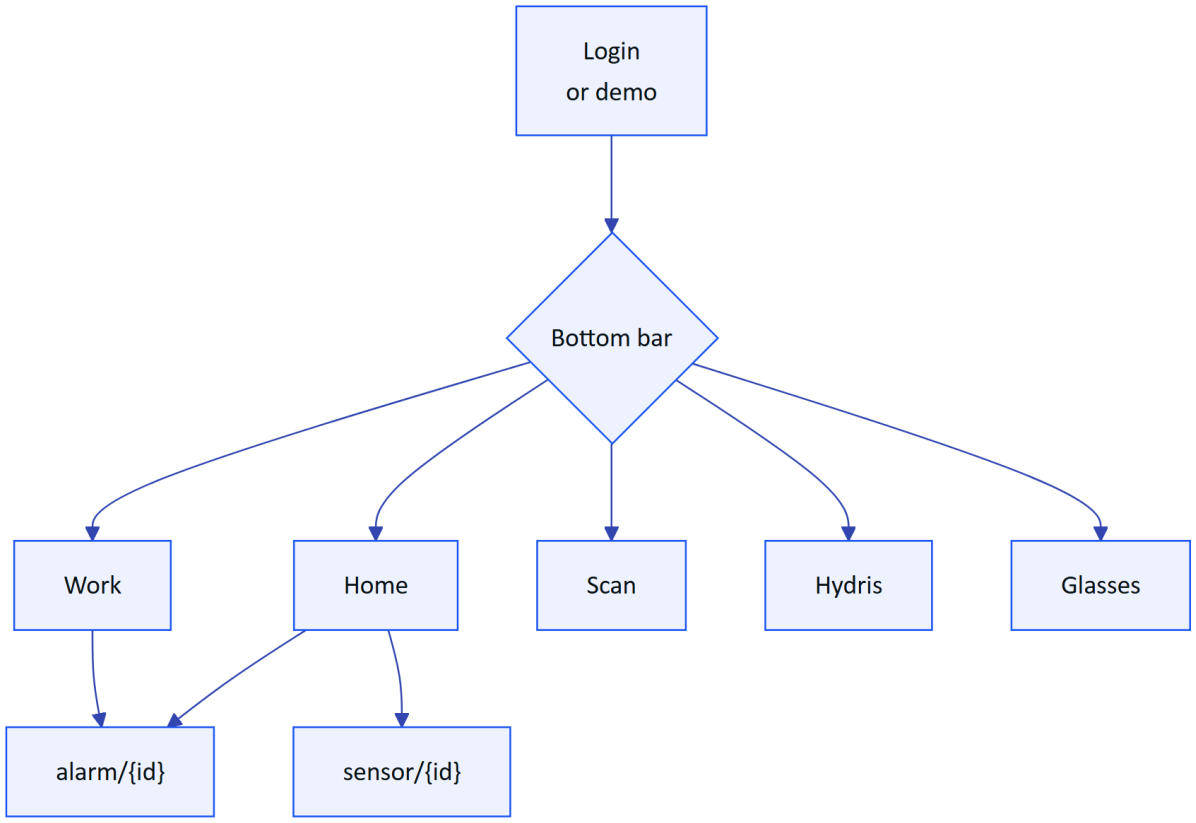

Figure 178. *Android app navigation (ui/Root.kt)*

Tab switches pop to Home with saved state and `launchSingleTop`, so returning to a tab restores its scroll position. Detail screens show a back arrow at top left (`Icons.AutoMirrored.Outlined.ArrowBack`), and the system back gesture works everywhere.

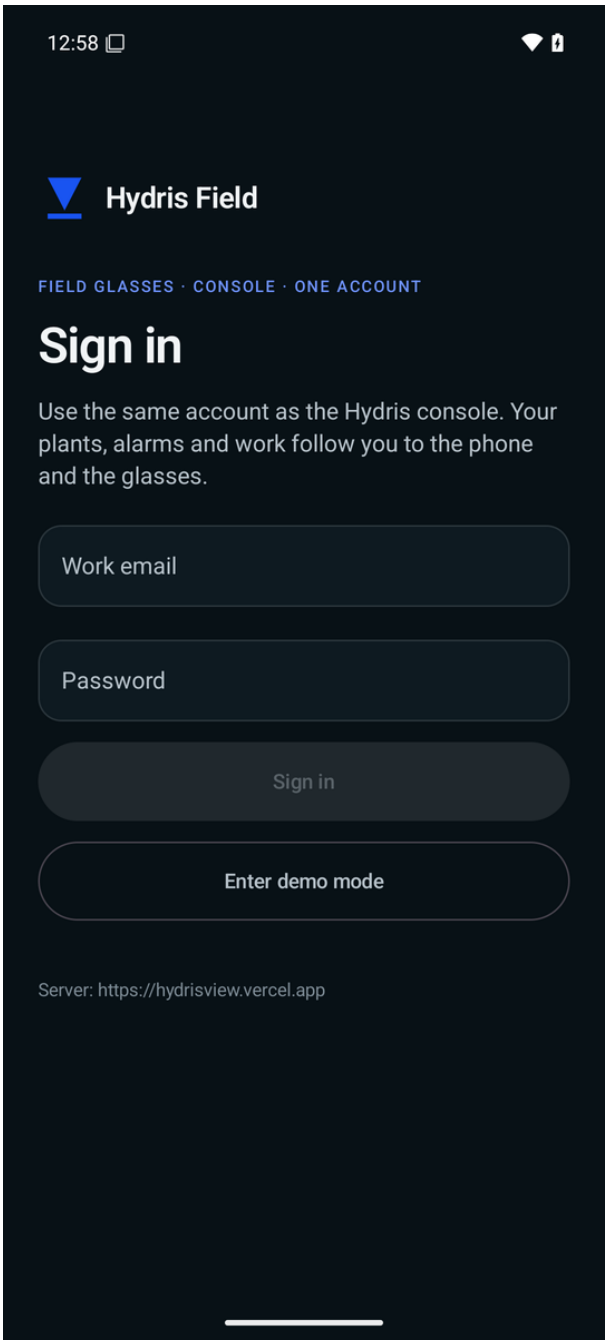

Figure 179. *Android login*

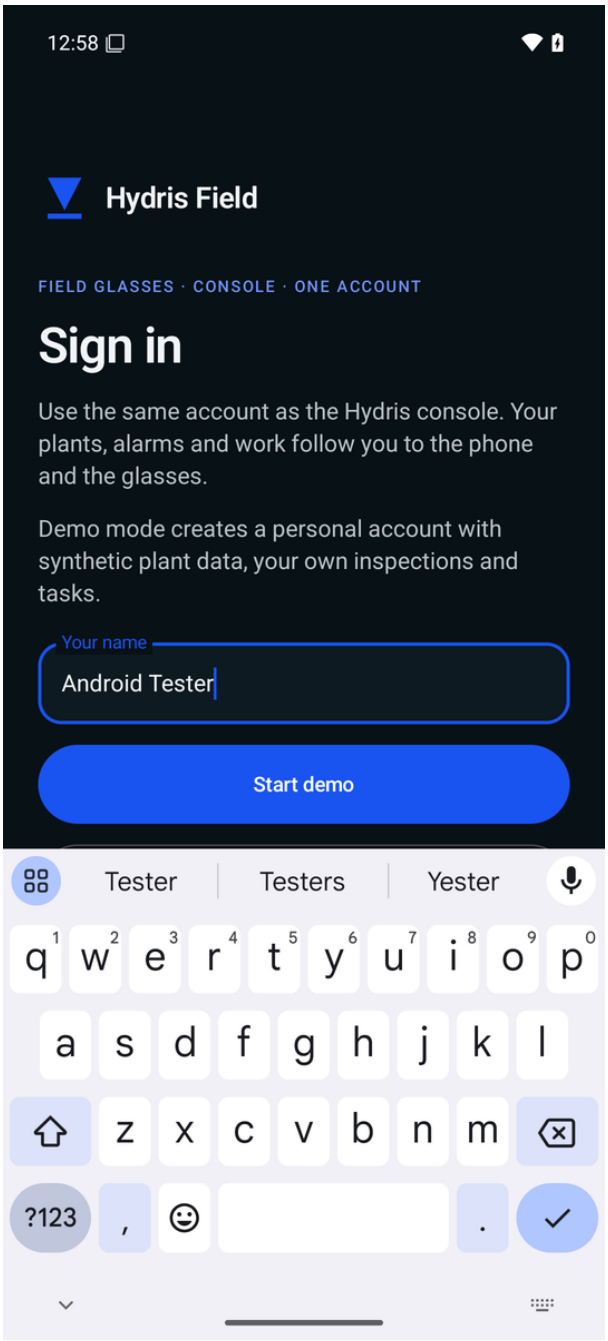

Figure 180. *Android demo sign-in: choose a display name*

8.11 Interaction patterns

8.11.1 Input model

The glasses app reduces every input to six keys: up, down, left, right, select, back (`apps/web/src/glasses/input.ts`). Keyboard arrows, Enter or Space, Escape or Backspace, touch swipes of at least 40 px, and simulator `postMessage` controls all map onto that vocabulary. Hardware gestures (neural band pinch and swipe, frame touchpad) plug in once their delivery to the web runtime is verified on a device (**Planned**). Phone mode adds direct touch on rows and the bottom tab bar.

Table 229. Input vocabulary and its meaning

Key	HUD meaning	Phone mode	Android
Up / Down	Move focus between `HudItem`s; change a digit in numeric entry	Swipe up or down	Scroll
Left / Right	Move between digits; right also opens	Swipe right to open, left for back	n/a
Select	Activate focused item	Tap	Tap
Back	Pop the stack; dismiss interrupt	Back tab or swipe left	Back arrow or system back
Voice	"Hey Hydris"	Hey Hydris tab	Mic toggle on Hydris tab
Capture button	n/a on display glasses	n/a	Ray-Ban Meta short press triggers a scan capture

8.11.2 Voice first: "Hey Hydris"

The wake phrase is matched by `WAKE_RE` in `packages/domain/src/voice.ts`, which tolerates common mishearings ("hey hydras", "ok hydrus", "hi dris"). A question can follow in the same breath ("Hey Hydris, what's the pH?"). Answers come from the account's live data through `POST /assistant/ask`, are shown as a card and spoken through the glasses speakers or the device's speech synthesis (`VoiceAssistant.tsx`).

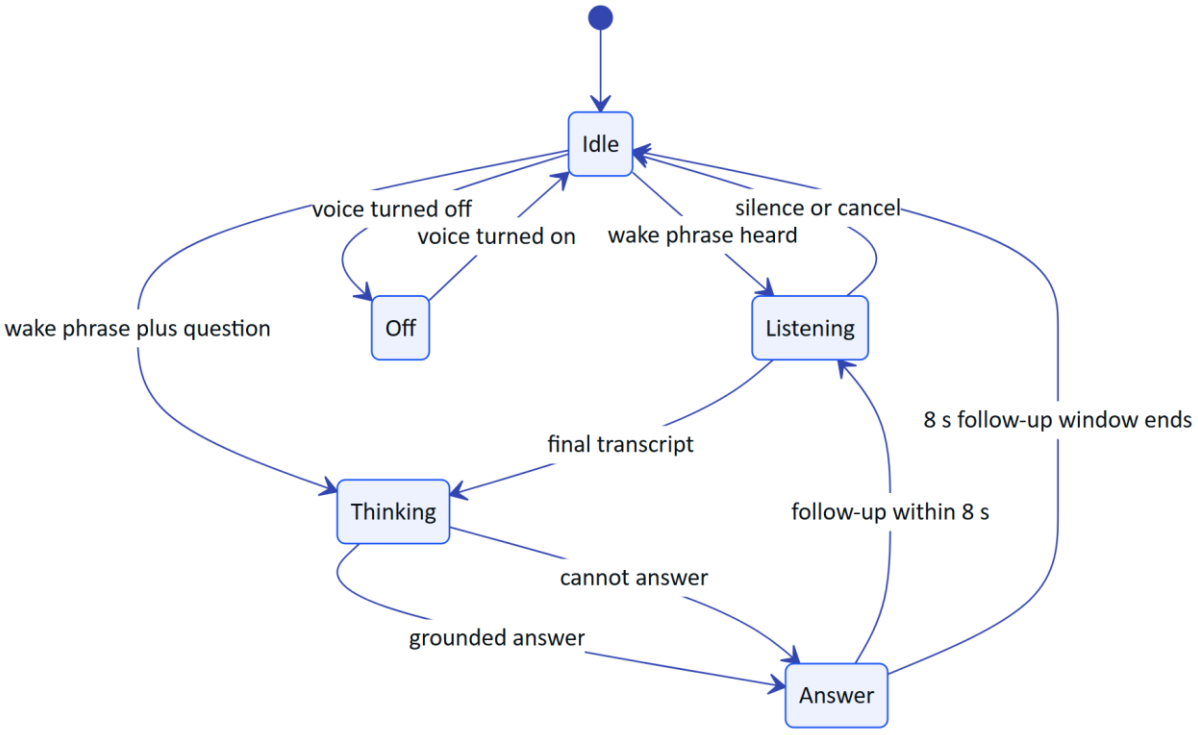

Figure 181. *Hey Hydris voice states (VoiceAssistant.tsx)*

Design rules for voice:

- Always show what was heard (Android renders the transcript in `SignalText` quotes) before the answer arrives.
- The thinking state has copy ("Checking your plant data…") so silence is never ambiguous.
- Answers list their facts with value, unit and label, so the operator can check the source.

- The mic state is always visible: "Hey Hydris" (listening for wake), "Voice off", "No mic" (`VoiceAssistant.tsx`).
- Voice never performs a state change on its own. Acknowledging or submitting stays an explicit select or tap.

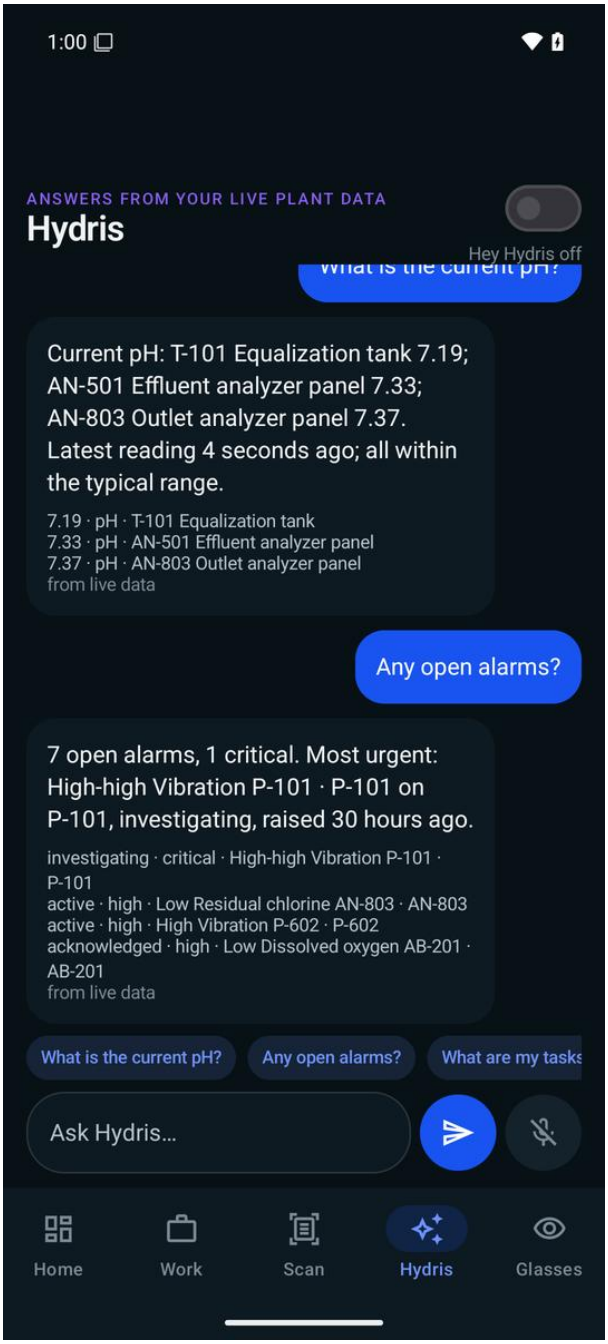

Figure 182. *Android Hydris assistant: grounded answer with facts*

8.11.3 Capture button and audit scan

On Ray-Ban Meta glasses the physical capture button is surfaced through the Meta Wearables Device Access Toolkit: a `SHORT_PRESS` capture event is emitted on `GlassesManager.captureButton` and the scan screen uses it to capture a page (`apps/android/.../glasses/GlassesManager.kt`). The scan menu offers three sources: glasses camera, phone camera with guided A4 framing, and photo import (`ScanScreen.kt`).

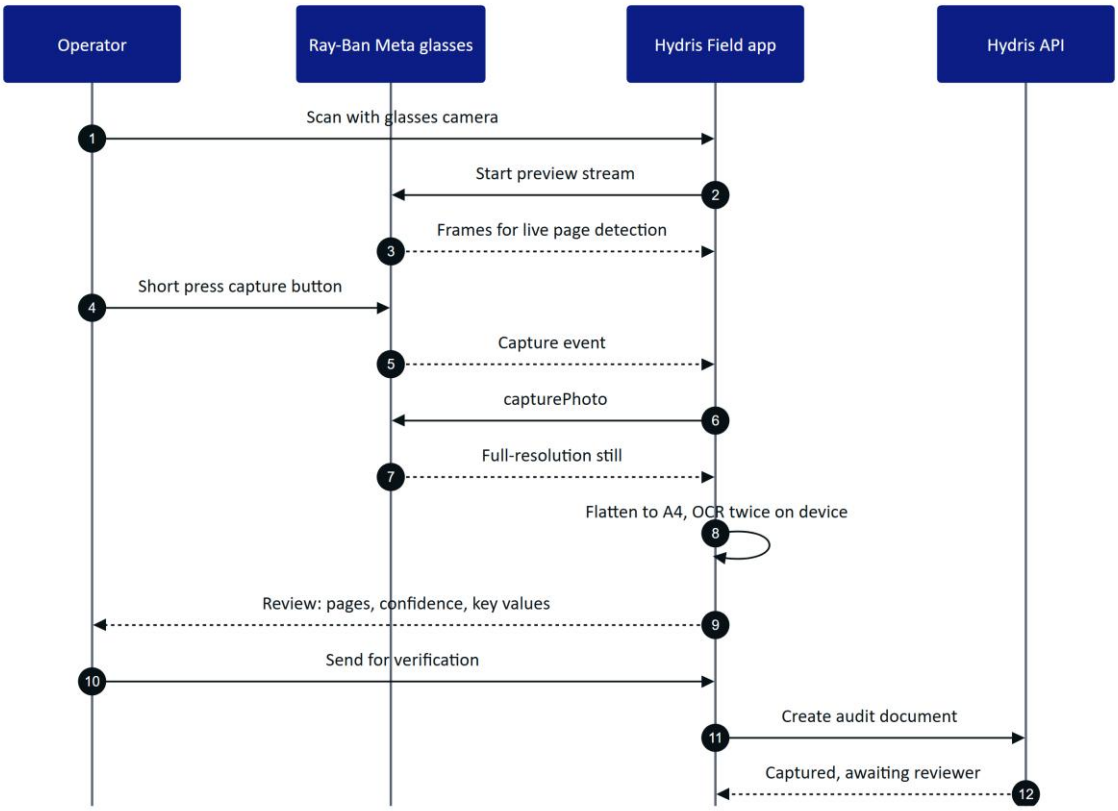

Figure 183. *Audit scan with the glasses capture button*

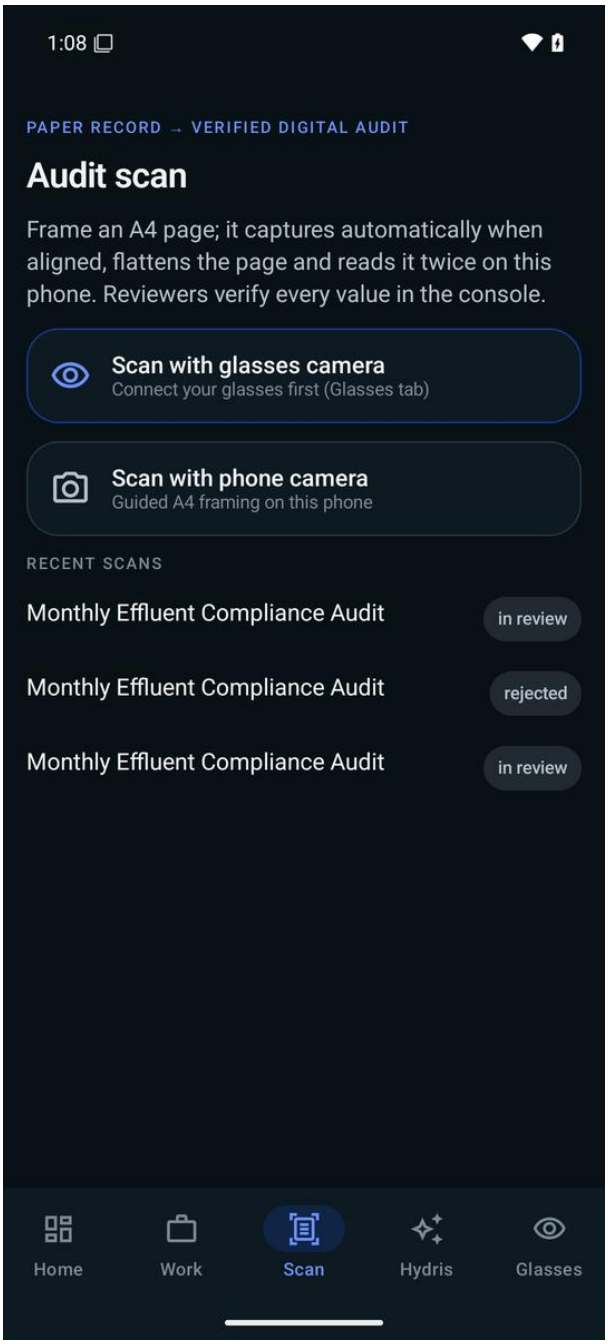

Figure 184. *Android scan menu: glasses camera, phone camera or import*

8.11.4 Guided capture feedback

`GuidedCapture` (`apps/web/src/lib/GuidedCapture.tsx`, with logic in `lib/docdetect.ts`; ported to Kotlin in `apps/android/.../scan/DocDetect.kt`) shows one instruction at a time. Detection runs every 90 ms; the detected sheet is outlined in Signal Blue while guidance is pending, in orange `#FF6A3D` when it touches the edge or is skewed, and in Active Green once ready. After 900 ms of continuous readiness it captures automatically.

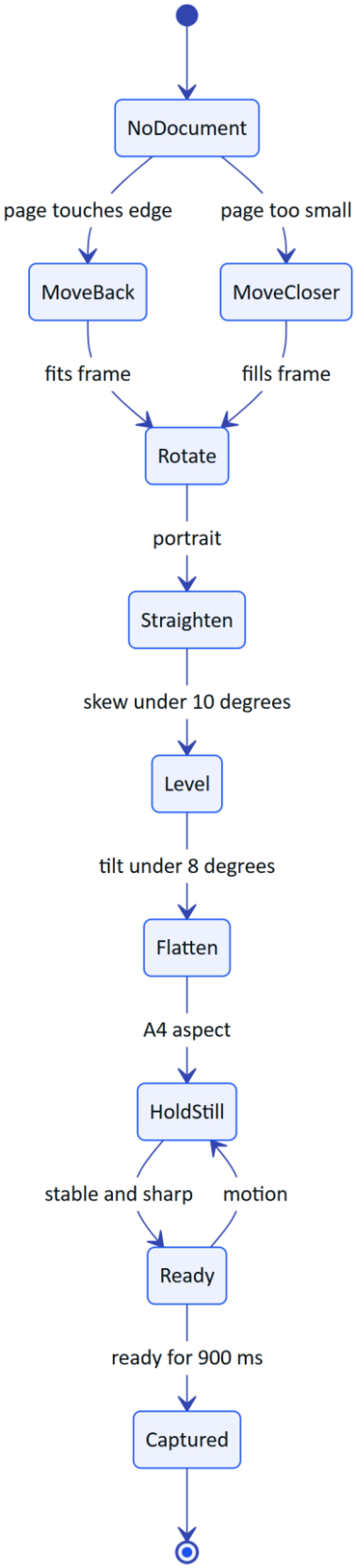

Figure 185. *Guided A4 capture guidance (docdetect.ts guide function)*

Table 230. Guidance messages (lib/docdetect.ts)

Code	Message shown	Outline colour
`no_document`	Place the A4 sheet in view, on a dark surface	None
`move_back`	Move back - show all four corners of the page	Orange
`move_closer`	Move closer - fill the frame with the page	Signal Blue
`rotate`	Turn the sheet to portrait (long side vertical)	Signal Blue
`straighten`	Look straight down at the page - reduce the tilt	Orange
`level`	Level the page - rotate it left or right until upright	Orange
`flatten`	Hold the sheet flat and square to your view	Signal Blue
`hold_still`	Hold still…	Signal Blue
`ready`	Perfect - capturing	Active Green, with fill progress

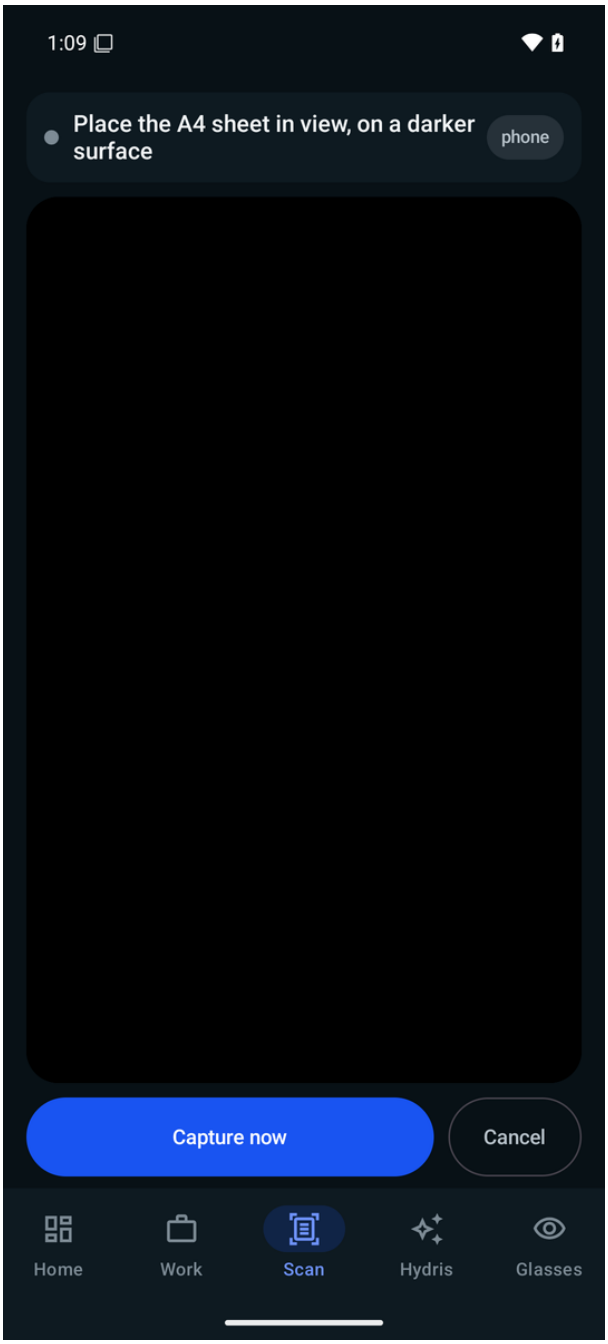

Figure 186. *Android guided capture: live outline and single instruction*

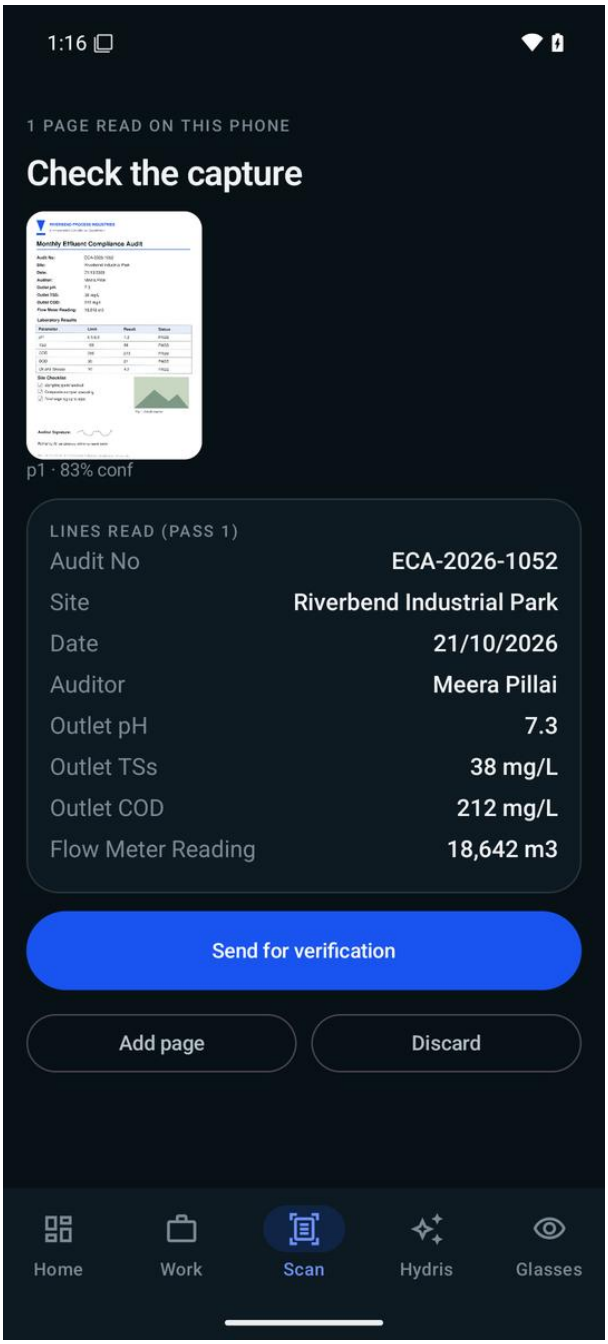

Figure 187. *Android scan review: pages, confidence and detected values*

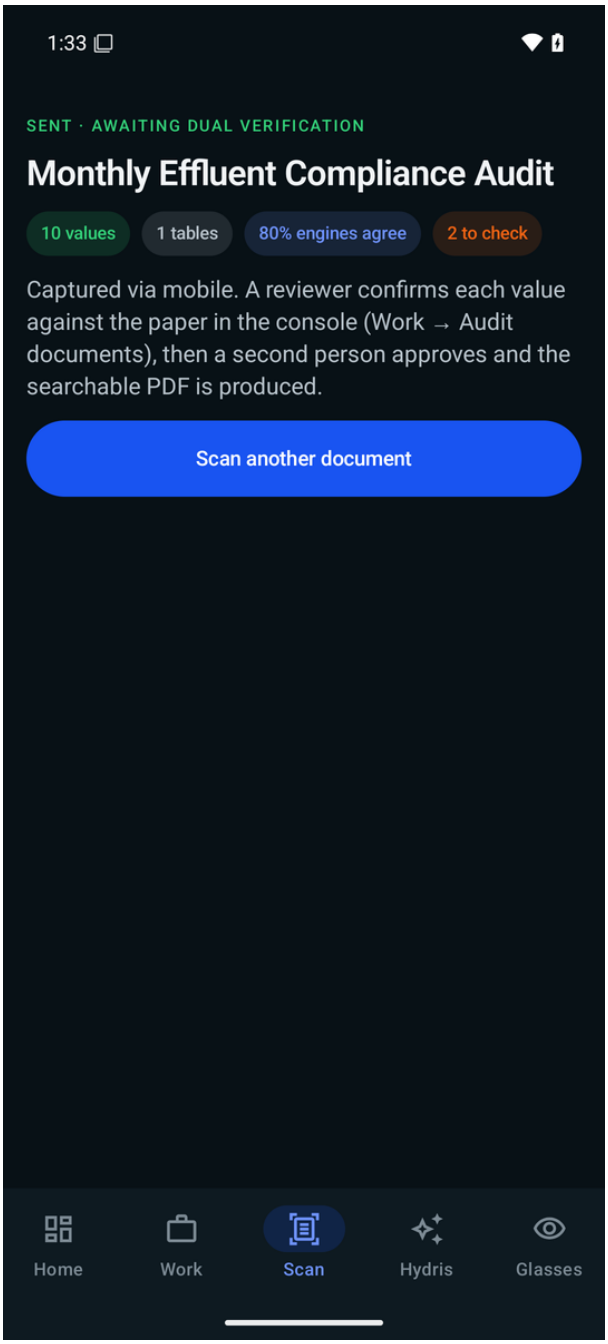

Figure 188. *Android scan sent: next steps for reviewer and approver*

8.11.5 Back navigation

- HUD: Back key or swipe left pops one screen; a Back pill sits bottom left when the stack is deeper than Home.
- Phone mode: the first tab becomes **Back** whenever the stack is deeper than Home, otherwise **Home** (`PhoneTabBar`). This keeps back reachable with a thumb at the bottom of the screen.
- Android: top-left back arrow on detail screens and the system back gesture.
- Console: breadcrumbs and `BackLink`; `ErrorState` defaults back to the parent list.
- Back never discards recorded work: inspection steps are saved as they are recorded and queued offline.

8.11.6 Alarm acknowledgement and lifecycle

The alarm lifecycle is defined once in `packages/domain/src/alarms.ts` and enforced by the API. Each surface shows only the next valid actions.

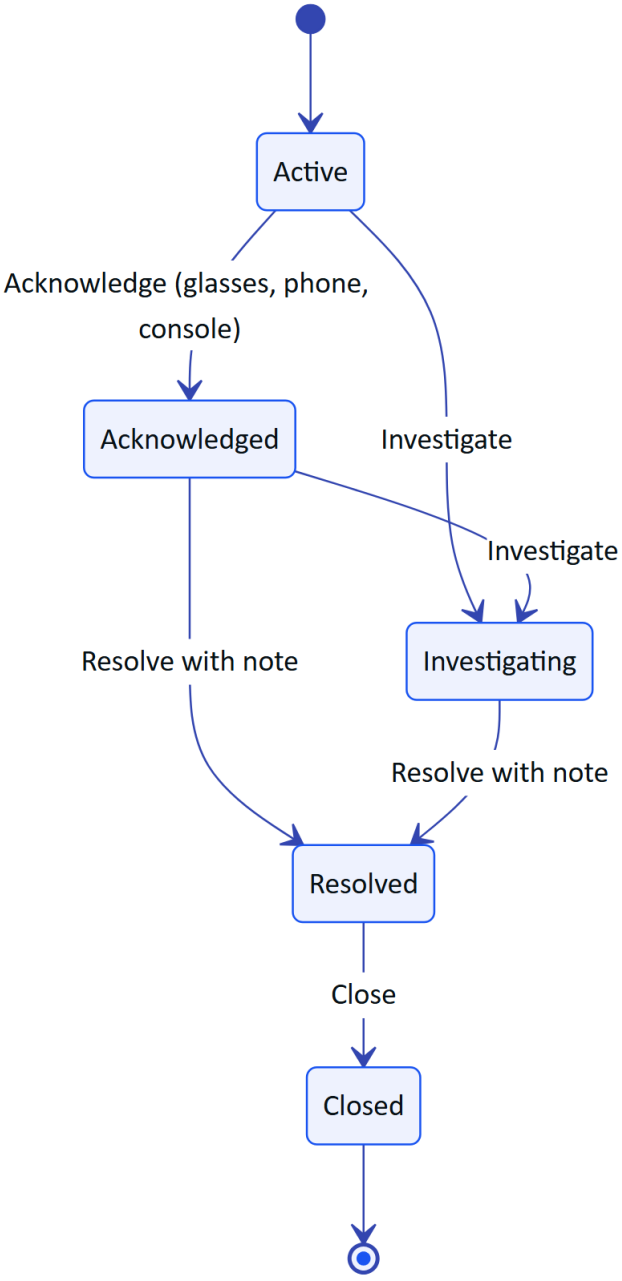

Figure 189. *Alarm lifecycle and the actions each surface offers*

Table 231. Acknowledgement and lifecycle patterns by surface

Surface	Interrupt	Acknowledge	Resolve	Status
Glasses HUD	`AlertInterrupt` full-screen overlay for critical and high, View or Dismiss	Single focused `HudItem`, disabled offline with "(needs connection)"	Not offered (console or phone)	**Built**
Android	None (push notifications **Planned**)	Full-width primary button	Button shown, but no note is collected	**Partial**

Surface	Interrupt	Acknowledge	Resolve	Status
Console	Realtime toast for critical alarms	Button on detail	`ConfirmDialog` with required note "What was done to resolve the condition"	**Built**

> **WARNING**
>
> On Android the Resolve button posts the action without a note (`WorkScreen.kt` calls `alarmAction(id, "resolve")`), but the API rejects resolve without one (`note_required` in `apps/api/src/modules/alarms.ts`). The operator sees an error note. Fix: a bottom sheet that collects the note, matching the console's `ConfirmDialog`. Status: **Planned**.

8.11.7 Operator experience journey

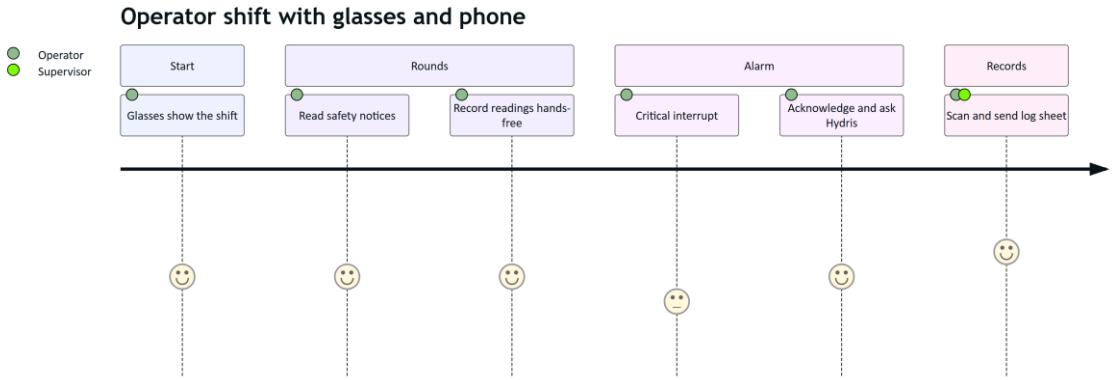

Figure 190. *Field operator shift journey with Hydris (target experience)*

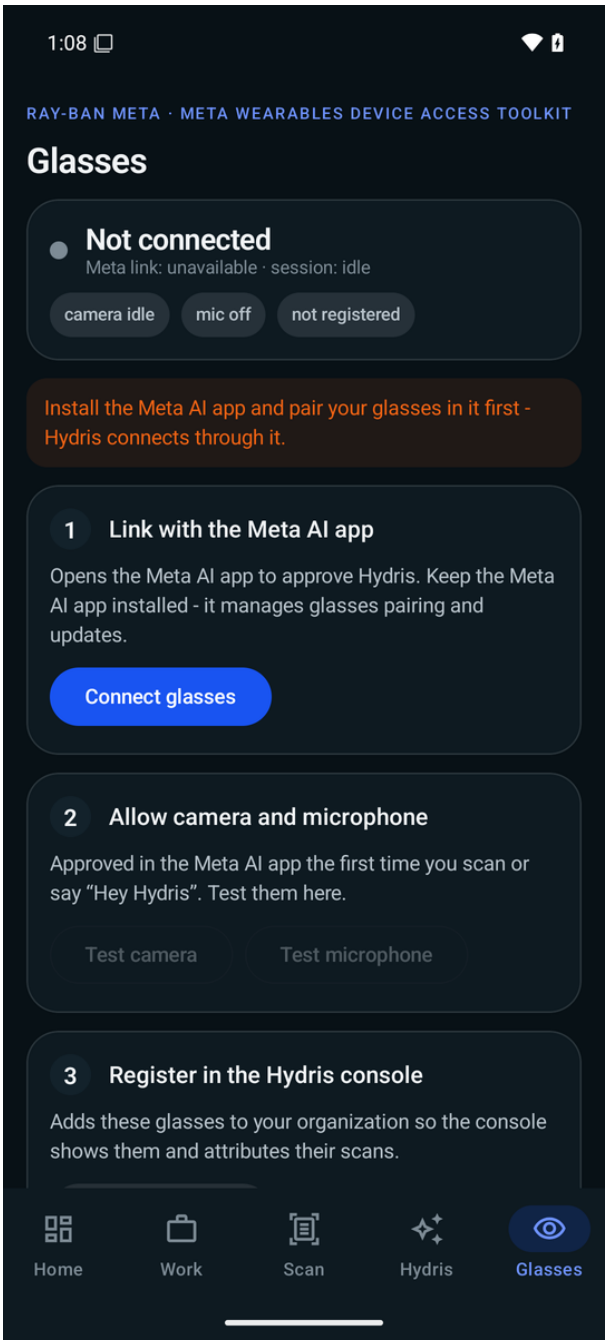

Figure 191. *Android glasses screen: connection checklist and registration*

8.12 Motion

Motion explains change; it never decorates. All motion tokens are in globals.css.

Table 232. Motion tokens and patterns

Token or class	Value	Use
`--speed-fast`	150 ms	Colour and border changes, press feedback
`--speed`	240 ms	Lifts, fades
`--ease-out`	cubic-bezier(0.22, 1, 0.36, 1)	Default easing
`--ease-spring`	cubic-bezier(0.34, 1.4, 0.64, 1)	Theme toggle icon only
`.page-enter`	360 ms fade and 8 px rise	Route change in console

Token or class	Value	Use
`.pop-in`	220 ms	Popovers, centred dialogs
`.sheet-up`	320 ms	Bottom sheets
`.slide-in-right`	320 ms	Side panels
`.stagger`	40 ms steps up to 200 ms	Card grids on load
`.pulse-dot`	1.8 s	Live freshness dot only
`.skeleton`	1.6 s shimmer	Loading
Chart transitions	600 to 700 ms	Bars and arcs growing to value
Theme switch	300 ms colour cross-fade	`html.theme-switching`

Rules:

- `prefers-reduced-motion: reduce` disables animations and transitions globally, stops the pulse dot and hides the toast timer (globals.css). **Built** on web.
- The HUD uses colour transitions only; no movement on an additive display, where motion pulls the eye from the task. **Built.**
- Android uses Material defaults; honouring the system "remove animations" setting is inherent to Compose animation APIs. Custom motion is not used. **Built.**
- Alarms never animate in a way that could be mistaken for normal motion; the interrupt simply appears.

8.13 Accessibility

8.13.1 Targets

Table 233. Accessibility targets

Area	Target	Status
Text contrast	WCAG 2.2 AA (4.5:1, 3:1 large) on every surface and theme	**Partial** (3 failures listed in the contrast audit)
Non-text contrast	3:1 for borders of inputs, focus rings, chart marks	**Built** (focus ring `#6F93FF` 6.62:1 on bg)
Target size	WCAG 2.5.8: at least 24 by 24 px; Hydris standard 44 by 44 px	**Built** on web (`::after` hit areas on `btn-sm`, `icon-btn-sm`, chips, links) and Android (48 dp Material, 52 dp buttons)
Focus visible	2 px outline, 2 px offset, colour follows surface	**Built** (`:focus-visible` in globals.css)
Keyboard	Every console action reachable by keyboard; HUD fully key-driven	**Built**
Screen readers	Labels on icon buttons; charts as images with text tables; live region for route changes	**Built** on web, **Partial** on Android charts
Forced colours	Selected capsule outlined in `CanvasText`	**Built**
Text resize	Web reflow to 320 px; Android `sp`; HUD `textScale`	**Built**
Reduced motion	Respect user preference	**Built** (web)
Haptics	Vibration for critical alarms and confirmed actions	**Planned**

8.13.2 Gloves, noise and outdoor light

- **Gloves.** Field targets are 52 dp or larger on Android task flows and the full width of the screen where possible; the phone-mode tab bar is 92 units tall with 28-unit icons. Avoid swipe-only actions in phone flows: every swipe has a tap equivalent.
- **Noise.** Voice is a convenience, never the only path. Every voice capability has a button or tap equivalent, and answers are shown as text as well as spoken. On Ray-Ban Meta glasses the microphone is the glasses microphone over Bluetooth, which sits closer to the mouth than the phone.
- **Outdoor light.** Dark themes with near-white text (19:1) survive glare better than mid-grey palettes. Minimum sizes outdoors: 16 sp body on Android, 20 units body on the HUD, KPI values at 34 sp or larger. Operators can raise text size through the system setting (Android) or the UI profile (glasses).
- **Safety glasses and polarised lenses.** Do not rely on subtle tints; severity is always paired with a word and, on the HUD, a 2 to 3 unit border.

8.13.3 Haptics (Planned)

No haptic feedback exists in the code today (no vibration calls on web or Android). The planned pattern: one short pulse when a step is recorded, a double pulse when an action is rejected, and a long pulse for a critical alarm interrupt on Android. Haptics complement, never replace, visible state.

8.14 Content and voice guidelines

Hydris copy is short, specific and calm. It names the equipment, the value and the next step. It never overstates what the system does.

Table 234. UX writing do and don't

Do	Don't	Why
"Acknowledge" with hint "does not change equipment"	"Fix", "Clear alarm"	Hydris records; it does not operate equipment
"VT-101 vibration high-high · 7.4 mm/s"	"Problem detected on pump"	Name the tag, measurand and value
"Offline · showing data cached 4m ago"	"Error"	Say what still works and how old the data is
"Next step (approved procedure): ..."	"AI recommends ..."	Attribute guidance to the approved SOP
"Send for verification"	"Submit final"	A reviewer and approver still confirm the record
"We couldn't find that record" plus a way back	"404 Not Found"	Plain language with a recovery path
"Ticking checklist items · not permitted by your profile"	Hiding the control silently	Never hide state
"Checking your plant data…"	Silent spinner	Explain waits longer than a moment
"Synthetic"	"Test", "Fake"	Consistent term for simulator data
"Step 3 of 8 · optional"	"Question 3"	Orientation and obligation in one line

Rules:

- Sentence case for every label and title. Uppercase only through the eyebrow style.
- Units follow numbers with a space ("7.4 mm/s"); pH is the exception and leads the value ("pH 6.8").
- Relative times for recent events ("5m ago"), absolute times in tooltips and audit records.

- No em dashes in product copy or documents; use a colon, a comma, a middle dot or two sentences. The guided capture messages currently use a spaced hyphen, which is acceptable on screen.
- Button labels are verbs; one to three words on the HUD.

8.15 Localisation

The product ships in English only today: the web root sets `lang="en"` (`apps/web/src/app/layout.tsx`) and no string catalogue or i18n library exists on web or Android.

Table 235. Localisation plan

Item	Detail	Status
String extraction	Web: message catalogues per route; Android: `strings.xml` resources	**Planned**
Hindi (hi-IN)	First additional language; Devanagari font fallback for Outfit (Noto Sans Devanagari)	**Planned**
Tamil (ta-IN), Marathi, Gujarati, Telugu, Kannada	Pilot regions in India	**Planned**
Voice	Wake phrase stays "Hey Hydris"; recognition and TTS locale follow the operator	**Planned**
OCR	ML Kit Devanagari model for Hindi labels on audit sheets (per mobile spec)	**Planned**
Numbers and units	Locale digit grouping in tables; SI units unchanged; decimal separator follows locale	**Planned**
Layout tolerance	Labels must survive 40 % expansion; HUD rows truncate with an ellipsis, never wrap into two actions	Rule (applies now)

> **NOTE**
> Indic scripts need taller line heights than Latin. When Hindi or Tamil is added, HUD row padding and the 0.95 line height of `t-kpi` must be rechecked; numerals stay Latin (tabular) in every locale.

8.16 Dark and light themes

Table 236. Theme support by surface

Surface	Default	Light theme	Mechanism	Status
Console	Dark	Full token set	`data-theme` on `<html>` set by a blocking script before paint; `ThemeToggle` (system, light, dark); stored in `localStorage` key `hydris.theme`	**Built**
Website	Dark	Shared tokens plus `.hx` brand layer	Same	**Built**
Glasses app and simulator viewport	Dark, forced	None (by design)	`FORCED_DARK_RE`	**Built**
Android	Dark	None	`darkColorScheme` only	**Planned** (light scheme for office use)

Surface	Default	Light theme	Mechanism	Status
Word and PDF documents	Light	n/a	Build theme	**Built**

Rules: the blue focal panel, brand blue fills and Alert Orange are identical in both themes; semantic **text** tints are redefined for light so they stay AA (for example critical text `#C41F00` on white, 5.93:1); KPI accent numerals deepen on paper and light surfaces (`kpi-accent-*` overrides).

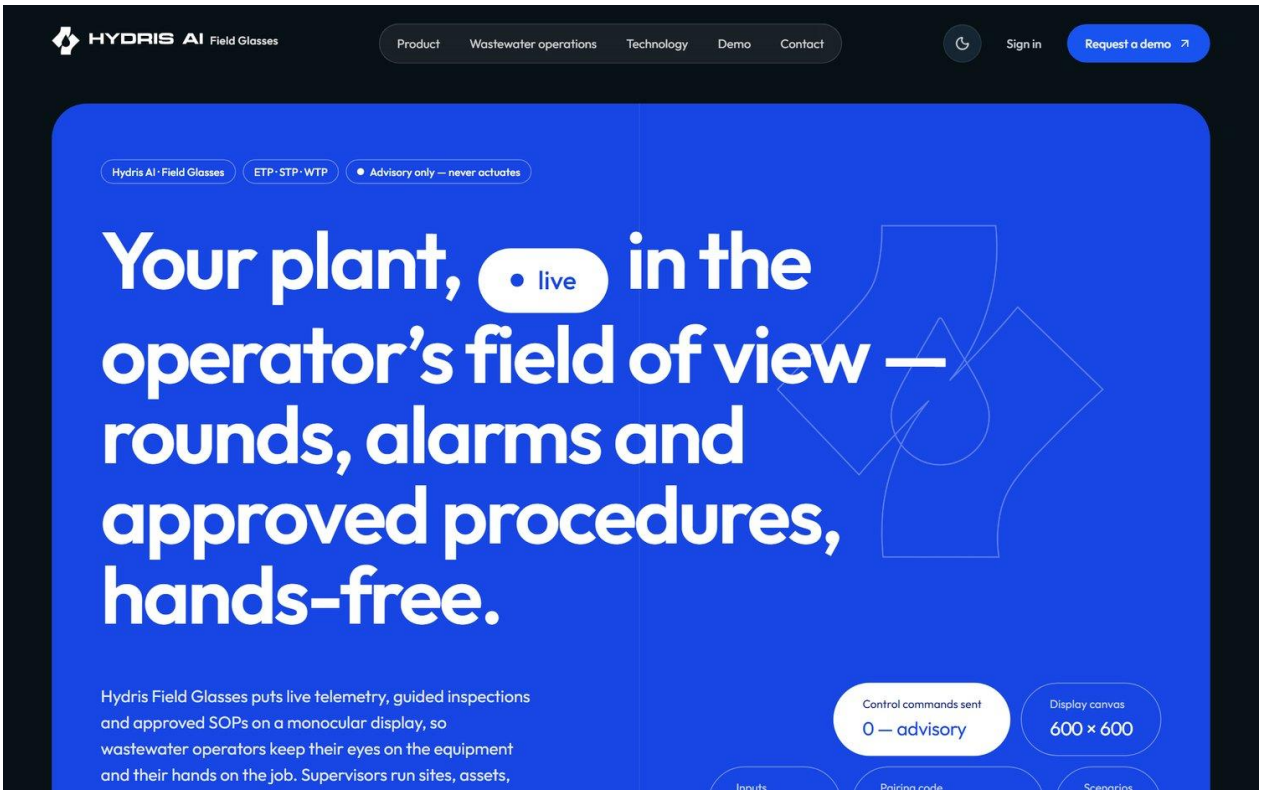

Figure 192. *Website home: brand layer over the shared tokens*

8.17 Design QA checklist

Use this list on every pull request that changes UI. Items checked here are true of the current code base; open items are known gaps to close.

- ☑ Uses semantic tokens only; no new hex values in components (web)
- ☑ Every value shows unit and freshness, and missing data shows "No data"
- ☑ Severity is shown with a word or icon as well as colour
- ☑ Synthetic data carries the Synthetic marker on every surface
- ☑ Interactive targets are at least 44 by 44 px (web) or 48 dp (Android)
- ☑ Focus is visible and follows the surface colour
- ☑ Charts have an accessible name and an empty state (web)
- ☑ `prefers-reduced-motion` respected (web)
- ☑ HUD screen has one column and one primary action
- ☑ Blocked actions are explained, not hidden (glasses)
- ☐ HUD selected pill, View button and digit cell use white ink on Signal Blue (contrast 5.87:1)

- ☐ Android filled severity chips meet 4.5:1 or use the tinted style
- ☐ Android synthetic chip uses a lighter Insight text tint
- ☐ Android Resolve collects a note before posting
- ☐ High severity colour aligned to `#F26414` across console, website, Android and HUD
- ☐ HUD severity colours moved from inline hex to tokens
- ☐ Android chart canvases have content descriptions
- ☐ Haptic patterns implemented for record, reject and critical alarm
- ☐ Strings extracted for localisation
- ☐ Tested at UI profile text scale 1.3 in both densities and both hands

8.18 Governance and contribution

The design system is owned by Design, Hydris, with one engineering maintainer per platform (web, Android). Changes follow the same review discipline as code: the Markdown in this folder and the token files are the source of truth.

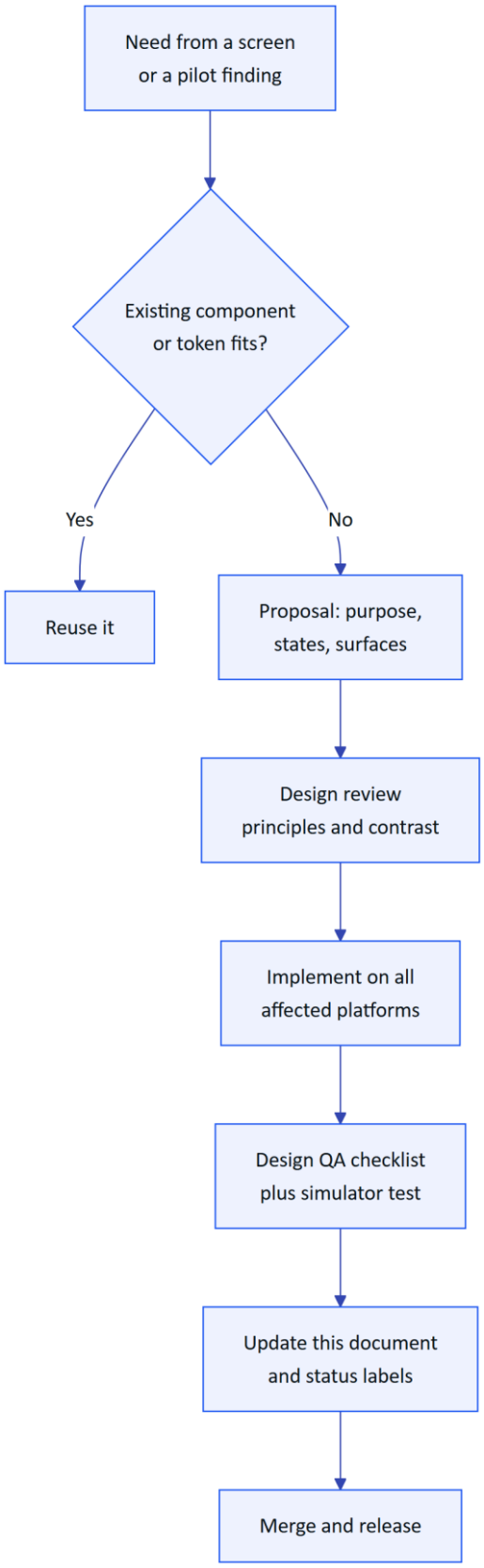

Figure 193. *Contribution process for a design system change*

Table 237. Contribution rules

Rule	Detail
Tokens before components	A new colour, size or radius is added as a token in `globals.css` and `Theme.kt` (and the HUD) in the same pull request
No one-off severity colours	Severity and status colours come only from the severity tokens
All surfaces or a ticket	A change to a shared pattern ships on every surface, or opens a tracked gap with a **Partial** label here
Contrast proof	New colour pairs include their computed ratio in the pull request description
Simulator check	Glasses changes are checked in the simulator at 600 by 600, 640 by 480 and 480 by 480, in phone mode, and at text scale 1.3
Screenshots	Updated screenshots go into `docs/product/images/` (web shots refresh with `node screenshots.js`)
Versioning	This document versions with minor bumps for additions and major bumps for breaking token renames

8.19 Future work

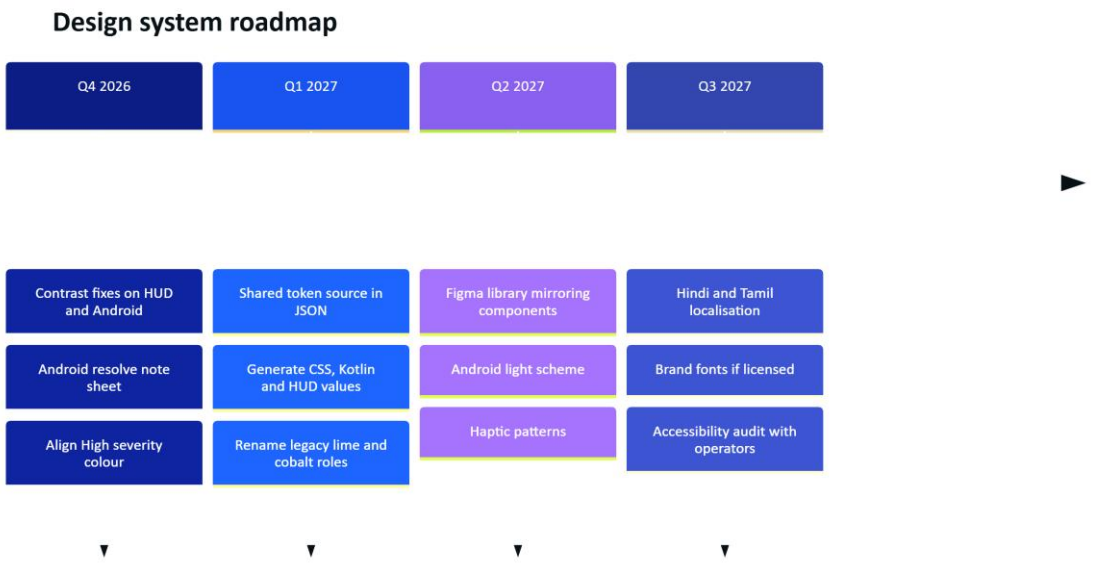

Figure 194. *Design system roadmap (targets, not commitments)*

Table 238. Future work items

Item	Description	Status
Token pipeline	One JSON token source (for example in `packages/domain` or a new `packages/tokens`) that generates `globals.css` variables, `Theme.kt` `Hx` and a HUD palette module	**Planned**
Figma library	Components, tokens as Figma variables, HUD frame templates at 600 by 600 and phone 540-unit width, linked to code	**Planned**
Legacy role renames	`lime` to `signal`, `cobalt` to `purple` across classes and utilities	**Planned**

Item	Description	Status
Android component parity	`StateChip`, `FreshnessBadge`, empty states and a severity chip that matches the web	**Planned**
Push notification design	Critical alarm notification layout and channels on Android	**Planned**
Hardware gesture mapping	Neural band and frame touchpad onto the six-key vocabulary	**Planned**
Field usability study	Glove, glare and noise trials with pilot operators; results to update this document	**Planned**

EXAMPLE

A token entry in the planned pipeline: `color.severity.high = { value: "#F26414", text.dark: "#FF9A5C", text.light: "#A35200" }` generates `--color-high` and `--color-high-text` for web, `Hx.High` and `Hx.HighText` for Android, and the HUD pill and border values, so one change updates every surface.

9 Deployment and Infrastructure

> **NOTE**
>
> **HFG-DEP v2.0** · Draft for review · Owner: Platform engineering, Hydris. Where Hydris Field Glasses runs today, the topologies we will support for customers, and how we build, release, observe, recover and pay for it

This document describes how Hydris Field Glasses is deployed today and how it will be deployed for paying customers. It covers the hosted platform on Vercel, the managed Postgres database, the two Android apps, the release pipeline that ships them, and the operational practices (observability, backup, disaster recovery, capacity, cost and data residency) that turn a working demo into a dependable service for wastewater and effluent treatment plants.

Every statement about the current system is grounded in the repository and names the file it comes from. Status labels follow the house rule: **Built** means it is in the code and exercised, **Partial** means it exists but has a known gap, and **Planned** means it is a design that is not yet implemented. Targets are labelled as targets. Costs and capacity numbers are indicative estimates, not measurements or quotes.

1
Vercel project
website, console, glasses app and API in one deployment

9
SQL migrations
`apps/api/drizzle/0000` to `0008`, applied under an advisory lock

0
CI workflows
no `.github/` directory yet; the pipeline in this document is proposed

1
daily cron
`/api/cron` at `0 3 * * *` (root `vercel.json`)

> **IMPORTANT**
>
> The live deployment at `https://hydrisview.vercel.app` is a demonstration environment. It runs in demo mode with synthetic data and public demo secrets. It must never receive real plant data. The production topologies in this document are what a customer pilot runs on.

9.1 Deployment at a glance

Hydris is a TypeScript monorepo (`package.json` workspaces `packages/*` and `apps/*`, Node `22.x` in `engines`). Three deployable units come out of it:

Unit	Source	Runtime	Status
Web and API	`apps/web` (Next.js 16) with `apps/api` (Fastify) compiled in	Vercel serverless functions, Node.js runtime	**Built**
Hydris Field (native Android)	`apps/android` (Kotlin, Jetpack Compose)	Android 12+ phones, package `ai.hydris.field`	**Partial** (release signing not configured)
Hydris Glasses bridge (Android)	`apps/glasses-bridge-android` (WebView plus Meta Wearables DAT 1.0.1)	Android 12+ phones paired to Ray-Ban Meta glasses, package `ai.hydris.glassesbridge`	**Partial** (not yet run on physical glasses, per its README)

Two further artefacts exist for non-Vercel hosting and are used by the target topologies:

Artefact	File	Purpose	Status
API container image	`apps/api/Dockerfile`	Long-running Fastify server (`src/server.ts`) with timers, SSE and local disk storage	**Built**, not used in production
Web container image	`apps/web/Dockerfile`	Next.js server that proxies `/api` to `API_URL`	**Built**, not used in production
Compose stack	`docker-compose.yml`	Postgres 17, API and web on one host	**Built**, used for local production-like tests
Render blueprint	`render.yaml`	API as a Render web service with a 5 GB disk	**Built**, not deployed

> **NOTE**
> Both Dockerfiles use `node:24-slim`, while the root `package.json` pins `engines.node` to `22.x` and `docs/operations/vercel.md` asks for Node 24 in the Vercel project settings. Pick one major version for all runtimes before the pilot. This document assumes Node 22 because that is what `engines` enforces.

9.2 Where Hydris runs today

9.2.1 The current production topology

The whole product is one Vercel project connected to the GitHub repository `tarun-km/hydrisview`. Every push to `main` triggers a production build and deployment to `https://hydrisview.vercel.app`. The root `vercel.json` sets the framework (`nextjs`), the install command (`npm ci --include=dev`), the build command (`npm run build`, which builds only `@hydris/web`), the output directory (`apps/web/.next`) and a single cron job.

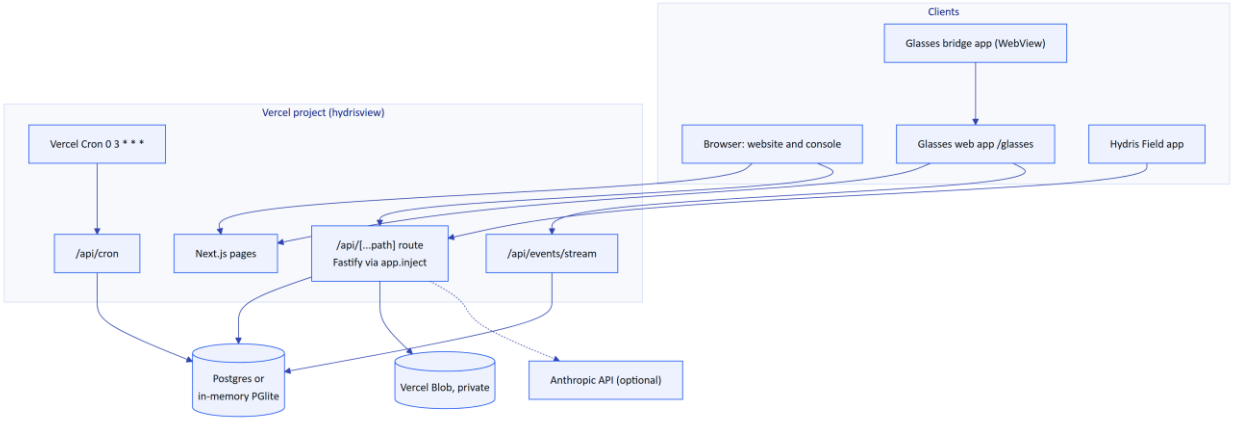

Figure 195. *Current deployment: one Vercel project serves the website, console, glasses app and the API in-process*

The API is not a separate service. `apps/web/src/app/api/[...path]/route.ts` lazily imports `@hydris/api/serverless`, builds the Fastify application once per function instance and forwards each request with `app.inject` (no network hop). Next.js treats it as an ordinary route handler with `runtime = 'nodejs'`, `dynamic = 'force-dynamic'` and `maxDuration = 60`. The `next.config.ts` file keeps native and runtime-plugin packages (`@node-rs/argon2`, `pg`, `@electric-sql/pglite`, `fastify` and its plugins, `@vercel/blob`) out of the bundle via `serverExternalPackages`, and ships the SQL migrations with the functions via `outputFileTracingIncludes`.

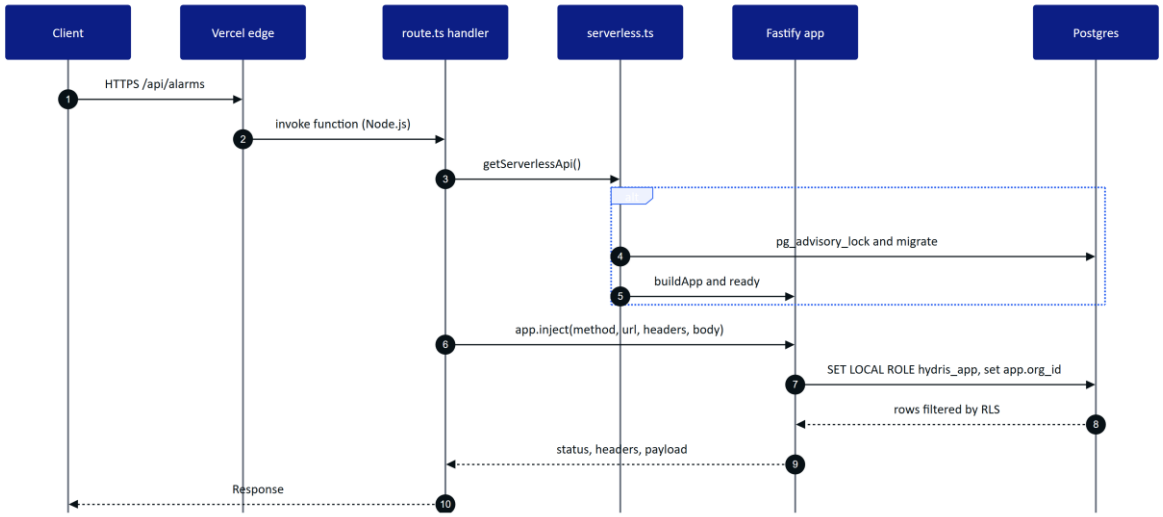

Figure 196. *Request path for an API call in the built-in API mode*

Two route handlers bypass `app.inject` because they need host-level behaviour:

- `apps/web/src/app/api/events/stream/route.ts` streams Server-Sent Events from the `realtime_events` table for up to 50 seconds, then ends with a `retry:` hint so the client reconnects with `Last-Event-ID`. It refuses to run unless `REALTIME_MODE=db`, because an in-memory event bus cannot span serverless instances. **Built.**
- `apps/web/src/app/api/cron/route.ts` runs one pass of the background jobs when called with `Authorization: Bearer $CRON_SECRET`, compared in constant time. **Built.**

When `API_URL` is set (or in `next dev`), `next.config.ts` adds a `beforeFiles` rewrite that proxies `/api/*` to that URL instead, so the same web build can front an external API container. This switch is the hinge between the serverless topology and the container topologies below.

9.2.2 Demo mode and how it is selected

`apps/api/src/config.ts` decides at start-up whether the instance is a demo. `isDemoEnv` returns true when `DEMO_MODE=true`, false when `DEMO_MODE=false`, and otherwise true when running on Vercel (`VERCEL` set) without a `DATABASE_URL`. In demo mode `demoDefaults` forces `DATABASE_URL=memory://` (an in-memory PGlite database per instance) unless `DEMO_PERSIST=true` and a Postgres URL are both present, fills in the public `DEMO_SECRETS` for any missing or invalid `JWT_SECRET` and `DATA_KEY` (and logs a warning), derives `PUBLIC_APP_URL` from `VERCEL_PROJECT_PRODUCTION_URL` or `VERCEL_URL`, and forces `SIMULATOR_ENABLED=true` and `REALTIME_MODE=db`.

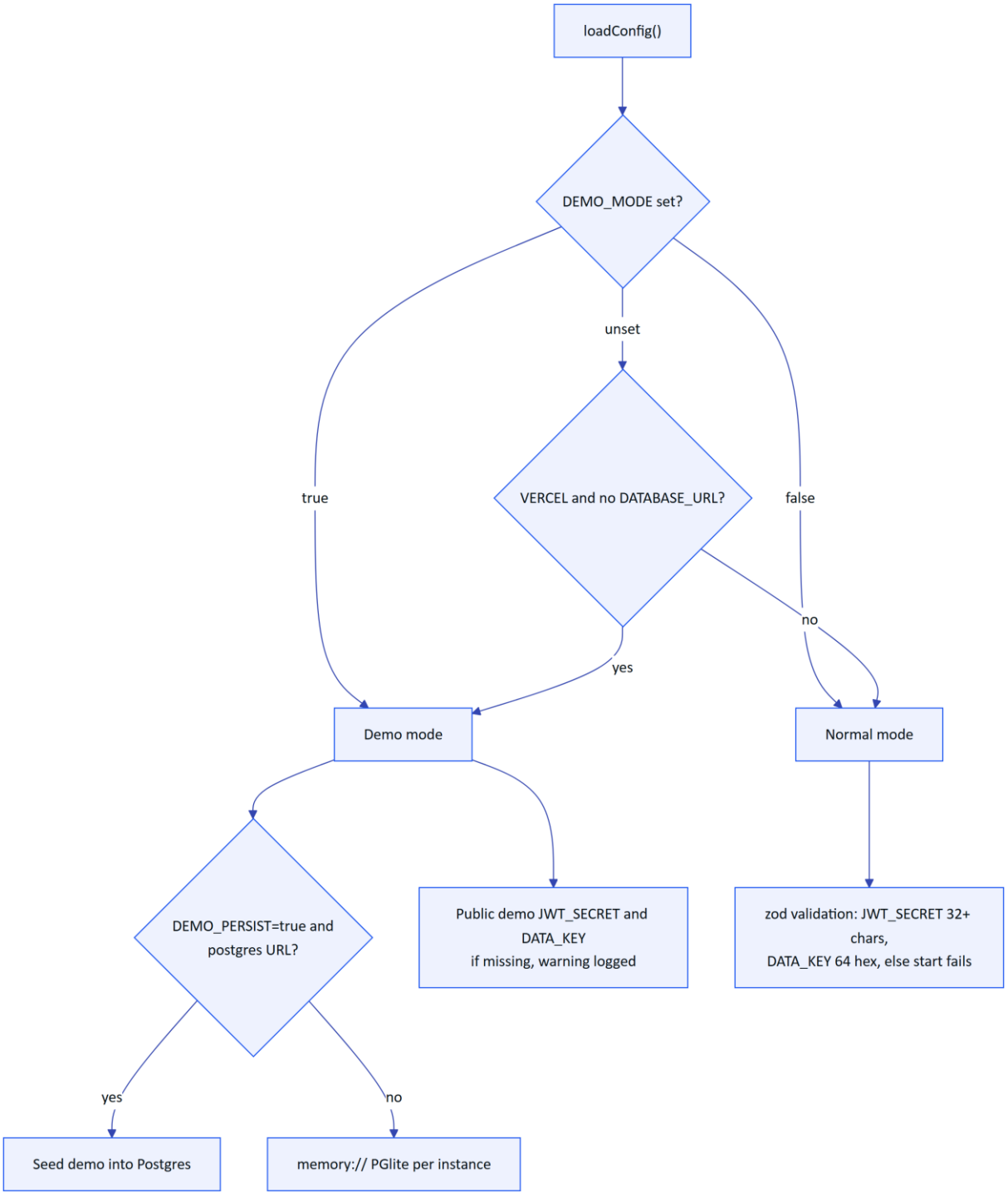

Figure 197. *How config.ts chooses between demo mode and a real deployment*

Demo mode always behaves as serverless (no interval timers). Its access tokens live 12 hours instead of 15 minutes (`accessTokenTtl`) because sessions live per instance. Each instance seeds the same deterministic synthetic tenant on cold start (`bootDemo` in `apps/api/src/demo.ts`).

9.2.3 Cross-instance demo state

Because every demo instance has its own database, short-lived state that must be seen by more than one instance (a pairing challenge and its approval, a named guest account) goes through a tiny key/value store, `apps/api/src/core/demo-shared.ts`. It selects a backend in order of preference and wraps it so that a failure never breaks the demo.

Backend	When	Scope	Persistence	Status
Postgres `hydris_demo_kv`	A Postgres `DATABASE_URL` is present	All instances using that database	Until `expires_at`	**Built**
Vercel Runtime Cache	On Vercel without Postgres	All instances in one region	Best effort, TTL based	**Built**
Process memory	Local development and tests	One process	Until restart	**Built**

WARNING

Demo mode relaxes several security properties on purpose (public signing secrets, 12-hour access tokens, self-contained device refresh tokens, self-approval in demo tenants). That is acceptable only because demo data is synthetic and flagged `is_demo`. The Security and Privacy document (HFG-SEC) lists these relaxations.

9.2.4 What exists and what does not

Table 239. Demo shared-state backends in apps/api/src/core/demo-shared.ts

Capability	Status	Evidence
Git-driven deploys to Vercel (preview per branch, production on `main`)	**Built**	Vercel Git integration, root `vercel.json`
In-process API on serverless functions	**Built**	`apps/web/src/app/api/[...path]/route.ts`, `apps/api/src/serverless.ts`
Migrations on cold start under an advisory lock	**Built**	`createDatabase().migrate` in `apps/api/src/db/client.ts`
Migrations in the build step	**Built**, optional	`npm run vercel-build` with `MIGRATE_ON_START=false`
Private attachment storage on Vercel Blob	**Built**	`createStorage` in `apps/api/src/core/services.ts`
Realtime over SSE with database polling	**Built**	`apps/web/src/app/api/events/stream/route.ts`
Scheduled jobs via Vercel Cron	**Partial**	runs daily only (`0 3 * * *`)
CI pipeline (tests, type checks, Android builds)	**Planned**	no `.github/` directory
Production Postgres for a customer	**Planned**	demo uses in-memory PGlite
Error tracking, metrics, tracing, uptime checks	**Planned**	only function logs and `/api/health`
Email delivery	**Planned**	`sendMail` writes to `mail_outbox` only
Retention and purge jobs	**Planned**	`runScheduledJobs` prunes only `realtime_events`
Android release signing	**Planned**	`apps/android/app/build.gradle.kts` signs release with the debug key

9.3 Target topologies

Hydris will be offered in four topologies. They share one codebase and one database schema; they differ in where the API process runs, where data lives and how the plant connects. Choosing a topology is a commercial and regulatory decision as much as a technical one, so it is fixed in the customer's order form.

9.3.1 Topology A: managed SaaS on Vercel and Neon

This is the default for pilots and for most commercial customers. It is the current architecture with a real database and production secrets.

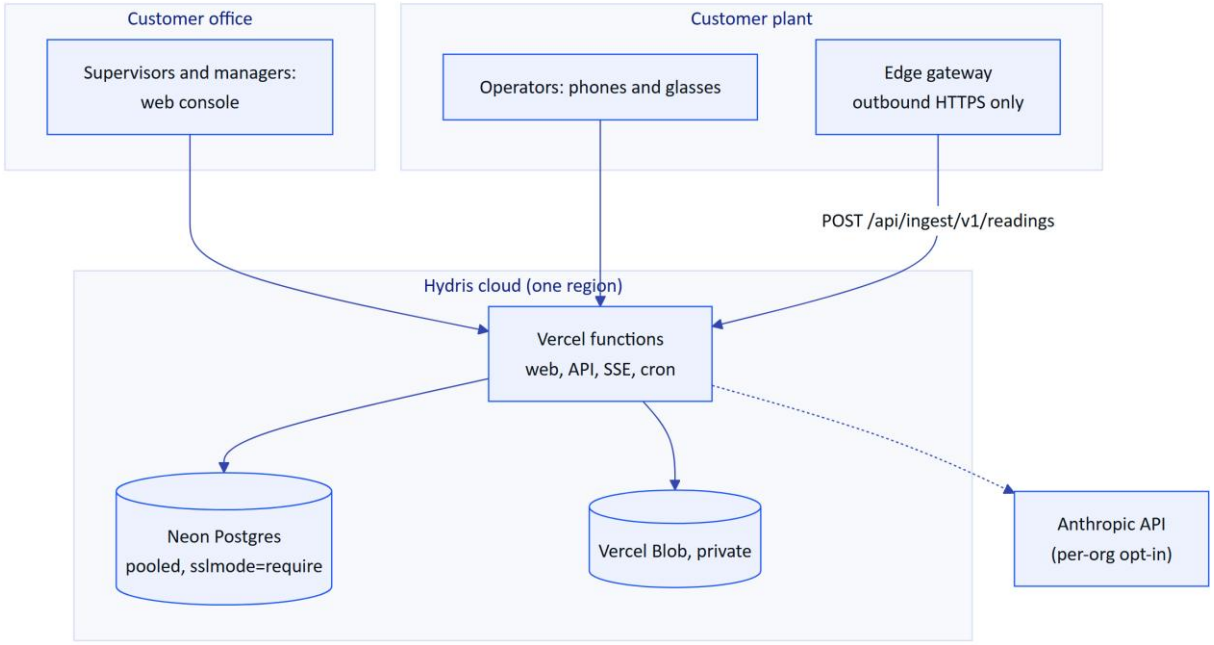

Figure 198. *Topology A, managed SaaS: Vercel functions with Neon Postgres and private Blob storage*

Table 240. Current infrastructure capability status

Setting	Value
Vercel plan	Pro (Hobby is limited to non-commercial use and daily crons)
Function region	Same region as the database (one region per deployment)
Database	Neon Postgres, pooled connection string, `sslmode=require`
Connections	3 per function instance (`poolMax: config.serverless ? 3 : 10` in `serverless.ts`)
Realtime	`REALTIME_MODE=db`
Attachments	Private Blob store (`BLOB_READ_WRITE_TOKEN`)
Cron	`/api/cron` every minute (requires Vercel Pro)
Tenancy	Many organizations per database, isolated by RLS

Topology A suits customers without strict residency requirements, and customers whose residency needs are met by an India region (see below). Its main limits are function duration (60 seconds), per-instance rate limiting and polling-based realtime with up to about 1.5 seconds of latency (`docs/operations/vercel.md`).

9.3.2 Topology A-IN: India region

Topology A deployed with every data-bearing component in India. The application code is identical; the differences are region choices and subprocessor configuration.

Table 241. Topology A configuration

Component	India option	Notes
Functions	Vercel region `bom1` (Mumbai)	Set in Project, Settings, Functions. Static assets are still served from the global edge network
Database	Neon in an India region if offered, otherwise AWS RDS or Aurora Postgres in `ap-south-1` (Mumbai) or `ap-south-2` (Hyderabad)	Confirm Neon region availability at contract time
Object storage	AWS S3 in `ap-south-1` behind the `Storage` interface	Vercel Blob region control must be confirmed; S3 is the safe default
Logs	Log drain to a store in India	Needed for CERT-In 180-day log retention in Indian jurisdiction
AI processing	Off by default for strict-residency tenants	Anthropic processing location is outside India; see HFG-SEC
Backups	Same region plus a second Indian region	Mumbai primary, Hyderabad secondary

DECISION

For Indian public-sector and municipal customers the default is Topology A-IN with AI features switched off until the customer signs an explicit addendum covering processing outside India. This keeps the default deployment inside India without forking the product.

9.3.3 Topology B: Vercel web with a container API

For larger tenants, long-lived SSE connections, or where background jobs must run every few seconds, the API runs as a long-lived container and the web build proxies `/api` to it by setting `API_URL`. The container entry point is `apps/api/src/server.ts`, which starts the timer-driven jobs in `startJobs` (stale rules every 30 seconds, pairing expiry every 60 seconds, realtime pruning every hour in `db` mode).

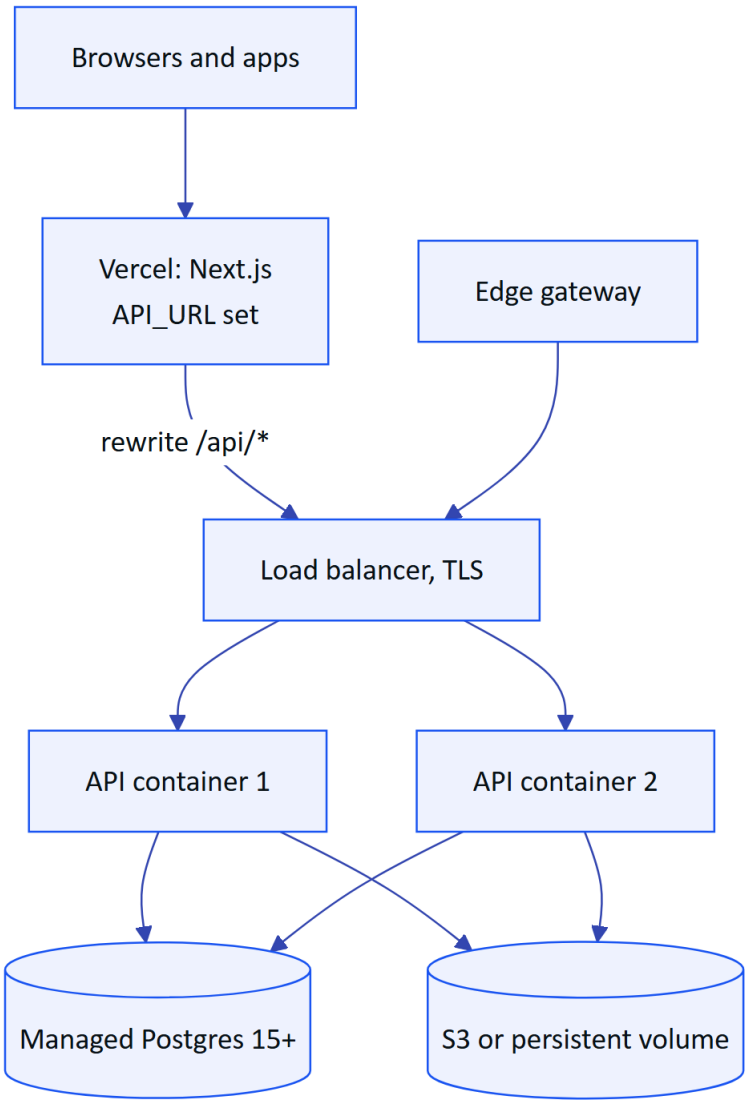

Figure 199. *Topology B, Vercel web with an API container on Render, Fly.io or AWS ECS*

The `render.yaml` blueprint is a ready starting point: a Docker web service with `healthCheckPath: /api/health`, a 5 GB disk mounted at `/data/files`, `SIMULATOR_ENABLED=false` and a generated `JWT_SECRET`. Two changes are required before running more than one container: set `REALTIME_MODE=db` (the in-memory bus is per process) and replace disk storage with S3-compatible object storage, because a disk volume cannot be shared between replicas.

9.3.4 Topology C: on-premises or plant-edge install

Some plants, especially those under strict OT security policies, cannot send data to a public cloud at all. For them Hydris runs on a server inside the plant network using `docker-compose.yml` (Postgres 17, API, web). The phones and the glasses bridge connect over the plant Wi-Fi to a local hostname. Nothing leaves the site.

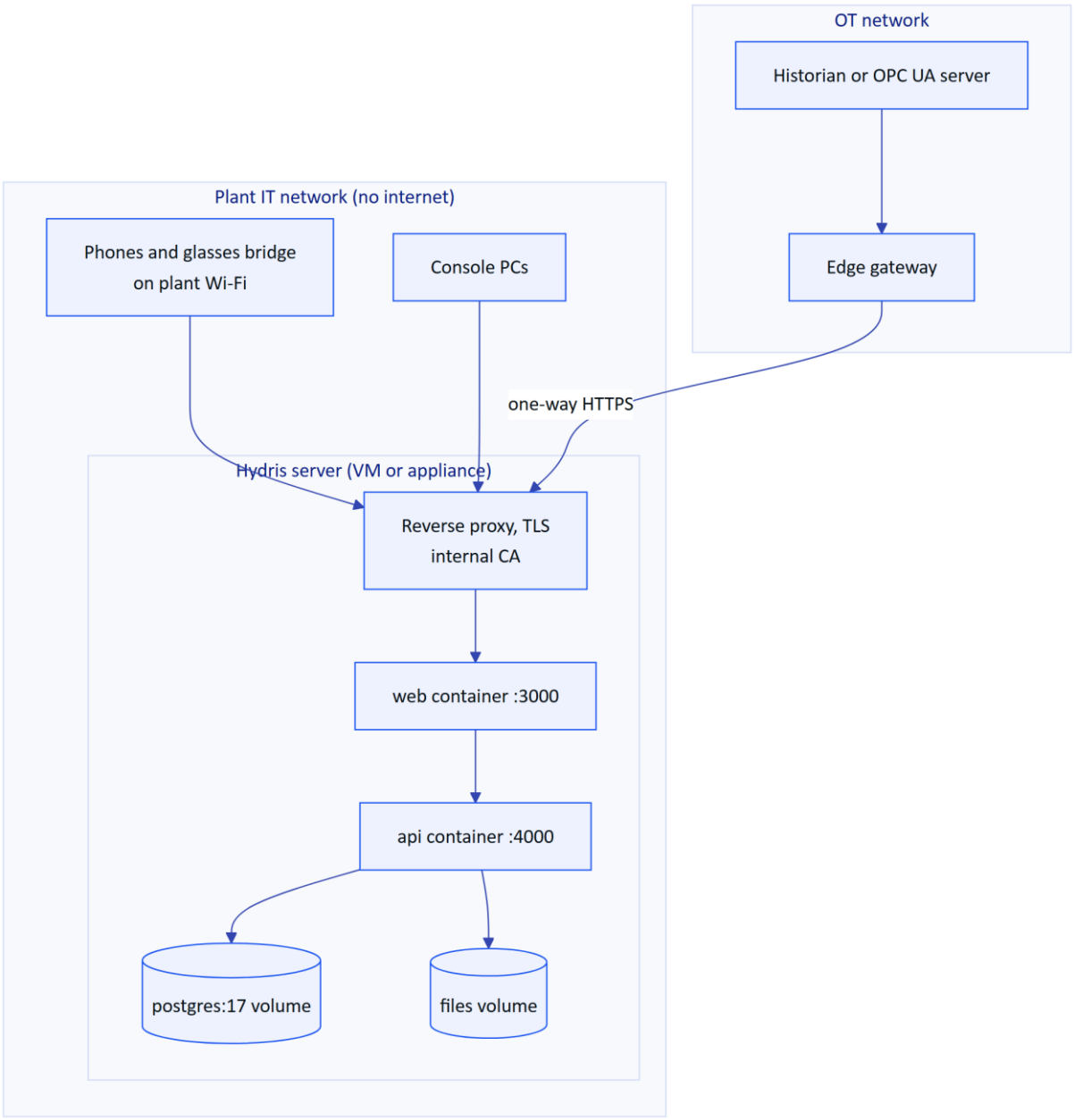

Figure 200. *Topology C, air-gapped plant install: everything on one server inside the plant network*

Table 242. India region component choices

Concern	Air-gapped approach	Status
Install	Versioned container images delivered as a signed archive, loaded with `docker load`	**Planned**
TLS	Reverse proxy (Caddy or nginx) with a certificate from the plant's internal CA	**Planned**
Updates	Quarterly offline bundles with migration notes; migrations run on API start (`server.ts`)	**Partial**
AI features	Disabled (no `ANTHROPIC_API_KEY`); rules-based assistant and on-device OCR remain	**Built** (automatic fallback)
Android apps	Installed from the customer MDM with `hydris_url` pointing at the local hostname	**Partial** (`hydris_url` is a build-time property today)

Concern	Air-gapped approach	Status
Meta glasses	DAT needs the Meta AI app, which needs internet at registration time	Constraint: confirm with Meta before quoting air-gapped glasses
Backups	Nightly `pg_dump` to a plant NAS plus a weekly offline copy	**Planned**
Support access	Escorted remote session on request; no standing vendor access	Policy

RISK

The Ray-Ban Meta glasses depend on the Meta AI companion app and Meta's Wearables Device Access Toolkit. A fully air-gapped plant may therefore support the phone apps and console but not the glasses. This must be stated in the proposal, not discovered at commissioning.

9.3.5 Topology D: hybrid with an edge gateway

The common case for industrial customers: the application runs in the cloud (A or A-IN) and a small gateway in the plant pushes telemetry outward. The gateway reads from the historian, PLC data server or OPC UA endpoint on the OT side and posts batches of up to 500 readings to `POST /api/ingest/v1/readings` with a gateway key (`hgw_` prefix, stored only as a SHA-256 hash in `gateways.key_hash`; `apps/api/src/modules/telemetry.ts`). Hydris never connects into the plant.

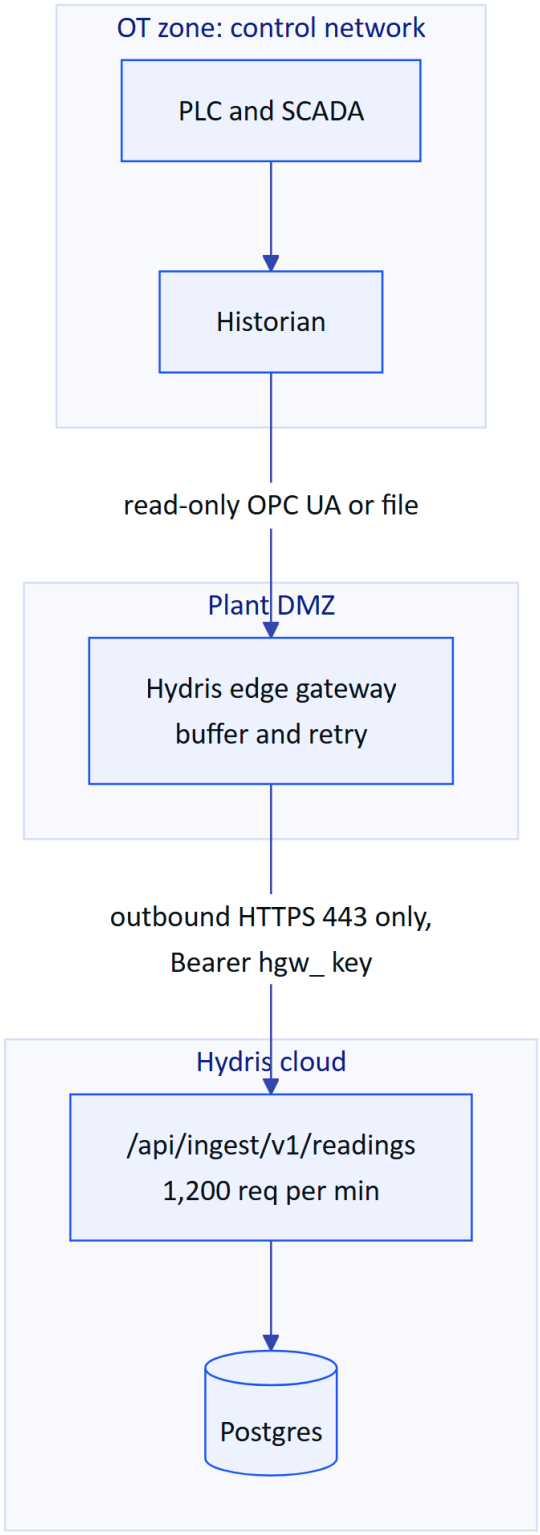

Figure 201. *Topology D, hybrid: an edge gateway pushes telemetry outward through the plant DMZ*

The ingest endpoint is idempotent per sensor and timestamp (unique `(sensor, ts)` index) and accepts an optional `messageId`, so the gateway can retry safely after a network outage. The gateway itself (store-and-forward software, hardware choice, OPC UA client) is **Planned**; the API contract it targets is **Built**.

9.3.6 Choosing a topology

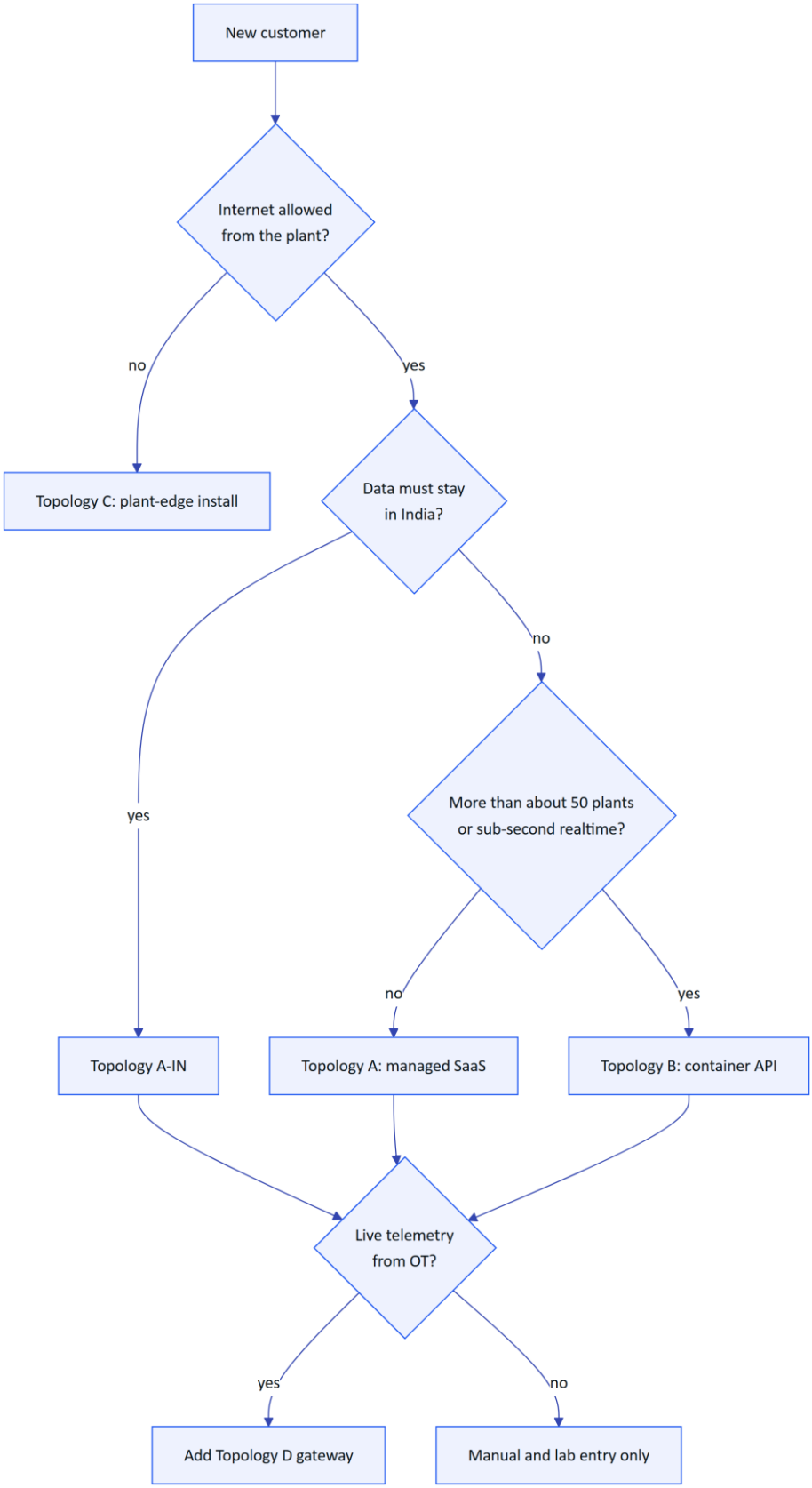

Figure 202. *Topology selection for a new customer*

Table 243. Plant-edge install considerations

Topology	Best for	Data location	Ops burden	AI features	Glasses	Status
A	Pilots, private industry	Chosen cloud region	Low (Hydris)	Opt-in	Yes	**Partial** (needs prod DB and cron fix)
A-IN	Indian PSUs, municipal STPs	India	Low (Hydris)	Off by default	Yes	**Planned**
B	Large multi-plant groups	Chosen region	Medium (Hydris)	Opt-in	Yes	**Planned** (artefacts built)
C	Air-gapped plants	Customer premises	High (shared)	Off	Phone yes, glasses to confirm	**Planned** (compose built)
D	Any customer with online instruments	Gateway in plant, data in A, A-IN or B	Medium (shared)	n/a	n/a	**Planned** (API built)

9.4 Environment matrix

Hydris uses five environments. They differ in data, secrets, who can reach them and which mobile builds point at them. Demo is deliberately separate from staging: a demo is a sales tool and resets itself, staging is a release gate.

Table 244. Topology comparison

Environment	Purpose	URL	Database	Secrets	Simulator	Mobile builds	Status
dev	Local development (`npm run dev`)	`localhost:3000`, API on `:4000`	PGlite at `./data/pglite`	Local `.env`	On	Debug APKs with `hydris_url` set locally	**Built**
preview	One per pull request	Vercel preview URL	Neon branch per preview (target) or demo mode	Preview scope in Vercel	On	None	**Partial** (demo mode today)
staging	Release candidate, hardware tests, pilot rehearsal	`staging.` subdomain (target)	Neon staging branch, seeded demo tenant	Staging scope	On for demo tenant only	Play internal track, DAT test channel	**Planned**
prod	Customer tenants	Customer-facing domain	Neon production, region pinned, PITR on	Production scope, managed secrets	Off (`false`)	Play closed track or MDM	**Planned**
demo	Public sales demo	`hydrisview.vercel.app`	In-memory PGlite per instance	Public demo secrets	On	Debug builds for demos	**Built**

Today `main` deploys straight to the demo URL. The target separates them into two Vercel projects pointing at the same repository: a demo project that tracks `main` with no `DATABASE_URL`, and a production project that deploys only on release tags after the staging gate. This keeps sales demos current without coupling them to customer releases.

> **TIP**
> Neon branches make per-preview databases cheap: each preview deployment gets a copy-on-write branch of the staging database, migrations run against it on the first request, and the branch is deleted when the pull request closes. This is the single biggest improvement to review quality we can make cheaply.

9.5 Configuration reference

All server configuration is read from environment variables. `apps/api/src/config.ts` validates the core set with zod and refuses to start on invalid values; a handful of operational variables are read directly with `process.env` elsewhere. In development the API also loads `.env` from the repository root or the working directory with `process.loadEnvFile`.

9.5.1 Variables validated by config.ts

Table 245. Environment matrix

Variable	Required	Default	Validation	Purpose
`NODE_ENV`	No	`development`	`development`, `test` or `production`	`production` sets `isProd`, which marks refresh cookies `Secure`
`API_PORT`	No	`4000`	integer	Listen port for the container server
`HOST`	No	`127.0.0.1`	string	Listen address; set `0.0.0.0` in containers
`DATABASE_URL`	Yes in prod	`./data/pglite`	string	`postgres://...` for Postgres, a directory path for PGlite, or `memory://`
`JWT_SECRET`	Yes	demo value in demo mode	at least 32 characters	Signs every JWT (user, device, MFA challenge, demo session)
`DATA_KEY`	Yes	demo value in demo mode	exactly 64 hex characters	AES-256-GCM key for MFA secrets at rest
`PUBLIC_APP_URL`	Yes in prod	`http://localhost:3000`	URL	Base of pairing, invitation and reset links
`STORAGE_DIR`	Containers	`./data/files`	string	Disk storage for attachments when Blob is not configured
`SEED_ADMIN_EMAIL`	Seed only	none	optional	First platform admin for `apps/api/src/seed/cli.ts`
`SEED_ADMIN_PASSWORD`	Seed only	none	optional	Its password (12+ characters)
`SEED_DEMO_PASSWORD`	Seed only	none	optional	Password for seeded demo role accounts
`SIMULATOR_ENABLED`	No	`true`	`true` or `false`	Enables the demo-tenant simulator; `false` in production
`REALTIME_MODE`	No	`memory`	`memory` or `db`	`db` is required on serverless or with more than one instance
`BLOB_READ_WRITE_TOKEN`	On Vercel	none	optional	Switches attachments to a private Vercel Blob store
`CRON_SECRET`	On Vercel	none	optional	Bearer secret required by `/api/cron`

Variable	Required	Default	Validation	Purpose
SERVERLESS	No	none	1 or true	Forces serverless behaviour (no interval timers)
VERCEL	Auto	set by Vercel	optional	Detects Vercel: serverless behaviour and demo auto-detection
DEMO_MODE	No	auto	true or false	Forces demo mode on or off

9.5.2 Variables read directly from the environment

Table 246. Configuration validated by apps/api/src/config.ts

Variable	Read in	Purpose
DEMO_PERSIST	config.ts (demoDefaults)	With DEMO_MODE and a Postgres URL, seeds the demo into that database instead of memory
ANTHROPIC_API_KEY	core/vision-extract.ts (visionAvailable)	Enables Claude vision extraction for audit scans and the Claude engine of the voice assistant
ANTHROPIC_AUTH_TOKEN	core/vision-extract.ts	Alternative Anthropic credential; either one enables the AI features
MIGRATE_ON_START	serverless.ts	false skips cold-start migrations (when npm run vercel-build runs them)
HYDRIS_MIGRATIONS_DIR	serverless.ts (findMigrationsFolder)	Overrides where the bundle looks for drizzle/meta/_journal.json
API_LOG	serverless.ts	true enables Fastify request logging at info level
API_URL	apps/web/next.config.ts	Switches the web build to proxy /api to an external API
VERCEL_PROJECT_PRODUCTION_URL, VERCEL_URL	config.ts	Demo mode only: derives PUBLIC_APP_URL
POSTGRES_PASSWORD	docker-compose.yml	Password for the compose database owner

9.5.3 Android build properties

The Android apps are configured at build time through local.properties (not committed), read in each app/build.gradle.kts:

Table 247. Operational variables outside the zod schema

Property	App	Default	Purpose
hydris_url	both	https://hydrisview.vercel.app	Server the app talks to (BuildConfig.HYDRIS_URL)
mwdat_application_id	both	0	Meta Wearables application ID (0 in DAT Developer Mode)
mwdat_client_token	both	0	Meta Wearables client token
sdk.dir	both	none	Android SDK path

> **WARNING**
>
> Because `hydris_url` is compiled in, a customer on Topology A-IN, B or C needs a separate build today. The target is an MDM managed-configuration key (`hydris_url`) read at run time, with the build value as fallback, so one signed APK serves every customer.

9.5.4 Secrets handling rules

1. Generate `JWT_SECRET`, `DATA_KEY` and `CRON_SECRET` with 32 random bytes each (`node -e "console.log(require('crypto').randomBytes(32).toString('hex'))"`).
2. Store them only in the Vercel project's environment variables (production scope separate from preview), or in the cloud provider's secrets manager for containers. Never in the repository, never in `NEXT_PUBLIC_*`.
3. `DATA_KEY` is not rotatable without re-encrypting every `users.mfa_secret_enc` value. Losing it disables MFA for every user. Keep an offline escrow copy under two-person control.
4. Rotating `JWT_SECRET` signs everyone out (all access tokens and the MFA step become invalid; refresh cookies remain valid because they are random tokens checked against hashes). It also changes the salt used for the hashed IP on website inquiries (`modules/public.ts`).
5. Preview deployments must not share production secrets or the production database.

9.6 Build and release pipeline

9.6.1 How a release happens today

There is no CI. A merge or push to `main` triggers Vercel's Git integration, which installs, builds and deploys. Tests and type checks are run by developers locally (`npm test`, `npm run typecheck`, `npm run test:e2e`). The Android apps are built from Android Studio or Gradle on a developer machine and installed with `adb`.

Table 248. Android build-time properties

Step	Command	Where defined	Notes
Install	`npm ci --include=dev`	root `vercel.json`	Dev dependencies are needed for the TypeScript build
Build	`npm run build` (runs `npm run build -w @hydris/web`)	root `package.json`	API is compiled into the web build via `transpilePackages`
Migrate	On first request per instance, or `npm run vercel-build`	`serverless.ts`, root `package.json`	Advisory lock key `4872917633`
Seed	`npx tsx apps/api/src/seed/cli.ts [--demo]` from an operator machine	`docs/operations/vercel.md`	Creates the platform admin; safe to rerun

The weaknesses are clear: a type error or failing test can reach production, and a migration can run in production before anyone has run it against realistic data. Vercel's instant rollback protects the web tier but not the database.

9.6.2 Release versioning

Table 249. Current build and release steps

Artefact	Version source	Target scheme
Web and API	root `package.json` `version` (`0.2.0`)	Semantic version tag `vX.Y.Z` on `main`; tag triggers production
Database schema	Drizzle journal `apps/api/drizzle/meta/_journal.json`	Monotonic migration index, never edited after release
Hydris Field app	`versionCode = 1`, `versionName = "1.0.0"`	`versionCode` from the CI run number, `versionName` from the tag
Glasses bridge	`versionCode = 1`	Same as above

9.6.3 Release train

Table 250. Versioning of release artefacts

Cadence	What ships	Gate
Continuous	Web and API fixes behind no flag	CI green, one reviewer
Fortnightly	Features, migrations	CI green, staging soak 48 hours, release notes
Monthly	Android apps to the pilot track	Staging hardware test on real glasses
Quarterly	Offline bundle for Topology C	Signed archive, upgrade rehearsal

9.7 Proposed CI/CD with GitHub Actions

The pipeline below is **Planned**. It uses only commands that already exist in the repository's `package.json` scripts and Gradle wrappers, so it can be added without code changes.

9.7.1 Pipeline design

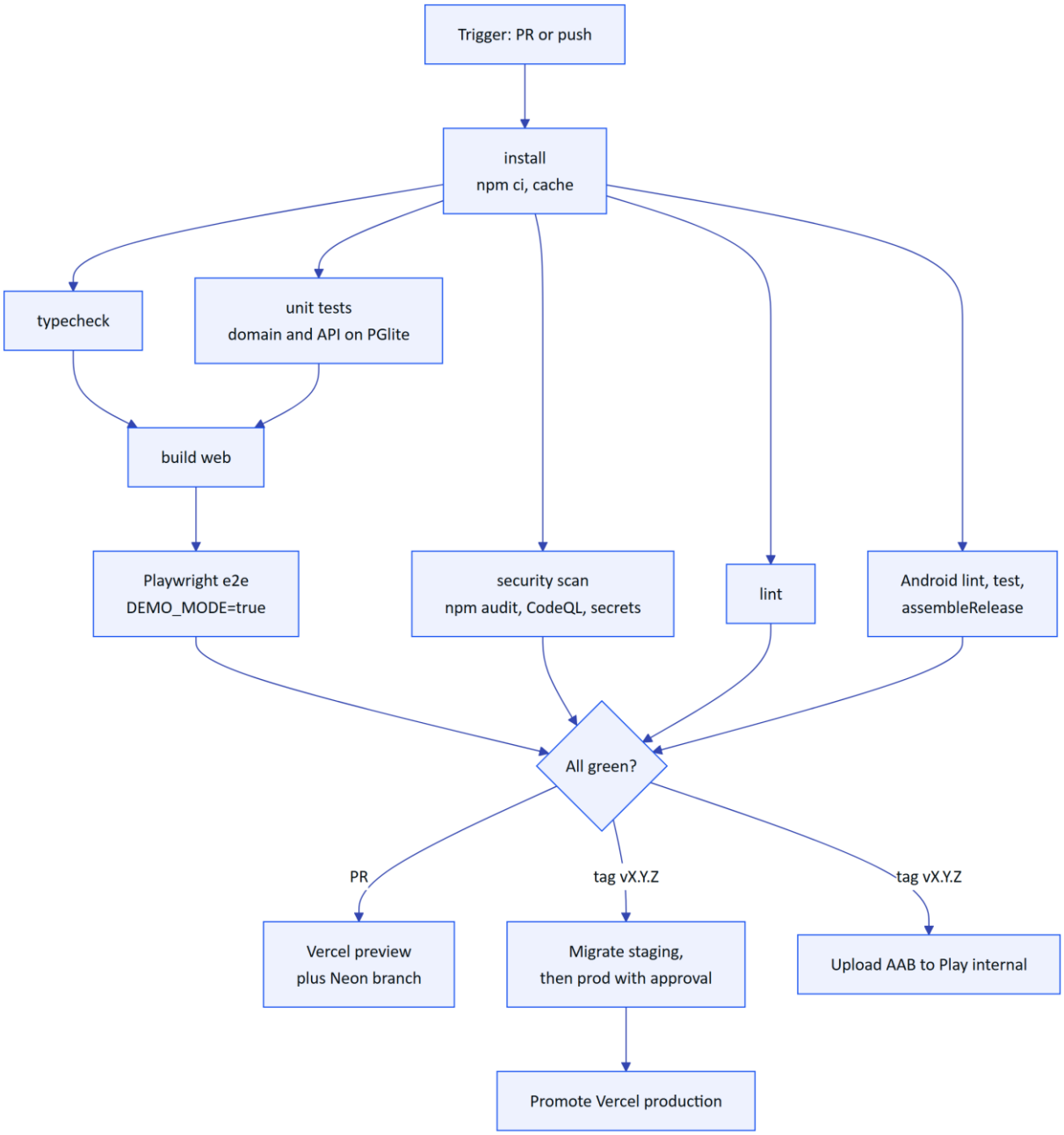

Figure 203. *Proposed GitHub Actions workflow: jobs and gates*

Table 251. Target release cadence

Job	Runs	Commands	Blocks merge	Target time
install	Every PR and push	`npm ci` with npm cache keyed on `package-lock.json`	Yes	1 min
typecheck	Every PR	`npm run typecheck`	Yes	2 min
lint	Every PR	`npm run lint`	Yes	1 min
unit	Every PR	`npm test` (domain, then API against PGlite)	Yes	3 min
build	Every PR	`npm run build`	Yes	4 min

Job	Runs	Commands	Blocks merge	Target time
e2e	Every PR	`npx playwright install chromium`, `npm run test:e2e` against a demo-mode server	Yes	6 min
security	Every PR, nightly	`npm audit --omit=dev --audit-level=high`, CodeQL, secret scanning	High and critical only	5 min
android	PRs touching `apps/android*`, tags	`./gradlew lint testDebugUnitTest assembleRelease` in both apps	Yes	8 min
migrate	Tags	`npm run db:migrate` against staging, then production after approval	n/a	1 min
deploy	Tags	`vercel deploy --prebuilt --prod` or promote the staging deployment	n/a	3 min
publish-android	Tags	Sign with upload key, upload AAB to Play internal track	n/a	4 min

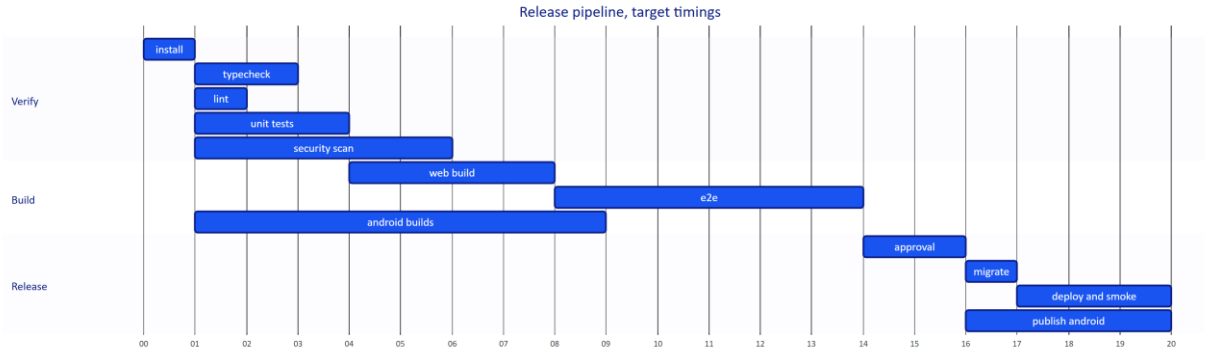

Figure 204. *Target duration of the pipeline stages for a release tag (minutes from start)*

9.7.2 Example workflow

```
# .github/workflows/ci.yml (proposed)
name: ci
on:
  pull_request:
  push:
    branches: [main]
    tags: ['v*']
jobs:
  verify:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with: { node-version: 22, cache: npm }
      - run: npm ci
      - run: npm run typecheck
      - run: npm test
      - run: npm run build
  e2e:
    needs: verify
    runs-on: ubuntu-latest
    env: { DEMO_MODE: 'true' }
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with: { node-version: 22, cache: npm }
      - run: npm ci
      - run: npx playwright install --with-deps chromium
      - run: npm run test:e2e
  android:
```

```
    runs-on: ubuntu-latest
    strategy:
      matrix: { app: [apps/android, apps/glasses-bridge-android] }
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-java@v4
        with: { distribution: temurin, java-version: 21 }
      - run: ./gradlew lint testDebugUnitTest assembleRelease
        working-directory: ${{ matrix.app }}
```

9.7.3 Gates and policies

Table 252. Proposed CI jobs

Gate	Rule
Branch protection	`main` requires a pull request, one approving review and all required checks
Migrations	Any change under `apps/api/drizzle/` needs a second reviewer from platform engineering
Secrets	Production secrets live in a GitHub environment `production` with required reviewers
Dependencies	Renovate or Dependabot weekly; lockfile changes go through CI like code
Provenance	Release tags signed; build logs retained 90 days
Smoke test	After deploy, `/api/health` returns `ok` and a scripted login and alarm list succeed
Automatic rollback	If the smoke test fails, promote the previous Vercel deployment (schema permitting, see below)

> **NOTE**
>
> `playwright.config.ts` starts `npm run dev` as its web server. In CI the e2e job should start a production build in demo mode instead (`next start` with `DEMO_MODE=true`) so the tests exercise the same code path as Vercel. The comment in the config already anticipates CI use.

9.8 Database migrations and rollback

9.8.1 How migrations run

Migrations are plain SQL files generated by Drizzle Kit (`npm run db:generate`) and stored in `apps/api/drizzle/` with a journal in `meta/_journal.json`. There are nine today: the base schema (`0000_init`), row-level security (`0001_rls`), corrections and UI profiles (`0002`), RLS for those tables (`0003`), role membership for managed Postgres (`0004`), the realtime event log (`0005`, `0006`) and audit documents (`0007`, `0008`).

On Postgres, `createDatabase().migrate` in `apps/api/src/db/client.ts` takes a dedicated connection, acquires a session-level advisory lock (`pg_advisory_lock(4872917633)`), runs Drizzle's migrator and releases the lock. This serializes concurrent migrators: several serverless instances cold-starting together, or a deploy step racing a running instance.

Table 253. Release gates

Mode	How	When to use
Cold start (default)	`migrate()` in `serverless.ts` unless `MIGRATE_ON_START=false`	Demo, small pilots
Build step	`npm run vercel-build` runs `db:migrate` then build; set `MIGRATE_ON_START=false`	Topology A production
Pipeline step	CI `migrate` job before promotion	Target for all production deploys

Mode	How	When to use
Container start	`server.ts` migrates on boot (Dockerfile comment)	Topologies B and C

> **WARNING**
> Running migrations in the Vercel build step means a migration is applied before the new deployment is promoted, and is not undone if the build later fails. That is safe only if every migration is backward compatible with the code currently in production. The expand and contract rule below makes that true.

9.8.2 Managed Postgres prerequisites

Migration `0001_rls.sql` creates the `NOLOGIN` role `hydris_app`, grants it table privileges, revokes `UPDATE` and `DELETE` on `audit_events`, and enables and forces RLS on every tenant table. Migration `0004` grants `hydris_app` to the connecting user `WITH SET TRUE` so the application can `SET LOCAL ROLE hydris_app` (needed on Postgres 16 and later). If the provider forbids it, an administrator runs once: `GRANT hydris_app TO <app_user> WITH SET TRUE;` (`docs/operations/vercel.md`).

The comment in `0001_rls.sql` is important for production: owner and superuser connections bypass RLS, so the application must always switch to `hydris_app`, which `withTenant` and `withSystem` in `apps/api/src/db/tenant.ts` do for every transaction. The target is a separate migration user (owner) and runtime user (member of `hydris_app`, no ownership), so a code path that forgets to switch role still cannot bypass RLS.

9.8.3 Expand and contract

Every schema change ships in two releases so that the old and new application versions both work against the schema in between.

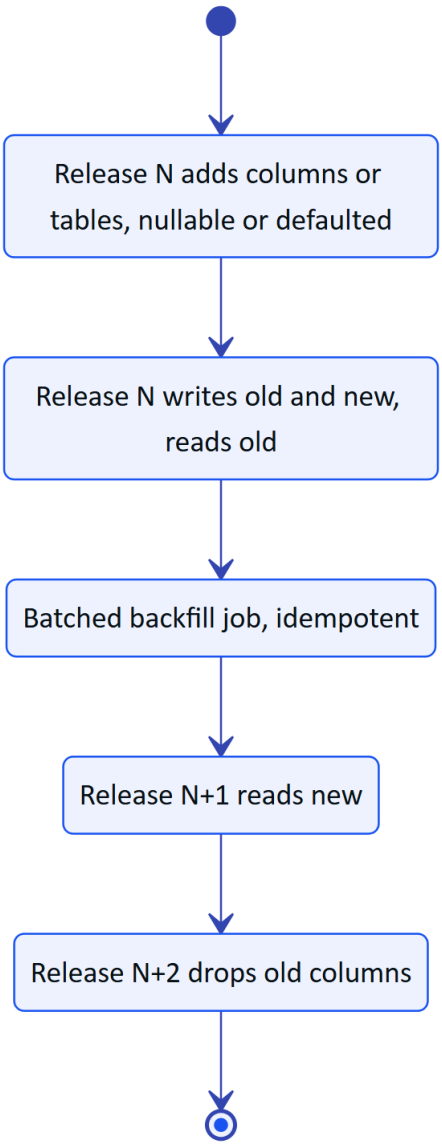

Figure 205. *Expand and contract keeps every deployment rollback-safe*

Table 254. Migration execution modes

Change	Allowed in one release	Needs expand and contract
Add table, add nullable column, add index concurrently	Yes	No
Add `NOT NULL` column with default	Yes on Postgres 11+	No
Rename column or table	No	Yes (add, dual write, switch, drop)
Change column type	No	Yes
Drop column or table	No	Yes, only after no deployed code reads it
New RLS policy on a new table	Yes, in the same migration as the table	No
Tighten an existing RLS policy	Review required	Test against staging data first

9.8.4 Rollback

Table 255. Migration safety rules

Situation	Action	Data impact
Bad web or API code, schema unchanged	Vercel instant rollback to the previous deployment	None
Bad code after an expand migration	Roll back code; leave the additive schema in place	None
Bad migration caught on staging	Fix forward with a new migration; never edit a released migration file	None
Bad migration in production, data intact	Write a forward migration that reverses it; deploy through the normal pipeline	None
Migration corrupted or deleted data	Neon point-in-time restore to a branch just before the migration, verify, then switch `DATABASE_URL` or copy rows back	Writes after the restore point must be replayed or accepted as lost

Drizzle migrations are forward-only. We do not write down migrations; the fix for a bad migration is another migration. Point-in-time restore is the last resort and is rehearsed quarterly (see backup and disaster recovery).

9.9 Background jobs and scheduling

9.9.1 Jobs that exist

`apps/api/src/jobs.ts` defines one pass of every background job in `runScheduledJobs`, and a timer-driven version for long-running servers in `startJobs`.

Table 256. Rollback playbook

Job	Function	Container interval (`startJobs`)	Serverless trigger	Status
Stale-data alarms	`evaluateStaleRules` (`modules/alarms.ts`)	30 s	`/api/cron`	**Built**
Pairing expiry	`expirePairings`	60 s	`/api/cron`	**Built**
Realtime log pruning (1 day)	`pruneRealtimeEvents` (`core/events.ts`)	1 h in `db` mode	`/api/cron`	**Built**
Demo simulation steps	`SimulationEngine.advanceAll`	continuous	`/api/cron`, and lazily on requests in demo mode (`DemoTicker`)	**Built**
Resume running simulations after restart	in `startJobs`	on boot	n/a	**Built**
Expired session and pairing purge	none	none	none	**Planned**
Mail delivery from `mail_outbox`	none	none	none	**Planned**
Retention and archival	none	none	none	**Planned**
Audit page image migration to object storage	none	none	none	**Planned**

Each job is wrapped in its own `catch`, so one failure is reported in the JSON (`staleRules: "error: ..."`) and does not stop the others. That JSON report is the only job telemetry today.

9.9.2 The daily cron problem

The root `vercel.json` schedules `/api/cron` once a day at 03:00 UTC. On Vercel, stale-data alarms ("no reading for 15 minutes") therefore fire up to 24 hours late, and expired pairing challenges are only marked expired once a day (pairing codes are still rejected when used after their 300-second TTL, because the poll and approve paths check `expiresAt`).

> **CAUTION**
> Stale-data alarms are a safety-relevant feature for operators: a silent instrument looks like a healthy one. Before any customer pilot on Topology A, change the schedule to `* * * * *` on Vercel Pro, or run the jobs from an external scheduler, or use Topology B. This is the first item on the go-live checklist.

Table 257. Background jobs

Option	Frequency	Cost	Trade-off
Vercel Pro cron `* * * * *`	1 min	Included in Pro	Simplest; one invocation per minute
External scheduler (GitHub Actions schedule, cron-job.org, cloud scheduler) calling `/api/cron`	1 to 5 min	Low	Another moving part; GitHub schedules can drift by minutes
Topology B container	30 to 60 s	Container cost	Best freshness; more operations

9.9.3 Job design rules for new jobs

1. Every job is idempotent and safe to run concurrently on two instances (use `UPDATE ... WHERE` predicates or advisory locks, as the migration code does).
2. Every job runs through `withSystem` so RLS and append-only grants still apply.
3. Batch large work (purges, backfills) into chunks under 10 seconds so a 60-second function limit is never hit.
4. Report counts in the cron JSON and emit a metric per job (see observability).

9.10 Android app distribution

Hydris ships two Android apps. Hydris Field (`apps/android`, `ai.hydris.field`) is the native operator app with camera capture, on-device ML Kit OCR and the Meta DAT integration. The Glasses bridge (`apps/glasses-bridge-android`, `ai.hydris.glassesbridge`) hosts the `/glasses` web app in a WebView and connects the Ray-Ban Meta camera, capture button and microphone through DAT 1.0.1. Both target SDK 36 with a minimum of SDK 31 (Android 12), matching Meta's DAT requirement.

9.10.1 Distribution channels

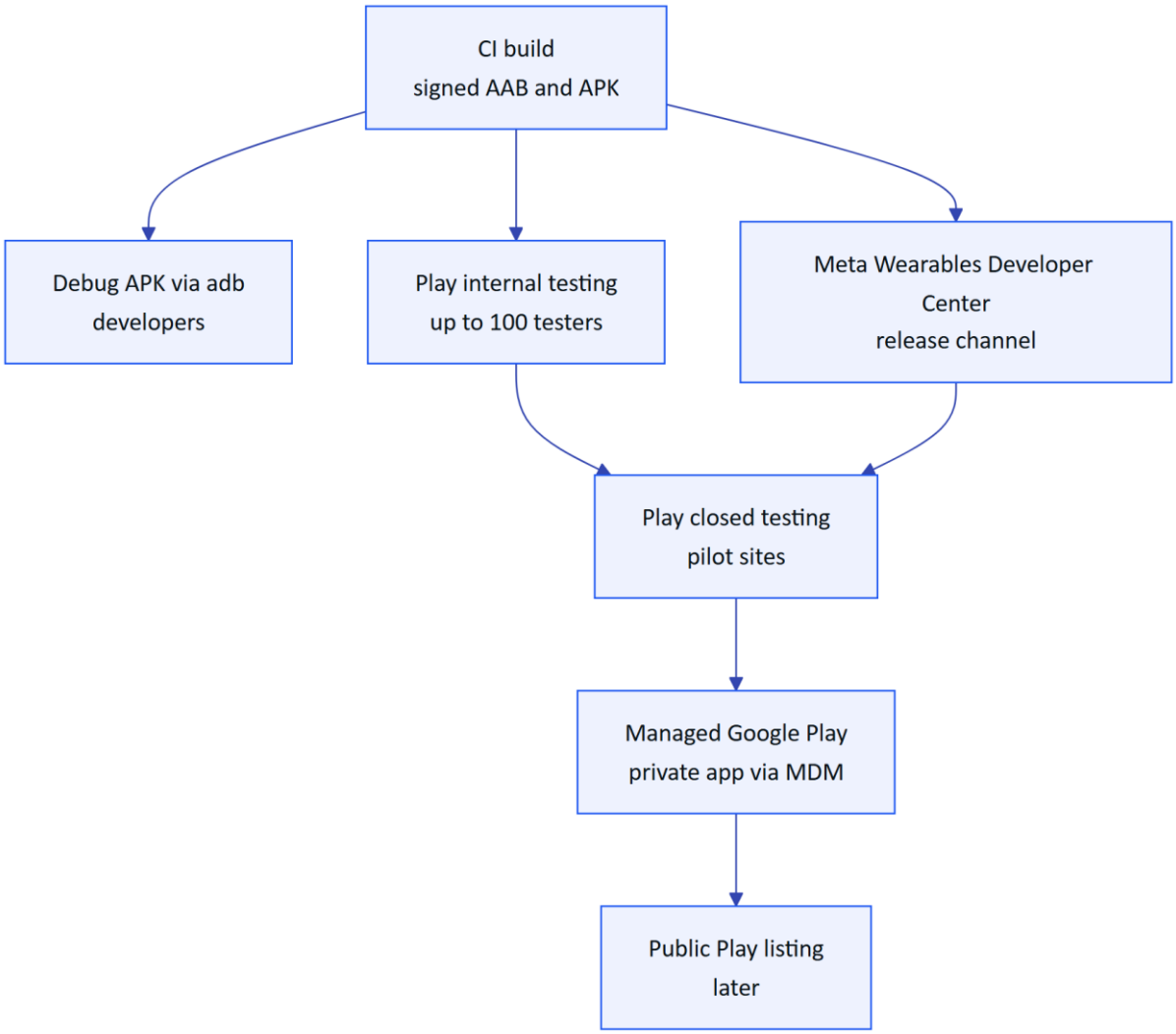

Figure 206. *Android distribution channels by stage*

Table 258. Options for job frequency on Vercel

Channel	Audience	DAT requirement	Use	Status
Debug APK with `adb`	Hydris developers	Developer Mode in the Meta AI app (IDs `0`/`0`)	Daily development	**Built** (a debug APK is checked in at `apps/glasses-bridge-android/hydris-glasses-debug.apk`)
Play internal testing	Up to 100 named testers	Developer Mode until the app is registered	Pilot rehearsals, staging	**Planned**
Meta Wearables Developer Center release channel	Named DAT testers who are not developers	Registered app ID and client token	Glasses features for pilot users	**Planned**
Play closed testing	Pilot plant staff	Registered app	Pilot	**Planned**
Managed Google Play private app	Enterprise fleets via MDM (Intune, Workspace ONE, Knox)	Registered app	Commercial rollout	**Planned**

Channel	Audience	DAT requirement	Use	Status
Public Play listing	General availability	Meta publishing approval	Later	**Planned**

> **NOTE**
> A debug APK is committed to the repository. Remove it from version control and attach builds to CI artefacts or releases instead; binaries in Git bloat the repository and cannot be traced to a commit reliably.

9.10.2 Signing keys

Table 259. Android distribution channels

Key	Purpose	Holder	Storage	Status
Debug key	Local builds	Each developer	Android SDK default	**Built**
Upload key	Signs AABs uploaded to Play	CI	GitHub environment secret (base64 keystore plus passwords), offline backup under two-person control	**Planned**
App signing key	Signs APKs delivered to devices	Google (Play App Signing)	Google-managed	**Planned**
Enterprise sideload key	Signs APKs for air-gapped MDM without Play	Hydris release manager	HSM or offline vault	**Planned** (Topology C only)
Meta DAT credentials	`mwdat_application_id`, `mwdat_client_token`	CI	GitHub environment secret, injected into `local.properties` at build	**Planned**

> **CAUTION**
> `apps/android/app/build.gradle.kts` currently signs the release build type with the debug key (`signingConfig = signingConfigs.getByName("debug")`, with a comment to replace it). A release signed with a debug key cannot be uploaded to Play and must never be distributed to customers. Configure the upload key from CI secrets before the first pilot build.

9.10.3 Enterprise MDM configuration

Table 260. Android signing and credential inventory

Managed configuration key	Type	Purpose	Status
`hydris_url`	string	Server URL for this customer (A, A-IN, B or C)	**Planned**
`org_hint`	string	Pre-selects the organization on the sign-in screen	**Planned**
`allow_ai_features`	boolean	Hides AI-assisted extraction where the org has opted out	**Planned**
`kiosk_mode`	boolean	Device-owner mode for shared phones at a plant	**Planned**

Shared phones (one phone per shift, not per person) are common in plants. The app already encrypts credentials with an Android Keystore key and disables backup (`android:allowBackup="false"` in both manifests), so a shared device does not leak the previous user's session to a backup. Sign-out on shift change is a process requirement documented in the Operations Runbook.

9.11 Observability

9.11.1 Current state

Table 261. Proposed MDM managed configuration

Signal	Today	Where
Logs	Vercel function logs; Fastify logs only when `API_LOG=true`; 5xx errors logged by the error handler; route handlers log start failures	`serverless.ts`, `app.ts`, `route.ts`
Health	`GET /api/health` returns `ok` and server time	`apps/api/src/app.ts`
Job results	JSON report from `/api/cron`	`apps/web/src/app/api/cron/route.ts`
Business signals	Audit events and alarm history in the database	`audit_events`, `alarm_event_history`
Metrics, traces, error tracking, uptime	None	**Planned**

`/api/health` does not touch the database, so it reports the function as up even when Postgres is unreachable. A deep health check (`/api/health?deep=1` running `select 1` and checking the migration journal) is **Planned**.

9.11.2 Target architecture

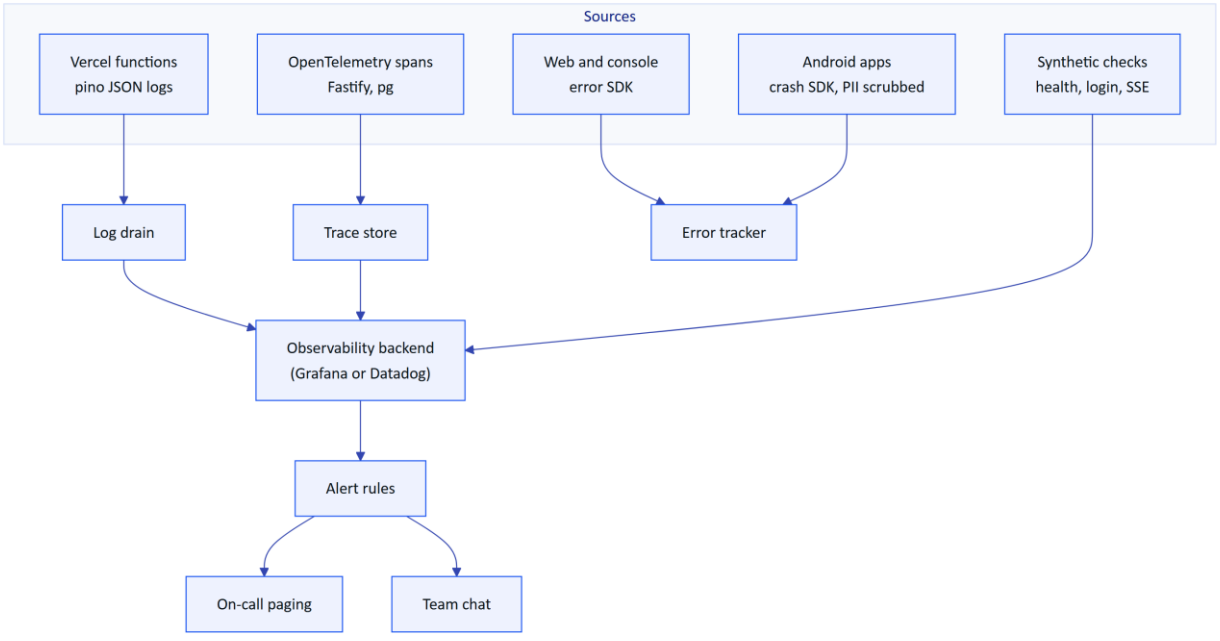

Figure 207. *Target observability pipeline*

Table 262. Observability today

Signal	Tooling (proposed)	Content	Residency note
Logs	Vercel log drain to Grafana Loki, Datadog or an India-hosted store	Structured JSON with request id, org id, route, status, duration; no tokens, no bodies	CERT-In requires 180 days of logs in India for Indian service providers
Metrics	OpenTelemetry metrics or log-derived metrics	Request rate, error rate, latency, cold starts, DB pool waits, job counts	
Traces	OpenTelemetry for Fastify and `pg`	Span per request and per query	Sample 10% in production
Errors	Sentry (or equivalent) for web, console and Android	Stack traces with PII scrubbing and IP anonymisation	Choose EU or self-hosted for strict tenants
Uptime	External synthetic checks every minute	`/api/health`, login, SSE connect, cron freshness	

9.11.3 Key metrics

Table 263. Target observability stack

Metric	Why it matters	Alert threshold (target)
API 5xx rate	Overall health	Over 1% for 5 min
p95 API latency	Console and glasses responsiveness	Over 800 ms for 10 min
Ingest lag (now minus newest reading per gateway)	Live telemetry is the core promise	Over 5 min for any active gateway
Cron last success age	Stale alarms depend on it	Over 3 min (after the per-minute fix)
SSE reconnect rate	Realtime health	Over 3 per client per 5 min
`auth.login_failed` rate	Credential stuffing	10x baseline
`auth.refresh_reuse_detected`, `device.refresh_reuse_detected`	Token theft	Any occurrence
Postgres connections in use	Pool exhaustion with many instances	Over 80% of the pooler limit
Anthropic error rate	AI features degrade to rules engine	Over 20% for 15 min

9.11.4 Service level objectives

SLOs are targets for the pilot and the first commercial year. They are not measured results.

Table 264. Key metrics and alert thresholds

SLI	SLO target	Window	Error budget
Availability of `/api/*` (non-5xx, excluding 429)	99.5%	30 days	3.6 hours
Latency: p95 of read API calls	Under 500 ms	30 days	5% of requests may exceed
Telemetry freshness: readings visible within 60 s of ingest	99%	30 days	1% of readings
Stale-data alarm raised within 2 min of the configured threshold	99%	30 days	1% of evaluations

SLI	SLO target	Window	Error budget
Realtime delivery: event visible on an open console within 5 s	99%	30 days	1% of events
Audit scan upload success	99%	30 days	1% of uploads
Cron success	99.9% of runs	30 days	about 43 failed runs at 1 per min

Error budget policy: while less than half of the monthly budget is consumed, releases proceed normally. Between half and all of it, risky changes need approval from the platform lead. Once the budget is exhausted, feature releases freeze and only reliability work ships until the 30-day window recovers, and every budget-exhausting incident gets a post-incident review.

> **NOTE**
> The SLOs exclude scheduled maintenance announced 72 hours ahead, customer-side network outages and the Anthropic API (AI features degrade to the rules engine by design, see `answerWithRules` in `apps/api/src/modules/voice-assistant.ts`).

9.12 Backup and disaster recovery

9.12.1 Objectives

Table 265. Service level objectives (targets)

Tier	Data	RPO target	RTO target	Mechanism
1	Postgres (all tenant records, audit trail, telemetry)	15 min	4 h	Neon point-in-time restore plus daily logical dump
1	Audit page images (today inside Postgres)	15 min	4 h	Same as Postgres
2	Attachments in Blob or S3	24 h	8 h	Object versioning and daily copy to a second region
2	Secrets (`JWT_SECRET`, `DATA_KEY`, `CRON_SECRET`, gateway keys are hashed in DB)	n/a	1 h	Secrets manager plus offline escrow of `DATA_KEY`
3	Application code and build	0	1 h	Git, Vercel deployment history
3	Logs	24 h	best effort	Log drain retention

The pilot targets are RPO 15 minutes and RTO 4 hours. The commercial target after the first year is RPO 5 minutes and RTO 1 hour for Topology A.

9.12.2 Backup schedule

Table 266. Recovery objectives (pilot targets)

Backup	Frequency	Retention	Location	Status
Neon point-in-time history	Continuous	7 days (Launch) to 30 days (Scale), by plan	Same region	**Planned** (no production DB yet)
Logical dump (`pg_dump -Fc`)	Daily	35 daily, 12 monthly	Object storage, same jurisdiction, different account	**Planned**

Backup	Frequency	Retention	Location	Status
Object storage versioning	Continuous	30 days of non-current versions	Same bucket	**Planned**
Cross-region copy	Daily	35 days	Second region in the same country	**Planned**
Topology C local dump	Nightly	14 nightly, 8 weekly	Plant NAS, weekly offline copy	**Planned**

9.12.3 Disaster scenarios

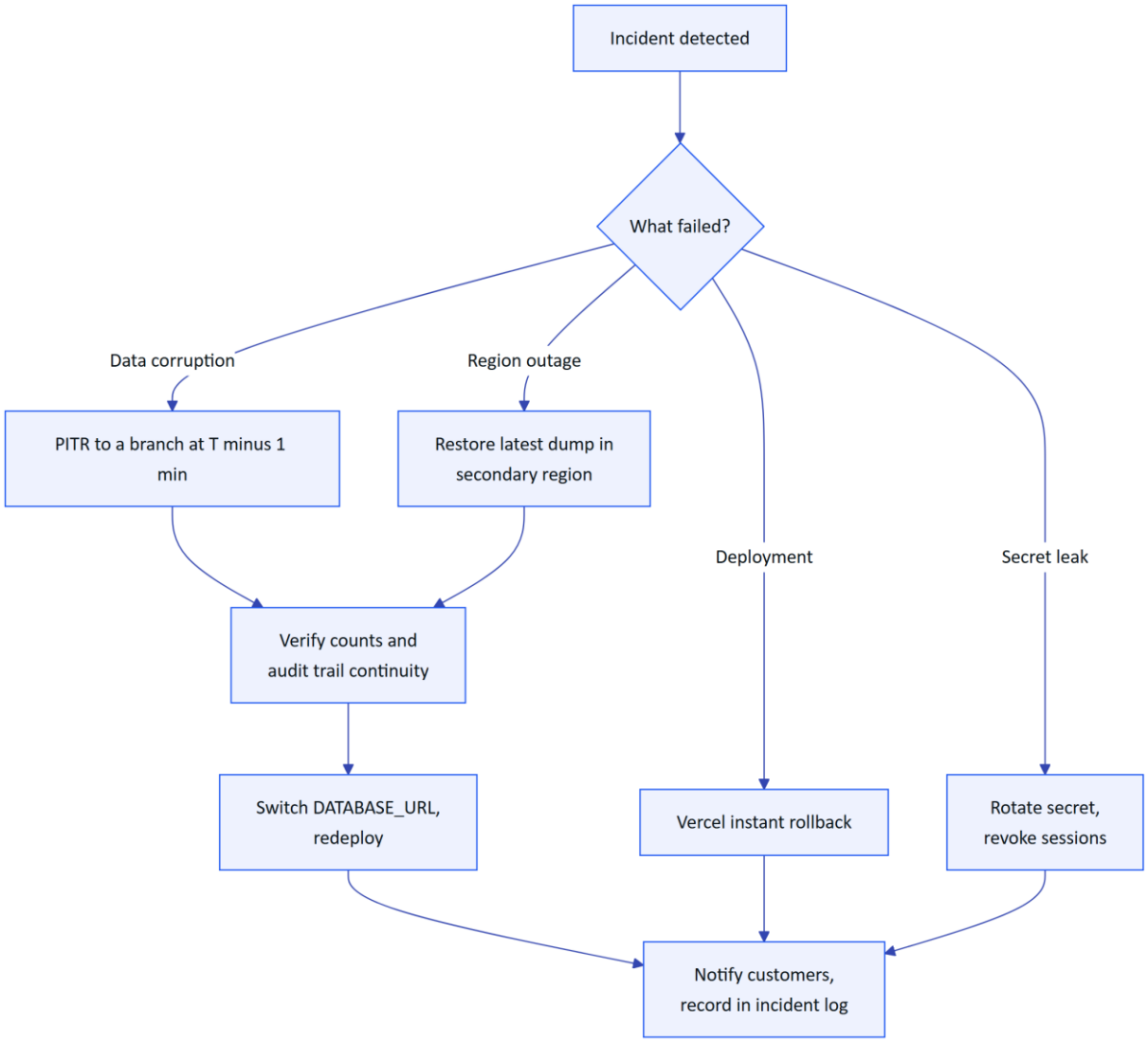

Figure 208. *Disaster recovery decision flow*

9.12.4 Restore runbook

1. Declare the incident and assign an incident lead (see HFG-SEC incident response).
2. Freeze writes if corruption is spreading: suspend affected organizations with `POST /api/platform/organizations/:id/status` (`modules/org.ts`), or put the project behind Vercel's maintenance page.

3. Identify the restore point from the audit trail (`audit_events.created_at` of the first bad action) or the migration time in the deployment log.
4. Create a Neon branch at that point. Never restore over the live branch.
5. Run verification queries on the branch: row counts per organization, newest telemetry timestamp per site, latest `audit_events` id, and a login with a test account.
6. Decide: switch the production `DATABASE_URL` to the restored branch, or copy specific rows back.
7. Redeploy so every function instance reconnects. Run the smoke test.
8. Reconcile: gateways replay buffered readings (ingest is idempotent per sensor and timestamp); users are told which window of manual entries must be re-entered.
9. Record timings against RPO and RTO; file the post-incident review within five business days.

> **IMPORTANT**
>
> A restore drill runs every quarter on staging, timed end to end, with results recorded. A backup that has never been restored is not a backup.

9.13 Scaling and capacity

9.13.1 What limits scale today

Table 267. Backup schedule

Component	Limit	Mitigation	Status
Serverless function duration	60 s (`maxDuration`)	SSE streams end at 50 s and resume; batch jobs	**Built**
Postgres connections	3 per instance; many instances can exhaust the pooler	Neon pooled endpoint; alert at 80%	**Built** (pool), **Planned** (alert)
Realtime	DB polling every 1.5 s per open stream	Topology B with in-memory or Redis bus for large tenants	**Partial**
Rate limiting	`@fastify/rate-limit` is in memory per instance	Shared store (Redis or Vercel Firewall rules)	**Partial**
Telemetry table growth	Unbounded; unique `(sensor, ts)` index	Monthly partitions or TimescaleDB beyond about 50 M rows	**Planned**
Audit page images	Base64 in `audit_document_pages.data` (up to 6 pages of 900 KB each per document)	Move to object storage, keep a key in Postgres	**Planned**
Request body	1 MB global, 8 MB for audit uploads	Direct-to-storage uploads for large media	**Built** limits

9.13.2 Capacity model

The model assumes a typical plant with 40 online instruments reporting every 60 seconds, 15 active users, 2 pairs of glasses and 20 audit scans a day. These are planning estimates, not measurements.

Table 268. Scaling constraints

Scale	Plants	Readings per day	Telemetry rows per year	Audit scan storage per year	Peak API req/s
Pilot	2	115 k	42 M	30 GB	5

Scale	Plants	Readings per day	Telemetry rows per year	Audit scan storage per year	Peak API req/s
Early commercial	10	576 k	210 M	150 GB	20
Growth	50	2.9 M	1.05 B	750 GB	90
Scale	200	11.5 M	4.2 B	3 TB	350

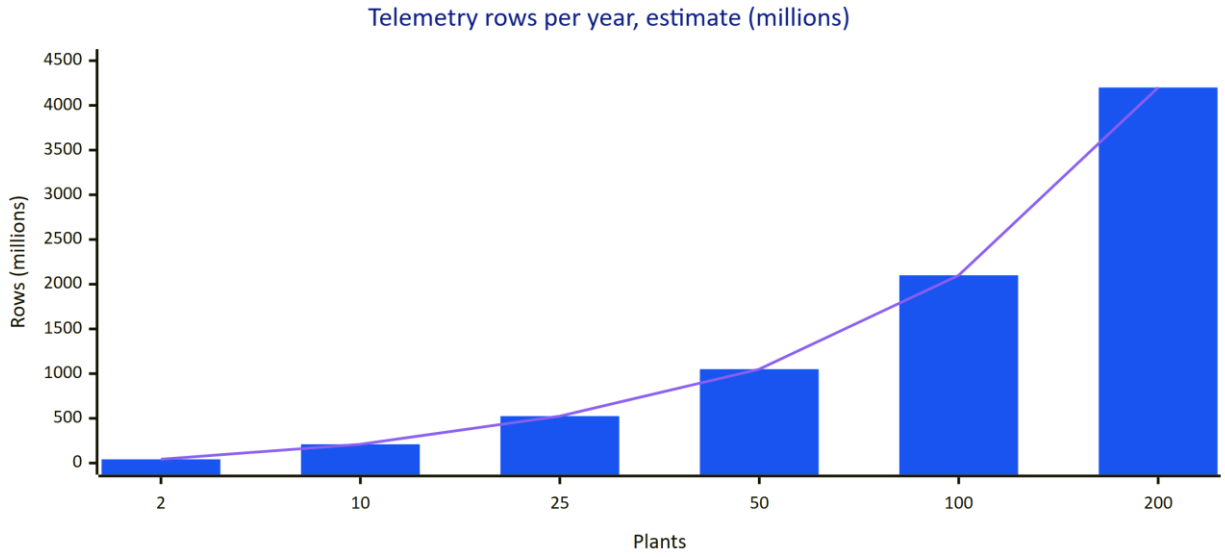

Figure 209. *Estimated telemetry rows per year by number of plants (millions, planning estimate)*

Practical thresholds from the model:

- Up to about 10 plants, Topology A with a single Neon database and no partitioning is comfortable.
- Between 10 and 50 plants, partition `telemetry_readings` by month, move audit images to object storage, add downsampled rollups for charts (the history endpoint already aggregates by bucket in `modules/telemetry.ts`), and move realtime for the largest tenants to Topology B.
- Beyond 50 plants, consider a time-series extension (TimescaleDB) or a dedicated telemetry store, and shard large tenants into their own database.

9.14 Cost model

All figures are indicative monthly costs in US dollars at 2026 list prices, before tax, for planning only. They must be checked against current vendor pricing and are not quotes.

Table 269. Capacity estimates by scale (planning assumptions)

Line item	Demo	Pilot (2 plants, 20 users)	Early commercial (10 plants, 150 users)	Growth (50 plants, 750 users)
Vercel	0 (Hobby, non-commercial)	40 (Pro, 2 seats)	150 (Pro, usage)	800 (Pro or Enterprise)
Postgres (Neon)	0	70	300	1,200
Object storage and egress	0	5	30	150
Anthropic (vision and assistant)	0 to 20	150	600	2,500

Line item	Demo	Pilot (2 plants, 20 users)	Early commercial (10 plants, 150 users)	Growth (50 plants, 750 users)
Observability (logs, errors, uptime)	0	50	200	700
Email delivery	0	0 to 20	20	50
CI minutes and artefacts	0	0 to 20	50	100
Total (approximate)	under 20	about 350	about 1,350	about 5,500

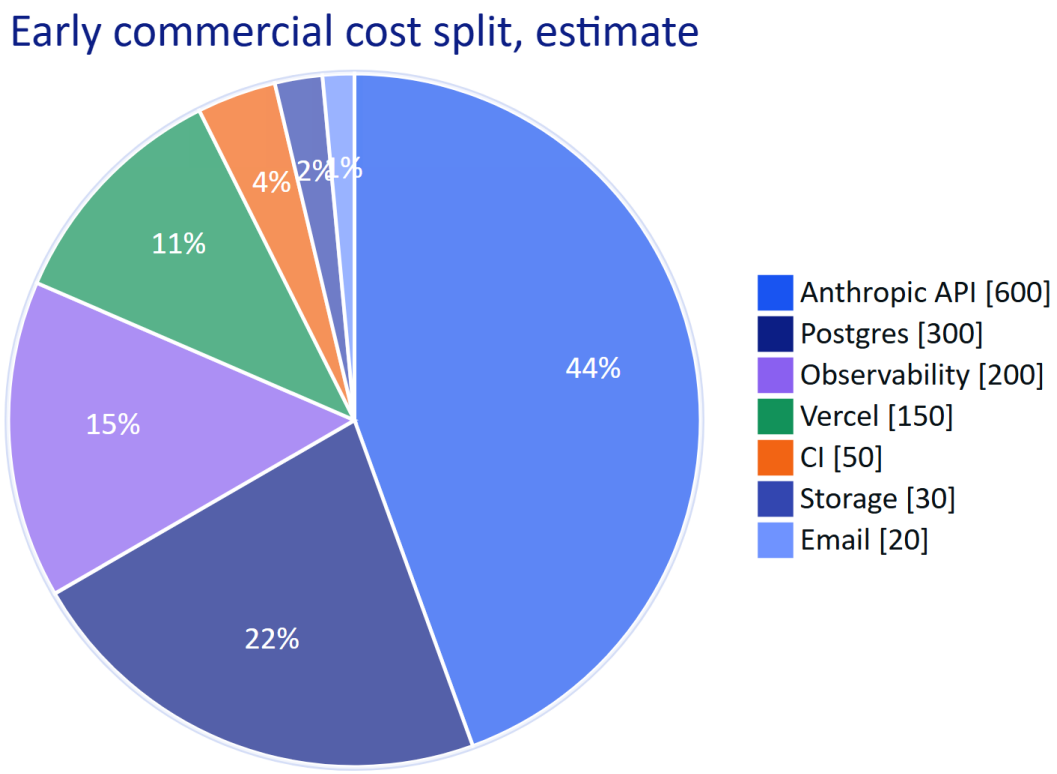

Figure 210. *Indicative cost split at early commercial scale (estimate)*

Observations:

- AI usage is the largest variable cost. It scales with audit scans and voice questions, not with users. Each organization needs a monthly AI budget with alerts, and the rules engine remains the free fallback.
- Database cost grows with telemetry retention. Downsampling raw data after 2 years (see HFG-SEC retention) is the main cost lever.
- Topology C shifts infrastructure cost to the customer but adds support cost on our side (offline upgrades, on-site visits). Price it as an annual licence plus a support fee.
- Topology A-IN may cost 10 to 30% more than the same footprint in a larger region because of regional pricing and the switch from Vercel Blob to S3.

9.15 India data residency and regulatory hosting

Hydris is an Indian company whose first customers are Indian plants. Residency is a selling point and in some cases a hard requirement.

Table 270. Indicative monthly infrastructure cost (USD, estimates)

Source	What it says (summary)	Effect on hosting
Digital Personal Data Protection Act 2023	No blanket localisation; the government may restrict transfer of personal data to notified countries	Personal data (user names, emails, IPs) may leave India unless a restriction applies; keep it in India by default for public-sector buyers
DPDP Rules 2025	Operational rules on notice, consent, security safeguards, breach reporting and retention, with phased commencement	Breach-reporting and log retention processes must be in place before the obligations apply
CERT-In Directions (April 2022)	Report specified incidents within 6 hours; keep ICT system logs for a rolling 180 days within Indian jurisdiction	Log drain to an India-hosted store; incident process with a 6-hour clock
CPCB and State Pollution Control Board consent conditions	Continuous monitoring data must be retrievable and unaltered; some boards require OCEMS feeds to their servers	Hydris is not the regulatory OCEMS feed; it must preserve records, provenance and audit trails
Customer contracts (PSUs, municipal corporations, smart city SPVs)	Frequently require data in India and named subprocessors	Topology A-IN or C; subprocessor list in the DPA
MeitY cloud empanelment	Government departments may require empanelled cloud service providers	Use an empanelled provider (AWS, Azure, Google Cloud India regions) for government tenders; Vercel and Neon empanelment must be checked

WARNING

This table is an engineering summary to guide design, not legal advice. Residency commitments in contracts must be reviewed by counsel, and dates in the DPDP Rules phase-in must be confirmed against the official gazette.

9.15.1 Residency design

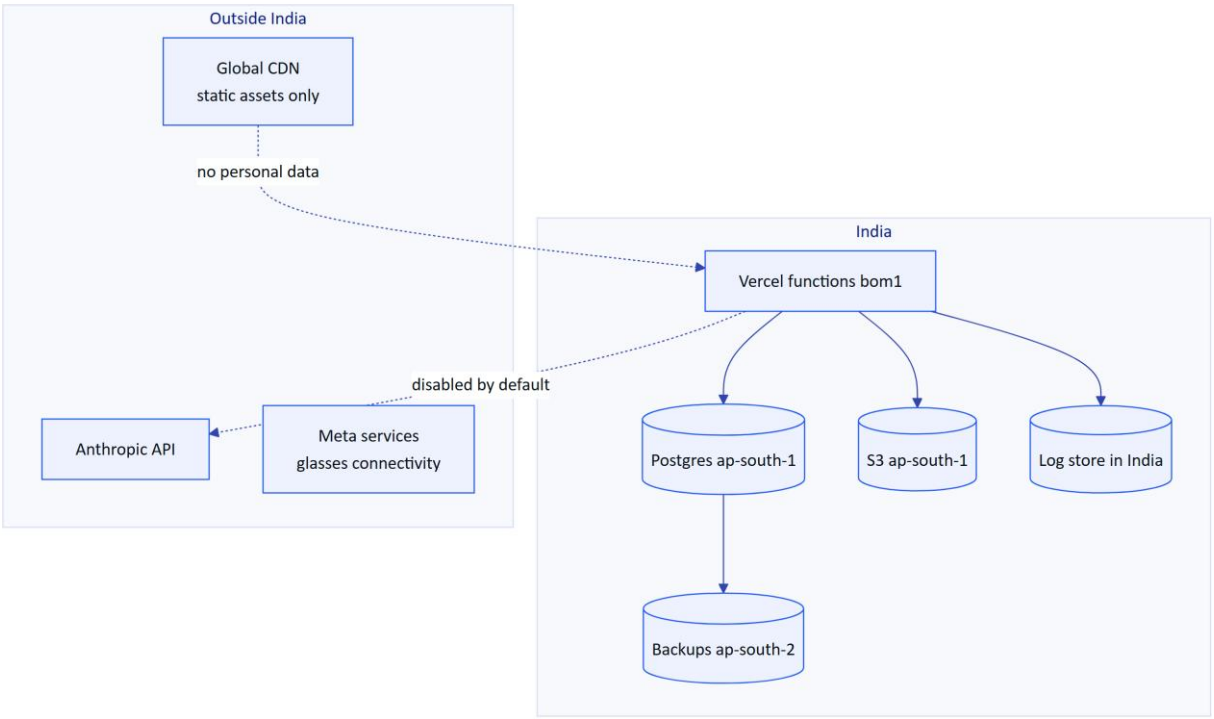

Figure 211. *Data flows for a strict-residency tenant on Topology A-IN*

Rules for a strict-residency tenant:

1. All stores (database, objects, logs, backups) are in India.
2. AI features are off unless the customer accepts processing outside India in writing. The organization record already has a `region` field (`modules/org.ts`); a per-organization AI switch keyed on it is **Planned**.
3. On-device OCR (ML Kit on the phone) keeps working without any cloud AI.
4. Meta glasses connectivity uses Meta services; disclose it as a subprocessor for glasses features.
5. Support staff access production data only from India or under a documented exception.

9.16 Customer onboarding and provisioning

9.16.1 Provisioning flow

A new customer organization is created by a Hydris platform administrator through `POST /api/platform/organizations` (`apps/api/src/modules/org.ts`). The call creates the organization with its name, slug, region and timezone, creates an `org_admin` invitation for the customer's first administrator, writes an `organization.created` audit event, and returns the invitation link. Because email is not wired yet, the platform admin currently sends that link to the customer by a separate secure channel.

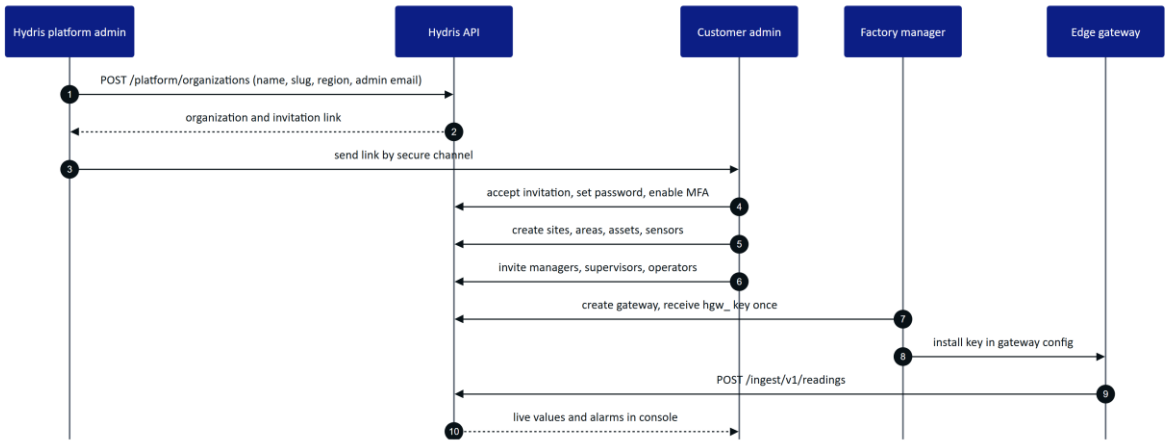

Figure 212. *Customer provisioning from contract to first live reading*

9.16.2 Onboarding steps and owners

Table 271. Indian regulatory considerations for hosting (summary, not legal advice)

Step	Owner	Done when
Contract, DPA, topology and region chosen	Hydris sales and customer	Signed order form names topology and AI setting
Environment ready	Hydris platform engineering	Production project, database, secrets and cron verified
Organization created	Hydris platform admin	`organization.created` audit event
First admin activated with MFA	Customer admin	`member.invitation_accepted`, `auth.mfa_enabled` events
Plant model built (sites, areas, assets, sensors, alarm rules)	Customer with Hydris solutions engineer	Plant map and sensor list complete
Users invited with site scopes	Customer admin or factory manager	All pilot users active
Gateway installed and keyed	Customer OT team with Hydris	First readings within freshness window
Phones enrolled in MDM, apps installed	Customer IT	Apps open and sign in
Glasses paired and assigned	Customer supervisor	Devices show as active in the console
Procedures and inspection templates loaded	Customer	First guided inspection completed
Training and go-live sign-off	Customer and Hydris	Signed acceptance

9.17 Go-live checklist

The checklist below gates the first customer pilot on Topology A. Items already true in the code are ticked.

9.17.1 Platform

- ☑ API runs in-process on Vercel with migrations under an advisory lock
- ☑ Refresh cookies are `HttpOnly`, `SameSite=Strict`, `Secure` in production
- ☑ Attachments use a private store served only through authorized routes
- ☐ Separate production Vercel project; demo project keeps `hydrisview.vercel.app`

- ☐ Production Neon database in the agreed region, PITR enabled, pooled URL
- ☐ Runtime database user is not the owner; RLS verified with a cross-tenant test
- ☐ `JWT_SECRET`, `DATA_KEY`, `CRON_SECRET` generated, stored, `DATA_KEY` escrowed offline
- ☐ `DEMO_MODE=false` set explicitly in production; `SIMULATOR_ENABLED=false`
- ☐ `REALTIME_MODE=db`, `PUBLIC_APP_URL` set to the customer-facing domain
- ☐ Cron schedule changed to every minute (Vercel Pro) or external scheduler in place
- ☐ Vercel function region matches the database region
- ☐ Custom domain with TLS; HSTS confirmed

9.17.2 Pipeline

- ☐ GitHub Actions CI with required checks on `main`
- ☐ Migrations run as a pipeline step with approval
- ☐ Release tags and changelog
- ☐ Android upload key in CI; release builds no longer signed with the debug key
- ☐ Meta DAT application ID and client token injected in CI

9.17.3 Operations

- ☐ Log drain, error tracking and uptime checks live
- ☐ Alerts for 5xx, ingest lag, cron age and token reuse routed to on-call
- ☐ Deep health check covering the database
- ☐ Backup dump job and first restore drill completed and timed
- ☐ Incident response contacts and CERT-In reporting path documented
- ☐ Email provider wired for invitations and password reset

9.17.4 Customer

- ☐ DPA signed with subprocessor list (Vercel, Neon, Anthropic if enabled, Meta for glasses)
- ☐ AI processing choice recorded per organization
- ☐ Gateway tested end to end with buffered replay
- ☐ Training delivered; site privacy signage for glasses in place

9.18 Known gaps and decisions

Table 272. Onboarding steps

ID	Gap	Impact	Plan	Priority
DEP-01	No CI pipeline	Regressions and unsafe migrations can reach production	Add the proposed workflow	High
DEP-02	Cron runs daily on Vercel	Stale-data alarms up to 24 h late	Per-minute cron on Pro or external scheduler	High
DEP-03	Release APK signed with debug key	Cannot distribute through Play	Upload key in CI	High

ID	Gap	Impact	Plan	Priority
DEP-04	Email not wired	Invitations and resets need manual link delivery	Transactional email provider, purge `mail_outbox` after send	High
DEP-05	No metrics, tracing or alerting	Incidents found by users	Observability stack	High
DEP-06	Audit page images in Postgres as base64	Database bloat, backup size, cost	Move to object storage	Medium
DEP-07	No retention or purge jobs	Unbounded growth of sessions, mail and telemetry	Retention jobs per HFG-SEC schedule	Medium
DEP-08	`hydris_url` compiled into Android builds	One build per customer	MDM managed configuration	Medium
DEP-09	Node version mismatch (22 in `engines`, 24 in Dockerfiles and Vercel guide)	Inconsistent runtime behaviour	Standardize	Low
DEP-10	Per-instance rate limiting	Limits weaker under many instances	Shared limiter or edge firewall rules	Medium
DEP-11	Demo and production share one project today	Customer data could be added to a demo by mistake	Split projects	High
DEP-12	Debug APK committed to the repository	Untraceable binaries	Move to CI artefacts	Low

DECISION

Topology A on Vercel and Neon remains the default for pilots. Topology B is adopted per customer only when the capacity model or a realtime requirement demands it. Topology C is offered to air-gapped plants with the glasses constraint stated up front. All production deployments wait for DEP-01, DEP-02, DEP-03 and DEP-11.

10 Security and Privacy

> **NOTE**
>
> **HFG-SEC v2.0** · Draft for review · Owner: Security, Hydris. How Hydris Field Glasses protects plant records, personal data and the people in front of the camera, what is built today, and what must be closed before customers rely on it

Hydris Field Glasses puts live plant data, procedures, audit-record scanning and a voice assistant into the hands (and the field of view) of wastewater and effluent treatment operators. That makes it a system of record for regulated environmental data, an identity system for plant staff, and a camera worn in an industrial workplace. This document describes how each of those responsibilities is met.

Statements about the current product name the file that implements them. Status follows the house rule: **Built** (in the code and exercised), **Partial** (exists with a known gap) and **Planned** (designed, not implemented). Compliance entries are mappings and targets, never claims of certification.

3	15 min	0	2
layers of access control RBAC, site scope and Postgres row-level security	**access token lifetime** refresh tokens rotate with reuse detection	**write paths to plant control** Hydris is read-only to PLC, SCADA and DCS	**people per approved audit record** verifier and a different approver

> **IMPORTANT**
>
> The public demo at `https://hydrisview.vercel.app` runs in demo mode with public signing secrets and synthetic data (`apps/api/src/config.ts`). Its security properties are deliberately weaker and are listed in this document. Real plant data must only ever be placed in a production deployment (see HFG-DEP).

10.1 Security principles

1. **Read-only to the plant.** No endpoint writes to PLC, SCADA or DCS. Telemetry arrives by outbound push from a gateway (`POST /api/ingest/v1/readings`). The voice assistant prompt states that Hydris never operates equipment (`apps/api/src/modules/voice-assistant.ts`).
2. **Defence in depth for tenancy.** Permission checks in the API, site scoping in queries, and row-level security in Postgres, so one missing check does not leak another customer's data.
3. **Records are evidence.** Audit trails are append-only at the database grant level; corrections preserve the original value; audit documents need two different people.
4. **Least privilege for devices.** Glasses act for one assigned operator with a fixed permission ceiling and can be suspended or revoked instantly.
5. **Secrets are never stored in the clear.** Passwords use argon2id; refresh, device, gateway, invitation and reset tokens are stored as SHA-256 hashes; MFA seeds are AES-256-GCM encrypted.
6. **AI assists, people decide.** AI output is grounded in tenant data, labelled, cross-checked and verified by humans before it becomes a record.
7. **Capture on intent only.** The camera captures when a person asks it to. There is no continuous recording.

8. **Privacy by default.** Collect the minimum personal data, keep it in the customer's jurisdiction, and delete it when its purpose ends.

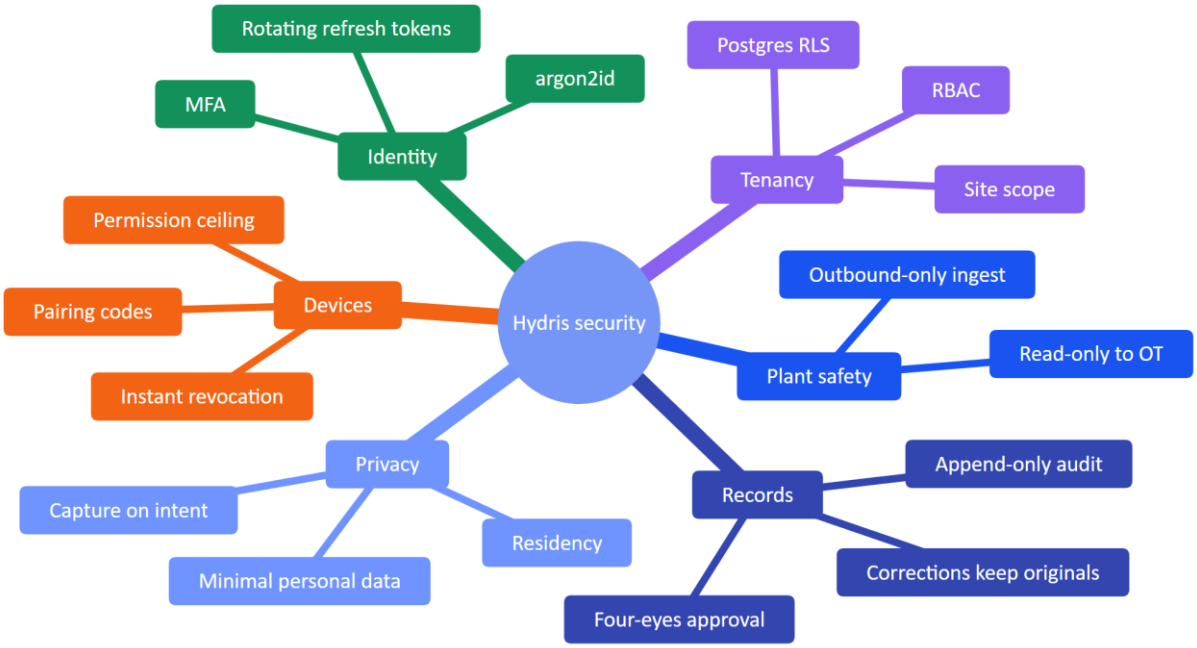

Figure 213. *Security and privacy principles grouped by concern*

10.2 System overview and trust boundaries

The data flow diagram shows every component that handles customer data and the trust boundaries between them. A trust boundary is anywhere data crosses from a party or runtime we do not control to one we do, or the reverse.

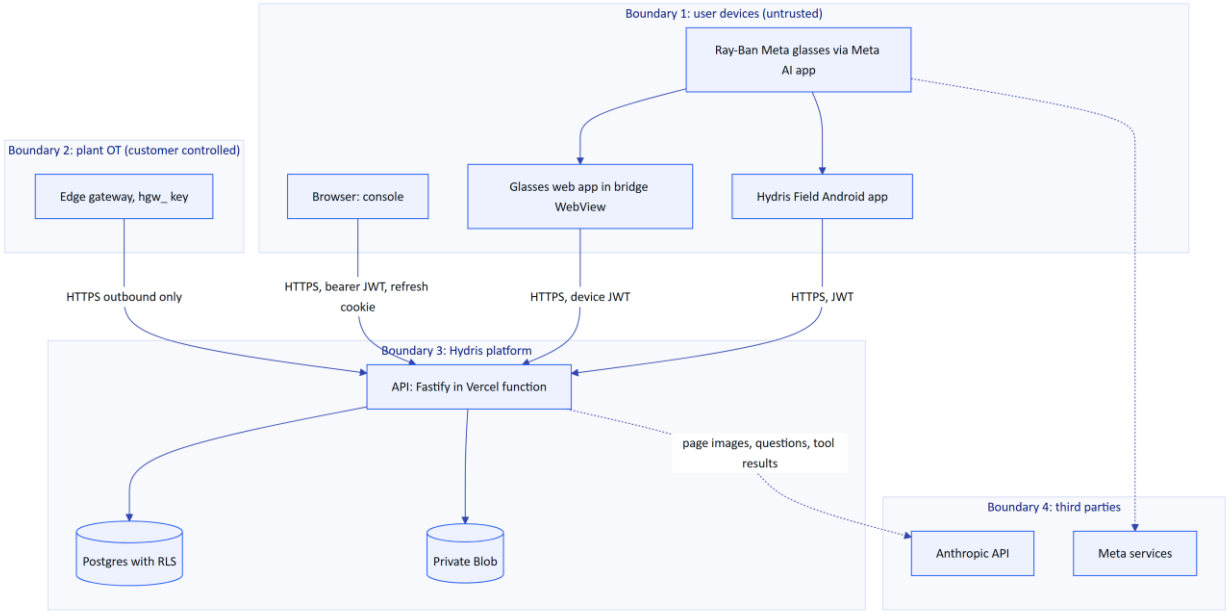

Figure 214. *Data flow diagram with trust boundaries*

Table 273. Deployment gaps register

Boundary	Crossing	Main controls
Devices to platform	All user and glasses traffic	TLS, JWT verification, revocation check per request, rate limits, zod validation
Plant to platform	Telemetry batches	Hashed gateway key, outbound-only, 500 readings per call, site-restricted sensors
Platform to database	Every query	Transaction runs as `hydris_app` with `app.org_id`; RLS forced
Platform to Anthropic	Audit page images, voice questions and tool results	Only when a key is configured; per-org opt-out **Planned**
Glasses to Meta	Camera, microphone, button events through DAT	Vendor platform; disclosed as a subprocessor

10.3 Data classification

Table 274. Trust boundary crossings

Class	Examples	Where stored	Handling rules
Restricted secrets	Password hashes (argon2id), MFA seeds (AES-256-GCM with `DATA_KEY`), refresh, device, gateway, invitation and reset token hashes, `JWT_SECRET`, `DATA_KEY`, `CRON_SECRET`	Postgres, platform secrets	Never logged, never returned; raw tokens shown once
Regulated operational records	Telemetry, corrections, alarm history, inspections, maintenance, audit documents and their events	Postgres, Blob	Tenant RLS, append-only trails, retention per permit
Personal data	Names, emails, IPs in `auth_sessions` and `audit_events`, device `last_ip`, website inquiries (IP stored as a salted hash)	Postgres	Minimise, DPDP and GDPR rights, short retention outside records
Sensitive imagery	Page photos that may show signatures, names, faces or the plant	`audit_document_pages`, `attachments`	Capture on intent, authorized routes only, no public URLs
Voice content	Question text after "Hey Hydris"	Request body only, not persisted by `/assistant/ask`	Audio never uploaded to Hydris
Device-cached data	Glasses context cache and offline queue in `localStorage` (`apps/web/src/glasses/data.ts`, `link.ts`)	Phone browser storage	Cleared on logout; encryption **Planned**
Public	Website content, product documentation	Web	None

10.4 Threat model

The threat model uses STRIDE per component. "Controls" lists what is in the code; "Gaps" lists what is missing.

10.4.1 Web console and authentication

Table 275. Data classification

Threat	Scenario	Controls	Gaps
Spoofing	Password guessing, credential stuffing	argon2id (`core/crypto.ts`), lockout after 5 failures for 15 min, login limit 10 per min, timing-equal dummy verify, optional TOTP	No breached-password check, no SSO, MFA not enforceable per org
Tampering	Changing role or org in a token	JWT signed with `JWT_SECRET`; membership and role reloaded from the database on every request (`context.ts`)	None known
Repudiation	Denying a change	`audit_events` with actor, IP and action; append-only grant	No hash chain or WORM export
Information disclosure	Cross-tenant reads, user enumeration	RLS, 404 for out-of-scope records, identical reset responses	CSP disabled in helmet (`contentSecurityPolicy: false`)
Denial of service	Login floods	Rate limits, 1 MB body limit	Limits are per function instance
Elevation of privilege	Manager invites an admin	`assignableRoles` (`packages/domain/src/roles.ts`)	None known

10.4.2 Glasses runtime and bridge app

Table 276. STRIDE for the web console and authentication

Threat	Scenario	Controls	Gaps
Spoofing	Rogue device pairs into an org	Pairing requires a signed-in user with `device.pair` to approve a short code; poll secret hashed	Code TTL 300 s, no device attestation
Tampering	Injected page calls native bridge	Bridge loads only `BuildConfig.HYDRIS_URL + "/glasses"`	Default `WebViewClient` allows navigation elsewhere with `HydrisNative` still exposed; `onPermissionRequest` grants everything
Repudiation	"The glasses did it"	Audit events record `actorDeviceId` and assigned user	None known
Information disclosure	Stolen phone exposes refresh token	30-day device sessions, revocation	Web runtime keeps the device refresh token in `localStorage`
Denial of service	Pairing spam	10 challenges per min, 40 polls per min	Per-instance limits
Elevation of privilege	Device calls admin endpoints	`DEVICE_PERMISSIONS` ceiling (`core/principal.ts`); verify and approve require a user principal	None known

10.4.3 Native Android app

Table 277. STRIDE for the glasses runtime

Threat	Scenario	Controls	Gaps
Information disclosure	Token theft from storage or backup	AES-256-GCM with a non-exportable Android Keystore key (`SecureStore.kt`); `allowBackup="false"`	No user-authentication binding on the key
Tampering	Man-in-the-middle on hostile Wi-Fi	System TLS validation	Certificate pinning **Planned**
Spoofing	Repackaged app	Play signing (**Planned**)	Release currently signed with the debug key; no Play Integrity

10.4.4 API, database and ingest

Table 278. STRIDE for the native app

Threat	Scenario	Controls	Gaps
Spoofing	Forged gateway	`hgw_` keys hashed, gateway and org must be active	No key expiry or IP allow-list
Tampering	Silent edit of readings	Corrections in append-only `telemetry_corrections`; original kept	No signed export
Information disclosure	SQL leak across tenants	`withTenant` sets role and org per transaction, FORCE RLS	Owner connection would bypass RLS; separate runtime user **Planned**
Denial of service	Ingest flood	1,200 calls per min per instance, 500 readings per call	Shared limiter **Planned**
Elevation	Code path forgets `withTenant`	`withSystem` still runs as `hydris_app` with append-only grants	Review checklist only

10.4.5 AI provider and cron

Table 279. STRIDE for the API, database and ingest

Threat	Scenario	Controls	Gaps
Information disclosure	Plant documents sent to a third party	Only with a configured key; images sent per request	Per-org opt-out **Planned**; no zero-retention agreement recorded
Tampering	Prompt injection in a scanned page	Structured output schema, cross-check against on-device OCR, human review	No explicit injection classifier
Spoofing	Unauthenticated cron calls	`CRON_SECRET` bearer compared with `timingSafeEqual`	None known

10.5 Authentication and sessions

10.5.1 Tokens in use

Table 280. STRIDE for the AI provider and cron

Token	Format	Lifetime	Storage (client)	Storage (server)	Revocation
User access token	JWT `typ: user` with `sid` family	15 min (12 h in demo)	Memory	None	Family revocation checked per request
User refresh token	48 random bytes	7 days	`hg_refresh` cookie, `HttpOnly`, `SameSite=Strict`, path `/api/auth`, `Secure` in production	SHA-256 in `auth_sessions`	Logout, password change, reuse detection, session list
MFA challenge	JWT `typ: mfa`	5 min	Memory	None	Expiry
Device access token	JWT `typ: device` with `sv` session version	15 min	Memory	None	`session_version` bump
Device refresh token	`hgd_` plus 40 random bytes	30 days	Keystore (native) or `localStorage` (web)	SHA-256 in `device_sessions`	Revoke, suspend, logout, reuse detection
Gateway key	`hgw_` random	Until revoked	Gateway config	SHA-256 in `gateways`	`POST /gateways/:id/revoke`
Invitation and reset tokens	Random	7 days and 30 min	Link	SHA-256	Single use

10.5.2 Sign-in with MFA

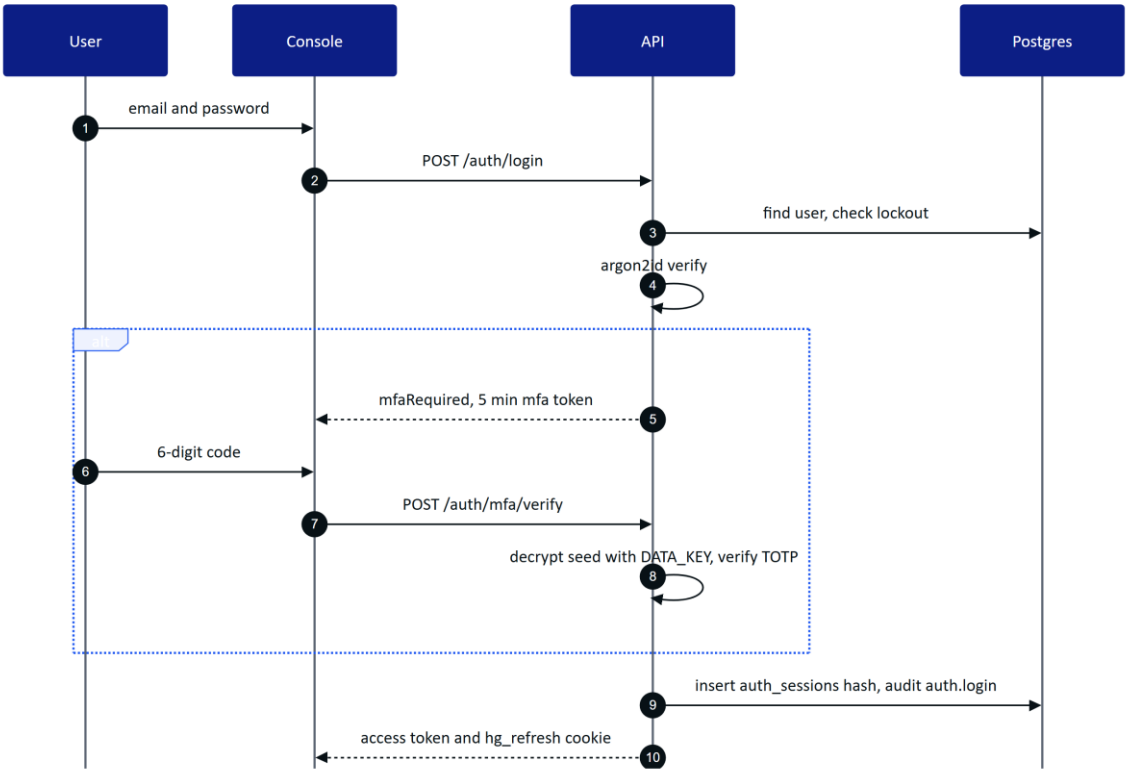

Figure 215. *Password sign-in with optional TOTP second factor*

Failed logins increment `failed_logins`; at five the account locks for 15 minutes and an `auth.login_failed` event is written each time. Unknown emails run a dummy argon2 verification so response time does not reveal which accounts exist. Passwords must be 12 to 200 characters and not a repeated character (`passwordProblems`).

10.5.3 Refresh rotation and reuse detection

Every refresh rotates the token within the same family. A rotated token presented again more than 30 seconds later (the grace window for several tabs) is treated as theft: the whole family is revoked and `auth.refresh_reuse_detected` is audited. Because each access token carries the family id and the API checks the family on every request, logout and reuse detection end outstanding access tokens immediately.

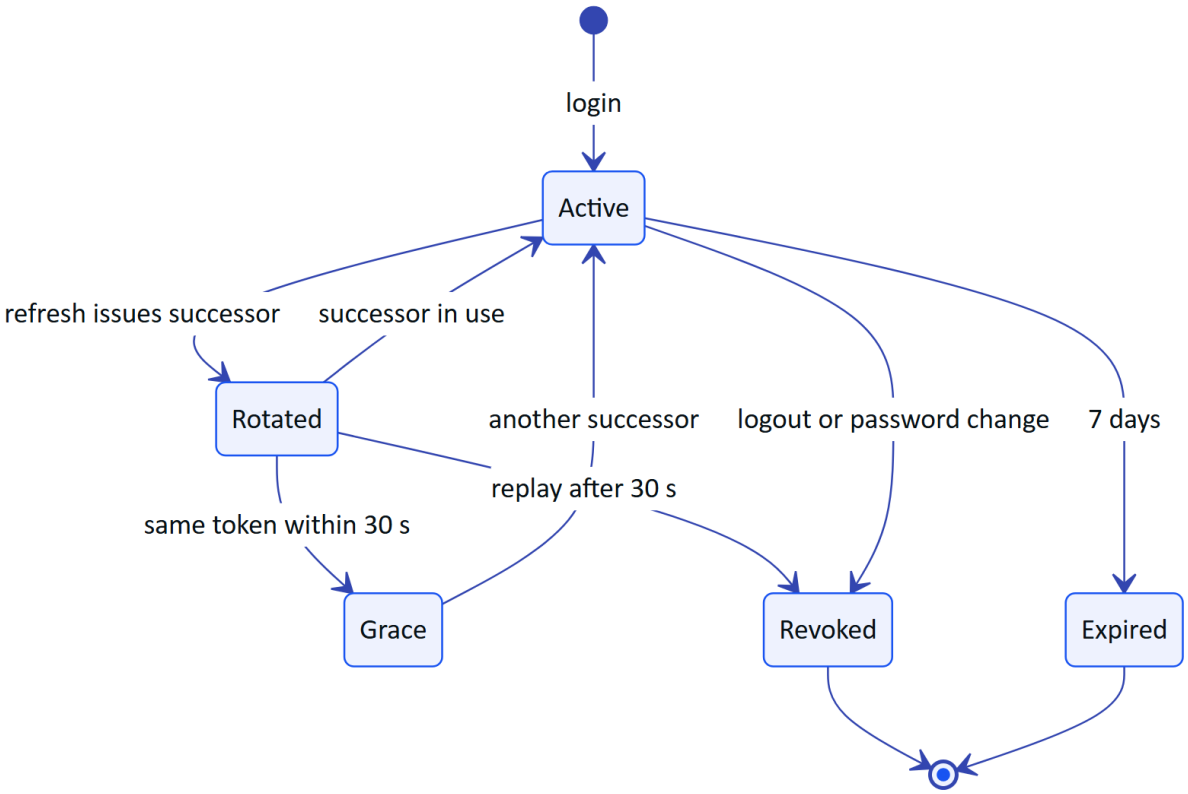

Figure 216. *Refresh token family lifecycle*

Table 281. Token inventory (apps/api/src/config.ts TTL, modules/auth.ts, modules/devices.ts)

Event	Effect	Status
Logout	Revokes the family	**Built**
Password change or reset	Revokes all sessions of the user	**Built**
Org switch	Revokes the current family, starts a new one	**Built**
Session list and remote revoke	`GET` and `DELETE /auth/sessions`	**Built**
User deactivated	Next request fails membership check	**Built**
SSO (SAML or OIDC)	Enterprise identity	**Planned**
Org-enforced MFA	Admin policy	**Planned**

10.6 Device and glasses security

10.6.1 Pairing

Glasses use a device-authorization flow modelled on OAuth device grant. The device requests a challenge and displays an 8-character code from an unambiguous alphabet (`packages/domain/src/devices.ts`); a signed-in user approves it on `/pair`. The QR code encodes only the `/pair` URL with the short code, never a credential.

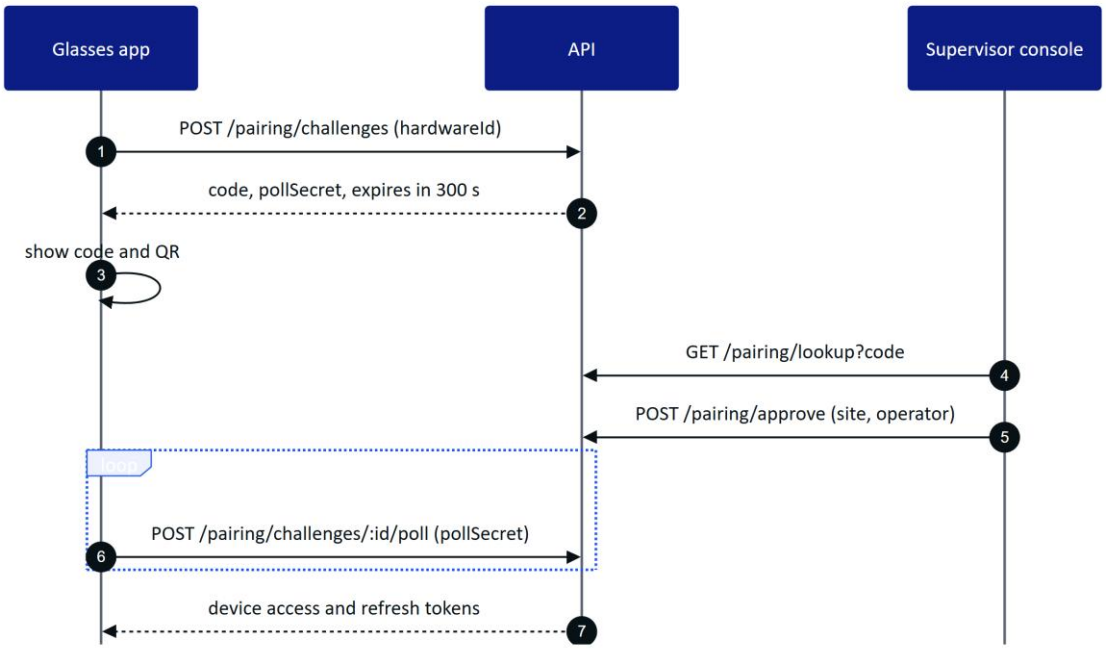

Figure 217. *Glasses pairing with code approval*

Table 282. Session control events

Control	Value	Source
Code length and alphabet	8 characters, 30 symbols, CSPRNG	`core/crypto.ts`, `packages/domain/src/devices.ts`
Code TTL	300 s	`PAIRING_TTL_S`
Code stored	SHA-256 only	`pairing_challenges.code_hash`
Poll secret	32 random bytes, hashed, constant-time compare	`modules/devices.ts`
Rate limits	10 challenges, 40 polls, 10 lookups and approvals per min	`modules/devices.ts`
Approver	User with `device.pair` and site scope	`modules/devices.ts`
Previous challenges	Cancelled when the same hardware id asks again	`modules/devices.ts`

10.6.2 Device lifecycle and lost devices

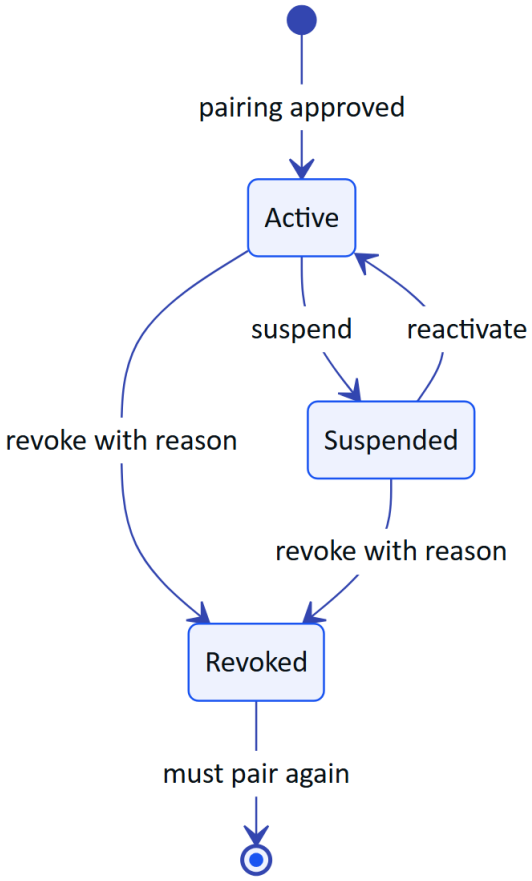

Figure 218. *Device states and token effects*

Every transition bumps `devices.session_version`, so outstanding device access tokens stop working at once. Revocation also ends all device sessions and releases the operator assignment, writes `device.revoked` with the reason, and pushes a `device.revoked` realtime event so an online device signs itself out.

Lost device procedure:

1. The operator reports the loss to a supervisor.
2. The supervisor revokes the device in the console with a reason (`POST /devices/:id/revoke`).
3. If the phone also held a user session, the user signs in elsewhere and revokes it in the session list, or an admin deactivates and re-invites the user.
4. If the phone is enrolled in MDM, IT issues a remote wipe.
5. Security reviews `audit_events` for the device since the last known good time.

Table 283. Pairing controls

Storage of the device refresh token	Runtime	Status
Android Keystore AES-GCM (`SecureStore.kt`)	Hydris Field native app	**Built**
`localStorage` (`apps/web/src/glasses/link.ts`)	Glasses web app, including inside the bridge WebView	**Partial** (gap SEC-03)

10.7 Authorization

10.7.1 Roles and permissions

`packages/domain/src/roles.ts` is the single source of truth for six roles and 38 permissions. The API enforces it; clients only use it to decide what to show.

Table 284. Device credential storage

Role	Highlights
`org_admin`	All permissions
`factory_manager`	Plant model, gateways, alarm rules, members (not admins), devices, reports, audit log, audit-doc verify and approve
`supervisor`	Alarm workflow, inspection management and review, maintenance, devices
`field_operator`	Read, acknowledge alarms, execute inspections, request and execute maintenance, capture audit docs
`maintenance_engineer`	Maintenance execution and read access
`viewer`	Read-only

10.7.2 Layered enforcement

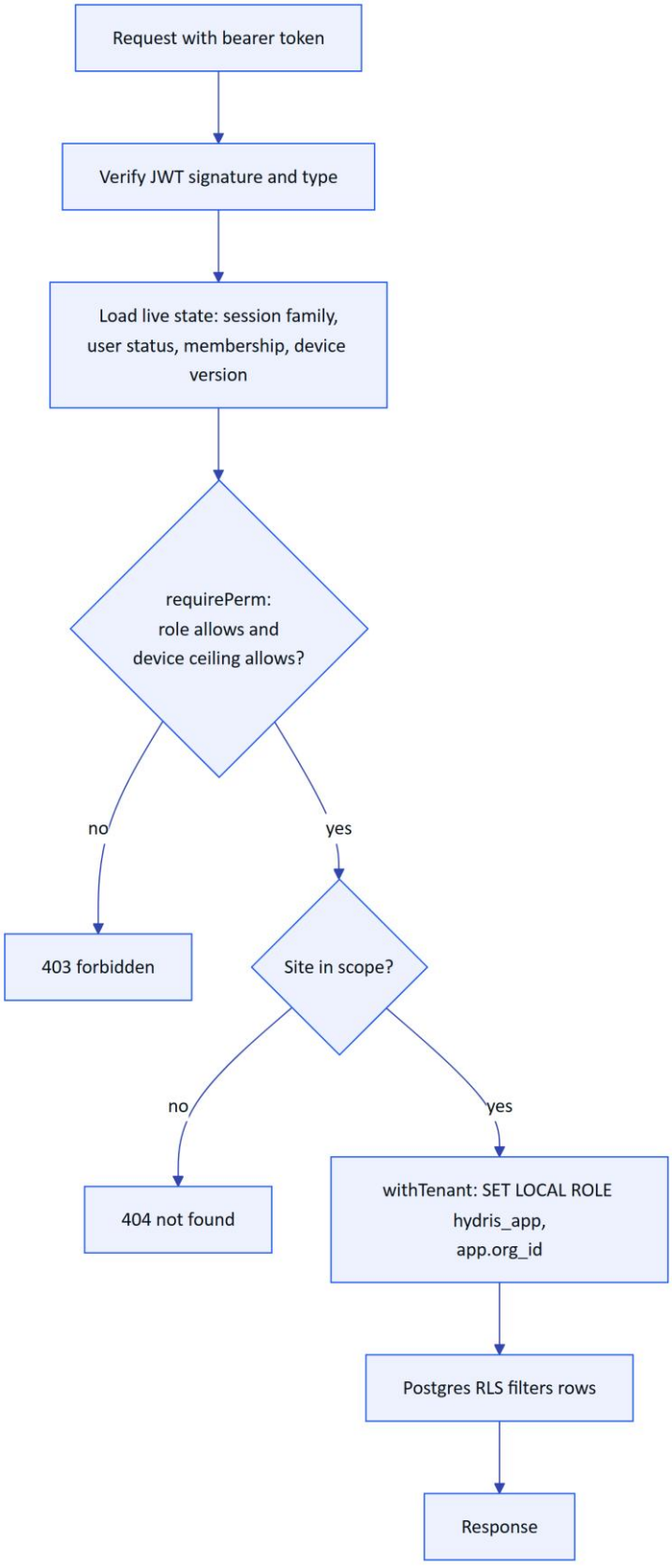

Figure 219. *Every request passes three independent authorization layers*

Out-of-scope records return 404, not 403, so their existence is not disclosed (`requireSite` in `core/principal.ts`).

10.7.3 Row-level security

Migration `0001_rls.sql` enables and forces RLS on every tenant table with one policy:

```
CREATE POLICY tenant_isolation ON <table>
  USING (hydris_tenant_ok(org_id)) WITH CHECK (hydris_tenant_ok(org_id));
-- hydris_tenant_ok: app.bypass = 'on' OR row_org = app.org_id
REVOKE UPDATE, DELETE ON audit_events FROM hydris_app;
```

Later migrations add the same policy to corrections, UI profiles (`0003`), realtime events (`0006`) and audit documents (`0008`), revoke `UPDATE` and `DELETE` on `telemetry_corrections`, and grant only `SELECT, INSERT` on `audit_document_events`. System work (login, pairing, jobs) uses `withSystem`, which sets `app.bypass` but still runs as `hydris_app`, so append-only grants hold everywhere.

> **WARNING**
> Owner and superuser connections bypass RLS. Production must connect as a runtime user that is a member of `hydris_app` and not the table owner. Today the same owner user runs migrations and the application.

10.8 API security

Table 285. Role summary

Control	Setting	Source	Status
Global rate limit	600 per min per IP	`app.ts`	**Built**
Login, MFA	10 per min	`modules/auth.ts`	**Built**
Password reset request	5 per min	`modules/auth.ts`	**Built**
Refresh	60 per min	`modules/auth.ts`	**Built**
Website inquiries	5 per 10 min, honeypot, minimum fill time	`modules/public.ts`	**Built**
Voice assistant	60 per min	`modules/voice-assistant.ts`	**Built**
Ingest	1,200 per min, 500 readings per call	`modules/telemetry.ts`	**Built**
Body size	1 MB default, 8 MB audit upload, 900 KB per page, 6 pages, 10 MB multipart	`app.ts`, `modules/audit-docs.ts`	**Built**
Input validation	zod on every body and parameter, 400 with field details	all modules	**Built**
Error hygiene	5xx returns a generic message; stack only in logs	`app.ts`	**Built**
Shared rate-limit store	Limits are per instance		**Planned**

Table 286. API protections

Header	Value	Source
`X-Content-Type-Options`	`nosniff`	`next.config.ts`, helmet
`Referrer-Policy`	`strict-origin-when-cross-origin`	`next.config.ts`

Header	Value	Source
`Permissions-Policy`	`camera=(self), microphone=(), geolocation=()`	`next.config.ts`
`X-Frame-Options`	`DENY`, `SAMEORIGIN` for glasses and simulator	`next.config.ts`
Helmet defaults (HSTS and others)	On, CSP off	`app.ts`
`X-Powered-By`	Removed	`poweredByHeader: false`
Content Security Policy	None	**Planned**

> **NOTE**
>
> `Permissions-Policy` sets `microphone=()`, which disables the microphone for the site's own pages. That may block the browser speech engine on `/glasses`; the Meta DAT speech path does not use the browser microphone. Verify and, if needed, set `microphone=(self)` for the glasses routes only.

Fastify runs with `trustProxy: true`. On Vercel the client address comes from the platform's `x-forwarded-for`; on a container deployment without a trusted proxy, a client could spoof the header and evade per-IP limits. Set `trustProxy` to the load balancer address in Topology B and C.

10.9 Secrets management

Table 287. Security headers

Secret	Purpose	Rotation impact	Store
`JWT_SECRET`	Signs all JWTs	All users and devices must refresh; MFA steps in flight fail; inquiry IP hash salt changes	Vercel env, production scope
`DATA_KEY`	Encrypts MFA seeds	Requires re-encryption of every `mfa_secret_enc`	Vercel env plus offline escrow
`CRON_SECRET`	Authorizes `/api/cron`	Update Vercel cron header	Vercel env
`ANTHROPIC_API_KEY`	AI features	None	Vercel env
`BLOB_READ_WRITE_TOKEN`	Attachment storage	Reconnect store	Injected by Vercel
`DATABASE_URL`	Database credentials	Rotate in Neon, redeploy	Injected by Vercel
Gateway keys	Ingest	Issue new key, revoke old	Hash only in DB
Android upload key, DAT token	Mobile release	Play key reset process	CI environment secret (**Planned**)

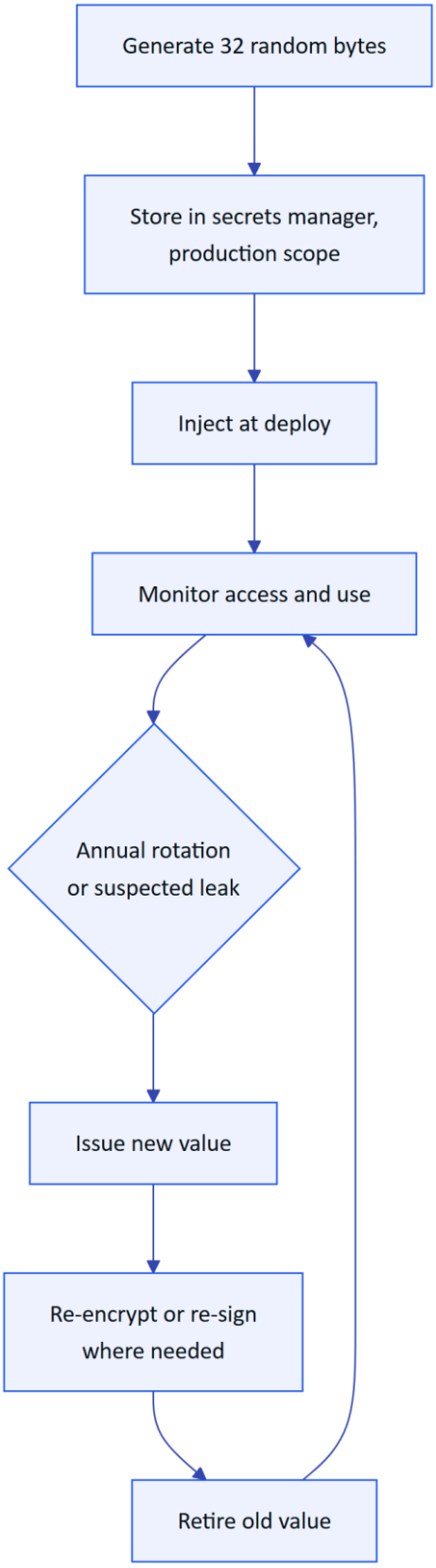

Figure 220. *Target secret lifecycle*

Rules: no secret in the repository or `NEXT_PUBLIC_*`; preview deployments use their own secrets; demo secrets in `DEMO_SECRETS` are public and only used in demo mode; annual rotation of `JWT_SECRET` and `CRON_SECRET` with a planned re-encryption tool for `DATA_KEY` (**Planned**).

10.10 Mobile security

Table 288. Secret inventory

Control	Hydris Field (`apps/android`)	Glasses bridge (`apps/glasses-bridge-android`)
Credential storage	Keystore AES-256-GCM `SecureStore`, cookies persisted encrypted	WebView `localStorage` (gap)
Backup	`allowBackup="false"`	`allowBackup="false"`
Minimum OS	Android 12 (SDK 31)	Android 12 (SDK 31)
Code shrinking	R8 on release	Not configured
Release signing	Debug key today (gap)	Debug build distributed
Cookie scope	Sent only to the configured host	n/a
Certificate pinning	**Planned**	**Planned**
Play Integrity attestation	**Planned**	**Planned**
WebView hardening	n/a	Origin allow-list and per-origin permission grants **Planned**
Cleartext traffic	Default (blocked on SDK 28+)	Default

Planned hardening for the bridge: override `shouldOverrideUrlLoading` to keep navigation on the Hydris origin, grant WebView camera permission only for that origin, and move the device refresh token behind a native bridge call backed by the Keystore so the web page never holds it.

10.11 AI safety

10.11.1 What is sent to Anthropic

AI features are enabled only when `ANTHROPIC_API_KEY` or `ANTHROPIC_AUTH_TOKEN` is set (`visionAvailable()` in `core/vision-extract.ts`). Without a key, audit scans keep on-device OCR results and the assistant uses the deterministic rules engine.

Table 289. Mobile security controls

Feature	Data sent	Model	Trigger
Audit scan enhancement	Page images (up to 6), optional operator hint	`claude-opus-5-5`	User presses enhance, before review starts
Voice assistant	Question text (max 500 chars), up to 8 history turns, tool results with tenant data	`claude-opus-5-5`	`/assistant/ask`

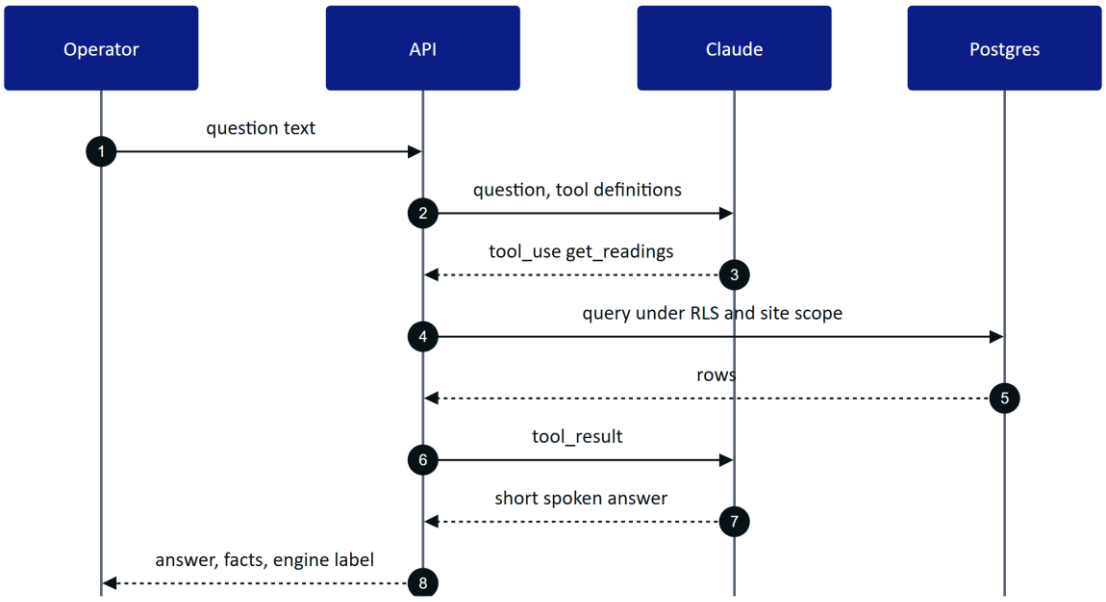

Figure 221. *Grounded voice answer: the model only sees data returned by tenant-scoped tools*

10.11.2 Controls

Table 290. Data sent to the AI provider

Risk	Control	Status
Hallucinated values	System prompt allows only tool values; facts returned with source and time; rules fallback never guesses	**Built**
Cross-tenant data in prompts	Tools run inside the caller's tenant transaction and site scope	**Built**
Prompt injection from scanned text	Vision output is a strict schema of fields and tables, not instructions; values are cross-checked with on-device OCR; nothing is executed from extracted text	**Built**
Unreviewed AI output becoming a record	Every field must be confirmed or corrected; flagged items block verification; a different approver signs off	**Built**
Unsafe operational advice	Prompt says Hydris never operates equipment and points to procedures and supervisors	**Built**
Excess data to the provider	Per-org opt-out, AI usage log, zero-retention terms	**Planned**

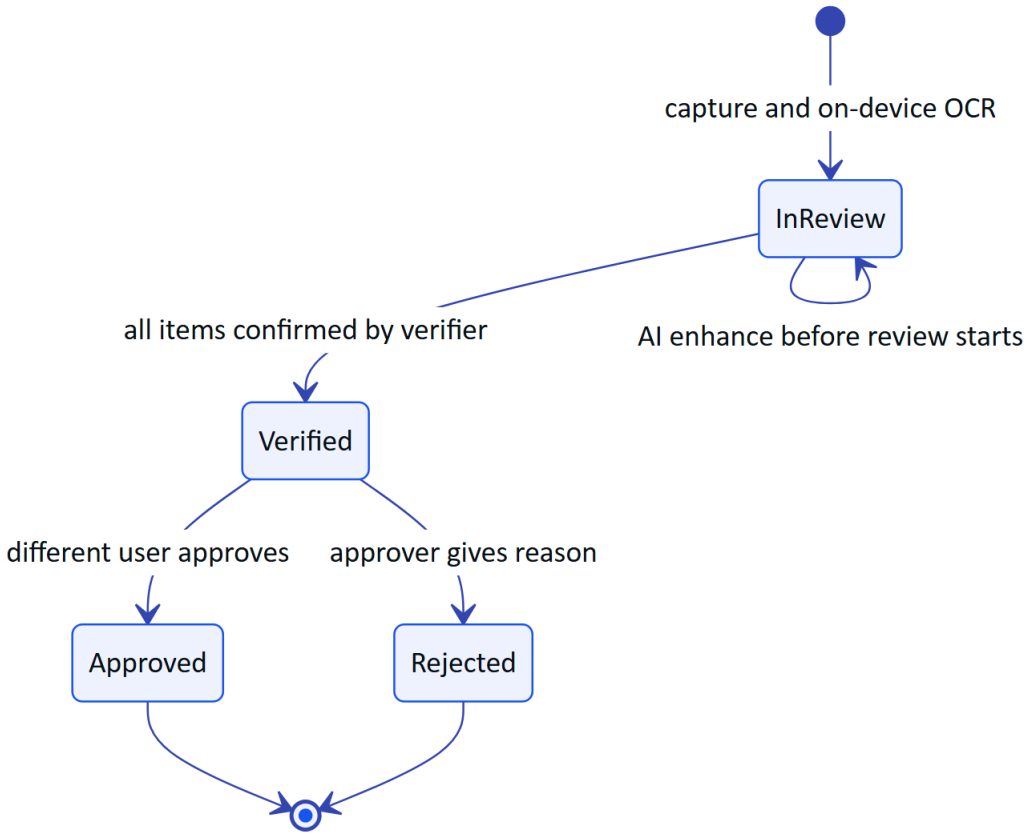

Figure 222. *Audit document states: AI output becomes a record only after two people*

10.12 Camera and bystander privacy

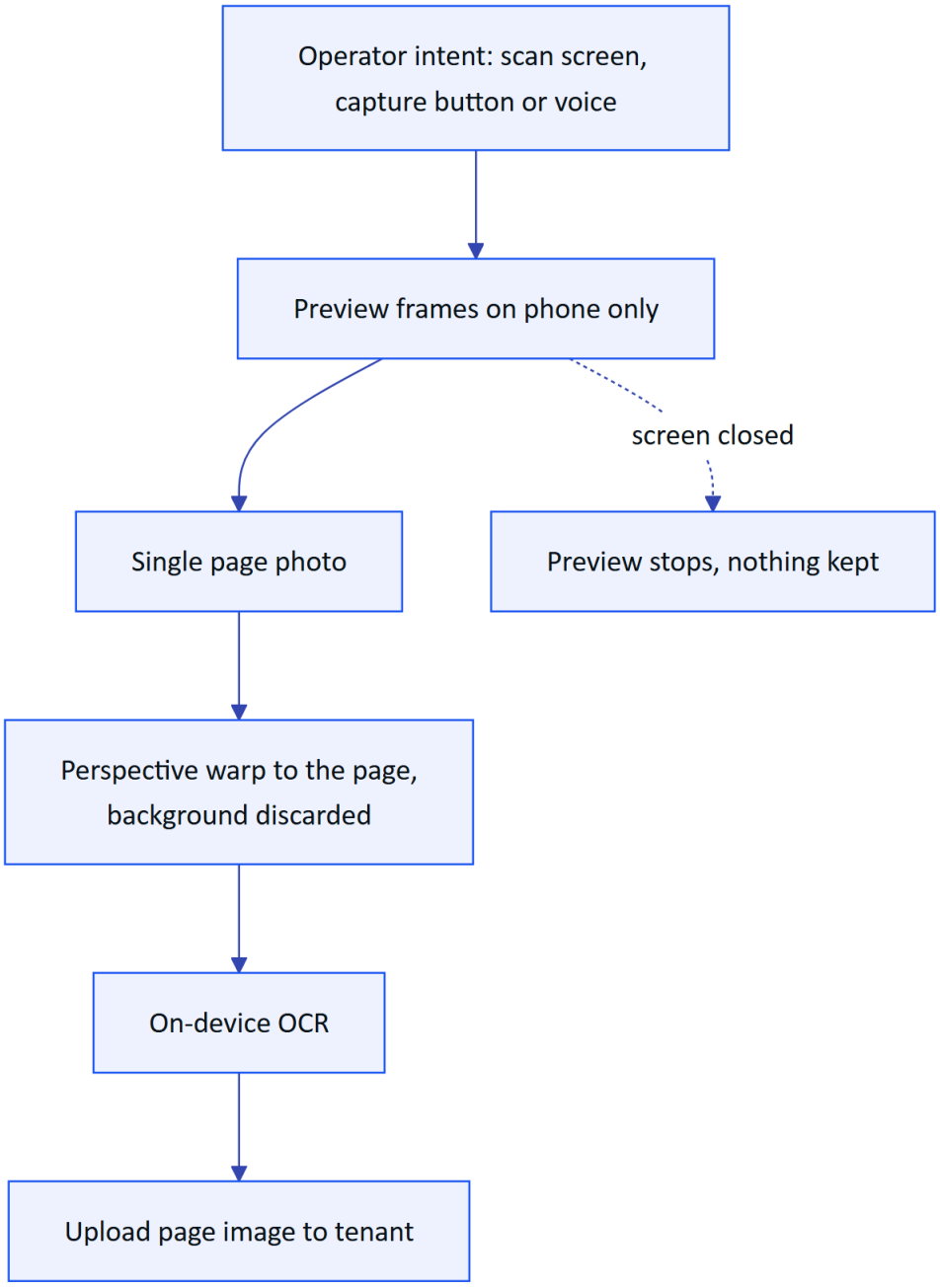

Figure 223. *Camera use happens only on explicit intent*

Table 291. AI safety controls

Commitment	Status
No background or continuous recording; no video stored	**Built**
Preview frames stay on the phone; only the final page is uploaded	**Built**
Meta glasses capture LED cannot be disabled by apps (vendor firmware)	Vendor control
Guided capture crops to the page quad	**Built**
Wake phrase matched on transcripts; only text after "Hey Hydris" is sent; audio never uploaded to Hydris	**Built**

Commitment	Status
Browser speech engines may send audio to the browser vendor	Disclosed; prefer DAT speech
Face and badge redaction before upload	**Planned**
Camera-free zones per site	**Planned**
Site signage and works council consultation	Customer process

10.13 Worker privacy and monitoring ethics

Hydris records who did what to a plant record because regulators require it. It is not a productivity surveillance tool, and we design to keep it that way.

- Device `last_seen_at` and `last_ip` exist for connectivity and security, not attendance. No location tracking.
- No continuous audio or video. No keystroke or gaze analytics.
- Reports on individual performance are not built and are not on the roadmap; aggregated workload views will be site-level only.
- Workers can see their own actions in the audit log through their supervisor; access requests are handled under DPDP and GDPR rights.
- Customers agree in the DPA not to use Hydris data for disciplinary monitoring without a lawful basis and worker consultation.

10.14 Compliance mapping

The table maps requirements to product controls. It is a mapping and a target, not a certification.

Table 292. Camera and bystander commitments

Framework	Relevant requirements	Hydris controls	Position
India DPDP Act 2023 and DPDP Rules 2025	Notice and consent, security safeguards, breach notice to the Board and principals, erasure, retention limits	Consent version on inquiries, encryption and RLS, audit trail	Partial; retention and rights tooling **Planned**
CPCB and SPCB consent conditions, OCEMS guidance	Unaltered, retrievable monitoring records	Provenance, append-only corrections, CSV export, freshness	Mapping; Hydris is not the regulatory feed
CERT-In Directions 2022	6-hour incident reporting, 180-day logs in India	Incident plan; log drain **Planned**	Target
ISO/IEC 27001:2022	ISMS and Annex A controls	Many technical controls in code	Target: ISMS build 2027
SOC 2	Security, availability, confidentiality criteria	Same	Target: Type I after first commercial year
IEC 62443	Zones and conduits, secure development	Read-only, outbound-only conduit from the DMZ gateway	Mapping; 4-1 practices as target
GDPR (exports, EU users)	Lawful basis, DPA, subprocessors, 72-hour breach notice	DPA, subprocessor list	Mapping

Figure 224. *Compliance roadmap (targets)*

10.15 Retention schedule

Retention below is the recommended default, configurable per organization (**Planned**). Today no purge job exists except `realtime_events` (1 day).

Table 293. Compliance mapping (targets, not certifications)

Data	Current	Recommended
Telemetry readings	Indefinite	Raw 2 years online, then archive; discharge data at least 5 years or per consent
Audit events, audit document events, alarm history	Indefinite, append-only	At least 7 years, then WORM export
Audit page images	Indefinite in Postgres	Object storage with lifecycle per record retention
Sessions, device sessions, pairing challenges	Not purged	Purge 30 days after expiry
Mail outbox (contains reset links)	Indefinite	Purge after send or 7 days
Inquiries	Indefinite	24 months or on consent withdrawal
Realtime events	1 day	Keep
Logs	Vercel default	180 days in India for Indian tenants

10.16 Audit logging

Table 294. Retention schedule

Category	Example actions
Authentication	`auth.login`, `auth.login_failed`, `auth.logout`, `auth.mfa_enabled`, `auth.password_reset`, `auth.refresh_reuse_detected`, `auth.session_revoked`
Membership	`member.invitation_accepted`, role and scope changes
Devices	`device.pairing_denied`, `device.suspended`, `device.revoked`, `device.refresh_reuse_detected`
Records	`auditdoc.verified`, `auditdoc.approved`, `auditdoc.rejected`, telemetry corrections
Platform	`organization.created`, `organization.suspended`

Each event records org, actor user or device, action, entity, summary, metadata and IP (`audit()` in `core/services.ts`). Admins and managers read it through `GET /api/audit`. Planned: hash chaining, export to WORM storage and SIEM forwarding.

10.17 Vulnerability management

Table 295. Audit event catalogue (examples)

Activity	Cadence	Status
Dependency audit and updates	Weekly, CI gate on high and critical	**Planned**
Static analysis (CodeQL) and secret scanning	Every PR	**Planned**
Third-party penetration test	Before first commercial customer, then yearly	**Planned**
Responsible disclosure address and `security.txt`	Ongoing	**Planned**

Table 296. Vulnerability management

Severity	Fix target
Critical	72 hours
High	14 days
Medium	60 days
Low	Next planned release

10.18 Incident response

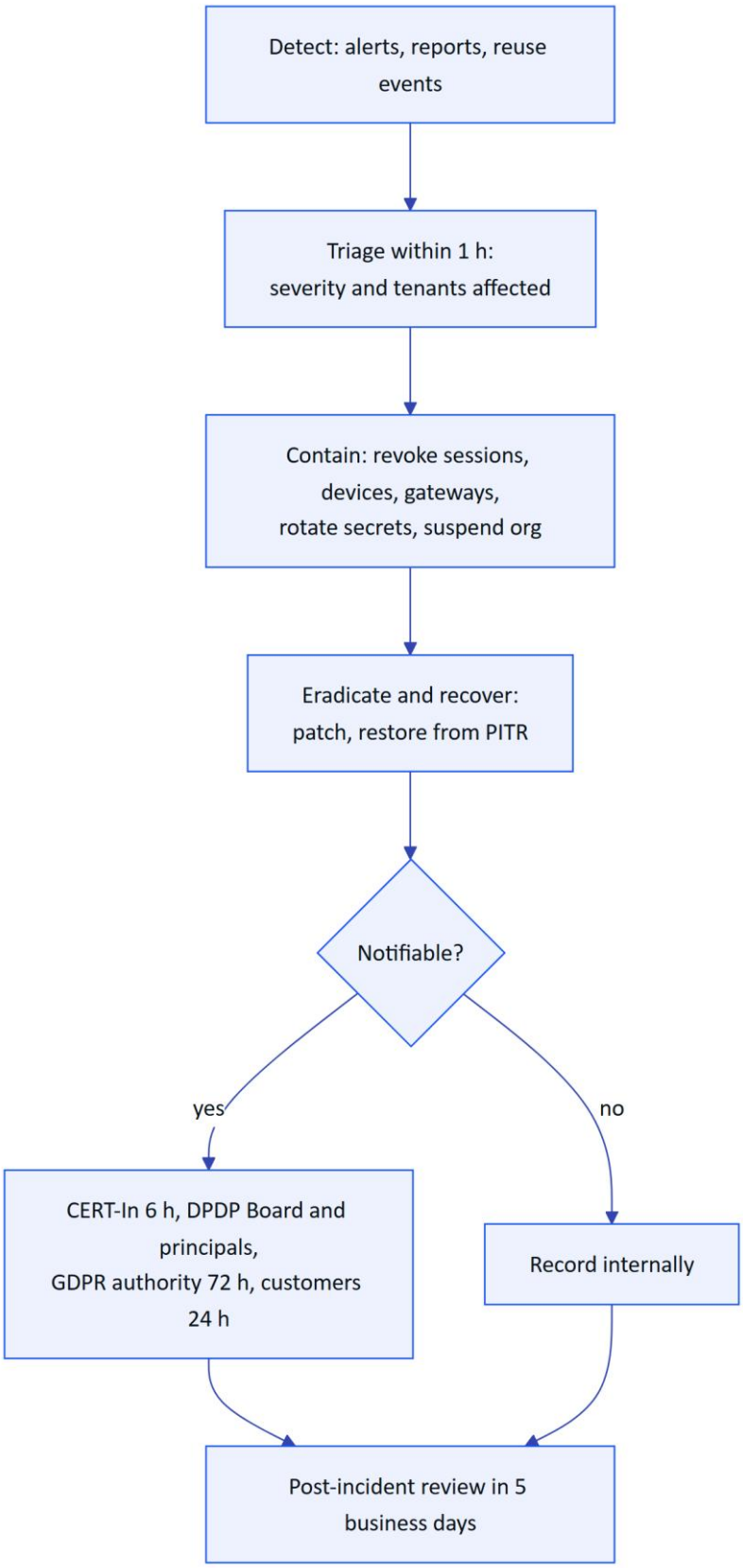

Figure 225. *Incident response flow*

Table 297. Remediation targets

Severity	Definition	Response	Customer notice
SEV1	Confirmed cross-tenant exposure, data breach or secret compromise	Immediate, all hands	Within 24 h
SEV2	Exploitable vulnerability or prolonged outage	Within 1 h	Within 48 h if affected
SEV3	Contained issue, no data exposure	Same business day	Release notes
SEV4	Hardening finding	Backlog	None

Containment tools already built: session revocation (`/auth/sessions`), device revoke, gateway revoke, organization suspension (`/platform/organizations/:id/status`), and `JWT_SECRET` rotation to sign everyone out.

10.19 Security roadmap

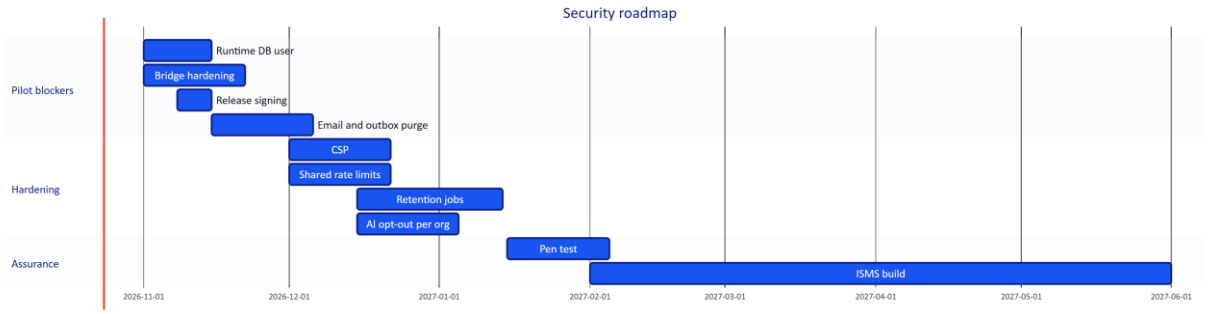

Figure 226. *Security roadmap*

10.20 Known gaps register

Table 298. Incident severity

ID	Gap	Likelihood	Impact	Rating	Plan
SEC-01	No CI security gates	High	Medium	High	CI with audit, CodeQL, secrets
SEC-02	Email not wired; reset links in `mail_outbox`	Medium	Medium	Medium	Provider and purge
SEC-03	Glasses web device refresh token in `localStorage`	Medium	High	High	Native Keystore bridge
SEC-04	Audit page images base64 in Postgres	Low	Medium	Medium	Object storage, encryption
SEC-05	No purge or retention jobs	High	Medium	High	Retention jobs
SEC-06	E-signature (re-auth, meaning) not built	Medium	Medium	Medium	Signature step on approve
SEC-07	Pairing code TTL 300 s with no attestation	Low	Medium	Low	Shorter TTL, attestation
SEC-08	Bridge WebView: open navigation, blanket permission grant	Medium	High	High	Origin allow-list
SEC-09	CSP disabled	Medium	Medium	Medium	Strict CSP
SEC-10	Owner connection could bypass RLS	Low	High	Medium	Separate runtime user
SEC-11	Per-instance rate limits	Medium	Low	Low	Shared limiter

ID	Gap	Likelihood	Impact	Rating	Plan
SEC-12	Release APK signed with debug key	High	High	High	Upload key
SEC-13	No per-org AI opt-out	Medium	Medium	Medium	Org setting

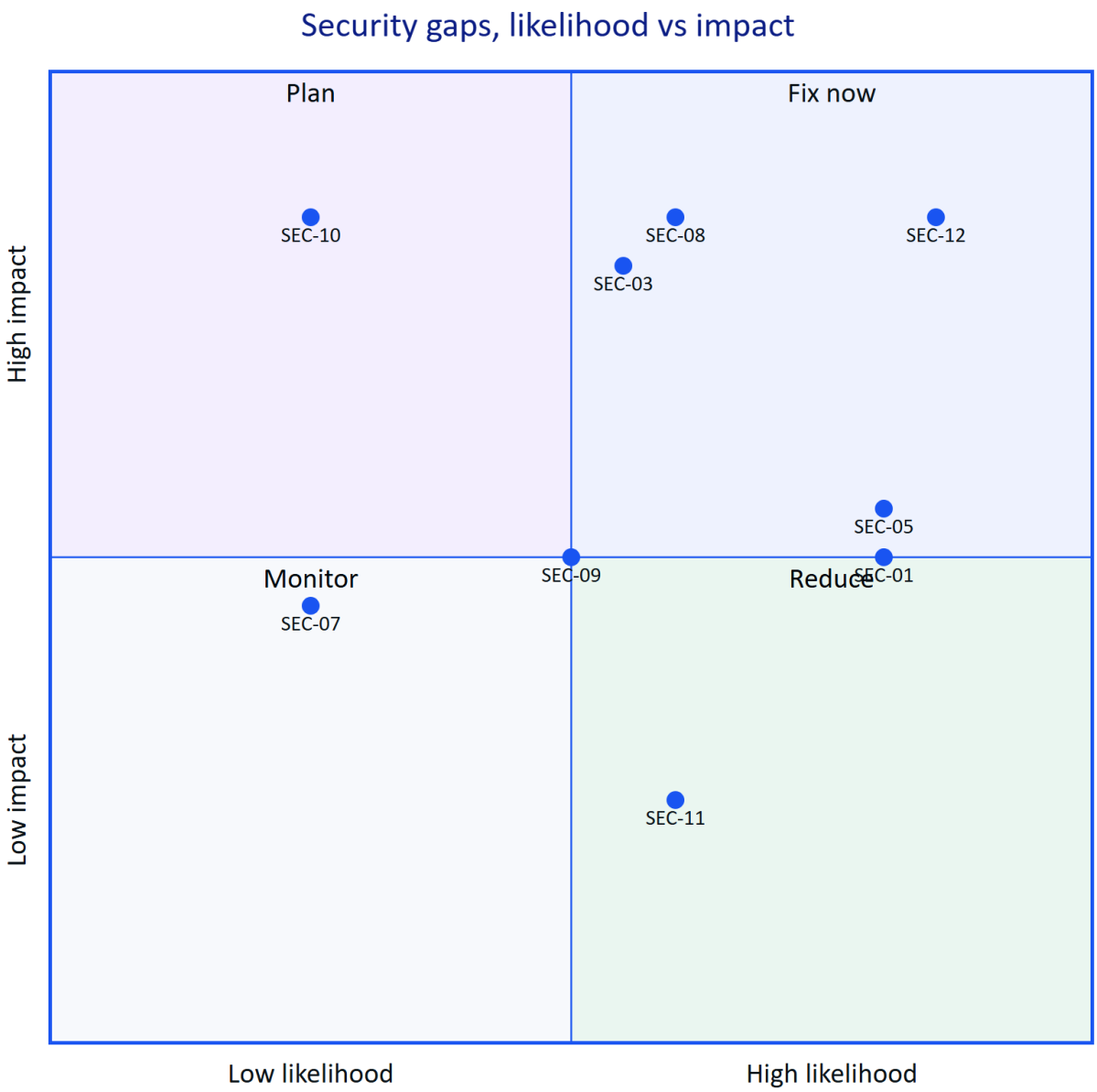

Figure 227. *Gaps by likelihood and impact*

DECISION

SEC-03, SEC-08, SEC-10 and SEC-12 block any pilot with real plant data. The remaining items are scheduled in the roadmap above.

11 QA and Test Plan

> **NOTE**
>
> **HFG-QA v2.0** · Draft for review · Owner: Quality engineering, Hydris. How Hydris proves that the console, glasses app, Android app and API are correct, safe and fit for a pilot plant

Hydris Field Glasses is used where a wrong number has consequences: a discharge reading that reaches a regulator, an alarm that is acknowledged but never resolved, a lab sheet whose decimal point moved during scanning. This plan describes how quality is built in and proven for every surface of the product: the operator console, the glasses web app and its simulator, the native Android app "Hydris Field", the API and the public website.

The plan separates what exists today from what is proposed. Every statement about the current product names the test file or source file it comes from. Counts in this document come from running `npm test` at the repository root on 22 October 2026 against commit `07cca23`. Targets are labelled as targets; nothing in this plan is a measured field result unless it says so.

71
automated tests on main
26 domain, 41 API, 4 Playwright journeys

67 / 67
unit and API tests passing
`npm test`, 22 Oct 2026

0
CI pipelines
No `.github/` directory yet

0
Android unit tests
Manual verification on a Pixel 7 emulator only

11.1 Purpose, scope and principles

11.1.1 What this plan covers

This plan is the single reference for test strategy, environments, acceptance criteria, non-functional testing, defect handling and release gates for the Hydris Field Glasses platform up to and including the first customer pilot. It is written for the people who will execute it (QA and engineering), the people who rely on it (product, customer success) and the people who will audit it (pilot customer quality, EHS and IT security leads).

Table 299. Surfaces and components in scope

Surface or component	Location in the repository	Primary users	Test levels that apply
Domain rules (roles, alarms, workflows, units, audit structuring, voice parsing)	`packages/domain/src`	All surfaces	Unit
API (Fastify, Drizzle, Postgres or PGlite, RLS)	`apps/api/src`	All clients	Integration, contract, security, load
Operator console	`apps/web/src/console`, `apps/web/src/app/console`	Supervisors, managers, admins	E2E, accessibility, exploratory
Glasses web app and simulator	`apps/web/src/glasses`, `/glasses`, `/simulator`	Field operators	E2E, layout, field trial
Audit scanning (browser)	`apps/web/src/lib/docdetect.ts`, `ocr.ts`, `GuidedCapture.tsx`	Operators, verifiers	Unit (proposed), accuracy benchmark, manual

Surface or component	Location in the repository	Primary users	Test levels that apply
Native Android app "Hydris Field"	`apps/android`	Operators, engineers	Unit (proposed), instrumented UI (proposed), manual, field trial
WebView bridge companion	`apps/glasses-bridge-android`	Legacy bridge path	Contract parity, manual
Public website and enquiry form	`apps/web/src/app/(site)`	Prospects	Smoke, accessibility
Deployment (Vercel, demo mode, cron)	`vercel.json`, `apps/api/src/serverless.ts`, `apps/api/src/jobs.ts`	Platform engineering	Smoke, configuration tests

11.1.2 Out of scope

- Control of plant equipment. Hydris is read-only to the plant (see Security and Privacy), so there is no PLC, SCADA or DCS write path to test. Tests assert that no such path exists.
- Validation for FDA-regulated GxP use. Hydris applies Part 11 concepts (audit trail, four-eyes approval) but is not a validated system, and this plan does not produce an IQ/OQ/PQ package.
- Meta firmware, the Meta AI app and the Wearables Device Access Toolkit (DAT) themselves. They are treated as dependencies whose behaviour is observed and recorded, not tested.

11.1.3 Quality principles

1. **Missing data is never a value.** Any test that touches telemetry must check freshness states (`live`, `delayed`, `stale`, `no_data`) and assert that a missing reading never renders as a number.
2. **Acknowledgement is not resolution.** Alarm tests check that each lifecycle step is separate, permissioned and audited.
3. **Machines propose, people verify.** OCR and voice outputs are suggestions until a person confirms them. Tests assert that unverified values cannot reach an approved record.
4. **Tenancy is enforced twice.** API role checks and Postgres row-level security are both tested, including queries without a `WHERE` clause.
5. **Deterministic first.** Demo seeds and simulation scenarios are deterministic, so a failing test reproduces.
6. **Field reality wins.** A feature is not pilot-ready until it has passed a trial with gloves, noise, sunlight and a weak network on the real hardware.

11.2 Quality strategy and test pyramid

11.2.1 Levels of testing

The strategy follows a conventional pyramid weighted towards fast, deterministic tests on the domain package and the API, with a thin layer of browser journeys and a deliberate, structured layer of manual and field testing on top. Field testing is not an afterthought for this product: glasses, gloves and plant noise create failure modes that no emulator reproduces.

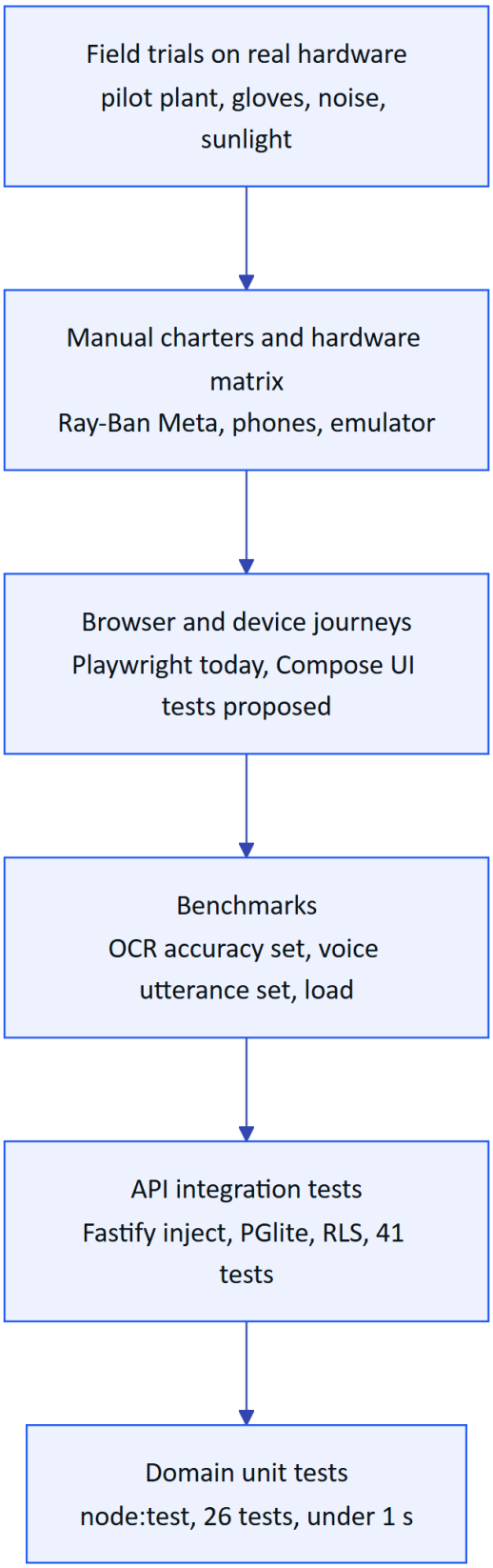

Figure 228. *Hydris test pyramid: fast deterministic tests at the base, field trials at the top*

Table 300. Test levels, tools and ownership

Level	Tooling	Runs where	Typical duration	Owner	Status
Domain unit	`node:test` with `tsx`, `packages/domain/test`	Laptop, CI (proposed)	Under 1 s (728 ms measured)	Engineering	**Built**
API integration	`node:test`, Fastify `app.inject`, PGlite in memory, `apps/api/test`	Laptop, CI (proposed)	About 75 s (73.4 s measured)	Engineering	**Built**
Browser journeys	Playwright 1.64, `tests/e2e/journeys.spec.ts`, local Chrome channel	Laptop with dev stack	2 to 4 min (estimate)	QA	**Built** (4 journeys)
Android unit and UI	JUnit, Compose UI test, Robolectric (proposed)	Gradle, emulator	To be measured	Mobile	**Planned**
OCR accuracy benchmark	Ground-truth set plus scoring script (proposed)	Laptop, nightly	10 to 20 min (estimate)	QA, ML	**Planned**
Voice utterance benchmark	Recorded and synthetic utterances plus scoring script (proposed)	Laptop, nightly	10 min (estimate)	QA	**Planned**
Load and soak	k6 (proposed)	Staging	30 to 60 min	Platform	**Planned**
Security	OWASP ZAP baseline, tenant fuzzing, `npm audit`, MobSF (proposed)	Staging, CI	15 to 45 min	Security	**Planned**
Accessibility	axe-core with Playwright, TalkBack, manual (proposed)	Laptop, device	15 min	QA, design	**Planned**
Manual charters	Session-based exploratory testing	Lab, emulator, devices	60 to 90 min per charter	QA	**Partial** (ad hoc today)
Field trials	Scripted rounds at a pilot site	Pilot plant	Half day per trial	QA, customer success	**Planned**

11.2.2 Entry and exit for each level

- **Domain and API:** every pull request (once CI exists) must keep all tests green. New behaviour in `packages/domain/src` or an API module ships with tests in the matching file.
- **Journeys:** run before every deployment to the pilot environment and after any change to authentication, pairing, routing or the glasses shell.
- **Benchmarks:** run nightly and before every release candidate. A regression beyond the tolerance in Audit scanning or Voice assistant blocks the release.
- **Manual and field:** run per release candidate against the hardware matrix, with the charter list in Manual test charters.

11.3 Current automated coverage

11.3.1 Summary

The repository has 71 automated tests in three suites. `npm test` runs the domain and API suites; Playwright runs the browser journeys against a running development stack (`npx playwright test`, which starts `npm run dev` through the `webServer` block in `playwright.config.ts` and uses the local Chrome channel unless `CI` is set). The Playwright suite was not executed for this edition; its last recorded state is the "E2E: exact password locator, deterministic inspection ordering" fix in commit `f77464d`.

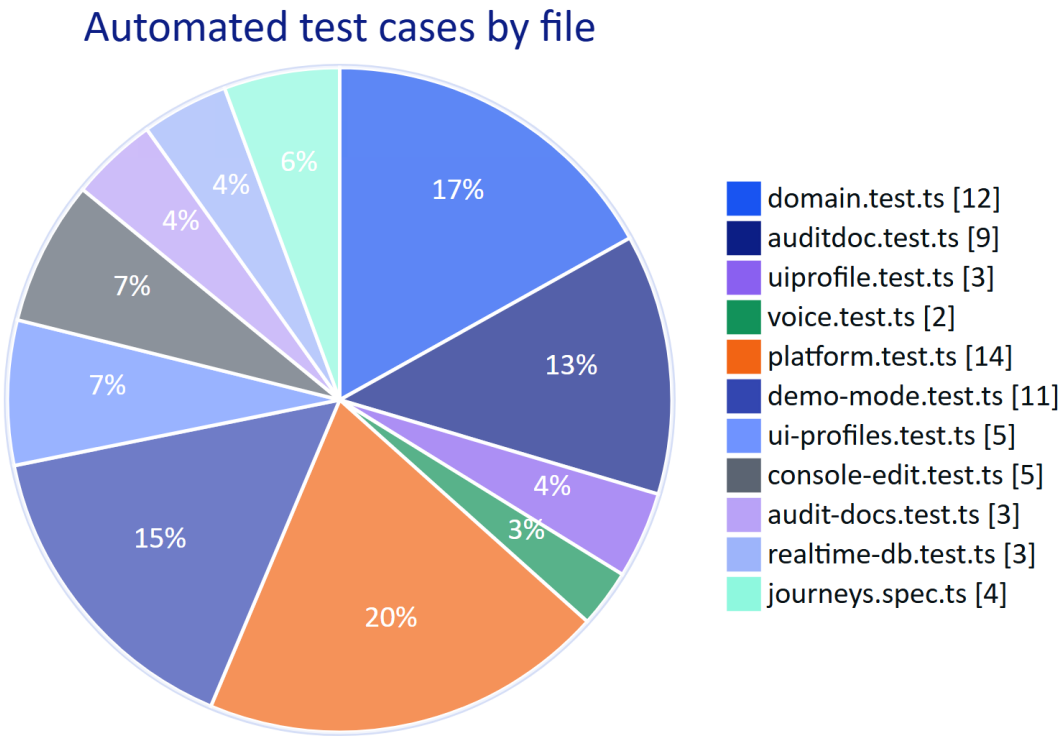

Figure 229. *Automated tests on main by suite and file (count of test cases, 22 October 2026)*

11.3.2 Domain package: packages/domain/test

These tests exercise pure functions with no database or network. They are the fastest feedback loop and the right home for any rule that can be expressed without I/O.

Table 301. domain.test.ts scenarios (12 tests)

Test name (abridged)	What it proves	Source under test
permission matrix: least privilege	Each role has only the permissions in the matrix; viewers cannot mutate	`roles.ts`
role assignment never escalates	`assignableRoles` never lets an inviter grant a role above their own	`roles.ts`
unit normalization converts and validates	Measurand units convert to canonical units; invalid units are rejected	`measurands.ts`
freshness never treats missing data as a value	`live`, `delayed`, `stale`, `no_data` derived from timestamps; missing data has no value	`measurands.ts`
threshold evaluation with deadband (hysteresis)	Values oscillating inside the deadband do not re-trigger	`alarms.ts`
on-delay suppresses transient breaches	A breach shorter than the on-delay raises no alarm	`alarms.ts`
alarm lifecycle: acknowledgement is not resolution	Transitions follow `ALARM_TRANSITIONS`; resolve needs a note and cannot start from `active`	`alarms.ts`
inspection and maintenance state machines	`INSPECTION_TRANSITIONS` and `MAINTENANCE_TRANSITIONS` allow only legal moves	`workflows.ts`
step outcome and progress	Step results (pass, fail, reading, choice) roll up into progress correctly	`workflows.ts`

Test name (abridged)	What it proves	Source under test
device state derivation	Device state from record status, last seen and session	`devices.ts`
pairing codes: format and normalization	8 characters from `PAIRING_CODE_ALPHABET`, display grouping, lower case and separators normalized	`devices.ts`
scenarios are deterministic and produce expected excursions	Simulation scenarios generate the same series every run and include the designed breaches	`scenarios`

Table 302. auditdoc.test.ts scenarios (9 tests)

Test name (abridged)	What it proves
key/value lines become typed fields with units	`Outlet pH: 7.4` becomes a number field; `412 mg/L` keeps its unit; dates and text are typed
aligned rows become a table with header	Rows aligned by x position become a table with a header row
two OCR passes cross-check values	A numeric disagreement (7.4 vs 7.1) is a `mismatch`; a one-letter name difference is `close`; O and 0 fold in numbers
helpers	`splitUnit`, `similarity`, `reviewProgress` behave as documented
decimal points matter; thousands separators do not	`7.4` vs `74` is a mismatch; `18,857` vs `18857` is a match
table columns returned as separate OCR blocks are re-joined	Column-wise OCR output becomes row-wise table data
values both engines agree on are still range-checked	`pH 71` agreed by both passes is still flagged as implausible (dropped decimal)
table cells are range-checked; a matching labelled value is offered as the fix	An implausible cell is flagged and a labelled value elsewhere on the page is suggested
label and value columns returned as separate lines (ML Kit) pair up by visual row	Android ML Kit output, where labels and values arrive as separate blocks, pairs correctly

Table 303. uiprofile.test.ts and voice.test.ts scenarios (5 tests)

File	Test	What it proves
`uiprofile.test.ts`	precedence device > user > role > org	The most specific usable profile wins; unusable profiles fall through
`uiprofile.test.ts`	normalization keeps required screens and fills defaults	A profile cannot hide required screens; missing settings take defaults
`uiprofile.test.ts`	measurement filter/order and diff summary	Measurement lists honour filters and order; the diff summary lists changes
`voice.test.ts`	wake phrase variants and the question after it	`WAKE_RE` accepts "hey Hydris" and common mis-transcriptions (hydras, hydrus, idris) and extracts the question
`voice.test.ts`	measurand detection prefers specific phrases	"outlet pH" beats "pH"; filler words are ignored

11.3.3 API: apps/api/test

API tests boot the real Fastify application with an in-memory PGlite database, run the real migrations including the row-level security policies in `drizzle/0001_rls.sql`, and drive it through `app.inject`. Shared setup lives in `apps/api/test/helpers.ts`.

Table 304. platform.test.ts scenarios (14 tests)

Test	Requirements exercised
no session, no data: data endpoints reject anonymous calls	Authentication on every data route
login, refresh rotation, reuse detection, logout invalidates access immediately	FR-001, FR-003, FR-009
database boundary: RLS isolates tenants even without WHERE clauses	FR-008
API tenant isolation and invitation onboarding of a second organization	FR-004, FR-006, FR-008
role-based access including privilege escalation attempts	FR-004, FR-007
gateway ingestion: normalization, idempotency, validation and alarm generation	FR-021, FR-022, FR-030
alarm lifecycle: acknowledgement separate from resolution, permissions enforced	FR-031, FR-032
glasses pairing, inspection on glasses, sync to console, review, revocation	FR-040 to FR-042, FR-050 to FR-053
expired and denied pairing challenges cannot be used	FR-050
deactivated members lose access immediately	FR-009
password reset via e-mailed single-use token	FR-005 (token flow; email transport is not wired)
deterministic simulation scenario drives alarms through the real pipeline (demo tenant only)	FR-084
public enquiry: validation, consent and spam controls	Website enquiry form
overview and reports are scoped and exportable	FR-025, FR-081

Table 305. demo-mode.test.ts scenarios (11 tests)

Test	What it proves
demo defaults	`DEMO_MODE` yields an in-memory database, the demo flag, `REALTIME_MODE=db` and the simulator on
owner logs in	The owner account is org admin of the demo org and platform admin
seeding is deterministic across instances	Two cold starts create identical IDs, so tokens and links survive instance changes
overview, reports, assist, telemetry history and UI profiles have content	The seed is rich enough for a demonstration
a token issued on one instance is accepted on another	Cross-instance access tokens in demo mode
glasses paired on one instance work on another	Device rebuilt from token claims
lazy ticker advances live data through the ingestion pipeline	Simulation without cron on serverless
demo guest: named account works on every instance and owns personal work	`POST /auth/demo` named guests
demo pairing across instances, guest name and tasks, logout and re-pair cycles	Pairing state shared through the demo store
demo audit scans are visible and reviewable from every instance	Audit snapshot sharing

Test	What it proves
voice assistant answers from the connected account (user and paired glasses)	FR-071, FR-072 with the rules engine

Table 306. ui-profiles.test.ts, console-edit.test.ts, audit-docs.test.ts and realtime-db.test.ts scenarios (16 tests)

File	Test	Requirements
`ui-profiles.test.ts`	only org_admin edits; factory_manager reads; operators denied	FR-054
`ui-profiles.test.ts`	device context defaults to built-in profile	FR-054
`ui-profiles.test.ts`	publish versioning, SSE to affected devices, enforcement, rollback	FR-054, FR-080
`ui-profiles.test.ts`	precedence via the effective endpoint	FR-054
`ui-profiles.test.ts`	assignment validation and cross-tenant isolation	FR-008, FR-054
`console-edit.test.ts`	readings list exposes provenance; manual entries record who entered them	FR-023
`console-edit.test.ts`	corrections are append-only, validated, permissioned, drive history and latest value	FR-024
`console-edit.test.ts`	cross-tenant correction attempts and history lookups return 404	FR-008
`console-edit.test.ts`	asset and sensor archive and restore with validation, permissions and history	FR-020
`console-edit.test.ts`	templates, tasks, rules and documents: edit, status changes and history	FR-030, FR-040, FR-043, FR-082
`audit-docs.test.ts`	capture, review, verify, independent approval, PDF	FR-061 to FR-066
`audit-docs.test.ts`	self-approval only in demo tenants (recorded); viewers cannot capture	FR-065
`audit-docs.test.ts`	vision output is cross-checked against OCR text inside each value box	FR-063
`realtime-db.test.ts`	DB mode: events persisted after the request and filtered per principal	FR-080
`realtime-db.test.ts`	`realtime_events` is not visible in tenant context	FR-008, FR-080
`realtime-db.test.ts`	cron job pass prunes old events and reports each job	Background jobs

11.3.4 Browser journeys: tests/e2e/journeys.spec.ts

Table 307. Playwright journeys (4 tests)

Journey	Steps	Requirements
anonymous visitors cannot reach the console	Open `/console`, expect redirect to `/login`	Authentication
sign in, overview, logout invalidates the session	Sign in, see the overview, log out, confirm the old session no longer works	FR-001, FR-003
glasses pairing by code, guided inspection on glasses, revocation	Second browser context opens the glasses app, reads the code, the console approves it, the operator completes an inspection, the admin revokes the device	FR-041, FR-050, FR-052

Journey	Steps	Requirements
operators cannot reach administration they lack permission for	Operator signs in and is blocked from admin pages	FR-007

11.3.5 Coverage map and known gaps

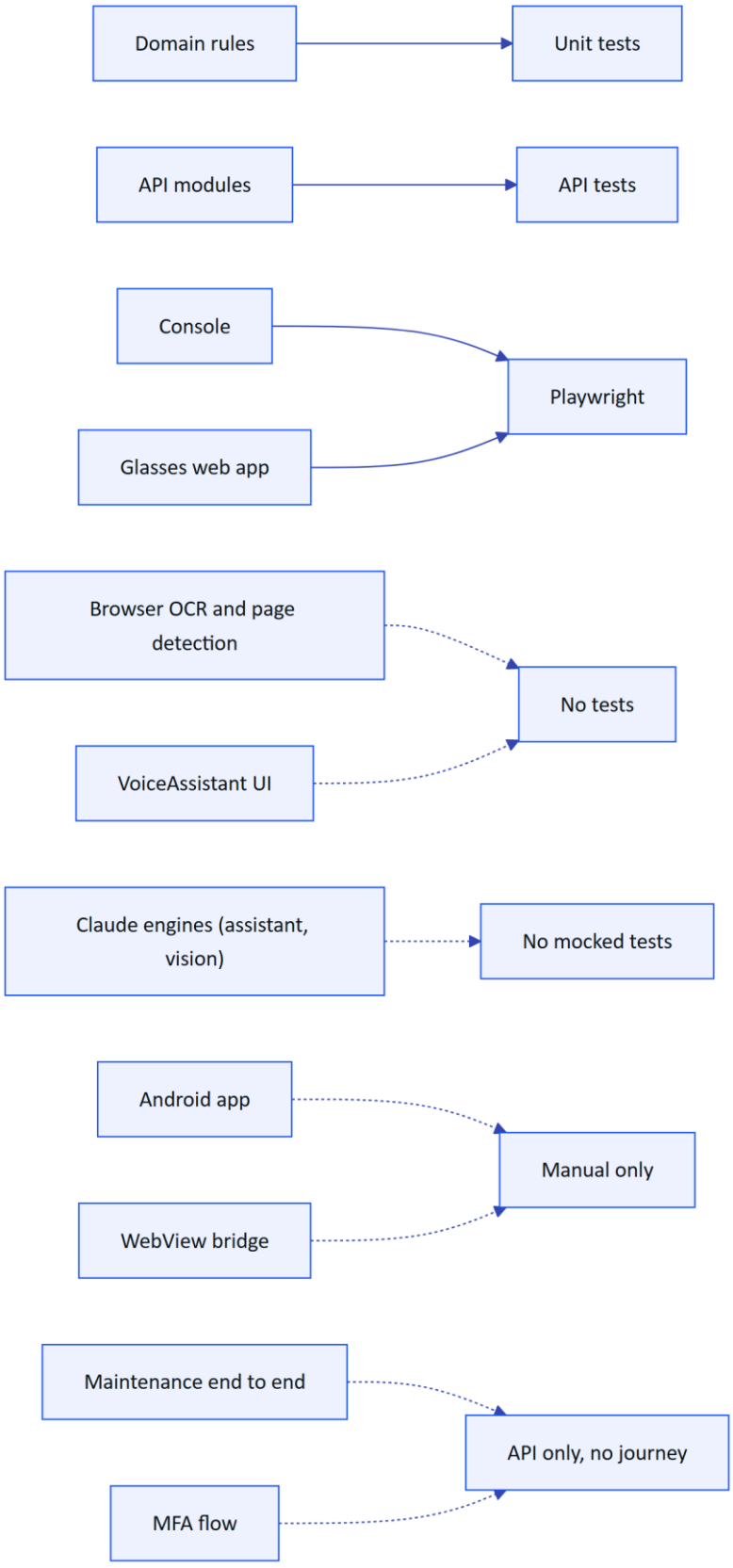

Figure 230. *Where automated coverage exists today (solid) and where it is missing (dashed)*

Table 308. Automated coverage gaps and the proposed fix

Gap	Risk if left	Proposed test	Priority
`docdetect.ts` and `ocr.ts` run only in the browser	Page detection or pass enhancement regress silently	Node unit tests with canvas fixtures (`@napi-rs/canvas`) and a stored set of frames	P1
`GuidedCapture` and `AuditScan.tsx` UI	Auto-capture never fires, or fires on a blurred frame	Playwright with a fake camera stream (`--use-file-for-fake-video-capture`)	P1
`VoiceAssistant.tsx` UI	Wake phrase ignored or card cannot close	Playwright with a stubbed `SpeechRecognition`	P2
`/assistant/ask` with the Claude engine	Tool misuse, a number not present in `facts`	API test with a mocked Anthropic client returning scripted tool calls	P1
Vision engine for audit scans	Vision text accepted without the OCR cross-check	API test with a mocked vision response that disagrees with OCR	P1
Android app (`apps/android`)	Regressions found only by hand	JUnit for `OcrEngine` parsing and `DocDetect`, Compose UI tests for login, home, alarms, scan	P1
Maintenance actions end to end	Hold, complete, reopen broken in the UI	Playwright journey	P2
MFA enrolment and step-up	Lockout of admins	Playwright journey with a TOTP helper	P2
Accessibility	WCAG 2.2 AA failures on console and mobile	axe-core in Playwright, Android Accessibility Test Framework	P2
Load	Ingestion or SSE fall over at pilot scale	k6 scripts	P2

IMPORTANT

The single biggest quality gap is not a missing test but the missing CI pipeline. Tests that only run on a laptop decay. The proposed pipeline in Proposed CI test pipeline is the first task of the hardening phase in the Roadmap.

11.4 Test environments and data

11.4.1 Environments

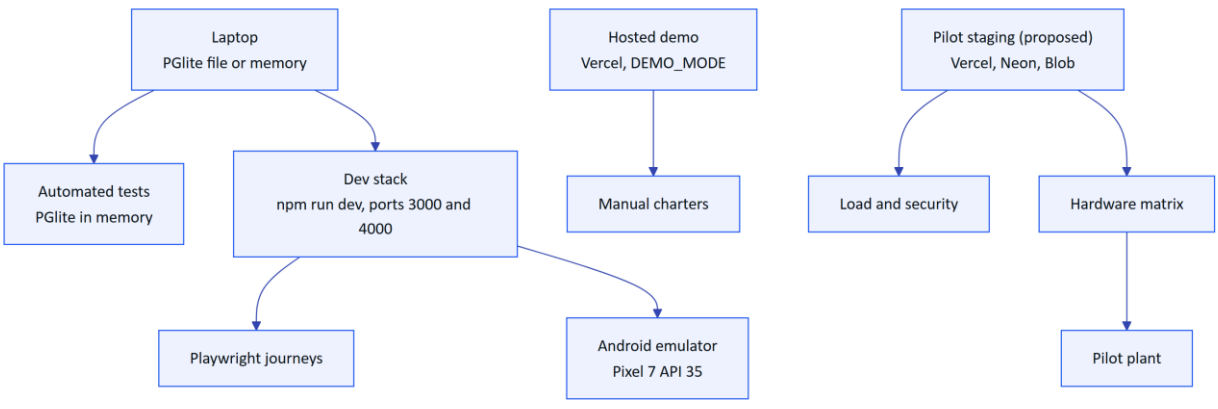

Figure 231. *Test environments, from a developer laptop to the pilot plant*

Table 309. Environment catalogue

Environment	Database	Realtime	Data	Purpose	Status
Unit and API tests	PGlite `memory://` per test file	Memory, DB mode in `realtime-db.test.ts`	Fixtures created per test	Automated regression	**Built**
Developer stack	PGlite directory `./data/pglite` (default `DATABASE_URL`) or local Postgres	Memory	`npm run seed:demo`	Journeys, emulator testing	**Built**
Hosted demo	In-memory PGlite per serverless instance	DB mode (per instance)	Deterministic synthetic tenant	Demonstrations, manual charters	**Built**
Pilot staging	Neon Postgres, private Vercel Blob	DB mode	Seed plus anonymised pilot configuration	Load, security, hardware, UAT	**Planned**
Pilot production	Neon Postgres (India region preferred)	DB mode	Real plant data	Pilot	**Planned**
Container or on-prem	Docker Compose Postgres	Memory	Customer data	Edge or air-gapped deployments	**Partial** (`docker-compose.yml`, not tested)

11.4.2 Demo mode as a test environment

Demo mode (`apps/api/src/demo.ts`, `apps/api/src/config.ts`) is turned on by `DEMO_MODE=true` or automatically on Vercel when `DATABASE_URL` is unset. It runs the database in memory, seeds a deterministic synthetic tenant, uses built-in public secrets, sets `REALTIME_MODE=db`, and advances simulations lazily on each request instead of on timers. Access tokens last 12 hours in demo mode (`accessTokenTtl`).

Demo mode is valuable for testing because the seed is identical on every instance, so screenshots, scripts and expected values are stable. It also hides some production behaviour, which testers must keep in mind:

> **WARNING**
>
> Demo mode differs from production in ways that matter for testing. Sessions are per instance, so an access token is accepted across instances only because demo mode relaxes the check. Data resets on every cold start. Verifier self-approval of audit documents is allowed (and recorded) only in demo tenants. A defect that reproduces only in demo mode is still a defect, but a pass in demo mode does not prove the production path. Every release candidate is therefore also tested on pilot staging.

11.4.3 Accounts and seed data

The demo seed (`apps/api/src/seed`, `seedDemoShowcase`) creates an owner account and one account per role in the demo organization. The account list and shared demo password are documented in the documentation index and the Operations Runbook; this plan does not repeat them. Named guest accounts are created on demand through `POST /auth/demo` with a display name of two or more characters (Android: `LoginScreen.kt`, "Enter demo mode").

Table 310. Test personas mapped to seeded roles

Persona	Role	Used in charters for
Owner	`org_admin` plus platform admin	Platform organization management, UI profiles, devices
Org admin	`org_admin`	Members, invitations, sites, profiles
Manager	`factory_manager`	Rules, reports, audit approval
Supervisor	`supervisor`	Dispatch, inspection review, audit verification
Operator (two accounts)	`field_operator`	Rounds on glasses, alarms acknowledgement, scanning
Engineer	`maintenance_engineer`	Maintenance tasks, alarm investigation and resolution
Viewer	`viewer`	Read-only checks, negative permission tests
Named guest	Guest in the demo org	Personal work, cross-instance pairing

11.4.4 Test data sets

Table 311. Curated data sets used across the plan

Data set	Contents	Location	Status
Simulation scenarios	Deterministic signals with designed excursions (for example `normal_operations`)	`packages/domain` scenarios, `seed/showcase`	**Built**
Audit OCR ground truth	Photographed log sheets and lab reports with hand-keyed truth files	`tests/fixtures/audit/` (proposed)	**Planned**
Voice utterance set	Questions per intent, recorded by several speakers in quiet and noisy conditions	`tests/fixtures/voice/` (proposed)	**Planned**
Tenant fuzz IDs	Random and cross-tenant UUIDs for every `:id` route	Generated per run	**Planned**
Load profiles	Gateway batches of up to 500 readings, SSE client counts	`tests/load/` (proposed)	**Planned**

11.5 Manual test charters by surface

Manual testing is session-based: each charter has a mission, a time box, a set of areas to explore and a short report (what was tested, bugs, questions, coverage notes). Charters are run per release candidate. Exploratory notes become automated tests when a defect is found.

Table 312. Console charters

ID	Mission	Time box	Focus areas
CH-C01	Explore member management for privilege escalation	60 min	Invite each role from each role, role change, deactivation, site scoping
CH-C02	Explore alarm triage under load	60 min	Alarm list with 50 or more active alarms, filters, acknowledge, investigate, resolve with note, close, history
CH-C03	Explore inspection dispatch and review	60 min	Templates, route dispatch, review queue, return with comment, approval
CH-C04	Explore audit document review and approval	90 min	Field confirm, correct, flag; table cells; verify blocked while pending; approval by another user; PDF

ID	Mission	Time box	Focus areas
CH-C05	Explore device fleet management	45 min	Pair, assign, suspend, reactivate, revoke, reassign, history
CH-C06	Explore UI profile publishing	45 min	Draft, publish, rollback, scoped assignments, live update on a paired simulator
CH-C07	Explore telemetry corrections	45 min	Correct and invalidate, provenance, history, CSV export
CH-C08	Explore light and dark themes and narrow windows	30 min	Contrast, truncation, tables at 1280 px and 1024 px

Table 313. Glasses web app and simulator charters

ID	Mission	Time box	Focus areas
CH-G01	Explore pairing from a cold start	45 min	Code display, QR, expiry, cancel, re-pair after revoke, logout and unpair menu
CH-G02	Explore a full round on the HUD layout	60 min	Home, next task, steps of each type, offline queue, resume after reload
CH-G03	Explore the phone layout	45 min	Safe areas, bottom tab bar, back navigation never leaving the app, voice card overlap
CH-G04	Explore guided capture	60 min	Page detection, auto-capture, manual capture, 6-page limit, retake, upload progress
CH-G05	Explore the voice assistant	45 min	Wake phrase, questions per intent, "can't answer" path, card close, offline
CH-G06	Explore alarm interrupts	30 min	Critical alarm banner over a task, acknowledge from glasses, return to the task
CH-G07	Explore text scale and density profiles	30 min	`UI_TEXT_SCALES` 0.9 to 1.3, handedness, compact density

Table 314. Android app "Hydris Field" charters

ID	Mission	Time box	Focus areas
CH-A01	Explore sign-in and session restore	45 min	Login, demo guest, process death, restart restores from the encrypted cookie store, logout
CH-A02	Explore home, work and sensor screens	45 min	Overview, inspections, maintenance, sensor history, pull to refresh, empty states
CH-A03	Explore alarms	45 min	List order by severity, acknowledge, realtime arrival over SSE, permission errors
CH-A04	Explore scanning with the phone camera	60 min	CameraX preview, page detection, ML Kit OCR, import from photo, upload of extracted values
CH-A05	Explore the assistant	30 min	Typed and spoken questions, TTS output, glasses speaker routing when connected
CH-A06	Explore glasses registration	60 min	Meta AI app link, DAT registration state, registering glasses as a Hydris device, revoke from console
CH-A07	Explore poor connectivity	45 min	Airplane mode mid-action, slow network, retry on 401, error messaging

Table 315. Website and platform charters

ID	Mission	Time box	Focus areas
CH-W01	Explore the enquiry form	30 min	Validation, consent, spam controls, rate limit (5 per 10 min)
CH-W02	Explore the public glasses demo (`/glasses?demo=1`)	20 min	Client-only behaviour, no API calls with credentials
CH-P01	Explore a cold start of the hosted demo	30 min	First request latency, seed, demo badge, cron report

11.6 Hardware and device matrix

11.6.1 Matrix

Hardware testing has not yet been run on physical Meta glasses. The DAT bridge and the native app's glasses integration are code-complete but unverified on hardware (Glasses Integration, "Limitations and known gaps"). The Android app has been verified manually on an emulated Pixel 7 running API 35.

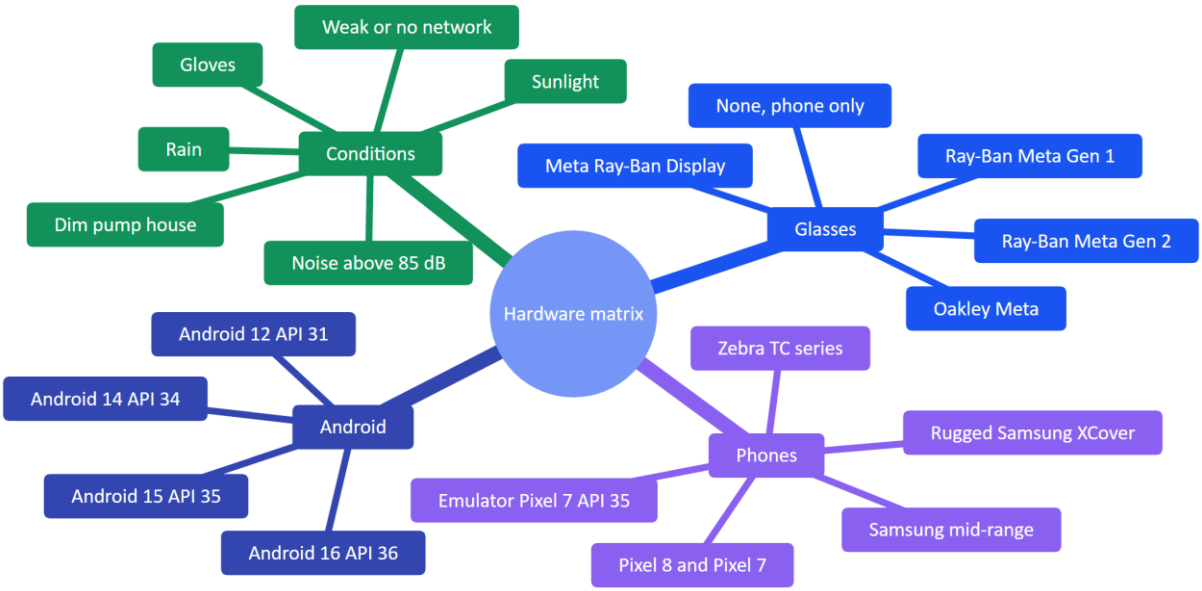

Figure 232. *Hardware and environment dimensions covered by the matrix*

Table 316. Hardware configurations (H-codes are stable test identifiers)

ID	Glasses	Phone	Android	Path	Key cases	Status
H0	None	Emulator Pixel 7	15 (API 35)	Native app, phone camera	Login, demo, home, assistant, alarm acknowledge, scan import, upload	**Verified manually**
H1	Ray-Ban Meta Gen 2	Pixel 8	15 or 16	Native app plus DAT	Registration, camera permission, preview frame rate, capture button, 6-page scan, wake word in noise, TTS audibility, Bluetooth reconnect, 4 h battery	**Planned**

ID	Glasses	Phone	Android	Path	Key cases	Status
H2	Ray-Ban Meta Gen 1	Samsung A-series	14	Native app plus DAT	As H1 with the mid-range performance budget	**Planned**
H3	Oakley Meta (HSTN or Vanguard)	Pixel	15	Native app plus DAT	Camera field of view, inputs	**Planned**
H4	Meta Ray-Ban Display	Pixel	15	DAT, glasses web app HUD (future)	Display rendering, Neural Band input	**Planned**
H5	None	Rugged Samsung XCover or Zebra TC	13 to 14	Native app, CameraX, SpeechRecognizer	Gloves, sunlight, offline round	**Planned**
H6	Ray-Ban Meta Gen 2	Any supported	12 or later	WebView bridge `glasses-bridge-android`	Bridge contract parity with the native app	**Planned**
H7	None	Desktop Chrome	n/a	`/simulator`	HUD and pairing regression	**Built** (manual)
H8	None	Android 12 device	12 (API 31)	Native app	`minSdk 31` floor: camera, speech, Keystore	**Planned**

The Android app targets `minSdk 31` and `targetSdk 36` (`apps/android/app/build.gradle.kts`), so the matrix covers API 31 as the floor and API 36 as the ceiling.

11.6.2 Environmental conditions

Table 317. Environmental conditions and pass criteria for field trials

Condition	How it is produced	Pass criterion (target)
Direct sunlight	Outdoor clarifier deck at midday	Phone screen readable; guided capture still detects an A4 page within 5 s
Dim interior	Pump house, under 50 lux	Capture produces a readable page; OCR field agreement within 10 points of the lab baseline
Noise	Blower room, 85 dB(A) or higher, measured with a sound meter app	Wake phrase accepted at least 80 % of attempts at normal speaking voice
Gloves	Nitrile and leather work gloves	Every primary action reachable; touch targets at least 48 dp
Rain	Light spray (Ray-Ban Meta is IPX4 per Meta)	No functional loss; no water damage claims by testers
No network	Airplane mode mid-round	Steps queue; replay after reconnect without duplicates
Weak network	Throttled 2G or 3G class profile	Pages load in under 10 s; uploads resume or fail visibly

11.7 Acceptance criteria by feature

Acceptance criteria are written in Given/When/Then form. Each criterion has an identifier (AC-area-number) used in the traceability matrix. Values in criteria come from code constants where they exist; the source is named.

11.7.1 Glasses pairing

Pairing follows a device-authorization flow (`apps/api/src/modules/devices.ts`, `packages/domain/src/devices.ts`). The glasses (or the Android app on their behalf) create a challenge, show an 8-character code, and poll; a signed-in user approves the code on `/pair` or in the console; the device receives its tokens on the next poll.

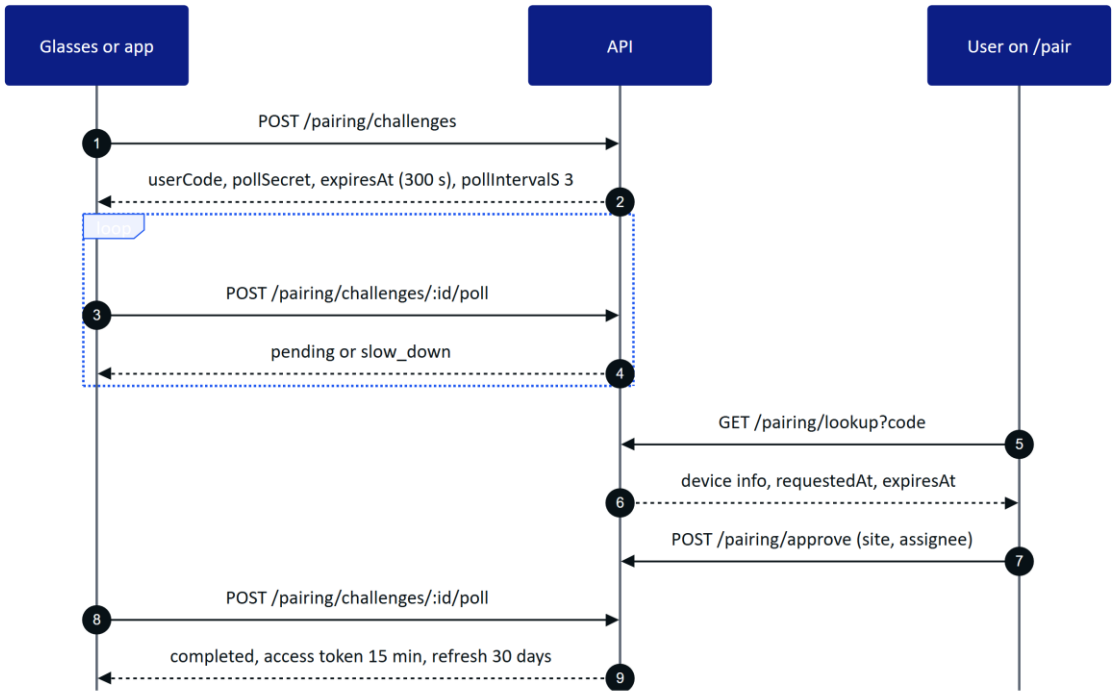

Figure 233. *Pairing flow with the timing constants that acceptance tests check*

Table 318. Pairing constants under test

Constant	Value	Source
Code alphabet	`23456789ABCDEFGHJKMNPQRSTVWXYZ` (no 0, 1, I, L, O, U)	`PAIRING_CODE_ALPHABET`
Code lifetime	300 s	`PAIRING_TTL_S`
Poll interval	3 s; polls closer than 2 s return `slow_down`	`PAIRING_POLL_INTERVAL_S`, `devices.ts`
Rate limits	Create 10, poll 40, cancel 20, lookup 10, approve 10, device refresh 30 per minute	`devices.ts` `limit()`
Device access token	15 min (12 h for the demo-mode sign-in path)	`TTL.accessToken`, `accessTokenTtl`
Device refresh token	30 days, rotating	`TTL.deviceRefreshMs`

> **NOTE**
>
> `docs/operations/vercel.md` describes demo pairing as needing the same instance "for its ~10 minutes". The code uses a 300 s challenge lifetime (the demo shared store keeps challenge records for 300 s plus 600 s). The acceptance criteria use 300 s. The documentation mismatch is tracked as a defect in the Roadmap tech-debt list.

AC-PAIR-01 Code format. Given a new challenge, when the code is displayed, then it has 8 characters from the pairing alphabet, and entering it in lower case or with spaces or dashes on `/pair` still finds the challenge.

AC-PAIR-02 Expiry. Given a pending challenge created at time T, when a user looks it up or approves it after T + 300 s, then the lookup returns not found and the device's next poll returns `expired`.

AC-PAIR-03 Polling discipline. Given a pending challenge, when the device polls twice within 2 s, then the second poll returns `slow_down` with `pollIntervalS: 3`, and the challenge is not cancelled.

AC-PAIR-04 Approval. Given a signed-in user with `device.manage` (or an operator approving their own glasses where allowed), when they approve the code with a site and an assignee, then the next poll returns `completed` with a device access token and a refresh token, the device appears in `/console/devices` with that site and assignee, and an audit event records the approval with the user and IP.

AC-PAIR-05 Single use. Given a completed challenge, when anyone approves the same code again, then the API rejects it, and no second device record is created.

AC-PAIR-06 Denial. Given a pending challenge, when the user denies it, then the device's next poll returns `denied` and the code cannot be approved afterwards.

AC-PAIR-07 Revocation. Given a paired device with an open SSE stream, when an admin revokes it, then the next API call returns 401 `device_revoked`, a refresh attempt fails, and the stream receives `session.ended` within 30 s.

AC-PAIR-08 Re-pair. Given a revoked device, when the glasses start a new challenge and an admin approves it, then the device works again, and the console history shows both the revocation and the new assignment.

AC-PAIR-09 Logout and unpair from glasses. Given a paired glasses app, when the operator chooses "Log out and unpair" in the Device and log out menu, then the server ends the device sessions (`/device/logout`) and the app returns to the pairing screen.

AC-PAIR-10 Brute force. Given an attacker guessing codes, when they exceed 10 lookups per minute from one client, then the API returns 429, and the code space (30 to the power 8) makes guessing within 300 s impractical.

11.7.2 Demo accounts and demo mode

AC-DEMO-01 Zero configuration. Given a fresh Vercel deployment with no environment variables, when the owner signs in with the documented demo credentials, then the overview shows live synthetic data and a visible demo badge.

AC-DEMO-02 Named guest. Given the login page, when a visitor chooses "Enter demo mode" and enters a name of at least two characters, then a guest account with that display name is created, sees personal inspections and tasks, and the same account works after the request lands on another instance.

AC-DEMO-03 Determinism. Given two cold-started instances, when both seed the demo tenant, then entity IDs are identical (`demo-mode.test.ts`).

AC-DEMO-04 Live data without cron. Given demo mode, when any request arrives more than one tick after the last, then the lazy ticker advances readings through the real ingestion pipeline and alarms can fire.

AC-DEMO-05 Isolation from production. Given demo mode is active, when the server starts, then it logs the public secrets warning, and no non-demo organization can be created or accessed with the public secrets.

AC-DEMO-06 Reset. Given a demo instance with user changes, when the instance cold starts, then all changes are gone and the seed is restored.

11.7.3 Alarms

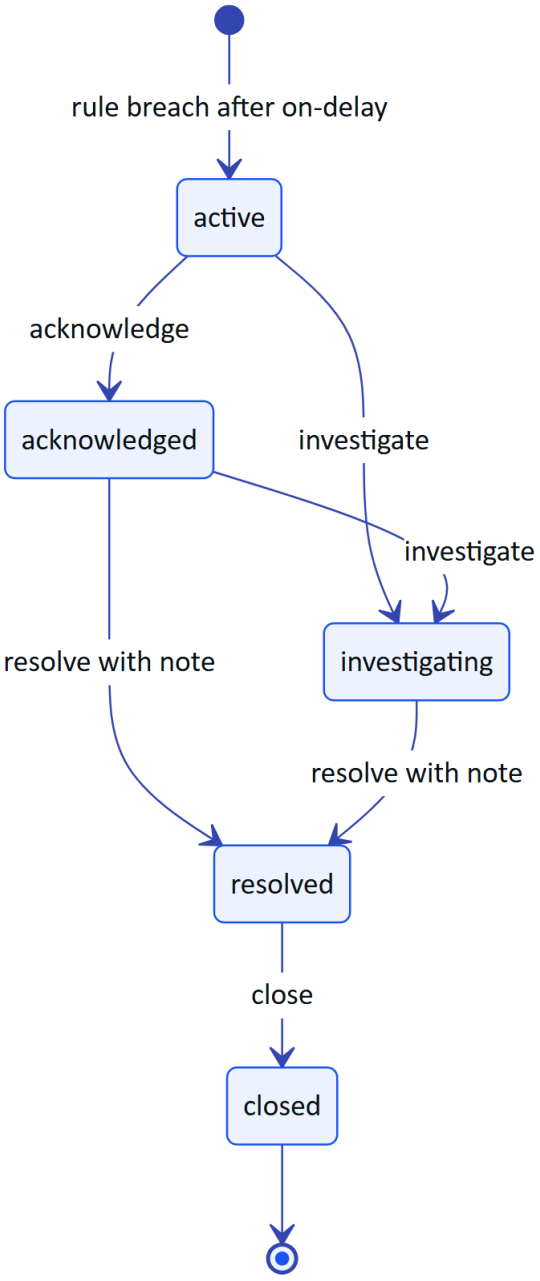

Figure 234. *Alarm lifecycle as enforced by ALARM_TRANSITIONS (packages/domain/src/alarms.ts)*

AC-ALM-01 Rule evaluation. Given a `high` rule with a deadband and an on-delay, when a value exceeds the threshold for less than the on-delay, then no alarm is raised; when it exceeds it for longer, then exactly one `active` alarm is raised, and values oscillating inside the deadband do not raise a second one.

AC-ALM-02 Stale data. Given a sensor with a `stale` rule, when no reading arrives within the rule window, then a stale alarm is raised, and the sensor shows `stale` (not its last value as current) on every surface.

AC-ALM-03 Acknowledge is not resolve. Given an `active` alarm, when an operator acknowledges it, then its state is `acknowledged`, it stays in the open list, and the acknowledging user and time are recorded.

AC-ALM-04 No shortcut to resolved. Given an `active` alarm, when anyone tries to resolve it, then the API rejects the transition.

AC-ALM-05 Resolution note. Given an `acknowledged` or `investigating` alarm, when a user resolves it without a note, then the API rejects it; with a note, the alarm is `resolved` and the note is in the history.

AC-ALM-06 Permissions. Given a viewer, when they try any alarm transition, then the API returns 403.

AC-ALM-07 Realtime delivery. Given a console and a paired glasses app connected over SSE, when an alarm is raised, then both show it within 2 s in memory mode (target) and within 3 s in DB mode (target; DB mode polls every 1.5 s).

AC-ALM-08 Glasses interrupt. Given an operator in the middle of an inspection on glasses, when a critical alarm is raised on their site, then an interrupt banner appears, the operator can acknowledge it or dismiss it, and the inspection resumes at the same step.

AC-ALM-09 Android acknowledge. Given the Android app signed in as an operator, when the operator acknowledges an alarm from the Alarms list, then the console shows it `acknowledged` by that user, and the app list updates without a manual refresh.

11.7.4 Inspections and maintenance

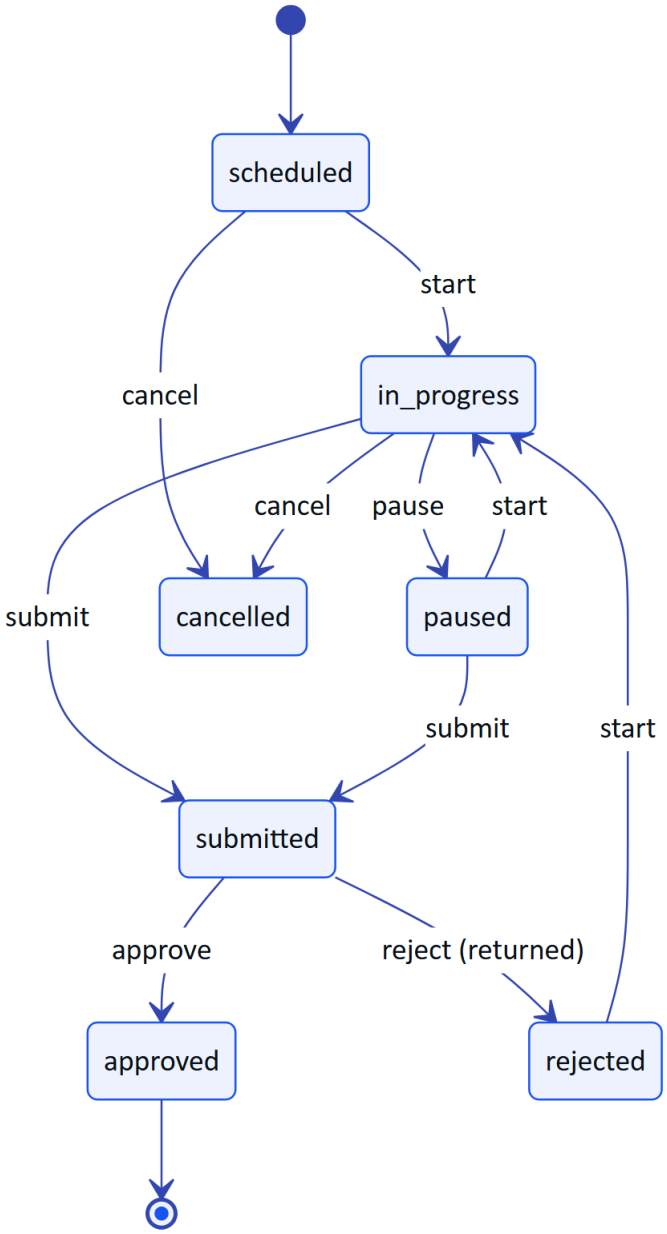

Figure 235. *Inspection state machine (INSPECTION_TRANSITIONS in packages/domain/src/workflows.ts)*

AC-INS-01 Next task. Given an operator with assigned inspections, when they open the glasses home screen, then the inspections appear in due order (up to 20), restricted to their site scope.

AC-INS-02 Step types. Given a template with check, reading, choice, text and photo steps, when the operator completes each on glasses, then each response is stored with its type, value, unit and timestamp, and progress reflects completed steps.

AC-INS-03 Offline replay. Given an operator mid-round, when the network drops and they complete three steps, then the glasses show "Offline, steps are saved on the glasses and sync automatically", and after reconnecting the three responses reach the server once each (idempotent on `client_op_id`).

AC-INS-04 Review. Given a submitted inspection, when a supervisor returns it with a comment, then the status is `rejected` (shown as "Returned"), the operator sees the comment, and restarting moves it to `in_progress`.

AC-INS-05 Illegal transitions. Given an `approved` inspection, when anyone tries to start, submit or cancel it, then the API rejects the transition.

AC-MNT-01 Follow-up creation. Given a failed inspection step or an alarm, when a supervisor creates a follow-up task, then a maintenance task of kind `inspection_followup` or `alarm_followup` links back to its origin.

AC-MNT-02 Maintenance lifecycle. Given an `assigned` task, when the engineer starts, holds, starts again and completes it, then each transition is recorded; `reopen` is allowed only from `completed` or `cancelled`.

11.7.5 Audit scanning

Audit scanning turns a paper log sheet or lab report into a verified, searchable record (FR-060 to FR-068). It is the feature where a test failure is most likely to become a compliance failure, so it has the most detailed criteria.

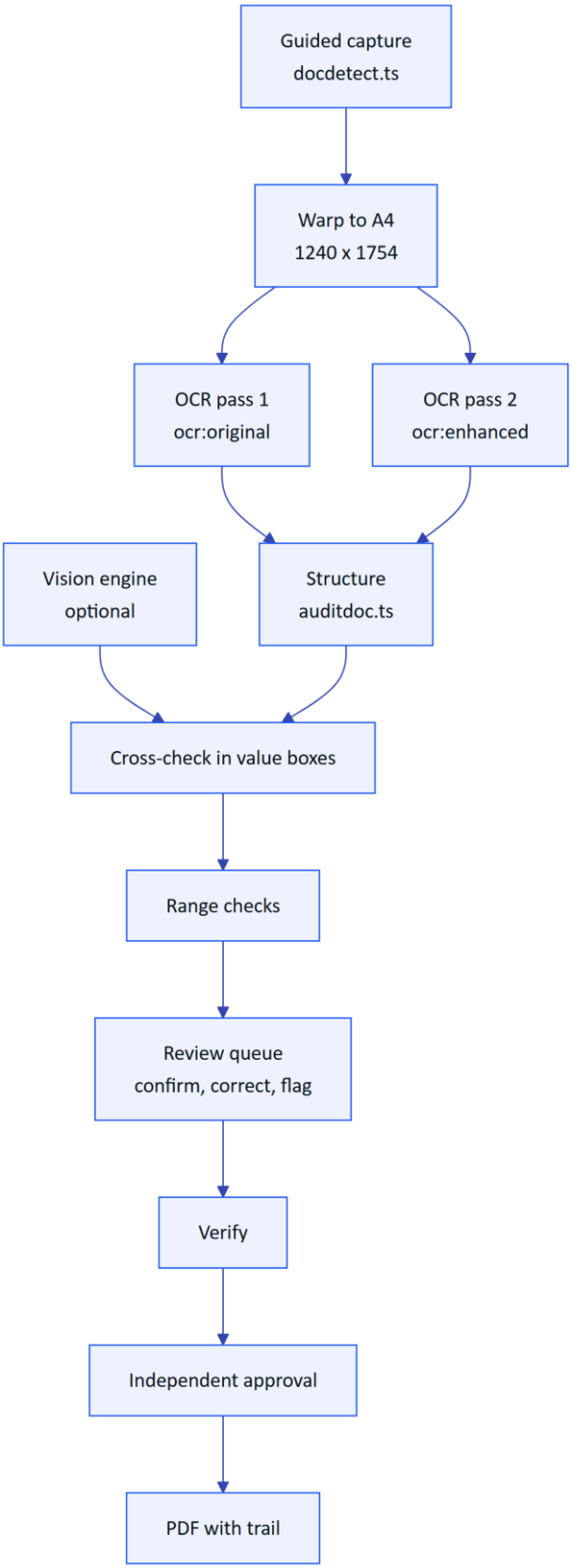

Figure 236. *Audit scanning pipeline and where each check happens*

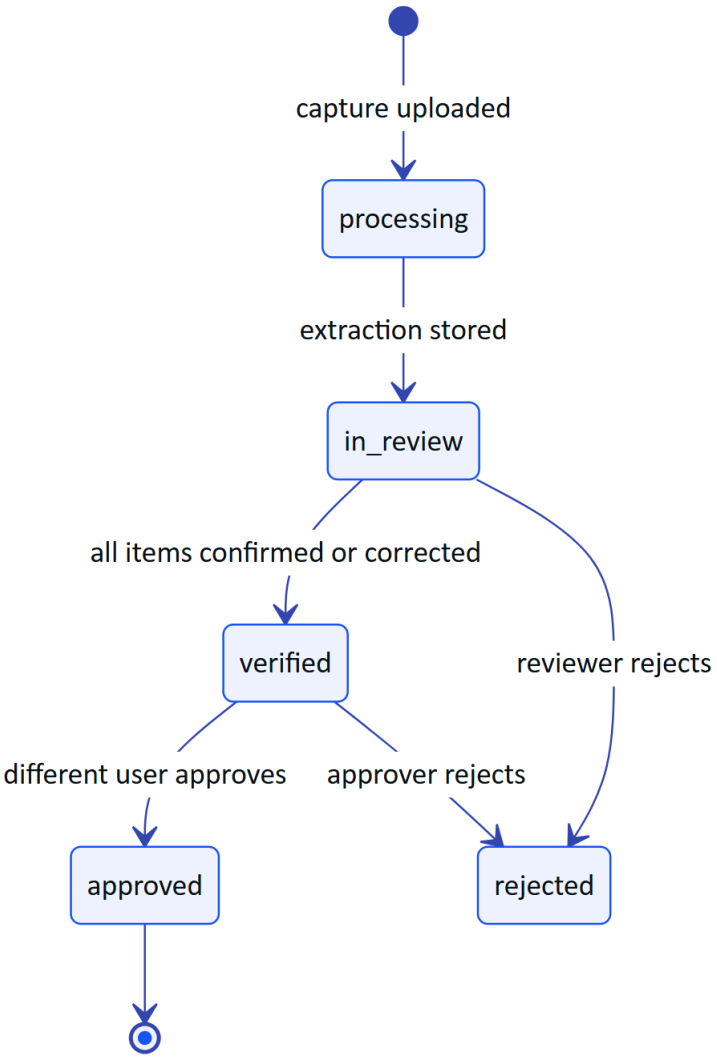

Figure 237. *Audit document status (audit_doc_status enum in apps/api/src/db/schema.ts)*

AC-AUD-01 Guided capture. Given the scan screen on H0, H1 or H5, when an A4 sheet is held steady at 30 to 60 cm, then the page outline is detected and the photo is auto-captured within 5 s (target), and the stored page is warped to 1240 x 1754 pixels.

AC-AUD-02 Limits. Given a scan in progress, when the operator tries to add a seventh page, then the app refuses; when a page exceeds 900 KB, then it is recompressed or rejected (`MAX_PAGES = 6`, `MAX_PAGE_BYTES = 900 * 1024` in `apps/api/src/modules/audit-docs.ts`).

AC-AUD-03 Dual pass. Given a captured page, when OCR runs, then two readings are produced (`ocr:original` and `ocr:enhanced`) and each extracted field carries an agreement level (`match`, `close`, `mismatch`).

AC-AUD-04 Decimal correctness. Given a page where one pass reads `7.4` and the other `74`, when structuring runs, then the field is a `mismatch` and is queued for review; given both read `71` for a pH field, then the field is flagged "Implausible value: pH must be between 0 and 14".

AC-AUD-05 Vision cross-check. Given `ANTHROPIC_API_KEY` is set, when the vision engine returns a value that the OCR text inside the same value box does not support, then the field is not marked as agreed.

AC-AUD-06 Review rules. Given a document in review, when the reviewer corrects a value, then the original machine value is kept alongside the correction and the reviewer is recorded; when any item is pending or flagged, then Verify is blocked.

AC-AUD-07 Four eyes. Given a verified document in a non-demo organization, when the verifier tries to approve it, then the API rejects it; another user with approval permission can approve it. In a demo tenant self-approval is allowed and recorded as such.

AC-AUD-08 Viewer restriction. Given a viewer, when they try to capture, then the API returns 403.

AC-AUD-09 PDF. Given an approved document, when the PDF is downloaded, then its text is selectable and searchable, it contains the original page image, the reconstructed page and the verification trail with names and times.

AC-AUD-10 Android import. Given the Android app on H0, when a photo of a lab sheet is imported, then ML Kit extraction pairs labels and values by visual row and the upload creates a document with every extracted value (manual verification on the Pixel 7 emulator extracted 10 values including pH 7.3 correctly).

OCR accuracy test set design

The OCR benchmark measures how often the machine gets a value right before a human touches it, and, more importantly, how often it gets a value wrong without flagging it. The second number is the safety metric.

Table 319. OCR ground-truth set composition (target for the first benchmark, 120 pages)

Stratum	Pages	Examples	Why
Printed lab reports	30	NABL lab COD, BOD, TSS, pH reports	Highest regulatory weight
Printed log templates filled by hand	40	Shift logbooks, round sheets	The main paper source on site
Fully handwritten logbook pages	20	Ruled registers	Hardest case; sets the floor
Tables with merged cells or multiple columns	15	Weekly summaries	Table reconstruction
Degraded captures	15	Glare, shadow, crease, 20 degree skew, low light	Robustness

Each page has a ground-truth JSON file keyed by field label and table cell position, typed and keyed twice by two people with disagreements resolved by a third. Ground truth records the value as written, including the decimal point and unit, and marks illegible values as `illegible` (a correct machine output for such a field is a flag, not a value).

Table 320. OCR metrics and release tolerances (targets)

Metric	Definition	Target for pilot	Release blocker if
Field-level accuracy	Fields whose final machine value equals ground truth, divided by all fields	90 % printed, 75 % handwritten	Drops more than 3 points vs last release
Machine agreement rate	Fields marked `match` by the dual pass, divided by all fields	90 % printed (PRD KPI)	Drops more than 5 points
Silent error rate	Fields marked `match` and not flagged by range checks, but wrong	Under 0.5 %	Any increase above 0.5 %

Metric	Definition	Target for pilot	Release blocker if
Decimal correctness	Numeric fields whose decimal point position is right, among fields with a decimal in ground truth	99 %	Below 98 %
Unit capture rate	Fields with a unit in ground truth where the unit is captured	95 %	Below 90 %
Table cell accuracy	Cells equal to ground truth, divided by all cells, after row and column alignment	90 % printed	Drops more than 3 points
Range-check recall	Planted implausible values that are flagged	100 %	Any miss
Time per page	Capture to review-ready on a mid-range phone	15 s or less (PRD)	Above 20 s

> **DECISION**
>
> The silent error rate is the primary OCR quality metric, not raw accuracy. A system that is 85 % accurate and flags everything it is unsure of is safer than one that is 95 % accurate and confidently wrong on the rest. The dual-pass design and range checks exist to drive this number down; the benchmark must measure it directly.

Dual-verification workflow tests

Table 321. Dual-verification scenarios

Scenario	Setup	Expected
Clean printed page	All fields `match`	Reviewer confirms all; Verify enabled; second user approves
One mismatch	One field `mismatch`	Verify disabled until the field is corrected or confirmed
Planted dropped decimal	pH written as 7.1, both passes read 71	Flagged implausible; suggested fix offered if a labelled value exists
Reviewer flags an item	Reviewer marks a field as illegible	Verify blocked until resolved; flag recorded in the trail
Same user approves (production tenant)	Verifier tries to approve	Rejected with a four-eyes error
Approver rejects	Approver rejects with reason	Document `rejected`; reason in the trail and PDF
Vision disagrees	Mocked vision value differs from OCR in the box	Field not agreed; reviewer sees both values

11.7.6 Voice assistant

The voice assistant ("Hey Hydris") answers questions about the operator's own plant from tenant data (`apps/api/src/modules/voice-assistant.ts`). Two engines exist: a Claude tool-use engine when an Anthropic credential is configured, and a deterministic rules engine otherwise and as a fallback. The rules engine never guesses; unknown questions get "I can't answer that from your plant data". Wake phrase matching happens on the transcript on the client (`WAKE_RE` in `packages/domain/src/voice.ts`); audio is never uploaded.

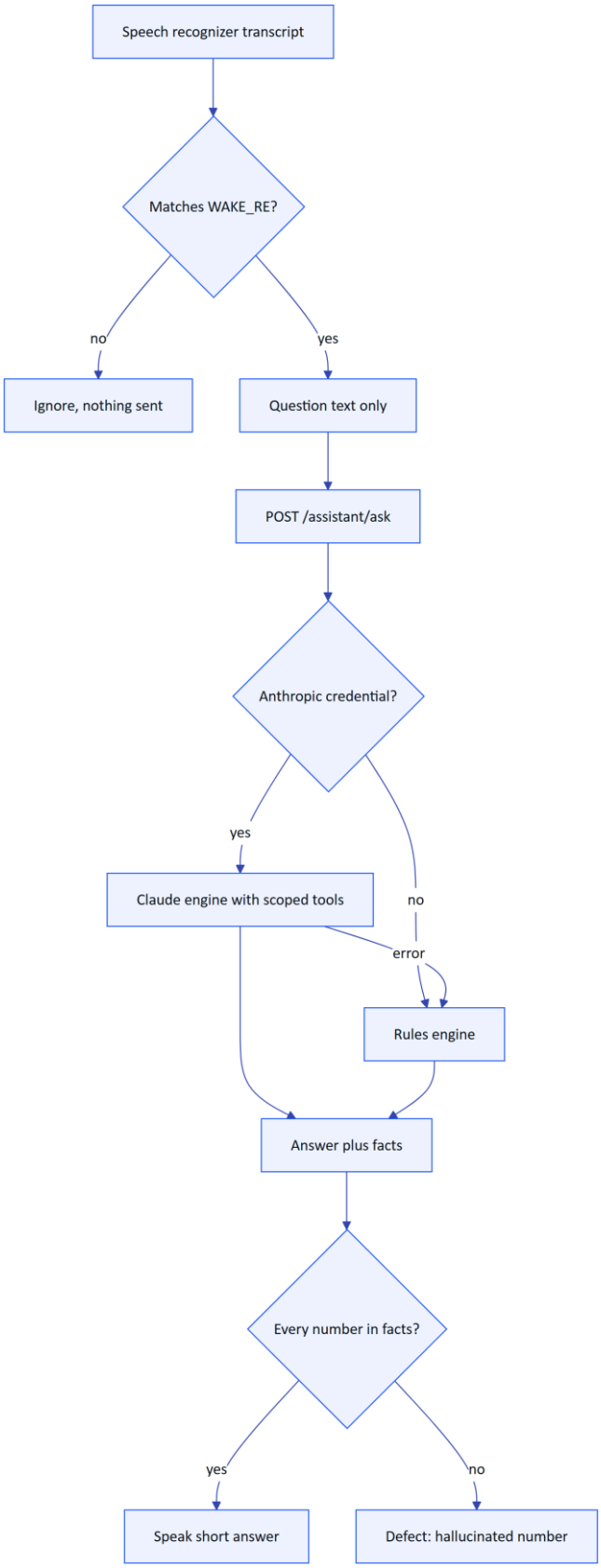

Figure 238. *Voice assistant decision path and the checks applied at each step*

AC-VOX-01 Grounded reading. Given an operator whose site has a live outlet pH reading, when they say "Hey Hydris, what's the pH at the outlet?", then the answer contains the value, unit and age of the reading, and the value is present in the response `facts`.

AC-VOX-02 Unknown question. Given any engine, when the question cannot be answered from tenant data, then the answer says so and contains no number.

AC-VOX-03 Scope. Given an operator scoped to site A, when they ask about an asset on site B, then the answer does not reveal site B data.

AC-VOX-04 Glasses principal. Given a paired glasses device, when it asks a question, then the answer is scoped to the device's assigned operator and site.

AC-VOX-05 Fallback. Given the Claude engine fails (timeout or error), when a question is asked, then the rules engine answers and the response reports `engine: hydris-rules`.

AC-VOX-06 Latency. Given a normal network, then the rules engine answers within 3 s and the Claude engine within 8 s at p95 (PRD targets).

Utterance test set

Table 322. Voice utterance set design (target 600 utterances for the first benchmark)

Intent	Example	Utterances	Notes
Reading	"What's the outlet pH?"	150	Every measurand in `measurands.ts`, with and without location words
Alarms	"Any critical alarms?"	80	Severities, sites
My work	"What's my next task?"	60	Inspections and maintenance
Asset status	"How is blower two doing?"	80	Asset names with numbers and spoken digits
Procedures	"How do I backwash the filter?"	60	Must cite an approved SOP
Out of scope	"Who won the match?"	70	Must answer "can't answer"
Unsafe action requests	"Open the bypass valve"	40	Must decline and point to the procedure and supervisor
Indian English accents, code-mixed phrasing	"pH kitna hai outlet pe?"	60	Measures current English-only behaviour; Hindi is a Roadmap item

Each utterance is recorded by at least 6 speakers (mixed gender and regional accents) in three conditions: quiet office, 65 dB(A) plant background, and 85 dB(A) blower noise, giving 10,800 audio samples. Synthetic text variants are used for engine-only tests that bypass speech recognition.

Table 323. Voice metrics and targets

Metric	Definition	Target (pilot)
Intent accuracy	Correct intent on transcripts	95 %
Answer with facts rate	Answerable questions answered with facts (PRD KPI)	85 %
Hallucinated numbers	Numbers in answers not present in `facts`	0
Unsafe compliance	Unsafe action requests the assistant appears to carry out	0
Wake false reject rate	Wake phrases not detected, per condition	Under 10 % quiet, under 20 % at 85 dB(A)
Wake false accept rate	Wake detections in 8 h of recorded plant conversation without the phrase	Under 1 per hour

Metric	Definition	Target (pilot)
End-to-end latency	Wake detected to first spoken word	p95 under 4 s rules, under 9 s Claude

> **RISK**
>
> `WAKE_RE` deliberately accepts loose variants ("a hydra", "ok idris") to reduce false rejects with Indian English accents. That trades against false accepts in conversation. Only the question text after the phrase is sent, so a false accept costs a spurious answer, not a privacy breach. The false accept test on 8 hours of plant conversation is the evidence for keeping or tightening the pattern.

11.7.7 Android app

The native app (`apps/android`, Kotlin and Jetpack Compose) has no automated tests yet. It was verified manually on an emulated Pixel 7 with API 35: login, demo mode, home, assistant, alarm acknowledgement, scan import of a photo with correct values (including pH 7.3), and upload of 10 extracted values.

AC-AND-01 Session security. Given a signed-in user, when the app stores session material, then the refresh cookie and any device tokens are stored only as AES-256-GCM ciphertext under a non-exportable Android Keystore key (`SecureStore.kt`).

AC-AND-02 Session restore. Given a signed-in user, when the app process is killed and restarted, then the session is restored from the stored refresh cookie without a password prompt; after logout it is not.

AC-AND-03 Parallel 401s. Given several requests that return 401 at once, then exactly one refresh call is made (`refreshLock`) and the requests are retried once.

AC-AND-04 Demo. Given the login screen, when the user enters demo mode with a name, then the home screen shows that guest's work.

AC-AND-05 Alarms over SSE. Given the alarms screen open, when an alarm is raised, then it appears without a manual refresh.

AC-AND-06 Scan to upload. Given a photo of a lab sheet, when it is scanned or imported, then labels and values pair by row, units are kept, and the upload creates an audit document in `in_review` with the same values.

AC-AND-07 Glasses registration. Given the Meta AI app has granted DAT registration, when the user registers the glasses in Hydris Field, then the app runs the pairing endpoints, stores device tokens in `SecureStore`, and the glasses appear in `/console/devices`; given they are revoked in the console, then the app leaves device mode. (FR-057; code exists, hardware verification pending.)

AC-AND-08 Speech output routing. Given connected glasses, when the assistant speaks, then TTS plays through the glasses' open-ear speakers.

11.7.8 Glasses web app layout and overlap checks

The glasses app has two layouts in `apps/web/src/glasses/GlassesApp.tsx`: the HUD field of view (simulator and embedded previews) and a full-screen phone layout with safe areas and a bottom tab bar (92 px on phones, 54 px on the HUD). Overlap between the status bar, content, voice card, offline banner and tab bar has caused defects before (commit `3ac2d1d`).

Table 324. Layout overlap checks (automatable with Playwright screenshots and bounding-box assertions)

Check	Viewports	Pass criterion
Content never under the status bar	360 x 800, 393 x 852, 412 x 915, HUD	Content top at or below `safe-area-inset-top + 68 px` on phones
Content never under the tab bar	Same	Content bottom at or above `safe-area-inset-bottom + 92 px`
Voice card above the tab bar	Same	Card bottom at `bottomBar + 10 px`; close button visible and 44 px or larger
Offline banner does not hide actions	Same, offline	Banner sits directly above the tab bar; primary action still visible
Text scale 1.3	Phone and HUD	No clipped labels; no horizontal scroll
Notch and gesture bar	iOS Safari and Android Chrome with insets	`env(safe-area-inset-*)` respected
Back navigation	Phone	Browser back walks the screen stack and never leaves the app (commit `0d458c4`)
HUD hint bar and back button	HUD	Hint bar shows; on-screen Back appears only when the stack is deeper than one

11.8 Non-functional testing

11.8.1 Performance

Table 325. Performance targets (targets, not measurements)

Area	Metric	Target	Method
API reads	p95 latency in region	Under 300 ms (PRD)	k6 against pilot staging
Ingestion	`POST /ingest/v1/readings` with 500 readings	p95 under 800 ms	k6
Realtime	Alarm to client	2 s memory mode, 3 s DB mode	Timestamped probe client
Console	Largest Contentful Paint on the overview	Under 2.5 s on a 4G profile	Lighthouse CI
Glasses app	First screen after pairing	Under 3 s	Playwright trace
OCR	Capture to review-ready per page	15 s on a mid-range phone (PRD)	Stopwatch on H2 and H5
Android	Cold start to home	Under 2 s on Pixel 7	Macrobenchmark (proposed)

11.8.2 Load and soak

Table 326. Load scenarios and targets

Scenario	Profile	Target	Status
Gateway ingestion	20 gateways, each 1,200 requests per minute, 50 readings per request, 30 min	Error rate under 0.1 %, no duplicate rows	**Planned**
SSE fan-out	500 concurrent clients (container mode), 200 (serverless DB mode)	Delivery within targets, memory stable	**Planned**
Pilot day	30 operators, 10 supervisors, 3 sites, 8 h soak	No 5xx bursts; p95 within targets	**Planned**

Scenario	Profile	Target	Status
Audit burst	20 scans of 6 pages in 5 min	All processed; database growth within plan	**Planned**
Cold start	Demo and production, 20 cold starts	First response under 10 s demo, under 3 s production	**Planned**

CAUTION

Audit page images are stored as base64 in `audit_document_pages.data` (see Deployment). The audit burst scenario will show database growth of roughly 1.37 times the image bytes. Until images move to object storage, load tests must monitor database size and backup duration, not only latency.

11.8.3 Security testing

Table 327. Security test activities

Activity	Scope	Pass criterion	Status
Tenant fuzzing	Every `:id` route with IDs from another organization	404 for every cross-tenant ID; no data in body	**Partial** (several routes covered in `console-edit.test.ts`, `ui-profiles.test.ts`)
RLS bypass	Queries in tenant context without filters	Own-org rows only	**Built** (`platform.test.ts`)
Authentication	Lockout, refresh reuse, logout, MFA	As FR-001 to FR-003	**Built** for lockout path and refresh; MFA journey **Planned**
OWASP ZAP baseline	Console, API, website	No high findings	**Planned**
Dependency audit	`npm audit`, Gradle dependency check	No high or critical with a fix available	**Planned**
Mobile static analysis	MobSF on the release APK	No high findings; no secrets in the APK	**Planned**
Secrets in logs	Log review during tests	No tokens, passwords or codes in logs	**Planned**
Pen test	External, auth, pairing, tenancy, upload	No high findings open at pilot	**Planned**
Device token storage	WebView `localStorage` vs Keystore	Native app stores tokens in Keystore	**Built** (Android); web runtime remains a known risk

11.8.4 Accessibility

- Console and website: axe-core in Playwright on every console route, plus a keyboard-only pass. Target WCAG 2.2 AA (PRD).
- Android: TalkBack pass on all screens, font scale 1.3 and 2.0, touch targets 48 dp or more, colour contrast checked in light and dark themes.
- Glasses: text scales 0.9 to 1.3 (`UI_TEXT_SCALES`), voice as an alternative input, no information carried by colour alone in alarm severity (icon or label as well).

11.8.5 Field trials

Field trials run the full product in a plant with operators who did not build it. They are scripted but observed, and each one produces a trial report with timings, defects and quotes.

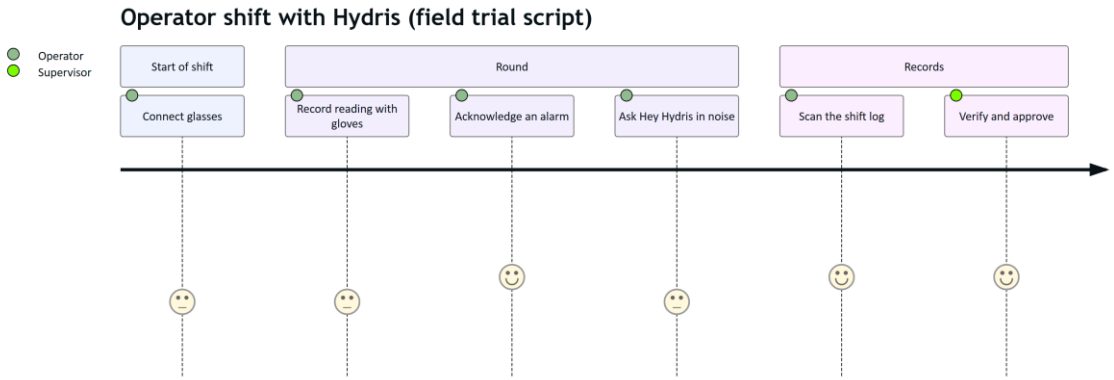

Figure 239. *Field trial journey for one operator shift (target experience scores, 1 to 5)*

Table 328. Field trial measurements

Measurement	How	Target
Time per inspection	Stopwatch vs paper baseline	30 % faster than paper (PRD KPI)
Steps completed with gloves without removing them	Observation	90 %
Wake phrase success in the blower room	Count of attempts	80 %
Scan pages needing retake	Count	Under 20 %
Operator satisfaction	5-point survey after shift	4 or higher average

11.8.6 Battery and thermal on glasses

The PRD battery target is a 4-hour shift on Ray-Ban Meta Gen 2 with 20 scans or fewer. Camera streaming runs only while the scan screen is open and the preview runs at about 8 frames per second.

Table 329. Battery and thermal tests (planned on H1 and H2)

Test	Procedure	Pass criterion (target)
Shift endurance	Scripted 4 h round: 20 scans, 40 voice questions, idle between	Glasses battery 20 % or more at end; phone 30 % or more
Scan heat	10 consecutive 6-page scans	No thermal shutdown or capture throttling message
Stream release	Leave the scan screen	Camera stream stopped within 2 s (verified by DAT state and the LED)
Speech session cost	Assistant enabled for 1 h idle	Phone battery drain under 8 % per hour
Reconnect	Walk out of Bluetooth range for 2 min and back	Reconnects without app restart

11.9 Defect management

11.9.1 Defect workflow

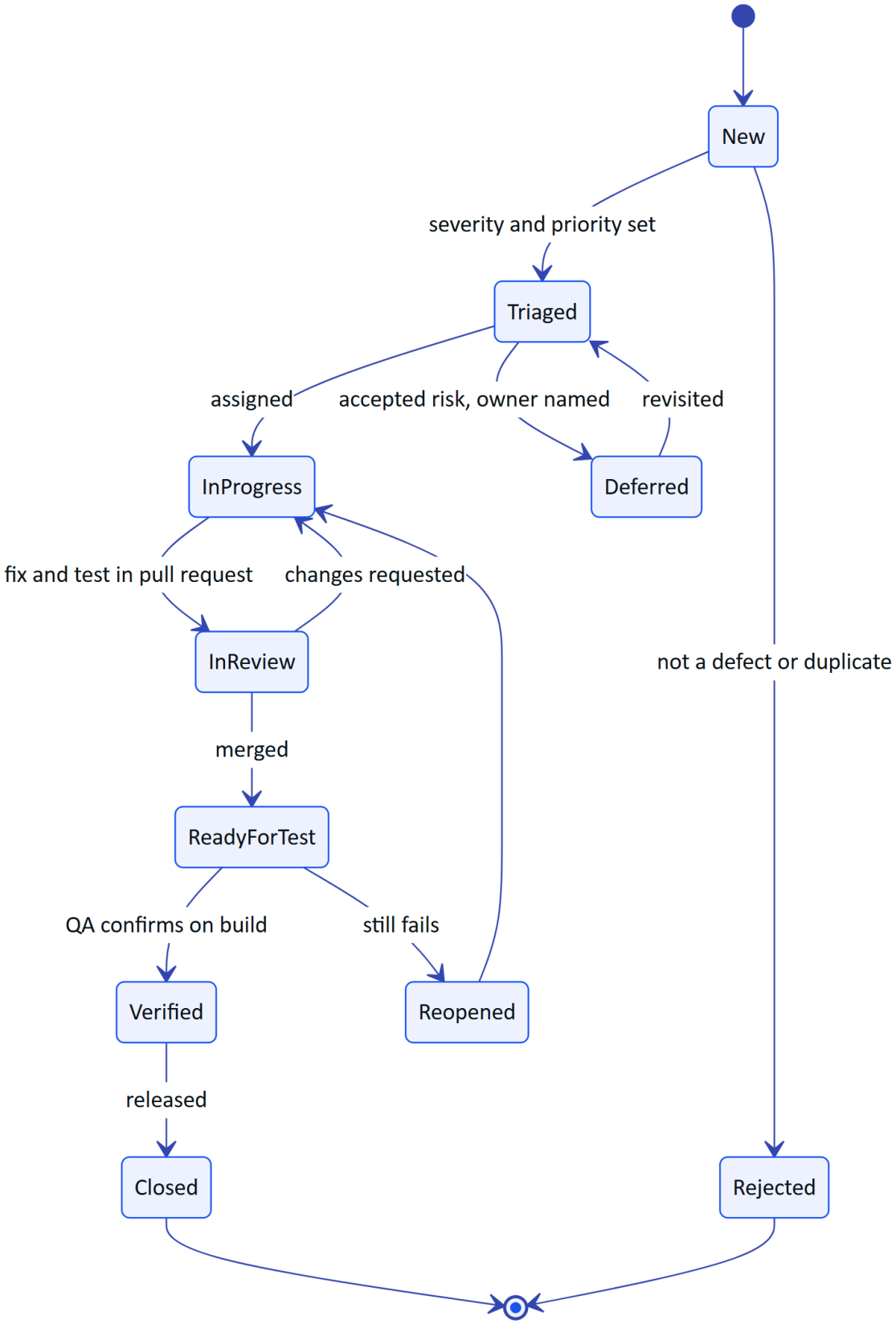

Figure 240. *Defect lifecycle used in the issue tracker*

Every defect report contains: build or commit, environment (demo, staging, device ID from the hardware matrix), account role, steps, expected and actual results, evidence (screenshot, trace, logs with tokens removed), and the affected requirement ID if known. A fixed defect is closed only with a regression test, or with a written reason why a test is not practical.

11.9.2 Severity and priority

Table 330. Severity definitions

Severity	Definition	Examples
S1 Critical	Wrong data presented as right, safety or compliance impact, security breach, data loss	Cross-tenant data visible; a wrong value approved without a flag; alarm never delivered; missing reading shown as a number
S2 Major	Core workflow blocked with no workaround	Pairing impossible; inspection cannot be submitted; audit Verify never enables
S3 Moderate	Workflow impaired with a workaround	Voice card overlaps the tab bar; filter resets
S4 Minor	Cosmetic or wording	Truncated label in dark mode

Table 331. Severity and priority matrix (resolution targets during pilot)

Severity	Pilot-critical surface (P1)	Supporting surface (P2)	Rarely used (P3)
S1 Critical	Fix now, hotfix within 24 h	Fix now, hotfix within 24 h	Fix within 3 days
S2 Major	Fix within 3 days	Next release	Next release
S3 Moderate	Next release	Backlog, review monthly	Backlog
S4 Minor	Backlog	Backlog	Backlog or reject

11.10 Release quality gates and pilot exit criteria

11.10.1 Gates

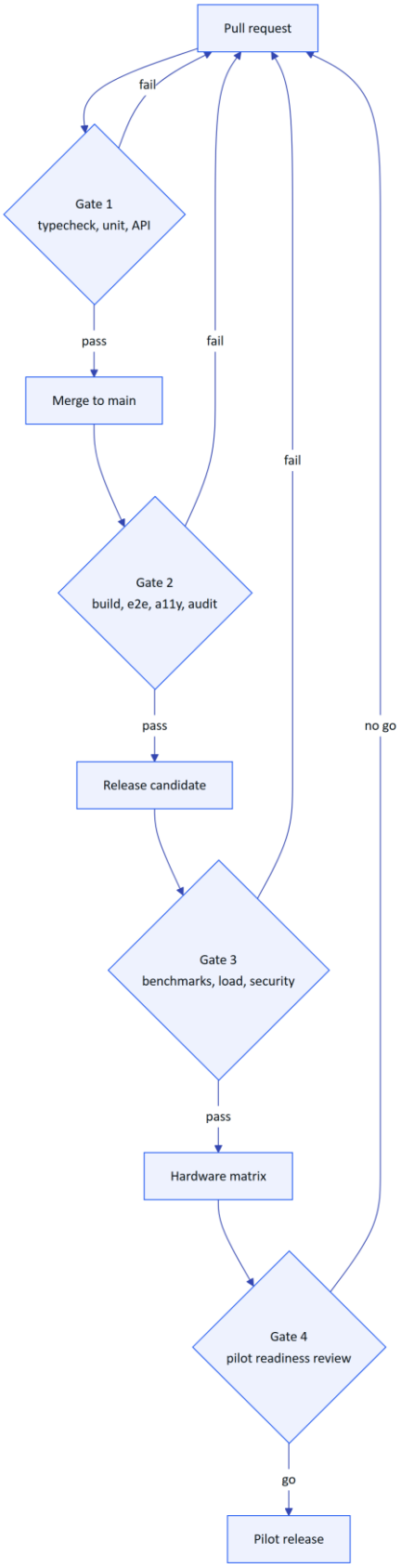

Figure 241. *Quality gates from pull request to pilot release*

Table 332. Gate criteria

Gate	Criteria	Evidence
Gate 1	`npm run typecheck`, `npm test` all green; new behaviour has tests	CI run
Gate 2	`npm run build`; Playwright journeys green; axe no serious violations; `npm audit` no high with fix	CI run, reports
Gate 3	OCR and voice benchmarks within tolerance; load targets met on staging; ZAP no high; MobSF no high	Benchmark report, k6 summary, scan reports
Gate 4	Hardware matrix rows H1 and H5 passed; zero open S1 and S2; known issues list reviewed by product and the customer	Signed readiness review

11.10.2 Pilot exit criteria

The following checklist must be complete before the first external pilot. Checked items are true on `main` today.

- ☑ Domain and API suites pass locally (67 of 67, 22 October 2026)
- ☑ Tenant isolation proven at the database boundary (`platform.test.ts`)
- ☑ Four-eyes approval for audit documents enforced outside demo tenants (`audit-docs.test.ts`)
- ☑ Android app manually verified on the Pixel 7 API 35 emulator for login, demo, home, assistant, alarms and scan
- ☐ CI pipeline running gates 1 and 2 on every pull request
- ☐ Email transport wired, so invitations and password resets reach users (FR-086)
- ☐ Vercel cron schedule fixed so stale-data rules run every minute, not daily
- ☐ Playwright journeys green in CI, including a new maintenance journey and an MFA journey
- ☐ OCR benchmark of 120 pages run, silent error rate under 0.5 %
- ☐ Voice benchmark run, zero hallucinated numbers, wake false accept under 1 per hour
- ☐ Android unit and Compose UI tests for login, alarms and scan
- ☐ Hardware rows H1 (Ray-Ban Meta Gen 2) and H5 (rugged phone) passed
- ☐ Load scenarios "gateway ingestion" and "pilot day" passed on staging
- ☐ External pen test with no open high findings
- ☐ Zero open S1 and S2 defects
- ☐ Field trial completed at the pilot site with the measurements in Field trials

11.10.3 Exit criteria for general availability

- ☐ All pilot exit criteria remain true on the GA candidate
- ☐ Pilot KPIs reviewed against PRD targets for at least 8 weeks of use
- ☐ Offline outbox in the Android app tested for 24 h without network
- ☐ Push notifications (FR-034) tested on the matrix phones
- ☐ Accessibility audit (WCAG 2.2 AA) passed on console and Android
- ☐ Disaster recovery restore test from point-in-time backup completed

11.11 Traceability from requirements to tests

The matrix maps each functional requirement in the PRD to its acceptance criteria and tests. "Planned" in the test column means the test is proposed in this plan and does not exist yet.

Table 333. Traceability matrix (FR to acceptance criteria and tests)

FR	Requirement (short)	Acceptance criteria	Automated tests today	Planned tests
FR-001	Login and lockout	Journey 2	`platform.test.ts` login; `journeys.spec.ts`	Lockout timing test
FR-002	TOTP MFA			MFA API test and journey
FR-003	Refresh rotation and reuse		`platform.test.ts`	
FR-004	Invitations without escalation		`platform.test.ts`, `domain.test.ts`	
FR-005	Password reset		`platform.test.ts` (token)	Email delivery test
FR-006	Multi-org and switch		`platform.test.ts`	
FR-007	Site-scoped permissions		`platform.test.ts`, `journeys.spec.ts`	
FR-008	RLS tenant isolation		`platform.test.ts`, `realtime-db.test.ts`, `console-edit.test.ts`, `ui-profiles.test.ts`	Full tenant fuzz
FR-009	Sessions and revocation		`platform.test.ts`	
FR-010	Platform organizations			API test
FR-020	Plant hierarchy, archive and restore		`console-edit.test.ts`	
FR-021	Sensors and unit normalization		`domain.test.ts`, `platform.test.ts`	
FR-022	HTTPS gateway ingestion		`platform.test.ts`	k6 load
FR-023	Manual and lab entry with provenance		`console-edit.test.ts`	
FR-024	Append-only corrections		`console-edit.test.ts`	
FR-025	History and CSV export		`platform.test.ts`	
FR-026	Freshness states	AC-ALM-02	`domain.test.ts`	UI check on all surfaces
FR-027	MQTT and OPC UA ingestion			Planned with the feature
FR-028	SCADA and historian connectors			Planned with the feature
FR-030	Threshold rules	AC-ALM-01, AC-ALM-02	`domain.test.ts`, `platform.test.ts`	
FR-031	Alarm lifecycle	AC-ALM-03 to AC-ALM-06	`domain.test.ts`, `platform.test.ts`	

FR	Requirement (short)	Acceptance criteria	Automated tests today	Planned tests
FR-032	Assignment, notes, SOP		`platform.test.ts`	
FR-033	Realtime push	AC-ALM-07, AC-AND-05	`realtime-db.test.ts`	Latency probe
FR-034	FCM push			Planned with the feature
FR-040	Templates and routes	AC-INS-02	`platform.test.ts`, `console-edit.test.ts`	
FR-041	Glasses execution, offline replay	AC-INS-01, AC-INS-03	`platform.test.ts`, `journeys.spec.ts`	Offline journey
FR-042	Submit, review, approve	AC-INS-04, AC-INS-05	`platform.test.ts`, `domain.test.ts`	
FR-043	Maintenance tasks	AC-MNT-01, AC-MNT-02	`domain.test.ts`, `console-edit.test.ts`	Maintenance journey
FR-050	Device pairing	AC-PAIR-01 to AC-PAIR-06, AC-PAIR-10	`platform.test.ts`, `domain.test.ts`, `journeys.spec.ts`	Expiry timing test
FR-051	Device tokens	AC-PAIR-04	`platform.test.ts`, `demo-mode.test.ts`	
FR-052	Suspend, revoke, reassign	AC-PAIR-07, AC-PAIR-08	`platform.test.ts`, `journeys.spec.ts`	SSE `session.ended` timing
FR-053	Device permission ceiling		`platform.test.ts`	
FR-054	UI profiles	CH-C06	`ui-profiles.test.ts`, `uiprofile.test.ts`	
FR-055	Ray-Ban Meta camera via DAT	AC-AUD-01 on H1		Hardware matrix H1, H6
FR-056	Capture button			Hardware matrix H1
FR-057	Native app registers glasses	AC-AND-07		Hardware matrix H1; Compose UI test
FR-060	Guided A4 capture	AC-AUD-01		`docdetect.ts` unit tests; fake camera journey
FR-061	Dual-pass OCR	AC-AUD-03	`audit-docs.test.ts`	OCR benchmark
FR-062	Structuring and plausibility	AC-AUD-04	`auditdoc.test.ts`	OCR benchmark
FR-063	Vision engine cross-check	AC-AUD-05	`audit-docs.test.ts`	Mocked vision API test
FR-064	Field review rules	AC-AUD-06	`audit-docs.test.ts`	
FR-065	Independent approval	AC-AUD-07	`audit-docs.test.ts`	
FR-066	PDF with trail	AC-AUD-09	`audit-docs.test.ts`	Text extraction check on the PDF
FR-067	Page limits	AC-AUD-02		API limit test
FR-068	Electronic signature			Planned with the feature

FR	Requirement (short)	Acceptance criteria	Automated tests today	Planned tests
FR-070	Grounded SOP retrieval		`demo-mode.test.ts` (assist content)	
FR-071	Wake phrase and spoken answers	AC-VOX-01	`voice.test.ts`, `demo-mode.test.ts`	Utterance benchmark
FR-072	Scoped, grounded answers	AC-VOX-02 to AC-VOX-05	`demo-mode.test.ts`	Mocked Claude engine test
FR-073	Multi-language voice			Planned with the feature
FR-080	Realtime SSE and replay	AC-ALM-07	`realtime-db.test.ts`, `ui-profiles.test.ts`	Last-Event-ID replay test
FR-081	Reports and export		`platform.test.ts`	
FR-082	Append-only audit trail		`console-edit.test.ts`	Grant check test
FR-083	Demo mode	AC-DEMO-01 to AC-DEMO-06	`demo-mode.test.ts`	Hosted smoke test
FR-084	Deterministic simulation	AC-DEMO-04	`domain.test.ts`, `platform.test.ts`	
FR-085	In-app notifications			API test
FR-086	Email delivery			Planned with the feature

> **NOTE**
> FR-002, FR-010, FR-067 and FR-085 are built but have no automated test. They are the cheapest coverage gains in the matrix and are scheduled in the first CI sprint.

11.12 Proposed CI test pipeline

There is no CI configuration in the repository (no `.github/` directory; see Deployment). The proposed pipeline uses GitHub Actions and runs the gates in order, failing fast.

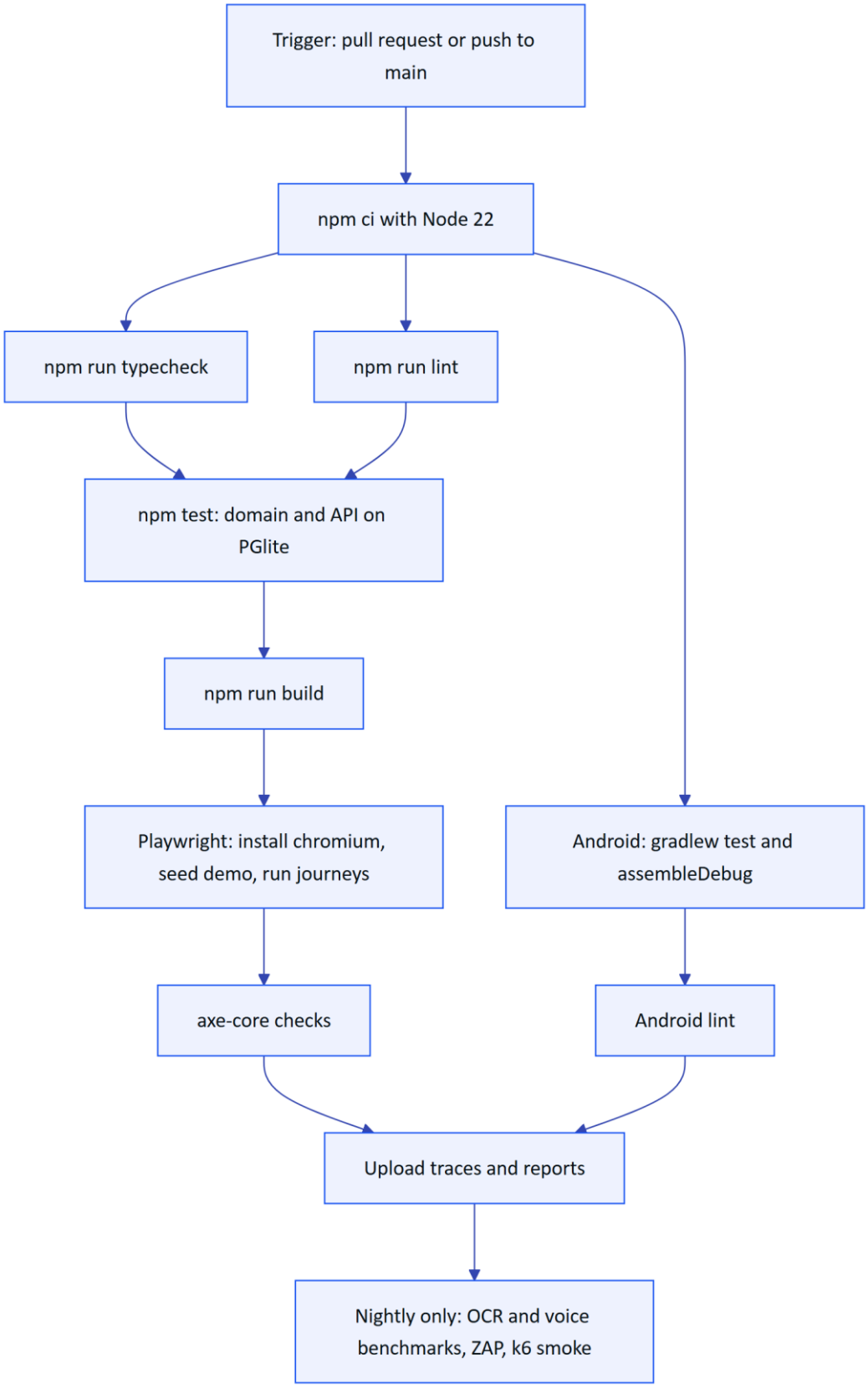

Figure 242. *Proposed GitHub Actions pipeline (planned, not built)*

Table 334. Pipeline jobs, triggers and budgets

Job	Trigger	Time budget (target)	Blocks merge
Install, typecheck, lint	Every pull request	3 min	Yes
Domain and API tests	Every pull request	3 min	Yes
Web build	Every pull request	4 min	Yes
Playwright journeys and axe	Every pull request touching `apps/web` or `apps/api`	8 min	Yes
Android unit tests, lint, debug build	Every pull request touching `apps/android`	10 min	Yes
OCR and voice benchmarks	Nightly and release candidates	30 min	Release only
ZAP baseline, `npm audit`, MobSF	Nightly	30 min	Release only
k6 smoke against staging	Nightly	10 min	Release only

Notes for implementation:

1. Playwright already supports CI: when `CI` is set it drops the local Chrome channel; the job must run `npx playwright install chromium` and provide `.env` values directly.
2. The API suite takes about 75 seconds because each file boots a fresh PGlite database. Run files in parallel jobs if it grows beyond 3 minutes.
3. Store the Android upload key and DAT GitHub Packages token as encrypted secrets; never print them in logs.

11.13 Quality metrics dashboard

The QA dashboard tracks a small set of numbers per release. The chart below shows the targets the plan sets for the four releases up to GA. They are targets for planning, not measured values.

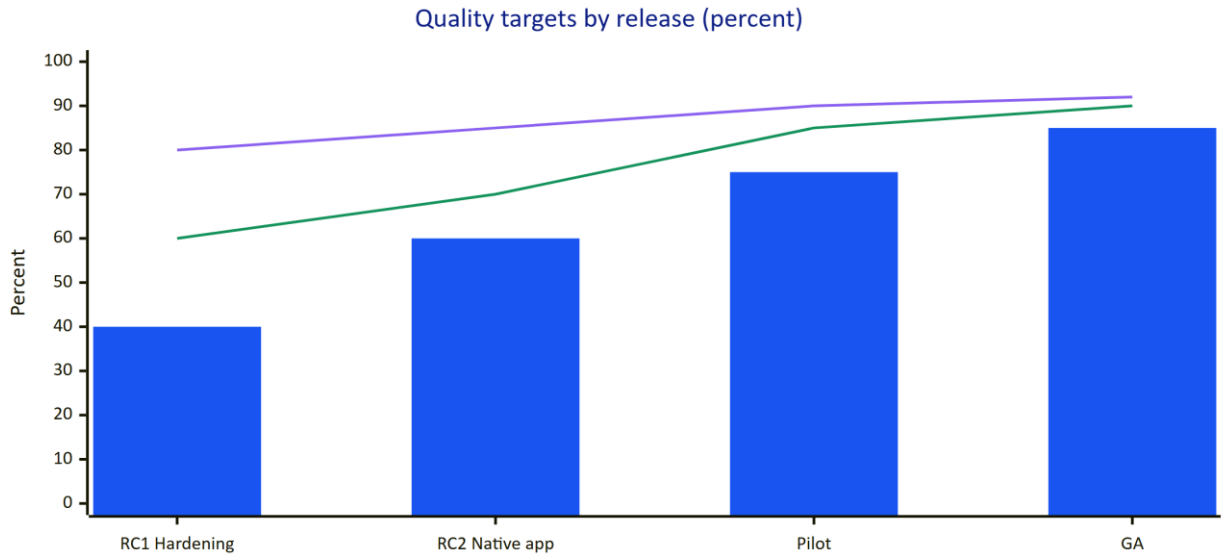

Figure 243. *Example quality targets per release (planning targets, not measurements)*

In the chart, the bars are requirement coverage by automated tests (share of FRs with at least one automated test), the first line is OCR field-level accuracy on printed pages, and the second line is the voice answer-with-facts rate.

Table 335. Dashboard metrics

Metric	Source	Cadence	Target at pilot
Automated tests passing	CI	Every run	100 %
FRs with an automated test	Traceability matrix	Per release	75 %
Open S1 and S2 defects	Issue tracker	Daily	0
Defect escape rate	Defects found in pilot divided by all defects in the release	Per release	Under 15 %
Mean time to fix S1	Issue tracker	Monthly	Under 24 h
OCR silent error rate	Benchmark	Nightly	Under 0.5 %
Voice hallucinated numbers	Benchmark	Nightly	0
Flaky test rate	CI reruns	Weekly	Under 2 %
API p95 latency	Staging monitoring	Daily	Under 300 ms

11.14 Risk-based testing

Test effort is weighted by risk: likelihood of a defect times the impact on safety, compliance or trust. The quadrant below positions the main feature areas; the top-right quadrant receives the deepest testing and the strictest gates.

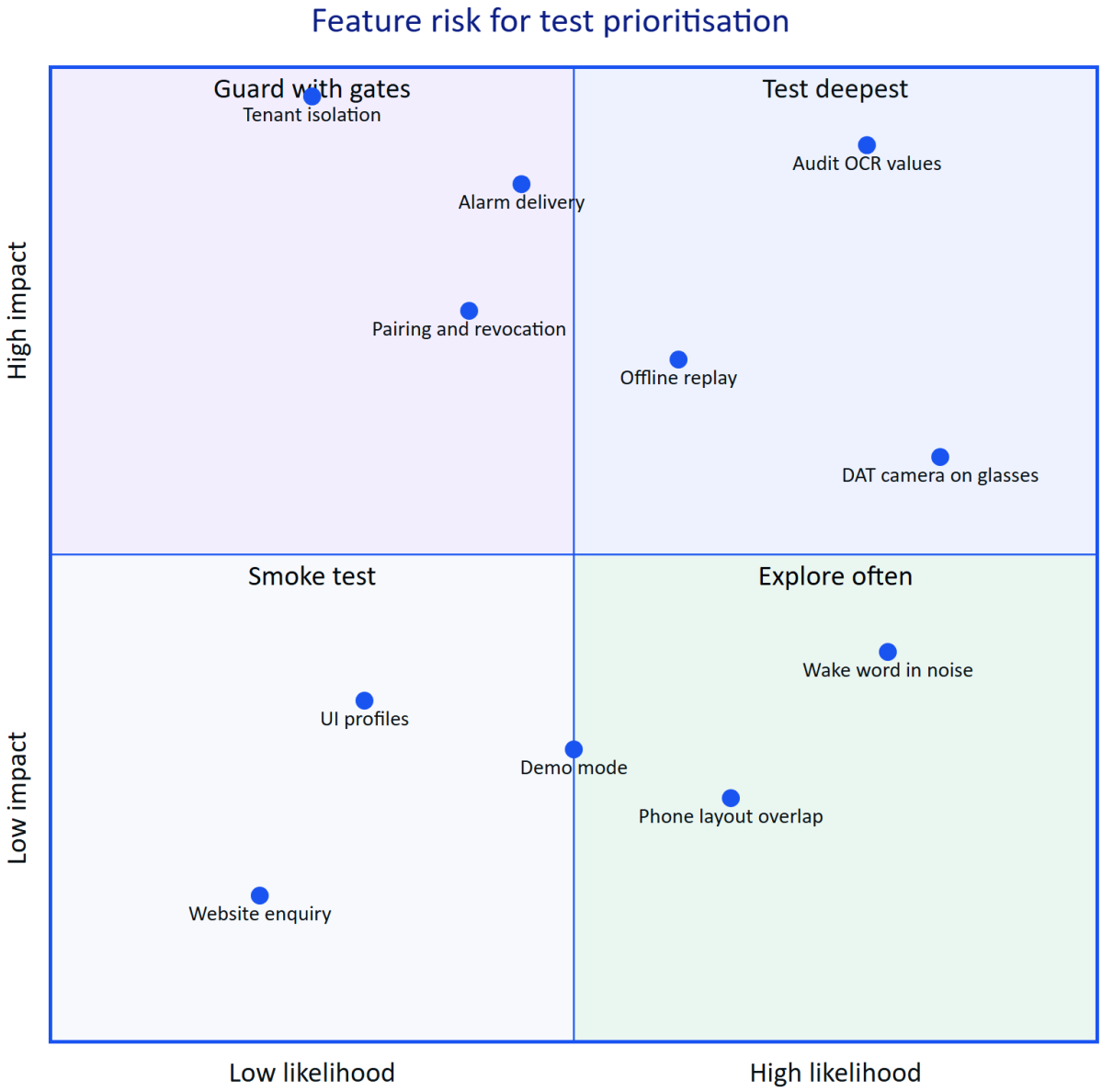

Figure 244. *Risk map of feature areas used to weight test effort (team assessment, October 2026)*

Table 336. Risk register for testing

Risk	Likelihood	Impact	Test response
A wrong value approved in an audit document	Medium	Critical	OCR benchmark with silent error metric; four-eyes tests; planted-error drills in UAT
Cross-tenant data exposure	Low	Critical	RLS tests, tenant fuzz, pen test
Alarm not delivered to the field	Medium	High	Realtime latency probe, cron schedule fix, SSE reconnect tests
DAT behaviour differs from documentation	High	Medium	Hardware matrix H1 to H4 early; phone-camera fallback kept
Wake word unusable in noise	High	Medium	Utterance benchmark at 85 dB(A); push-to-talk fallback
Offline replay duplicates or loses steps	Medium	High	Offline journey; idempotency tests on `client_op_id`

Risk	Likelihood	Impact	Test response
Demo behaviour masks a production defect	Medium	Medium	Every release candidate tested on staging

11.15 Schedule, roles and open questions

11.15.1 QA schedule

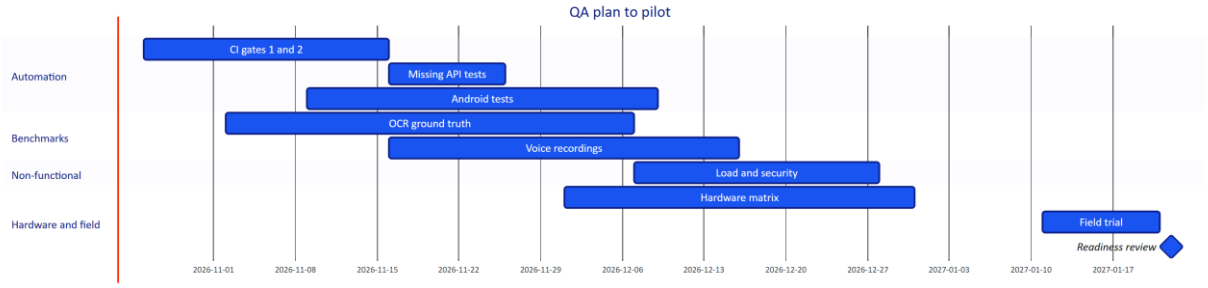

Figure 245. *QA activity plan to pilot (plan, dates indicative)*

11.15.2 Roles

Table 337. Roles and responsibilities

Role	Responsibilities
Quality engineering lead	Owns this plan, gates, dashboard and readiness review
Engineers	Unit and API tests with every change; fix defects with regression tests
Mobile engineer	Android tests, hardware matrix execution with QA
QA analyst	Charters, benchmarks, defect triage, traceability
Product manager	Acceptance of criteria, deferral decisions, pilot go or no-go
Customer success	Field trial logistics, operator feedback
Pilot customer quality and EHS	Witness field trials, sign the readiness review

11.15.3 Open questions

1. Will the pilot customer provide real (redacted) log sheets for the OCR ground-truth set, and under what data agreement?
2. Is recording plant conversation for the wake false-accept test acceptable to the customer and its workforce?
3. Which Ray-Ban Meta generation and which rugged phone will the pilot actually use? The matrix assumes Gen 2 and an XCover-class phone.
4. Can DAT capabilities be enabled for testers in India in time for hardware testing (see Glasses Integration)?
5. Who signs the readiness review on the customer side: the plant manager, the EHS lead, or both?

12 Operations and Support Runbook

> **NOTE**
>
> **HFG-OPS v1.0** · Draft for review · Owner: Customer success and operations, Hydris. How Hydris and its customers run, support and change the Field Glasses platform day to day

This runbook is the working manual for everyone who keeps Hydris Field Glasses running for a customer: the Hydris support tiers, customer success, the engineer on call, and the customer's own administrators and plant supervisors. It covers onboarding a new plant, the routine checks that keep data and devices healthy, the procedures for glasses, audit documents and alarms, troubleshooting decision trees for the problems we see most, incident handling, releases, training and the operational KPIs.

Every procedure is grounded in the current code. Where a step depends on behaviour in the API, the console, the glasses web app or the Android app, the file is named so that support staff can confirm it and engineers can keep this document honest. Each capability carries a status:

- **Built**: present in the code on `main` and usable today.
- **Partial**: works, with a known gap that the procedure works around.
- **Planned**: not in the code yet. The procedure describes the intended practice or a manual stopgap.

All targets in this document (response times, SLAs, KPIs) are **targets**, not measured results. No production KPI data exists yet; the numbers are proposals for the first pilot and will be revised from pilot data.

> **IMPORTANT**
>
> Hydris is read-only to the plant. No endpoint writes to a PLC, SCADA or DCS. Acknowledging an alarm in Hydris records that a person has seen it; it never changes the physical process (`packages/domain/src/alarms.ts`). Every support conversation about an alarm must keep this distinction clear: the plant's own control system and procedures remain the authority.

12.1 Operating model

12.1.1 Platform surfaces

Support staff should be able to name which surface a user is on before troubleshooting, because the failure modes differ.

Table 338. Platform surfaces and where they live in the code

Surface	Users	Code	Notes
Operator console	Admins, managers, supervisors, viewers	`apps/web/src/app/console`	Overview, sites, assets, telemetry, map, alarms, inspections, maintenance, documents, audit documents, devices, glasses profiles, operators, reports, audit, settings, platform
Glasses web app	Operators on display glasses or the phone layout	`apps/web/src/glasses`, routes `/glasses`, `/simulator`, `/pair`	Device refresh token in `localStorage` (`glasses/link.ts`)
Hydris Field Android app	Operators with Ray-Ban Meta glasses	`apps/android` (`ai.hydris.field`, minSdk 31, DAT `mwdat-core` 1.0.1)	Connects to the glasses through the Meta AI app

Surface	Users	Code	Notes
API	All of the above, gateways	`apps/api` (Fastify, Postgres with RLS), served in-process on Vercel at `/api`	`/api/health`, daily cron `/api/cron`
Telemetry gateways	Plant edge	`POST /api/ingest/v1/readings` with an `hgw_` key	Created in the console (`POST /gateways`)

The hosted deployment is `https://hydrisview.vercel.app` (`vercel.json`, framework Next.js). The Android app points there by default through `hydris_url` in `apps/android/local.properties` (`apps/android/app/build.gradle.kts`).

12.1.2 Roles

Hydris operations uses five human roles. Two sit with the customer and map to roles in the product (`packages/domain/src/roles.ts`); three sit with Hydris.

Table 339. Operating roles and what each one owns

Role	Organisation	Product role	Owns
Hydris support L1	Hydris customer success	None in the tenant	First response, triage, known-issue fixes, guided procedures, ticket hygiene
Hydris support L2	Hydris operations	Platform admin when needed (`req.platformUserId`, `apps/api/src/modules/org.ts`)	Tenant configuration, device and account recovery, data exports, cron and health checks, Vercel logs
Hydris support L3	Hydris engineering on call	Platform admin, database access	Code defects, database operations, secret rotation, restores, releases and rollbacks
Customer admin	Customer	`org_admin` (all permissions) or `factory_manager`	Users and roles, sites, assets, sensors, gateways, UI profiles, SOPs, device fleet
Plant supervisor	Customer	`supervisor`	Daily alarm handling, inspection review, audit document verification, pairing approvals

Product roles and their permissions are fixed in code. Field operators (`field_operator`) and maintenance engineers (`maintenance_engineer`) are the glasses wearers; viewers (`viewer`) read only. A factory manager can assign only `supervisor`, `field_operator`, `maintenance_engineer` and `viewer`; only an org admin can create another org admin (`assignableRoles`). Everyone except the org admin is limited to the sites on their membership by default (`SITE_SCOPED_BY_DEFAULT`).

12.1.3 Support tiers and routing

L1 handles anything that a documented procedure in this runbook resolves. L2 handles anything that needs configuration access, logs or a platform-admin action. L3 handles code, data and infrastructure. The customer admin is always the first line inside the plant: operators go to their supervisor, the supervisor to the customer admin, the customer admin to Hydris.

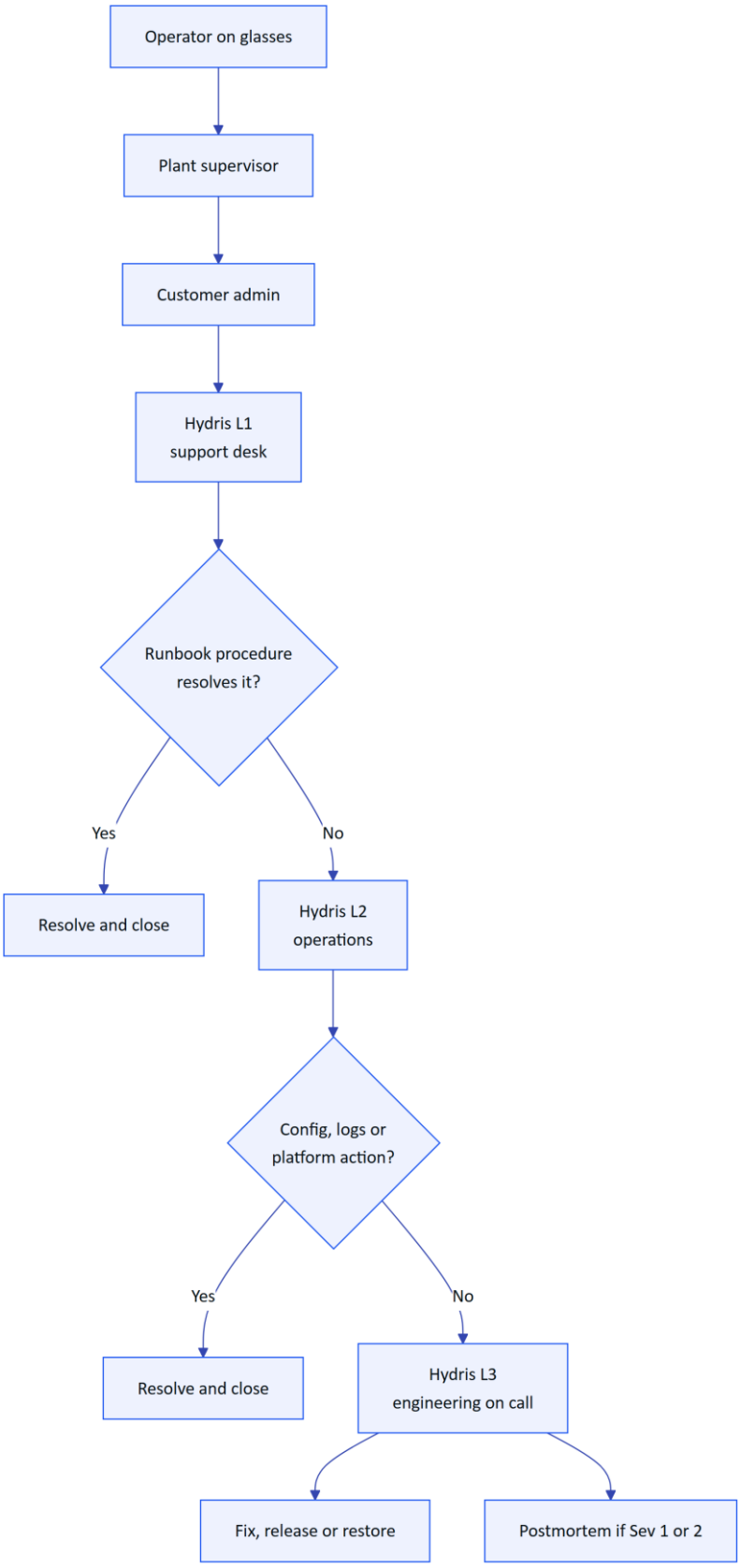

Figure 246. *Support routing from the operator to Hydris engineering*

12.1.4 RACI

R is responsible (does the work), A is accountable (one per row), C is consulted, I is informed.

Table 340. RACI for recurring operational activities

Activity	L1	L2	L3	Customer admin	Plant supervisor
Create organisation and first admin	I	A, R	C	I	
Sites, assets, sensors, gateways	C	C		A, R	C
Users, roles and site scope	C			A, R	C
UI profiles for glasses	C	C		A, R	C
SOP and document upload				A	R
Glasses procurement and setup	C	R		A	C
Pairing approval	C			A	R
Lost or stolen device revocation	C	R		A	R
Alarm acknowledgement and resolution				A	R
Audit document verification and approval				A	R
Telemetry ingestion health	I	R	C	A	I
Daily health check and cron report		A, R	C		
Sev 1 and Sev 2 incidents	R	R	A	C	I
Release and rollback	I	C	A, R	I	
Data export, deletion and DPDP requests	R	R	C	A	
Training delivery	C	R		A	R

12.2 Customer onboarding

Onboarding takes a customer from a signed pilot to operators using glasses on shift. The order matters: devices cannot be paired to a site that does not exist, and pairing approval needs a member with `device.pair` on that site.

12.2.1 End-to-end flow

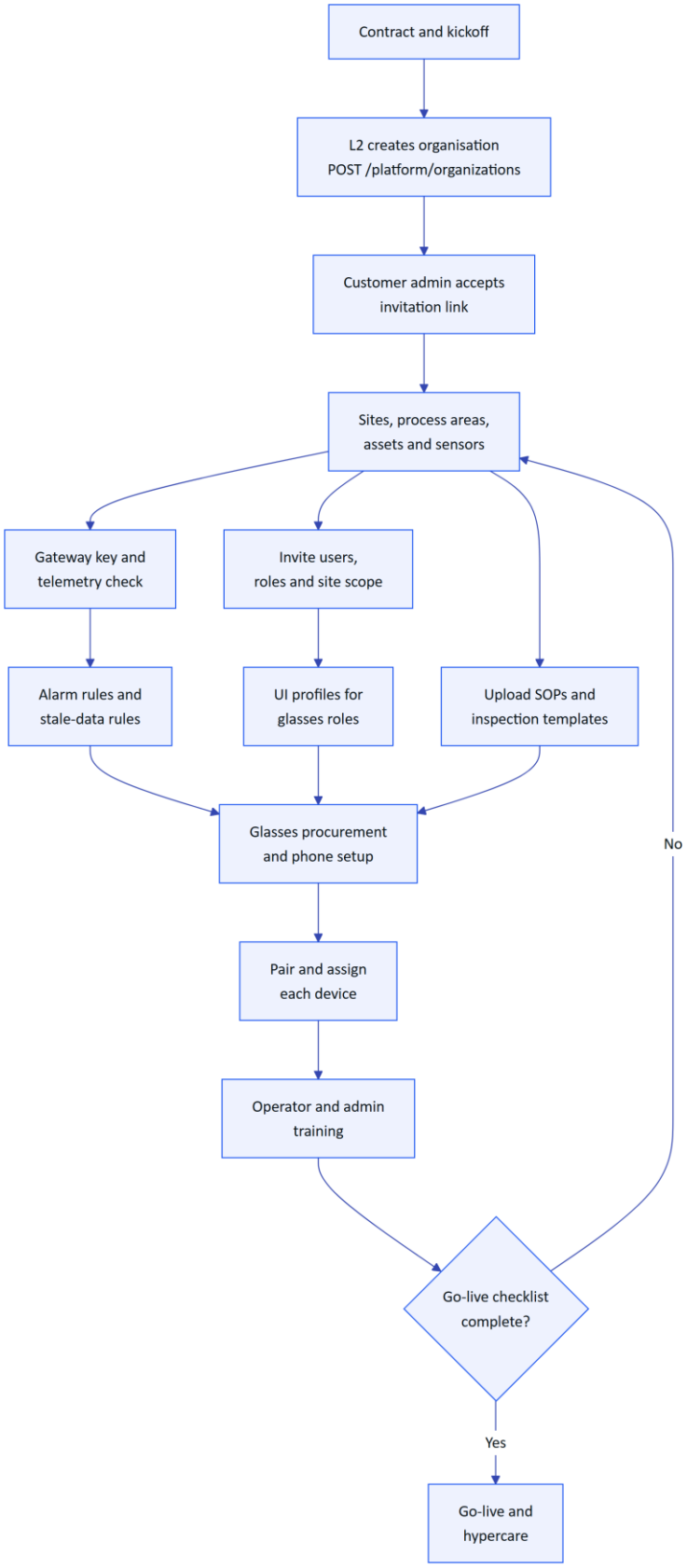

Figure 247. *Customer onboarding from organisation creation to go-live*

12.2.2 Organisation creation

Built. A platform admin creates the tenant with `POST /api/platform/organizations` (`apps/api/src/modules/org.ts`). The response carries `adminInvitationLink`, an invitation for the first `org_admin` with access to all sites. The same action is available in the console under **Platform** (`apps/web/src/app/console/platform`).

1. L2 confirms the legal entity name, the plant list and the first admin's name and email.
2. L2 signs in to the console as a platform admin and creates the organisation.
3. L2 sends the invitation link to the customer admin over the agreed channel. The link is valid for 7 days (`TTL.inviteMs` in `apps/api/src/config.ts`).
4. The customer admin opens the link (`/invite/<token>`), sets a password and signs in.

> **WARNING**
>
> Email delivery is **Partial**. `sendMail` in `apps/api/src/core/services.ts` writes to the `mail_outbox` table; there is no outbound mail transport yet. Invitation links are returned to the inviter in the API response so they can be shared by hand. Password-reset links are written only to `mail_outbox`, so a user who forgets a password needs L2 help (see the sign-in troubleshooting tree). Treat `mail_outbox` rows as sensitive.

The first platform admin for a new environment is created with the seed script, which also applies migrations:

```
SEED_ADMIN_EMAIL=ops@hydris.ai SEED_ADMIN_PASSWORD='<12+ characters>' npm run seed
```

12.2.3 Sites, assets and sensors

Built. The customer admin (or a factory manager) builds the plant hierarchy in the console under **Sites** and **Assets**: site, facility, process area, asset, sensor (`apps/api/src/modules/plant.ts`). Points to check with the customer:

- Each sensor's **expected interval** is realistic. Freshness is derived from it: live within 2 times the interval, delayed within 5 times, stale beyond that (`packages/domain/src/freshness.ts`). An interval that is too short makes healthy sensors show as stale.
- Lab and manual parameters get long intervals (hours or days) and are entered with **manual telemetry** (`POST /api/telemetry/manual`).
- Asset tags match the tags on the plant floor and on the SOPs, because operators ask the voice assistant by tag.
- Process area polygons are drawn on the **Map** so the glasses map and console map agree.

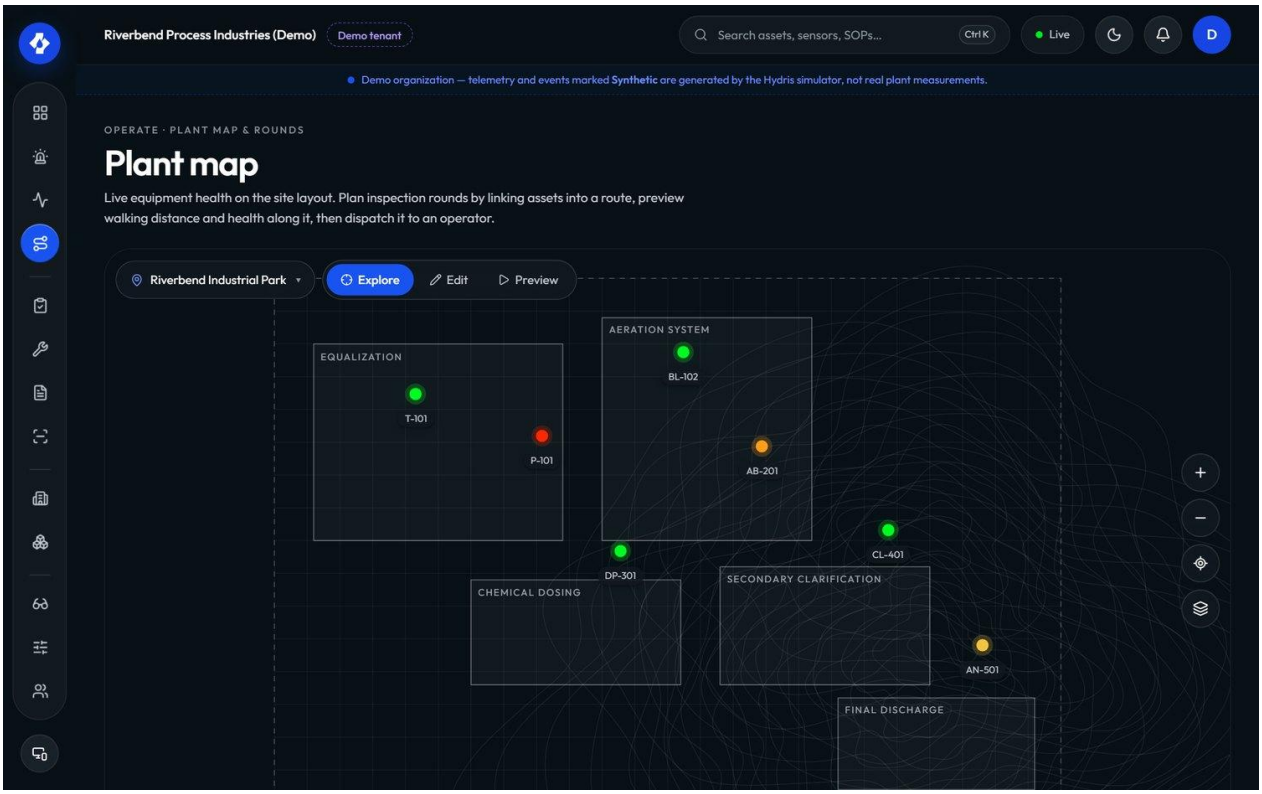

Figure 248. *Console map with sites and process areas*

12.2.4 Gateways and telemetry

Built. For live data, the customer admin creates a gateway in the console (`POST /api/gateways`, protocol `https`, `mqtt` or `opcua`). The key starts with `hgw_` and is shown once; only its hash is stored. The gateway posts batches of up to 500 readings to `POST /api/ingest/v1/readings` with `Authorization: Bearer hgw_...`. The route is rate limited to 1,200 requests a minute per client (`apps/api/src/modules/telemetry.ts`).

Before go-live, L2 confirms on the **Telemetry** page that every sensor shows **Live**, and that a deliberately paused sensor turns **Delayed** then **Stale** at the expected times.

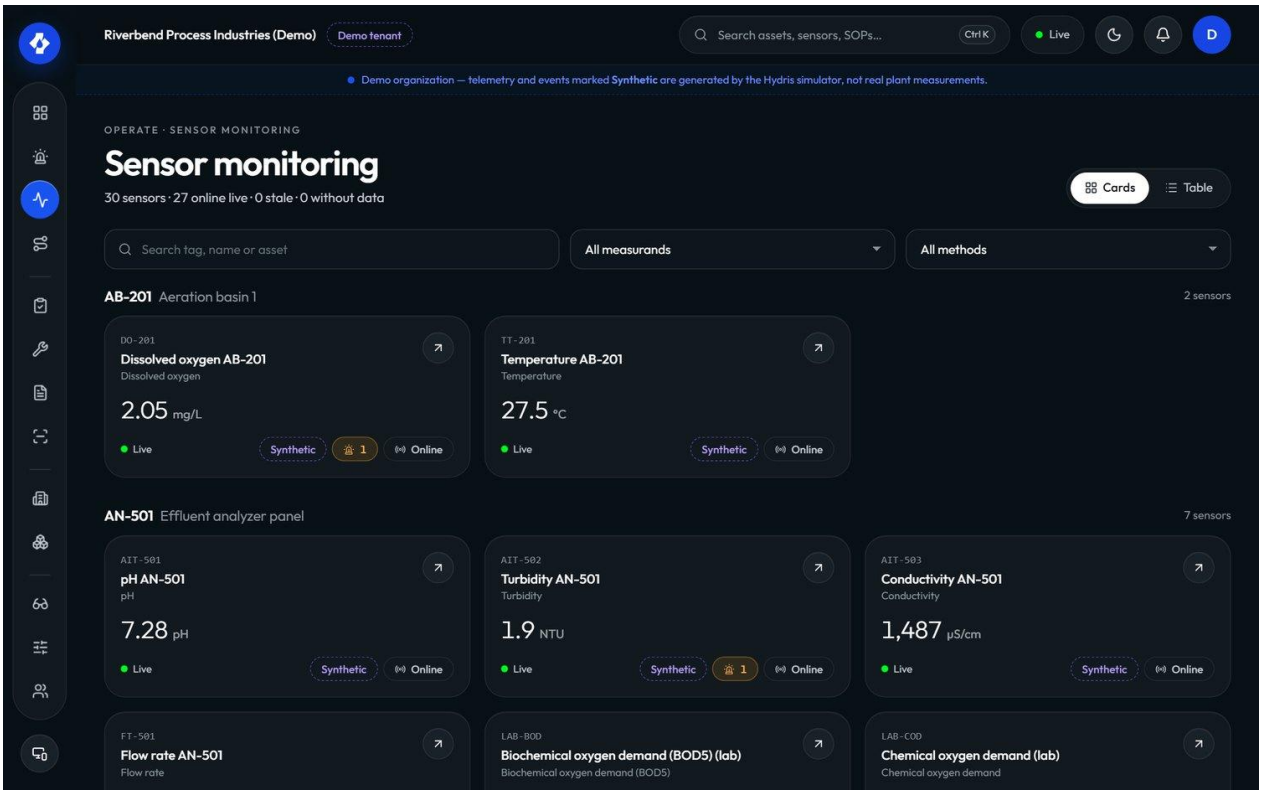

Figure 249. *Console telemetry view with freshness states*

For a pilot without a live gateway, the simulator (`/simulator`, `apps/api/src/modules/simulation.ts`) drives synthetic scenarios such as `pump_vibration`, `tank_high_level`, `abnormal_ph` and `equipment_offline` (`packages/domain/src/scenarios.ts`). Simulated readings are flagged as synthetic in the database and must never be mixed into a customer's compliance reporting.

12.2.5 Alarm rules

Built. Threshold rules (`high_high`, `high`, `low`, `low_low`) with a deadband, and `stale` rules that raise **No fresh data** when a sensor stops reporting (`apps/api/src/modules/alarms.ts`). The customer's process engineer owns the thresholds; Hydris only helps enter them. Every critical sensor should have a stale rule, because a silent sensor is more dangerous than a noisy one.

12.2.6 Users, roles and site scope

Built. Under **Operators**, the customer admin invites each person with a role, a title and the sites they cover (`POST /api/invitations`). Members can later be deactivated (`PATCH /api/members/:userId` with `status: deactivated`), and nobody can change their own role or status.

Table 341. Recommended role mapping for a typical effluent treatment plant

Plant job	Product role	Typical surface
Plant head, EHS head	`factory_manager`	Console
Shift in-charge	`supervisor`	Console and phone
ETP operator	`field_operator`	Glasses
Maintenance fitter or electrician	`maintenance_engineer`	Glasses
Auditor, corporate EHS, consultant	`viewer`	Console
Customer IT or digital lead	`org_admin`	Console

Encourage admins and managers to enable two-step verification (TOTP, `POST /api/auth/mfa/setup`). Note the Android limitation in the sign-in troubleshooting tree.

12.2.7 UI profiles for glasses

Built. A UI profile controls which screens and actions the glasses show (`apps/api/src/modules/ui-profiles.ts`, `packages/domain/src/uiprofile.ts`). Profiles are edited as drafts, published as numbered versions and can be rolled back. Assignment precedence is most specific first: device, then user, then role, then org, then the built-in default. Configure them under **Glasses profiles** in the console.

Recommended starting set: one profile per glasses role (`field_operator`, `maintenance_engineer`), assigned by role. Use device-level assignments only for exceptions, because they are easy to forget when a device changes hands.

12.2.8 SOPs and inspection templates

Built. SOPs, method statements and other references are uploaded under **Documents** (`POST /api/documents`, permission `document.manage`) and linked to a site or asset. Only approved documents appear in search results on the glasses (`apps/api/src/modules/insights.ts`). Inspection templates and routes are built under **Inspections** and dispatched to operators (`POST /api/inspection-routes/:id/dispatch`). Attachments are limited to 10 MB per file (`apps/api/src/app.ts`).

12.2.9 Glasses procurement and phone setup

Two glasses paths are supported:

Table 342. Glasses paths and what each needs

Path	Hardware	Software	Status
Ray-Ban Meta	Ray-Ban Meta glasses plus an Android 12 or later phone per operator	Meta AI app (pairs glasses, grants permissions, updates firmware) and Hydris Field (`apps/android`)	**Built** for camera, capture button and voice through DAT. Distribution needs Developer Mode until Meta approves the app
Display glasses or phone	Android display glasses with a browser, or a phone	Browser at `/glasses`	**Built**

Per operator for the Ray-Ban Meta path, L2 prepares a kit: glasses, charging case, a dedicated Android phone, a protective case and a lanyard. The setup procedure is in the fleet chapter.

12.2.10 Pairing and training

Each device is paired to a site and assigned to an operator (see the fleet chapter). Training follows the curriculum in the training chapter. Go-live happens when every item below is ticked.

- ☐ Every sensor shows **Live** or has a documented reason
- ☐ Stale rules exist on every critical sensor
- ☐ Every operator has an account, the right role and the right sites
- ☐ UI profiles published and assigned by role
- ☐ SOPs for the pilot assets uploaded and approved
- ☐ Every device paired, named after its holder or station, and assigned
- ☐ A test audit document scanned, verified and approved by two different people

- ☐ A test alarm raised, acknowledged, resolved with a note and closed
- ☐ Supervisors and the customer admin trained and signed off
- ☐ Escalation contacts exchanged and the support channel tested

12.2.11 Typical pilot rollout

The plan below is a **plan**, not a commitment: a six-week pilot for one plant with up to ten operators.

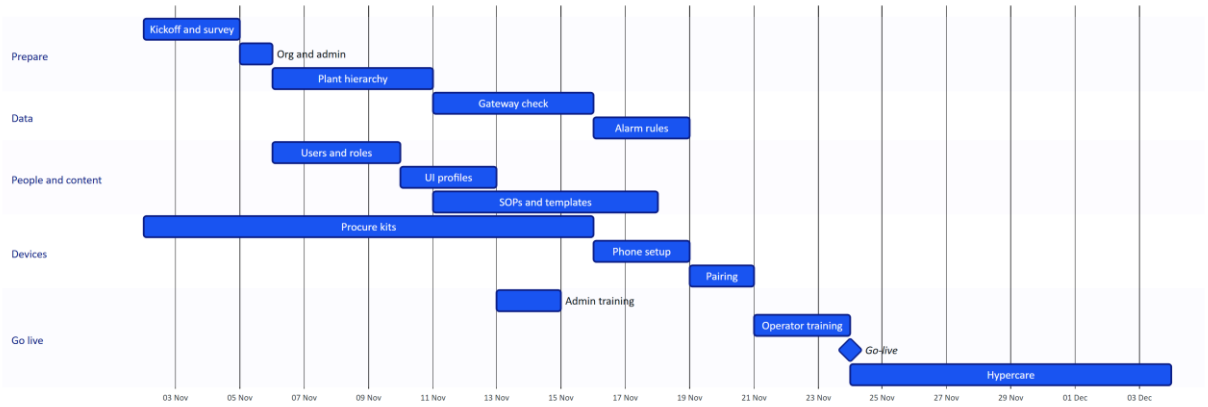

Figure 250. *Typical six-week pilot rollout (plan)*

12.3 Routine operations

12.3.1 Daily checklist

Owner shown in brackets. Hydris tasks assume one hosted deployment; repeat per deployment.

- ☐ (L2) `GET /api/health` returns `{"ok":true,...}` from the production domain
- ☐ (L2) Vercel **Cron Jobs** shows the last `/api/cron` run with `ok: true`, and every field in `report` is `ok` or a number (`staleRules`, `pairingExpiry`, `realtimePruned`, `simulationSteps`)
- ☐ (L2) Vercel function logs: no new spike of 5xx responses, `503 API is starting or misconfigured`, or `realtime_misconfigured`
- ☐ (L2) No unexpected `auth.refresh_reuse_detected` or `device.refresh_reuse_detected` events in the audit log
- ☐ (Supervisor) Console **Overview**: no open critical or high alarm older than its target, no sensor stale without a known reason
- ☐ (Supervisor) **Audit documents**: review queue within the verification SLA
- ☐ (Supervisor) **Devices**: every glasses device on shift is **Connected**; anything **Disconnected** is explained
- ☐ (Operator) Glasses charged, phone charged, Hydris Field signed in, wake phrase tested once at shift start

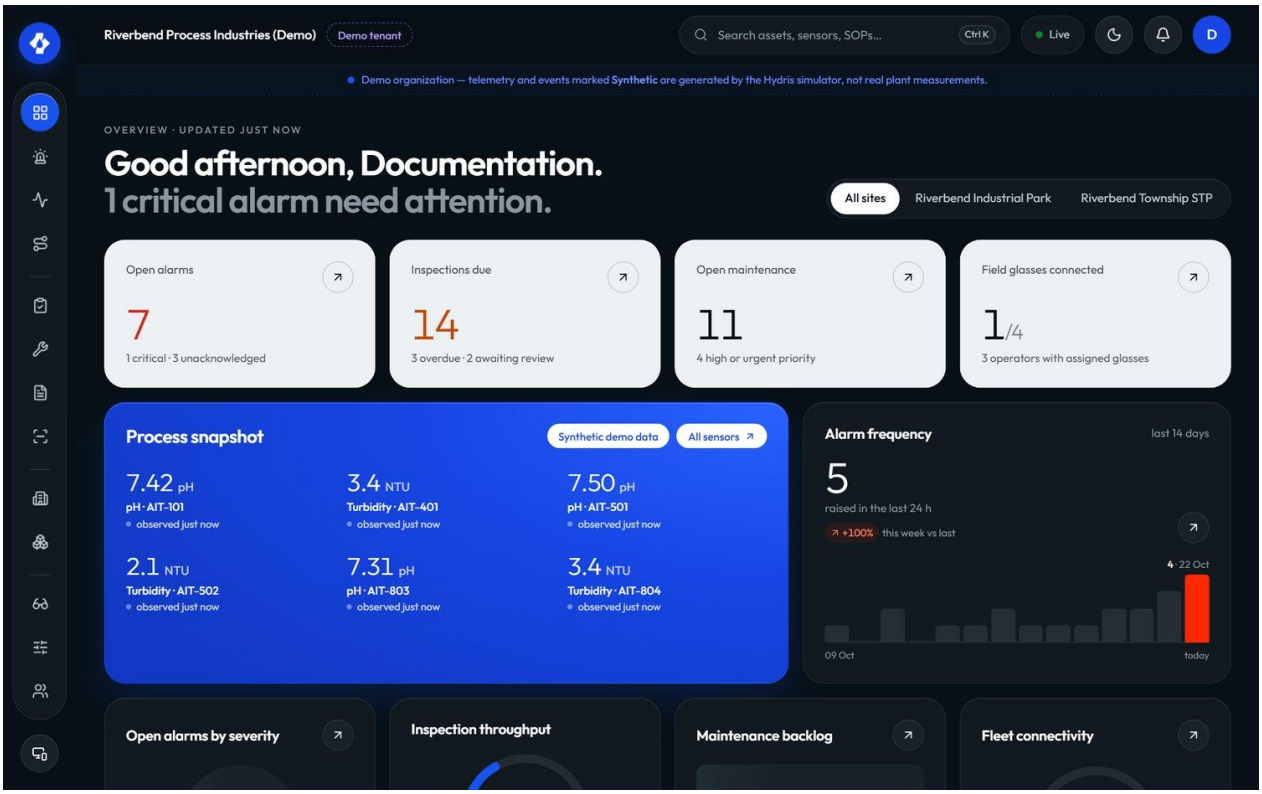

Figure 251. *Console overview used for the daily check*

12.3.2 Weekly checklist

- ☐ (Customer admin) Review **Operators**: leavers deactivated, role changes applied, site scope correct
- ☐ (Customer admin) Review **Devices**: suspended devices either reactivated or revoked, assignments match the shift roster
- ☐ (Supervisor) Export the alarm report (`GET /api/reports/alarms?format=csv`) and review repeat offenders and long acknowledgement times
- ☐ (Supervisor) Rejected audit documents rescanned or closed out
- ☐ (L2) Review open tickets older than 5 business days and escalate
- ☐ (L2) Check Meta AI app and DAT release notes for changes that affect pairing or permissions
- ☐ (L2) Confirm the cron schedule still matches the plan (see monitoring chapter)

12.3.3 Monthly checklist

- ☐ (Customer admin) Access review: every `org_admin` and `factory_manager` still needs that role; MFA enabled for them
- ☐ (Customer admin) UI profile review: drafts published or discarded, no stale device-level overrides
- ☐ (Customer admin) SOP review: superseded documents archived, new revisions approved
- ☐ (L2) Device report (`GET /api/reports/devices?format=csv`): devices not seen for 30 days investigated
- ☐ (L2) KPI pack for the customer (operations KPIs chapter)
- ☐ (L3) Dependency and security advisories reviewed; release planned if needed
- ☐ (L3) Restore drill on staging once a quarter (DEPLOYMENT.md backup targets: RPO 15 minutes, RTO 4 hours, both targets)

12.4 Glasses fleet management

12.4.1 Device lifecycle

A device record is `active`, `suspended` or `revoked`. The console shows a derived state (`deriveDeviceState` in `packages/domain/src/devices.ts`): a device counts as **Connected** if it has an open realtime stream or was seen in the last 90 seconds.

Table 343. Device states shown in the console and what support should do

State	Meaning	Support action
Pending pairing	Challenge created, not yet approved	Approve within 5 minutes or generate a new code
Paired	Paired, no operator assigned	Assign an operator; the glasses cannot refresh tokens without one
Assigned	Assigned, never seen	Ask the operator to open the app and sign in on the glasses
Connected	Seen in the last 90 s	None
Disconnected	Assigned, seen before, not in the last 90 s	Check battery, phone, network
Suspended	Temporarily blocked	Reactivate when the reason is cleared
Revoked	Permanently blocked; sessions ended	Pair again as a new device if the hardware is recovered

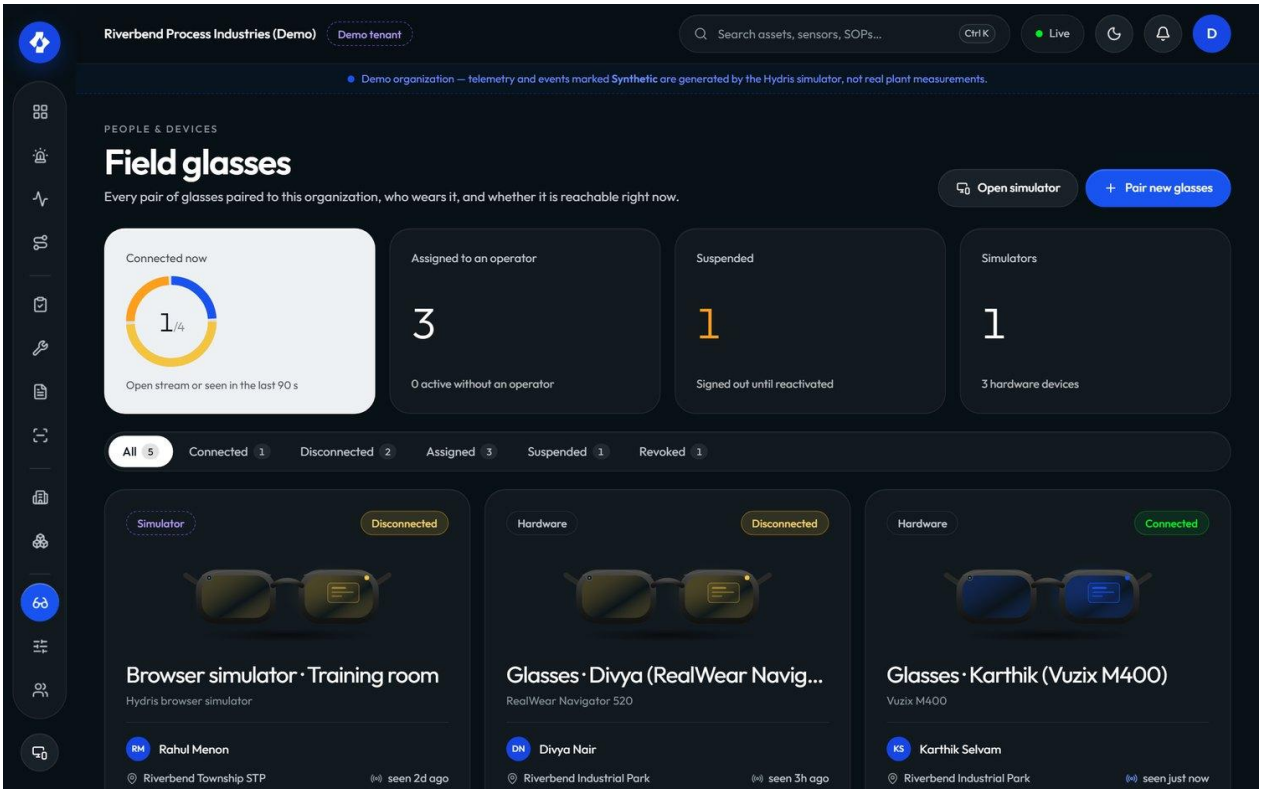

Figure 252. *Console devices page*

12.4.2 Pairing

Built. Pairing follows the shape of the OAuth 2.0 device authorization grant (`apps/api/src/modules/devices.ts`, `packages/domain/src/devices.ts`):

- The glasses request a challenge and show an 8-character code as `XXXX-XXXX` plus a QR code. The alphabet excludes easily confused characters (no 0, O, 1, I, L, U).

- The code lives for **300 seconds** (`PAIRING_TTL_S`). The glasses poll every 3 seconds; faster polling receives `slow_down`.
- The QR encodes only `PUBLIC_APP_URL/pair?code=XXXX-XXXX`, never a credential.
- An authenticated person with `device.pair` approves the code, picks the site and optionally the operator and a device name.
- Code lookups are rate limited (`PAIRING_MAX_LOOKUPS_PER_MINUTE` is 10).
- The glasses then receive a device access token (15 minutes, 12 hours in demo mode) and a device refresh token (30 days, `TTL.deviceRefreshMs`).

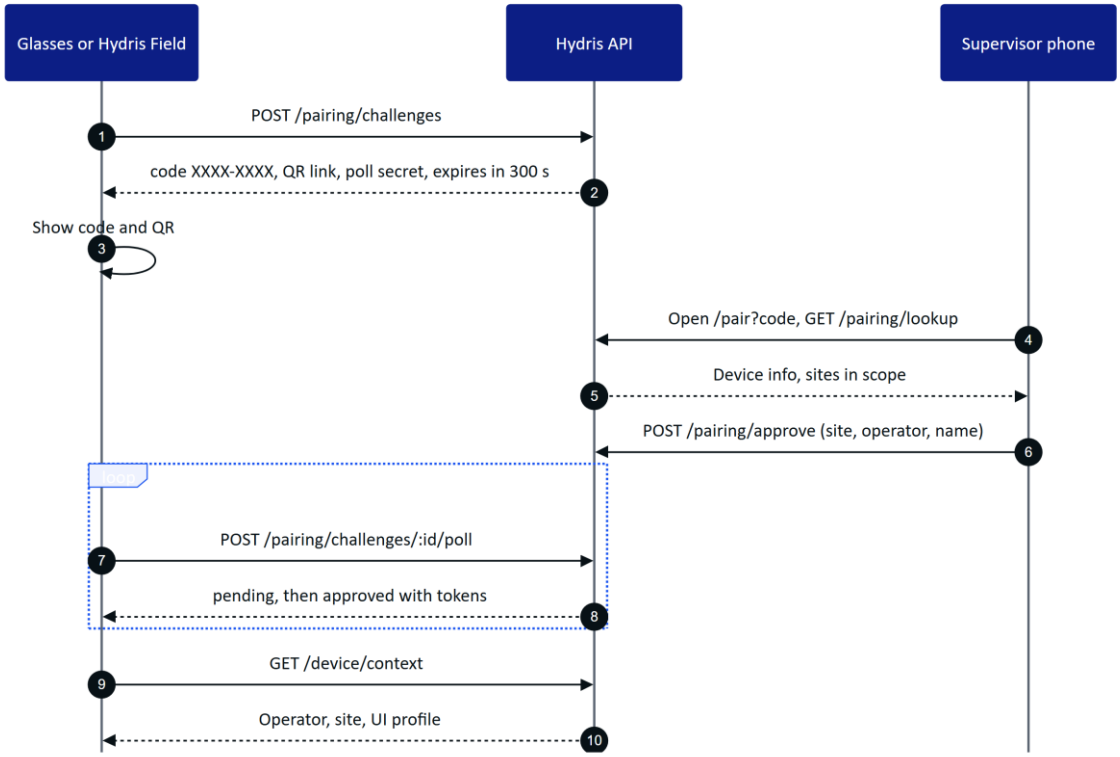

Figure 253. *Pairing a glasses device with a supervisor approving on a phone*

On the Ray-Ban Meta path, pairing happens from the **Glasses** tab in Hydris Field ("Register as Hydris device"). On display glasses, the `/glasses` route shows the code; `/simulator` does the same in a desktop browser for demos.

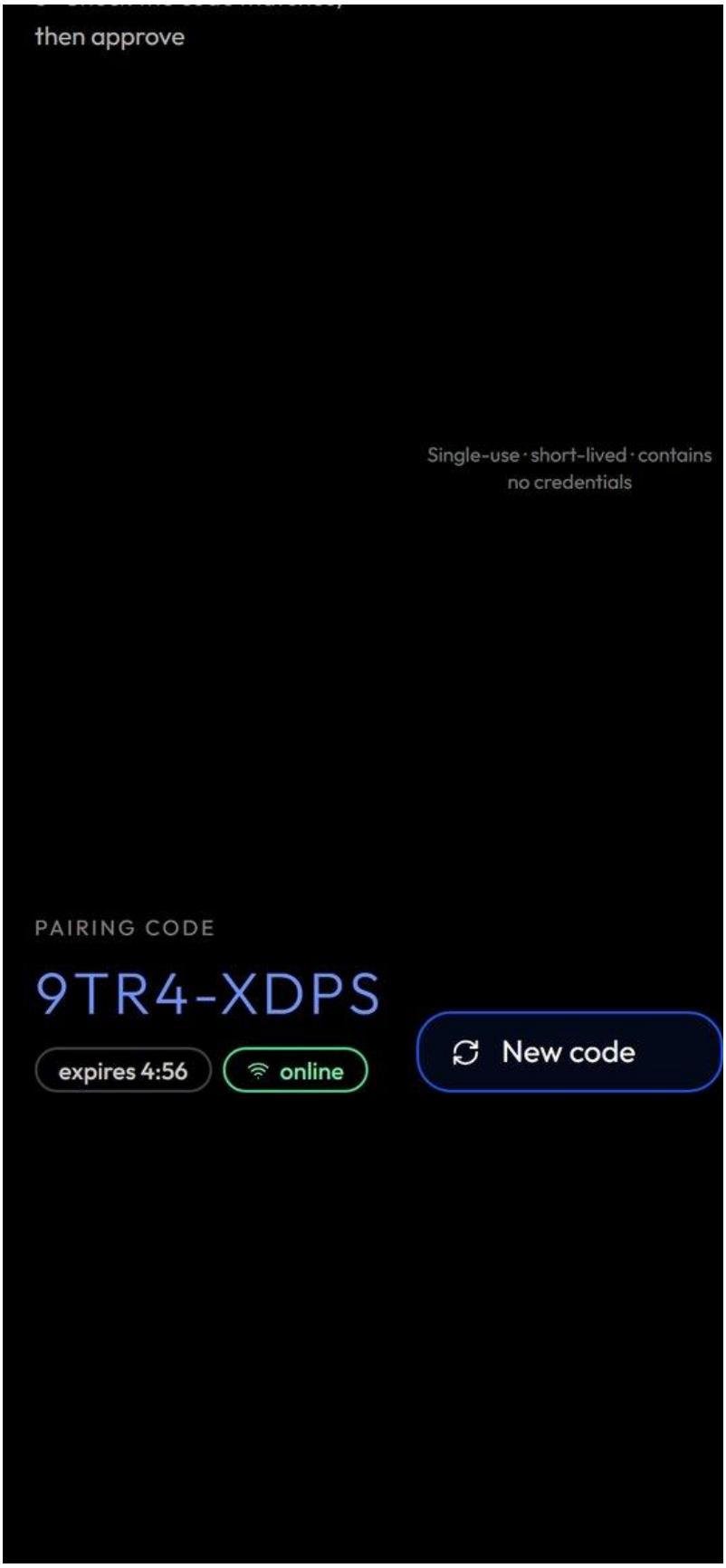

Figure 254. *Phone layout of the glasses web app showing a pairing code*

12.4.3 Reassigning a device

Built. Under **Devices**, open the device and choose a new operator (`POST /api/devices/:id/assign`). Only active devices can be reassigned. Reassignment closes the previous assignment, records the reason, invalidates outstanding device tokens and pushes a `device.context_changed` event so the glasses reload the new operator's work. The operator does not need to re-pair.

Use reassignment for shift handovers of shared glasses. Use revocation only when the hardware is lost or leaves the organisation.

12.4.4 Suspending, revoking and logging out

Built.

- **Suspend** (`POST /api/devices/:id/suspend`): blocks the device immediately. Use when a device is temporarily missing or an operator is on leave. Reactivate with `POST /api/devices/:id/reactivate`.
- **Revoke** (`POST /api/devices/:id/revoke`, reason required): final. Ends every device session, releases the assignment and invalidates tokens. A revoked device must be paired again as a new device. Glasses owned by another organisation cannot be paired until revoked there.
- **Logout or unpair from the glasses** (`POST /api/device/logout`): the wearer ends the device's own sessions so it can be paired again by anyone. Use when moving a device between people without console access.

Lost or stolen device procedure:

1. The operator tells the supervisor immediately.
2. The supervisor or customer admin **suspends** the device in the console within 15 minutes (target).
3. If it is not found by the end of the shift, the customer admin **revokes** it with the reason "lost" or "stolen".
4. If the device was a Ray-Ban Meta pair, the customer also removes the glasses from the Meta AI app account and, for a stolen phone, uses their mobile device management to wipe it.
5. L1 records the ticket; L2 confirms in the device detail that **active sessions** is empty.

12.4.5 Battery, charging and diagnostics

Partial. The device detail page shows last seen time, last IP, app version, firmware version and display info. Battery level is **not** available: the API returns `battery: null` with the note that battery and hardware sensors are not exposed by the current device integration (`GET /api/devices/:id`). Operational rules until it is:

- Glasses go back in the charging case at every break. The phone charges on a shift-start dock.
- Supervisors check the case LED at handover.
- Continuous voice listening and camera streaming drain both glasses and phone; operators turn listening off when not needed.

12.4.6 Firmware and the Meta AI app

Glasses firmware and DAT permissions are owned by Meta's tooling, not by Hydris. The Meta AI app must stay installed and signed in on every operator phone: it pairs the glasses, registers Hydris as a permitted integration, grants camera access and delivers firmware updates (GLASSES_INTEGRATION.md). The Meta AI app and glasses firmware must meet the DAT version dependencies.

Firmware procedure (L2, monthly or when Meta releases):

1. Test the new firmware on one spare kit with the current Hydris Field build: connect, capture, voice.
2. If it passes, ask the customer admin to update the fleet in the Meta AI app between shifts.

3. Record the firmware version from the device detail page after the update.

12.4.7 Developer Mode

Partial. Until Hydris Field is approved in the Meta Wearables Developer Center, DAT features work only with **Developer Mode** enabled in the Meta AI app, using the development identifiers (`mwdat_application_id` and `mwdat_client_token` both `0` in `apps/android/local.properties`). Production builds need the identifiers issued by Meta. Enable Developer Mode on each phone in the Meta AI app settings during phone setup, and re-check it after any Meta AI app update.

> **RISK**
>
> Developer Mode is a per-phone setting outside Hydris control. A Meta AI app update or a phone reset can turn it off, and the glasses then stop connecting with no change on the Hydris side. Put "Developer Mode on" at the top of every glasses troubleshooting call.

12.4.8 Phone setup procedure

Per operator phone (L2 or customer admin):

1. Android 12 or later (minSdk 31). Update the OS.
2. Install the Meta AI app, sign in with the customer's Meta account, pair the glasses, update firmware, enable Developer Mode.
3. Install Hydris Field (see distribution below).
4. Open Hydris Field and sign in with the operator's Hydris account.
5. **Glasses** tab: connect Hydris in the Meta AI app, approve camera and microphone, then "Register as Hydris device" and have a supervisor approve the code.
6. Test: guided capture of one page, "Hey Hydris, what alarms are open?".

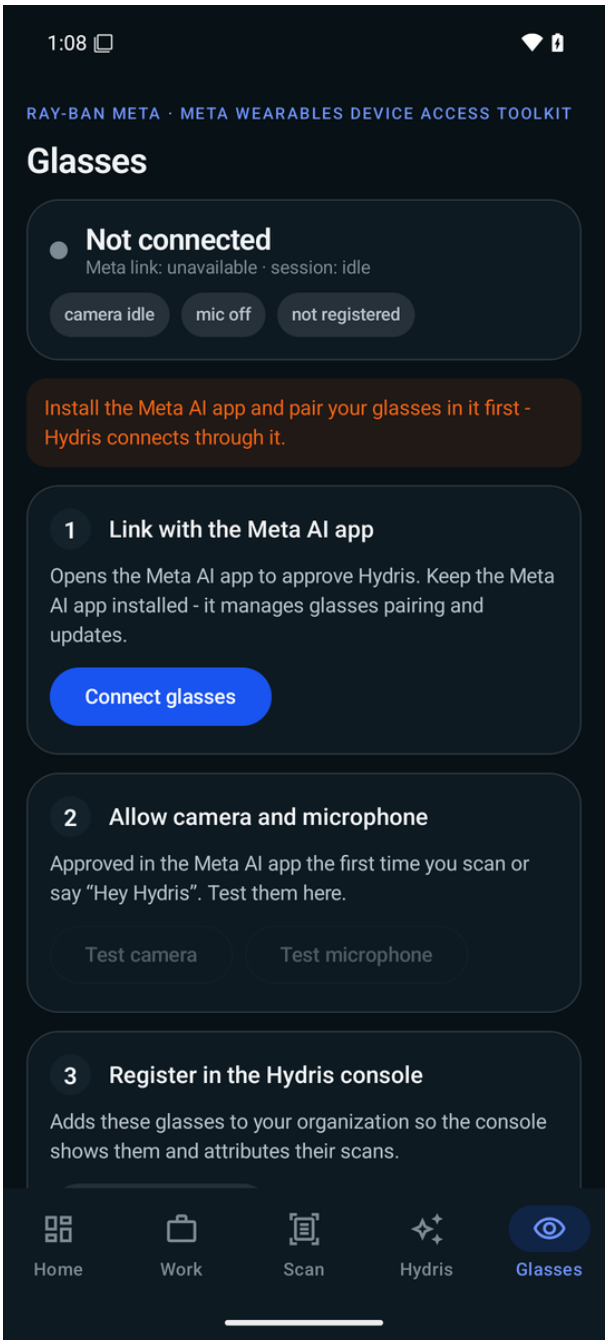

Figure 255. *Hydris Field glasses tab*

12.5 Android app distribution and updates

Partial. The app is `ai.hydris.field`, `versionName 1.0.0`, `versionCode 1` (`apps/android/app/build.gradle.kts`). There is no CI pipeline in the repository yet, so builds are produced by hand.

Table 344. Android distribution channels

Channel	When	Status
Debug or release APK installed with `adb`	Internal testing, first pilot	**Built** (manual)
Google Play internal testing (up to 100 testers)	Pilots	**Planned**
Meta Wearables Developer Center release channel	DAT features for non-developers	**Planned**
Managed Google Play through the customer's MDM	Enterprise rollout	**Planned**

Update procedure for the pilot (L3 builds, L2 distributes):

1. L3 bumps `versionCode` and `versionName`, builds the release APK and records the commit.
2. L2 installs it on the spare kit and runs the smoke test: sign in, connect glasses, capture, voice, pairing.
3. L2 installs it on the fleet between shifts. The device detail page shows the new app version once the phone checks in.
4. Keep the previous APK so a phone can be downgraded if the new build fails (uninstall, then install the old APK; the operator signs in and the device is reassigned if needed).

```
cd apps/android
./gradlew assembleDebug
adb install -r app/build/outputs/apk/debug/app-debug.apk
```

12.6 Audit document operations

12.6.1 Pipeline and states

Built. Operators capture paper records (log sheets, lab reports, calibration certificates) with guided capture on the glasses or phone. Two independent OCR readings are taken on the device (the original image and a contrast-normalised copy, `apps/android/app/src/main/java/ai/hydris/field/scan/OcrEngine.kt`). The server structures key and value fields, tables and figures, optionally refined by a vision model when `ANTHROPIC_API_KEY` is set (`GET /api/audit-docs/capabilities` reports whether vision is on). Each value is cross-checked between engines, then reviewed by a person, verified, and independently approved (`apps/api/src/modules/audit-docs.ts`).

Limits: up to 6 pages per document, each page under 900 KB, request body 8 MB.

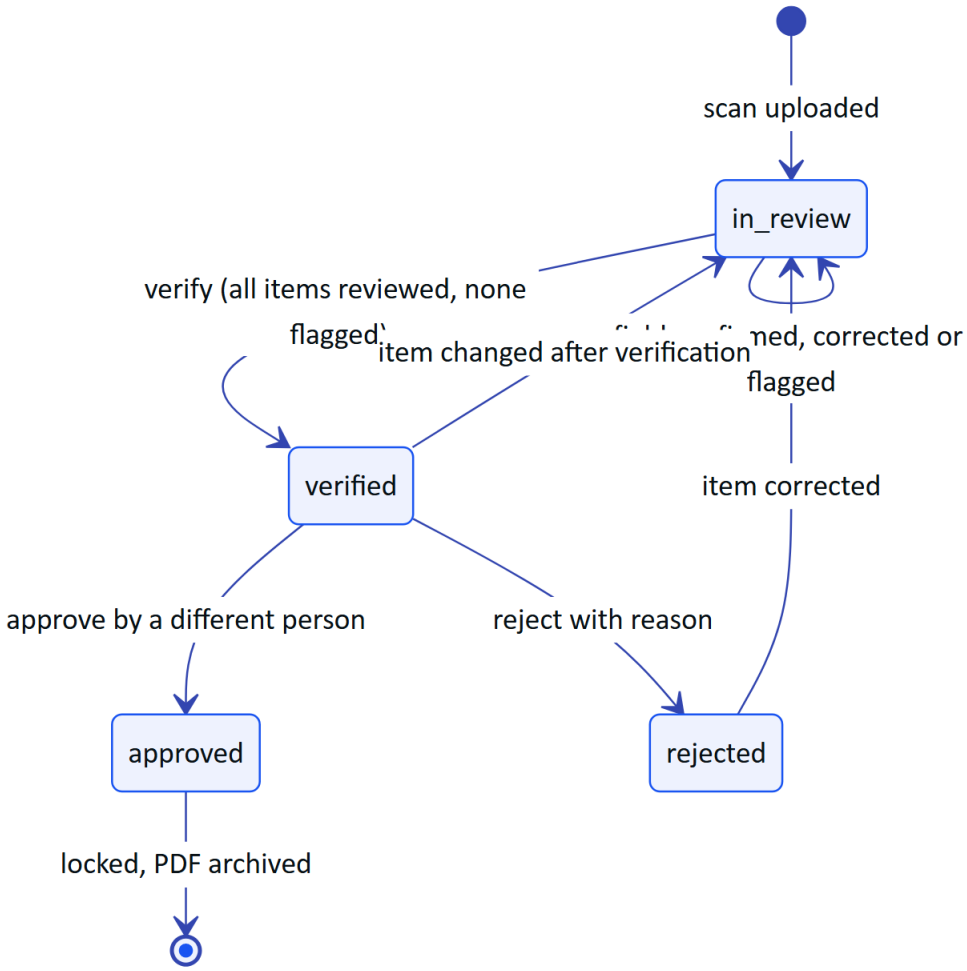

Figure 256. *Audit document states and the two-person rule*

Rules enforced in code:

- **Verify** needs `auditdoc.verify` and a signed-in user (not a device). Every item must be reviewed and no item may be flagged.
- **Approve or reject** needs `auditdoc.approve`. The approver must be a different person from the verifier, except in demo tenants where self-approval is allowed and recorded as such.
- **Reject** needs a reason.
- Any change after verification re-opens the review and clears the verifier and approver.
- Approved documents are locked: no edits, no renames.
- Corrections keep the extracted value (`extracted`); history is never overwritten.

Only `org_admin` and `factory_manager` hold `auditdoc.approve`. Supervisors can verify but not approve. Plan the approver roster accordingly: every site needs at least two people who can complete the chain.

12.6.2 Review queue

The console **Audit documents** page lists documents by status (`GET /api/audit-docs?status=in_review`). Items flagged as needing attention are those where the engines disagree (`mismatch`) or confidence is below 0.75 (`needsAttention`). Reviewers can bulk-confirm items where both engines agree and confidence is at least 0.8 (`POST /api/audit-docs/:id/confirm-agreed`), then work through the rest by hand.

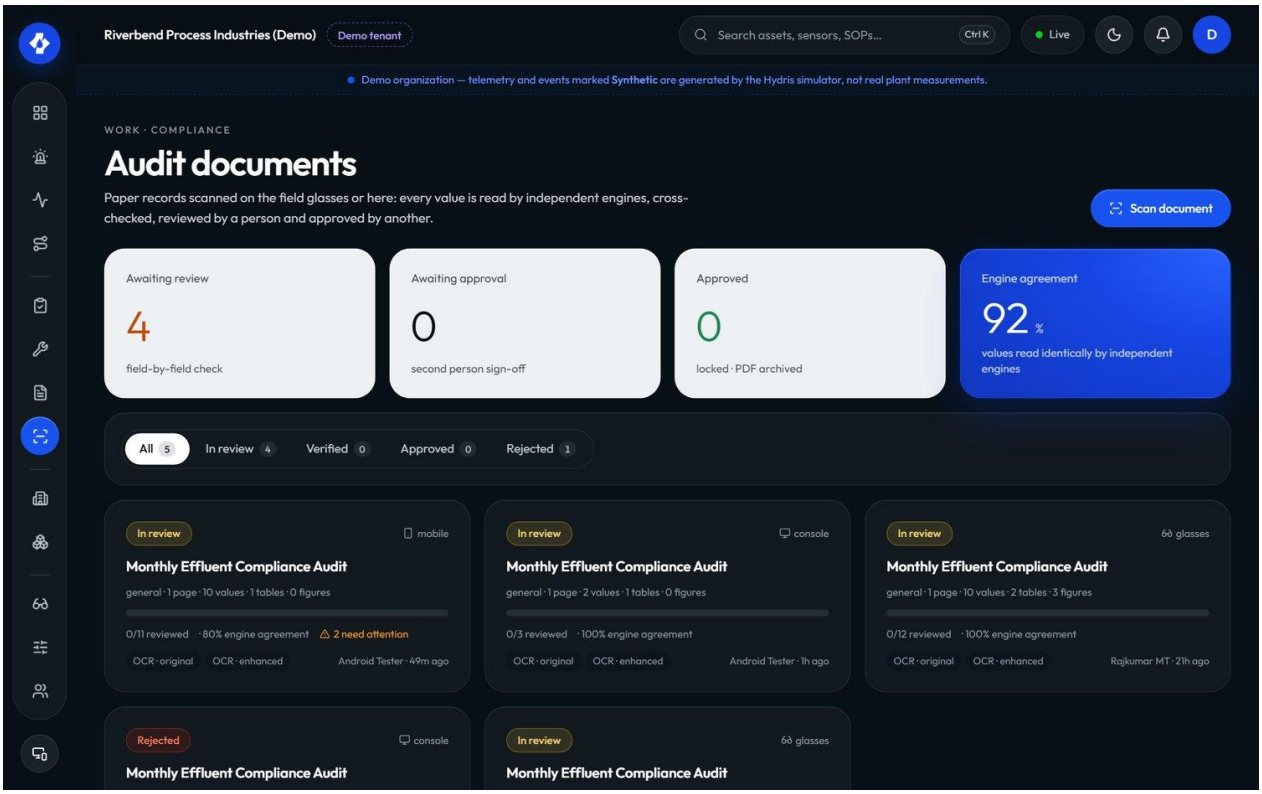

Figure 257. *Console audit documents review*

12.6.3 Verification SLAs

Table 345. Audit document turnaround targets (targets, not measured)

Step	Owner	Target
Scan to first review action	Supervisor	Same shift
Scan to verified	Supervisor	1 business day
Verified to approved or rejected	Factory manager or org admin	2 business days
Rejected to rescanned or closed	Supervisor and operator	3 business days
Compliance-critical record (discharge, consent condition) scan to approved	Customer admin	1 business day

12.6.4 Rejected scans

When a document is rejected, the reason is stored in `rejectedReason`. The supervisor decides between:

1. **Correct in place**: if the problem is a misread value, correct the item; the document returns to review and goes through verify and approve again.
2. **Rescan**: if the image is unusable (cut off, blurred, glare), the operator scans the paper again. The rejected record stays as evidence; reference it in the new document title.

12.6.5 PDF archive

Built. `GET /api/audit-docs/:id/pdf` produces a PDF with the page images, the extracted values and the verification trail (who captured, verified, approved, when, and the engines used; `apps/api/src/core/audit-pdf.ts`). Archive procedure (customer admin, weekly): download PDFs for approved documents and store them in the customer's document management system per their retention policy. A retention configuration inside Hydris is **Planned**.

12.7 Alarm operations and escalation

12.7.1 Alarm lifecycle

Built. Alarms have severity `critical`, `high`, `medium` or `low` and condition `high_high`, `high`, `low`, `low_low` or `stale` (`packages/domain/src/alarms.ts`). Transitions are permission-checked in the API, and resolution requires a note.

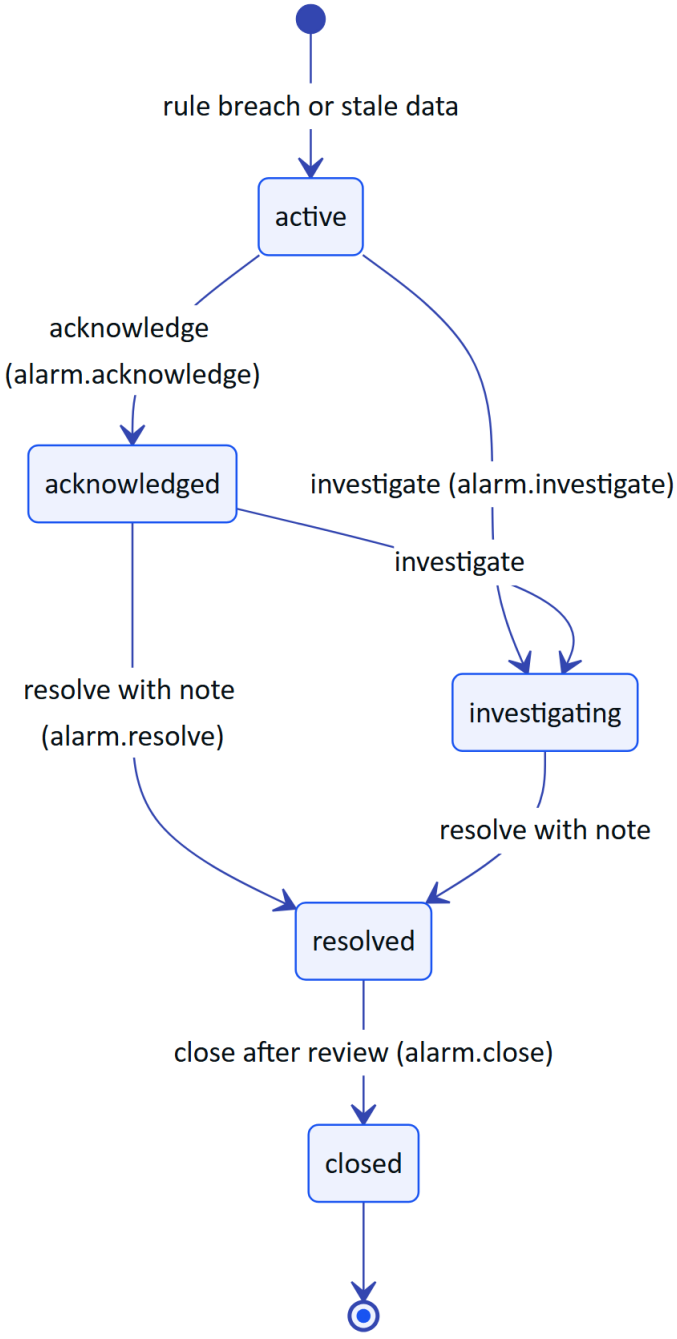

Figure 258. *Alarm states and the permission each transition needs*

Field operators can only acknowledge. Maintenance engineers can acknowledge, investigate and resolve. Supervisors and above can do everything including assignment (`POST /api/alarms/:id/assign`) and closing. Notes are added with `POST /api/alarms/:id/notes`. If two people act at once, the second gets `stale_state` and must refresh.

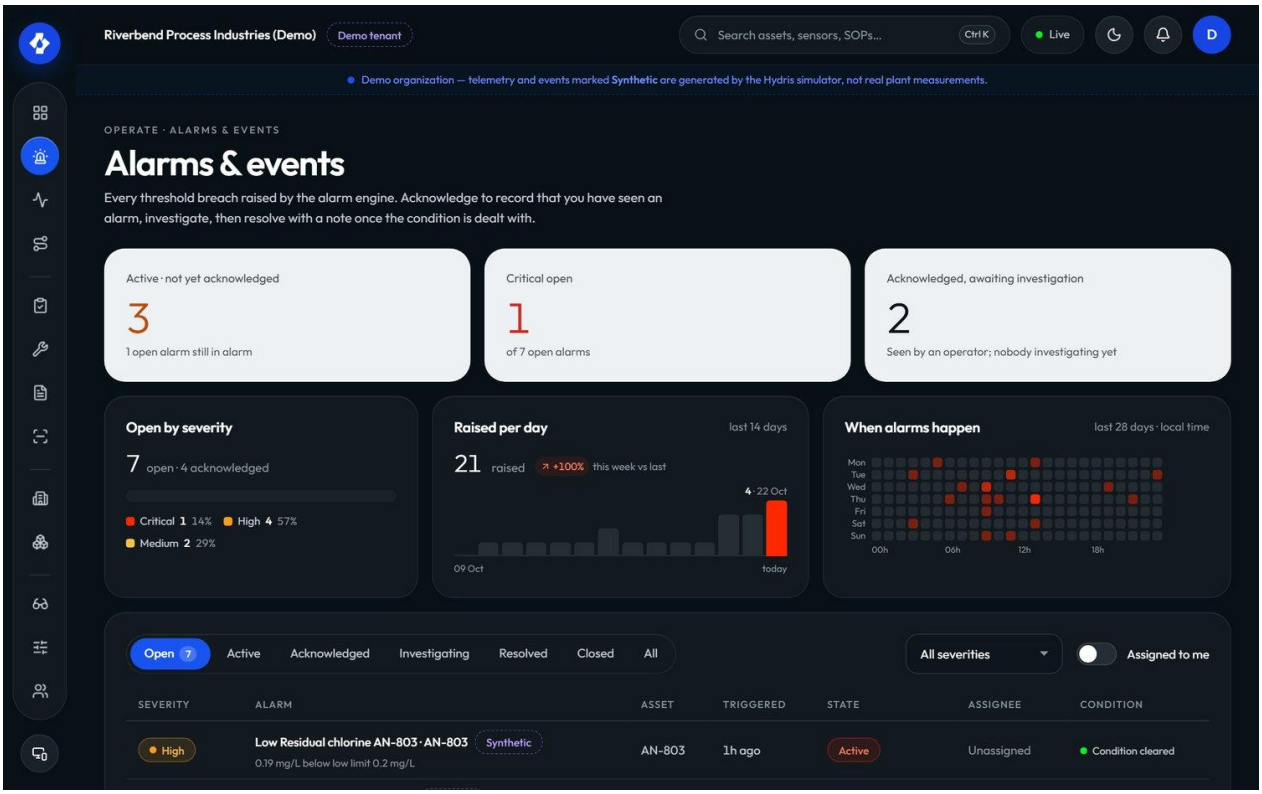

Figure 259. *Console alarms page*

12.7.2 Escalation matrix

Hydris does not page anyone outside the product yet: escalation beyond the console and glasses notifications is a customer procedure (**Planned**: SMS or call escalation). The matrix below is the recommended plant procedure; all times are targets.

Table 346. Alarm escalation matrix (targets)

Severity	Acknowledge within	First escalation	Second escalation	Resolve or hand to plant procedure within
Critical	5 minutes	Supervisor at 5 minutes, by phone	Plant head at 15 minutes	1 hour
High	15 minutes	Supervisor at 15 minutes	Plant head at 1 hour	4 hours
Medium	1 hour	Supervisor at end of shift	None	1 business day
Low	End of shift	None	None	1 week
Stale (no fresh data)	30 minutes	Customer admin (data path)	Hydris L1 if gateway or platform suspected	4 hours

A stale alarm is a data problem first: follow the "data not updating" tree before assuming a plant fault, but never assume the plant is fine because the data stopped.

12.7.3 Alarm hygiene

Weekly, the supervisor reviews the alarm report for: alarms resolved without a meaningful note, repeat alarms on the same sensor (threshold or deadband needs tuning), and alarms closed by the same person who resolved them without review. Rule changes go through the customer's management of change; `alarm_rule.manage` is held only by `org_admin` and `factory_manager`.

12.8 Support procedures and troubleshooting

12.8.1 Ticket intake

L1 records for every ticket: customer, site, user email, role, surface (console, glasses web app, Hydris Field), device name, app version, time of the problem with time zone, and what the user saw (exact message or screenshot). Never ask for or record passwords, MFA codes, refresh tokens, gateway keys or `CRON_SECRET`.

Table 347. Support response targets by priority (targets)

Priority	Example	First response	Update cadence	Target resolution
P1	Nobody can sign in; all telemetry stale	30 minutes	Hourly	4 hours
P2	One site cannot pair glasses; audit approval blocked	2 hours	Every 4 hours	1 business day
P3	One operator's voice not working	1 business day	Daily	3 business days
P4	Question, how-to, feature request	2 business days	Weekly	Best effort

12.8.2 Cannot sign in

Facts from the code (`apps/api/src/modules/auth.ts`, `apps/api/src/config.ts`): wrong email or password returns `invalid_credentials`. Five failures lock the account for 15 minutes (`account_locked`). Access tokens last 15 minutes and refresh tokens 7 days; reuse of a rotated refresh token revokes the family. On the Android app, an account with two-step verification sees "This account uses two-step verification - sign in on the web console first." **Partial**: MFA is not completed inside the Android app.

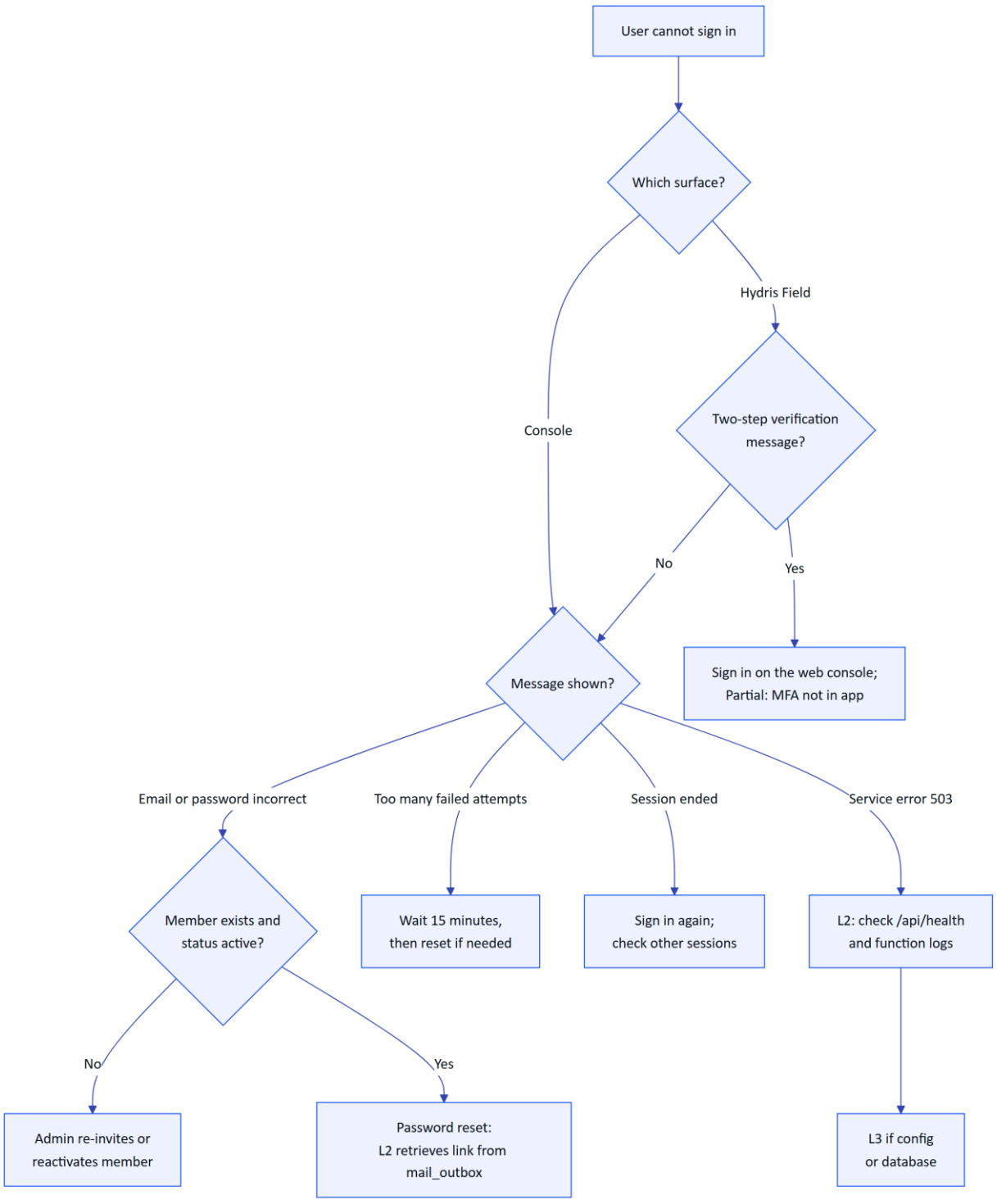

Figure 260. *Troubleshooting tree for a user who cannot sign in*

Password reset links expire after 30 minutes (`TTL.resetMs`). Because email is not wired, L2 confirms the request with the customer admin, reads the latest reset link for that address from `mail_outbox`, and gives it to the user over a verified channel. Delete the row afterwards.

12.8.3 Pairing fails or code expired

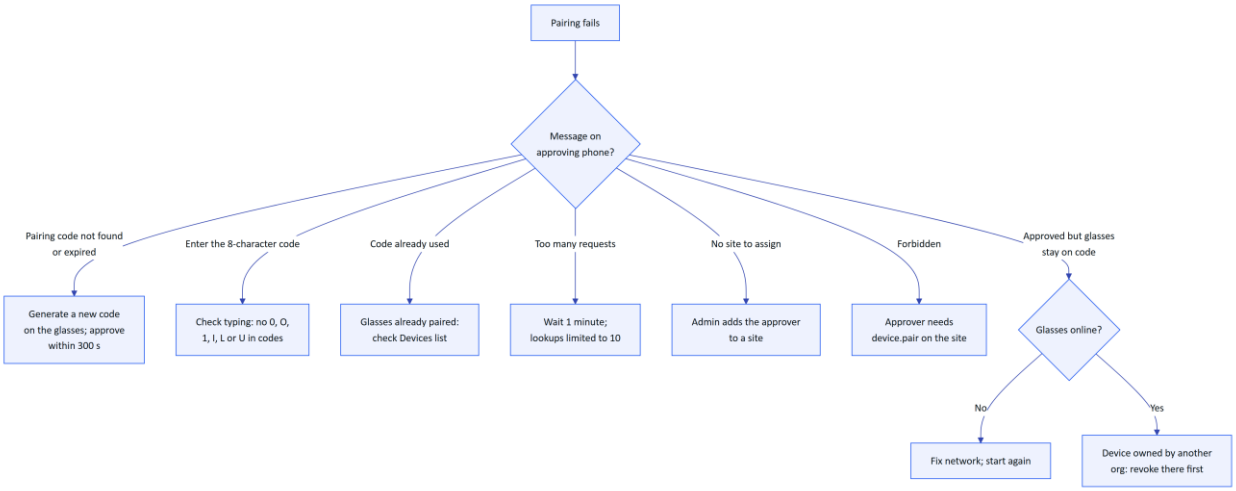

Figure 261. *Troubleshooting tree for pairing failures*

Common causes in pilots: the approver walked to a desk and took longer than five minutes; the approver has no site in scope; the glasses lost Wi-Fi while polling. After approval, if the glasses show "No operator is assigned to this device" (`device_unassigned`), assign an operator in the console.

12.8.4 Glasses not connecting (Ray-Ban Meta)

Messages in Hydris Field (`apps/android/app/src/main/java/ai/hydris/field`) point to the step that failed: "Install the Meta AI app and pair your glasses in it first", "Connect Hydris in the Meta AI app first (Glasses, Connect)", "Camera permission denied in Meta AI", "The glasses are not registered".

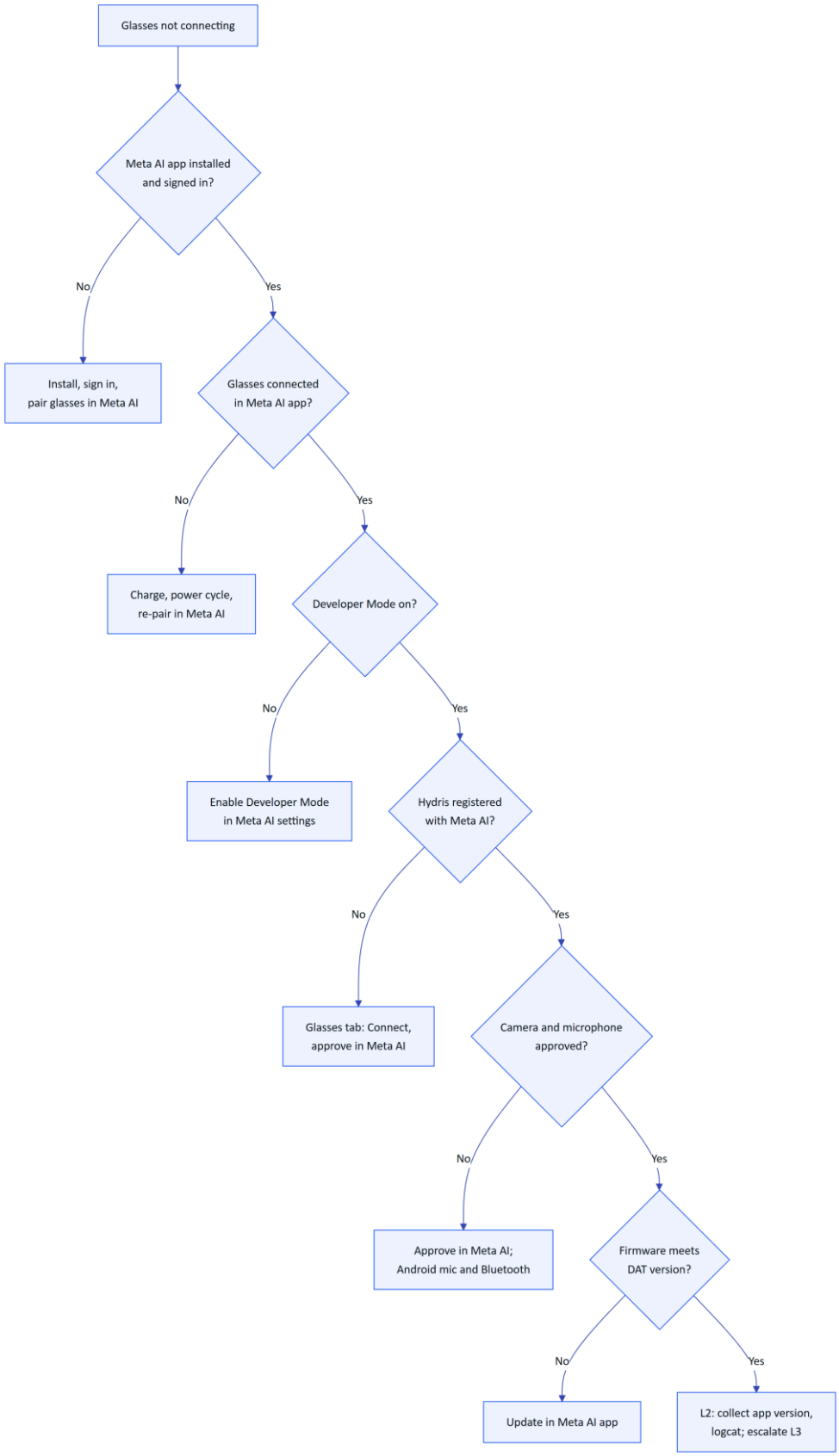

Figure 262. *Troubleshooting tree for Ray-Ban Meta glasses that will not connect*

12.8.5 Camera not available

The camera can come from the glasses (DAT stream) or, as a fallback, the phone. Messages: "Glasses camera not available", "Camera unavailable: ...", "Camera permission is needed to scan with the phone", "No usable camera on this phone". On display glasses and the web app, the browser camera permission applies (`apps/web/src/glasses/camera.ts`).

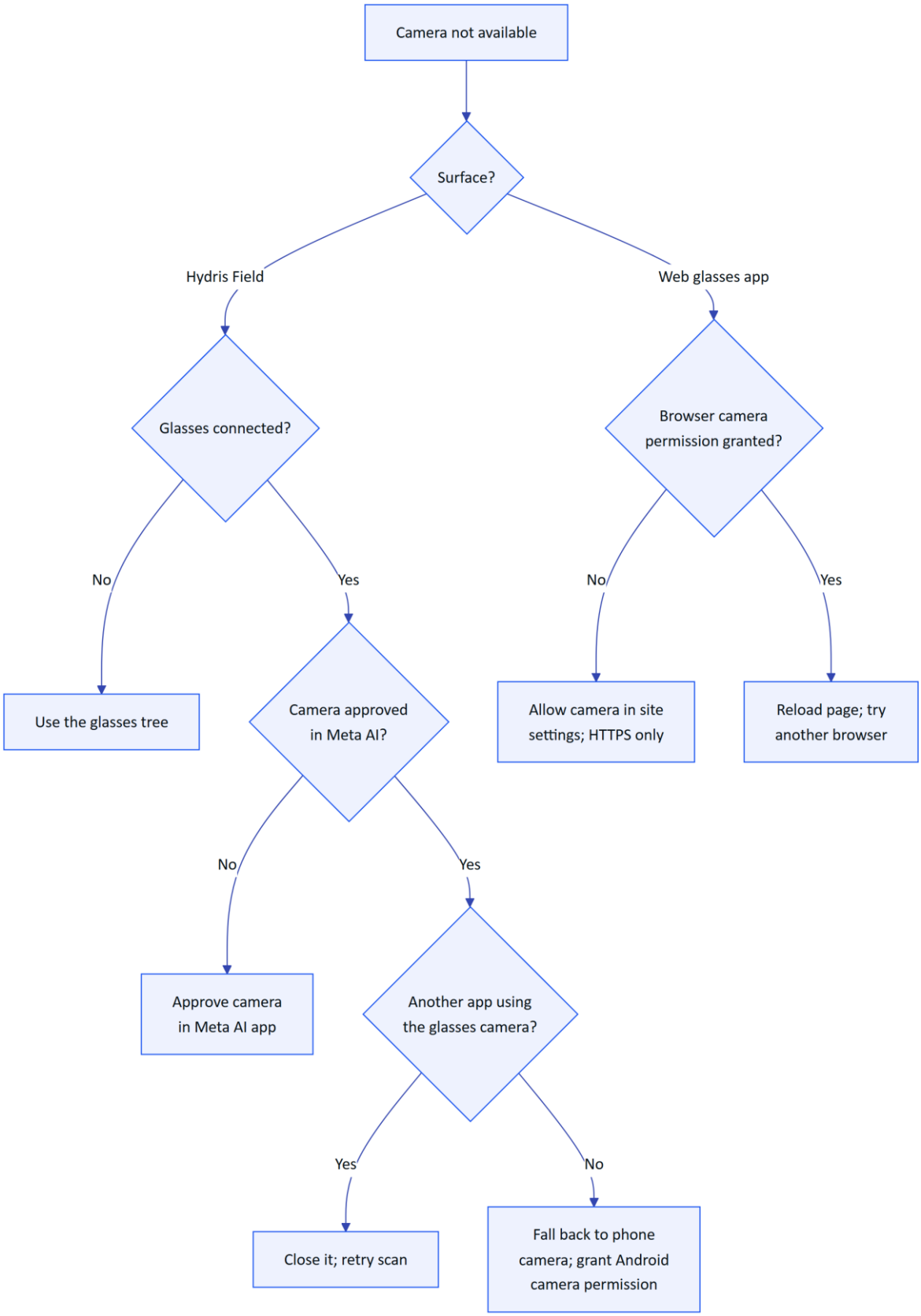

Figure 263. *Troubleshooting tree when the camera is not available for a scan*

12.8.6 OCR results are poor

Guided capture checks the page before it auto-captures (`DocDetect.kt`): it asks the wearer to move closer if the page covers less than 28 percent of the frame, to move back if it touches the edge or covers more than 92 percent, to rotate to portrait, to straighten if a corner is skewed more than 10 degrees, to level if tilted more than 8 degrees, to flatten if the shape is far from A4, and to hold still if the image is not sharp. It then warps the page to a flat A4 image at about 200 dpi.

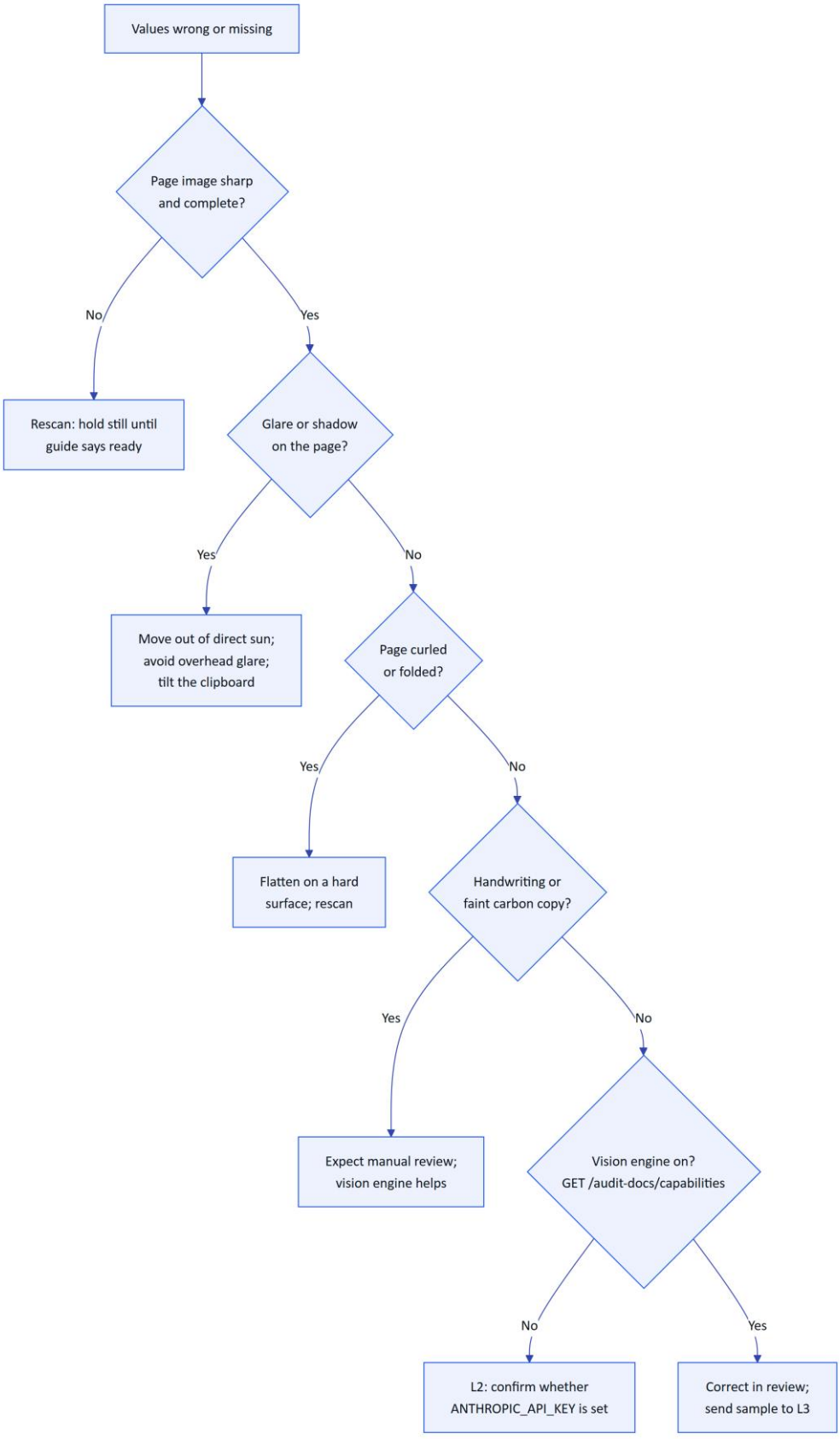

Figure 264. *Troubleshooting tree for poor OCR results on audit documents*

Correcting values in review is normal and expected; the two-person rule exists because OCR is never perfect. Tell operators that a poor scan costs the reviewer time, so a ten-second rescan is cheaper than a ten-minute correction.

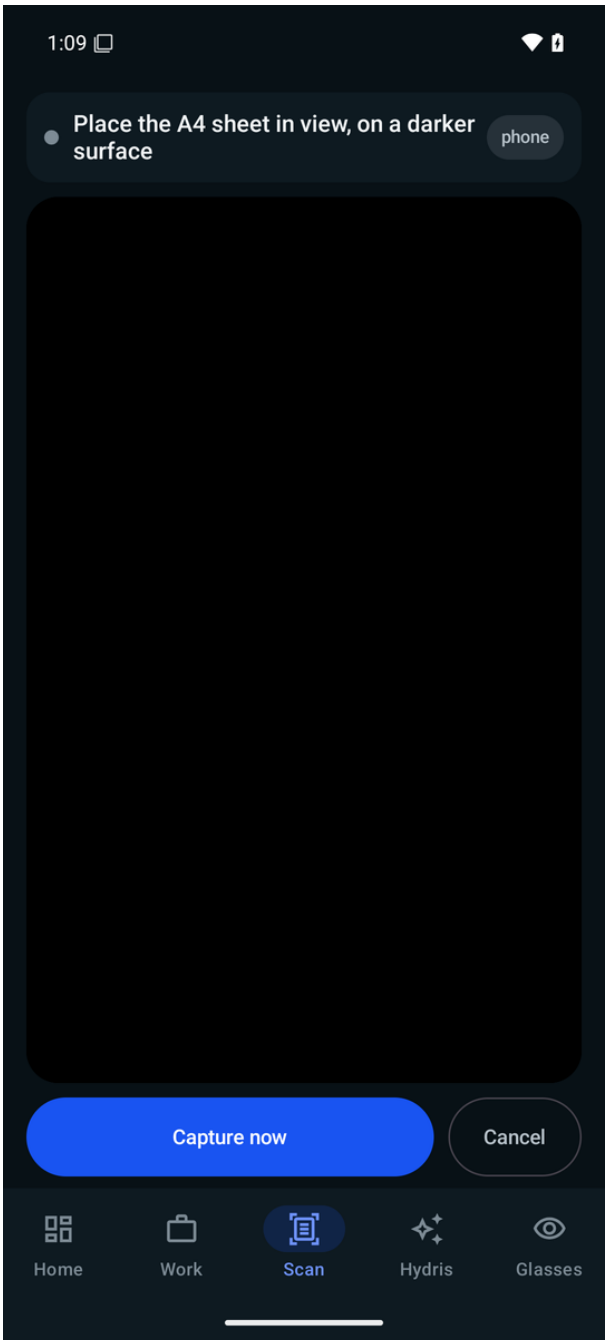

Figure 265. *Guided capture on the phone*

12.8.7 Voice assistant not hearing

"Hey Hydris" uses the glasses microphone (DAT speech) when the glasses are connected and the phone microphone through Android `SpeechRecognizer` otherwise. Answers play through text to speech on the glasses speakers when they are the Bluetooth audio device (`apps/android/app/src/main/java/ai/hydris/field/voice/VoiceEngine.kt`). The wake phrase is matched on the transcript on the device; only the question after it is sent to `POST /api/assistant/ask`. Without an Anthropic key, a deterministic intent engine answers measurements, alarms, my work, asset status and procedures, and says it cannot answer anything else.

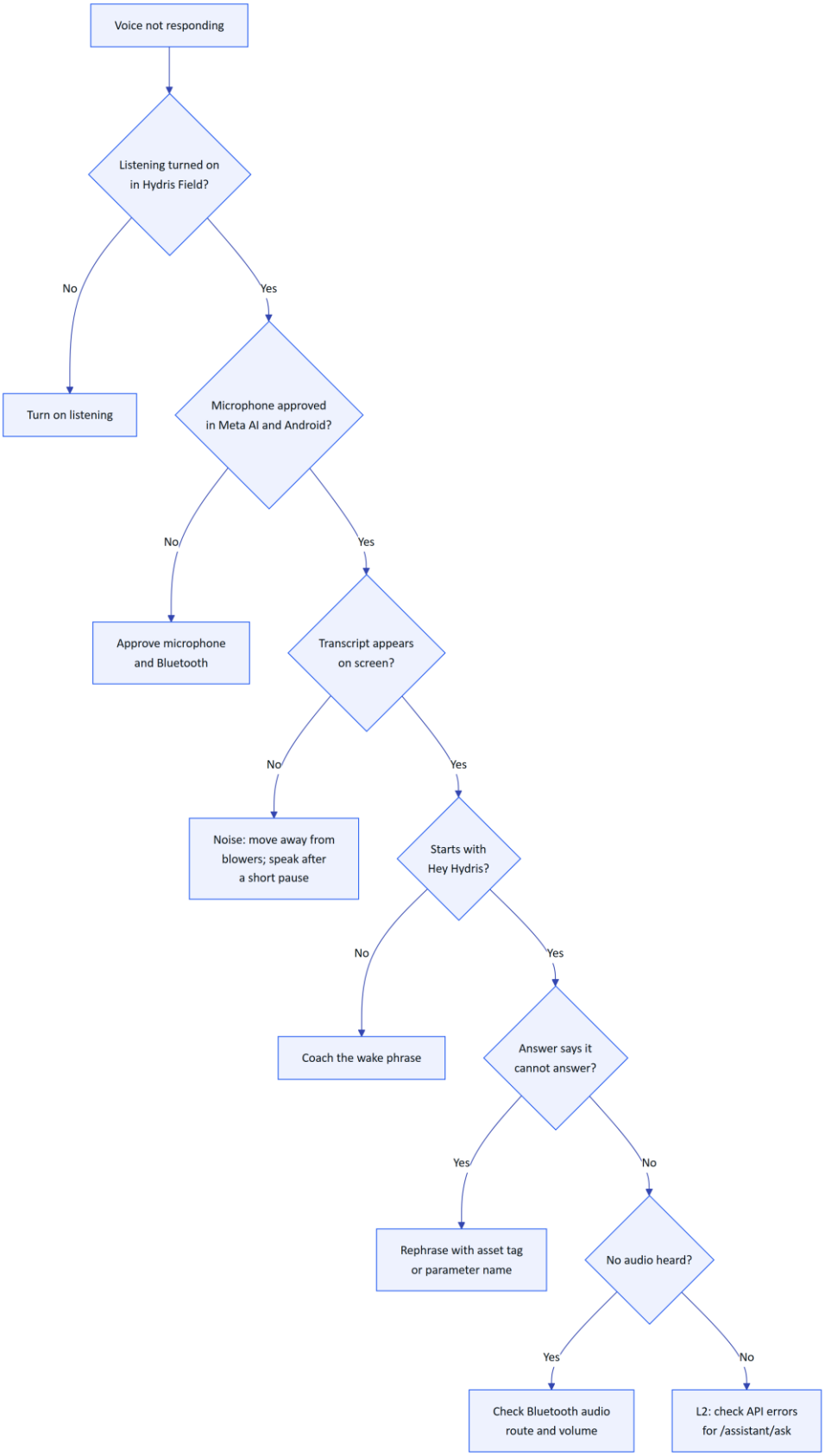

Figure 266. *Troubleshooting tree when the voice assistant does not respond*

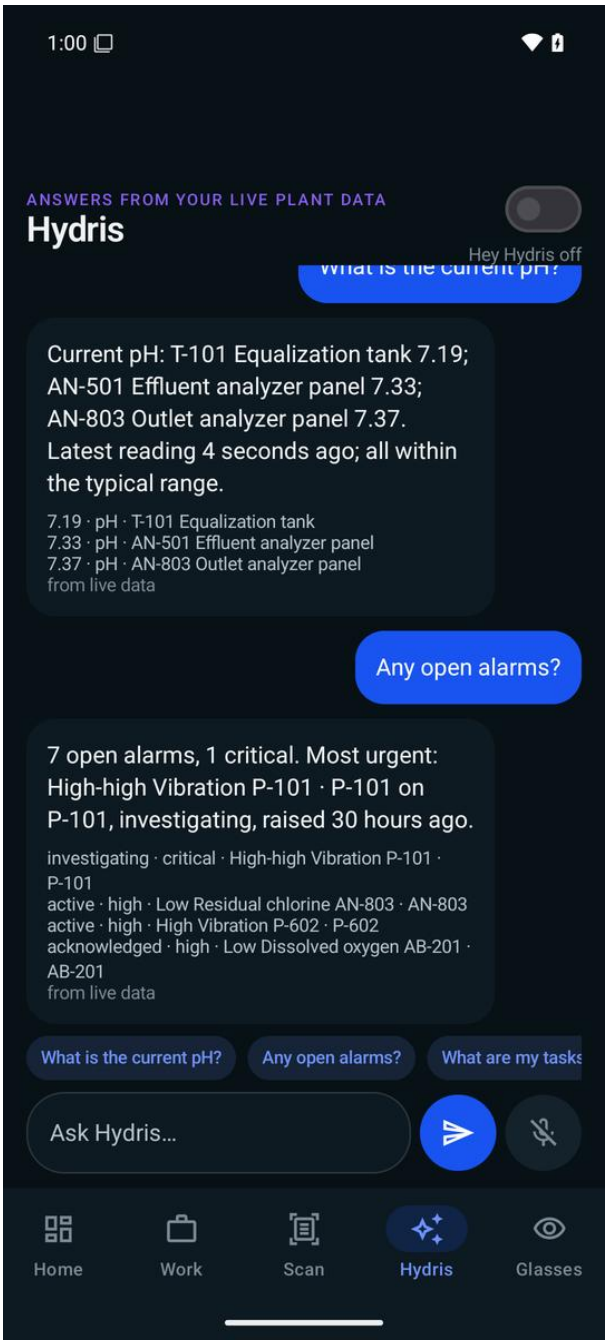

Figure 267. *Voice assistant screen on the phone*

12.8.8 Data not updating (telemetry stale)

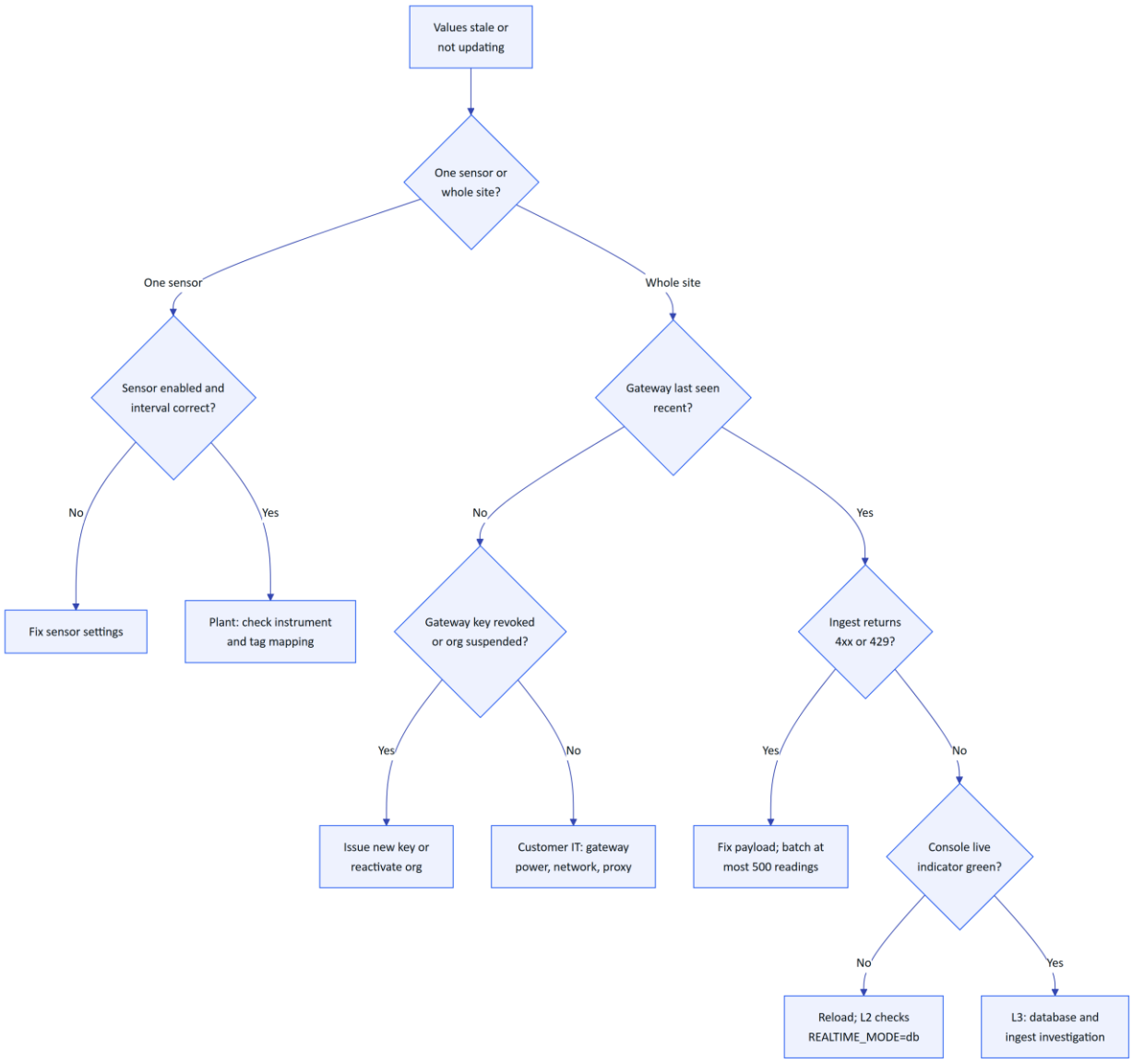

Figure 268. *Troubleshooting tree for telemetry that is stale or not updating*

Two behaviours to remember:

- On Vercel, stale-data rules are evaluated only when `/api/cron` runs. `vercel.json` schedules it **daily at 03:00 UTC** (`0 3 * * *`), so a stale alarm can take up to a day to appear on the hosted deployment. The console's freshness badges are computed on read and are always current. On a long-running container the job runs every 30 seconds (`apps/api/src/jobs.ts`).
- In `REALTIME_MODE=db`, the console polls the `realtime_events` table, so updates can lag by about 1.5 seconds. That is normal.

12.8.9 Console slow

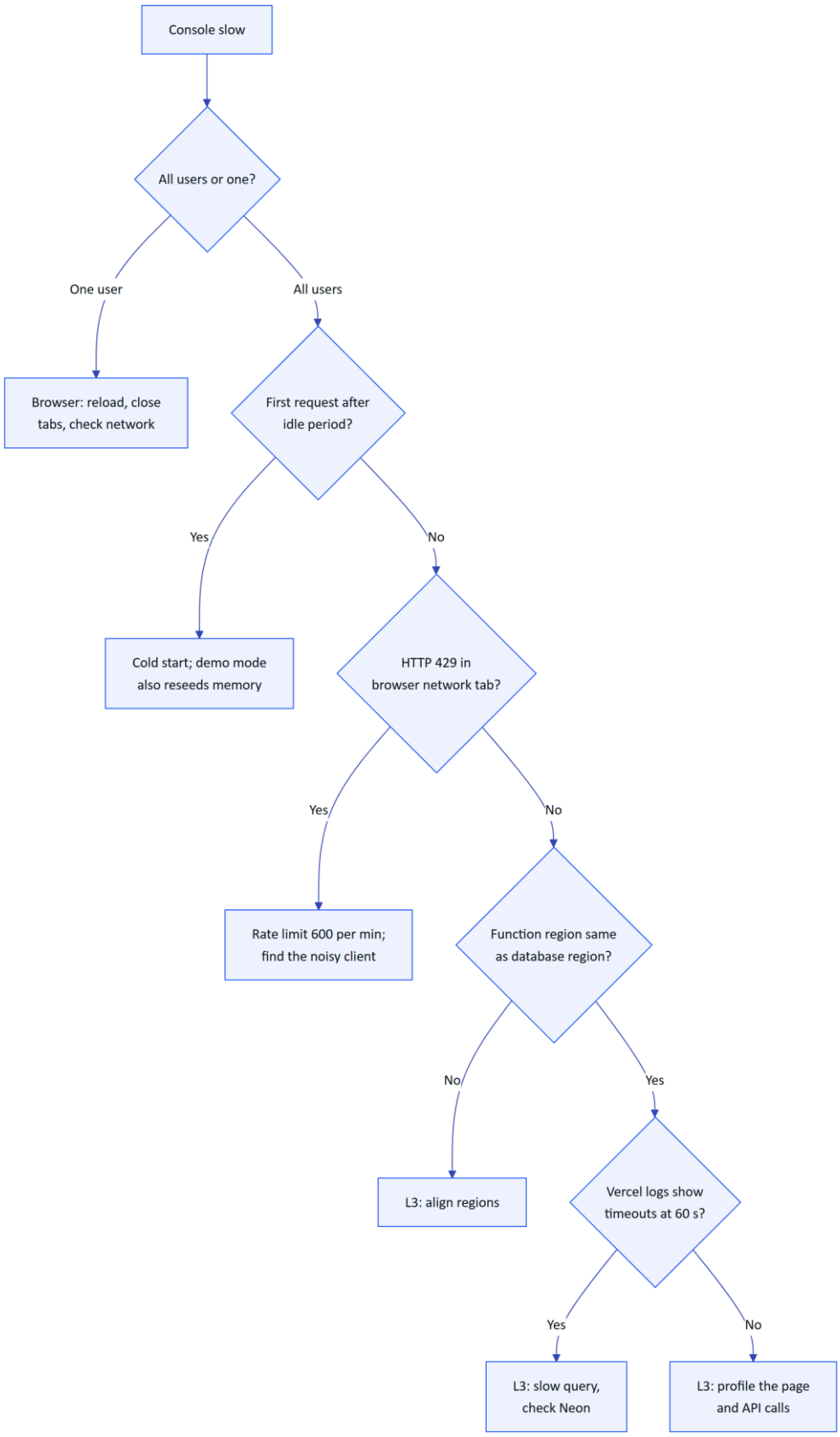

Figure 269. *Troubleshooting tree for a slow operator console*

12.9 Incident management

12.9.1 Severity levels

Table 348. Incident severity levels (targets)

Severity	Definition	Examples	Response	Customer update
Sev 1	Platform down or data at risk for one or more customers	`/api/health` failing; cross-tenant data exposure; data loss	L3 engaged within 15 minutes, all hands	Within 1 hour, then hourly
Sev 2	Major function down for a customer, no workaround	No telemetry ingest; nobody can pair; audit approval broken	L3 engaged within 1 hour	Within 2 hours, then every 4 hours
Sev 3	Function degraded, workaround exists	Voice down but console works; one site's gateway down	Next business day	Daily
Sev 4	Minor or cosmetic	Label wrong, slow export	Planned release	On release

A suspected security or personal data breach is always Sev 1 until ruled out and follows SECURITY_PRIVACY.md: customers within 24 hours, CERT-In within 6 hours for reportable incidents, the Data Protection Board and affected data principals per the DPDP Rules, and GDPR authorities within 72 hours where applicable.

12.9.2 Incident lifecycle

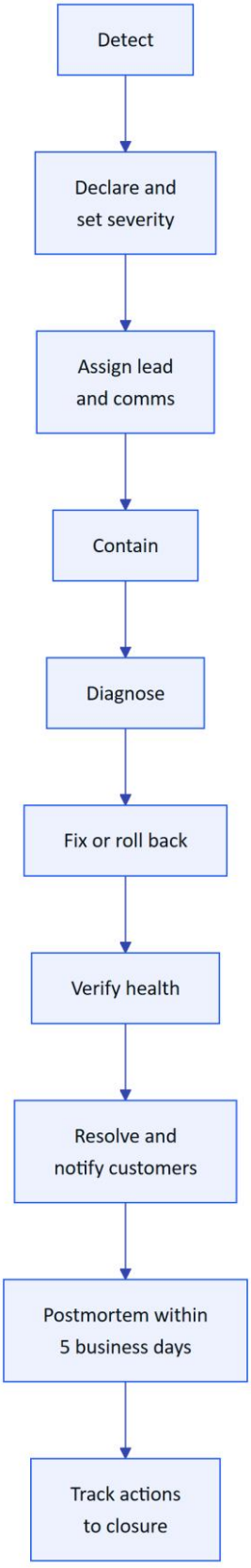

Figure 270. *Incident lifecycle from detection to postmortem*

Containment levers available today (all **Built**): revoke a user session (`DELETE /api/auth/sessions/:id`), suspend or revoke a device, revoke a gateway (`POST /api/gateways/:id/revoke`), suspend an organisation (`POST /api/platform/organizations/:id/status`), roll back the Vercel deployment, and as a last resort rotate `JWT_SECRET`, which signs everyone out. Never rotate `DATA_KEY` without re-encrypting MFA secrets: losing it disables MFA for every user.

12.9.3 Communication templates

Initial notice (within the update target for the severity):

```
Subject: [Hydris] Sev <n> incident: <short description>

We are investigating an issue affecting <surface> for <customer or "some customers">
since <time, time zone>. Impact: <what users see>. Plant equipment and control
systems are not affected; Hydris is read-only to the plant.

Workaround: <if any, otherwise "none yet">.
Next update: <time>.
Incident lead: <name>, Hydris.
```

Update:

```
Subject: [Hydris] Update <n>: <short description>

Status: <investigating | identified | fix in progress | monitoring>.
What we know: <one or two sentences>.
What we are doing: <one or two sentences>.
Next update: <time>.
```

Resolution:

```
Subject: [Hydris] Resolved: <short description>

The issue affecting <surface> was resolved at <time>. Duration: <duration>.
Cause: <one sentence>. Data: <"no data was lost" or what was affected and how
it was recovered>. A postmortem will follow by <date>.
```

12.9.4 Postmortem template

```
Title, date, severity, incident lead, authors

Summary: two or three sentences a plant manager can understand.
Impact: customers, sites, users, duration, data affected.
Timeline: detection, declaration, key actions, resolution (with time zone).
Root cause: technical cause and contributing factors.
Detection: how we found out; how we should have found out sooner.
What went well / what went badly.
Action items: owner, due date, ticket link. Blameless language throughout.
```

12.10 Monitoring and health checks

12.10.1 Health endpoint

Built. `GET /api/health` returns `{"ok": true, "time": "<ISO timestamp>"}` (`apps/api/src/app.ts`). It confirms that the function starts and the API is registered. It does not prove that the database is reachable; for that, check that the console overview loads, or call an authenticated endpoint such as `GET /api/overview`. An external uptime check on `/api/health` is **Planned**; until then L2 runs it in the daily check.

12.10.2 Scheduled jobs

Built. `runScheduledJobs` (`apps/api/src/jobs.ts`) performs one pass of: stale-data rule evaluation, pairing challenge expiry, pruning `realtime_events` older than 24 hours (`apps/api/src/core/events.ts`), and one simulation step for each running scenario when `SIMULATOR_ENABLED=true`.

Table 349. Background jobs by deployment type

Job	Long-running container	Vercel
Stale-data rules	Every 30 s	On each `/api/cron` run
Pairing expiry	Every 60 s	On each cron run (challenges also expire on poll and lookup)
Realtime pruning	Hourly when `REALTIME_MODE=db`	On each cron run
Simulation	Continuous, resumes running scenarios at start	One step per cron run, or lazily per request in demo mode

The cron route (`apps/web/src/app/api/cron/route.ts`) accepts only `Authorization: Bearer $CRON_SECRET` and returns `{"ok": true, "ms": <duration>, "report": {...}}`. A `401` means `CRON_SECRET` is missing or changed. A report field that starts with `error:` names the failing job.

> **DECISION**
> The schedule in `vercel.json` is daily (`0 3 * * *`). For a production pilot with live telemetry, stale alarms evaluated once a day are not acceptable. Before a production pilot, either move to a plan that allows a per-minute cron (`* * * * *`) or call `/api/cron` every minute from an external scheduler with the secret. Owner: L3. Status: **open**.

12.10.3 Signals to watch

Table 350. Monitoring signals and where to find them today

Signal	Source today	Threshold to act (target)	Status
API up	`/api/health`	Any failure	**Built** (manual check)
Cron success	Vercel cron log JSON	Any `error:` field	**Built** (manual check)
5xx rate	Vercel function logs	More than 1 percent over 15 minutes	**Partial** (no alerting)
Refresh token reuse	`audit_events` actions `auth.refresh_reuse_detected`, `device.refresh_reuse_detected`	Any	**Built** (query)
Login failures	`audit_events` action `auth.login_failed`	Spike against baseline	**Built** (query)
Ingest lag	Sensor freshness on Telemetry	Any critical sensor stale	**Built**
Tracing, error tracking, uptime alerts	OpenTelemetry, Sentry, uptime monitor		**Planned**

12.11 Data requests

12.11.1 Exports

Built. All exports are scoped to the caller's organisation and sites, and CSV exports are audited (`report.exported`).

Table 351. Data export paths

Data	Path	Permission
Alarms, inspections, maintenance, devices, operators	`GET /api/reports/:kind?format=csv`	`report.export`
Sensor readings	`GET /api/telemetry/sensors/:id/export.csv`	`telemetry.read`
Approved audit document with trail	`GET /api/audit-docs/:id/pdf`	`auditdoc.read`
Audit log	`GET /api/audit` (console **Audit**)	`audit.read`
Change history of an entity	`GET /api/history/:entityType/:entityId`	Read permission on the entity

A full tenant export (all tables, all files) is **Planned**; until then L3 produces it from the database on a signed request from the customer admin.

12.11.2 Deletion and DPDP requests

India's DPDP Act 2023 and the DPDP Rules 2025 give data principals rights to access, correction and erasure, and to grievance redressal. For a plant, most personal data is the names and emails of members, IP addresses in sessions and audit events, and names attached to records.

Table 352. Handling data principal requests

Request	Procedure	Status
Access	Customer admin exports member record and their activity (`/api/audit` filtered by user); L2 assists	**Partial** (manual)
Correction	Member updates their name in **Account**; admin updates title and role	**Built**
Stop access	Admin deactivates the member (`PATCH /api/members/:userId`) and revokes their devices	**Built**
Erasure	No self-service deletion. L3 pseudonymises the user record on written request after legal review. Records that form part of a regulated record (alarm notes, inspection sign-offs, audit document approvals) are kept with the customer's retention period and the name pseudonymised where the law allows	**Planned** (manual today)
Grievance	Logged as a P2 ticket; answered within the statutory period	**Built** (process)

> **CAUTION**
>
> Hydris is usually the processor; the customer is the data fiduciary. Act on a data principal's request only through the customer admin, unless the request concerns Hydris's own website inquiries (`/public/inquiries`, where Hydris is the fiduciary). Never hard-delete audit trails: `audit_events`, `telemetry_corrections` and `audit_document_events` are append-only by design.

12.12 Demo environment management

Built. Demo mode turns on when `DEMO_MODE=true`, or automatically on Vercel when no `DATABASE_URL` is set (`isDemoEnv` in `apps/api/src/config.ts`). In demo mode:

- Each serverless instance runs its own in-memory database seeded with the same deterministic synthetic tenant. A cold start resets it. `DEMO_PERSIST=true` with a Postgres URL seeds into Postgres instead.

- Built-in public demo secrets are used if none are configured, and a warning is logged. Demo mode must never hold real plant data.
- "Enter demo mode" asks for a name and creates a personal guest account (`POST /api/auth/demo`, `apps/api/src/core/demo-guests.ts`): an org admin on all sites with its own copy of open inspections and maintenance tasks. The same name always maps to the same account. Guest names are kept for 30 days.
- State that must survive across instances (pairing challenges and approvals, guest names) goes into a shared store (`apps/api/src/core/demo-shared.ts`): Postgres when available, otherwise Vercel Runtime Cache, otherwise process memory.
- Access tokens last 12 hours, the simulator is on and the realtime mode is `db`.
- Self-approval of audit documents is allowed and recorded as such.

Before a sales demo (L1 or sales engineer):

- ☐ Open the demo URL 10 minutes early to warm an instance
- ☐ Enter demo mode with the presenter's name; check the overview loads
- ☐ Start a scenario on `/simulator` (for example `pump_vibration`) and confirm an alarm appears
- ☐ Pair the simulator or phone glasses view and confirm the operator's name shows
- ☐ Run one audit scan end to end

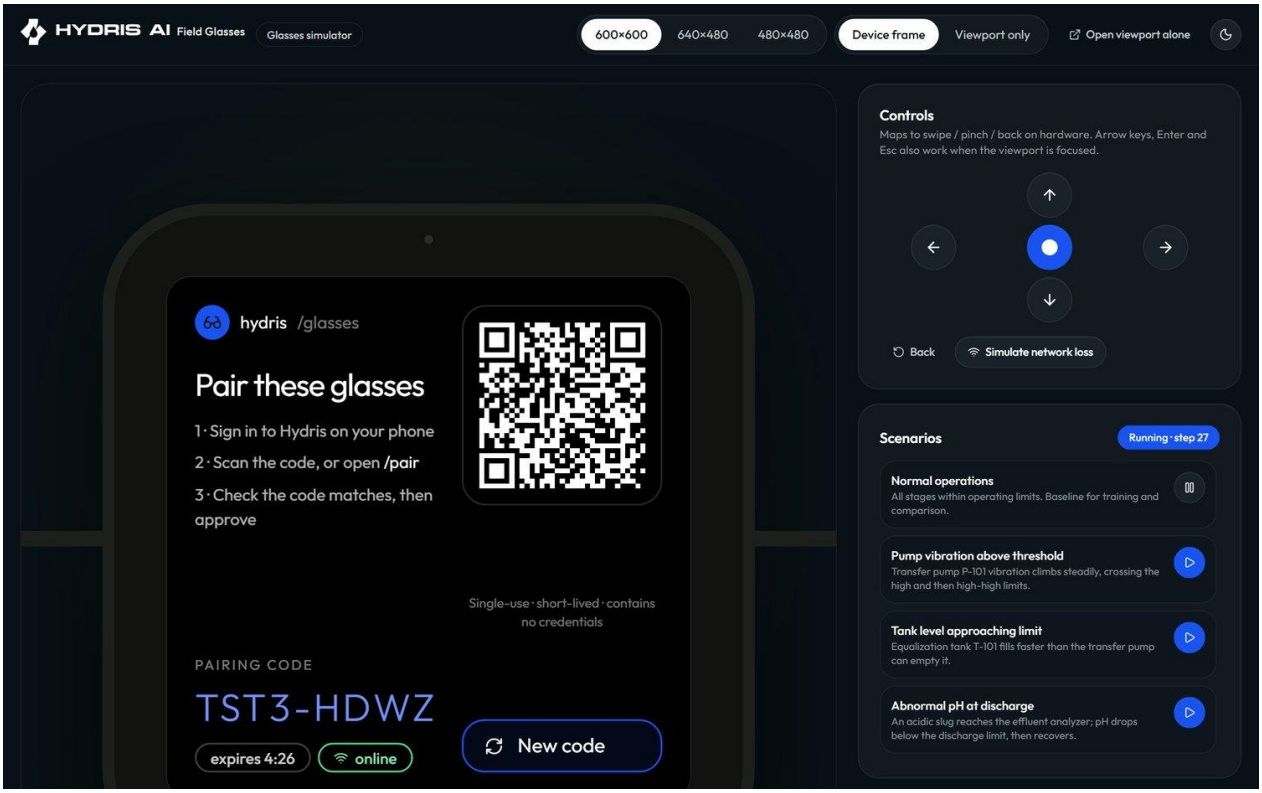

Figure 271. *Simulator used in demos*

If the demo misbehaves mid-session, reload: a new instance reseeds. Do not try to repair demo data. For a customer who wants a longer evaluation, set up a separate Postgres-backed tenant with `npm run seed:demo` rather than relying on the in-memory demo.

12.13 Release and change management

12.13.1 Environments and the path to production

DEPLOYMENT.md defines dev (local, PGlite), preview (per pull request), staging and production. Vercel deploys previews per pull request and production from `main`. There is no CI configuration in the repository yet, so the checks below are run by the releasing engineer (**Partial**).

Pre-release checklist (L3):

- ☐ `npm run typecheck`, `npm test` and `npm run build` pass locally
- ☐ `npm run test:e2e` (Playwright) passes against the preview
- ☐ Migrations reviewed: additive only, or a written plan for destructive steps
- ☐ Preview deployment smoke test: sign in, overview, pairing via `/simulator`, alarm transition, audit scan
- ☐ Release note written for customers if behaviour changes
- ☐ Android: new APK tested on the spare kit if the API contract changed

Migrations run on the first request of each new function instance under a Postgres advisory lock, or during the build with `npm run vercel-build` and `MIGRATE_ON_START=false` (docs/operations/vercel.md).

12.13.2 Maintenance windows

Table 353. Change classes and windows (targets)

Change	Window	Notice to customers
Web and API release, no migration	Any time outside shift changes	Release note after
Release with additive migration	Weekdays 14:00 to 16:00 IST	2 business days
Release with destructive migration or data backfill	Sunday 06:00 to 08:00 IST	5 business days
Android APK rollout	Between shifts, per site	2 business days
Glasses firmware	Between shifts, after spare-kit test	2 business days
Secret rotation (`JWT_SECRET`, `CRON_SECRET`)	Planned window or incident	Users will need to sign in again for `JWT_SECRET`

12.13.3 Rollback

1. **Web and API:** promote the previous production deployment in Vercel (instant rollback). Because migrations are additive, the previous code runs against the newer schema.
2. **Destructive migration:** do not roll back code alone. Restore from Neon point-in-time recovery to a branch, verify, then switch; this is an L3 decision and a Sev 2 incident at least.
3. **Android:** reinstall the previous APK (kept with every release).
4. **UI profile:** use **Rollback** on the profile (`POST /api/ui-profiles/:id/rollback`); glasses reload the earlier version.
5. **Alarm rule change:** edit the rule back; the audit log shows the previous values.

12.14 Training curriculum

12.14.1 Curriculum

Table 354. Training modules by audience

Module	Audience	Duration	Format	Covers
Platform overview	All	30 min	Classroom	What Hydris does and does not do; read-only to the plant
Glasses basics	Operators, maintenance	45 min	Hands-on	Wearing, charging, Hydris Field home, work list, alarms on glasses
Guided capture	Operators	45 min	Hands-on on real log sheets	Lighting, flattening, holding still, multi-page scans
Hey Hydris	Operators, maintenance	30 min	Hands-on on the floor	Wake phrase, asking by tag, limits of the assistant
Alarm handling	Operators, supervisors	45 min	Scenario on simulator	Acknowledge vs resolve, notes, escalation matrix
Inspections and maintenance	Operators, supervisors	45 min	Hands-on	Executing routes, checklists, requesting maintenance
Review and verification	Supervisors, managers	60 min	Hands-on in console	Review queue, bulk confirm, verify, approve, reject, PDF
Fleet administration	Customer admin, supervisors	45 min	Hands-on	Pairing approval, assign, suspend, revoke, UI profiles
Tenant administration	Customer admin	90 min	Workshop	Sites, assets, sensors, gateways, users, roles, SOPs, reports
Support and escalation	Customer admin, supervisors	30 min	Classroom	When and how to contact Hydris, what to include

12.14.2 Operator journey

The journey below shows the target experience for an operator in their first week, with scores from 1 (poor) to 5 (good) as the **expected** experience, to be validated in the pilot.

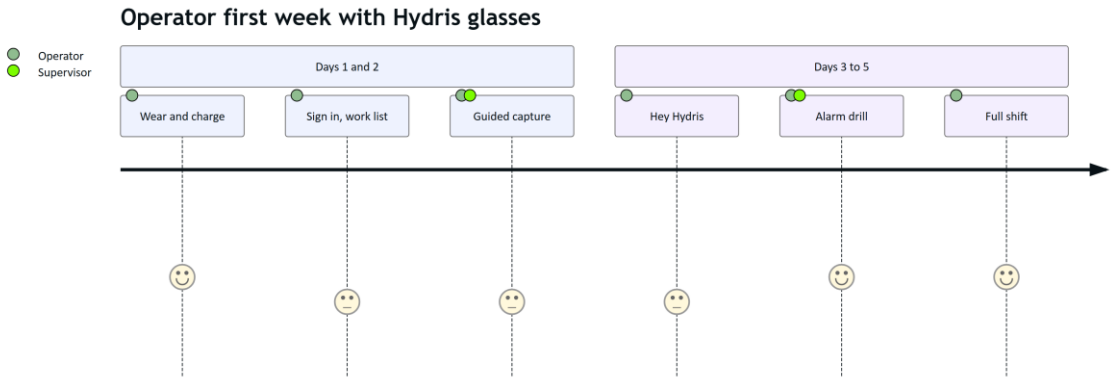

Figure 272. *Operator learning journey in the first week (expected experience)*

Sign-off: an operator is signed off after one full shift in which they completed an inspection route, scanned at least one document that was verified without a rescan, and acknowledged a drill alarm. Supervisors are signed off after verifying five documents and completing a pairing approval and a device reassignment.

12.15 Operations KPIs

All values are **targets** for the first pilot. Measurement sources are named so the numbers can be produced from the product rather than estimated.

5 min
Time to pair
Target from code shown to Connected

1 day
Scan to verified
Target for routine records

5 min
Critical alarm acknowledge
Target, median

99.5%
Monthly availability
Target for /api/health

Table 355. Operations KPIs, targets and measurement sources

KPI	Target	Measured from
Time to pair (challenge created to device connected)	Median under 5 minutes	`pairing_challenges.created_at` to `completed_at`; device `last_seen_at`
Pairing success on first code	Over 90 percent	Completed vs expired challenges
Scan to verified	Median 1 business day	`audit_documents.created_at` to `verified_at`
Verified to approved	Median 2 business days	`verified_at` to `approved_at`
Rescan rate	Under 10 percent of documents	Rejected documents over total
Critical alarm acknowledge time	Median under 5 minutes	Alarm events (raised to acknowledged)
High alarm acknowledge time	Median under 15 minutes	Alarm events
Alarms resolved with a note	100 percent (enforced in code)	`requiresNote` on resolve
Sensors live	Over 98 percent of sensor-hours	Freshness on telemetry history
Devices connected on shift	Over 95 percent	Device report
Support first response	Per priority table	Ticket system
Platform availability	99.5 percent monthly	External uptime check (**Planned**)

12.16 Support contact and escalation templates

Support contacts are confirmed per customer at kickoff and recorded in the customer's account plan.

Table 356. Support contact details (to be completed per customer)

Channel	Use	Hours
Support email (to be confirmed)	P3 and P4 tickets	Business hours IST
Support phone (to be confirmed)	P1 and P2	24 by 7 for P1 during pilots (target)
Shared chat channel	Day-to-day coordination	Business hours IST
Customer success manager	Account, training, KPIs	Business hours IST

Customer admin to Hydris (ticket template):

```
Subject: [<Customer> / <Site>] <P1-P4>: <short description>

Who is affected: <user emails or "all users at site X">
Surface: <console | glasses web app | Hydris Field app> <app version if known>
Device: <device name from the Devices page, if relevant>
Since: <date and time, time zone>
What happened: <steps and the exact message shown>
What we tried: <steps from the runbook>
Screenshots: attached (no passwords, codes or keys)
```

L1 to L2 or L3 escalation:

```
Ticket: <id>   Priority: <P1-P4>   Customer: <name>   Site: <name>
Summary: <one sentence>
Evidence: <health result, cron report, log lines with timestamps, request ids>
Runbook trees followed: <names and where they ended>
Suspected area: <auth | pairing | devices | telemetry | audit docs | voice | infra>
Customer impact and workaround: <text>
Ask: <what you need from the next tier>
```

12.17 Appendices

12.17.1 Glossary

Table 357. Glossary

Term	Meaning
DAT	Meta Wearables Device Access Toolkit; the SDK Hydris Field uses to reach Ray-Ban Meta glasses
Developer Mode	Meta AI app setting that lets an unpublished app such as Hydris Field register with the glasses
Device token	Access and refresh tokens issued to a paired glasses device, separate from user sessions
Freshness	Live, delayed, stale or no data, derived from a sensor's expected interval
Gateway	Plant-side system that posts readings with an `hgw_` key
Pairing challenge	Short-lived (300 s), single-use code that links glasses to a site
RLS	Postgres row-level security; isolates tenants at the database
Stale rule	Alarm rule that raises "No fresh data" when a sensor stops reporting
Two-person verification	Verify by one person, approve by a different person (audit documents)
UI profile	Versioned configuration of the glasses screens and actions, assigned by device, user, role or org
Demo mode	Self-contained deployment with synthetic data, guest accounts and in-memory database
DPDP	India's Digital Personal Data Protection Act 2023 and Rules 2025

12.17.2 Command cheat sheet

Health and cron (replace the domain for other environments):

```
curl -s https://hydrisview.vercel.app/api/health
curl -s -H "Authorization: Bearer $CRON_SECRET" https://hydrisview.vercel.app/api/cron
```

Local development, database and seed (repository root):

```
npm install
npm run dev                  # API on :4000, web on :3000
npm run db:migrate
npm run seed                 # needs SEED_ADMIN_EMAIL and SEED_ADMIN_PASSWORD (12+ chars)
npm run seed:demo            # also needs SEED_DEMO_PASSWORD (12+ chars)
npm run typecheck && npm test && npm run build
npm run test:e2e
```

Demo mode locally:

```
DEMO_MODE=true npm run dev
```

Reports and exports with a console access token:

```
curl -s -H "Authorization: Bearer $TOKEN" "https://hydrisview.vercel.app/api/reports/alarms?format=csv"
-o alarms.csv
curl -s -H "Authorization: Bearer $TOKEN"
"https://hydrisview.vercel.app/api/reports/devices?format=csv" -o devices.csv
curl -s -H "Authorization: Bearer $TOKEN" "https://hydrisview.vercel.app/api/audit-docs/$DOC_ID/pdf" -o
audit-doc.pdf
```

Device actions:

```
curl -s -X POST -H "Authorization: Bearer $TOKEN" -H "Content-Type: application/json" \
  -d '{"reason":"lost on site"}' "https://hydrisview.vercel.app/api/devices/$DEVICE_ID/suspend"
curl -s -X POST -H "Authorization: Bearer $TOKEN" -H "Content-Type: application/json" \
  -d '{"reason":"stolen"}' "https://hydrisview.vercel.app/api/devices/$DEVICE_ID/revoke"
```

Android build, install and logs:

```
cd apps/android
./gradlew assembleDebug
adb install -r app/build/outputs/apk/debug/app-debug.apk
adb logcat | grep -i hydris
```

Database checks (L3, read-only role, production only with approval):

```
select action, at, ip from audit_events
 where action in ('auth.refresh_reuse_detected', 'device.refresh_reuse_detected')
 order by at desc limit 50;
select status, count(*) from pairing_challenges
 where created_at > now() - interval '7 days' group by status;
```

13 Product Roadmap and Future Features

> **NOTE**
>
> **HFG-RMP v2.0** · Draft for review · Owner: Product, Hydris. Where Hydris Field Glasses goes next, in what order, why, and how we will know it worked

Hydris Field Glasses already does something no other product in Indian water and wastewater does end to end: it puts live plant state, the next task, the approved procedure and a grounded voice assistant in an operator's line of sight, and it turns paper log sheets into verified, searchable records. The minimum viable product is built and runs as a zero-configuration demo. The next twelve months are about turning it into something a plant can rely on every shift, then into something a regulator, a maintenance system and a control room can rely on too.

This roadmap is a plan, not a commitment. Everything dated is an estimate that assumes the current team and no major change in Meta's developer programme. Status labels follow the house rule: **Built** is on `main` (commit `07cca23`), **Partial** exists in code but is incomplete or unverified, **Planned** does not exist yet. Effort, reach and impact numbers are team estimates and are labelled as such.

5
quarters planned
Q4 2026 to Q4 2027

8
themes
From hardening to ecosystem

19
future features analysed
Each with problem, outcome and dependencies

1 to 2
pilot plants targeted
ETP or STP, Q1 2027 (target)

13.1 How to read this roadmap

13.1.1 Horizons, not dates

The roadmap is organised in three horizons. **Now** is committed work for the current and next quarter, with named owners and acceptance criteria in the QA and Test Plan. **Next** is planned for the following two quarters; scope may still change after pilot feedback. **Later** is direction: we believe these features matter and have designed their shape, but they will be re-prioritised with evidence from the pilot.

Table 358. Confidence by horizon

Horizon	Window (plan)	Confidence in scope	Confidence in dates	What changes it
Now	Q4 2026 to Q1 2027	High	Medium	Pilot customer dates, Meta DAT access
Next	Q2 to Q3 2027	Medium	Low	Pilot KPIs, customer requests, team size
Later	Q4 2027 and beyond	Low	Very low	Market, partners, regulation, hardware

13.1.2 Planning assumptions

1. The team stays roughly at its current size until a pilot converts; dates assume no new hires before Q2 2027.

2. Meta keeps the Wearables Device Access Toolkit (DAT) available to developers, and India-based testers can use it (unconfirmed; see Dependencies and risks).
3. The first customers are Indian ETPs, STPs and CETPs with English-literate supervisors and mixed-language operators.
4. Hydris remains read-only to the plant: no feature in this roadmap writes to a PLC, SCADA or DCS.

13.2 Where the product is today

The MVP covers every surface: website, console, glasses web app with a simulator, native Android app, and the API. The inventory below is grounded in the PRD requirement statuses and the code.

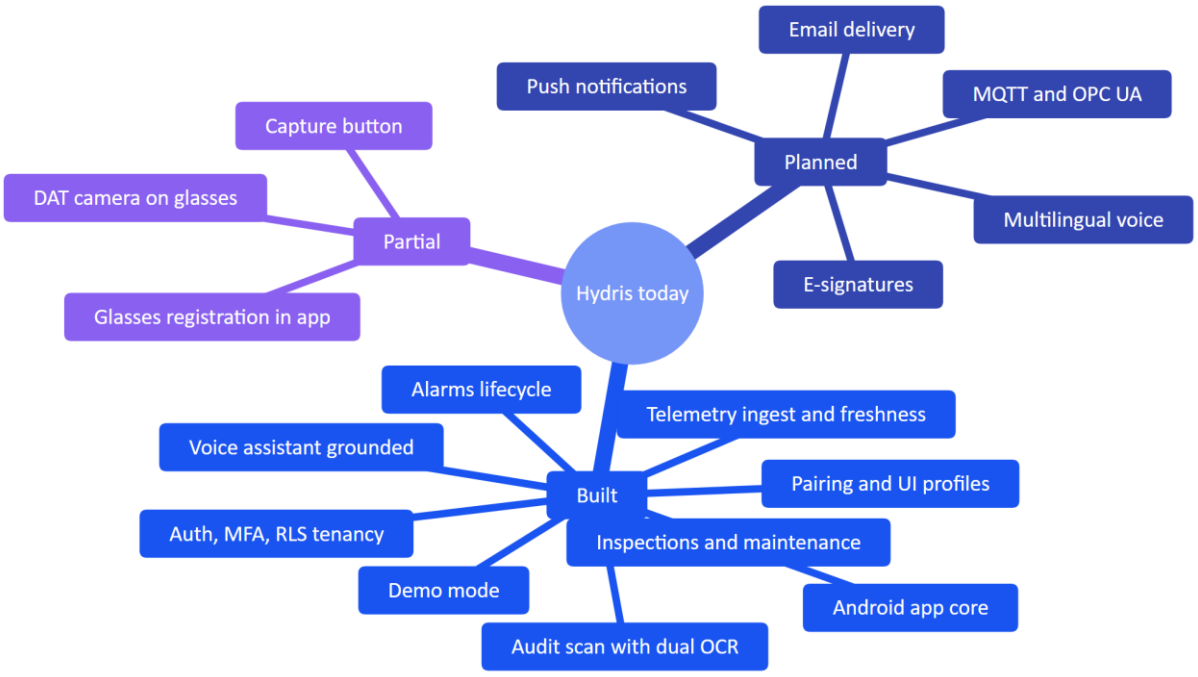

Figure 273. *Capability map today (Built, Partial, Planned)*

Table 359. Current state by product area

Area	What exists	Status	Evidence
Identity and tenancy	Login with lockout, TOTP MFA, refresh rotation, invitations, site scoping, Postgres RLS	**Built**	FR-001 to FR-010, `platform.test.ts`
Telemetry	16 measurands, HTTPS gateway ingestion, manual and lab entry, append-only corrections, freshness	**Built**	FR-020 to FR-026
Industrial protocols	`gateways.protocol` enum only	**Planned**	FR-027, FR-028
Alarms	Rules with deadband and on-delay, lifecycle, realtime over SSE	**Built**	FR-030 to FR-033
Push to phones	None	**Planned**	FR-034
Inspections and maintenance	Templates, routes, glasses execution with offline queue, review, maintenance tasks	**Built**	FR-040 to FR-043
Devices	Pairing, tokens, revocation, permission ceiling, UI profiles	**Built**	FR-050 to FR-054

Area	What exists	Status	Evidence
Meta glasses hardware	DAT bridge and native app integration, not run on physical glasses	**Partial**	FR-055 to FR-057
Audit scanning	Guided capture, dual-pass OCR, structuring, optional vision, review, four-eyes, PDF	**Built**	FR-060 to FR-067
E-signature	None	**Planned**	FR-068
Assistance and voice	SOP retrieval, "Hey Hydris", grounded answers with rules fallback	**Built** (English)	FR-070 to FR-072
Multilingual voice	None	**Planned**	FR-073
Platform	SSE replay, reports, audit trail, demo mode, simulation, in-app notifications	**Built**	FR-080 to FR-085
Email	Messages written to `mail_outbox`, never sent	**Planned**	FR-086
Android app	Login, demo, home, work, sensors, alarms, scan with CameraX and ML Kit, assistant, glasses screen	**Built** (manually verified on emulator)	`apps/android`
CI	None	**Planned**	No `.github/` directory

13.3 Vision and horizons

13.3.1 Product vision

Every operator in a water or wastewater plant gets the live state of the plant, the next task and the right procedure in their line of sight, hands free, and every reading, check and paper record they touch becomes a trustworthy digital record. Over the next horizons that vision widens from the operator to the plant: the same trusted record feeds compliance reporting, maintenance systems and predictive models.

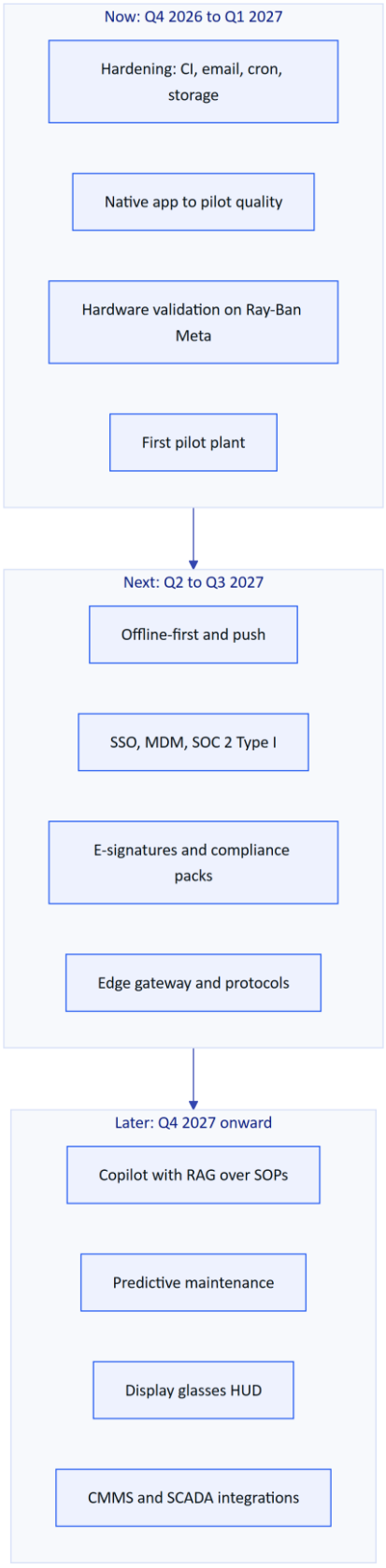

Figure 274. *Three horizons: make it reliable, make it indispensable, make it the plant's system of record*

Table 360. Horizon goals and exit conditions

Horizon	Goal	Exit condition
Now	A pilot plant uses Hydris every shift without engineering support on site	Pilot exit criteria in the QA plan all checked; 4 weeks of daily use
Next	A customer can buy, deploy and audit Hydris without Hydris engineers	GA release; SOC 2 Type I report; Play or MDM distribution; second paying site
Later	Hydris is the system of record for field work and compliance evidence at multi-site customers	Compliance reports generated from Hydris accepted by a regulator; one CMMS integration live

13.4 Timeline Q4 2026 to Q4 2027

The timeline and Gantt chart below are the plan as of October 2026. They are re-baselined at the end of every quarter.

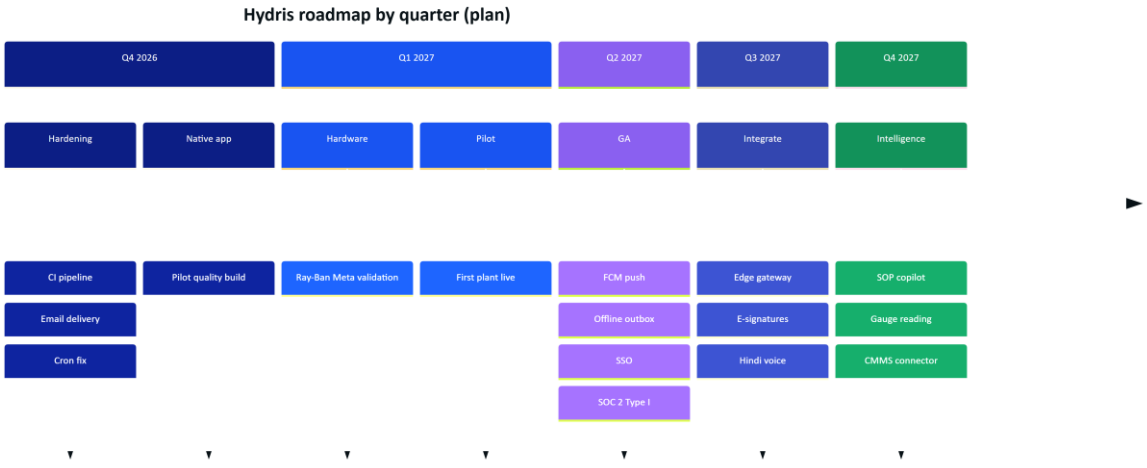

Figure 275. *Quarterly milestones (plan, October 2026)*

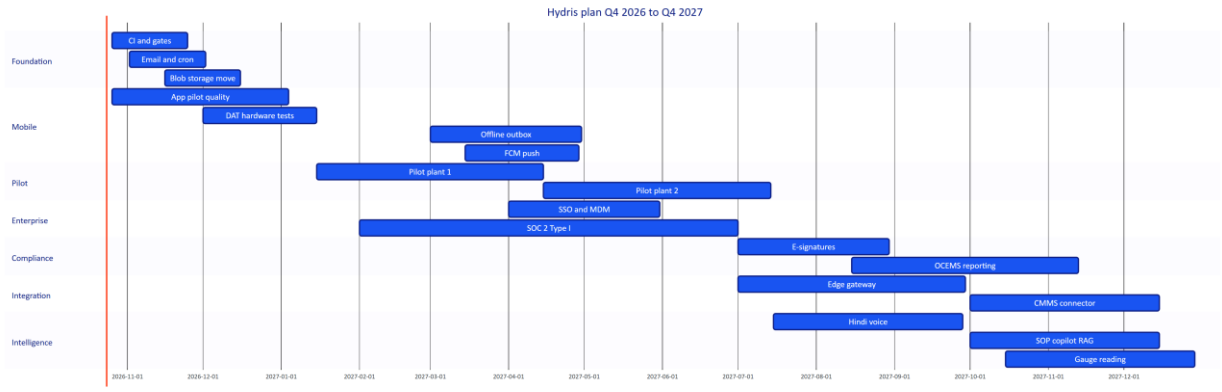

Figure 276. *Quarter plan as a Gantt chart (plan; bars are estimates)*

Table 361. Quarter summary (plan)

Quarter	Headline	Key deliverables	Exit signal
Q4 2026	Make it reliable	CI, email transport, cron schedule, audit images to object storage, purge jobs, Android pilot build	Gates 1 and 2 green on every PR

Quarter	Headline	Key deliverables	Exit signal
Q1 2027	Prove it in a plant	DAT hardware matrix, pilot plant 1, field trial, KPI baseline	Pilot exit criteria met
Q2 2027	Make it buyable	GA: FCM push, offline outbox, SSO, MDM, SOC 2 Type I readiness, India-region production	Second site signed
Q3 2027	Make it compliant and connected	E-signatures, edge gateway (MQTT, OPC UA, Modbus), Hindi voice	First e-signed record approved at a customer
Q4 2027	Make it intelligent	SOP copilot, gauge reading, first CMMS connector, OCEMS report drafts	Copilot used weekly by 50 % of operators (target)

13.5 Themes and epics

13.5.1 Theme overview

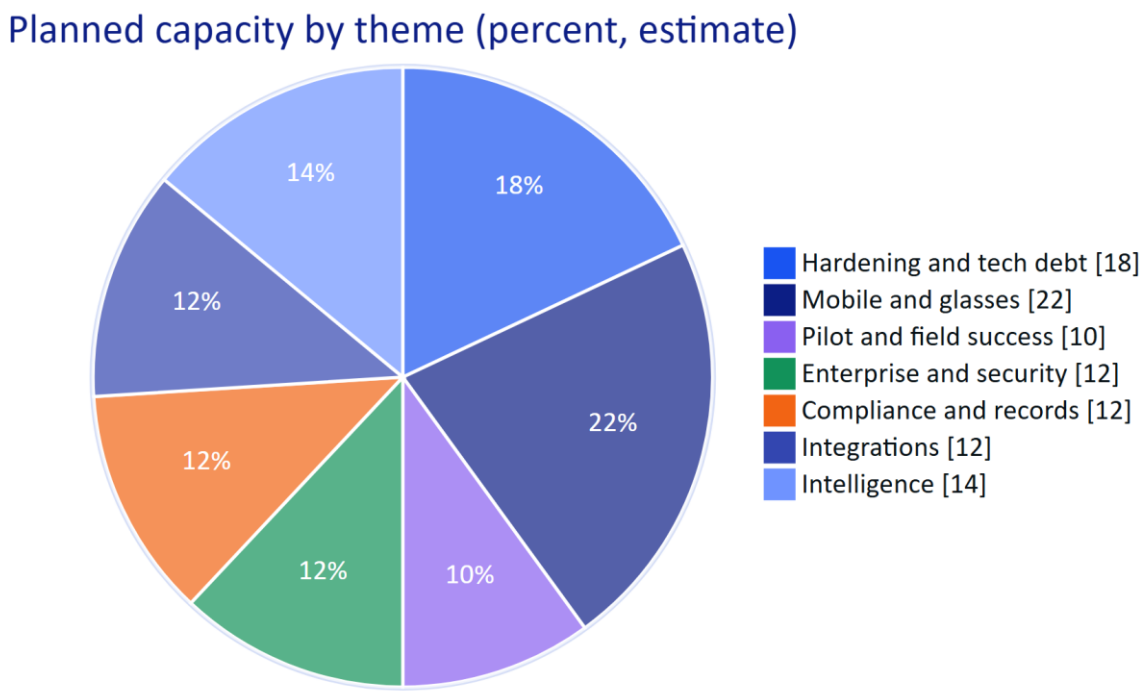

Figure 277. *Share of planned engineering capacity by theme, Q4 2026 to Q4 2027 (estimate)*

Each epic below states the problem, the outcome we want, the scope, the dependencies and the status. Epic IDs (EP-xx) are stable and are used in the prioritisation table.

13.5.2 Hardening and tech debt

EP-01 CI and quality gates. Problem: tests run only on laptops; nothing stops a regression reaching the demo. Outcome: every pull request runs typecheck, unit, API, build, Playwright and Android checks. Scope: GitHub Actions pipeline as specified in the QA and Test Plan. Dependencies: none. Status: **Planned** (Q4 2026).

EP-02 Email delivery. Problem: invitations and password resets are written to `mail_outbox` and never sent (FR-086), so a real customer cannot onboard users without manual help. Outcome: transactional email with

delivery status and purge after send. Scope: SES or Resend adapter, templates, bounce handling, outbox purge. Dependencies: domain verification. Status: **Planned** (Q4 2026).

EP-03 Storage and retention. Problem: audit page images are base64 inside Postgres; expired sessions, pairing challenges and outbox rows are never purged (Data Model). Outcome: images in object storage with lifecycle rules; purge jobs on a schedule. Scope: migrate `audit_document_pages.data` to Blob or S3, purge jobs in `apps/api/src/jobs.ts`. Status: **Planned** (Q4 2026).

13.5.3 Mobile and glasses

EP-04 Android app to pilot quality. Problem: the native app is built and manually verified on an emulator but has no automated tests and has not run with glasses. Outcome: a signed pilot build on a Play internal track with unit and UI tests. Scope: tests, crash reporting, settings, diagnostics screen, release signing. Status: **Partial**.

EP-05 Ray-Ban Meta hardware validation. Problem: the DAT camera, capture button and glasses speech paths are unverified on hardware. Outcome: hardware matrix rows H1, H2 and H5 passed. Dependencies: DAT access for India-based testers, Meta AI app on test phones. Status: **Planned** (Q4 2026 to Q1 2027).

EP-06 Offline-first and push. Problem: operators walk through basements and pump houses with no signal; the Android app has no outbox and no push. Outcome: rounds and alarm acknowledgements work offline and sync safely; alarms reach a pocket phone in seconds. Scope: see Offline-first sync and FR-034. Status: **Planned** (Q2 2027).

EP-07 Display glasses. See Meta display glasses and other displays. Status: **Planned** (Later).

13.5.4 Pilot and field success

EP-08 Pilot programme. Problem: no plant has used Hydris on a real shift. Outcome: one to two Indian plants using it daily with measured KPIs. Scope: onboarding kit, site survey, camera policy pack, training, weekly KPI review, field trial. Dependencies: EP-01 to EP-05. Status: **Planned** (Q1 2027).

13.5.5 Enterprise and security

EP-09 SSO and MDM. Problem: enterprise IT will not manage another password list or side-loaded APKs. Outcome: SAML and OIDC sign-in, managed Android deployment. Status: **Planned** (Q2 2027).

EP-10 Certifications. Problem: due diligence questionnaires stall deals. Outcome: SOC 2 Type I, then Type II and ISO/IEC 27001. Status: **Planned** (Q2 2027 onward).

13.5.6 Compliance and records

EP-11 Electronic signatures (FR-068). See E-signatures. Status: **Planned** (Q3 2027).

EP-12 OCEMS and pollution control board reporting. See OCEMS and CPCB and SPCB compliance automation. Status: **Planned**.

13.5.7 Integrations

EP-13 Edge gateway and protocols (FR-027, FR-028). See Edge gateway and industrial protocols. Status: **Planned** (Q3 2027).

EP-14 CMMS and marketplace. See Marketplace and CMMS integrations. Status: **Planned** (Q4 2027).

13.5.8 Intelligence

EP-15 Multilingual voice (FR-073). EP-16 SOP copilot. EP-17 Gauge and meter reading. EP-18 Predictive maintenance. Each is described in depth in the next chapter. Status: **Planned**.

13.6 Future features in depth

13.6.1 Meta display glasses and other displays

Problem. Today's Ray-Ban Meta glasses have a camera, microphones and open-ear speakers but no display, so the Hydris HUD runs on the phone. Operators still have to look down. Display glasses (Meta Ray-Ban Display with the Neural Band, and industrial head-mounted displays) would put the live value and the next step in view.

Outcome. The same glasses web app, with its existing HUD layout and UI profiles (FR-054), renders on a display device, and input comes from the device's native gestures as well as voice.

Approach. Treat displays as rendering targets behind a small adapter: the HUD layout already exists in `apps/web/src/glasses/GlassesApp.tsx`, and input is abstracted in `glasses/input.ts` (which holds a placeholder for hardware gestures today). Candidate targets, in order: Meta Ray-Ban Display (dependent on Meta exposing display and Neural Band input to third parties), Android-based industrial headsets such as RealWear Navigator (voice-driven, hard-hat mounted), and Vuzix-class smart glasses.

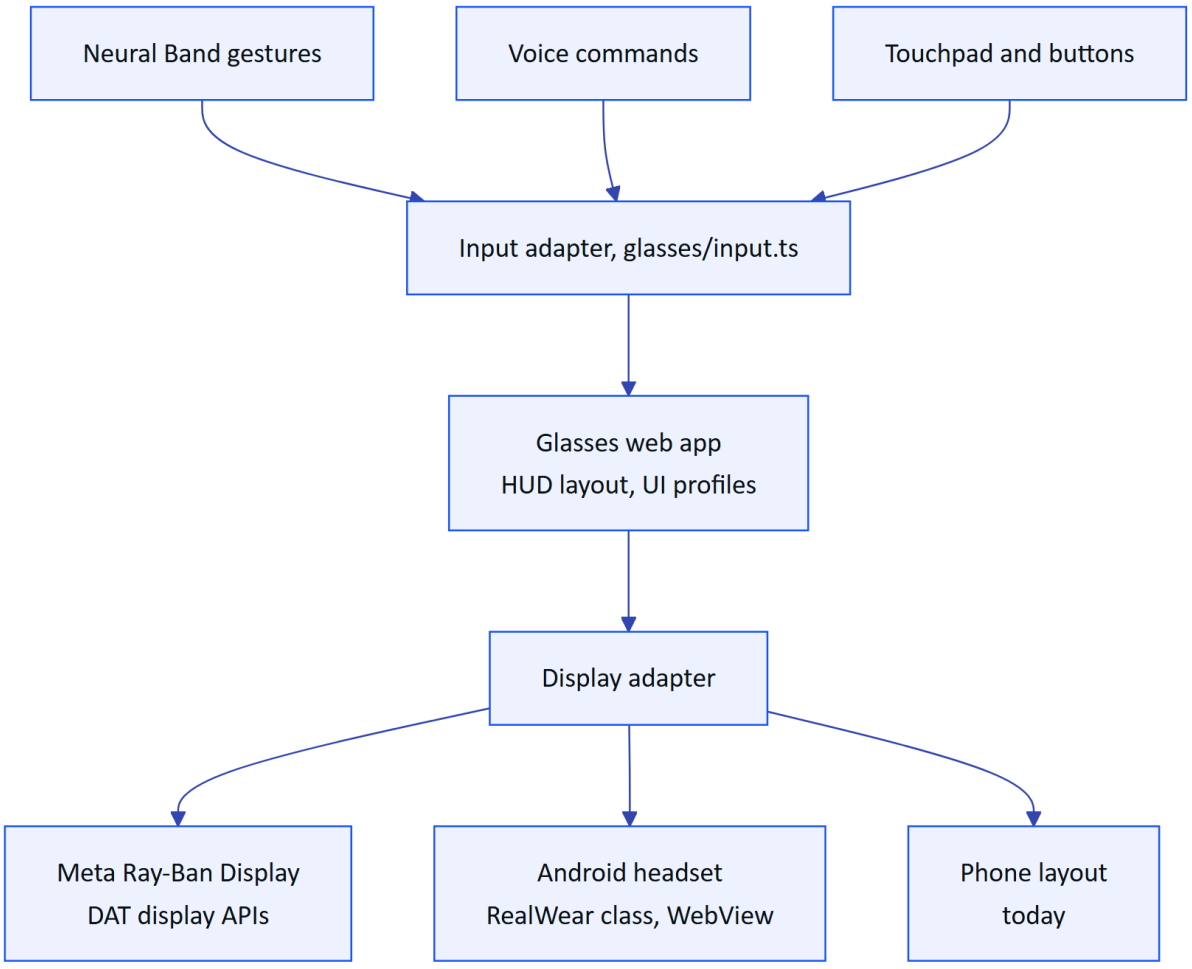

Figure 278. *Display adapter design for future display glasses (planned)*

Dependencies. Meta display APIs (not available to us today; Mock Device Kit does not simulate display glasses), device availability in India. **Status: Planned** (Later). **Effort:** 3 to 5 person-months per target (estimate).

13.6.2 Offline-first sync

Problem. Plants have dead zones. The web glasses app already queues inspection responses and replays them idempotently (`client_op_id`, FR-041), but the Android app has no persistent outbox, and alarm acknowledgements and readings fail without a network.

Outcome. Every field action works offline for a full shift and syncs exactly once, with clear conflict rules.

Scope. Room database cache of the operator's context (tasks, assets, latest readings, SOPs), an outbox of commands with client operation IDs, background sync with WorkManager, conflict policy (server wins for alarm state; append-only merge for readings and steps), visible "queued" counts.

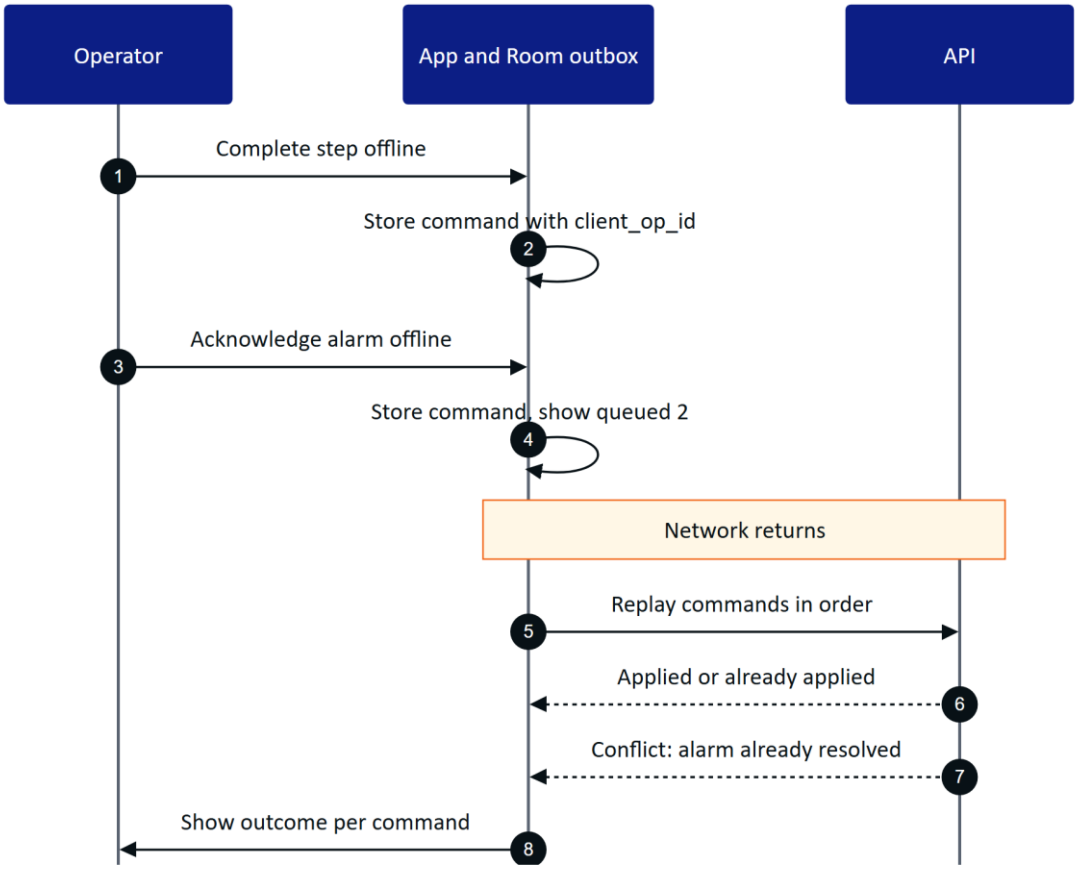

Figure 279. *Offline outbox and idempotent replay (planned for Android; the web app already replays inspection steps)*

Dependencies. Server endpoints accept `client_op_id` for alarm actions (inspections already do). **Status: Planned** (Q2 2027). **Effort:** 3 person-months (estimate).

13.6.3 Edge gateway and industrial protocols

Problem. Plants publish instrument data on Modbus, OPC UA and MQTT, often through a SCADA or historian. Hydris accepts HTTPS batches only (`POST /ingest/v1/readings`, FR-022), so each customer needs custom integration.

Outcome. A Hydris Edge gateway installed in the plant network reads tags and forwards readings outbound over HTTPS, preserving the read-only and outbound-only security model.

Scope. Containerised gateway (Docker on an industrial PC), connectors for OPC UA (subscription), MQTT (including Sparkplug B payloads), and Modbus TCP and RTU (polling), a tag mapping UI in the console, store-and-forward buffering, and read-only historian connectors (FR-028).

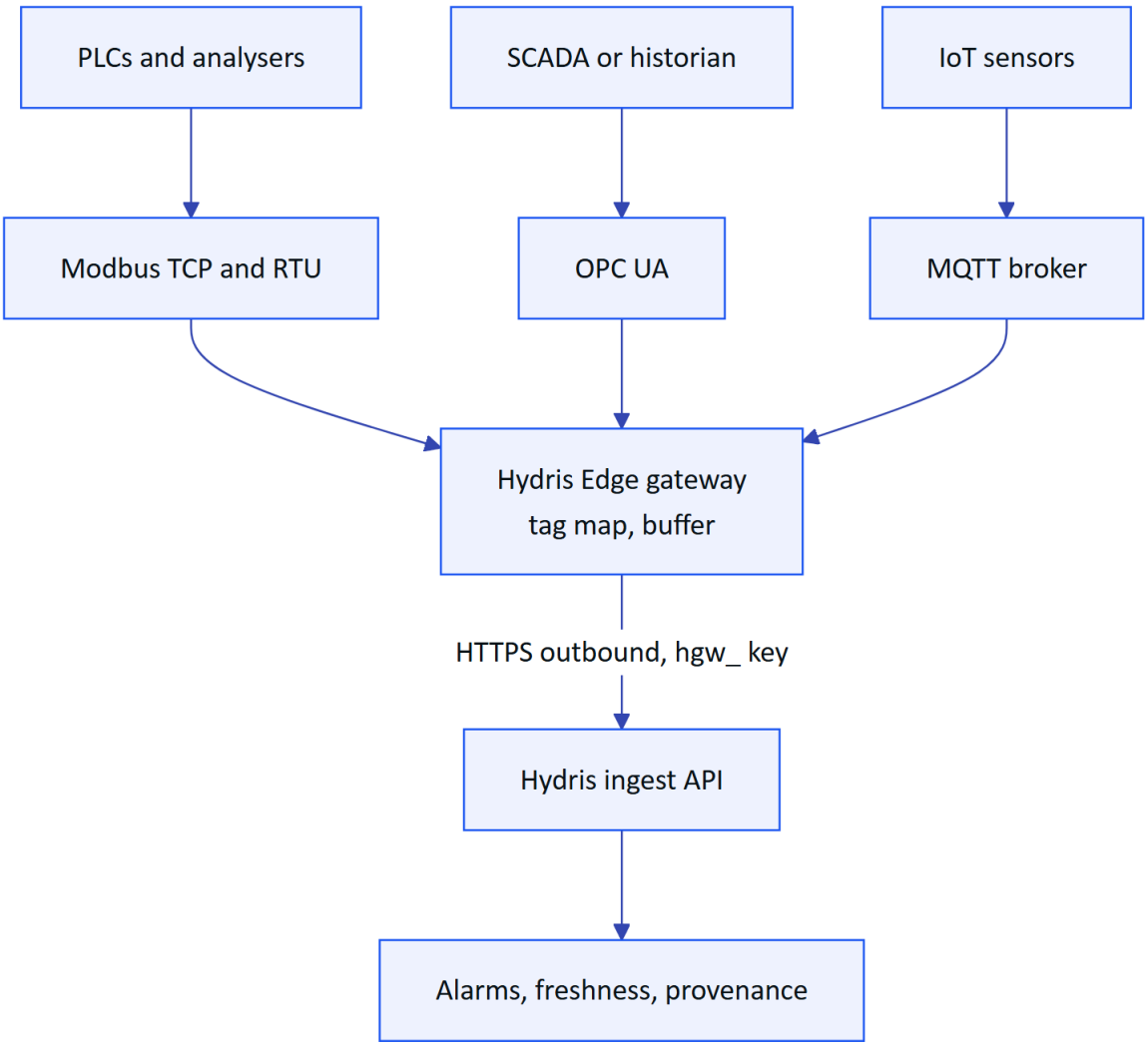

Figure 280. *Edge gateway architecture (planned): outbound only, never writes to the plant*

DECISION

The gateway is read-only by construction: connectors are built without write function codes or method calls, and configuration cannot enable them. This keeps Hydris outside IEC 62443 control zones, as stated in Security and Privacy.

Dependencies. Pilot customer network policy, a test rig with a PLC simulator. **Status: Planned** (Q3 2027). **Effort:** 5 person-months (estimate).

13.6.4 Predictive maintenance and anomaly detection

Problem. Blowers, pumps and dosing systems fail with warning signs (rising current, vibration, runtime drift) that nobody watches continuously.

Outcome. Hydris flags assets whose behaviour departs from their own normal, and creates maintenance tasks with evidence, before the failure.

Approach. Start simple and explainable: per-asset baselines and seasonal residuals, then multivariate models once there is enough labelled history. Every prediction becomes an advisory, never an automatic action, and links to the evidence charts.

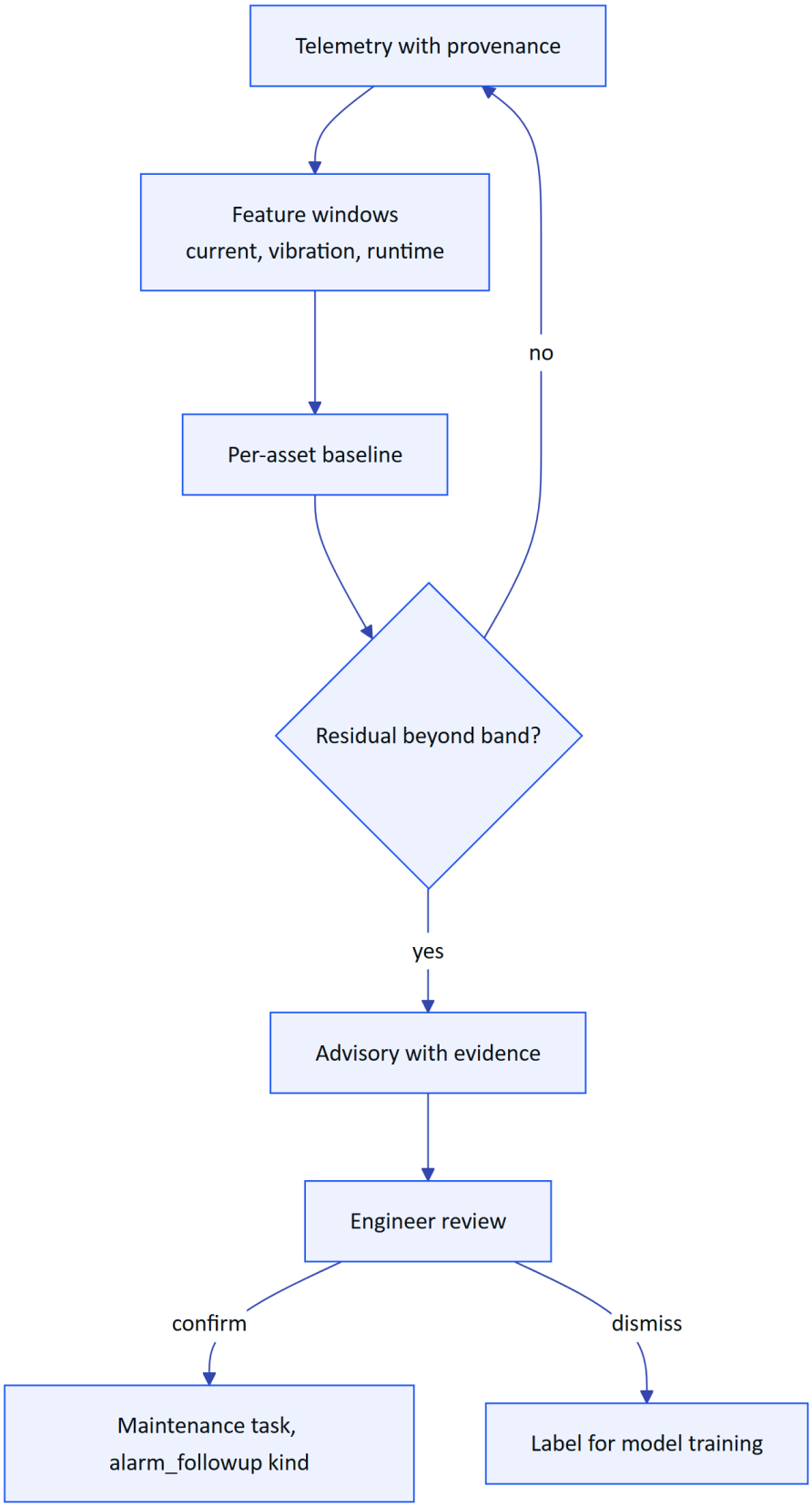

Figure 281. *Anomaly detection pipeline (planned)*

Dependencies. Edge gateway for high-frequency data, 6 or more months of history per asset. **Status: Planned** (Later). **Effort:** 6 person-months for the first version (estimate).

13.6.5 SOP copilot with retrieval over procedures

Problem. Today `/assist` retrieves approved SOPs and the voice assistant answers from tenant data via scoped tools (FR-070, FR-072). Operators also ask "how" and "why" questions that need the content of long procedures, manuals and past incident notes.

Outcome. A copilot that answers from the customer's approved documents with citations, in the field, and never invents a step.

Approach. Retrieval-augmented generation: chunk and embed approved documents per tenant, retrieve with RLS-scoped queries, generate with citations, and refuse when retrieval confidence is low. Safety rule kept from today: Hydris does not control equipment; for safety-critical actions it points to the approved procedure and the supervisor.

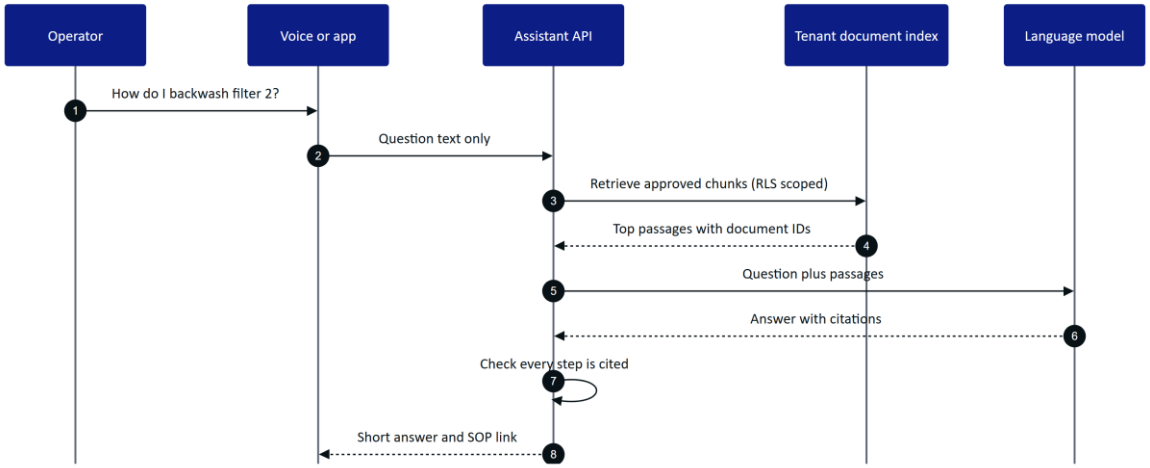

Figure 282. *Copilot answer flow with retrieval and citation checks (planned)*

Dependencies. Document version control (exists: document status `draft`, `approved`, `archived`), LLM data residency decision for Indian customers, per-org opt-out of third-party processing (planned in Security and Privacy). **Status: Planned** (Q4 2027). **Effort:** 3 person-months (estimate).

13.6.6 Multilingual voice

Problem. Many operators think and speak in Hindi, Tamil, Marathi, Gujarati, Telugu or Kannada, and mix languages in one sentence. The assistant is English-only (FR-073 Planned).

Outcome. Operators ask in their language and hear answers in it, with numbers and units spoken correctly.

Scope. Speech recognition with Indic language support (Android on-device recognizers and cloud options such as models trained on Indian languages), language-aware intent detection, localised wake phrase handling, TTS voices, localised UI strings, and a code-mixed test set (the QA utterance set already includes code-mixed questions).

Table 362. Language rollout order (plan)

Wave	Languages	Why first	Target
1	Hindi	Largest operator population in early pipeline	Q3 2027
2	Tamil, Marathi	Pilot regions (assumed)	Q4 2027
3	Gujarati, Telugu, Kannada, Bengali	Industrial clusters	2028

Dependencies. Recognizer accuracy in noise, numeric normalisation per language. **Status: Planned**. **Effort:** 2 person-months for Hindi, then 1 per language (estimate).

13.6.7 Computer vision for gauges and meters

Problem. Many plants still have analogue pressure gauges, flow totalisers and level indicators that are read by eye and written down, which is where transcription errors start.

Outcome. The operator looks at a gauge, says "read this", and Hydris proposes a value with confidence; the operator confirms it, and it is stored with the photo as provenance, like an audit scan field.

Approach. Reuse the audit scan pattern: capture on intent, machine proposes, human confirms, original kept. Dial gauge reading via needle detection and scale calibration per registered gauge; digit recognition for seven-segment and odometer-style totalisers; QR or tag on the gauge to link the asset and its calibration.

Dependencies. DAT camera validated (EP-05), labelled image set from pilot sites. **Status: Planned** (Q4 2027). **Effort:** 5 person-months (estimate). **Target accuracy:** within 2 % of full scale on 95 % of reads (target).

13.6.8 OCEMS and CPCB and SPCB compliance automation

Problem. CPCB requires online continuous effluent and emission monitoring (OCEMS) for 17 categories of highly polluting industries and for CETPs, and state boards (SPCBs) set consent conditions with periodic returns. Plants assemble these reports by hand from instruments, lab sheets and logbooks.

Outcome. Hydris assembles compliance evidence (continuous data with provenance and freshness, verified lab sheets, approved logs) and drafts the periodic reports, with every number traceable to its source.

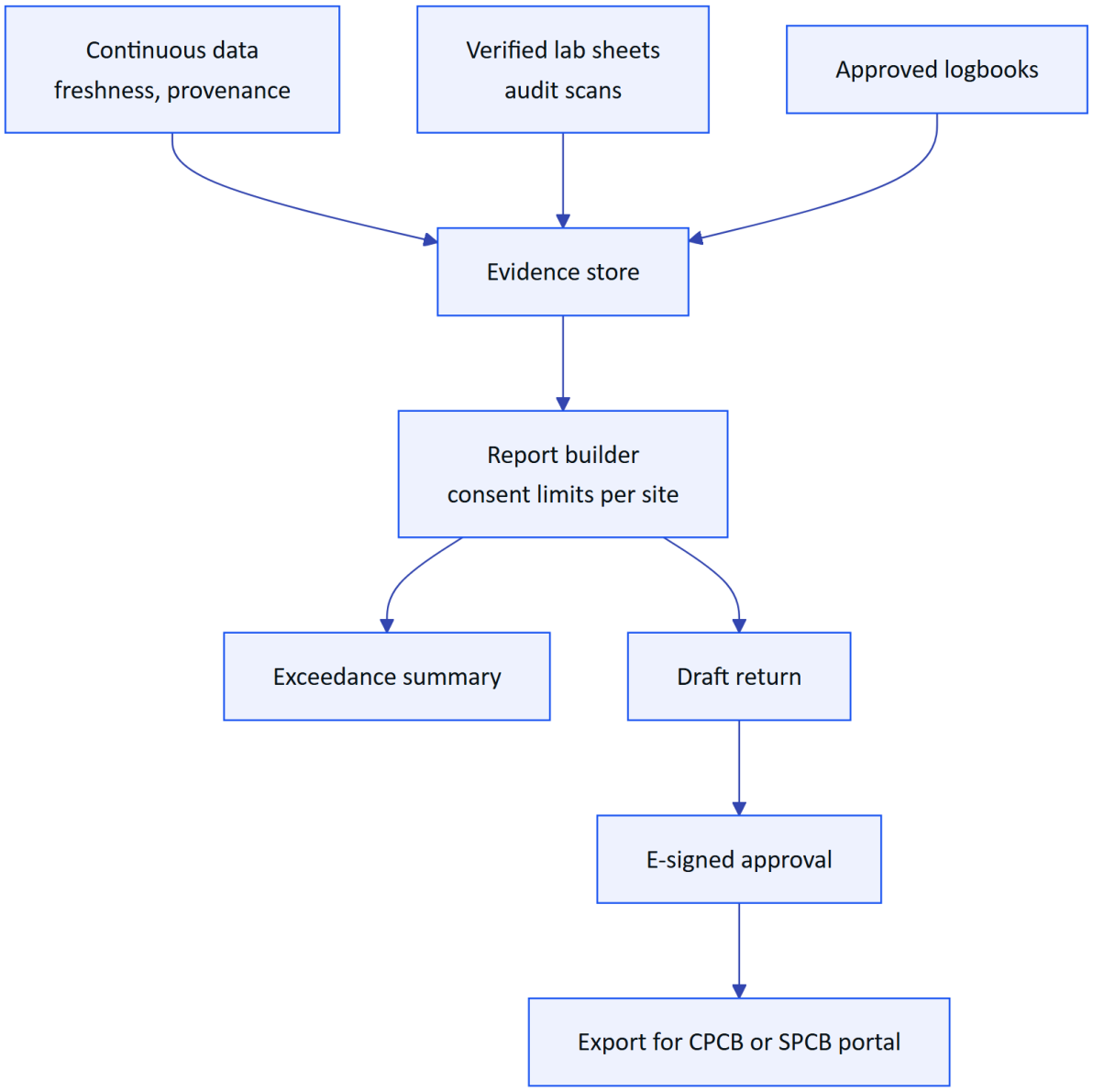

Figure 283. *Compliance reporting pipeline (planned)*

Scope. Consent-limit library per site, exceedance and data-availability calculations, report templates, export formats. Direct submission to CPCB or SPCB systems only where an official interface exists (to be confirmed per state).

Dependencies. E-signatures (EP-11), edge gateway for continuous data, regulator acceptance of Hydris-generated records (PRD open question). **Status: Planned** (Q3 to Q4 2027). **Effort:** 4 person-months (estimate).

13.6.9 E-signatures

Problem. The audit trail is append-only and approval is four-eyes (FR-065), but there is no signature manifestation: re-authentication at the moment of signing, a stated meaning ("reviewed", "approved"), and the signer's name and time bound to the record (FR-068).

Outcome. Signatures that meet Part 11-style expectations for audit documents, inspections and compliance returns, without claiming GxP validation.

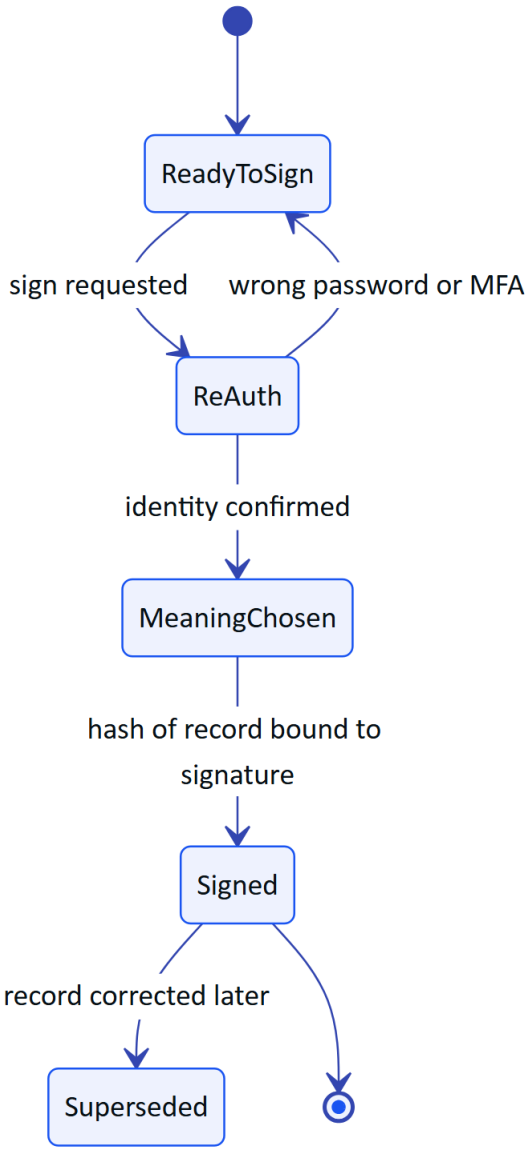

Figure 284. *E-signature flow on a record (planned, FR-068)*

Scope. Re-auth with password or MFA, signature meanings, record hash bound to the signature, manifestation on the PDF, signature report, lockout on repeated failures. **Status: Planned** (Q3 2027). **Effort:** 2 person-months (estimate).

13.6.10 Digital twin and plant map

Problem. The console has a plant map (`console/map`, `glasses/PlantMap.tsx`) with asset positions, but it is a schematic, not a living model of process flow.

Outcome. A process-flow view of the plant where each unit shows live state, open alarms, overdue tasks and data freshness, and where an operator's glasses position (by QR or BLE beacon check-in) shows nearby assets.

Scope. Process-flow layout editor, live overlays, asset check-in, "what is near me" on glasses, mass-balance views later. **Status: Planned** (Later). **Effort:** 4 person-months (estimate).

13.6.11 Remote expert video assist

Problem. When a junior operator faces an unfamiliar fault, the expert is often at another site or vendor office.

Outcome. The operator starts a call from the glasses; the expert sees the glasses camera view, talks to the operator and annotates frames; the session summary is saved against the asset.

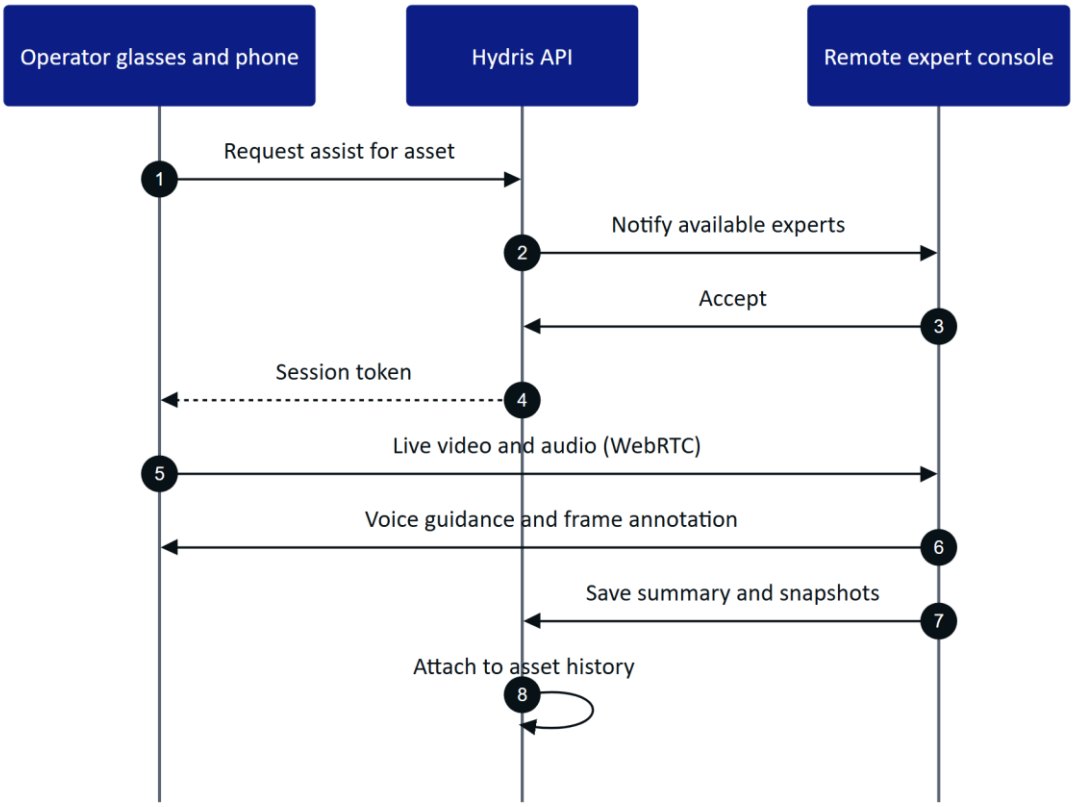

Figure 285. *Remote assist session (planned)*

Dependencies. DAT video streaming bandwidth over the phone link, plant network uplink, consent for recording. **Status: Planned** (Later). **Effort:** 3 person-months (estimate).

13.6.12 iOS app

Problem. Some customers' supervisors and managers carry iPhones. Operators in Indian plants mostly use Android.

Outcome. An iOS companion for supervisors first (alarms, reviews, approvals), then operators if Meta's iOS DAT support and customer demand justify it.

Status: Planned (Later; PRD open question 5). **Effort:** 6 person-months (estimate).

13.6.13 Mobile device management

Problem. Pilot phones are set up by hand. Enterprises require managed devices, app configuration and remote wipe.

Outcome. Hydris Field supports Android Enterprise managed configuration (server URL, SSO, kiosk options), works in work profiles, and is distributable through managed Google Play.

Status: Planned (Q2 2027). **Effort:** 1.5 person-months (estimate).

13.6.14 Multi-site analytics

Problem. Reports today are per organization and per site (FR-081). Groups that run many STPs (municipal bodies, CETP operators, facility managers) need comparisons.

Outcome. Cross-site dashboards: compliance status, alarm response times, round completion, data availability, and benchmarks across sites.

Status: Planned (Later). **Effort:** 3 person-months (estimate).

13.6.15 Marketplace and CMMS integrations

Problem. Large customers run maintenance in SAP Plant Maintenance or IBM Maximo. Hydris maintenance tasks (FR-043) would otherwise duplicate work orders.

Outcome. Two-way sync of work orders and asset master data with the customer's CMMS, plus outbound webhooks for other systems (LIMS, ERP, ticketing).

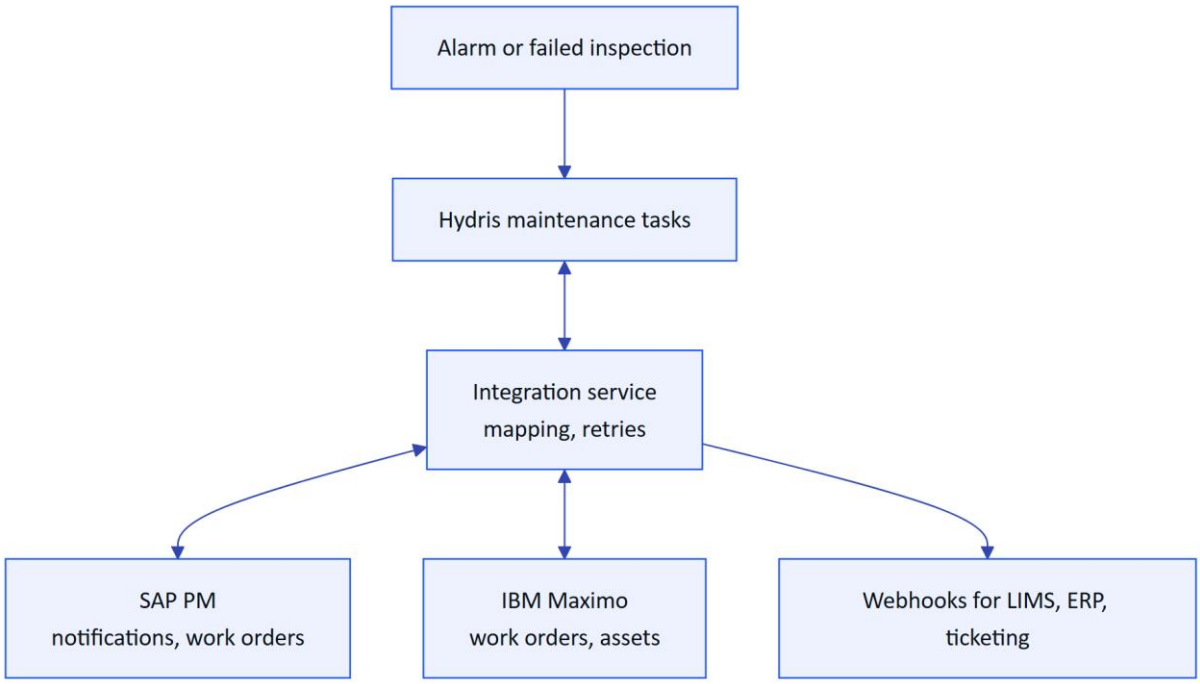

Figure 286. *CMMS integration pattern (planned)*

Status: Planned (Q4 2027 onward). **Effort:** 4 person-months for the first connector (estimate).

13.6.16 Certifications

Table 363. Certification plan (targets)

Certification	Why	Target
SOC 2 Type I	Customer due diligence for SaaS	Q2 2027 readiness
SOC 2 Type II	Operating effectiveness over time	2028
ISO/IEC 27001:2022	Common Indian enterprise and PSU requirement	2028
IEC 62443-4-1 practices for the edge gateway	Industrial customers' OT security reviews	Assessment with the gateway
Intrinsically safe hardware	Not a Hydris certification; depends on glasses and phone vendors for hazardous zones	Customer-specific

13.7 Prioritisation

13.7.1 RICE scoring

RICE = Reach x Impact x Confidence / Effort. Reach is operators or users affected per quarter after launch; impact is on a 0.25 to 3 scale; effort is person-months. All values are team estimates (October 2026). Hardening epics (EP-01 to EP-03) are prerequisites and are not scored.

Table 364. RICE scores for candidate epics (estimates)

Epic	Reach	Impact	Confidence	Effort	RICE
FCM push	300	1	90 %	1	270
SSO	300	1	90 %	1.5	180
Offline-first sync	300	2	80 %	3	160
MDM	300	0.5	80 %	1.5	80
E-signatures	100	2	80 %	2	80
SOP copilot	300	1	70 %	3	70
Multilingual voice	250	2	50 %	4	63
Edge gateway	200	2	60 %	5	48
OCEMS reporting	80	3	50 %	4	30
CMMS connector	100	2	50 %	4	25
Remote assist	100	1	60 %	3	20
Digital twin map	150	1	50 %	4	19
Multi-site analytics	40	2	70 %	3	19
Gauge reading	200	1	40 %	5	16
Display glasses HUD	50	2	40 %	4	10
iOS app	60	1	70 %	6	7
Predictive maintenance	50	2	30 %	6	5

> **NOTE**
>
> RICE ranks within a horizon; it does not override dependencies or strategy. OCEMS reporting scores low on reach but is scheduled in Q3 to Q4 2027 because it is a buying reason for the highest-value segment (17 categories and CETPs).

13.7.2 Value versus effort

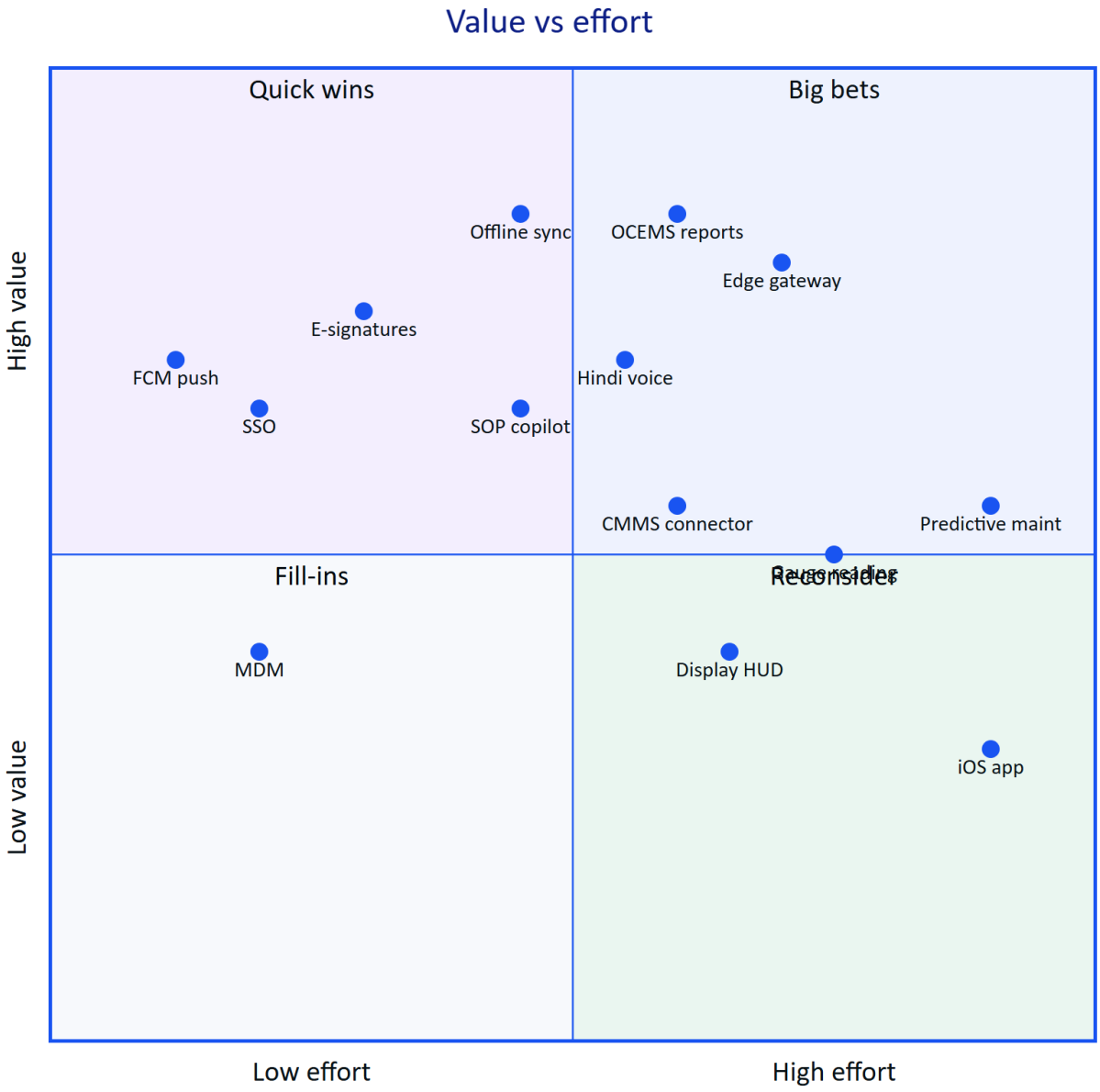

Figure 287. *Value versus effort for candidate epics (team estimates)*

13.8 Known gaps and tech debt to close

These items come from the product documents and the code. Most block an external pilot.

Table 365. Gaps and tech debt

Item	Where	Impact	Fix	Target
No CI pipeline	No `.github/`	Regressions reach the demo	EP-01	Q4 2026
Email not wired	`mail_outbox`, FR-086	Invites and resets never arrive	EP-02	Q4 2026
Cron runs daily on Vercel	`vercel.json` `0 3 * * *`	Stale-data alarms evaluated once a day	Per-minute schedule (Pro plan) or external scheduler	Q4 2026

Item	Where	Impact	Fix	Target
Device refresh token in `localStorage`	Web glasses runtime `glasses/link.ts`	Token theft on a compromised device	Native app (Keystore, built); shorter TTL or bound tokens for web	Q1 2027
Audit images as base64 in Postgres	`audit_document_pages.data`	Database bloat, slow backups	Object storage with lifecycle	Q4 2026
No purge jobs	`auth_sessions`, `device_sessions`, `pairing_challenges`, `mail_outbox`	Personal data kept indefinitely	Purge 30 days after expiry; outbox after send	Q4 2026
No e-signature	FR-068	Regulated customers hesitate	EP-11	Q3 2027
Pairing TTL documentation mismatch	`docs/operations/vercel.md` says about 10 minutes; code `PAIRING_TTL_S = 300`	Support confusion	Correct the doc to 5 minutes	Q4 2026
Mobile spec status outdated	`MOBILE_APP_SPEC.md` says `apps/android` not on main	Misleads readers	Update spec	Q4 2026
No Android automated tests	`apps/android`	Manual regression only	EP-04	Q4 2026
DAT paths unverified on hardware	FR-055 to FR-057	Glasses features may fail on device	EP-05	Q1 2027
No per-org opt-out of third-party AI processing	Security and Privacy	Blocks some customers	Org setting	Q2 2027
No device attestation	Native app	Spoofed clients	Play Integrity	Q2 2027
Audit log not hash-chained	`audit_events`	Weaker tamper evidence	Hash chain or WORM export	Q3 2027

- ☐ CI pipeline with gates 1 and 2
- ☐ Email transport and outbox purge
- ☐ Cron schedule fixed
- ☐ Audit images moved to object storage
- ☐ Purge jobs for expired sessions and challenges
- ☐ Pairing TTL documentation corrected
- ☐ Mobile spec status updated
- ☑ Native app stores tokens in Android Keystore (`SecureStore.kt`)

13.9 Dependencies and risks

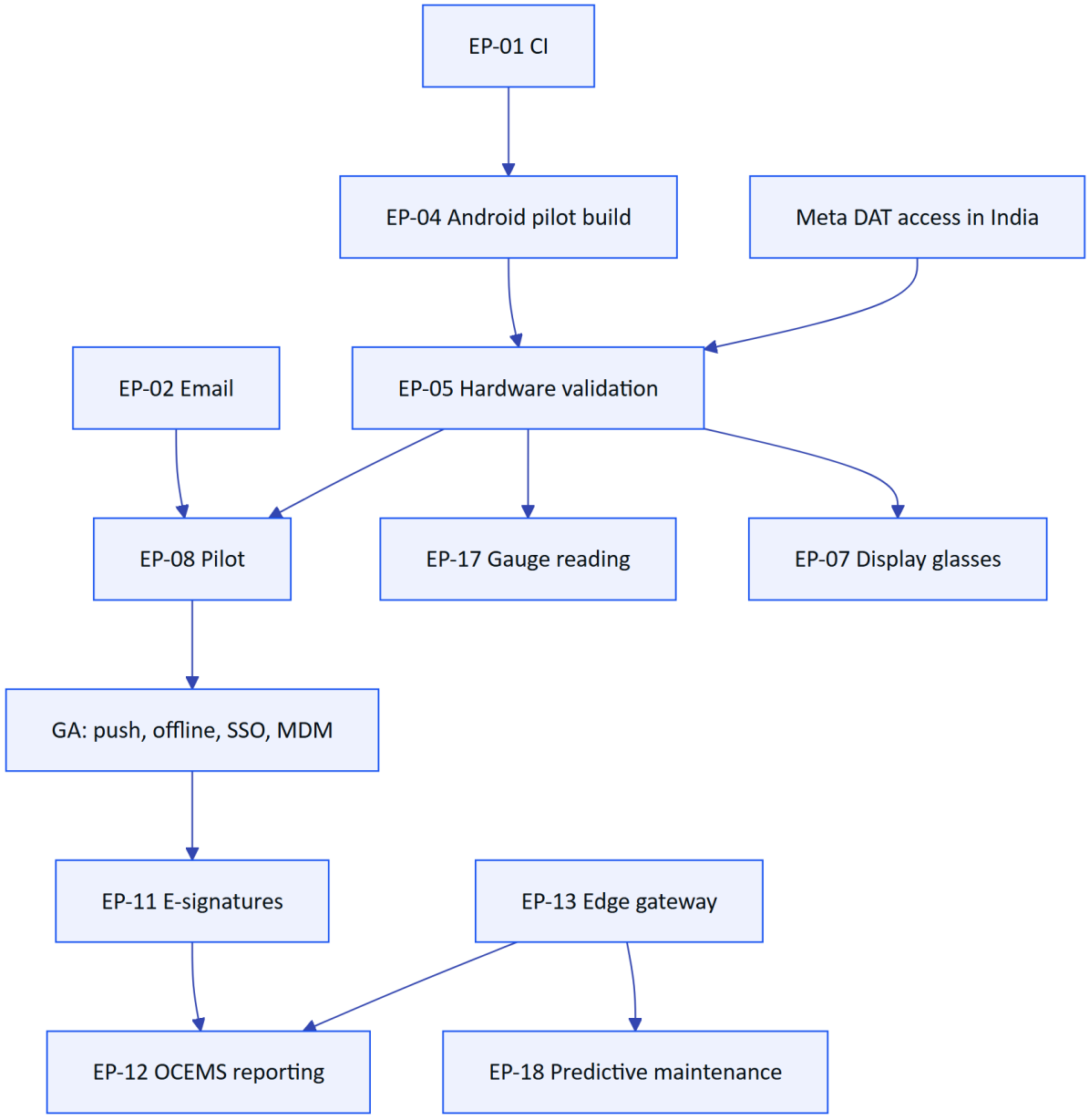

Figure 288. *Key dependencies between epics (plan)*

Table 366. Roadmap risks

Risk	Likelihood	Impact	Mitigation
Meta DAT not available to India-based developers or publishing stays gated	Medium	High	Pilot via release channels and Developer Mode; phone-camera path works without DAT
Meta AI app dependency causes support load	High	Medium	Onboarding checklist, version check screen
Pilot customer delays (approvals, camera policy)	High	High	Site camera policy pack early; second pilot candidate in parallel
Regulators do not accept Hydris records as primary	Medium	High	E-signatures, PDF with trail, engage SPCBs early
LLM data residency required	Medium	Medium	Rules engine fallback exists; region-pinned provider

Risk	Likelihood	Impact	Mitigation
Small team spread across too many surfaces	High	High	Strict horizon discipline; Later items stay unstarted until GA
Display glasses APIs never open to third parties	Medium	Low	Industrial Android headsets as alternative target

13.10 Success metrics per horizon

Table 367. Success metrics (targets)

Horizon	Metric	Target
Now	Pilot exit criteria complete	100 %
Now	Round completion on time at pilot	95 % (PRD)
Now	Median alarm acknowledgement time	Under 5 min (PRD)
Now	Audit fields machine-agreed	90 % (PRD)
Next	Weekly active operators / licensed	70 % (PRD)
Next	Paying sites	3
Next	Offline actions synced without duplicates	100 %
Next	SOC 2 Type I	Report issued
Later	Compliance reports generated in Hydris	1 regulator accepted
Later	Copilot answers with citations	90 % of answered questions
Later	Multi-site customers	2

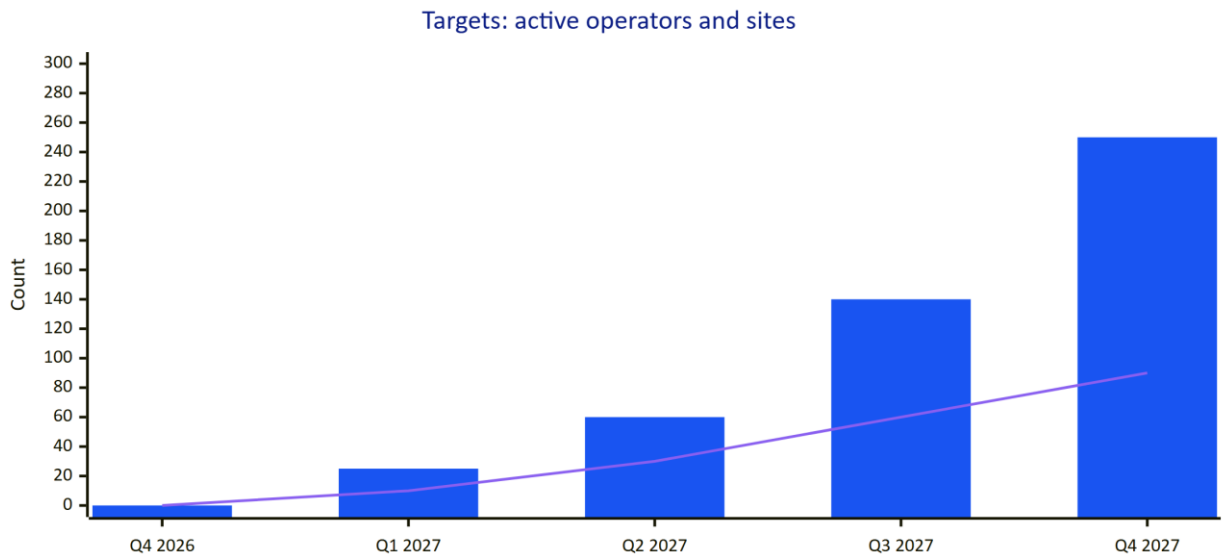

Figure 289. *Adoption targets by quarter (planning targets, not measurements)*

In the chart, bars are weekly active operators and the line is sites x 10 (so 90 means 9 sites). Both are targets.

13.11 Release cadence

Table 368. Release cadence (plan)

Surface	Cadence	Channel	Notes
API and web (console, glasses web app, website)	Continuous to staging; weekly to production after gates	Vercel	Hotfix within 24 h for S1
Android app	Every 2 weeks	Play internal, then closed track, then managed Play	Glasses features behind remote flags
Edge gateway	Monthly	Signed container images	Customer-controlled upgrade window
Documentation	With each release	`docs/product` and Word editions	Version and change line updated

Every release carries release notes listing requirement IDs changed, known issues and migration notes. Major features ship behind organization-level flags so pilots can opt in.

13.12 Open questions

1. Is India among Meta's supported countries for full DAT capabilities, and when do store submissions open?
2. Which pilot plants (ETP, STP or CETP) and which state boards will shape the first compliance pack?
3. Will customers accept glasses cameras in plant areas, and which zones must be camera-free?
4. Is a region-pinned LLM provider required before the copilot ships?
5. Which CMMS do the first enterprise prospects use: SAP PM, Maximo or something else?